

Alex's Anthology of Algorithms
Common Code for Contests in Concise C++

Draft v0.93, Jun 1, 2026

Alex Li

© 2026. All rights reserved.

This page is intentionally left blank.

Preface

Visit github.com/alxli/algorithm-anthology for the most up-to-date digital version of this book.

Introduction

Welcome to a comprehensive collection of common algorithms and data structures. The ultimate goal of this book is not to explain concepts from the ground up, but instead to explore the finer details behind their *implementations*. There are many potential ways you can use this, for instance:

- as a reference to help you better understand topics that you have only studied on a high level,
- as a printed codebook, which is a permitted resource for contests such as the ACM ICPC, or
- to cross-check existing code you have written for contest or coding interview questions.

Before diving into any section, it is strongly recommended that you have already studied the algorithms involved. Reading the code first is never an ideal approach to properly understand an algorithm. You should instead try proving (its correctness and time/space complexity) and implementing it from scratch.

Every topic to be explored is easily researchable online. Thus instead of including theoretical discussions, I document just enough to establish the problem being solved, notation being used, and any special trickery involved. I have also included small, non-rigorous examples to demonstrate usage of the code.

We mentioned that the implementation itself is the focus, but what makes an implementation good? The code is written with the following principles in mind:

- *Clarity*: A reader already familiar with an algorithm should have no problem understanding how its implementation works. Consistency in naming conventions should be emphasized, and any tricks or language-specific hacks should be documented.
- *Concision*: To minimize the amount of scrolling and searching during the frenzy of time contests, it is helpful for code to be compact. Shorter code is also generally easier to understand, as long as it is not overly cryptic. Finally, each implementation should fit in a single source file as required by nearly all online judging systems.
- *Efficiency*: The code here is designed to be performant on real contests, and should maintain a low constant overhead. This is often challenging in the face of clarity and tweakability, but we can hope for contest setters to be liberal with time and memory limits. If the code here times out, you can reasonably rule out insufficient constant optimization and assume that you are choosing an algorithm from a suboptimal complexity class.
- *Genericness*: Implementations should be easy to adapt to achieve slightly different goals. One may want to tweak some core logic, parameters, data types, etc. In timed contests, we would certainly prefer this process to be as painless as possible. C++ templates are often used to increase tweakability at a slight cost to simplicity.

- *Portability*: Different contest environments use different compiler builds. In order to maximize compatibility, non-standard and newer features are avoided. The decision to follow C++98 standards is due to many contest systems being stuck on an older version of the language. Moreover, minimizing newer C++ features will make the code more language-agnostic.

As these points and the title both suggest, there is a slight bias towards contests. Compiling a codebook for my personal reference during contests was indeed how this project got started. This work has become much more multipurpose now. Whatever your use case is, I hope you discover something enlightening.

Cheers.
— Alex

Portability Notes

All programs were tested with GCC and compiled for a 32-bit target using the switches below:

```
g++ -std=gnu++98 -pedantic -Wall -Wno-long-long -O2
```

This means the following are assumed about data types:

- `bool` and `char` are 8-bit.
- `int` and `float` are 32-bit.
- `double` and `long long` are 64-bit.
- `long double` is 96-bit.

Programs are highly portable (ISO C++ 1998 compliant), except in the following regards:

- Usage of `long long` and dependent features, which are compliant in C99/C++0x or later. 64-bit integers are a must for many contest problems.
- Usage of variable sized arrays. While easily replaced by vectors, they are generally simpler and avoid dynamic memory (which some argue is a bad idea for contests).
- Usage of GCC's built-in functions like `__builtin_popcount()` and `__builtin_clz()`. These can be extremely convenient, but are straightforward to implement if unavailable. See here for a reference: <https://gcc.gnu.org/onlinedocs/gcc/Other-Builtins.html>
- Usage of compound-literals, e.g. `vec.push_back((mystruct){a, b, c})`. This adds a little more concision by not requiring a constructor definition.
- Hacks that may depend on the platform (e.g. endianness), such as getting the signbit with type-punned pointers. Be wary of portability for all bitwise/lower level code.

Contents

Preface	i
1 Elementary Algorithms	1
1.1 Array Transformations	1
1.1.1 Sorting Algorithms	1
1.1.2 Array Rotation	6
1.1.3 Counting Inversions	8
1.1.4 Coordinate Compression	10
1.1.5 Selection (Quickselect)	11
1.1.6 Dutch National Flag	13
1.2 Arrays and Dynamic Programming	14
1.2.1 Prefix Sums	14
1.2.2 Difference Array	15
1.2.3 Longest Increasing Subsequence	16
1.2.4 Subset Sum (Meet-in-the-Middle)	17
1.2.5 Knapsack (0-1)	18
1.2.6 Knapsack (Unbounded and Coin Change)	19
1.2.7 Maximal Zero Submatrix	20
1.2.8 Minimum Excluded Value (MEX)	21
1.2.9 Two Pointers and Sliding Window	23
1.3 Greedy and Scheduling	24
1.3.1 Interval Scheduling	24
1.3.2 Weighted Interval Scheduling	25
1.3.3 Interval Partitioning	26
1.3.4 Deadline Scheduling	27
1.3.5 Minimizing Late Jobs (Moore-Hodgson)	28
1.4 Streaming and Randomized Algorithms	29
1.4.1 Fisher-Yates Shuffle	29
1.4.2 Reservoir Sampling	30
1.4.3 Maximal Subarray Sum (Kadane)	31

1.4.4	Majority Element (Boyer-Moore)	33
1.4.5	Online Statistics (Welford)	34
1.4.6	Frequent Elements (Misra-Gries)	35
1.5	Cycle Detection	36
1.5.1	Cycle Detection (Floyd)	36
1.5.2	Cycle Detection (Brent)	38
2	Data Structures	40
2.1	Linked Lists	40
2.1.1	Singly Linked List Operations	40
2.1.2	Linked List Splicing	42
2.1.3	LRU Cache	43
2.2	Heaps and Priority Queues	45
2.2.1	Binary Heap	45
2.2.2	Skew Heap	46
2.2.3	Pairing Heap	49
2.3	Dictionaries and Ordered Sets	51
2.3.1	Binary Search Tree	51
2.3.2	Treap	54
2.3.3	AVL Tree	57
2.3.4	Red-Black Tree	60
2.3.5	Splay Tree	66
2.3.6	Size Balanced Tree	69
2.3.7	Interval Treap	73
2.3.8	Hash Map	77
2.3.9	Skip List	80
2.4	Range Queries in One Dimension	83
2.4.1	Sparse Table (Range Minimum Query)	83
2.4.2	Square Root Decomposition	84
2.4.3	Segment Tree (Point Update)	86
2.4.4	Segment Tree (Range Update)	88
2.4.5	Segment Tree (Compressed)	91
2.4.6	Segment Tree (Persistent)	94
2.4.7	Cartesian Tree	96
2.4.8	Implicit Treap	97
2.5	Range Queries in Two Dimensions	101
2.5.1	Quadtree (Point Update)	101
2.5.2	Quadtree (Range Update)	104
2.5.3	2D Segment Tree	107

2.5.4	2D Range Tree	111
2.5.5	K-d Tree (2D Range Query)	113
2.5.6	K-d Tree (Nearest Neighbor)	115
2.6	Fenwick Trees	117
2.6.1	Fenwick Tree (Simple)	117
2.6.2	Fenwick Tree (Range Update, Point Query)	119
2.6.3	Fenwick Tree (Point Update, Range Query)	120
2.6.4	Fenwick Tree (Range Update, Range Query)	121
2.6.5	Fenwick Tree (Compressed)	123
2.6.6	Fenwick Tree (Order Statistic)	124
2.6.7	2D Fenwick Tree (Simple)	126
2.6.8	2D Fenwick Tree (Compressed)	127
2.7	Disjoint Sets and Tree Structures	129
2.7.1	Disjoint Set Forest (Simple)	129
2.7.2	Disjoint Set Forest (Compressed)	131
2.7.3	Disjoint Set Forest (Rollback)	133
2.7.4	Lowest Common Ancestor (Sparse Table)	134
2.7.5	Lowest Common Ancestor (Segment Tree)	136
2.7.6	Heavy Light Decomposition	137
2.7.7	Link-Cut Tree	142
3	Optimization	148
3.1	Search on Answer	148
3.1.1	Binary Search	148
3.1.2	Exponential Search	150
3.2	Unimodal and Continuous Search	151
3.2.1	Ternary Search	151
3.2.2	Hill Climbing	152
3.2.3	Simulated Annealing	153
3.3	Dynamic Programming Optimization	155
3.3.1	Monotone Queue DP Optimization	155
3.3.2	Divide-and-Conquer DP Optimization	156
3.3.3	Knuth DP Optimization	158
3.3.4	Convex Hull Trick (Semi-Dynamic)	159
3.3.5	Convex Hull Trick (Fully-Dynamic)	160
3.3.6	Li Chao Tree	162
3.4	Linear and Convex Optimization	164
3.4.1	Linear Programming (Simplex)	164
3.4.2	Hungarian Algorithm	166

4	Strings	168
4.1	String Utilities	168
4.2	Hashing	172
4.2.1	Hash Functions	172
4.2.2	Rolling Hash	175
4.2.3	Zobrist Hashing	178
4.3	String Searching	180
4.3.1	String Searching (KMP)	180
4.3.2	String Searching (Z Algorithm)	181
4.3.3	String Searching (Aho-Corasick)	182
4.3.4	Palindrome Searching (Manacher)	184
4.4	Expression Parsing	186
4.4.1	Recursive Descent Parsing (Simple)	186
4.4.2	Recursive Descent Parsing (Generic)	187
4.4.3	Shunting Yard Parsing	191
4.5	Sequence Dynamic Programming	196
4.5.1	Longest Common Substring	196
4.5.2	Longest Common Subsequence	196
4.5.3	Sequence Alignment	198
4.6	Suffix Array and LCP	201
4.6.1	Suffix Array and LCP (Manber-Myers)	201
4.6.2	Suffix Array and LCP (Counting Sort)	203
4.6.3	Suffix Array and LCP (Linear DC3)	205
4.7	String Data Structures	208
4.7.1	Trie	208
4.7.2	Radix Tree	211
4.7.3	Suffix Automaton	214
4.7.4	Eertree	217
4.8	Encoding and Compression	219
4.8.1	Entropy and Frequency Tables	219
4.8.2	Run-Length Encoding	221
4.8.3	Base64 Encoding	222
4.8.4	Huffman Coding	223
5	Graphs	226
5.1	DFS and Tree Algorithms	226
5.1.1	Graph Class and Depth-First Search	226
5.1.2	Connected Components	228
5.1.3	Topological Sorting	229

5.1.4	Tree Centers and Diameter	231
5.1.5	Centroid Decomposition	232
5.1.6	Prufer Code	234
5.1.7	Eulerian Cycles	236
5.2	BFS and Shortest Path	237
5.2.1	Shortest Path (BFS)	237
5.2.2	Shortest Path (0-1 BFS)	239
5.2.3	Shortest Path (Dijkstra)	240
5.2.4	Shortest Path (Bellman-Ford)	242
5.2.5	Shortest Path (SPFA)	243
5.2.6	Shortest Path (Floyd-Warshall)	244
5.3	Connectivity	246
5.3.1	Strongly Connected Components (Kosaraju)	246
5.3.2	Strongly Connected Components (Tarjan)	247
5.3.3	Cut-points, Bridges and Bridge Forest	249
5.3.4	Bridge Finding (Online)	252
5.3.5	2-SAT	254
5.4	Spanning Trees	256
5.4.1	Minimum Spanning Tree (Prim)	256
5.4.2	Minimum Spanning Tree (Kruskal)	257
5.5	Flow	259
5.5.1	Maximum Flow (Ford-Fulkerson)	259
5.5.2	Maximum Flow (Edmonds-Karp)	260
5.5.3	Maximum Flow (Dinic)	261
5.5.4	Maximum Flow (Push-Relabel)	263
5.5.5	Min-Cost Max-Flow	264
5.6	Maximum Matching	266
5.6.1	Maximum Bipartite Matching (Kuhn)	266
5.6.2	Maximum Bipartite Matching (Hopcroft-Karp)	267
5.6.3	Maximum Graph Matching (Edmonds)	269
5.7	Hard Problems	271
5.7.1	Maximum Clique (Bron-Kerbosch)	271
5.7.2	Graph Coloring	273
5.7.3	Shortest Hamiltonian Cycle (TSP)	275
5.7.4	Shortest Hamiltonian Path	276
6	Mathematics	279
6.1	Math Utilities	279
6.2	Combinatorics	285

6.2.1	Combinatorial Calculations	285
6.2.2	Enumerating Arrangements	289
6.2.3	Enumerating Permutations	291
6.2.4	Enumerating Combinations	294
6.2.5	Enumerating Partitions	298
6.2.6	Enumerating Generic Combinatorial Sequences	301
6.3	Number Theory	304
6.3.1	GCD, LCM, Mod Inverse, Chinese Remainder	304
6.3.2	Modular Arithmetic	307
6.3.3	Prime Generation	310
6.3.4	Primality Testing	312
6.3.5	Integer Factorization	315
6.3.6	Euler's Totient Function	319
6.3.7	Binary Exponentiation	320
6.4	Arbitrary Precision Arithmetic	321
6.4.1	Big Integers (Simple)	321
6.4.2	Big Integers	324
6.4.3	Rational Numbers	334
6.5	Linear Algebra	337
6.5.1	Matrix Utilities	337
6.5.2	Row Reduction	344
6.5.3	Determinant and Inverse	346
6.5.4	LU Decomposition	348
6.6	Root Finding and Calculus	351
6.6.1	Root Finding (Bracketing)	351
6.6.2	Root Finding (Iteration)	353
6.6.3	Polynomial Root Finding (Differentiation)	354
6.6.4	Polynomial Root Finding (Laguerre)	356
6.6.5	Polynomial Root Finding (Ehrlich-Aberth)	358
6.6.6	Integration (Simpson)	361
6.7	Game Theory	362
6.7.1	Nim	362
6.7.2	Sprague-Grundy	363
6.7.3	Grundy Numbers on DAG	365
6.7.4	Minimax and Alpha-Beta Pruning	366
7	Geometry	368
7.1	Geometric Classes	368
7.1.1	Point	368

7.1.2	Line	370
7.1.3	Circle	372
7.1.4	Triangle	374
7.1.5	Rectangle	376
7.2	Elementary Geometric Calculations	378
7.2.1	Angles	378
7.2.2	Distances	380
7.2.3	Line Intersection	382
7.2.4	Circle Intersection	385
7.3	Intermediate Geometric Calculations	389
7.3.1	Polygon Sorting and Area	389
7.3.2	Point-in-Polygon (Ray Casting)	391
7.3.3	Convex Hull and Diametral Pair	392
7.3.4	Minimum Enclosing Circle	394
7.3.5	Closest Pair	396
7.3.6	Segment Intersection Finding	398
7.4	Advanced Geometric Computations	401
7.4.1	Convex Polygon Cut	401
7.4.2	Polygon Intersection and Union	403
7.4.3	Delaunay Triangulation (Simple)	406
7.4.4	Delaunay Triangulation (Fast)	409
7.5	Combined Geometry Template	413
7.5.1	Geometry Library in One File	413

Chapter 1

Elementary Algorithms

1.1 Array Transformations

1.1.1 Sorting Algorithms

These functions are equivalent to `std::sort()`, taking random-access iterators as a range `[lo, hi)` to be sorted. Elements between `lo` and `hi` (including the element pointed to by `lo` but excluding the element pointed to by `hi`) will be sorted into ascending order after the function call. Optionally, a comparison function object specifying a strict weak ordering may be specified to replace the default `operator <`.

These functions are not meant to compete with standard library implementations in terms of speed. Instead, they are meant to demonstrate how common sorting algorithms can be concisely implemented in C++.

```
#include <algorithm>
#include <functional>
#include <iterator>
#include <vector>
```

Quicksort repeatedly selects a pivot and partitions the range so that elements comparing less than the pivot precede the pivot, and elements comparing greater or equal follow it. Divide and conquer is then applied to both sides of the pivot until the original range is sorted. Despite having a worst case of $O(n^2)$, quicksort is often faster in practice than merge sort and heapsort, which both have a worst case time complexity of $O(n \log n)$.

The pivot chosen in this implementation is always a middle element of the range to be sorted. To reduce the likelihood of encountering the worst case, the pivot can be chosen in better ways (e.g. randomly, or using the "median of three" technique).

Time Complexity (Average): $O(n \log n)$.

Time Complexity (Worst): $O(n^2)$.

Space Complexity: $O(\log n)$ auxiliary stack space.

Stable?: No.

```
template<class It, class Compare>
void quicksort(It lo, It hi, Compare comp) {
    if (hi - lo < 2) {
        return;
    }
    typedef typename std::iterator_traits<It>::value_type T;
    T pivot = *(lo + (hi - lo) / 2);
    It i, j;
```

```

for (i = lo, j = hi - 1;; ++i, --j) {
    while (comp(*i, pivot)) {
        ++i;
    }
    while (comp(pivot, *j)) {
        --j;
    }
    if (i >= j) {
        break;
    }
    std::swap(*i, *j);
}
quicksort(lo, i, comp);
quicksort(i, hi, comp);
}

template<class It>
void quicksort(It lo, It hi) {
    typedef typename std::iterator_traits<It>::value_type T;
    quicksort(lo, hi, std::less<T>());
}

```

Merge sort first divides a list into n sublists of one element each, then recursively merges the sublists into sorted order until only a single sorted sublist remains. Merge sort is a stable sort, meaning that it preserves the relative order of elements which compare equal by `operator <` or the custom comparator given.

An analogous function in the C++ standard library is `std::stable_sort()`, except that the implementation here requires sufficient memory to be available. When $O(n)$ auxiliary memory is not available, `std::stable_sort()` falls back to a time complexity of $O(n \log^2 n)$ whereas the implementation here will simply fail.

Time Complexity (Average): $O(n \log n)$.

Time Complexity (Worst): $O(n \log n)$.

Space Complexity: $O(\log n)$ auxiliary stack space and $O(n)$ auxiliary heap space.

Stable?: Yes.

```

template<class It, class Compare>
void mergesort(It lo, It hi, Compare comp) {
    if (hi - lo < 2) {
        return;
    }
    It mid = lo + (hi - lo - 1) / 2, a = lo, c = mid + 1;
    mergesort(lo, mid + 1, comp);
    mergesort(mid + 1, hi, comp);
    typedef typename std::iterator_traits<It>::value_type T;
    std::vector<T> merged;
    while (a <= mid && c < hi) {
        merged.push_back(comp(*c, *a) ? *c++ : *a++);
    }
    if (a > mid) {
        for (It it = c; it < hi; ++it) {
            merged.push_back(*it);
        }
    } else {
        for (It it = a; it <= mid; ++it) {
            merged.push_back(*it);
        }
    }
    for (int i = 0; i < hi - lo; i++) {
        *(lo + i) = merged[i];
    }
}

template<class It>

```

```
void mergesort(It lo, It hi) {
    typedef typename std::iterator_traits<It>::value_type T;
    mergesort(lo, hi, std::less<T>());
}
```

Heapsort first rearranges an array to satisfy the max-heap property. Then, it repeatedly pops the max element of the heap (the left, unsorted subrange), moving it to the beginning of the right, sorted subrange until the entire range is sorted. Heapsort has a better worst case time complexity than quicksort and also a better space complexity than merge sort.

The C++ standard library equivalent is calling `std::make_heap(lo, hi)`, followed by `std::sort_heap(lo, hi)`.

Time Complexity (Average): $O(n \log n)$.

Time Complexity (Worst): $O(n \log n)$.

Space Complexity: $O(1)$ auxiliary.

Stable?: No.

```
template<class It, class Compare>
void heapsort(It lo, It hi, Compare comp) {
    typename std::iterator_traits<It>::value_type tmp;
    It i = lo + (hi - lo) / 2, j = hi, parent, child;
    for (;;) {
        if (i <= lo) {
            if (--j == lo) {
                return;
            }
            tmp = *j;
            *j = *lo;
        } else {
            tmp = *(--i);
        }
        parent = i;
        child = lo + 2 * (i - lo) + 1;
        while (child < j) {
            if (child + 1 < j && comp(*child, *(child + 1))) {
                child++;
            }
            if (!comp(tmp, *child)) {
                break;
            }
            *parent = *child;
            parent = child;
            child = lo + 2 * (parent - lo) + 1;
        }
        *(lo + (parent - lo)) = tmp;
    }
}

template<class It>
void heapsort(It lo, It hi) {
    typedef typename std::iterator_traits<It>::value_type T;
    heapsort(lo, hi, std::less<T>());
}
```

Comb sort is an improved bubble sort. While bubble sort increments the gap between swapped elements for every inner loop iteration, comb sort fixes the gap size in the inner loop, decreasing it by a particular shrink factor in every iteration of the outer loop. The shrink factor of 1.3 is empirically determined to be the most effective.

Even though the worst case time complexity is $O(n^2)$, a well chosen shrink factor ensures that the gap sizes are co-prime, in turn requiring astronomically large n to make the algorithm exceed $O(n \log n)$ steps. On random arrays, comb sort is only 2-3 times slower than merge sort. Its short code length length relative to its good

performance makes it a worthwhile algorithm to remember.

Time Complexity (Worst): $O(n^2)$.

Space Complexity: $O(1)$ auxiliary.

Stable?: No.

```
template<class It, class Compare>
void combsort(It lo, It hi, Compare comp) {
    int gap = hi - lo;
    bool swapped = true;
    while (gap > 1 || swapped) {
        if (gap > 1) {
            gap = (int)((double)gap / 1.3);
        }
        swapped = false;
        for (It it = lo; it + gap < hi; ++it) {
            if (comp(*(it + gap), *it)) {
                std::swap(*it, *(it + gap));
                swapped = true;
            }
        }
    }
}

template<class It>
void combsort(It lo, It hi) {
    typedef typename std::iterator_traits<It>::value_type T;
    combsort(lo, hi, std::less<T>());
}
```

Radix sort is used to sort integer elements with a constant number of bits in linear time. This implementation only works on ranges pointing to unsigned integer primitives. The elements in the input range do not strictly have to be unsigned types, as long as their values are nonnegative integers.

In this implementation, a power of two is chosen to be the base for the sort so that bitwise operations can be easily used to extract digits. This avoids the need to use modulo and exponentiation, which are much more expensive operations. In practice, it's been demonstrated that 2^8 is the best choice for sorting 32-bit integers (approximately 5 times faster than `std::sort()`, and typically 2-4 faster than radix sort using any other power of two chosen as the base).

Time Complexity: $O(n \cdot w)$ for n integers of w bits each.

Space Complexity: $O(n + w)$ auxiliary.

```
template<class UnsignedIt>
void radix_sort(UnsignedIt lo, UnsignedIt hi) {
    if (hi - lo < 2) {
        return;
    }
    const int radix_bits = 8;
    const int radix_base = 1 << radix_bits; // e.g. 2^8 = 256
    const int radix_mask = radix_base - 1; // e.g. 2^8 - 1 = 0xFF
    int num_bits = 8 * sizeof(*lo); // 8 bits per byte
    typedef typename std::iterator_traits<UnsignedIt>::value_type T;
    T *buf = new T[hi - lo];
    for (int pos = 0; pos < num_bits; pos += radix_bits) {
        int count[radix_base] = {0};
        for (UnsignedIt it = lo; it != hi; ++it) {
            count[( *it >> pos) & radix_mask]++;
        }
        T *bucket[radix_base], *curr = buf;
        for (int i = 0; i < radix_base; curr += count[i++]) {
            bucket[i] = curr;
        }
    }
```

```

    for (UnsignedIt it = lo; it != hi; ++it) {
        *bucket[( *it >> pos) & radix_mask]++ = *it;
    }
    std::copy(buf, buf + (hi - lo), lo);
}
delete[] buf;
}

```

Example Usage

```

#include <cassert>
#include <cstdlib>
#include <ctime>
#include <iomanip>
#include <iostream>
#include <vector>
using namespace std;

template<class It>
void print_range(It lo, It hi) {
    while (lo != hi) {
        cout << *lo++ << " ";
    }
    cout << endl;
}

template<class It>
bool sorted(It lo, It hi) {
    while (++lo != hi) {
        if (*lo < *(lo - 1)) {
            return false;
        }
    }
    return true;
}

bool compare_as_ints(double i, double j) {
    return (int)i < (int)j;
}

int main() {
    { // Can be used to sort arrays like std::sort().
        int a[] = {32, 71, 12, 45, 26, 80, 53, 33};
        quicksort(a, a + 8);
        assert(sorted(a, a + 8));
    }
    { // STL containers work too.
        int a[] = {32, 71, 12, 45, 26, 80, 53, 33};
        vector<int> v(a, a + 8);
        quicksort(v.begin(), v.end());
        assert(sorted(v.begin(), v.end()));
    }
    { // Reverse iterators work as expected.
        int a[] = {32, 71, 12, 45, 26, 80, 53, 33};
        vector<int> v(a, a + 8);
        heapsort(v.rbegin(), v.rend());
        assert(sorted(v.rbegin(), v.rend()));
    }
    { // We can sort doubles just as well.
        double a[] = {1.1, -5.0, 6.23, 4.123, 155.2};
        vector<double> v(a, a + 5);
        combsort(v.begin(), v.end());
        assert(sorted(v.begin(), v.end()));
    }
    { // Must use radix_sort with unsigned values, but sorting in reverse works!
        int a[] = {32, 71, 12, 45, 26, 80, 53, 33};
    }
}

```



```

    vector<int> v(a, a + 8);
    radix_sort(v.rbegin(), v.rend());
    assert(sorted(v.rbegin(), v.rend()));
}

// Example from: http://www.cplusplus.com/reference/algorithm/stable\_sort
double a[] = {3.14, 1.41, 2.72, 4.67, 1.73, 1.32, 1.62, 2.58};
{
    vector<double> v(a, a + 8);
    cout << "mergesort() with default comparisons: ";
    mergesort(v.begin(), v.end());
    print_range(v.begin(), v.end());
}
{
    vector<double> v(a, a + 8);
    cout << "mergesort() with 'compare_as_ints()': ";
    mergesort(v.begin(), v.end(), compare_as_ints);
    print_range(v.begin(), v.end());
}
cout << "-----" << endl;

vector<int> v, v2;
for (int i = 0; i < 5000000; i++) {
    v.push_back((rand() & 0x7fff) | ((rand() & 0x7fff) << 15));
}
v2 = v;
cout << "Sorting five million integers..." << endl;
cout.precision(3);

#define test(sort_function) \
{ \
    clock_t start = clock(); \
    sort_function(v.begin(), v.end()); \
    double t = (double)(clock() - start) / CLOCKS_PER_SEC; \
    cout << setw(14) << left << #sort_function "()": "; \
    cout << fixed << t << "s" << endl; \
    assert(sorted(v.begin(), v.end())); \
    v = v2; \
}

test(std::sort);
test(quicksort);
test(mergesort);
test(heap_sort);
test(combsort);
test(radix_sort);
return 0;
}

```

Example Output

```

mergesort() with default comparisons: 1.32 1.41 1.62 1.73 2.58 2.72 3.14 4.67
mergesort() with 'compare_as_ints()': 1.41 1.73 1.32 1.62 2.72 2.58 3.14 4.67
-----
Sorting five million integers...
std::sort(): 0.355s
quicksort(): 0.426s
mergesort(): 1.263s
heap_sort(): 1.093s
combsort(): 0.827s
radix_sort(): 0.076s

```

1.1.2 Array Rotation

These functions are equivalent to `std::rotate()`, taking three iterators `lo`, `mid`, and `hi` ($lo \leq mid \leq hi$) to perform a left rotation on the range `[lo, hi)`. After the function call, `[lo, hi)` will comprise of the concatenation

of the elements originally in `[mid, hi)` + `[lo, mid)`. That is, the range `[lo, hi)` will be rearranged in such a way that the element at `mid` becomes the first element of the new range and the element at `mid - 1` becomes the last element, all while preserving the relative ordering of elements within the two rotated subarrays.

All three versions below achieve the same result using in-place algorithms. Version 1 uses a straightforward swapping algorithm requiring `ForwardIterators`. Version 2 requires `BidirectionalIterators`, employing a well-known trick with three simple inversions. Version 3 requires random-access iterators, applying a juggling algorithm which first divides the range into `gcd(hi - lo, mid - lo)` sets and then rotates the corresponding elements in each set.

Time Complexity: $O(n)$ per call to both functions, where n is the distance between `lo` and `hi`.

Space Complexity: $O(1)$ auxiliary for all versions

```
#include <algorithm>

template<class It>
void rotate1(It lo, It mid, It hi) {
    It next = mid;
    while (lo != next) {
        std::iter_swap(lo++, next++);
        if (next == hi) {
            next = mid;
        } else if (lo == mid) {
            mid = next;
        }
    }
}

template<class It>
void rotate2(It lo, It mid, It hi) {
    std::reverse(lo, mid);
    std::reverse(mid, hi);
    std::reverse(lo, hi);
}

int gcd(int a, int b) {
    return (b == 0) ? a : gcd(b, a % b);
}

template<class It>
void rotate3(It lo, It mid, It hi) {
    int n = hi - lo, jump = mid - lo;
    int g = gcd(jump, n), cycle = n / g;
    for (int i = 0; i < g; i++) {
        int curr = i, next;
        for (int j = 0; j < cycle - 1; j++) {
            next = curr + jump;
            if (next >= n) {
                next -= n;
            }
            std::iter_swap(lo + curr, lo + next);
            curr = next;
        }
    }
}
```

Example Usage

```
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>
using namespace std;

int main() {
```

```

vector<int> v0, v1, v2, v3;
for (int i = 0; i < 10000; i++) {
    v0.push_back(i);
}
v1 = v2 = v3 = v0;
int mid = 5678;
std::rotate(v0.begin(), v0.begin() + mid, v0.end());
rotate1(v1.begin(), v1.begin() + mid, v1.end());
rotate2(v2.begin(), v2.begin() + mid, v2.end());
rotate3(v3.begin(), v3.begin() + mid, v3.end());
assert(v0 == v1 && v0 == v2 && v0 == v3);

// Example from: http://en.cppreference.com/w/cpp/algorithm/rotate
int a[] = {2, 4, 2, 0, 5, 10, 7, 3, 7, 1};
vector<int> v(a, a + 10);
cout << "before sort: ";
for (int i = 0; i < (int)v.size(); i++) {
    cout << v[i] << " ";
}
cout << endl;

// Insertion sort.
for (vector<int>::iterator i = v.begin(); i != v.end(); ++i) {
    rotate1(std::upper_bound(v.begin(), i, *i), i, i + 1);
}
cout << "after sort: ";
for (int i = 0; i < (int)v.size(); i++) {
    cout << v[i] << " ";
}
cout << endl;

// Simple rotation to the left.
rotate2(v.begin(), v.begin() + 1, v.end());
cout << "rotate left: ";
for (int i = 0; i < (int)v.size(); i++) {
    cout << v[i] << " ";
}
cout << endl;

// Simple rotation to the right.
rotate3(v.rbegin(), v.rbegin() + 1, v.rend());
cout << "rotate right: ";
for (int i = 0; i < (int)v.size(); i++) {
    cout << v[i] << " ";
}
cout << endl;

return 0;
}

```

Example Output

```

before sort:  2 4 2 0 5 10 7 3 7 1
after sort:   0 1 2 2 3 4 5 7 7 10
rotate left:  1 2 2 3 4 5 7 7 10 0
rotate right: 0 1 2 2 3 4 5 7 7 10

```

1.1.3 Counting Inversions

The number of inversions for an array $a[]$ the number of ordered pairs (i, j) such that $i < j$ and $a[i] > a[j]$. This is roughly how "close" an array is to being sorted, but is *not* the minimum number of swaps required to sort the array. If the array is sorted, then the inversion count is 0. If the array is sorted in decreasing order, then the inversion count is maximal. The following two functions are each techniques to efficiently count inversions.

- `inversions(lo, hi)` uses merge sort to return the number of inversions given two random-access iterators as a range `[lo, hi)`. The input range will be sorted after the function call. This requires `operator <` to be defined on the iterators' value type.
- `inversions(n, a[])` uses a power-of-two trick to return the number of inversions for an array `a[]` of n nonnegative integers. After calling the function, every value of `a[]` will be set to 0. The time and space complexity of this operation are functions of the magnitude of the maximum value in `a[]`. To instead obtain a running time of $O(n \log n)$ on the number of elements, coordinate compression may be applied to `a[]` beforehand so that its maximum is strictly less than the length n itself.

Time Complexity:

- $O(n \log n)$ per call to `inversion(lo, hi)`, where n is the distance between `lo` and `hi`.
- $O(n \log m)$ per call to `inversions(n, a[])` where n is the distance between `lo` and `hi` and m is maximum value in `a[]`.

Space Complexity:

- $O(n)$ auxiliary space and $O(\log n)$ stack space for `inversions(lo, hi)`.
- $O(m)$ auxiliary heap space for `inversions(n, a[])`.

```
#include <algorithm>
#include <iterator>
#include <vector>

template<class It>
long long inversions(It lo, It hi) {
    if (hi - lo < 2) {
        return 0;
    }
    It mid = lo + (hi - lo - 1) / 2, a = lo, c = mid + 1;
    long long res = 0;
    res += inversions(lo, mid + 1);
    res += inversions(mid + 1, hi);
    typedef typename std::iterator_traits<It>::value_type T;
    std::vector<T> merged;
    while (a <= mid && c < hi) {
        if (*c < *a) {
            merged.push_back(*(c++));
            res += (mid - a) + 1;
        } else {
            merged.push_back(*(a++));
        }
    }
    if (a > mid) {
        for (It it = c; it != hi; ++it) {
            merged.push_back(*it);
        }
    } else {
        for (It it = a; it <= mid; ++it) {
            merged.push_back(*it);
        }
    }
    for (It it = lo; it != hi; ++it) {
        *it = merged[it - lo];
    }
    return res;
}

long long inversions(int n, int a[]) {
    int mx = 0;
    for (int i = 0; i < n; i++) {
        mx = std::max(mx, a[i]);
    }
    long long res = 0;
    std::vector<int> count(mx);
    while (mx > 0) {
```

```

std::fill(count.begin(), count.end(), 0);
for (int i = 0; i < n; i++) {
    if (a[i] % 2 == 0) {
        res += count[a[i] / 2];
    } else {
        count[a[i] / 2]++;
    }
}
mx = 0;
for (int i = 0; i < n; i++) {
    mx = std::max(mx, a[i] / 2);
}
return res;
}

```

Example Usage

```

#include <cassert>

int main() {
    {
        int a[] = {6, 9, 1, 14, 8, 12, 3, 2};
        assert(inversions(a, a + 8) == 16);
    }
    {
        int a[] = {6, 9, 1, 14, 8, 12, 3, 2};
        assert(inversions(8, a) == 16);
    }
    return 0;
}

```

1.1.4 Coordinate Compression

Given two ForwardIterators as a range `[lo, hi)` of n numerical elements, reassign each element in the range to an integer in $[0, k)$, where k is the number of distinct elements in the original range, while preserving the initial relative ordering of elements. That is, if a is the array of original values and b is the array of compressed values, then every pair of indices i, j in $[0, n)$ shall satisfy $a[i] < a[j]$ if and only if $b[i] < b[j]$.

Both implementations below require `operator <` to be defined on the iterator's value type. Version 1 performs the compression by sorting the array, removing duplicates, and binary searching for the position of each original value. Version 2 achieves the same result by inserting all values in a balanced binary search tree (`std::map`) which automatically removes duplicate values and supports efficient lookups of the compressed values.

Time Complexity: $O(n \log n)$ per call to either function, where n is the distance between `lo` and `hi`.

Space Complexity: $O(n)$ auxiliary heap space.

```

#include <algorithm>
#include <iterator>
#include <map>
#include <vector>

template<class It>
void compress1(It lo, It hi) {
    typedef typename std::iterator_traits<It>::value_type T;
    std::vector<T> v(lo, hi);
    std::sort(v.begin(), v.end());
    v.resize(std::unique(v.begin(), v.end()) - v.begin());
    for (It it = lo; it != hi; ++it) {
        *it = (int)(std::lower_bound(v.begin(), v.end(), *it) - v.begin());
    }
}

```

```

}

template<class It>
void compress2(It lo, It hi) {
    typedef typename std::iterator_traits<It>::value_type T;
    std::map<T, int> m;
    for (It it = lo; it != hi; ++it) {
        m[*it] = 0;
    }
    typename std::map<T, int>::iterator it = m.begin();
    for (int id = 0; it != m.end(); it++) {
        it->second = id++;
    }
    for (It it = lo; it != hi; ++it) {
        *it = m[*it];
    }
}

```

Example Usage

```

#include <iostream>
using namespace std;

template<class It>
void print_range(It lo, It hi) {
    while (lo != hi) {
        cout << *lo++ << " ";
    }
    cout << endl;
}

int main() {
    {
        int a[] = {1, 30, 30, 7, 9, 8, 99, 99};
        compress1(a, a + 8);
        print_range(a, a + 8);
    }
    {
        int a[] = {1, 30, 30, 7, 9, 8, 99, 99};
        compress2(a, a + 8);
        print_range(a, a + 8);
    }
    { // Non-integral types work too, as long as ints can be assigned to them.
        double a[] = {0.5, -1.0, 3, -1.0, 20, 0.5};
        compress1(a, a + 6);
        print_range(a, a + 6);
    }
    return 0;
}

```

Example Output

```

0 4 4 1 3 2 5 5
0 4 4 1 3 2 5 5
1 0 2 0 3 1

```

1.1.5 Selection (Quickselect)

`nth_element2()` is equivalent to `std::nth_element()`, taking random-access iterators `lo`, `nth`, and `hi` as the range `[lo, hi)` to be partially sorted. The values in `[lo, hi)` are rearranged such that the value pointed to by `nth` is the element that would be there if the range were sorted. Furthermore, the range is partitioned such that no value in `[lo, nth)` compares greater than the value pointed to by `nth` and no value in `(nth, hi)` compares less. This implementation requires `operator <` to be defined on the iterator's value type.

Time Complexity: $O(n)$ on average per call to `nth_element2()`, where n is the distance between `lo` and `hi`.

Space Complexity: $O(1)$ auxiliary.

```
#include <algorithm>
#include <cstdlib>
#include <iterator>

int rand32() {
    return (rand() & 0x7fff) | ((rand() & 0x7fff) << 15);
}

template<class It>
void nth_element2(It lo, It nth, It hi) {
    for (;;) {
        std::iter_swap(lo + rand32() % (hi - lo), hi - 1);
        typename std::iterator_traits<It>::value_type mid = *(hi - 1);
        It k = lo - 1;
        for (It it = lo; it != hi; ++it) {
            if (!(mid < *it)) {
                std::iter_swap(++k, it);
            }
        }
        if (nth < k) {
            hi = k;
        } else if (k < nth) {
            lo = k + 1;
        } else {
            return;
        }
    }
}
```

Example Usage

```
#include <cassert>
#include <iostream>
using namespace std;

template<class It>
void print_range(It lo, It hi) {
    while (lo != hi) {
        cout << *lo++ << " ";
    }
    cout << endl;
}

int main() {
    int n = 9;
    int a[] = {5, 6, 4, 3, 2, 6, 7, 9, 3};
    nth_element2(a, a + n / 2, a + n);
    assert(a[n / 2] == 5);
    print_range(a, a + n);
    return 0;
}
```

Example Output

```
2 3 3 4 5 6 6 7 9
```

1.1.6 Dutch National Flag

Partitions a range into three consecutive groups according to a pivot value: elements less than the pivot, elements equal to the pivot, and elements greater than the pivot. This is the Dutch National Flag algorithm, and is the core partitioning step used in three-way quicksort and arrays with many duplicate keys.

- `partition_three_way(lo, hi, pivot)` rearranges the range `[lo, hi)` in-place and returns iterators `(mid_lo, mid_hi)`, where `[lo, mid_lo)` is less than `pivot`, `[mid_lo, mid_hi)` is equal to `pivot`, and `[mid_hi, hi)` is greater than `pivot`.
- `sort_012(lo, hi)` sorts a range whose values are all `0`, `1`, or `2`.

Time Complexity: $O(n)$ per call, where n is the distance between `lo` and `hi`.

Space Complexity: $O(1)$ auxiliary.

```
#include <algorithm>
#include <utility>

template<class It, class T>
std::pair<It, It> partition_three_way(It lo, It hi, const T &pivot) {
    It lt = lo, i = lo, gt = hi;
    while (i != gt) {
        if (*i < pivot) {
            std::iter_swap(lt++, i++);
        } else if (pivot < *i) {
            std::iter_swap(i, --gt);
        } else {
            ++i;
        }
    }
    return std::make_pair(lt, gt);
}

template<class It>
void sort_012(It lo, It hi) {
    partition_three_way(lo, hi, 1);
}
```

Example Usage

```
#include <cassert>
#include <vector>
using namespace std;

int main() {
    int raw[] = {2, 0, 1, 2, 1, 0, 1};
    vector<int> a(raw, raw + 7);
    sort_012(a.begin(), a.end());
    for (int i = 0; i < 7; i++) {
        assert(a[i] == (i < 2 ? 0 : i < 5 ? 1 : 2));
    }

    int b_raw[] = {4, 2, 5, 3, 3, 1};
    vector<int> b(b_raw, b_raw + 6);
    pair<vector<int>::iterator, vector<int>::iterator> mid =
        partition_three_way(b.begin(), b.end(), 3);
    for (vector<int>::iterator it = b.begin(); it != mid.first; ++it) {
        assert(*it < 3);
    }
    for (vector<int>::iterator it = mid.first; it != mid.second; ++it) {
        assert(*it == 3);
    }
    for (vector<int>::iterator it = mid.second; it != b.end(); ++it) {
        assert(*it > 3);
    }
}
```



```
    return 0;
}
```

1.2 Arrays and Dynamic Programming

1.2.1 Prefix Sums

Precomputes prefix sums so range-sum queries can be answered in constant time. This is one of the most common array preprocessing tools, and also appears as a building block for subarray sums, difference arrays, and two-dimensional grids.

- `prefix_sums(a)` returns array `pref` with `pref[0] = 0` and `pref[i + 1] = a[0] + ... + a[i]`.
- `range_sum(pref, l, r)` returns the sum of the half-open range `[l, r)`.
- `prefix_sums_2d(a)` returns a two-dimensional prefix sum table for matrix `a`.
- `rectangle_sum(pref, r1, c1, r2, c2)` returns the sum of rows `[r1, r2)` and columns `[c1, c2)`.

Time Complexity:

- $O(n)$ per call to `prefix_sums(a)`, where n is the array size.
- $O(RC)$ per call to `prefix_sums_2d(a)`, where R and C are matrix dimensions.
- $O(1)$ per range or rectangle query.

Space Complexity:

- $O(n)$ for one-dimensional prefix sums.
- $O(RC)$ for two-dimensional prefix sums.

```
#include <vector>

std::vector<long long> prefix_sums(const std::vector<int> &a) {
    std::vector<long long> pref(a.size() + 1, 0);
    for (int i = 0; i < (int)a.size(); i++) {
        pref[i + 1] = pref[i] + a[i];
    }
    return pref;
}

long long range_sum(const std::vector<long long> &pref, int l, int r) {
    return pref[r] - pref[l];
}

std::vector<std::vector<long long>> prefix_sums_2d(const std::vector<std::vector<int>> &a) {
    int rows = a.size(), cols = rows == 0 ? 0 : a[0].size();
    std::vector<std::vector<long long>> pref(rows + 1, std::vector<long long>(cols + 1, 0));
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            pref[r + 1][c + 1] = a[r][c] + pref[r][c + 1] + pref[r + 1][c] - pref[r][c];
        }
    }
    return pref;
}

long long rectangle_sum(
    const std::vector<std::vector<long long>> &pref, int r1, int c1, int r2, int c2
) {
    return pref[r2][c2] - pref[r1][c2] - pref[r2][c1] + pref[r1][c1];
}
```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    int raw[] = {3, -1, 4, 1, 5};
    vector<int> a(raw, raw + 5);
    vector<long long> pref = prefix_sums(a);
    assert(range_sum(pref, 0, 5) == 12);
    assert(range_sum(pref, 1, 4) == 4);

    vector<vector<int>> grid(2, vector<int>(3));
    grid[0][0] = 1;
    grid[0][1] = 2;
    grid[0][2] = 3;
    grid[1][0] = 4;
    grid[1][1] = 5;
    grid[1][2] = 6;
    vector<vector<long long>> ps = prefix_sums_2d(grid);
    assert(rectangle_sum(ps, 0, 1, 2, 3) == 16);
    return 0;
}

```

1.2.2 Difference Array

Applies many range-add updates to an array in linear total time using a difference array. Instead of immediately adding `delta` to every element of `[l, r)`, add `delta` at `diff[l]` and subtract it at `diff[r]`; a final prefix sum reconstructs the updated values.

- `difference_array(n)` constructs an initially zero array of size `n`.
- `add(l, r, delta)` adds `delta` to every position in the half-open range `[l, r)`.
- `build()` returns the final array after all range updates.

Time Complexity:

- $O(1)$ per call to `add(l, r, delta)`.
- $O(n)$ per call to `build()`.

Space Complexity: $O(n)$ for the difference array.

```

#include <vector>

class difference_array {
    std::vector<long long> diff;

public:
    explicit difference_array(int n) : diff(n + 1, 0) {}

    void add(int l, int r, long long delta) {
        diff[l] += delta;
        diff[r] -= delta;
    }

    std::vector<long long> build() const {
        std::vector<long long> res(diff.size() - 1);
        long long cur = 0;
        for (int i = 0; i < (int)res.size(); i++) {
            cur += diff[i];
            res[i] = cur;
        }
        return res;
    }
};

```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    difference_array d(5);
    d.add(0, 3, 2);
    d.add(1, 5, 4);
    d.add(2, 4, -1);
    vector<long long> a = d.build();
    long long expected[] = {2, 6, 5, 3, 4};
    for (int i = 0; i < 5; i++) {
        assert(a[i] == expected[i]);
    }
    return 0;
}
```

1.2.3 Longest Increasing Subsequence

Given two random-access iterators `lo` and `hi` specifying a range `[lo, hi)`, determine a longest subsequence of the range such that all of its elements are in strictly ascending order. This implementation requires `operator <` to be defined on the iterator's value type. The subsequence is not necessarily contiguous or unique, so only one such answer will be found. The answer is computed using binary search and dynamic programming.

Time Complexity: $O(n \log n)$ per call to `longest_increasing_subsequence()`, where n is the distance between `lo` and `hi`.

Space Complexity: $O(n)$ auxiliary heap space for `longest_increasing_subsequence()`.

```
#include <iterator>
#include <vector>

template<class It>
std::vector<typename std::iterator_traits<It>::value_type> longest_increasing_subsequence(
    It lo, It hi
) {
    int len = 0, n = hi - lo;
    if (n == 0) {
        return std::vector<typename std::iterator_traits<It>::value_type>();
    }
    std::vector<int> prev(n), tail(n);
    for (int i = 0; i < n; i++) {
        int l = -1, h = len;
        while (h - l > 1) {
            int mid = (l + h) / 2;
            if (*(lo + tail[mid]) < *(lo + i)) {
                l = mid;
            } else {
                h = mid;
            }
        }
        if (len < h + 1) {
            len = h + 1;
        }
        prev[i] = h > 0 ? tail[h - 1] : -1;
        tail[h] = i;
    }
    std::vector<typename std::iterator_traits<It>::value_type> res(len);
    for (int i = tail[len - 1]; i != -1; i = prev[i]) {
        res[--len] = *(lo + i);
    }
    return res;
}
```

```
}
```

Example Usage

```
#include <iostream>
using namespace std;

template<class It>
void print_range(It lo, It hi) {
    while (lo != hi) {
        cout << *lo++ << " ";
    }
    cout << endl;
}

int main() {
    int a[] = {-2, -5, 1, 9, 10, 8, 11, 10, 13, 11};
    vector<int> res = longest_increasing_subsequence(a, a + 10);
    print_range(res.begin(), res.end());
    return 0;
}
```

Example Output

```
-5 1 9 10 11 13
```

1.2.4 Subset Sum (Meet-in-the-Middle)

Given random-access iterators `lo` and `hi` specifying a range `[lo, hi)` of integers, return the minimum sum of any subset of the range that is greater than or equal to a given integer `v`. This is a generalization of the NP-complete subset sum problem, which asks whether a subset summing to 0 exists (equivalent in this case to checking if $v = 0$ yields an answer of 0). This implementation uses a meet-in-the-middle algorithm to precompute and search for a lower bound. Note that 64-bit integers are used in intermediate calculations to avoid overflow.

Time Complexity: $O(n \cdot 2^{n/2})$ per call to `sum_lower_bound()`, where n is the distance between `lo` and `hi`.

Space Complexity: $O(n)$ auxiliary heap space, where n is the number of array elements.

```
#include <algorithm>
#include <limits>
#include <vector>

template<class It>
long long sum_lower_bound(It lo, It hi, long long v) {
    int n = hi - lo, llen = 1 << (n / 2), hlen = 1 << (n - n / 2);
    std::vector<long long> lsum(llen), hsum(hlen);
    for (int mask = 0; mask < llen; mask++) {
        for (int i = 0; i < n / 2; i++) {
            if ((mask >> i) & 1) {
                lsum[mask] += *(lo + i);
            }
        }
    }
    for (int mask = 0; mask < hlen; mask++) {
        for (int i = 0; i < (n - n / 2); i++) {
            if ((mask >> i) & 1) {
                hsum[mask] += *(lo + i + n / 2);
            }
        }
    }
    std::sort(lsum.begin(), lsum.end());
    std::sort(hsum.begin(), hsum.end());
```

```

int l = 0, h = hlen - 1;
long long curr = std::numeric_limits<long long>::min();
while (l < llen && h >= 0) {
    if (lsum[l] + hsum[h] <= v) {
        curr = std::max(curr, lsum[l] + hsum[h]);
        l++;
    } else {
        h--;
    }
}
return curr;
}

```

Example Usage

```

#include <cassert>

int main() {
    int a[] = {9, 1, 5, 0, 1, 11, 5};
    assert(sum_lower_bound(a, a + 7, 8) == 7);
    int b[] = {-7, -3, -2, 5, 8};
    assert(sum_lower_bound(b, b + 5, 0) == 0);
    return 0;
}

```

1.2.5 Knapsack (0-1)

Solves the 0/1 knapsack problem: given items with nonnegative integer weights and values, choose each item at most once to maximize total value without exceeding a capacity limit.

The one-dimensional dynamic programming array `dp[w]` stores the best value that can be achieved with total weight at most `w` after processing some prefix of the items. Iterating weights in decreasing order is what enforces the 0/1 constraint; it prevents the same item from being reused during the current item update.

- `knapsack_01(weight, value, capacity)` returns the maximum value achievable with total weight at most `capacity`.
- `weight[i]` and `value[i]` are the weight and value of item `i`.
- All weights must be nonnegative integers. Items with weight `0` are allowed and can each be taken once.

Time Complexity: $O(nW)$, where n is the number of items and W is `capacity`.

Space Complexity: $O(W)$ auxiliary heap space.

```

#include <algorithm>
#include <cassert>
#include <vector>

long long knapsack_01(
    const std::vector<int> &weight, const std::vector<long long> &value, int capacity
) {
    int n = weight.size();
    std::vector<long long> dp(capacity + 1, 0);
    for (int i = 0; i < n; i++) {
        for (int w = capacity; w >= weight[i]; w--) {
            dp[w] = std::max(dp[w], dp[w - weight[i]] + value[i]);
        }
    }
    return dp[capacity];
}

```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    int w_raw[] = {3, 4, 5, 9};
    long long v_raw[] = {4, 5, 7, 10};
    vector<int> weight(w_raw, w_raw + 4);
    vector<long long> value(v_raw, v_raw + 4);

    assert(knapsack_01(weight, value, 8) == 11); // Items of weight 3 and 5.
    assert(knapsack_01(weight, value, 10) == 12); // Items of weight 4 and 5.
    return 0;
}
```

1.2.6 Knapsack (Unbounded and Coin Change)

Solves two common unbounded knapsack variants, where each item may be used any number of times. In the maximum-value version, each item has a weight and value, and the goal is to maximize total value subject to a capacity limit. In the coin change version, each coin denomination may be used any number of times, and the goal is to count unordered ways to form a target sum.

For unbounded maximum-value knapsack, weights are iterated in increasing order so the current item may be reused. For unordered coin change, coins are the outer loop so each multiset of coins is counted once, independent of ordering.

- `unbounded_knapsack(weight, value, capacity)` returns the maximum value achievable with total weight at most `capacity`.
- `count_coin_change(coins, target)` returns the number of unordered ways to make sum `target` using the given coin denominations.
- All capacities must be nonnegative integers. Weights and coin denominations should be positive to avoid infinitely many uses of a zero-weight item.

Time Complexity:

- $O(nW)$ for `unbounded_knapsack(weight, value, capacity)`, where n is the number of items and W is `capacity`.
- $O(ct)$ for `count_coin_change(coins, target)`, where c is the number of coin denominations and t is `target`.

Space Complexity:

- $O(W)$ auxiliary heap space for `unbounded_knapsack(weight, value, capacity)`.
- $O(t)$ auxiliary heap space for `count_coin_change(coins, target)`.

```
#include <algorithm>
#include <vector>

long long unbounded_knapsack(
    const std::vector<int> &weight, const std::vector<long long> &value, int capacity
) {
    int n = weight.size();
    std::vector<long long> dp(capacity + 1, 0);
    for (int i = 0; i < n; i++) {
        for (int w = weight[i]; w <= capacity; w++) {
            dp[w] = std::max(dp[w], dp[w - weight[i]] + value[i]);
        }
    }
    return dp[capacity];
}

long long count_coin_change(const std::vector<int> &coins, int target) {
    std::vector<long long> dp(target + 1, 0);
```

```

dp[0] = 1;
for (int i = 0; i < (int)coins.size(); i++) {
    for (int sum = coins[i]; sum <= target; sum++) {
        dp[sum] += dp[sum - coins[i]];
    }
}
return dp[target];
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    int w_raw[] = {3, 4, 5};
    long long v_raw[] = {4, 5, 7};
    vector<int> weight(w_raw, w_raw + 3);
    vector<long long> value(v_raw, v_raw + 3);
    assert(unbounded_knapsack(weight, value, 10) == 14); // Two weight-5 items.

    int c_raw[] = {1, 2, 5};
    vector<int> coins(c_raw, c_raw + 3);
    assert(count_coin_change(coins, 5) == 4); // 5, 2+2+1, 2+1+1+1, 1*5.
    return 0;
}

```

1.2.7 Maximal Zero Submatrix

Given a rectangular matrix with n rows and m columns consisting of only 0's and 1's as a two-dimensional vector of bool, return the area of the largest rectangular submatrix consisting of only 0's. This solution uses a reduction to the problem of finding the maximum rectangular area under a histogram, which is efficiently solved using a stack algorithm.

Time Complexity: $O(n \cdot m)$ per call to `max_zero_submatrix()`, where n is the number of rows and m is the number of columns in the matrix.

Space Complexity: $O(m)$ auxiliary heap space, where m is the number of columns in the matrix.

```

#include <algorithm>
#include <stack>
#include <vector>

int max_zero_submatrix(const std::vector<std::vector<bool>> &matrix) {
    if (matrix.empty()) {
        return 0;
    }
    int n = matrix.size(), m = matrix[0].size(), res = 0;
    std::vector<int> d(m, -1), d1(m), d2(m);
    for (int r = 0; r < n; r++) {
        for (int c = 0; c < m; c++) {
            if (matrix[r][c]) {
                d[c] = r;
            }
        }
        std::stack<int> s;
        for (int c = 0; c < m; c++) {
            while (!s.empty() && d[s.top()] <= d[c]) {
                s.pop();
            }
            d1[c] = s.empty() ? -1 : s.top();
            s.push(c);
        }
        res = max(res, (d2[c] - d1[c] + 1) * (d[c] - d[c] + 1));
    }
    return res;
}

```

```

    }
    while (!s.empty()) {
        s.pop();
    }
    for (int c = m - 1; c >= 0; c--) {
        while (!s.empty() && d[s.top()] <= d[c]) {
            s.pop();
        }
        d2[c] = s.empty() ? m : s.top();
        s.push(c);
    }
    for (int j = 0; j < m; j++) {
        res = std::max(res, (r - d[j]) * (d2[j] - d1[j] - 1));
    }
}
return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    const int n = 5, m = 6;
    bool a[n][m] = {
        {1, 0, 1, 1, 0, 0},
        {1, 0, 0, 1, 0, 0},
        {0, 0, 0, 0, 0, 1},
        {1, 0, 0, 1, 0, 0},
        {1, 0, 1, 0, 0, 1}
    };
    vector<vector<bool>> matrix(n);
    for (int i = 0; i < n; i++) {
        matrix[i] = vector<bool>(a[i], a[i] + m);
    }
    assert(max_zero_submatrix(matrix) == 6);
    return 0;
}

```

1.2.8 Minimum Excluded Value (MEX)

Computes the minimum excluded nonnegative integer, commonly called the MEX. The MEX of a set is the smallest integer $x \geq 0$ that does not appear in the set. It appears frequently in array problems, game theory with Grundy numbers, and dynamic programming states.

- `mex(lo, hi)` returns the MEX of the values in `[lo, hi)`.
- `dynamic_mex(n)` maintains counts of values in `[0, n]`, enough to track the MEX of a multiset of at most `n` relevant nonnegative values.
- `add(x)` inserts one copy of value `x` if `0 <= x <= n`.
- `remove(x)` removes one copy of value `x` if present.
- `get()` returns the current MEX.

Time Complexity:

- $O(n)$ per call to `mex(lo, hi)`, where n is the number of values.
- $O(\log n)$ per call to `add(x)`, `remove(x)`, and `get()` for `dynamic_mex`.

Space Complexity: $O(n)$ auxiliary heap space for both approaches.

```

#include <set>

```



```

#include <vector>

template<class It>
int mex(It lo, It hi) {
    int n = hi - lo;
    std::vector<bool> seen(n + 1, false);
    for (It it = lo; it != hi; ++it) {
        if (0 <= *it && *it <= n) {
            seen[*it] = true;
        }
    }
    for (int x = 0; x <= n; x++) {
        if (!seen[x]) {
            return x;
        }
    }
    return n + 1;
}

class dynamic_mex {
    std::vector<int> count;
    std::set<int> missing;

public:
    explicit dynamic_mex(int n) : count(n + 1, 0) {
        for (int x = 0; x <= n; x++) {
            missing.insert(x);
        }
    }

    void add(int x) {
        if (0 <= x && x < (int)count.size() && count[x]++ == 0) {
            missing.erase(x);
        }
    }

    void remove(int x) {
        if (0 <= x && x < (int)count.size() && count[x] > 0 && --count[x] == 0) {
            missing.insert(x);
        }
    }

    int get() const { return *missing.begin(); }
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    int raw[] = {0, 1, 4, 2, 1};
    assert(mex(raw, raw + 5) == 3);

    dynamic_mex m(5);
    m.add(0);
    m.add(1);
    m.add(3);
    assert(m.get() == 2);
    m.add(2);
    assert(m.get() == 4);
    m.remove(1);
    assert(m.get() == 1);
    return 0;
}

```

1.2.9 Two Pointers and Sliding Window

Uses two moving indices to process subarrays in linear time. This technique is most useful when expanding the right endpoint of a window monotonically and shrinking the left endpoint monotonically preserves enough state to test a condition or update an answer.

The examples below cover two common patterns: finding the shortest subarray with sum at least a target when all values are nonnegative, and finding the longest subarray with at most k distinct values.

- `min_length_at_least(a, target)` returns the minimum length of a contiguous subarray with sum at least `target`, or `-1` if none exists. Values in `a` must be nonnegative.
- `longest_at_most_k_distinct(a, k)` returns the maximum length of a contiguous subarray containing at most k distinct values.

Time Complexity: $O(n)$ per call, where n is the array size.

Space Complexity:

- $O(1)$ auxiliary for `min_length_at_least(a, target)`.
- $O(k)$ auxiliary heap space for `longest_at_most_k_distinct(a, k)`.

```
#include <algorithm>
#include <map>
#include <vector>

int min_length_at_least(const std::vector<int> &a, long long target) {
    int best = (int)a.size() + 1;
    long long sum = 0;
    for (int l = 0, r = 0; r < (int)a.size(); r++) {
        sum += a[r];
        while (sum >= target) {
            best = std::min(best, r - l + 1);
            sum -= a[l++];
        }
    }
    return best == (int)a.size() + 1 ? -1 : best;
}

int longest_at_most_k_distinct(const std::vector<int> &a, int k) {
    std::map<int, int> count;
    int best = 0;
    for (int l = 0, r = 0; r < (int)a.size(); r++) {
        count[a[r]]++;
        while ((int)count.size() > k) {
            if (--count[a[l]] == 0) {
                count.erase(a[l]);
            }
            l++;
        }
        best = std::max(best, r - l + 1);
    }
    return best;
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    int raw[] = {2, 3, 1, 2, 4, 3};
    vector<int> a(raw, raw + 6);
    assert(min_length_at_least(a, 7) == 2); // [4, 3].

    int b_raw[] = {1, 2, 1, 3, 4, 3, 5};
```

```
vector<int> b(b_raw, b_raw + 7);
assert(longest_at_most_k_distinct(b, 2) == 3); // [1, 2, 1].
return 0;
}
```

1.3 Greedy and Scheduling

1.3.1 Interval Scheduling

Selects the maximum number of non-overlapping intervals using the classic greedy algorithm that sorts by earliest finish time. This is also known as activity selection.

Intervals are represented as half-open ranges `[start, finish)`, so two intervals are compatible if the next interval's `start` is at least the previous interval's `finish`.

- `interval_scheduling(intervals)` returns one maximum-size compatible subset of intervals, in the order they are selected.
- `interval_t` stores `start`, `finish`, and the original `id`.

Time Complexity: $O(n \log n)$ per call due to sorting, where n is the number of intervals.

Space Complexity: $O(n)$ auxiliary heap space for sorting and the answer.

```
#include <algorithm>
#include <vector>

struct interval_t {
    int start, finish, id;

    interval_t(int start = 0, int finish = 0, int id = 0) : start(start), finish(finish), id(id) {}
};

bool earlier_finish(const interval_t &a, const interval_t &b) {
    return a.finish != b.finish ? a.finish < b.finish : a.start < b.start;
}

std::vector<interval_t> interval_scheduling(std::vector<interval_t> intervals) {
    std::sort(intervals.begin(), intervals.end(), earlier_finish);
    std::vector<interval_t> res;
    int last_finish = 0;
    bool first = true;
    for (int i = 0; i < (int)intervals.size(); i++) {
        if (first || intervals[i].start >= last_finish) {
            res.push_back(intervals[i]);
            last_finish = intervals[i].finish;
            first = false;
        }
    }
    return res;
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    vector<interval_t> intervals;
    intervals.push_back(interval_t(1, 4, 0));
    intervals.push_back(interval_t(3, 5, 1));
}
```

```

intervals.push_back(interval_t(0, 6, 2));
intervals.push_back(interval_t(5, 7, 3));
intervals.push_back(interval_t(8, 9, 4));
vector<interval_t> chosen = interval_scheduling(intervals);
assert(chosen.size() == 3);
assert(chosen[0].id == 0);
assert(chosen[1].id == 3);
assert(chosen[2].id == 4);
return 0;
}

```

1.3.2 Weighted Interval Scheduling

Selects a maximum-weight subset of non-overlapping intervals using dynamic programming and binary search. Unlike unweighted interval scheduling, the earliest finish-time greedy choice is not sufficient when intervals have weights.

Intervals are represented as half-open ranges `[start, finish)`, so two intervals are compatible if the next interval's `start` is at least the previous interval's `finish`.

- `weighted_interval_scheduling(intervals)` returns the maximum total weight of a compatible subset.
- `weighted_interval_t` stores `start`, `finish`, and `weight`.

Time Complexity: $O(n \log n)$ per call due to sorting and binary searching compatible intervals.

Space Complexity: $O(n)$ auxiliary heap space.

```

#include <algorithm>
#include <vector>

struct weighted_interval_t {
    int start, finish;
    long long weight;

    weighted_interval_t(int start = 0, int finish = 0, long long weight = 0)
        : start(start), finish(finish), weight(weight) {}
};

bool weighted_earlier_finish(const weighted_interval_t &a, const weighted_interval_t &b) {
    return a.finish != b.finish ? a.finish < b.finish : a.start < b.start;
}

long long weighted_interval_scheduling(std::vector<weighted_interval_t> intervals) {
    std::sort(intervals.begin(), intervals.end(), weighted_earlier_finish);
    int n = intervals.size();
    std::vector<int> finish(n);
    for (int i = 0; i < n; i++) {
        finish[i] = intervals[i].finish;
    }
    std::vector<long long> dp(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        int j = std::upper_bound(finish.begin(), finish.end(), intervals[i - 1].start) - finish.begin();
        dp[i] = std::max(dp[i - 1], dp[j] + intervals[i - 1].weight);
    }
    return dp[n];
}

```

Example Usage

```

#include <cassert>
using namespace std;

```

```
int main() {
    vector<weighted_interval_t> intervals;
    intervals.push_back(weighted_interval_t(1, 3, 5));
    intervals.push_back(weighted_interval_t(2, 5, 6));
    intervals.push_back(weighted_interval_t(4, 6, 5));
    intervals.push_back(weighted_interval_t(6, 7, 4));
    intervals.push_back(weighted_interval_t(5, 8, 11));
    assert(weighted_interval_scheduling(intervals) == 17);
    return 0;
}
```

1.3.3 Interval Partitioning

Assigns intervals to the minimum number of groups so that no two intervals in the same group overlap. This is the classic interval partitioning problem, also known as finding the minimum number of rooms needed for meetings.

The greedy algorithm sorts intervals by start time and reuses the group whose current finish time is earliest whenever possible. Otherwise, it creates a new group.

- `interval_partitioning(intervals)` returns `room[id]`, the assigned room for each original interval id.
- The number of rooms used is `1 + max(room[id])`, or 0 if there are no intervals.

Time Complexity: $O(n \log n)$ per call due to sorting and priority queue operations.

Space Complexity: $O(n)$ auxiliary heap space.

```
#include <algorithm>
#include <queue>
#include <utility>
#include <vector>

struct partition_interval_t {
    int start, finish, id;

    partition_interval_t(int start = 0, int finish = 0, int id = 0)
        : start(start), finish(finish), id(id) {}
};

bool earlier_start(const partition_interval_t &a, const partition_interval_t &b) {
    return a.start != b.start ? a.start < b.start : a.finish < b.finish;
}

std::vector<int> interval_partitioning(std::vector<partition_interval_t> intervals) {
    std::sort(intervals.begin(), intervals.end(), earlier_start);
    std::vector<int> room(intervals.size());
    std::priority_queue<
        std::pair<int, int>, std::vector<std::pair<int, int>>, std::greater<std::pair<int, int>>>
    > pq;
    int rooms = 0;
    for (int i = 0; i < (int)intervals.size(); i++) {
        int id = intervals[i].id;
        if (!pq.empty() && pq.top().first <= intervals[i].start) {
            int r = pq.top().second;
            pq.pop();
            room[id] = r;
        } else {
            room[id] = rooms++;
        }
        pq.push(std::make_pair(intervals[i].finish, room[id]));
    }
    return room;
}
```

Example Usage

```
#include <algorithm>
#include <cassert>
using namespace std;

int main() {
    vector<partition_interval_t> intervals;
    intervals.push_back(partition_interval_t(0, 30, 0));
    intervals.push_back(partition_interval_t(5, 10, 1));
    intervals.push_back(partition_interval_t(15, 20, 2));
    vector<int> room = interval_partitioning(intervals);
    assert(1 + *max_element(room.begin(), room.end()) == 2);
    assert(room[1] == room[2]);
    assert(room[0] != room[1]);
    return 0;
}
```

1.3.4 Deadline Scheduling

Schedules unit-time jobs with deadlines and profits to maximize total profit. Each job takes one time slot and must be scheduled at or before its deadline. The greedy algorithm processes jobs in decreasing profit order and places each chosen job in the latest available slot before its deadline.

This is a compact scheduling pattern that combines sorting with a disjoint-set forest over available time slots.

- `schedule_deadline_jobs(jobs)` returns the maximum total profit.
- `job_t` stores `deadline` and `profit`. Deadlines are positive integer time slots.

Time Complexity: $O(n \log n)$ per call due to sorting, with near-constant disjoint-set operations.

Space Complexity: $O(n)$ auxiliary heap space.

```
#include <algorithm>
#include <vector>

struct job_t {
    int deadline;
    long long profit;

    job_t(int deadline = 0, long long profit = 0) : deadline(deadline), profit(profit) {}
};

bool higher_profit(const job_t &a, const job_t &b) {
    return a.profit != b.profit ? a.profit > b.profit : a.deadline < b.deadline;
}

class slot_dsu {
    std::vector<int> root;

public:
    explicit slot_dsu(int n) : root(n + 1) {
        for (int i = 0; i <= n; i++) {
            root[i] = i;
        }
    }

    int find_root(int u) {
        if (root[u] != u) {
            root[u] = find_root(root[u]);
        }
        return root[u];
    }
}
```

```

    void occupy(int u) { root[u] = find_root(u - 1); }
};

long long schedule_deadline_jobs(std::vector<job_t> jobs) {
    std::sort(jobs.begin(), jobs.end(), higher_profit);
    int max_deadline = 0;
    for (int i = 0; i < (int)jobs.size(); i++) {
        max_deadline = std::max(max_deadline, jobs[i].deadline);
    }
    slot_dsu slots(max_deadline);
    long long res = 0;
    for (int i = 0; i < (int)jobs.size(); i++) {
        int slot = slots.find_root(jobs[i].deadline);
        if (slot > 0) {
            res += jobs[i].profit;
            slots.occupy(slot);
        }
    }
    return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<job_t> jobs;
    jobs.push_back(job_t(2, 100));
    jobs.push_back(job_t(1, 19));
    jobs.push_back(job_t(2, 27));
    jobs.push_back(job_t(1, 25));
    jobs.push_back(job_t(3, 15));
    assert(schedule_deadline_jobs(jobs) == 142);
    return 0;
}

```

1.3.5 Minimizing Late Jobs (Moore-Hodgson)

Selects a maximum-size subset of jobs that can all be completed by their deadlines. Each job has a processing time and deadline, and all jobs are available at time 0. The Moore-Hodgson algorithm sorts by deadline and keeps the accepted jobs in a max-heap by processing time; whenever the schedule becomes late, it removes the longest accepted job.

- `maximize_on_time_jobs(jobs)` returns the maximum number of jobs that can finish by their deadlines.
- `timed_job_t` stores `duration` and `deadline`.

Time Complexity: $O(n \log n)$ per call due to sorting and heap operations.

Space Complexity: $O(n)$ auxiliary heap space.

```

#include <algorithm>
#include <queue>
#include <vector>

struct timed_job_t {
    int duration, deadline;

    timed_job_t(int duration = 0, int deadline = 0) : duration(duration), deadline(deadline) {}
};

bool earlier_deadline(const timed_job_t &a, const timed_job_t &b) {

```

```

    return a.deadline != b.deadline ? a.deadline < b.deadline : a.duration < b.duration;
}

int maximize_on_time_jobs(std::vector<timed_job_t> jobs) {
    std::sort(jobs.begin(), jobs.end(), earlier_deadline);
    std::priority_queue<int> durations;
    int time = 0;
    for (int i = 0; i < (int)jobs.size(); i++) {
        time += jobs[i].duration;
        durations.push(jobs[i].duration);
        if (time > jobs[i].deadline) {
            time -= durations.top();
            durations.pop();
        }
    }
    return durations.size();
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<timed_job_t> jobs;
    jobs.push_back(timed_job_t(3, 4));
    jobs.push_back(timed_job_t(2, 3));
    jobs.push_back(timed_job_t(1, 2));
    jobs.push_back(timed_job_t(2, 7));
    assert(maximize_on_time_jobs(jobs) == 3);
    return 0;
}

```

1.4 Streaming and Randomized Algorithms

1.4.1 Fisher-Yates Shuffle

Randomly permutes a range in-place using the Fisher-Yates shuffle. Each possible permutation is produced with equal probability, assuming the random number source is uniform.

The algorithm scans from right to left. At position `i`, it chooses a random position `j` from `[0, i]` and swaps the two elements. This fixes one uniformly random remaining element at a time.

- `fisher_yates_shuffle(lo, hi)` randomly shuffles the range `[lo, hi)`.

Time Complexity: $O(n)$ per call, where n is the distance between `lo` and `hi`.

Space Complexity: $O(1)$ auxiliary.

```

#include <algorithm>
#include <cstdlib>

template<class It>
void fisher_yates_shuffle(It lo, It hi) {
    int n = hi - lo;
    for (int i = n - 1; i > 0; i--) {
        int j = rand() % (i + 1);
        std::iter_swap(lo + i, lo + j);
    }
}

```


Example Usage

```
#include <algorithm>
#include <cassert>
#include <vector>
using namespace std;

int main() {
    srand(1);
    int raw[] = {1, 2, 3, 4, 5};
    vector<int> a(raw, raw + 5), original(raw, raw + 5);
    fisher_yates_shuffle(a.begin(), a.end());
    sort(a.begin(), a.end());
    assert(a == original);
    return 0;
}
```

1.4.2 Reservoir Sampling

Selects uniformly random samples from a stream whose length may be unknown in advance. Reservoir sampling keeps only the chosen sample or samples in memory while processing each element once.

- `reservoir_sample_one(lo, hi)` returns one uniformly random element from `[lo, hi)`. The range must be nonempty.
- `reservoir_sample_k(lo, hi, k)` returns `k` uniformly random elements without replacement from `[lo, hi)`, or the whole range if it has fewer than `k` elements.

Time Complexity: $O(n)$ per call, where n is the number of stream elements.

Space Complexity:

- $O(1)$ auxiliary for `reservoir_sample_one(lo, hi)`.
- $O(k)$ auxiliary heap space for `reservoir_sample_k(lo, hi, k)`.

```
#include <algorithm>
#include <cassert>
#include <cstdlib>
#include <iterator>
#include <vector>

template<class It>
typename std::iterator_traits<It>::value_type reservoir_sample_one(It lo, It hi) {
    assert(lo != hi);
    typename std::iterator_traits<It>::value_type sample = *lo;
    int seen = 0;
    for (It it = lo; it != hi; ++it) {
        seen++;
        if (rand() % seen == 0) {
            sample = *it;
        }
    }
    return sample;
}

template<class It>
std::vector<typename std::iterator_traits<It>::value_type> reservoir_sample_k(It lo, It hi, int k) {
    std::vector<typename std::iterator_traits<It>::value_type> res;
    int seen = 0;
    for (It it = lo; it != hi; ++it) {
        seen++;
        if ((int)res.size() < k) {
            res.push_back(*it);
        } else {
```

```

    int j = rand() % seen;
    if (j < k) {
        res[j] = *it;
    }
}
}
return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    srand(1);
    int raw[] = {10, 20, 30, 40, 50};
    vector<int> a(raw, raw + 5);

    int one = reservoir_sample_one(a.begin(), a.end());
    assert(one == 10 || one == 20 || one == 30 || one == 40 || one == 50);

    vector<int> sample = reservoir_sample_k(a.begin(), a.end(), 3);
    assert(sample.size() == 3);
    for (int i = 0; i < (int)sample.size(); i++) {
        bool found = false;
        for (int j = 0; j < (int)a.size(); j++) {
            found = found || sample[i] == a[j];
        }
        assert(found);
    }
    return 0;
}

```

1.4.3 Maximal Subarray Sum (Kadane)

Given an array of numbers, determine the maximum possible sum of any contiguous subarray. Kadane's algorithm scans through the array, at each index computing the maximum nonnegative sum subarray ending there. Either this subarray is empty (in which case its sum is zero), or it consists of one more element than the maximum sequence ending at the previous position. This can be adapted to compute the maximal submatrix sum as well.

- `max_subarray_sum(lo, hi, &res_lo, &res_hi)` returns the maximal subarray sum for the range `[lo, hi)`, where `lo` and `hi` are random-access iterators to numeric types. This implementation requires operators `+` and `<` to be defined on the iterators' value type. Optionally, two `int` pointers may be passed to store the inclusive boundary indices `[res_lo, res_hi]` of the resulting subarray. By convention, the empty subarray is allowed, so an input range consisting of only negative values returns `0` with an empty result interval.
- `max_submatrix_sum(matrix, &r1, &c1, &r2, &c2)` returns the largest sum of any rectangular submatrix for a matrix of n rows by m columns. The matrix should be given as a 2-dimensional vector, where the outer vector must contain n vectors each of size m . This implementation requires operators `+` and `<` to be defined on the iterators' value type. Optionally, four `int` pointers may be passed to store the boundary indices of the resulting subarray, with `(r1, c1)` specifying the top-left index and `(r2, c2)` specifying the bottom-right index. By convention, the empty submatrix is allowed, so an input matrix consisting of only negative values returns `0` with an empty result interval.

Time Complexity:

- $O(n)$ per call to `max_subarray_sum()`, where n is the distance between `lo` and `hi`.
- $O(n \cdot m^2)$ per call to `max_submatrix_sum()`, where n is the number of rows and m is the number of columns in the matrix.

Space Complexity:

- $O(1)$ auxiliary for `max_subarray_sum()`.
- $O(n)$ auxiliary heap space for `max_submatrix_sum()`, where n is the number of rows in the matrix.

```

#include <algorithm>
#include <cstdint>
#include <iterator>
#include <vector>

template<class It>
typename std::iterator_traits<It>::value_type max_subarray_sum(
    It lo, It hi, int *res_lo = NULL, int *res_hi = NULL
) {
    typedef typename std::iterator_traits<It>::value_type T;
    if (lo == hi) {
        return T();
    }
    int curr_begin = 0, begin = 0, end = -1;
    T sum = 0, max_sum = 0;
    for (It it = lo; it != hi; ++it) {
        sum += *it;
        if (sum < 0) {
            sum = 0;
            curr_begin = (it - lo) + 1;
        } else if (max_sum < sum) {
            max_sum = sum;
            begin = curr_begin;
            end = it - lo;
        }
    }
    if (res_lo != NULL && res_hi != NULL) {
        *res_lo = begin;
        *res_hi = end;
    }
    return max_sum;
}

template<class T>
T max_submatrix_sum(
    const std::vector<std::vector<T>> &matrix, int *r1 = NULL, int *c1 = NULL, int *r2 = NULL,
    int *c2 = NULL
) {
    if (matrix.empty() || matrix[0].empty()) {
        return T();
    }
    int n = matrix.size(), m = matrix[0].size();
    std::vector<T> sums(n);
    T sum, max_sum = 0;
    for (int clo = 0; clo < m; clo++) {
        std::fill(sums.begin(), sums.end(), 0);
        for (int chi = clo; chi < m; chi++) {
            for (int i = 0; i < n; i++) {
                sums[i] += matrix[i][chi];
            }
            int rlo, rhi;
            sum = max_subarray_sum(sums.begin(), sums.end(), &rlo, &rhi);
            if (max_sum < sum) {
                max_sum = sum;
                if (r1 != NULL && c1 != NULL && r2 != NULL && c2 != NULL) {
                    *r1 = rlo;
                    *c1 = clo;
                    *r2 = rhi;
                    *c2 = chi;
                }
            }
        }
    }
}

```

```
    return max_sum;
}
```

Example Usage

```
#include <cassert>
#include <iostream>
using namespace std;

int main() {
    {
        int a[] = {-2, -1, -3, 4, -1, 2, 1, -5, 4};
        int lo = 0, hi = 0;
        assert(max_subarray_sum(a, a + 3) == 0);
        assert(max_subarray_sum(a, a + 9, &lo, &hi) == 6);
        cout << "Maximal sum subarray:" << endl;
        for (int i = lo; i <= hi; i++) {
            cout << a[i] << " ";
        }
        cout << endl;
    }
    {
        const int n = 4, m = 5;
        int a[n][m] = {{0, -2, -7, 0, 5},
                       {9, 2, -6, 2, -4},
                       {-4, 1, -4, 1, 0},
                       {-1, 8, 0, -2, 3}};
        vector<vector<int>> matrix(n);
        for (int i = 0; i < n; i++) {
            matrix[i] = vector<int>(a[i], a[i] + m);
        }
        int r1 = 0, c1 = 0, r2 = 0, c2 = 0;
        assert(max_submatrix_sum(matrix, &r1, &c1, &r2, &c2) == 15);
        cout << "\nMaximal sum submatrix:" << endl;
        for (int i = r1; i <= r2; i++) {
            for (int j = c1; j <= c2; j++) {
                cout << matrix[i][j] << " ";
            }
            cout << endl;
        }
    }
    return 0;
}
```

Example Output

```
Maximal sum subarray:
4 -1 2 1

Maximal sum submatrix:
9 2
-4 1
-1 8
```

1.4.4 Majority Element (Boyer-Moore)

Given two ForwardIterators `lo` and `hi` specifying a range `[lo, hi)` of n elements, return an iterator to the first occurrence of the majority element, or the iterator `hi` if there is no majority element. The majority element is defined as an element which occurs strictly more than $\lfloor n/2 \rfloor$ times in the range. This implementation requires `operator ==` to be defined on the iterator's value type.

Time Complexity: $O(n)$ per call to `majority()`, where n is the size of the array.

Space Complexity: $O(1)$ auxiliary.

```
template<class It>
It majority(It lo, It hi) {
    int count = 0;
    It candidate = lo;
    for (It it = lo; it != hi; ++it) {
        if (count == 0) {
            candidate = it;
            count = 1;
        } else if (*it == *candidate) {
            count++;
        } else {
            count--;
        }
    }
    count = 0;
    for (It it = lo; it != hi; ++it) {
        if (*it == *candidate) {
            count++;
        }
    }
    if (count <= (hi - lo) / 2) {
        return hi;
    }
    return candidate;
}
```

Example Usage

```
#include <cassert>

int main() {
    int a[] = {3, 2, 3, 1, 3};
    assert(*majority(a, a + 5) == 3);
    int b[] = {2, 3, 3, 3, 2, 1};
    assert(majority(b, b + 6) == b + 6);
    return 0;
}
```

1.4.5 Online Statistics (Welford)

Maintains the mean and variance of a stream of numbers in one pass using Welford's algorithm. This is more numerically stable than maintaining the sum of values and the sum of squares separately.

- `online_statistics()` constructs an empty summary.
- `add(x)` incorporates one more value `x` into the summary.
- `count()` returns the number of values seen so far.
- `mean()` returns the current arithmetic mean.
- `variance_population()` returns population variance, dividing by n .
- `variance_sample()` returns sample variance, dividing by $n - 1$.

Time Complexity: $O(1)$ per call to `add(x)` and each query.

Space Complexity: $O(1)$ auxiliary.

```
class online_statistics {
    int n;
    double avg, m2;

public:
```

```

online_statistics() : n(0), avg(0), m2(0) {}

void add(double x) {
    n++;
    double delta = x - avg;
    avg += delta / n;
    double delta2 = x - avg;
    m2 += delta * delta2;
}

int count() const { return n; }
double mean() const { return avg; }
double variance_population() const { return n == 0 ? 0 : m2 / n; }
double variance_sample() const { return n <= 1 ? 0 : m2 / (n - 1); }
};

```

Example Usage

```

#include <cassert>
#include <cmath>

bool close(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    online_statistics stats;
    for (int x = 1; x <= 5; x++) {
        stats.add(x);
    }
    assert(stats.count() == 5);
    assert(close(stats.mean(), 3.0));
    assert(close(stats.variance_population(), 2.0));
    assert(close(stats.variance_sample(), 2.5));
    return 0;
}

```

1.4.6 Frequent Elements (Misra-Gries)

Finds candidate frequent elements in a stream using the Misra-Gries algorithm. With parameter `k`, the algorithm keeps at most `k - 1` counters and guarantees that every value occurring more than $\lfloor n/k \rfloor$ times appears among the returned candidates.

The candidates are not automatically verified, since the algorithm intentionally uses sublinear memory and does not retain the stream. If exact frequencies are needed, make a second pass over the input and count only the returned candidates.

- `misra_gries(lo, hi, k)` returns a map of candidate values to their residual counters. Use `k = 2` for the Boyer-Moore majority-candidate special case.

Time Complexity: $O(nk)$ worst-case for this compact map-based implementation, where n is the number of stream values.

Space Complexity: $O(k)$ auxiliary heap space.

```

#include <map>
#include <vector>

template<class T, class It>
std::map<T, int> misra_gries(It lo, It hi, int k) {
    typedef std::map<T, int> counter_map;
    counter_map count;

```

```

for (It it = lo; it != hi; ++it) {
    typename counter_map::iterator found = count.find(*it);
    if (found != count.end()) {
        found->second++;
    } else if ((int)count.size() < k - 1) {
        count[*it] = 1;
    } else {
        std::vector<T> erase;
        for (typename counter_map::iterator jt = count.begin(); jt != count.end(); ++jt) {
            if (--jt->second == 0) {
                erase.push_back(jt->first);
            }
        }
        for (int i = 0; i < (int)erase.size(); i++) {
            count.erase(erase[i]);
        }
    }
}
return count;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    int raw[] = {1, 2, 1, 3, 1, 2, 1, 4, 2, 2, 2};
    map<int, int> candidates = misra_gries<int>(raw, raw + 11, 3);
    assert(candidates.count(1));
    assert(candidates.count(2));
    assert(!candidates.count(3));
    return 0;
}

```

1.5 Cycle Detection

1.5.1 Cycle Detection (Floyd)

Given a function f mapping a set of integers to itself and an x-coordinate in the set, return a pair containing the (position, length) of a cycle in the sequence of numbers obtained from repeatedly composing f with itself starting with the initial x . Formally, since f maps a finite set S to itself, some value is guaranteed to eventually repeat in the sequence: $x_0, x_1 = f(x_0), x_2 = f(x_1), \dots, x_n = f(x_{n-1}), \dots$

There must exist a pair of indices i and j ($i < j$) such that $x_i = x_j$. When this happens, the rest of the sequence will consist of the subsequence from x_i to x_{j-1} repeating indefinitely. The cycle detection problem asks to find such an i , along with the length of the repeating subsequence. A well-known special case is the problem of cycle-detection in a degenerate linked list.

Floyd's cycle-finding algorithm, a.k.a. the "tortoise and the hare algorithm", is a space-efficient algorithm that moves two pointers through the sequence at different speeds. Each step in the algorithm moves the "tortoise" one step forward and the "hare" two steps forward in the sequence, comparing the sequence values at each step. The first value which is simultaneously pointed to by both pointers is the start of the sequence.

Time Complexity: $O(m + n)$ per call to `find_cycle_floyd()`, where m is the smallest index of the sequence which is the beginning of a cycle, and n is the cycle's length.

Space Complexity: $O(1)$ auxiliary.

```

#include <utility>

template<class IntFunction>
std::pair<int, int> find_cycle_floyd(IntFunction f, int x0) {
    int tortoise = f(x0), hare = f(f(x0));
    while (tortoise != hare) {
        tortoise = f(tortoise);
        hare = f(f(hare));
    }
    int start = 0;
    tortoise = x0;
    while (tortoise != hare) {
        tortoise = f(tortoise);
        hare = f(hare);
        start++;
    }
    int length = 1;
    hare = f(tortoise);
    while (tortoise != hare) {
        hare = f(hare);
        length++;
    }
    return std::make_pair(start, length);
}

```

Example Usage

```

#include <cassert>
#include <set>
using namespace std;

int f(int x) {
    return (123 * x * x + 4567890) % 1337;
}

void verify(int x0, int start, int length) {
    set<int> s;
    int x = x0;
    for (int i = 0; i < start; i++) {
        assert(!s.count(x));
        s.insert(x);
        x = f(x);
    }
    int startx = x;
    s.clear();
    for (int i = 0; i < length; i++) {
        assert(!s.count(x));
        s.insert(x);
        x = f(x);
    }
    assert(startx == x);
}

int main() {
    int x0 = 0;
    pair<int, int> res = find_cycle_floyd(f, x0);
    assert(res == make_pair(4, 2));
    verify(x0, res.first, res.second);
    return 0;
}

```


1.5.2 Cycle Detection (Brent)

Given a function f mapping a set of integers to itself and an x -coordinate in the set, return a pair containing the (position, length) of a cycle in the sequence of numbers obtained from repeatedly composing f with itself starting with the initial x . Formally, since f maps a finite set S to itself, some value is guaranteed to eventually repeat in the sequence: $x_0, x_1 = f(x_0), x_2 = f(x_1), \dots, x_n = f(x_{n-1}), \dots$

There must exist a pair of indices i and j ($i < j$) such that $x_i = x_j$. When this happens, the rest of the sequence will consist of the subsequence from x_i to x_{j-1} repeating indefinitely. The cycle detection problem asks to find such an i , along with the length of the repeating subsequence. A well-known special case is the problem of cycle-detection in a degenerate linked list.

While Floyd's cycle-finding algorithm finds cycles by simultaneously moving two pointers at different speeds, Brent's algorithm keeps the tortoise pointer stationary in every iteration, only teleporting it to the hare pointer at every power of two. The smallest power of two at which they meet is the start of the first cycle. This improves upon the constant factor of Floyd's algorithm by reducing the number of calls made to f .

Time Complexity: $O(m + n)$ per call to `find_cycle_brent()`, where m is the smallest index of the sequence which is the beginning of a cycle, and n is the cycle's length.

Space Complexity: $O(1)$ auxiliary.

```
#include <utility>

template<class IntFunction>
std::pair<int, int> find_cycle_brent(IntFunction f, int x0) {
    int power = 1, length = 1, tortoise = x0, hare = f(x0);
    while (tortoise != hare) {
        if (power == length) {
            tortoise = hare;
            power *= 2;
            length = 0;
        }
        hare = f(hare);
        length++;
    }
    hare = x0;
    for (int i = 0; i < length; i++) {
        hare = f(hare);
    }
    int start = 0;
    tortoise = x0;
    while (tortoise != hare) {
        tortoise = f(tortoise);
        hare = f(hare);
        start++;
    }
    return std::make_pair(start, length);
}
```

Example Usage

```
#include <cassert>
#include <set>
using namespace std;

int f(int x) {
    return (123 * x * x + 4567890) % 1337;
}

void verify(int x0, int start, int length) {
    set<int> s;
    int x = x0;
    for (int i = 0; i < start; i++) {
```

```
    assert(!s.count(x));
    s.insert(x);
    x = f(x);
}
int startx = x;
s.clear();
for (int i = 0; i < length; i++) {
    assert(!s.count(x));
    s.insert(x);
    x = f(x);
}
assert(startx == x);
}

int main() {
    int x0 = 0;
    pair<int, int> res = find_cycle_brent(f, x0);
    assert(res == make_pair(4, 2));
    verify(x0, res.first, res.second);
    return 0;
}
```

Chapter 2

Data Structures

2.1 Linked Lists

2.1.1 Singly Linked List Operations

Implements common singly linked list operations using raw nodes. Linked lists are rarely the best contest choice compared with arrays, vectors, and `std::list`, but the pointer patterns are useful for interview-style problems and for understanding constant-time splicing.

For production C++, prefer containers such as `std::list`, `std::forward_list`, or `std::vector` when they fit the problem. Use the raw-node style below when the problem statement already gives nodes, or when manual pointer manipulation is the point of the exercise.

- `reverse_list(head)` reverses a singly linked list and returns the new head.
- `merge_sorted_lists(a, b)` merges two sorted lists using a dummy node and returns the merged head.
- `split_half(head, &second)` cuts a list into two halves, returning the first half and storing the second half in `second`.
- For linked-list cycle detection, use Floyd or Brent cycle detection from Chapter 1, Section 4.

Time Complexity:

- $O(n)$ per call to `reverse_list(head)` and `split_half(head, &second)`.
- $O(n + m)$ per call to `merge_sorted_lists(a, b)`.

Space Complexity: $O(1)$ auxiliary for all operations.

```
#include <cstddef>

struct list_node {
    int value;
    list_node *next;

    explicit list_node(int value = 0) : value(value), next(NULL) {}
};

list_node *reverse_list(list_node *head) {
    list_node *prev = NULL;
    while (head != NULL) {
        list_node *next = head->next;
        head->next = prev;
        prev = head;
        head = next;
    }
    return prev;
}
```

```

}

list_node *merge_sorted_lists(list_node *a, list_node *b) {
    list_node dummy;
    list_node *tail = &dummy;
    while (a != NULL && b != NULL) {
        if (b->value < a->value) {
            tail->next = b;
            b = b->next;
        } else {
            tail->next = a;
            a = a->next;
        }
        tail = tail->next;
    }
    tail->next = a != NULL ? a : b;
    return dummy.next;
}

list_node *split_half(list_node *head, list_node **second) {
    if (head == NULL || head->next == NULL) {
        *second = NULL;
        return head;
    }
    list_node *slow = head, *fast = head->next;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    *second = slow->next;
    slow->next = NULL;
    return head;
}

```

Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

vector<int> values(list_node *head) {
    vector<int> res;
    for (; head != NULL; head = head->next) {
        res.push_back(head->value);
    }
    return res;
}

int main() {
    list_node a[3] = {list_node(1), list_node(3), list_node(5)};
    list_node b[3] = {list_node(2), list_node(4), list_node(6)};
    for (int i = 0; i + 1 < 3; i++) {
        a[i].next = &a[i + 1];
        b[i].next = &b[i + 1];
    }
    list_node *merged = merge_sorted_lists(&a[0], &b[0]);
    vector<int> merged_values = values(merged);
    for (int i = 0; i < 6; i++) {
        assert(merged_values[i] == i + 1);
    }

    list_node *second = NULL;
    list_node *first = split_half(merged, &second);
    assert(values(first).size() == 3);
    assert(values(second).size() == 3);
    assert(values(reverse_list(first))[0] == 3);
}

```

```
    return 0;
}
```

2.1.2 Linked List Splicing

Moves nodes or ranges between singly linked lists in $O(1)$ pointer changes once the neighboring nodes are known. Splicing is the main operation that makes linked lists useful: it moves existing nodes without copying values or invalidating pointers to the moved nodes.

The functions below use a dummy head node for each list. This makes insertions and removals at the beginning of a list match all other positions. In the C++ standard library, `std::list::splice()` provides analogous operations for doubly linked lists, while `std::forward_list::splice_after()` provides analogous operations for singly linked lists.

- `splice_after(pos, before)` moves the node after `before` so it appears after `pos`.
- `splice_range_after(pos, before_first, last)` moves the half-open range `(before_first, last)` so it appears after `pos`.

Time Complexity:

- $O(1)$ per call to `splice_after(pos, before)`.
- $O(k)$ per call to `splice_range_after(pos, before_first, last)`, where k is the number of moved nodes. The actual pointer splicing is $O(1)$, but this compact version walks to the end of the moved range.

Space Complexity: $O(1)$ auxiliary.

```
#include <cstddef>

struct list_node {
    int value;
    list_node *next;

    explicit list_node(int value = 0) : value(value), next(NULL) {}
};

void splice_after(list_node *pos, list_node *before) {
    list_node *node = before->next;
    if (node == NULL || pos == before || pos == node) {
        return;
    }
    before->next = node->next;
    node->next = pos->next;
    pos->next = node;
}

void splice_range_after(list_node *pos, list_node *before_first, list_node *last) {
    list_node *first = before_first->next;
    if (first == last) {
        return;
    }
    list_node *tail = first;
    while (tail->next != last) {
        tail = tail->next;
    }
    before_first->next = last;
    tail->next = pos->next;
    pos->next = first;
}
```

Example Usage

```
#include <cassert>
```

```

#include <vector>
using namespace std;

vector<int> values(list_node *dummy) {
    vector<int> res;
    for (list_node *p = dummy->next; p != NULL; p = p->next) {
        res.push_back(p->value);
    }
    return res;
}

int main() {
    list_node dummy(0), a(1), b(2), c(3), d(4), e(5);
    dummy.next = &a;
    a.next = &b;
    b.next = &c;
    c.next = &d;
    d.next = &e;

    splice_after(&dummy, &c); // Move 4 to the front: 4, 1, 2, 3, 5.
    vector<int> v = values(&dummy);
    int expected1[] = {4, 1, 2, 3, 5};
    for (int i = 0; i < 5; i++) {
        assert(v[i] == expected1[i]);
    }

    splice_range_after(&d, &a, &e); // Move 2, 3 after 4: 4, 2, 3, 1, 5.
    v = values(&dummy);
    int expected2[] = {4, 2, 3, 1, 5};
    for (int i = 0; i < 5; i++) {
        assert(v[i] == expected2[i]);
    }
    return 0;
}

```

2.1.3 LRU Cache

Maintains a least-recently-used cache with fixed capacity. The cache supports lookups and updates by key, evicting the least recently used entry whenever an insertion would exceed capacity.

This implementation combines a doubly linked list (`std::list`) with a map from keys to list iterators. The front of the list stores the most recently used item. The operation `items.splice(items.begin(), items, it)` is the standard-library way to move an existing list node to the front in $O(1)$ time.

- `lru_cache(capacity)` constructs an empty cache holding at most `capacity` keys.
- `get(key, &value)` returns whether `key` is present. If present, it stores the value in `value` and marks the key as most recently used.
- `put(key, value)` inserts or updates `key`, marking it as most recently used and evicting the least recently used key if needed.
- `size()` returns the number of keys currently stored.

Time Complexity: $O(\log n)$ per call to `get(key, &value)` and `put(key, value)` due to `std::map`. In C++11 and later, replacing `std::map` with `std::unordered_map` gives expected $O(1)$ time.

Space Complexity: $O(n)$, where n is the cache capacity.

```

#include <list>
#include <map>
#include <utility>

template<class Key, class Value>
class lru_cache {
    typedef typename std::list<std::pair<Key, Value>>::iterator list_iter;

```

```

int cap;
std::list<std::pair<Key, Value>> items;
std::map<Key, list_iter> where;

public:
explicit lru_cache(int capacity) : cap(capacity) {}

int size() const { return items.size(); }

bool get(const Key &key, Value *value) {
    typename std::map<Key, list_iter>::iterator it = where.find(key);
    if (it == where.end()) {
        return false;
    }
    items.splice(items.begin(), items, it->second);
    it->second = items.begin();
    *value = it->second->second;
    return true;
}

void put(const Key &key, const Value &value) {
    typename std::map<Key, list_iter>::iterator it = where.find(key);
    if (it != where.end()) {
        it->second->second = value;
        items.splice(items.begin(), items, it->second);
        it->second = items.begin();
        return;
    }
    if (cap == 0) {
        return;
    }
    if ((int)items.size() == cap) {
        where.erase(items.back().first);
        items.pop_back();
    }
    items.push_front(std::make_pair(key, value));
    where[key] = items.begin();
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    lru_cache<int, int> cache(2);
    int value = 0;
    cache.put(1, 10);
    cache.put(2, 20);
    assert(cache.get(1, &value) && value == 10);
    cache.put(3, 30); // Evicts key 2.
    assert(!cache.get(2, &value));
    assert(cache.get(3, &value) && value == 30);
    cache.put(1, 11);
    assert(cache.get(1, &value) && value == 11);
    return 0;
}

```

2.2 Heaps and Priority Queues

2.2.1 Binary Heap

Maintain a min-priority queue, that is, a collection of elements with support for querying and extraction of the minimum. This implementation requires an ordering on the set of possible elements defined by `operator <`. A binary min-heap implements a priority queue by inserting and deleting nodes into a binary tree such that the parent of any node is always less than its children.

- `binary_heap()` constructs an empty priority queue.
- `binary_heap(lo, hi)` constructs a priority queue from two ForwardIterators, consisting of elements in the range `[lo, hi)`.
- `size()` returns the size of the priority queue.
- `empty()` returns whether the priority queue is empty.
- `push(v)` inserts the value `v` into the priority queue.
- `pop()` removes the minimum element from the priority queue.
- `top()` returns the minimum element in the priority queue.

Time Complexity:

- $O(1)$ per call to the first constructor, `size()`, `empty()`, and `top()`.
- $O(\log n)$ per call to `push()` and `pop()`, where n is the number of elements in the priority queue.
- $O(n)$ per call to the second constructor on the distance between `lo` and `hi`.

Space Complexity:

- $O(n)$ for storage of the priority queue elements.
- $O(1)$ auxiliary for all operations.

```
#include <algorithm>
#include <stdexcept>
#include <vector>

template<class T>
class binary_heap {
    std::vector<T> heap;

public:
    binary_heap() {}

    template<class It>
    binary_heap(It lo, It hi) {
        while (lo != hi) {
            push(*(lo++));
        }
    }

    int size() const { return heap.size(); }
    bool empty() const { return heap.empty(); }

    void push(const T &v) {
        heap.push_back(v);
        int i = heap.size() - 1;
        while (i > 0) {
            int parent = (i - 1) / 2;
            if (!(heap[i] < heap[parent])) {
                break;
            }
            std::swap(heap[i], heap[parent]);
            i = parent;
        }
    }
}
```



```

void pop() {
    if (heap.empty()) {
        throw std::runtime_error("Cannot pop from empty heap.");
    }
    heap[0] = heap.back();
    heap.pop_back();
    int i = 0;
    for (;;) {
        int child = 2 * i + 1;
        if (child >= (int)heap.size()) {
            break;
        }
        if (child + 1 < (int)heap.size() && heap[child + 1] < heap[child]) {
            child++;
        }
        if (heap[child] < heap[i]) {
            std::swap(heap[i], heap[child]);
            i = child;
        } else {
            break;
        }
    }
}

T top() const {
    if (heap.empty()) {
        throw std::runtime_error("Cannot get top of empty heap.");
    }
    return heap[0];
}
};

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    int a[] = {0, 5, -1, 12};
    binary_heap<int> h(a, a + 4);
    h.push(10);
    while (!h.empty()) {
        cout << h.top() << endl;
        h.pop();
    }
    return 0;
}

```

Example Output

```

-1
0
5
10
12

```

2.2.2 Skew Heap

Maintain a mergeable min-priority queue, that is, a collection of elements with support for querying and extraction of the minimum as well as efficient merging with other instances. This implementation requires an ordering on the set of possible elements defined by `operator <`. A skew heap attempts to maintain balance by unconditionally swapping all nodes in the merge path when merging.

- `skew_heap()` constructs an empty priority queue.
- `skew_heap(lo, hi)` constructs a priority queue from two ForwardIterators, consisting of elements in the range `[lo, hi)`.
- `size()` returns the size of the priority queue.
- `empty()` returns whether the priority queue is empty.
- `push(v)` inserts the value `v` into the priority queue.
- `pop()` removes the minimum element from the priority queue.
- `top()` returns the minimum element in the priority queue.
- `absorb(h)` inserts every value from `h` and sets `h` to the empty priority queue.

Time Complexity:

- $O(1)$ per call to the first constructor, `size()`, `empty()`, and `top()`.
- $O(\log n)$ amortized auxiliary per call to `push()`, `pop()`, and `absorb()`, where n is the number of elements in the priority queue.
- $O(n)$ per call to the second constructor, where n is the distance between `lo` and `hi`.

Space Complexity:

- $O(n)$ for storage of the priority queue elements.
- $O(\log n)$ amortized auxiliary stack space for `push()`, `pop()`, and `absorb()`.
- $O(1)$ auxiliary for all other operations.

```
#include <algorithm>
#include <cstddef>
#include <stdexcept>

template<class T>
class skew_heap {
    struct node_t {
        T value;
        node_t *left, *right;

        node_t(const T &v) : value(v), left(NULL), right(NULL) {}
    } *root;

    int num_nodes;

    static node_t *merge(node_t *a, node_t *b) {
        if (a == NULL) {
            return b;
        }
        if (b == NULL) {
            return a;
        }
        if (b->value < a->value) {
            std::swap(a, b);
        }
        std::swap(a->left, a->right);
        a->left = merge(b, a->left);
        return a;
    }

    static void clean_up(node_t *n) {
        if (n != NULL) {
            clean_up(n->left);
            clean_up(n->right);
            delete n;
        }
    }

public:
    skew_heap() : root(NULL), num_nodes(0) {}

    template<class It>
```

```

skew_heap(It lo, It hi) : root(NULL), num_nodes(0) {
    while (lo != hi) {
        push(*(lo++));
    }
}

~skew_heap() { clean_up(root); }
int size() const { return num_nodes; }
bool empty() const { return root == NULL; }

void push(const T &v) {
    root = merge(root, new node_t(v));
    num_nodes++;
}

void pop() {
    if (empty()) {
        throw std::runtime_error("Cannot pop from empty heap.");
    }
    node_t *tmp = root;
    root = merge(root->left, root->right);
    delete tmp;
    num_nodes--;
}

T top() const {
    if (empty()) {
        throw std::runtime_error("Cannot get top of empty heap.");
    }
    return root->value;
}

void absorb(skew_heap &h) {
    root = merge(root, h.root);
    h.root = NULL;
}
};

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    skew_heap<int> h, h2;
    h.push(12);
    h.push(10);
    h2.push(5);
    h2.push(-1);
    h2.push(0);
    h.absorb(h2);
    while (!h.empty()) {
        cout << h.top() << endl;
        h.pop();
    }
    return 0;
}

```

Example Output

```

-1
0
5
10
12

```

2.2.3 Pairing Heap

Maintain a mergeable min-priority queue, that is, a collection of elements with support for querying and extraction of the minimum as well as efficient merging with other instances. This implementation requires an ordering on the set of possible elements defined by `operator <`. A pairing heap is a heap-ordered multi-way tree, using a two-pass merge to self-adjust during each deletion.

- `pairing_heap()` constructs an empty priority queue.
- `pairing_heap(lo, hi)` constructs a priority queue from two ForwardIterators, consisting of elements in the range `[lo, hi)`.
- `size()` returns the size of the priority queue.
- `empty()` returns whether the priority queue is empty.
- `push(v)` inserts the value `v` into the priority queue.
- `pop()` removes the minimum element from the priority queue.
- `top()` returns the minimum element in the priority queue.
- `absorb(h)` inserts every value from `h` and sets `h` to the empty priority queue.

Time Complexity:

- $O(1)$ per call to the first constructor, `size()`, `empty()`, `top()`, `push()`, and `absorb()`.
- $O(\log n)$ amortized per call to `pop()`.
- $O(n)$ per call to the second constructor on the distance between `lo` and `hi`.

Space Complexity:

- $O(n)$ for storage of the priority queue elements.
- $O(\log n)$ amortized auxiliary stack space for `pop()`.
- $O(1)$ auxiliary for all other operations.

```
#include <cstddef>
#include <stdexcept>

template<class T>
class pairing_heap {
    struct node_t {
        T value;
        node_t *left, *next;

        node_t(const T &v) : value(v), left(NULL), next(NULL) {}

        void add_child(node_t *n) {
            if (left == NULL) {
                left = n;
            } else {
                n->next = left;
                left = n;
            }
        }
    };
    *root;

    int num_nodes;

    static node_t *merge(node_t *a, node_t *b) {
        if (a == NULL) {
            return b;
        }
        if (b == NULL) {
            return a;
        }
        if (a->value < b->value) {
            a->add_child(b);
            return a;
        }
        b->add_child(a);
```

```

    return b;
}

static node_t *merge_pairs(node_t *n) {
    if (n == NULL || n->next == NULL) {
        return n;
    }
    node_t *a = n, *b = n->next, *c = n->next->next;
    a->next = b->next = NULL;
    return merge(merge(a, b), merge_pairs(c));
}

static void clean_up(node_t *n) {
    if (n != NULL) {
        clean_up(n->left);
        clean_up(n->next);
        delete n;
    }
}

public:
pairing_heap() : root(NULL), num_nodes(0) {}

template<class It>
pairing_heap(It lo, It hi) : root(NULL), num_nodes(0) {
    while (lo != hi) {
        push(*(lo++));
    }
}

~pairing_heap() { clean_up(root); }
int size() const { return num_nodes; }
bool empty() const { return root == NULL; }

void push(const T &v) {
    root = merge(root, new node_t(v));
    num_nodes++;
}

void pop() {
    if (empty()) {
        throw std::runtime_error("Cannot pop from empty heap.");
    }
    node_t *tmp = root;
    root = merge_pairs(root->left);
    delete tmp;
    num_nodes--;
}

T top() const {
    if (empty()) {
        throw std::runtime_error("Cannot get top of empty heap.");
    }
    return root->value;
}

void absorb(pairing_heap &h) {
    root = merge(root, h.root);
    h.root = NULL;
}
};

```

Example Usage

```

#include <iostream>
using namespace std;

```

```
int main() {
    pairing_heap<int> h, h2;
    h.push(12);
    h.push(10);
    h2.push(5);
    h2.push(-1);
    h2.push(0);
    h.absorb(h2);
    while (!h.empty()) {
        cout << h.top() << endl;
        h.pop();
    }
    return 0;
}
```

Example Output

```
-1
0
5
10
12
```

2.3 Dictionaries and Ordered Sets

2.3.1 Binary Search Tree

Maintain a map, that is, a collection of key-value pairs such that each possible key appears at most once in the collection. This implementation requires an ordering on the set of possible keys defined by `operator <` on the key type. A binary search tree implements this map by inserting and deleting keys into a binary tree such that every node's left child has a lesser key and every node's right child has a greater key.

- `binary_search_tree()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `NULL` if the key was not found.
- `walk(f)` calls the function `f(k, v)` on each entry of the map, in ascending order of keys.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(n)$ per call to `insert()`, `erase()`, `find()`, and `walk()`, where n is the number of nodes currently in the map.

Space Complexity:

- $O(n)$ for storage of the map elements.
- $O(n)$ auxiliary stack space for `insert()`, `erase()`, and `walk()`.
- $O(1)$ auxiliary for all other operations.

```
#include <cstddef>

template<class K, class V>
class binary_search_tree {
```

```

struct node_t {
    K key;
    V value;
    node_t *left, *right;

    node_t(const K &k, const V &v) : key(k), value(v), left(NULL), right(NULL) {}
} *root;

int num_nodes;

static bool insert(node_t *&n, const K &k, const V &v) {
    if (n == NULL) {
        n = new node_t(k, v);
        return true;
    }
    if (k < n->key) {
        return insert(n->left, k, v);
    } else if (n->key < k) {
        return insert(n->right, k, v);
    }
    return false;
}

static bool erase(node_t *&n, const K &k) {
    if (n == NULL) {
        return false;
    }
    if (k < n->key) {
        return erase(n->left, k);
    } else if (n->key < k) {
        return erase(n->right, k);
    }
    if (n->left != NULL && n->right != NULL) {
        node_t *tmp = n->right, *parent = NULL;
        while (tmp->left != NULL) {
            parent = tmp;
            tmp = tmp->left;
        }
        n->key = tmp->key;
        n->value = tmp->value;
        if (parent != NULL) {
            return erase(parent->left, parent->left->key);
        }
        return erase(n->right, n->right->key);
    }
    node_t *tmp = (n->left != NULL) ? n->left : n->right;
    delete n;
    n = tmp;
    return true;
}

template<class KVFunction>
static void walk(node_t *n, KVFunction f) {
    if (n != NULL) {
        walk(n->left, f);
        f(n->key, n->value);
        walk(n->right, f);
    }
}

static void clean_up(node_t *n) {
    if (n != NULL) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

```

```

public:
    binary_search_tree() : root(NULL), num_nodes(0) {}

    ~binary_search_tree() { clean_up(root); }
    int size() const { return num_nodes; }
    bool empty() const { return root == NULL; }

    bool insert(const K &k, const V &v) {
        if (insert(root, k, v)) {
            num_nodes++;
            return true;
        }
        return false;
    }

    bool erase(const K &k) {
        if (erase(root, k)) {
            num_nodes--;
            return true;
        }
        return false;
    }

    const V *find(const K &k) const {
        node_t *n = root;
        while (n != NULL) {
            if (k < n->key) {
                n = n->left;
            } else if (n->key < k) {
                n = n->right;
            } else {
                return &(n->value);
            }
        }
        return NULL;
    }

    template<class KVFunction>
    void walk(KVFunction f) const {
        walk(root, f);
    }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

void printch(int k, char v) {
    cout << v;
}

int main() {
    binary_search_tree<int, char> t;
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
    assert(t.insert(4, 'd'));
    assert(*t.find(4) == 'd');
    assert(!t.insert(4, 'd'));
    t.walk(printch);
    cout << endl;
    assert(t.erase(1));
    assert(!t.erase(1));
}

```



```

assert(t.find(1) == NULL);
t.walk(printch);
cout << endl;
return 0;
}

```

Example Output

```

abcde
bcde

```

2.3.2 Treap

Maintain a map, that is, a collection of key-value pairs such that each possible key appears at most once in the collection. This implementation requires an ordering on the set of possible keys defined by `operator <` on the key type. A treap is a binary search tree that is balanced by preserving a heap property on the randomly generated priority value assigned to every node, thereby making insertions and deletions run in $O(\log n)$ with high probability.

- `treap()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key was not removed.
- `find(k)` returns a pointer to a const value associated with key `k`, or `NULL` if the key was not found.
- `walk(f)` calls the function `f(k, v)` on each entry of the map, in ascending order of keys.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$ on average per call to `insert()`, `erase()`, and `find()`, where n is the number of entries currently in the map.
- $O(n)$ per call to `walk()`.

Space Complexity:

- $O(n)$ for storage of the map elements.
- $O(\log n)$ auxiliary stack space on average for `insert()`, `erase()`, and `walk()`.
- $O(1)$ auxiliary for all other operations.

```

#include <cstdlib>

template<class K, class V>
class treap {
    struct node_t {
        static inline int rand32() { return (rand() & 0x7fff) | ((rand() & 0x7fff) << 15); }

        K key;
        V value;
        int priority;
        node_t *left, *right;

        node_t(const K &k, const V &v)
            : key(k), value(v), priority(rand32()), left(NULL), right(NULL) {}
    } *root;

    int num_nodes;

```

```

static void rotate_left(node_t *&n) {
    node_t *tmp = n;
    n = n->right;
    tmp->right = n->left;
    n->left = tmp;
}

static void rotate_right(node_t *&n) {
    node_t *tmp = n;
    n = n->left;
    tmp->left = n->right;
    n->right = tmp;
}

static bool insert(node_t *&n, const K &k, const V &v) {
    if (n == NULL) {
        n = new node_t(k, v);
        return true;
    }
    if (k < n->key && insert(n->left, k, v)) {
        if (n->left->priority < n->priority) {
            rotate_right(n);
        }
        return true;
    }
    if (n->key < k && insert(n->right, k, v)) {
        if (n->right->priority < n->priority) {
            rotate_left(n);
        }
        return true;
    }
    return false;
}

static bool erase(node_t *&n, const K &k) {
    if (n == NULL) {
        return false;
    }
    if (k < n->key) {
        return erase(n->left, k);
    } else if (n->key < k) {
        return erase(n->right, k);
    }
    if (n->left != NULL && n->right != NULL) {
        if (n->left->priority < n->right->priority) {
            rotate_right(n);
            return erase(n->right, k);
        }
        rotate_left(n);
        return erase(n->left, k);
    }
    node_t *tmp = (n->left != NULL) ? n->left : n->right;
    delete n;
    n = tmp;
    return true;
}

template<class KVFunction>
static void walk(node_t *n, KVFunction f) {
    if (n != NULL) {
        walk(n->left, f);
        f(n->key, n->value);
        walk(n->right, f);
    }
}

static void clean_up(node_t *n) {
    if (n != NULL) {

```

```

        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    treap() : root(NULL), num_nodes(0) {}

    ~treap() { clean_up(root); }
    int size() const { return num_nodes; }
    bool empty() const { return root == NULL; }

    bool insert(const K &k, const V &v) {
        if (insert(root, k, v)) {
            num_nodes++;
            return true;
        }
        return false;
    }

    bool erase(const K &k) {
        if (erase(root, k)) {
            num_nodes--;
            return true;
        }
        return false;
    }

    const V *find(const K &k) const {
        node_t *n = root;
        while (n != NULL) {
            if (k < n->key) {
                n = n->left;
            } else if (n->key < k) {
                n = n->right;
            } else {
                return &(n->value);
            }
        }
        return NULL;
    }

    template<class KVFunction>
    void walk(KVFunction f) const {
        walk(root, f);
    }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

void printch(int k, char v) {
    cout << v;
}

int main() {
    treap<int, char> t;
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
    assert(t.insert(4, 'd'));
}

```

```

assert(*t.find(4) == 'd');
assert(!t.insert(4, 'd'));
t.walk(printch);
cout << endl;
assert(t.erase(1));
assert(!t.erase(1));
assert(t.find(1) == NULL);
t.walk(printch);
cout << endl;
return 0;
}

```

Example Output

```

abcde
bcde

```

2.3.3 AVL Tree

Maintain a map, that is, a collection of key-value pairs such that each possible key appears at most once in the collection. This implementation requires an ordering on the set of possible keys defined by `operator <` on the key type. An AVL tree is a binary search tree balanced by height, guaranteeing $O(\log n)$ worst-case running time in insertions and deletions by making sure that the heights of the left and right subtrees at every node differ by at most 1.

- `avl_tree()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `NULL` if the key was not found.
- `walk(f)` calls the function `f(k, v)` on each entry of the map, in ascending order of keys.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$ per call to `insert()`, `erase()`, and `find()`, where n is the number of entries currently in the map.
- $O(n)$ per call to `walk()`.

Space Complexity:

- $O(n)$ for storage of the map elements.
- $O(\log n)$ auxiliary stack space for `insert()`, `erase()`, and `walk()`.
- $O(1)$ auxiliary for all other operations.

```

#include <algorithm>
#include <cstdint>

template<class K, class V>
class avl_tree {
    struct node_t {
        K key;
        V value;
        int height;
        node_t *left, *right;

        node_t(const K &k, const V &v) : key(k), value(v), height(1), left(NULL), right(NULL) {}
    } *root;
};

```

```

int num_nodes;

static int height(node_t *n) { return (n != NULL) ? n->height : 0; }

static void update_height(node_t *n) {
    if (n != NULL) {
        n->height = 1 + std::max(height(n->left), height(n->right));
    }
}

static void rotate_left(node_t *&n) {
    node_t *tmp = n;
    n = n->right;
    tmp->right = n->left;
    n->left = tmp;
    update_height(tmp);
    update_height(n);
}

static void rotate_right(node_t *&n) {
    node_t *tmp = n;
    n = n->left;
    tmp->left = n->right;
    n->right = tmp;
    update_height(tmp);
    update_height(n);
}

static int balance_factor(node_t *n) {
    return (n != NULL) ? (height(n->left) - height(n->right)) : 0;
}

static void rebalance(node_t *&n) {
    if (n == NULL) {
        return;
    }
    update_height(n);
    int bf = balance_factor(n);
    if (bf > 1 && balance_factor(n->left) >= 0) {
        rotate_right(n);
    } else if (bf > 1 && balance_factor(n->left) < 0) {
        rotate_left(n->left);
        rotate_right(n);
    } else if (bf < -1 && balance_factor(n->right) <= 0) {
        rotate_left(n);
    } else if (bf < -1 && balance_factor(n->right) > 0) {
        rotate_right(n->right);
        rotate_left(n);
    }
}

static bool insert(node_t *&n, const K &k, const V &v) {
    if (n == NULL) {
        n = new node_t(k, v);
        return true;
    }
    if ((k < n->key && insert(n->left, k, v)) || (n->key < k && insert(n->right, k, v))) {
        rebalance(n);
        return true;
    }
    return false;
}

static bool erase(node_t *&n, const K &k) {
    if (n == NULL) {
        return false;
    }
    if (!(k < n->key || n->key < k)) {

```

```

    if (n->left != NULL && n->right != NULL) {
        node_t *tmp = n->right, *parent = NULL;
        while (tmp->left != NULL) {
            parent = tmp;
            tmp = tmp->left;
        }
        n->key = tmp->key;
        n->value = tmp->value;
        if (parent != NULL) {
            if (!erase(parent->left, parent->left->key)) {
                return false;
            }
        } else if (!erase(n->right, n->right->key)) {
            return false;
        }
    } else {
        node_t *tmp = (n->left != NULL) ? n->left : n->right;
        delete n;
        n = tmp;
    }
    rebalance(n);
    return true;
}
if ((k < n->key && erase(n->left, k)) || (n->key < k && erase(n->right, k))) {
    rebalance(n);
    return true;
}
return false;
}

template<class KVFunction>
static void walk(node_t *n, KVFunction f) {
    if (n != NULL) {
        walk(n->left, f);
        f(n->key, n->value);
        walk(n->right, f);
    }
}

static void clean_up(node_t *n) {
    if (n != NULL) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    avl_tree() : root(NULL), num_nodes(0) {}

    ~avl_tree() { clean_up(root); }
    int size() const { return num_nodes; }
    bool empty() const { return root == NULL; }

    bool insert(const K &k, const V &v) {
        if (insert(root, k, v)) {
            num_nodes++;
            return true;
        }
        return false;
    }

    bool erase(const K &k) {
        if (erase(root, k)) {
            num_nodes--;
            return true;
        }
        return false;
    }

```

```

}

const V *find(const K &k) const {
    node_t *n = root;
    while (n != NULL) {
        if (k < n->key) {
            n = n->left;
        } else if (n->key < k) {
            n = n->right;
        } else {
            return &(n->value);
        }
    }
    return NULL;
}

template<class KVFunction>
void walk(KVFunction f) const {
    walk(root, f);
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

void printch(int k, char v) {
    cout << v;
}

int main() {
    avl_tree<int, char> t;
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
    assert(t.insert(4, 'd'));
    assert(*t.find(4) == 'd');
    assert(!t.insert(4, 'd'));
    t.walk(printch);
    cout << endl;
    assert(t.erase(1));
    assert(!t.erase(1));
    assert(t.find(1) == NULL);
    t.walk(printch);
    cout << endl;
    return 0;
}

```

Example Output

```

abcde
bcde

```

2.3.4 Red-Black Tree

Maintain a map, that is, a collection of key-value pairs such that each possible key appears at most once in the collection. This implementation requires an ordering on the set of possible keys defined by `operator <` on the key type. A red black tree is a binary search tree balanced by coloring its nodes red or black, then constraining node colors on any simple path from the root to a leaf.

- `red_black_tree()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `NULL` if the key was not found.
- `walk(f)` calls the function `f(k, v)` on each entry of the map, in ascending order of keys.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$ per call to `insert()`, `erase()`, and `find()`, where n is the number of entries currently in the map.
- $O(n)$ per call to `walk()`.

Space Complexity:

- $O(n)$ for storage of the map elements.
- $O(\log n)$ auxiliary stack space for `walk()`.
- $O(1)$ auxiliary for all other operations.

```
#include <algorithm>
#include <cstddef>

template<class K, class V>
class red_black_tree {
    enum color_t { RED, BLACK };
    struct node_t {
        K key;
        V value;
        color_t color;
        node_t *left, *right, *parent;

        node_t(const K &k, const V &v, color_t c)
            : key(k), value(v), color(c), left(NULL), right(NULL), parent(NULL) {}
    } *root, *LEAF_NIL;

    int num_nodes;

    void rotate_left(node_t *n) {
        node_t *tmp = n->right;
        if ((n->right = tmp->left) != LEAF_NIL) {
            n->right->parent = n;
        }
        if ((tmp->parent = n->parent) == LEAF_NIL) {
            root = tmp;
        } else if (n->parent->left == n) {
            n->parent->left = tmp;
        } else {
            n->parent->right = tmp;
        }
        tmp->left = n;
        n->parent = tmp;
    }

    void rotate_right(node_t *n) {
        node_t *tmp = n->left;
        if ((n->left = tmp->right) != LEAF_NIL) {
            n->left->parent = n;
        }
        if ((tmp->parent = n->parent) == LEAF_NIL) {
            root = tmp;
        } else if (n->parent->right == n) {
            n->parent->right = tmp;
        } else {
            n->parent->left = tmp;
        }
        tmp->right = n;
        n->parent = tmp;
    }
};
```



```

    n->parent->right = tmp;
} else {
    n->parent->left = tmp;
}
tmp->right = n;
n->parent = tmp;
}

void insert_fix(node_t *n) {
    while (n->parent->color == RED) {
        node_t *parent = n->parent;
        node_t *grandparent = n->parent->parent;
        if (parent == grandparent->left) {
            node_t *uncle = grandparent->right;
            if (uncle->color == RED) {
                grandparent->color = RED;
                parent->color = BLACK;
                uncle->color = BLACK;
                n = grandparent;
            } else {
                if (n == parent->right) {
                    rotate_left(parent);
                    n = parent;
                    parent = n->parent;
                }
                rotate_right(grandparent);
                std::swap(parent->color, grandparent->color);
                n = parent;
            }
        } else if (parent == grandparent->right) {
            node_t *uncle = grandparent->left;
            if (uncle->color == RED) {
                grandparent->color = RED;
                parent->color = BLACK;
                uncle->color = BLACK;
                n = grandparent;
            } else {
                if (n == parent->left) {
                    rotate_right(parent);
                    n = parent;
                    parent = n->parent;
                }
                rotate_left(grandparent);
                std::swap(parent->color, grandparent->color);
                n = parent;
            }
        }
    }
    root->color = BLACK;
}

void replace(node_t *n, node_t *replacement) {
    if (n->parent == LEAF_NIL) {
        root = replacement;
    } else if (n == n->parent->left) {
        n->parent->left = replacement;
    } else {
        n->parent->right = replacement;
    }
    replacement->parent = n->parent;
}

void erase_fix(node_t *n) {
    while (n != root && n->color == BLACK) {
        node_t *parent = n->parent;
        if (n == parent->left) {
            node_t *sibling = parent->right;
            if (sibling->color == RED) {

```

```

        sibling->color = BLACK;
        parent->color = RED;
        rotate_left(parent);
        sibling = parent->right;
    }
    if (sibling->left->color == BLACK && sibling->right->color == BLACK) {
        sibling->color = RED;
        n = parent;
    } else {
        if (sibling->right->color == BLACK) {
            sibling->left->color = BLACK;
            sibling->color = RED;
            rotate_right(sibling);
            sibling = parent->right;
        }
        sibling->color = parent->color;
        parent->color = BLACK;
        sibling->right->color = BLACK;
        rotate_left(parent);
        n = root;
    }
} else {
    node_t *sibling = parent->left;
    if (sibling->color == RED) {
        sibling->color = BLACK;
        parent->color = RED;
        rotate_right(parent);
        sibling = parent->left;
    }
    if (sibling->left->color == BLACK && sibling->right->color == BLACK) {
        sibling->color = RED;
        n = parent;
    } else {
        if (sibling->left->color == BLACK) {
            sibling->right->color = BLACK;
            sibling->color = RED;
            rotate_left(sibling);
            sibling = parent->left;
        }
        sibling->color = parent->color;
        parent->color = BLACK;
        sibling->left->color = BLACK;
        rotate_right(parent);
        n = root;
    }
}
}
n->color = BLACK;
}

template<class KVFunction>
void walk(node_t *n, KVFunction f) const {
    if (n != LEAF_NIL) {
        walk(n->left, f);
        f(n->key, n->value);
        walk(n->right, f);
    }
}

void clean_up(node_t *n) {
    if (n != LEAF_NIL) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:

```

```

red_black_tree() : num_nodes(0) { root = LEAF_NIL = new node_t(K(), V(), BLACK); }

~red_black_tree() {
    clean_up(root);
    delete LEAF_NIL;
}

int size() const { return num_nodes; }
bool empty() const { return num_nodes == 0; }

bool insert(const K &k, const V &v) {
    node_t *curr = root, *prev = LEAF_NIL;
    while (curr != LEAF_NIL) {
        prev = curr;
        if (k < curr->key) {
            curr = curr->left;
        } else if (curr->key < k) {
            curr = curr->right;
        } else {
            return false;
        }
    }
    node_t *n = new node_t(k, v, RED);
    n->parent = prev;
    if (prev == LEAF_NIL) {
        root = n;
    } else if (k < prev->key) {
        prev->left = n;
    } else {
        prev->right = n;
    }
    n->left = n->right = LEAF_NIL;
    insert_fix(n);
    num_nodes++;
    return true;
}

bool erase(const K &k) {
    node_t *n = root;
    while (n != LEAF_NIL) {
        if (k < n->key) {
            n = n->left;
        } else if (n->key < k) {
            n = n->right;
        } else {
            break;
        }
    }
    if (n == LEAF_NIL) {
        return false;
    }
    color_t color = n->color;
    node_t *replacement;
    if (n->left == LEAF_NIL) {
        replacement = n->right;
        replace(n, n->right);
    } else if (n->right == LEAF_NIL) {
        replacement = n->left;
        replace(n, n->left);
    } else {
        node_t *tmp = n->right;
        while (tmp->left != LEAF_NIL) {
            tmp = tmp->left;
        }
        color = tmp->color;
        replacement = tmp->right;
        if (tmp->parent == n) {
            replacement->parent = tmp;

```

```

    } else {
        replace(tmp, tmp->right);
        tmp->right = n->right;
        tmp->right->parent = tmp;
    }
    replace(n, tmp);
    tmp->left = n->left;
    tmp->left->parent = tmp;
    tmp->color = n->color;
}
delete n;
if (color == BLACK) {
    erase_fix(replacement);
}
return true;
}

const V *find(const K &k) const {
    node_t *n = root;
    while (n != LEAF_NIL) {
        if (k < n->key) {
            n = n->left;
        } else if (n->key < k) {
            n = n->right;
        } else {
            return &(n->value);
        }
    }
    return NULL;
}

template<class KVFunction>
void walk(KVFunction f) const {
    walk(root, f);
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

void printch(int k, char v) {
    cout << v;
}

int main() {
    red_black_tree<int, char> t;
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
    assert(t.insert(4, 'd'));
    assert(*t.find(4) == 'd');
    assert(!t.insert(4, 'd'));
    t.walk(printch);
    cout << endl;
    assert(t.erase(1));
    assert(!t.erase(1));
    assert(t.find(1) == NULL);
    t.walk(printch);
    cout << endl;
    return 0;
}

```

Example Output

```
abcde
bcde
```

2.3.5 Splay Tree

Maintain a map, that is, a collection of key-value pairs such that each possible key appears at most once in the collection. This implementation requires an ordering on the set of possible keys defined by `operator <` on the key type. A splay tree is a balanced binary search tree with the additional property that recently accessed elements are quick to access again.

- `splay_tree()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `NULL` if the key was not found.
- `walk(f)` calls the function `f(k, v)` on each entry of the map, in ascending order of keys.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$ per call to `insert()`, `erase()`, and `find()`, where n is the number of entries currently in the map.
- $O(n)$ per call to `walk()`.

Space Complexity:

- $O(n)$ for storage of the map elements.
- $O(\log n)$ auxiliary stack space for `insert()`, `erase()`, and `walk()`.
- $O(1)$ auxiliary for all other operations.

```
#include <cstddef>

template<class K, class V>
class splay_tree {
    struct node_t {
        K key;
        V value;
        node_t *left, *right;

        node_t(const K &k, const V &v) : key(k), value(v), left(NULL), right(NULL) {}
    } *root;

    int num_nodes;

    static void rotate_left(node_t *&n) {
        node_t *tmp = n;
        n = n->right;
        tmp->right = n->left;
        n->left = tmp;
    }

    static void rotate_right(node_t *&n) {
        node_t *tmp = n;
        n = n->left;
        tmp->left = n->right;
        n->right = tmp;
    }
}
```

```

static void splay(node_t *&n, const K &k) {
    if (n == NULL) {
        return;
    }
    if (k < n->key && n->left != NULL) {
        if (k < n->left->key) {
            splay(n->left->left, k);
            rotate_right(n);
        } else if (n->left->key < k) {
            splay(n->left->right, k);
            if (n->left->right != NULL) {
                rotate_left(n->left);
            }
        }
        if (n->left != NULL) {
            rotate_right(n);
        }
    } else if (n->key < k && n->right != NULL) {
        if (k < n->right->key) {
            splay(n->right->left, k);
            if (n->right->left != NULL) {
                rotate_right(n->right);
            }
        } else if (n->right->key < k) {
            splay(n->right->right, k);
            rotate_left(n);
        }
        if (n->right != NULL) {
            rotate_left(n);
        }
    }
}

static bool insert(node_t *&n, const K &k, const V &v) {
    if (n == NULL) {
        n = new node_t(k, v);
        return true;
    }
    splay(n, k);
    if (k < n->key) {
        node_t *tmp = new node_t(k, v);
        tmp->left = n->left;
        tmp->right = n;
        n->left = NULL;
        n = tmp;
    } else if (n->key < k) {
        node_t *tmp = new node_t(k, v);
        tmp->left = n;
        tmp->right = n->right;
        n->right = NULL;
        n = tmp;
    } else {
        return false;
    }
    return true;
}

static bool erase(node_t *&n, const K &k) {
    if (n == NULL) {
        return false;
    }
    splay(n, k);
    if (k < n->key || n->key < k) {
        return false;
    }
    node_t *tmp = n;
    if (n->left == NULL) {
        n = n->right;
    }

```

```

    } else {
        splay(n->left, k);
        n = n->left;
        n->right = tmp->right;
    }
    delete tmp;
    return true;
}

template<class KVFunction>
static void walk(node_t *n, KVFunction f) {
    if (n != NULL) {
        walk(n->left, f);
        f(n->key, n->value);
        walk(n->right, f);
    }
}

static void clean_up(node_t *n) {
    if (n != NULL) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    splay_tree() : root(NULL), num_nodes(0) {}

    ~splay_tree() { clean_up(root); }
    int size() const { return num_nodes; }
    bool empty() const { return root == NULL; }

    bool insert(const K &k, const V &v) {
        if (insert(root, k, v)) {
            num_nodes++;
            return true;
        }
        return false;
    }

    bool erase(const K &k) {
        if (erase(root, k)) {
            num_nodes--;
            return true;
        }
        return false;
    }

    const V *find(const K &k) {
        splay(root, k);
        return (k < root->key || root->key < k) ? NULL : &(root->value);
    }

    template<class KVFunction>
    void walk(KVFunction f) const {
        walk(root, f);
    }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

```

```

void printch(int k, char v) {
    cout << v;
}

int main() {
    splay_tree<int, char> t;
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
    assert(t.insert(4, 'd'));
    assert(*t.find(4) == 'd');
    assert(!t.insert(4, 'd'));
    t.walk(printch);
    cout << endl;
    assert(t.erase(1));
    assert(!t.erase(1));
    assert(t.find(1) == NULL);
    t.walk(printch);
    cout << endl;
    return 0;
}

```

Example Output

```

abcde
bcde

```

2.3.6 Size Balanced Tree

Maintain a map, that is, a collection of key-value pairs such that each possible key appears at most once in the collection. In addition, support queries for keys given their ranks as well as queries for the ranks of given keys. This implementation requires an ordering on the set of possible keys defined by `operator <` on the key type. A size balanced tree augments each nodes with the size of its subtree, using it to maintain balance and compute order statistics.

- `size_balanced_tree()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `NULL` if the key was not found.
- `select(r)` returns a key-value pair of the node with a key of zero-based rank `r` in the map, throwing an exception if the rank is not between 0 and `size() - 1`.
- `rank(k)` returns the zero-based rank of key `k` in the map, throwing an exception if the key was not found in the map.
- `walk(f)` calls the function `f(k, v)` on each entry of the map, in ascending order of keys.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$ per call to `insert()`, `erase()`, `find()`, `select()`, and `rank()`, where n is the number of entries currently in the map.
- $O(n)$ per call to `walk()`.

Space Complexity:

- $O(n)$ for storage of the map elements.

- $O(\log n)$ auxiliary stack space for `insert()`, `erase()`, and `walk()`.
- $O(1)$ auxiliary for all other operations.

```

#include <cstddef>
#include <stdexcept>
#include <utility>

template<class K, class V>
class size_balanced_tree {
    struct node_t {
        K key;
        V value;
        int size;
        node_t *left, *right;

        node_t(const K &k, const V &v) : key(k), value(v), size(1), left(NULL), right(NULL) {}

        inline node_t *&child(int c) { return (c == 0) ? left : right; }

        void update() {
            size = 1;
            if (left != NULL) {
                size += left->size;
            }
            if (right != NULL) {
                size += right->size;
            }
        }
    } *root;

    static inline int size(node_t *n) { return (n == NULL) ? 0 : n->size; }

    static void rotate(node_t *&n, int c) {
        node_t *tmp = n->child(c);
        n->child(c) = tmp->child(!c);
        tmp->child(!c) = n;
        n->update();
        tmp->update();
        n = tmp;
    }

    static void maintain(node_t *&n, int c) {
        if (n == NULL || n->child(c) == NULL) {
            return;
        }
        node_t *tmp = n->child(c);
        if (size(tmp->child(c)) > size(n->child(!c))) {
            rotate(n, c);
        } else if (size(tmp->child(!c)) > size(n->child(!c))) {
            rotate(tmp, !c);
            rotate(n, c);
        } else {
            return;
        }
        maintain(n->left, 0);
        maintain(n->right, 1);
        maintain(n, 0);
        maintain(n, 1);
    }

    static bool insert(node_t *&n, const K &k, const V &v) {
        if (n == NULL) {
            n = new node_t(k, v);
            return true;
        }
        bool result;
        if (k < n->key) {

```

```

    result = insert(n->left, k, v);
    maintain(n, 0);
} else if (n->key < k) {
    result = insert(n->right, k, v);
    maintain(n, 1);
} else {
    return false;
}
n->update();
return result;
}

static bool erase(node_t *n, const K &k) {
    if (n == NULL) {
        return false;
    }
    bool result;
    int c = (k < n->key);
    if (k < n->key) {
        result = erase(n->left, k);
    } else if (n->key < k) {
        result = erase(n->right, k);
    } else {
        if (n->right == NULL || n->left == NULL) {
            node_t *tmp = n;
            n = (n->right == NULL) ? n->left : n->right;
            delete tmp;
            return true;
        }
        node_t *p = n->right;
        while (p->left != NULL) {
            p = p->left;
        }
        n->key = p->key;
        result = erase(n->right, p->key);
    }
    maintain(n, c);
    n->update();
    return result;
}

static std::pair<K, V> select(node_t *n, int r) {
    int rank = size(n->left);
    if (r < rank) {
        return select(n->left, r);
    } else if (r > rank) {
        return select(n->right, r - rank - 1);
    }
    return std::make_pair(n->key, n->value);
}

static int rank(node_t *n, const K &k) {
    if (n == NULL) {
        throw std::runtime_error("Cannot rank key that's not in tree.");
    }
    int r = size(n->left);
    if (k < n->key) {
        return rank(n->left, k);
    } else if (n->key < k) {
        return rank(n->right, k) + r + 1;
    }
    return r;
}

template<class KVFunction>
static void walk(node_t *n, KVFunction f) {
    if (n != NULL) {
        walk(n->left, f);

```

```

        f(n->key, n->value);
        walk(n->right, f);
    }
}

static void clean_up(node_t *n) {
    if (n != NULL) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    size_balanced_tree() : root(NULL) {}

    ~size_balanced_tree() { clean_up(root); }
    int size() const { return size(root); }
    bool empty() const { return root == NULL; }
    bool insert(const K &k, const V &v) { return insert(root, k, v); }
    bool erase(const K &k) { return erase(root, k); }
    int rank(const K &k) const { return rank(root, k); }

    const V *find(const K &k) const {
        node_t *n = root;
        while (n != NULL) {
            if (k < n->key) {
                n = n->left;
            } else if (n->key < k) {
                n = n->right;
            } else {
                return &(n->value);
            }
        }
        return NULL;
    }

    std::pair<K, V> select(int r) const {
        if (r < 0 || r >= size(root)) {
            throw std::runtime_error("Select rank must be between 0 and size() - 1.");
        }
        return select(root, r);
    }

    template<class KVFunction>
    void walk(KVFunction f) const {
        walk(root, f);
    }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

void printch(int k, char v) {
    cout << v;
}

int main() {
    size_balanced_tree<int, char> t;
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
}

```

```

assert(t.insert(4, 'd'));
assert(*t.find(4) == 'd');
assert(!t.insert(4, 'd'));
t.walk(printch);
cout << endl;
assert(t.erase(1));
assert(!t.erase(1));
assert(t.find(1) == NULL);
t.walk(printch);
cout << endl;
assert(t.rank(2) == 0);
assert(t.rank(3) == 1);
assert(t.rank(5) == 3);
assert(t.select(0).first == 2);
assert(t.select(1).first == 3);
assert(t.select(2).first == 4);
return 0;
}

```

Example Output

```

abcde
bcde

```

2.3.7 Interval Treap

Maintain a map from closed, one-dimensional intervals to values while supporting efficient reporting of any or all entries that intersect with a given query interval. This implementation uses `std::pair` to represent intervals, requiring operators `<` and `==` to be defined on the numeric key type. A treap is used to process the entries, where keys are compared lexicographically as pairs.

- `interval_treap()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(lo, hi, v)` adds an entry with key `[lo, hi]` and value `v` to the map, returning `true` if a new interval was added or `false` if the interval already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(lo, hi)` removes the entry with key `[lo, hi]` from the map, returning `true` if the removal was successful or `false` if the interval was not found.
- `find_key(lo, hi)` returns a pointer to a const `std::pair` representing the key of some interval in the map which intersects with `[lo, hi]`, or `NULL` if no such entry was found.
- `find_value(lo, hi)` returns a pointer to a const value of some entry in the map with a key that intersects with `[lo, hi]`, or `NULL` if no such entry was found.
- `find_all(lo, hi, f)` calls the function `f(lo, hi, v)` on each entry in the map that overlaps with `[lo, hi]`, in lexicographically ascending order of intervals.
- `walk(f)` calls the function `f(lo, hi, v)` on each interval in the map, in lexicographically ascending order of intervals.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$ on average per call to `insert()`, `erase()`, and `find_any()`, where n is the number of intervals currently in the set.
- $O(\log n + m)$ on average per call to `find_all()`, where m is the number of intersecting intervals that are reported.
- $O(n)$ per call to `walk()`.

Space Complexity:

- $O(n)$ for storage of the map elements.

- $O(1)$ auxiliary for `size()` and `empty()`.
- $O(\log n)$ auxiliary stack space on average for all other operations.

```
#include <cstdlib>
#include <utility>

template<class K, class V>
class interval_treap {
    typedef std::pair<K, K> interval_t;

    struct node_t {
        static inline int rand32() { return (rand() & 0x7fff) | ((rand() & 0x7fff) << 15); }

        interval_t interval;
        V value;
        K max;
        int priority;
        node_t *left, *right;

        node_t(const interval_t &i, const V &v)
            : interval(i), value(v), max(i.second), priority(rand32()), left(NULL), right(NULL) {}

        void update() {
            max = interval.second;
            if (left != NULL && left->max > max) {
                max = left->max;
            }
            if (right != NULL && right->max > max) {
                max = right->max;
            }
        }
    } *root;

    int num_nodes;

    static void rotate_left(node_t *&n) {
        node_t *tmp = n;
        n = n->right;
        tmp->right = n->left;
        n->left = tmp;
        tmp->update();
    }

    static void rotate_right(node_t *&n) {
        node_t *tmp = n;
        n = n->left;
        tmp->left = n->right;
        n->right = tmp;
        tmp->update();
    }

    static bool insert(node_t *&n, const interval_t &i, const V &v) {
        if (n == NULL) {
            n = new node_t(i, v);
            return true;
        }
        if (i < n->interval && insert(n->left, i, v)) {
            if (n->left->priority < n->priority) {
                rotate_right(n);
            }
            n->update();
            return true;
        }
        if (i > n->interval && insert(n->right, i, v)) {
            if (n->right->priority < n->priority) {
                rotate_left(n);
            }
        }
    }
}
```

```

    n->update();
    return true;
}
return false;
}

static bool erase(node_t *&n, const interval_t &i) {
    if (n == NULL) {
        return false;
    }
    if (i < n->interval) {
        return erase(n->left, i);
    }
    if (i > n->interval) {
        return erase(n->right, i);
    }
    if (n->left != NULL && n->right != NULL) {
        bool res;
        if (n->left->priority < n->right->priority) {
            rotate_right(n);
            res = erase(n->right, i);
        } else {
            rotate_left(n);
            res = erase(n->left, i);
        }
        n->update();
        return res;
    }
    node_t *tmp = (n->left != NULL) ? n->left : n->right;
    delete n;
    n = tmp;
    return true;
}

static node_t *find_any(node_t *n, const interval_t &i) {
    if (n == NULL) {
        return NULL;
    }
    if (n->interval.first <= i.second && i.first <= n->interval.second) {
        return n;
    }
    if (n->left != NULL && i.first <= n->left->max) {
        return find_any(n->left, i);
    }
    return find_any(n->right, i);
}

template<class KVFunction>
static void find_all(node_t *n, const interval_t &i, KVFunction f) {
    if (n == NULL || n->max < i.first) {
        return;
    }
    if (n->interval.first <= i.second && i.first <= n->interval.second) {
        f(n->interval.first, n->interval.second, n->value);
    }
    find_all(n->left, i, f);
    find_all(n->right, i, f);
}

template<class KVFunction>
static void walk(node_t *n, KVFunction f) {
    if (n != NULL) {
        walk(n->left, f);
        f(n->interval.first, n->interval.second, n->value);
        walk(n->right, f);
    }
}

```

```

static void clean_up(node_t *n) {
    if (n != NULL) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    interval_treap() : root(NULL), num_nodes(0) {}

    ~interval_treap() { clean_up(root); }
    int size() const { return num_nodes; }
    bool empty() const { return root == NULL; }

    bool insert(const K &lo, const K &hi, const V &v) {
        if (insert(root, std::make_pair(lo, hi), v)) {
            num_nodes++;
            return true;
        }
        return false;
    }

    bool erase(const K &lo, const K &hi) {
        if (erase(root, std::make_pair(lo, hi))) {
            num_nodes--;
            return true;
        }
        return false;
    }

    const interval_t *find_key(const K &lo, const K &hi) const {
        node_t *n = find_any(root, std::make_pair(lo, hi));
        return (n == NULL) ? NULL : &(n->interval);
    }

    const V *find_value(const K &lo, const K &hi) const {
        node_t *n = find_any(root, std::make_pair(lo, hi));
        return (n == NULL) ? NULL : &(n->value);
    }

    template<class KVFunction>
    void find_all(const K &lo, const K &hi, KVFunction f) const {
        find_all(root, std::make_pair(lo, hi), f);
    }

    template<class KVFunction>
    void walk(KVFunction f) const {
        walk(root, f);
    }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

void print(int lo, int hi, char v) {
    cout << " [" << lo << ", " << hi << "]" << v;
}

int main() {
    interval_treap<int, char> t;
    t.insert(15, 20, 'a');
    t.insert(10, 30, 'b');
}

```

```

t.insert(17, 19, 'c');
t.insert(5, 20, 'd');
t.insert(12, 15, 'e');
t.insert(10, 40, 'f');
assert(t.size() == 6);
assert(!t.insert(5, 20, 'x'));
t.erase(17, 19);
assert(t.size() == 5);
assert(*t.find_key(3, 9) == make_pair(5, 20));
assert(*t.find_value(3, 9) == 'd');
cout << "Intervals intersecting [16, 20]:";
t.find_all(16, 20, print);
cout << "\nAll intervals:";
t.walk(print);
cout << endl;
return 0;
}

```

Example Output

```

Intervals intersecting [16, 20]: [15, 20] [10, 30] [5, 20] [10, 40]
All intervals: [5, 20] [10, 30] [10, 40] [12, 15] [15, 20]

```

2.3.8 Hash Map

Maintain a map, that is, a collection of key-value pairs such that each possible key appears at most once in the collection. This implementation requires `operator ==` to be defined on the key type. A hash map implements a map by hashing keys into buckets using a hash function. This implementation resolves collisions by chaining entries hashed to the same bucket into a linked list.

- `hash_map()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `NULL` if the key was not found.
- `operator[]` returns a reference to key `k`'s associated value (which may be modified), or if necessary, inserts and returns a new entry with the default constructed value if key `k` was not originally found.
- `walk(f)` calls the function `f(k, v)` on each entry of the map, in no guaranteed order.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(1)$ amortized per call to `insert()`, `erase()`, `find()`, and `operator[]`.
- $O(n)$ per call to `walk()`, where n is the number of entries in the map.

Space Complexity:

- $O(n)$ for storage of the map elements.
- $O(n)$ auxiliary heap space for `insert()`.
- $O(1)$ auxiliary for all other operations.

```

#include <cstdlib>
#include <list>

template<class K, class V, class Hash>
class hash_map {
    struct entry_t {

```



```

    K key;
    V value;

    entry_t(const K &k, const V &v) : key(k), value(v) {}
};

std::list<entry_t> *table;
int table_size, num_entries;

void double_capacity_and_rehash() {
    std::list<entry_t> *old = table;
    int old_size = table_size;
    table_size = 2 * table_size;
    table = new std::list<entry_t>[table_size];
    num_entries = 0;
    typename std::list<entry_t>::iterator it;
    for (int i = 0; i < old_size; i++) {
        for (it = old[i].begin(); it != old[i].end(); ++it) {
            insert(it->key, it->value);
        }
    }
    delete[] old;
}

public:
hash_map(int size = 128) : table_size(size), num_entries(0) {
    table = new std::list<entry_t>[table_size];
}

~hash_map() { delete[] table; }
int size() const { return num_entries; }
bool empty() const { return num_entries == 0; }

bool insert(const K &k, const V &v) {
    if (find(k) != NULL) {
        return false;
    }
    if (num_entries >= table_size) {
        double_capacity_and_rehash();
    }
    unsigned int i = Hash()(k) % table_size;
    table[i].push_back(entry_t(k, v));
    num_entries++;
    return true;
}

bool erase(const K &k) {
    unsigned int i = Hash()(k) % table_size;
    typename std::list<entry_t>::iterator it = table[i].begin();
    while (it != table[i].end() && !(it->key == k)) {
        ++it;
    }
    if (it == table[i].end()) {
        return false;
    }
    table[i].erase(it);
    num_entries--;
    return true;
}

V *find(const K &k) const {
    unsigned int i = Hash()(k) % table_size;
    typename std::list<entry_t>::iterator it = table[i].begin();
    while (it != table[i].end() && !(it->key == k)) {
        ++it;
    }
    if (it == table[i].end()) {
        return NULL;
    }
}

```

```

    }
    return &(it->value);
}

V &operator[](const K &k) {
    V *ret = find(k);
    if (ret != NULL) {
        return *ret;
    }
    insert(k, V());
    return *find(k);
}

template<class KVFunction>
void walk(KVFunction f) const {
    for (int i = 0; i < table_size; i++) {
        typename std::list<entry_t>::iterator it;
        for (it = table[i].begin(); it != table[i].end(); ++it) {
            f(it->key, it->value);
        }
    }
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

struct class_hash {
    unsigned int operator()(int k) { return class_hash()((unsigned int)k); }

    unsigned int operator()(long long k) { return class_hash()((unsigned long long)k); }

    // Knuth's one-to-one multiplicative method.
    unsigned int operator()(unsigned int k) {
        return k * 2654435761u; // Or just return k.
    }

    // Jenkins's 64-bit hash.
    unsigned int operator()(unsigned long long k) {
        k += ~(k << 32);
        k ^= (k >> 22);
        k += ~(k << 13);
        k ^= (k >> 8);
        k += (k << 3);
        k ^= (k >> 15);
        k += ~(k << 27);
        k ^= (k >> 31);
        return k;
    }

    // Jenkins's one-at-a-time hash.
    unsigned int operator()(const std::string &k) {
        unsigned int hash = 0;
        for (unsigned int i = 0; i < k.size(); i++) {
            hash += ((hash + k[i]) << 10);
            hash ^= (hash >> 6);
        }
        hash += (hash << 3);
        hash ^= (hash >> 11);
        return hash + (hash << 15);
    }
};

```

```

void printch(const string &k, char v) {
    cout << v;
}

int main() {
    hash_map<string, char, class_hash> m;
    m["foo"] = 'a';
    m.insert("bar", 'b');
    assert(m["foo"] == 'a');
    assert(m["bar"] == 'b');
    assert(m["baz"] == '\0');
    m["baz"] = 'c';
    m.walk(printch);
    cout << endl;
    assert(m.erase("foo"));
    assert(m.size() == 2);
    assert(m["foo"] == '\0');
    assert(m.size() == 3);
    return 0;
}

```

Example Output

cab

2.3.9 Skip List

Maintain a map, that is, a collection of key-value pairs such that each possible key appears at most once in the collection. This implementation requires operators `<` and `==` to be defined on the key type. A skip list maintains a linked hierarchy of sorted subsequences with each successive subsequence skipping over fewer elements than the previous one.

- `skip_list()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `NULL` if the key was not found.
- `operator[]` returns a reference to key `k`'s associated value (which may be modified), or if necessary, inserts and returns a new entry with the default constructed value if key `k` was not originally found.
- `walk(f)` calls the function `f(k, v)` on each entry of the map, in ascending order of keys.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$ on average per call to `insert()`, `erase()`, `find()`, and `operator[]`, where n is the number of entries currently in the map.
- $O(n)$ per call to `walk()`.

Space Complexity:

- $O(n)$ on average for storage of the map elements.
- $O(n)$ auxiliary heap space for `insert()` and `erase()`.
- $O(1)$ auxiliary for all other operations.

```

#include <cmath>
#include <cstdlib>
#include <vector>

```

```

template<class K, class V>
class skip_list {
    static const int MAX_LEVELS = 32; // log2(max possible keys)

    struct node_t {
        K key;
        V value;
        std::vector<node_t *> next;

        node_t(const K &k, const V &v, int levels) : key(k), value(v), next(levels, (node_t *)NULL) {}
    } *head;

    int num_nodes;

    static int random_level() {
        static const double p = 0.5;
        int level = 1;
        while (((double)rand() / RAND_MAX) < p && std::abs(level) < MAX_LEVELS) {
            level++;
        }
        return std::abs(level);
    }

    static int node_level(const std::vector<node_t *> &v) {
        int i = 0;
        while (i < (int)v.size() && v[i] != NULL) {
            i++;
        }
        return i + 1;
    }

public:
    skip_list() : head(new node_t(K(), V(), MAX_LEVELS)), num_nodes(0) {
        for (int i = 0; i < (int)head->next.size(); i++) {
            head->next[i] = NULL;
        }
    }

    ~skip_list() { delete head; }
    int size() const { return num_nodes; }
    bool empty() const { return num_nodes == 0; }

    bool insert(const K &k, const V &v) {
        std::vector<node_t *> update(head->next);
        int curr_level = node_level(update);
        node_t *n = head;
        for (int i = curr_level; i-- > 0;) {
            while (n->next[i] != NULL && n->next[i]->key < k) {
                n = n->next[i];
            }
            update[i] = n;
        }
        n = n->next[0];
        if (n != NULL && n->key == k) {
            return false;
        }
        int new_level = random_level();
        if (new_level > curr_level) {
            for (int i = curr_level; i < new_level; i++) {
                update[i] = head;
            }
        }
        n = new node_t(k, v, new_level);
        for (int i = 0; i < new_level; i++) {
            n->next[i] = update[i]->next[i];
            update[i]->next[i] = n;
        }
    }
}

```

```

    num_nodes++;
    return true;
}

bool erase(const K &k) {
    std::vector<node_t *> update(head->next);
    node_t *n = head;
    for (int i = node_level(update); i-- > 0;) {
        while (n->next[i] != NULL && n->next[i]->key < k) {
            n = n->next[i];
        }
        update[i] = n;
    }
    n = n->next[0];
    if (n != NULL && n->key == k) {
        for (int i = 0; i < (int)update.size(); i++) {
            if (update[i]->next[i] != n) {
                break;
            }
            update[i]->next[i] = n->next[i];
        }
        delete n;
        num_nodes--;
        return true;
    }
    return false;
}

V *find(const K &k) const {
    node_t *n = head;
    for (int i = node_level(n->next); i-- > 0;) {
        while (n->next[i] != NULL && n->next[i]->key < k) {
            n = n->next[i];
        }
    }
    n = n->next[0];
    return (n != NULL && n->key == k) ? &(n->value) : NULL;
}

V &operator[](const K &k) {
    V *ret = find(k);
    if (ret != NULL) {
        return *ret;
    }
    insert(k, V());
    return *find(k);
}

template<class KVFunction>
void walk(KVFunction f) const {
    node_t *n = head->next[0];
    while (n != NULL) {
        f(n->key, n->value);
        n = n->next[0];
    }
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

void printch(int k, char v) {
    cout << v;
}

```

```

}

int main() {
    skip_list<int, char> l;
    l.insert(2, 'b');
    l.insert(1, 'a');
    l.insert(3, 'c');
    l.insert(5, 'e');
    assert(l.insert(4, 'd'));
    assert(*l.find(4) == 'd');
    assert(!l.insert(4, 'd'));
    l.walk(printch);
    cout << endl;
    assert(l.erase(1));
    assert(!l.erase(1));
    assert(l.find(1) == NULL);
    l.walk(printch);
    cout << endl;
    return 0;
}

```

Example Output

```

abcde
bcde

```

2.4 Range Queries in One Dimension

2.4.1 Sparse Table (Range Minimum Query)

Given a static array with indices from 0 to $n - 1$, precompute a table that may later be used to perform range minimum queries on the array in constant time. This version is simplified to only work on integer arrays.

The dynamic programming state $dp[i][j]$ holds the index of the minimum value in the sub-array starting at i and having length 2^j . Each $dp[i][j]$ will always be equal to either $dp[i][j - 1]$ or $dp[i + 2^{j-1}][j - 1]$, whichever of the indices corresponds to the smaller value in the array.

Time Complexity:

- $O(n \log n)$ per call to `build()`, where n is the size of the array.
- $O(1)$ per call to `query()`.

Space Complexity:

- $O(n \log n)$ for storage of the sparse table, where n is the size of the array.
- $O(1)$ auxiliary for `query()`.

```

#include <vector>

const int MAXN = 1000;
std::vector<int> table, dp[MAXN];

void build(int n, int a[]) {
    table.resize(n + 1);
    for (int i = 2; i <= n; i++) {
        table[i] = table[i >> 1] + 1;
    }
    for (int i = 0; i < n; i++) {
        dp[i].resize(table[n] + 1);
        dp[i][0] = i;
    }
    for (int j = 1; (1 << j) < n; j++) {
        for (int i = 0; i + (1 << j) <= n; i++) {

```

```

        int x = dp[i][j - 1];
        int y = dp[i + (1 << (j - 1))][j - 1];
        dp[i][j] = (a[x] < a[y]) ? x : y;
    }
}

int query(int a[], int lo, int hi) {
    int j = table[hi - lo];
    int x = dp[lo][j];
    int y = dp[hi - (1 << j) + 1][j];
    return (a[x] < a[y]) ? a[x] : a[y];
}

```

Example Usage

```

#include <cassert>

int main() {
    int arr[5] = {6, -2, 1, 8, 10};
    build(5, arr);
    assert(query(arr, 0, 3) == -2);
    return 0;
}

```

2.4.2 Square Root Decomposition

Maintain a fixed-size array (from 0 to `size() - 1`) while supporting dynamic queries of contiguous sub-arrays and dynamic updates of individual indices.

The query operation is defined by an associative `join_values()` function which satisfies

`join_values(x, join_values(y, z)) = join_values(join_values(x, y), z)` for all values `x`, `y`, and `z` in the array. The default definition below assumes a numerical array type, supporting queries for the "min" of the target range. Another possible query operation is "sum", in which the `join_values()` function should be defined to return $a + b$.

The update operation is defined by the `join_value_with_delta()` function which determines the change made to array values. The default definition below supports updates that "set" the chosen array index to a new value. Another possible update operation is "increment", in which `join_value_with_delta(v, d)` should be defined to return $v + d$.

The operations supported by this data structure are identical to those of the point update segment tree found in this section.

Time Complexity:

- $O(n)$ per call to both constructors, where n is the size of the array.
- $O(1)$ per call to `size()`.
- $O(\sqrt{n})$ per call to `at()`, `update()`, and `query()`.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(1)$ auxiliary for all operations.

```

#include <algorithm>
#include <cmath>
#include <vector>

template<class T>
class sqrt_decomposition {
    static T join_values(const T &a, const T &b) { return std::min(a, b); }
}

```

```

static T join_value_with_delta(const T &v, const T &d) { return d; }

int len, blocklen;
std::vector<T> value, block;

void init() {
    blocklen = (int)sqrt(len);
    int nblocks = (len + blocklen - 1) / blocklen;
    for (int i = 0; i < nblocks; i++) {
        T blockval = value[i * blocklen];
        int blockhi = std::min(len, (i + 1) * blocklen);
        for (int j = i * blocklen + 1; j < blockhi; j++) {
            blockval = join_values(blockval, value[j]);
        }
        block.push_back(blockval);
    }
}

public:
sqrt_decomposition(int n, const T &v = T()) : len(n), value(n, v) { init(); }

template<class It>
sqrt_decomposition(It lo, It hi) : len(hi - lo), value(lo, hi) {
    init();
}

int size() const { return len; }
T at(int i) const { return query(i, i); }

T query(int lo, int hi) const {
    T res;
    int blocklo = ceil((double)lo / blocklen), blockhi = (hi + 1) / blocklen - 1;
    if (blocklo > blockhi) {
        res = value[lo];
        for (int i = lo + 1; i <= hi; i++) {
            res = join_values(res, value[i]);
        }
    } else {
        res = block[blocklo];
        for (int i = blocklo + 1; i <= blockhi; i++) {
            res = join_values(res, block[i]);
        }
        for (int i = lo; i < blocklo * blocklen; i++) {
            res = join_values(res, value[i]);
        }
        for (int i = (blockhi + 1) * blocklen; i <= hi; i++) {
            res = join_values(res, value[i]);
        }
    }
    return res;
}

void update(int i, const T &d) {
    value[i] = join_value_with_delta(value[i], d);
    int b = i / blocklen;
    int blockhi = std::min(len, (b + 1) * blocklen);
    block[b] = value[b * blocklen];
    for (int i = b * blocklen + 1; i < blockhi; i++) {
        block[b] = join_values(block[b], value[i]);
    }
}
};

```

Example Usage


```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {6, -2, 1, 8, 10};
    sqrt_decomposition<int> sd(arr, arr + 5);
    sd.update(2, 4);
    cout << "Values:";
    for (int i = 0; i < sd.size(); i++) {
        cout << " " << sd.at(i);
    }
    cout << endl;
    assert(sd.query(0, 3) == -2);
    return 0;
}

```

Example Output

Values: 6 -2 4 8 10

2.4.3 Segment Tree (Point Update)

Maintain a fixed-size array while supporting dynamic queries of contiguous sub-arrays and dynamic updates of individual indices.

The query operation is defined by an associative `join_values()` function which satisfies

`join_values(x, join_values(y, z)) = join_values(join_values(x, y), z)` for all values `x`, `y`, and `z` in the array. The default code below assumes a numerical array type, defining queries for the "min" of the target range. Another possible query operation is "sum", in which case the `join_values()` function should be defined to return $a + b$.

The update operation is defined by the `join_value_with_delta()` function, which determines the change made to array values. The default definition below supports updates that "set" the chosen array index to a new value. Another possible update operation is "increment", in which `join_value_with_delta(v, d)` should be defined to return $v + d$.

- `segment_tree(n, v)` constructs an array of size `n` with indices from 0 to `n - 1`, inclusive, and all values initialized to `v`.
- `segment_tree(lo, hi)` constructs an array from two random-access iterators as a range `[lo, hi)`, initialized to the elements of the range in the same order.
- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `query(lo, hi)` returns the result of `join_values()` applied to all indices from `lo` to `hi`, inclusive. If the distance between `lo` and `hi` is 1, then the single specified value is returned.
- `update(i, d)` assigns the value `v` at index `i` to `join_value_with_delta(v, d)`.

Time Complexity:

- $O(n)$ per call to both constructors, where n is the size of the array.
- $O(1)$ per call to `size()`.
- $O(\log n)$ per call to `at()`, `update()`, and `query()`.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(\log n)$ auxiliary stack space for `update()` and `query()`.
- $O(1)$ auxiliary for `size()`.

```

#include <algorithm>
#include <vector>

```

```

template<class T>
class segment_tree {
    static T join_values(const T &a, const T &b) { return std::min(a, b); }
    static T join_value_with_delta(const T &v, const T &d) { return d; }

    int len;
    std::vector<T> value;

    void build(int i, int lo, int hi, const T &v) {
        if (lo == hi) {
            value[i] = v;
            return;
        }
        int mid = lo + (hi - lo) / 2;
        build(i * 2 + 1, lo, mid, v);
        build(i * 2 + 2, mid + 1, hi, v);
        value[i] = join_values(value[i * 2 + 1], value[i * 2 + 2]);
    }

    template<class It>
    void build(int i, int lo, int hi, It arr) {
        if (lo == hi) {
            value[i] = *(arr + lo);
            return;
        }
        int mid = lo + (hi - lo) / 2;
        build(i * 2 + 1, lo, mid, arr);
        build(i * 2 + 2, mid + 1, hi, arr);
        value[i] = join_values(value[i * 2 + 1], value[i * 2 + 2]);
    }

    T query(int i, int lo, int hi, int tgt_lo, int tgt_hi) const {
        if (lo == tgt_lo && hi == tgt_hi) {
            return value[i];
        }
        int mid = lo + (hi - lo) / 2;
        if (tgt_lo <= mid && mid < tgt_hi) {
            return join_values(
                query(i * 2 + 1, lo, mid, tgt_lo, std::min(tgt_hi, mid)),
                query(i * 2 + 2, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi)
            );
        }
        if (tgt_lo <= mid) {
            return query(i * 2 + 1, lo, mid, tgt_lo, std::min(tgt_hi, mid));
        }
        return query(i * 2 + 2, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi);
    }

    void update(int i, int lo, int hi, int target, const T &d) {
        if (target < lo || target > hi) {
            return;
        }
        if (lo == hi) {
            value[i] = join_value_with_delta(value[i], d);
            return;
        }
        int mid = lo + (hi - lo) / 2;
        update(i * 2 + 1, lo, mid, target, d);
        update(i * 2 + 2, mid + 1, hi, target, d);
        value[i] = join_values(value[i * 2 + 1], value[i * 2 + 2]);
    }

public:
    segment_tree(int n, const T &v = T()) : len(n), value(4 * len) {
        if (len > 0) {
            build(0, 0, len - 1, v);
        }
    }
}

```

```

template<class It>
segment_tree(It lo, It hi) : len(hi - lo), value(4 * len) {
    if (len > 0) {
        build(0, 0, len - 1, lo);
    }
}

int size() const { return len; }
T at(int i) const { return query(i, i); }
T query(int lo, int hi) const { return query(0, 0, len - 1, lo, hi); }
void update(int i, const T &d) { update(0, 0, len - 1, i, d); }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {6, -2, 1, 8, 10};
    segment_tree<int> t(arr, arr + 5);
    t.update(2, 4);
    cout << "Values:";
    for (int i = 0; i < t.size(); i++) {
        cout << " " << t.at(i);
    }
    cout << endl;
    assert(t.query(0, 3) == -2);
    return 0;
}

```

Example Output

```

Values: 6 -2 4 8 10
The minimum value in the range [0, 3] is -2.

```

2.4.4 Segment Tree (Range Update)

Maintain a fixed-size array while supporting both dynamic queries and updates of contiguous subarrays via the lazy propagation technique.

The query operation is defined by an associative `join_values()` function which satisfies

`join_values(x, join_values(y, z)) = join_values(join_values(x, y), z)` for all values `x`, `y`, and `z` in the array. The default code below assumes a numerical array type, defining queries for the "min" of the target range. Another possible query operation is "sum", in which case the `join_values()` function should be defined to return $a + b$.

The update operation is defined by the `join_value_with_delta()` and `join_deltas()` functions, which determines the change made to array values. These must satisfy:

- `join_deltas(d1, join_deltas(d2, d3)) = join_deltas(join_deltas(d1, d2), d3)`.
- `join_value_with_delta(join_values(v, ..., v), d, m)` should be equal to `join_values(join_value_with_delta(v, d, 1), ...)`, with m values on each side.
- if a sequence $d_1, \dots, d_m$ of deltas is used to update a value v , then `join_value_with_delta(v, join_deltas(d_1, ..., d_m), 1)` should be equivalent to m sequential calls to `join_value_with_delta(v, d_i, 1)` for $i = 1, \dots, m$. The default code below defines updates that "set" the chosen array index to a new value. Another possible update operation is "increment", in which case `join_value_with_delta(v, d, len)` should be defined to return $v + d \cdot \text{len}$ and `join_deltas(d1, d2)` should be defined to return $d1 + d2$.

- `segment_tree(n, v)` constructs an array of size `n` with indices from 0 to `n - 1`, inclusive, and all values initialized to `v`.
- `segment_tree(lo, hi)` constructs an array from two random-access iterators as a range `[lo, hi)`, initialized to the elements of the range in the same order.
- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`, where `i` is between 0 and `size() - 1`.
- `query(lo, hi)` returns the result of `join_values()` applied to all indices from `lo` to `hi`, inclusive. If the distance between `lo` and `hi` is 1, then the single specified value is returned.
- `update(i, d)` assigns the value `v` at index `i` to `join_value_with_delta(v, d)`.
- `update(lo, hi, d)` modifies the value at each array index from `lo` to `hi`, inclusive, by respectively joining them with `d` using `join_value_with_delta()`.

Time Complexity:

- $O(n)$ per call to both constructors, where n is the size of the array.
- $O(1)$ per call to `size()`.
- $O(\log n)$ per call to `at()`, `update()`, and `query()`.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(\log n)$ auxiliary stack space for `update()` and `query()`.
- $O(1)$ auxiliary for `size()`.

```
#include <algorithm>
#include <vector>

template<class T>
class segment_tree {
    static T join_values(const T &a, const T &b) { return std::min(a, b); }
    static T join_value_with_delta(const T &v, const T &d, int len) { return d; }

    static T join_deltas(const T &d1, const T &d2) {
        return d2; // For "set" updates, the more recent delta prevails.
    }

    int len;
    std::vector<T> value, delta;
    std::vector<bool> pending;

    void build(int i, int lo, int hi, const T &v) {
        if (lo == hi) {
            value[i] = v;
            return;
        }
        int mid = lo + (hi - lo) / 2;
        build(i * 2 + 1, lo, mid, v);
        build(i * 2 + 2, mid + 1, hi, v);
        value[i] = join_values(value[i * 2 + 1], value[i * 2 + 2]);
    }

    template<class It>
    void build(int i, int lo, int hi, It arr) {
        if (lo == hi) {
            value[i] = *(arr + lo);
            return;
        }
        int mid = lo + (hi - lo) / 2;
        build(i * 2 + 1, lo, mid, arr);
        build(i * 2 + 2, mid + 1, hi, arr);
        value[i] = join_values(value[i * 2 + 1], value[i * 2 + 2]);
    }

    void push_delta(int i, int lo, int hi) {
        if (pending[i]) {
```

```

    value[i] = join_value_with_delta(value[i], delta[i], hi - lo + 1);
    if (lo != hi) {
        int l = 2 * i + 1, r = 2 * i + 2;
        delta[l] = pending[l] ? join_deltas(delta[l], delta[i]) : delta[i];
        delta[r] = pending[r] ? join_deltas(delta[r], delta[i]) : delta[i];
        pending[l] = pending[r] = true;
    }
    pending[i] = false;
}

T query(int i, int lo, int hi, int tgt_lo, int tgt_hi) {
    push_delta(i, lo, hi);
    if (lo == tgt_lo && hi == tgt_hi) {
        return value[i];
    }
    int mid = lo + (hi - lo) / 2;
    if (tgt_lo <= mid && mid < tgt_hi) {
        return join_values(
            query(i * 2 + 1, lo, mid, tgt_lo, std::min(tgt_hi, mid)),
            query(i * 2 + 2, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi)
        );
    }
    if (tgt_lo <= mid) {
        return query(i * 2 + 1, lo, mid, tgt_lo, std::min(tgt_hi, mid));
    }
    return query(i * 2 + 2, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi);
}

void update(int i, int lo, int hi, int tgt_lo, int tgt_hi, const T &d) {
    push_delta(i, lo, hi);
    if (hi < tgt_lo || lo > tgt_hi) {
        return;
    }
    if (tgt_lo <= lo && hi <= tgt_hi) {
        delta[i] = d;
        pending[i] = true;
        push_delta(i, lo, hi);
        return;
    }
    update(2 * i + 1, lo, (lo + hi) / 2, tgt_lo, tgt_hi, d);
    update(2 * i + 2, (lo + hi) / 2 + 1, hi, tgt_lo, tgt_hi, d);
    value[i] = join_values(value[2 * i + 1], value[2 * i + 2]);
}

public:
    segment_tree(int n, const T &v = T())
        : len(n), value(4 * len), delta(4 * len), pending(4 * len, false) {
        if (len > 0) {
            build(0, 0, len - 1, v);
        }
    }

    template<class It>
    segment_tree(It lo, It hi)
        : len(hi - lo), value(4 * len), delta(4 * len), pending(4 * len, false) {
        if (len > 0) {
            build(0, 0, len - 1, lo);
        }
    }

    int size() const { return len; }
    T at(int i) { return query(i, i); }
    T query(int lo, int hi) { return query(0, 0, len - 1, lo, hi); }
    void update(int i, const T &d) { update(0, 0, len - 1, i, i, d); }
    void update(int lo, int hi, const T &d) { update(0, 0, len - 1, lo, hi, d); }
};

```

Example Usage

```
#include <cassert>
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {6, -2, 1, 8, 10};
    segment_tree<int> t(arr, arr + 5);
    t.update(2, 4);
    cout << "Values:";
    for (int i = 0; i < t.size(); i++) {
        cout << " " << t.at(i);
    }
    cout << endl;
    assert(t.query(0, 3) == -2);
    t.update(0, 4, 5);
    t.update(3, 2);
    t.update(3, 1);
    cout << "Values:";
    for (int i = 0; i < t.size(); i++) {
        cout << " " << t.at(i);
    }
    cout << endl;
    assert(t.query(0, 3) == 1);
    return 0;
}
```

Example Output

```
Values: 6 -2 4 8 10
Values: 5 5 5 1 5
```

2.4.5 Segment Tree (Compressed)

Maintain a fixed-size array while supporting both dynamic queries and updates of contiguous subarrays via the lazy propagation technique. This implementation uses lazy initialization of nodes to conserve memory while supporting large indices.

The query operation is defined by an associative `join_values()` function which satisfies

`join_values(x, join_values(y, z)) = join_values(join_values(x, y), z)` for all values `x`, `y`, and `z` in the array. The default code below assumes a numerical array type, defining queries for the "min" of the target range. Another possible query operation is "sum", in which case the `join_values()` function should be defined to return $a + b$.

The update operation is defined by the `join_value_with_delta()` and `join_deltas()` functions, which determines the change made to array values. These must satisfy:

- `join_deltas(d1, join_deltas(d2, d3)) = join_deltas(join_deltas(d1, d2), d3)`.
- `join_value_with_delta(join_values(v, ..., v), d, m)` should be equal to `join_values(join_value_with_delta(v, d, 1), ...)`, with m values on each side.
- if a sequence $d_1, \dots, d_m$ of deltas is used to update a value v , then `join_value_with_delta(v, join_deltas(d_1, ..., d_m), 1)` should be equivalent to m sequential calls to `join_value_with_delta(v, d_i, 1)` for $i = 1, \dots, m$. The default code below defines updates that "set" the chosen array index to a new value. Another possible update operation is "increment", in which case `join_value_with_delta(v, d, len)` should be defined to return $v + d \cdot len$ and `join_deltas(d1, d2)` should be defined to return $d1 + d2$.
- `segment_tree(n, v)` constructs an array of size `n` with indices from 0 to `n - 1`, inclusive, and all values initialized to `v`.
- `segment_tree(lo, hi)` constructs an array from two random-access iterators as a range `[lo, hi)`, initialized to the elements of the range in the same order.

- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`, where `i` is between 0 and `size() - 1`.
- `query(lo, hi)` returns the result of `join_values()` applied to all indices from `lo` to `hi`, inclusive. If the distance between `lo` and `hi` is 1, then the single specified value is returned.
- `update(i, d)` assigns the value `v` at index `i` to `join_value_with_delta(v, d)`.
- `update(lo, hi, d)` modifies the value at each array index from `lo` to `hi`, inclusive, by respectively joining them with `d` using `join_value_with_delta()`.

Time Complexity:

- $O(n)$ per call to both constructors, where n is the size of the array.
- $O(1)$ per call to `size()`.
- $O(\log n)$ per call to `at()`, `update()`, and `query()`.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(\log n)$ auxiliary stack space for `update()` and `query()`.
- $O(1)$ auxiliary for `size()`.

```
#include <algorithm>
#include <cstdint>

template<class T>
class segment_tree {
    static const int MAXN = 1000000000;

    static T join_values(const T &a, const T &b) { return std::min(a, b); }
    static T join_segment(const T &v, int len) { return v; }
    static T join_value_with_delta(const T &v, const T &d, int len) { return d; }

    static T join_deltas(const T &d1, const T &d2) {
        return d2; // For "set" updates, the more recent delta prevails.
    }

    struct node_t {
        T value, delta;
        bool pending;
        node_t *left, *right;

        node_t(const T &v) : value(v), pending(false), left(NULL), right(NULL) {}
    } *root;

    T init;

    void update_delta(node_t *&n, const T &d, int len) {
        if (n == NULL) {
            n = new node_t(join_segment(init, len));
        }
        n->delta = n->pending ? join_deltas(n->delta, d) : d;
        n->pending = true;
    }

    void push_delta(node_t *&n, int lo, int hi) {
        if (n == NULL) {
            return;
        }
        if (n->pending) {
            n->value = join_value_with_delta(n->value, n->delta, hi - lo + 1);
            if (lo != hi) {
                int mid = lo + (hi - lo) / 2;
                update_delta(n->left, n->delta, mid - lo + 1);
                update_delta(n->right, n->delta, hi - mid);
            }
        }
        n->pending = false;
    }
};
```

```

}

T query(node_t *n, int lo, int hi, int tgt_lo, int tgt_hi) {
    if (n == NULL) {
        return join_segment(init, hi - lo + 1);
    }
    push_delta(n, lo, hi);
    if (lo == tgt_lo && hi == tgt_hi) {
        return n->value;
    }
    int mid = lo + (hi - lo) / 2;
    if (tgt_lo <= mid && mid < tgt_hi) {
        return join_values(
            query(n->left, lo, mid, tgt_lo, std::min(tgt_hi, mid)),
            query(n->right, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi)
        );
    }
    if (tgt_lo <= mid) {
        return query(n->left, lo, mid, tgt_lo, std::min(tgt_hi, mid));
    }
    return query(n->right, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi);
}

void update(node_t *&n, int lo, int hi, int tgt_lo, int tgt_hi, const T &d) {
    if (n == NULL) {
        n = new node_t(join_segment(init, hi - lo + 1));
    }
    push_delta(n, lo, hi);
    if (hi < tgt_lo || lo > tgt_hi) {
        return;
    }
    if (tgt_lo <= lo && hi <= tgt_hi) {
        n->delta = d;
        n->pending = true;
        push_delta(n, lo, hi);
        return;
    }
    int mid = lo + (hi - lo) / 2;
    update(n->left, lo, mid, tgt_lo, tgt_hi, d);
    update(n->right, mid + 1, hi, tgt_lo, tgt_hi, d);
    n->value = join_values(n->left->value, n->right->value);
}

void clean_up(node_t *n) {
    if (n != NULL) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    segment_tree(const T &v = T()) : root(NULL), init(v) {}

    ~segment_tree() { clean_up(root); }
    T at(int i) { return query(i, i); }
    T query(int lo, int hi) { return query(root, 0, MAXN, lo, hi); }
    void update(int i, const T &d) { return update(i, i, d); }
    void update(int lo, int hi, const T &d) { return update(root, 0, MAXN, lo, hi, d); }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

```



```

int main() {
    segment_tree<int> t(0);
    t.update(0, 6);
    t.update(1, -2);
    t.update(2, 4);
    t.update(3, 8);
    t.update(4, 10);
    cout << "Values:";
    for (int i = 0; i < 5; i++) {
        cout << " " << t.at(i);
    }
    cout << endl;
    assert(t.query(0, 3) == -2);
    t.update(0, 4, 5);
    t.update(3, 2);
    t.update(3, 1);
    cout << "Values:";
    for (int i = 0; i < 5; i++) {
        cout << " " << t.at(i);
    }
    cout << endl;
    assert(t.query(0, 3) == 1);
    return 0;
}

```

Example Output

```

Values: 6 -2 4 8 10
Values: 5 5 5 1 5

```

2.4.6 Segment Tree (Persistent)

Maintains immutable versions of a segment tree under point updates. Each update creates a new root by copying only the nodes on the path from the root to the updated leaf, while all unchanged subtrees are shared with previous versions.

The implementation below stores sums over a fixed index range $[0, n - 1]$. Persistent segment trees are useful for offline order-statistics queries, rollback-like histories, and functional dynamic programming states.

- `persistent_segment_tree(a)` constructs version 0 from array `a`.
- `update(version, index, value)` returns a new version where `a[index]` is set to `value`, leaving the old version unchanged.
- `query(version, lo, hi)` returns the sum of the inclusive range `[lo, hi]` in the specified version.
- `versions()` returns the number of stored roots.

Time Complexity:

- $O(n)$ for construction.
- $O(\log n)$ per call to `update(version, index, value)` and `query(version, lo, hi)`.

Space Complexity:

- $O(n + q \log n)$ for q updates.
- $O(\log n)$ auxiliary stack space per update and query.

```

#include <vector>

class persistent_segment_tree {
    struct node {
        long long sum;
        int left, right;

        node(long long sum = 0, int left = -1, int right = -1) : sum(sum), left(left), right(right) {}
    };

```

```

};

int n;
std::vector<node> tree;
std::vector<int> root;

int build(const std::vector<int> &a, int lo, int hi) {
    if (lo == hi) {
        tree.push_back(node(a[lo]));
        return (int)tree.size() - 1;
    }
    int mid = lo + (hi - lo) / 2;
    int left = build(a, lo, mid);
    int right = build(a, mid + 1, hi);
    tree.push_back(node(tree[left].sum + tree[right].sum, left, right));
    return (int)tree.size() - 1;
}

int update(int cur, int lo, int hi, int index, int value) {
    if (lo == hi) {
        tree.push_back(node(value));
        return (int)tree.size() - 1;
    }
    int mid = lo + (hi - lo) / 2;
    int left = tree[cur].left, right = tree[cur].right;
    if (index <= mid) {
        left = update(left, lo, mid, index, value);
    } else {
        right = update(right, mid + 1, hi, index, value);
    }
    tree.push_back(node(tree[left].sum + tree[right].sum, left, right));
    return (int)tree.size() - 1;
}

long long query(int cur, int lo, int hi, int qlo, int qhi) const {
    if (qlo <= lo && hi <= qhi) {
        return tree[cur].sum;
    }
    int mid = lo + (hi - lo) / 2;
    long long res = 0;
    if (qlo <= mid) {
        res += query(tree[cur].left, lo, mid, qlo, qhi);
    }
    if (mid < qhi) {
        res += query(tree[cur].right, mid + 1, hi, qlo, qhi);
    }
    return res;
}

public:
explicit persistent_segment_tree(const std::vector<int> &a) : n(a.size()) {
    if (n > 0) {
        root.push_back(build(a, 0, n - 1));
    }
}

int versions() const { return root.size(); }

int update(int version, int index, int value) {
    root.push_back(update(root[version], 0, n - 1, index, value));
    return (int)root.size() - 1;
}

long long query(int version, int lo, int hi) const {
    return query(root[version], 0, n - 1, lo, hi);
}
};

```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    int raw[] = {1, 2, 3, 4};
    vector<int> a(raw, raw + 4);
    persistent_segment_tree t(a);
    int v1 = t.update(0, 1, 10);
    int v2 = t.update(v1, 3, -1);
    assert(t.query(0, 0, 3) == 10);
    assert(t.query(v1, 0, 3) == 18);
    assert(t.query(v2, 1, 3) == 12);
    assert(t.versions() == 3);
    return 0;
}
```

2.4.7 Cartesian Tree

Builds the min Cartesian tree of an array in linear time. A Cartesian tree is a binary tree whose inorder traversal is the original array order and whose heap property is determined by array values. For a min Cartesian tree, each parent has value less than or equal to its children.

Cartesian trees connect arrays, stacks, range minimum queries, treaps, and largest rectangle style problems. Equal values are broken by position, so earlier equal values become ancestors of later equal values.

- `cartesian_tree(a)` constructs the tree for array `a`.
- `root` stores the root index.
- `parent[i]`, `left[i]`, and `right[i]` store neighboring node indices, or `-1` if absent.

Time Complexity: $O(n)$ construction time, where n is the array size.

Space Complexity: $O(n)$ for tree arrays and the monotone stack.

```
#include <vector>

struct cartesian_tree {
    int root;
    std::vector<int> parent, left, right;

    explicit cartesian_tree(const std::vector<int> &a)
        : root(-1), parent(a.size(), -1), left(a.size(), -1), right(a.size(), -1) {
        std::vector<int> st;
        for (int i = 0; i < (int)a.size(); i++) {
            int last = -1;
            while (!st.empty() && a[i] < a[st.back()]) {
                last = st.back();
                st.pop_back();
            }
            if (!st.empty()) {
                right[st.back()] = i;
                parent[i] = st.back();
            }
            if (last != -1) {
                left[i] = last;
                parent[last] = i;
            }
            st.push_back(i);
        }
        root = st.empty() ? -1 : st[0];
    }
};
```

Example Usage

```
#include <cassert>
using namespace std;

void inorder(const cartesian_tree &t, int u, vector<int> *order) {
    if (u == -1) {
        return;
    }
    inorder(t, t.left[u], order);
    order->push_back(u);
    inorder(t, t.right[u], order);
}

int main() {
    int raw[] = {3, 2, 6, 1, 9};
    vector<int> a(raw, raw + 5);
    cartesian_tree t(a);
    assert(t.root == 3);
    assert(t.parent[1] == 3);
    assert(t.parent[4] == 3);

    vector<int> order;
    inorder(t, t.root, &order);
    for (int i = 0; i < 5; i++) {
        assert(order[i] == i);
    }
    return 0;
}
```

2.4.8 Implicit Treap

Maintain a dynamically-sized array using a balanced binary search tree while supporting both dynamic queries and updates of contiguous subarrays via the lazy propagation technique. A treap maintains a balanced binary tree structure by preserving the heap property on the randomly generated priority values of nodes, thereby making insertions and deletions run in $O(\log n)$ with high probability.

The query operation is defined by an associative `join_values()` function which satisfies

`join_values(x, join_values(y, z)) = join_values(join_values(x, y), z)` for all values `x`, `y`, and `z` in the array. The default code below assumes a numerical array type, defining queries for the "min" of the target range. Another possible query operation is "sum", in which case the `join_values()` function should be defined to return $a + b$.

The update operation is defined by the `join_value_with_delta()` and `join_deltas()` functions, which determines the change made to array values. These must satisfy:

- `join_deltas(d1, join_deltas(d2, d3)) = join_deltas(join_deltas(d1, d2), d3)`.
- `join_value_with_delta(join_values(v, ..., v), d, m)` should be equal to `join_values(join_value_with_delta(v, d, 1), ...)`, with m values on each side.
- if a sequence $d_1, \dots, d_m$ of deltas is used to update a value v , then `join_value_with_delta(v, join_deltas(d1, ..., dm), 1)` should be equivalent to m sequential calls to `join_value_with_delta(v, d_i, 1)` for $i = 1, \dots, m$. The default code below defines updates that "set" the chosen array index to a new value. Another possible update operation is "increment", in which case `join_value_with_delta(v, d, len)` should be defined to return $v + d \cdot \text{len}$ and `join_deltas(d1, d2)` should be defined to return $d1 + d2$.

This data structure shares every operation of one-dimensional segment trees in this section, with the additional operations `empty()`, `insert()`, `erase()`, `push_back()`, and `pop_back()` analogous to those of `std::vector` (here, `insert()` and `erase()` both take an index instead of an iterator).

Time Complexity:

- $O(n)$ per call to both constructors, where n is the size of the array.
- $O(1)$ per call to `size()` and `empty()`.
- $O(\log n)$ on average per call to all other operations.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(1)$ auxiliary for `size()` and `empty()`.
- $O(\log n)$ auxiliary stack space for all other operations.

```
#include <cstdlib>

template<class T>
class implicit_treap {
    static T join_values(const T &a, const T &b) { return a < b ? a : b; }
    static T join_value_with_delta(const T &v, const T &d, int len) { return d; }
    static T join_deltas(const T &d1, const T &d2) { return d2; }

    struct node_t {
        static inline int rand32() { return (rand() & 0x7fff) | ((rand() & 0x7fff) << 15); }

        T value, subtree_value, delta;
        bool pending;
        int size, priority;
        node_t *left, *right;

        node_t(const T &v)
            : value(v),
              subtree_value(v),
              pending(false),
              size(1),
              priority(rand32()),
              left(NULL),
              right(NULL) {}
    } *root;

    static int size(node_t *n) { return (n == NULL) ? 0 : n->size; }

    static void update_value(node_t *n) {
        if (n == NULL) {
            return;
        }
        n->subtree_value = n->value;
        if (n->left != NULL) {
            n->subtree_value = join_values(n->subtree_value, n->left->subtree_value);
        }
        if (n->right != NULL) {
            n->subtree_value = join_values(n->subtree_value, n->right->subtree_value);
        }
        n->size = 1 + size(n->left) + size(n->right);
    }

    static void update_delta(node_t *n, const T &d) {
        if (n != NULL) {
            n->value = join_value_with_delta(n->value, d, 1);
            n->subtree_value = join_value_with_delta(n->subtree_value, d, n->size);
            n->delta = n->pending ? join_deltas(n->delta, d) : d;
            n->pending = true;
        }
    }

    static void push_delta(node_t *n) {
        if (n == NULL || !n->pending) {
            return;
        }
        if (n->size > 1) {
            update_delta(n->left, n->delta);
        }
    }
};
```

```

    update_delta(n->right, n->delta);
}
n->pending = false;
}

static void merge(node_t *&n, node_t *left, node_t *right) {
    push_delta(left);
    push_delta(right);
    if (left == NULL) {
        n = right;
    } else if (right == NULL) {
        n = left;
    } else if (left->priority < right->priority) {
        merge(left->right, left->right, right);
        n = left;
    } else {
        merge(right->left, left, right->left);
        n = right;
    }
    update_value(n);
}

static void split(node_t *n, node_t *&left, node_t *&right, int i) {
    push_delta(n);
    if (n == NULL) {
        left = right = NULL;
    } else if (i <= size(n->left)) {
        split(n->left, left, n->left, i);
        right = n;
    } else {
        split(n->right, n->right, right, i - size(n->left) - 1);
        left = n;
    }
    update_value(n);
}

static void insert(node_t *&n, node_t *new_node, int i) {
    push_delta(n);
    if (n == NULL) {
        n = new_node;
    } else if (new_node->priority < n->priority) {
        split(n, new_node->left, new_node->right, i);
        n = new_node;
    } else if (i <= size(n->left)) {
        insert(n->left, new_node, i);
    } else {
        insert(n->right, new_node, i - size(n->left) - 1);
    }
    update_value(n);
}

static void erase(node_t *&n, int i) {
    push_delta(n);
    if (i == size(n->left)) {
        node_t *left = n->left, *right = n->right;
        delete n;
        merge(n, left, right);
    } else if (i < size(n->left)) {
        erase(n->left, i);
    } else {
        erase(n->right, i - size(n->left) - 1);
    }
    update_value(n);
}

static node_t *select(node_t *n, int i) {
    push_delta(n);
    if (i < size(n->left)) {

```

```

        return select(n->left, i);
    }
    if (i > size(n->left)) {
        return select(n->right, i - size(n->left) - 1);
    }
    return n;
}

void clean_up(node_t *&n) {
    if (n != NULL) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    implicit_treap(int n = 0, const T &v = T()) : root(NULL) {
        for (int i = 0; i < n; i++) {
            push_back(v);
        }
    }

    template<class It>
    implicit_treap(It lo, It hi) : root(NULL) {
        for (; lo != hi; ++lo) {
            push_back(*lo);
        }
    }

    ~implicit_treap() { clean_up(root); }
    int size() const { return size(root); }
    bool empty() const { return root == NULL; }
    void insert(int i, const T &v) { insert(root, new node_t(v), i); }
    void erase(int i) { erase(root, i); }
    void push_back(const T &v) { insert(size(), v); }
    void pop_back() { erase(size() - 1); }
    T at(int i) const { return select(root, i)->value; }

    T query(int lo, int hi) {
        node_t *l1, *r1, *l2, *r2, *t;
        split(root, l1, r1, hi + 1);
        split(l1, l2, r2, lo);
        T res = r2->subtree_value;
        merge(t, l2, r2);
        merge(root, t, r1);
        return res;
    }

    void update(int lo, int hi, const T &d) {
        node_t *l1, *r1, *l2, *r2, *t;
        split(root, l1, r1, hi + 1);
        split(l1, l2, r2, lo);
        update_delta(r2, d);
        merge(t, l2, r2);
        merge(root, t, r1);
    }

    void update(int i, const T &d) { update(i, i, d); }
};

```

Example Usage

```

#include <iostream>
using namespace std;

```

```

void print(implicit_treap<int> &t) {
    cout << "Values:";
    for (int i = 0; i < t.size(); i++) {
        cout << " " << t.at(i);
    }
    cout << " (min: " << t.query(0, t.size() - 1) << ")" << endl;
}

int main() {
    int arr[5] = {99, -2, 1, 8, 10};
    implicit_treap<int> t(arr, arr + 5);
    t.push_back(11);
    t.push_back(12);
    t.pop_back();
    print(t);
    t.insert(0, 90);
    t.erase(1);
    print(t);
    t.update(0, 1, 2);
    print(t);
    return 0;
}

```

Example Output

```

Values: 99 -2 1 8 10 11 (min: -2)
Values: 90 -2 1 8 10 11 (min: -2)
Values: 2 2 1 8 10 11 (min: 1)

```

2.5 Range Queries in Two Dimensions

2.5.1 Quadtree (Point Update)

Maintain a two-dimensional array while supporting dynamic queries of rectangular sub-arrays and dynamic updates of individual indices. This implementation uses lazy initialization of nodes to conserve memory while supporting large indices.

The query operation is defined by the `join_values()` and `join_region()` functions where

`join_values(x, join_values(y, z)) = join_values(join_values(x, y), z)` for all values `x`, `y`, and `z` in the array. The `join_region(v, area)` function must be defined in conjunction to efficiently return the result of `join_values()` applied to a rectangular sub-array of `area` elements. The default code below assumes a numerical array type, defining queries for the "min" of the target range. Another possible query operation is "sum", in which case `join_values(a, b)` should return $a + b$ and `join_region(v, area)` should return $v \cdot area$.

The update operation is defined by the `join_value_with_delta()` function, which determines the change made to array values. The default code below defines updates that "set" the chosen array index to a new value. Another possible update operation is "increment", in which `join_value_with_delta(v, d)` should be defined to return $v + d$.

- `quadtree(v)` constructs a two-dimensional array with rows from 0 to `MAXR` and columns from 0 to `MAXC`, inclusive. All values are implicitly initialized to `v`.
- `at(r, c)` returns the value at row `r`, column `c`.
- `query(r1, c1, r2, c2)` returns the result of `join_values()` applied to every value in the rectangular region consisting of rows from `r1` to `r2`, inclusive, and columns from `c1` to `c2`, inclusive.
- `update(r, c, d)` assigns the value `v` at `(r, c)` to `join_value_with_delta(v, d)`.

Time Complexity:

- $O(1)$ per call to the constructor.
- $O(\max(\text{MAXR}, \text{MAXC}))$ per call to `at()`, `update()`, and `query()`.

Space Complexity:

- $O(n)$ for storage of the array elements, where n is the number of updated entries in the array.
- $O(\sqrt{\max(\text{MAXR}, \text{MAXC})})$ auxiliary stack space for `update()`, `query()`, and `at()`.

```
#include <algorithm>
#include <cstdio>

template<class T>
class quadtree {
    static const int MAXR = 1000000000;
    static const int MAXC = 1000000000;

    static T join_values(const T &a, const T &b) { return std::min(a, b); }
    static T join_region(const T &v, int area) { return v; }
    static T join_value_with_delta(const T &v, const T &d) { return d; }

    struct node_t {
        T value;
        node_t *child[4];

        node_t(const T &v) : value(v) {
            for (int i = 0; i < 4; i++) {
                child[i] = NULL;
            }
        }
    };

    node_t *root;
    T init;

    // Helper variables for query().
    int tgt_r1, tgt_c1, tgt_r2, tgt_c2;
    T res;
    bool found;

    void query(node_t *n, int r1, int c1, int r2, int c2) {
        if (tgt_r2 < r1 || tgt_r1 > r2 || tgt_c2 < c1 || tgt_c1 > c2) {
            return;
        }
        if (n == NULL) {
            int rlen = std::min(r2, tgt_r2) - std::max(r1, tgt_r1) + 1;
            int clen = std::min(c2, tgt_c2) - std::max(c1, tgt_c1) + 1;
            T v = join_region(init, rlen * clen);
            res = found ? join_values(res, v) : v;
            found = true;
            return;
        }
        if (tgt_r1 <= r1 && r2 <= tgt_r2 && tgt_c1 <= c1 && c2 <= tgt_c2) {
            res = found ? join_values(res, n->value) : n->value;
            found = true;
            return;
        }
        int rmid = r1 + (r2 - r1) / 2, cmid = c1 + (c2 - c1) / 2;
        query(n->child[0], r1, c1, rmid, cmid);
        query(n->child[1], rmid + 1, c1, r2, cmid);
        query(n->child[2], r1, cmid + 1, rmid, c2);
        query(n->child[3], rmid + 1, cmid + 1, r2, c2);
    }

    // Helper variables for update().
    int tgt_r, tgt_c;
    T delta;

    void update(node_t *&n, int r1, int c1, int r2, int c2) {
        if (n == NULL) {
            n = new node_t(join_region(init, (r2 - r1 + 1) * (c2 - r1 + 1)));
        }
        if (tgt_r < r1 || tgt_r > r2 || tgt_c < c1 || tgt_c > c2) {
```

```

    return;
}
if (r1 == r2 && c1 == c2) {
    n->value = join_value_with_delta(n->value, delta);
    return;
}
int rmid = r1 + (r2 - r1) / 2, cmid = c1 + (c2 - c1) / 2;
update(n->child[0], r1, c1, rmid, cmid);
update(n->child[1], rmid + 1, c1, r2, cmid);
update(n->child[2], r1, cmid + 1, rmid, c2);
update(n->child[3], rmid + 1, cmid + 1, r2, c2);
bool found = false;
for (int i = 0; i < 4; i++) {
    n->value = found ? join_values(n->value, n->child[i]->value) : n->child[i]->value;
    found = true;
}
}

static void clean_up(node_t *n) {
    if (n != NULL) {
        for (int i = 0; i < 4; i++) {
            clean_up(n->child[i]);
        }
        delete n;
    }
}

public:
quadtree(const T &v = T()) : root(NULL), init(v) {}

~quadtree() { clean_up(root); }
T at(int r, int c) { return query(r, c, r, c); }

T query(int r1, int c1, int r2, int c2) {
    tgt_r1 = r1;
    tgt_c1 = c1;
    tgt_r2 = r2;
    tgt_c2 = c2;
    found = false;
    query(root, 0, 0, MAXR, MAXC);
    return found ? res : join_region(init, (r2 - r1 + 1) * (c2 - c1 + 1));
}

void update(int r, int c, const T &d) {
    tgt_r = r;
    tgt_c = c;
    delta = d;
    update(root, 0, 0, MAXR, MAXC);
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    quadtree<int> t(0);
    t.update(0, 0, 7);
    t.update(0, 1, 6);
    t.update(1, 0, 5);
    t.update(1, 1, 4);
    t.update(2, 1, 1);
    t.update(2, 2, 9);
    cout << "Values:" << endl;
}

```

```

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        cout << t.at(i, j) << " ";
    }
    cout << endl;
}
assert(t.query(0, 0, 0, 1) == 6);
assert(t.query(0, 0, 1, 0) == 5);
assert(t.query(1, 1, 2, 2) == 0);
assert(t.query(0, 0, 1000000000, 1000000000) == 0);
t.update(500000000, 500000000, -100);
assert(t.query(0, 0, 1000000000, 1000000000) == -100);
return 0;
}

```

Example Output

```

Values:
7 6 0
5 4 0
0 1 9

```

2.5.2 Quadtree (Range Update)

Maintain a two-dimensional array while supporting both dynamic queries and updates of rectangular sub-arrays via the lazy propagation technique. This implementation uses lazy initialization of nodes to conserve memory while supporting large indices.

The query operation is defined by the `join_values()` and `join_region()` functions where

`join_values(x, join_values(y, z)) = join_values(join_values(x, y), z)` for all values `x`, `y`, and `z` in the array. The `join_region(v, area)` function must be defined in conjunction to efficiently return the result of `join_values()` applied to a rectangular sub-array of `area` elements. The default code below assumes a numerical array type, defining queries for the "min" of the target range. Another possible query operation is "sum", in which case `join_values(a, b)` should return $a + b$ and `join_region(v, area)` should return $v \cdot area$.

The update operation is defined by the `join_value_with_delta()` and `join_deltas()` functions, which determines the change made to array values. These must satisfy:

- `join_deltas(d1, join_deltas(d2, d3)) = join_deltas(join_deltas(d1, d2), d3)`.
- `join_value_with_delta(join_values(v, ..., v), d, m)` should be equal to `join_values(join_value_with_delta(v, d, 1), ...)`, with m values on each side.
- if a sequence $d_1, \dots, d_m$ of deltas is used to update a value v , then `join_value_with_delta(v, join_deltas(d_1, ..., d_m), 1)` should be equivalent to m sequential calls to `join_value_with_delta(v, d_i, 1)` for $i = 1, \dots, m$. The default code below defines updates that "set" the chosen array index to a new value. Another possible update operation is "increment", in which case `join_value_with_delta(v, d, area)` should be defined to return $v + d \cdot area$ and `join_deltas(d1, d2)` should be defined to return $d1 + d2$.
- `quadtree(v)` constructs a two-dimensional array with rows from 0 to `MAXR` and columns from 0 to `MAXC`, inclusive. All values are implicitly initialized to `v`.
- `at(r, c)` returns the value at row `r`, column `c`.
- `query(r1, c1, r2, c2)` returns the result of `join_values()` applied to every value in the rectangular region consisting of rows from `r1` to `r2` and columns from `c1` to `c2`, inclusive.
- `update(r, c, d)` assigns the value `v` at `(r, c)` to `join_value_with_delta(v, d)`.
- `update(r1, c1, r2, c2)` modifies the value at each index of the rectangular region consisting of rows from `r1` to `r2` and columns from `c1` to `c2`, inclusive, by respectively joining them with `d` using `join_value_with_delta()`.

Time Complexity:

- $O(1)$ per call to the constructor.
- $O(\max(\text{MAXR}, \text{MAXC}))$ per call to `at()`, `update()`, and `query()`.

Space Complexity:

- $O(n)$ for storage of the array elements, where n is the number of updated entries in the array.
- $O(\sqrt{\max(\text{MAXR}, \text{MAXC})})$ auxiliary stack space for `update()`, `query()`, and `at()`.

```
#include <algorithm>
#include <cstdint>

template<class T>
class quadtree {
    static const int MAXR = 1000000000;
    static const int MAXC = 1000000000;

    static T join_values(const T &a, const T &b) { return std::min(a, b); }
    static T join_region(const T &v, int area) { return v; }
    static T join_value_with_delta(const T &v, const T &d, int area) { return d; }

    static T join_deltas(const T &d1, const T &d2) {
        return d2; // For "set" updates, the more recent delta prevails.
    }

    struct node_t {
        T value, delta;
        bool pending;
        node_t *child[4];

        node_t(const T &v) : value(v), pending(false) {
            for (int i = 0; i < 4; i++) {
                child[i] = NULL;
            }
        }
    } *root;

    T init;

    void update_delta(node_t *&n, const T &d, int area) {
        if (n == NULL) {
            n = new node_t(join_region(init, area));
        }
        n->delta = n->pending ? join_deltas(n->delta, d) : d;
        n->pending = true;
    }

    void push_delta(node_t *n, int r1, int c1, int r2, int c2) {
        if (n->pending) {
            int rmid = r1 + (r2 - r1) / 2, cmid = c1 + (c2 - c1) / 2;
            int rlen = r2 - r1 + 1, clen = c2 - c1 + 1;
            n->value = join_value_with_delta(n->value, n->delta, rlen * clen);
            if (rlen * clen > 1) {
                int rlen1 = rmid - r1 + 1, rlen2 = rlen - rlen1;
                int clen1 = cmid - c1 + 1, clen2 = clen - clen1;
                update_delta(n->child[0], n->delta, rlen1 * clen1);
                update_delta(n->child[1], n->delta, rlen2 * clen1);
                update_delta(n->child[2], n->delta, rlen1 * clen2);
                update_delta(n->child[3], n->delta, rlen2 * clen2);
            }
            n->pending = false;
        }
    }

    // Helper variables for query() and update().
    int tgt_r1, tgt_c1, tgt_r2, tgt_c2;
    T res, delta;
    bool found;
};
```

```

void query(node_t *n, int r1, int c1, int r2, int c2) {
    if (tgt_r2 < r1 || tgt_r1 > r2 || tgt_c2 < c1 || tgt_c1 > c2) {
        return;
    }
    if (n == NULL) {
        int rlen = std::min(r2, tgt_r2) - std::max(r1, tgt_r1) + 1;
        int clen = std::min(c2, tgt_c2) - std::max(c1, tgt_c1) + 1;
        T v = join_region(init, rlen * clen);
        res = found ? join_values(res, v) : v;
        found = true;
        return;
    }
    push_delta(n, r1, c1, r2, c2);
    if (tgt_r1 <= r1 && r2 <= tgt_r2 && tgt_c1 <= c1 && c2 <= tgt_c2) {
        res = found ? join_values(res, n->value) : n->value;
        found = true;
        return;
    }
    int rmid = r1 + (r2 - r1) / 2, cmid = c1 + (c2 - c1) / 2;
    query(n->child[0], r1, c1, rmid, cmid);
    query(n->child[1], rmid + 1, c1, r2, cmid);
    query(n->child[2], r1, cmid + 1, rmid, c2);
    query(n->child[3], rmid + 1, cmid + 1, r2, c2);
}

void update(node_t *&n, int r1, int c1, int r2, int c2) {
    if (n == NULL) {
        n = new node_t(join_region(init, (r2 - r1 + 1) * (c2 - r1 + 1)));
    }
    if (tgt_r2 < r1 || tgt_r1 > r2 || tgt_c2 < c1 || tgt_c1 > c2) {
        return;
    }
    push_delta(n, r1, c1, r2, c2);
    if (tgt_r1 <= r1 && r2 <= tgt_r2 && tgt_c1 <= c1 && c2 <= tgt_c2) {
        n->delta = delta;
        n->pending = true;
        push_delta(n, r1, c1, r2, c2);
        return;
    }
    int rmid = r1 + (r2 - r1) / 2, cmid = c1 + (c2 - c1) / 2;
    update(n->child[0], r1, c1, rmid, cmid);
    update(n->child[1], rmid + 1, c1, r2, cmid);
    update(n->child[2], r1, cmid + 1, rmid, c2);
    update(n->child[3], rmid + 1, cmid + 1, r2, c2);
    n->value = n->child[0]->value;
    for (int i = 1; i < 4; i++) {
        n->value = join_values(n->value, n->child[i]->value);
    }
}

static void clean_up(node_t *n) {
    if (n != NULL) {
        for (int i = 0; i < 4; i++) {
            clean_up(n->child[i]);
        }
        delete n;
    }
}

public:
quadtree(const T &v = T()) : root(NULL), init(v) {}

~quadtree() { clean_up(root); }
T at(int r, int c) { return query(r, c, r, c); }

T query(int r1, int c1, int r2, int c2) {
    tgt_r1 = r1;

```

```

    tgt_c1 = c1;
    tgt_r2 = r2;
    tgt_c2 = c2;
    found = false;
    query(root, 0, 0, MAXR, MAXC);
    return found ? res : join_region(init, (r2 - r1 + 1) * (c2 - c1 + 1));
}

void update(int r, int c, const T &d) { update(r, c, r, c, d); }

void update(int r1, int c1, int r2, int c2, const T &d) {
    tgt_r1 = r1;
    tgt_c1 = c1;
    tgt_r2 = r2;
    tgt_c2 = c2;
    delta = d;
    update(root, 0, 0, MAXR, MAXC);
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    quadtree<int> t(0);
    t.update(0, 0, 7);
    t.update(0, 1, 6);
    t.update(1, 0, 5);
    t.update(1, 1, 4);
    t.update(2, 1, 1);
    t.update(2, 2, 9);
    cout << "Values:" << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << t.at(i, j) << " ";
        }
        cout << endl;
    }
    assert(t.query(0, 0, 0, 1) == 6);
    assert(t.query(0, 0, 1, 0) == 5);
    assert(t.query(1, 1, 2, 2) == 0);
    assert(t.query(0, 0, 1000000000, 1000000000) == 0);
    t.update(500000000, 500000000, -100);
    assert(t.query(0, 0, 1000000000, 1000000000) == -100);
    return 0;
}

```

Example Output

```

Values:
7 6 0
5 4 0
0 1 9

```

2.5.3 2D Segment Tree

Maintain a two-dimensional array while supporting dynamic queries of rectangular sub-arrays and dynamic updates of individual indices. This implementation uses lazy initialization of nodes to conserve memory while supporting large indices.

The query operation is defined by the `join_values()` and `join_region()` functions where

`join_values(x, join_values(y, z)) = join_values(join_values(x, y), z)` for all values `x`, `y`, and `z` in the array. The `join_region(v, area)` function must be defined in conjunction to efficiently return the result of `join_values()` applied to a rectangular sub-array of `area` elements. The default code below assumes a numerical array type, defining queries for the "min" of the target range. Another possible query operation is "sum", in which case `join_values(a, b)` should return $a + b$ and `join_region(v, area)` should return $v \cdot area$.

The update operation is defined by the `join_value_with_delta()` function, which determines the change made to array values. The default code below defines updates that "set" the chosen array index to a new value. Another possible update operation is "increment", in which `join_value_with_delta(v, d)` should be defined to return $v + d$.

- `segment_tree_2d(v)` constructs a two-dimensional array with rows from 0 to `MAXR` and columns from 0 to `MAXC`, inclusive. All values are implicitly initialized to `v`.
- `at(r, c)` returns the value at row `r`, column `c`.
- `query(r1, c1, r2, c2)` returns the result of `join_values()` applied to every value in the rectangular region consisting of rows from `r1` to `r2`, inclusive, and columns from `c1` to `c2`, inclusive.
- `update(r, c, d)` assigns the value `v` at `(r, c)` to `join_value_with_delta(v, d)`.

Time Complexity:

- $O(1)$ per call to the constructor.
- $O(\log(\text{MAXR}) \cdot \log(\text{MAXC}))$ per call to `at()`, `update()`, and `query()`.

Space Complexity:

- $O(n)$ for storage of the array elements, where n is the number of updated entries in the array.
- $O(\log(\text{MAXR}) + \log(\text{MAXC}))$ auxiliary stack space for `update()`, `query()`, and `at()`.

```
#include <algorithm>
#include <cstdlib>

template<class T>
class segment_tree_2d {
    static const int MAXR = 1000000000;
    static const int MAXC = 1000000000;

    static T join_values(const T &a, const T &b) { return std::min(a, b); }
    static T join_region(const T &v, int area) { return v; }
    static T join_value_with_delta(const T &v, const T &d) { return d; }

    struct inner_node_t {
        T value;
        int low, high;
        inner_node_t *left, *right;

        inner_node_t(int lo, int hi, const T &v)
            : value(v), low(lo), high(hi), left(NULL), right(NULL) {}
    };

    struct outer_node_t {
        inner_node_t root;
        int low, high;
        outer_node_t *left, *right;

        outer_node_t(int lo, int hi, const T &v)
            : root(0, MAXC, v), low(lo), high(hi), left(NULL), right(NULL) {}
    } *root;

    T init;

    // Helper variables for query() and update().
    int tgt_r1, tgt_c1, tgt_r2, tgt_c2, width;

    template<class node_t>
    inline T call_query(node_t *n, int area) {
```

```

    return (n != NULL) ? query(n) : join_region(init, area);
}

T query(inner_node_t *n) {
    int lo = n->low, hi = n->high, mid = lo + (hi - lo) / 2;
    if (tgt_c1 <= lo && hi <= tgt_c2) {
        T res = n->value;
        if (tgt_c1 < lo) {
            res = join_values(res, join_region(init, lo - tgt_c1 + 1));
        }
        if (hi < tgt_c2) {
            res = join_values(res, join_region(init, tgt_c2 - hi + 1));
        }
        return res;
    } else if (tgt_c2 <= mid) {
        return call_query(n->left, tgt_c2 - tgt_c1 + 1);
    } else if (mid < tgt_c1) {
        return call_query(n->right, tgt_c2 - tgt_c1 + 1);
    }
    return join_values(
        call_query(n->left, std::min(tgt_c2, mid) - tgt_c1 + 1),
        call_query(n->right, tgt_c2 - std::max(tgt_c1, mid + 1) + 1)
    );
}

T query(outer_node_t *n) {
    int lo = n->low, hi = n->high, mid = lo + (hi - lo) / 2;
    if (tgt_r1 <= lo && hi <= tgt_r2) {
        T res = query(&(n->root));
        if (tgt_r1 < lo) {
            res = join_values(res, join_region(init, width * (lo - tgt_r1 + 1)));
        }
        if (hi < tgt_r2) {
            res = join_values(res, join_region(init, width * (tgt_r2 - hi + 1)));
        }
        return res;
    } else if (tgt_r2 <= mid) {
        return call_query(n->left, tgt_r2 - tgt_r1 + 1);
    } else if (mid < tgt_r1) {
        return call_query(n->right, tgt_r2 - tgt_r1 + 1);
    }
    return join_values(
        call_query(n->left, width * (std::min(tgt_r2, mid) - tgt_r1 + 1)),
        call_query(n->right, width * (tgt_r2 - std::max(tgt_r1, mid + 1) + 1))
    );
}

void update(inner_node_t *n, int c, const T &d, bool leaf_row) {
    int lo = n->low, hi = n->high, mid = lo + (hi - lo) / 2;
    if (lo == hi) {
        if (leaf_row) {
            n->value = join_value_with_delta(n->value, d);
        } else {
            n->value = d;
        }
        return;
    }
    inner_node_t *target = (c <= mid) ? n->left : n->right;
    if (target == NULL) {
        target = new inner_node_t(c, c, init);
    }
    if (target->low <= c && c <= target->high) {
        update(target, c, d, leaf_row);
    } else {
        do {
            if (c <= mid) {
                hi = mid;
            } else {

```



```

        lo = mid + 1;
    }
    mid = lo + (hi - lo) / 2;
} while ((c <= mid) == (target->low <= mid));
inner_node_t *tmp = new inner_node_t(lo, hi, init);
if (target->low <= mid) {
    tmp->left = target;
} else {
    tmp->right = target;
}
target = tmp;
update(tmp, c, d, leaf_row);
}
T left_value = (n->left != NULL) ? n->left->value : join_region(init, mid - lo + 1);
T right_value = (n->right != NULL) ? n->right->value : join_region(init, hi - mid);
n->value = join_values(left_value, right_value);
}

void update(outer_node_t *n, int r, int c, const T &d) {
    int lo = n->low, hi = n->high, mid = lo + (hi - lo) / 2;
    if (lo == hi) {
        update(&(n->root), c, d, true);
        return;
    }
    if (r <= mid) {
        if (n->left == NULL) {
            n->left = new outer_node_t(lo, mid, init);
        }
        update(n->left, r, c, d);
    } else {
        if (n->right == NULL) {
            n->right = new outer_node_t(mid + 1, hi, init);
        }
        update(n->right, r, c, d);
    }
    T value = join_region(init, hi - lo + 1);
    if (n->left != NULL || n->right != NULL) {
        tgt_c1 = tgt_c2 = c;
        T left_value = (n->left != NULL) ? query(&(n->left->root)) : join_region(init, mid - lo + 1);
        T right_value = (n->right != NULL) ? query(&(n->right->root)) : join_region(init, hi - mid);
        value = join_values(left_value, right_value);
    }
    update(&(n->root), c, value, false);
}

static void clean_up(inner_node_t *n) {
    if (n != NULL) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

static void clean_up(outer_node_t *n) {
    if (n != NULL) {
        clean_up(n->root.left);
        clean_up(n->root.right);
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
segment_tree_2d(const T &v = T()) : root(new outer_node_t(0, MAXR, v)), init(v) {}

~segment_tree_2d() { clean_up(root); }

```

```

T at(int r, int c) { return query(r, c, r, c); }

T query(int r1, int c1, int r2, int c2) {
    tgt_r1 = r1;
    tgt_c1 = c1;
    tgt_r2 = r2;
    tgt_c2 = c2;
    width = c2 - c1;
    return query(root);
}

void update(int r, int c, const T &d) { update(root, r, c, d); }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    segment_tree_2d<int> t(0);
    t.update(0, 0, 7);
    t.update(0, 1, 6);
    t.update(1, 0, 5);
    t.update(1, 1, 4);
    t.update(2, 1, 1);
    t.update(2, 2, 9);
    cout << "Values:" << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << t.at(i, j) << " ";
        }
        cout << endl;
    }
    assert(t.query(0, 0, 0, 1) == 6);
    assert(t.query(0, 0, 1, 0) == 5);
    assert(t.query(1, 1, 2, 2) == 0);
    assert(t.query(0, 0, 1000000000, 1000000000) == 0);
    t.update(5000000000, 5000000000, -100);
    assert(t.query(0, 0, 10000000000, 10000000000) == -100);
    return 0;
}

```

Example Output

```

Values:
7 6 0
5 4 0
0 1 9

```

2.5.4 2D Range Tree

Maintain a set of two-dimensional points while supporting queries for all points that fall inside given rectangular regions. This implementation uses `std::pair` to represent points, requiring operators `<` and `==` to be defined on the numeric template type.

- `range_tree(lo, hi)` constructs a set from two random-access iterators to `std::pair` as a range `[lo, hi)` of points.
- `query(x1, y1, x2, y2, f)` calls the function `f(i, p)` on each point in the set that falls into the rectangular region consisting of rows from `x1` to `x2`, inclusive, and columns from `y1` to `y2`, inclusive. The first argument to `f` is the zero-based index of the point in the original range given to the constructor. The

second argument is the point itself as an `std::pair`.

Time Complexity:

- $O(n \log n)$ per call to the constructor, where n is the number of points.
- $O(\log^2(n) + m)$ per call to `query()`, where m is the number of points that are reported by the query.

Space Complexity:

- $O(n \log n)$ for storage of the points.
- $O(\log^2(n))$ auxiliary stack space for `query()`.

```
#include <algorithm>
#include <iterator>
#include <utility>
#include <vector>

template<class T>
class range_tree {
    typedef std::pair<T, T> point;
    typedef std::pair<int, T> colindex;

    std::vector<point> points;
    std::vector<std::vector<colindex>> columns;

    static inline bool comp1(const colindex &a, const colindex &b) { return a.second < b.second; }
    static inline bool comp2(const colindex &a, const T &v) { return a.second < v; }

    void build(int n, int lo, int hi) {
        if (points[lo].first == points[hi].first) {
            for (int i = lo; i <= hi; i++) {
                columns[n].push_back(point(i, points[i].second));
            }
            return;
        }
        int l = n * 2 + 1, r = n * 2 + 2, mid = lo + (hi - lo) / 2;
        build(l, lo, mid);
        build(r, mid + 1, hi);
        columns[n].resize(columns[l].size() + columns[r].size());
        std::merge(
            columns[l].begin(), columns[l].end(), columns[r].begin(), columns[r].end(),
            columns[n].begin(), comp1
        );
    }

    // Helper variables for query().
    T x1, y1, x2, y2;

    template<class ReportFunction>
    void query(int n, int lo, int hi, ReportFunction f) {
        if (points[hi].first < x1 || x2 < points[lo].first) {
            return;
        }
        if (!(points[lo].first < x1 || x2 < points[hi].first)) {
            if (!columns[n].empty() && !(y2 < y1)) {
                typename std::vector<point>::iterator it;
                it = std::lower_bound(columns[n].begin(), columns[n].end(), y1, comp2);
                for (; it != columns[n].end() && it->second <= y2; ++it) {
                    f(it->first, points[it->first]);
                }
            }
        }
        else if (lo != hi) {
            int mid = lo + (hi - lo) / 2;
            query(n * 2 + 1, lo, mid, f);
            query(n * 2 + 2, mid + 1, hi, f);
        }
    }
};
```

```

public:
    template<class It>
    range_tree(It lo, It hi) : points(lo, hi) {
        int n = std::distance(lo, hi);
        columns.resize(4 * n + 1);
        std::sort(points.begin(), points.end());
        build(0, 0, n - 1);
    }

    template<class ReportFunction>
    void query(const T &x1, const T &y1, const T &x2, const T &y2, ReportFunction f) {
        this->x1 = x1;
        this->y1 = y1;
        this->x2 = x2;
        this->y2 = y2;
        query(0, 0, points.size() - 1, f);
    }
};

```

Example Usage

```

#include <iostream>
using namespace std;

void print(int i, const pair<int, int> &p) {
    cout << "(" << p.first << ", " << p.second << ") ";
}

int main() {
    const int n = 10;
    int points[n][2] = {{1, 4}, {5, 4}, {2, 2}, {3, 1}, {6, -5},
                       {5, -1}, {3, -3}, {-1, -2}, {-1, -1}, {2, -1}};
    vector<pair<int, int>> v;
    for (int i = 0; i < n; i++) {
        v.push_back(make_pair(points[i][0], points[i][1]));
    }
    range_tree<int> t(v.begin(), v.end());
    t.query(-1, -1, 2, 5, print);
    cout << endl;
    t.query(1, 1, 4, 8, print);
    cout << endl;
    return 0;
}

```

Example Output

```

(-1, -1) (2, -1) (2, 2) (1, 4)
(1, 4) (2, 2) (3, 1)

```

2.5.5 K-d Tree (2D Range Query)

Maintain a set of two-dimensional points while supporting queries for all points that fall inside given rectangular regions. This implementation uses `std::pair` to represent points, requiring operators `<` and `==` to be defined on the numeric template type.

- `kd_tree(lo, hi)` constructs a set from two random-access iterators to `std::pair` as a range `[lo, hi)` of points.
- `query(x1, y1, x2, y2, f)` calls the function `f(i, p)` on each point in the set that falls into the rectangular region consisting of rows from `x1` to `x2`, inclusive, and columns from `y1` to `y2`, inclusive. The first argument to `f` is the zero-based index of the point in the original range given to the constructor. The second argument is the point itself as an `std::pair`.

Time Complexity:

- $O(n \log n)$ per call to the constructor, where n is the number of points.
- $O(\log(n) + m)$ on average per call to `query()`, where m is the number of points that are reported by the query.

Space Complexity:

- $O(n)$ for storage of the points.
- $O(\log n)$ auxiliary stack space for `query()`.

```
#include <algorithm>
#include <utility>
#include <vector>

template<class T>
class kd_tree {
    typedef std::pair<T, T> point;

    static inline bool comp1(const point &a, const point &b) { return a.first < b.first; }
    static inline bool comp2(const point &a, const point &b) { return a.second < b.second; }

    std::vector<point> tree, minp, maxp;
    std::vector<int> l_index, h_index;

    void build(int lo, int hi, bool div_x) {
        if (lo >= hi) {
            return;
        }
        int mid = lo + (hi - lo) / 2;
        std::nth_element(
            tree.begin() + lo, tree.begin() + mid, tree.begin() + hi, div_x ? comp1 : comp2
        );
        l_index[mid] = lo;
        h_index[mid] = hi;
        minp[mid].first = maxp[mid].first = tree[lo].first;
        minp[mid].second = maxp[mid].second = tree[lo].second;
        for (int i = lo + 1; i < hi; i++) {
            minp[mid].first = std::min(minp[mid].first, tree[i].first);
            minp[mid].second = std::min(minp[mid].second, tree[i].second);
            maxp[mid].first = std::max(maxp[mid].first, tree[i].first);
            maxp[mid].second = std::max(maxp[mid].second, tree[i].second);
        }
        build(lo, mid, !div_x);
        build(mid + 1, hi, !div_x);
    }

    // Helper variables for query().
    T x1, y1, x2, y2;

    template<class ReportFunction>
    void query(int lo, int hi, ReportFunction f) {
        if (lo >= hi) {
            return;
        }
        int mid = lo + (hi - lo) / 2;
        T ax = minp[mid].first, ay = minp[mid].second;
        T bx = maxp[mid].first, by = maxp[mid].second;
        if (x2 < ax || bx < x1 || y2 < ay || by < y1) {
            return;
        }
        if (!(ax < x1 || x2 < bx || ay < y1 || y2 < by)) {
            for (int i = l_index[mid]; i < h_index[mid]; i++) {
                f(tree[i]);
            }
            return;
        }
        query(lo, mid, f);
        query(mid + 1, hi, f);
    }
};
```

```

    if (tree[mid].first < x1 || x2 < tree[mid].first || tree[mid].second < y1 ||
        y2 < tree[mid].second) {
        return;
    }
    f(tree[mid]);
}

public:
    template<class It>
    kd_tree(It lo, It hi) : tree(lo, hi) {
        int n = std::distance(lo, hi);
        l_index.resize(n);
        h_index.resize(n);
        minp.resize(n);
        maxp.resize(n);
        build(0, n, true);
    }

    template<class ReportFunction>
    void query(const T &x1, const T &y1, const T &x2, const T &y2, ReportFunction f) {
        this->x1 = x1;
        this->y1 = y1;
        this->x2 = x2;
        this->y2 = y2;
        query(0, tree.size(), f);
    }
};

```

Example Usage

```

#include <iostream>
using namespace std;

void print(const pair<int, int> &p) {
    cout << "(" << p.first << ", " << p.second << ")" << " ";
}

int main() {
    const int n = 10;
    int points[n][2] = {{1, 4}, {5, 4}, {2, 2}, {3, 1}, {6, -5},
                        {5, -1}, {3, -3}, {-1, -2}, {-1, -1}, {2, -1}};
    vector<pair<int, int>> v;
    for (int i = 0; i < n; i++) {
        v.push_back(make_pair(points[i][0], points[i][1]));
    }
    kd_tree<int> t(v.begin(), v.end());
    t.query(-1, -1, 2, 5, print);
    cout << endl;
    t.query(1, 1, 4, 8, print);
    cout << endl;
    return 0;
}

```

Example Output

```

(2, -1) (1, 4) (2, 2) (-1, -1)
(1, 4) (2, 2) (3, 1)

```

2.5.6 K-d Tree (Nearest Neighbor)

Maintain a set of two-dimensional points while supporting queries for the closest point in the set to a given query point. This implementation uses `std::pair` to represent points, requiring operators `<`, `==`, `-`, and `long double` casting to be defined on the numeric template type.

- `kd_tree(lo, hi)` constructs a set from two random-access iterators to `std::pair` as a range `[lo, hi)` of points.
- `nearest(x, y, can_equal)` returns a point in the set that is closest to `(x, y)` by Euclidean distance. This may be equal to `(x, y)` only if `can_equal` is `true`.

Time Complexity:

- $O(n \log n)$ per call to the constructor, where n is the number of points.
- $O(\log n)$ on average per call to `nearest()`.

Space Complexity:

- $O(n)$ for storage of the points.
- $O(\log n)$ auxiliary stack space for `nearest()`.

```
#include <algorithm>
#include <limits>
#include <stdexcept>
#include <utility>
#include <vector>

template<class T>
class kd_tree {
    typedef std::pair<T, T> point;

    static inline bool comp1(const point &a, const point &b) { return a.first < b.first; }
    static inline bool comp2(const point &a, const point &b) { return a.second < b.second; }

    std::vector<point> tree;
    std::vector<bool> div_x;

    void build(int lo, int hi) {
        if (lo >= hi) {
            return;
        }
        int mid = lo + (hi - lo) / 2;
        T minx, maxx, miny, maxy;
        minx = maxx = tree[lo].first;
        miny = maxy = tree[lo].second;
        for (int i = lo + 1; i < hi; i++) {
            minx = std::min(minx, tree[i].first);
            miny = std::min(miny, tree[i].second);
            maxx = std::max(maxx, tree[i].first);
            maxy = std::max(maxy, tree[i].second);
        }
        div_x[mid] = !((maxx - minx) < (maxy - miny));
        std::nth_element(
            tree.begin() + lo, tree.begin() + mid, tree.begin() + hi, div_x[mid] ? comp1 : comp2
        );
        if (lo + 1 == hi) {
            return;
        }
        build(lo, mid);
        build(mid + 1, hi);
    }

    // Helper variables for nearest().
    long double min_dist;
    int id;

    void nearest(int lo, int hi, const T &x, const T &y, bool can_equal) {
        if (lo >= hi) {
            return;
        }
        int mid = lo + (hi - lo) / 2;
        T dx = x - tree[mid].first, dy = y - tree[mid].second;
        long double d = dx * (long double)dx + dy * (long double)dy;
        if (d < min_dist && (can_equal || d != 0)) {
```

```

        min_dist = d;
        id = mid;
    }
    if (lo + 1 == hi) {
        return;
    }
    d = (long double)(div_x[mid] ? dx : dy);
    int l1 = lo, r1 = mid, l2 = mid + 1, r2 = hi;
    if (d > 0) {
        std::swap(l1, l2);
        std::swap(r1, r2);
    }
    nearest(l1, r1, x, y, can_equal);
    if (d * (long double)d < min_dist) {
        nearest(l2, r2, x, y, can_equal);
    }
}

public:
    template<class It>
    kd_tree(It lo, It hi) : tree(lo, hi) {
        int n = std::distance(lo, hi);
        if (n <= 1) {
            throw std::runtime_error("K-d tree must be have at least 2 points.");
        }
        div_x.resize(n);
        build(0, n);
    }

    point nearest(const T &x, const T &y, bool can_equal = true) {
        min_dist = std::numeric_limits<long double>::max();
        nearest(0, tree.size(), x, y, can_equal);
        return tree[id];
    }
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    pair<int, int> p[3];
    p[0] = make_pair(0, 2);
    p[1] = make_pair(0, 3);
    p[2] = make_pair(-1, 0);
    kd_tree<int> t(p, p + 3);
    assert(t.nearest(0, 2, true) == make_pair(0, 2));
    assert(t.nearest(0, 2, false) == make_pair(0, 3));
    assert(t.nearest(0, 0) == make_pair(-1, 0));
    assert(t.nearest(-10000, 0) == make_pair(-1, 0));
    return 0;
}

```

2.6 Fenwick Trees

2.6.1 Fenwick Tree (Simple)

Maintain an array of numerical type, allowing for updates of individual indices (point update) and queries for the sum of contiguous sub-arrays (range queries). This implementation assumes that the array is 1-based (i.e. has

valid indices from 1 to `MAXN - 1`, inclusive).

- `initialize()` resets the data structure.
- `a[i]` stores the value at index `i`.
- `add(i, x)` adds `x` to the value at index `i`.
- `set(i, x)` assigns the value at index `i` to `x`.
- `sum(hi)` returns the sum of all values at indices from 1 to `hi`, inclusive.
- `sum(lo, hi)` returns the sum of all values at indices from `lo` to `hi`, inclusive.

Time Complexity:

- $O(n)$ per call to `initialize()`, where n is the size of the array.
- $O(\log n)$ per call to all other operations.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(1)$ auxiliary for all operations.

```
const int MAXN = 1000;
int a[MAXN + 1], t[MAXN + 1];

void initialize() {
    for (int i = 0; i <= MAXN; i++) {
        a[i] = t[i] = 0;
    }
}

void add(int i, int x) {
    a[i] += x;
    for (; i <= MAXN; i += i & -i) {
        t[i] += x;
    }
}

void set(int i, int x) {
    add(i, x - a[i]);
}

int sum(int hi) {
    int res = 0;
    for (; hi > 0; hi -= hi & -hi) {
        res += t[hi];
    }
    return res;
}

int sum(int lo, int hi) {
    return sum(hi) - sum(lo - 1);
}
```

Example Usage

```
#include <cassert>
#include <iostream>
using namespace std;

int main() {
    int v[] = {10, 1, 2, 3, 4};
    initialize();
    for (int i = 1; i <= 5; i++) {
        set(i, v[i - 1]);
    }
    add(1, -5);
    cout << "Values: ";
```

```

for (int i = 1; i <= 5; i++) {
    cout << a[i] << " ";
}
cout << endl;
assert(sum(2, 4) == 6);
return 0;
}

```

Example Output

Values: 5 1 2 3 4

2.6.2 Fenwick Tree (Range Update, Point Query)

Maintain an array of numerical type, allowing for contiguous sub-arrays to be simultaneously incremented by arbitrary values (range update) and values at individual indices to be queried (point query). This implementation assumes that the array is 0-based (i.e. has valid indices from 0 to `size() - 1`, inclusive).

- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `add(i, x)` adds `x` to the value at index `i`.
- `add(lo, hi, x)` adds `x` to the values at all indices from `lo` to `hi`, inclusive.

Time Complexity:

- $O(n)$ per call to the constructor, where n is the size of the array.
- $O(1)$ per call to `size()`.
- $O(\log n)$ per call to `at()` and both `add()` functions.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(1)$ auxiliary for all operations.

```

#include <vector>

template<class T>
class fenwick_tree {
    int len;
    std::vector<int> t;

public:
    fenwick_tree(int n) : len(n), t(n + 2) {}

    int size() const { return len; }

    T at(int i) const {
        T res = 0;
        for (i++; i > 0; i -= i & -i) {
            res += t[i];
        }
        return res;
    }

    void add(int i, const T &x) {
        for (i++; i <= len + 1; i += i & -i) {
            t[i] += x;
        }
    }

    void add(int lo, int hi, const T &x) {
        add(lo, x);
        add(hi + 1, -x);
    }
}

```

```
};
```

Example Usage

```
#include <iostream>
using namespace std;

int main() {
    fenwick_tree<int> t(5);
    t.add(0, 1, 5);
    t.add(1, 2, 5);
    t.add(2, 4, 10);
    cout << "Values: ";
    for (int i = 0; i < t.size(); i++) {
        cout << t.at(i) << " ";
    }
    cout << endl;
    return 0;
}
```

Example Output

```
Values: 5 10 15 10 10
```

2.6.3 Fenwick Tree (Point Update, Range Query)

Maintain an array of numerical type, allowing for updates of individual indices (point update) and queries for the sum of contiguous sub-arrays (range queries). This implementation assumes that the array is 0-based (i.e. has valid indices from 0 to `size() - 1`, inclusive).

- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `add(i, x)` adds `x` to the value at index `i`.
- `set(i, x)` assigns the value at index `i` to `x`.
- `sum(hi)` returns the sum of all values at indices from 0 to `hi`, inclusive.
- `sum(lo, hi)` returns the sum of all values at indices from `lo` to `hi`, inclusive.

Time Complexity:

- $O(n)$ per call to the constructor, where n is the size of the array.
- $O(1)$ per call to `size()` and `at()`.
- $O(\log n)$ per call to `add()`, `set()`, and both `sum()` functions.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(1)$ auxiliary for all operations.

```
#include <vector>

template<class T>
class fenwick_tree {
    int len;
    std::vector<int> a, t;

public:
    fenwick_tree(int n) : len(n), a(n + 1), t(n + 1) {}

    int size() const { return len; }
    T at(int i) const { return a[i + 1]; }
```

```

void add(int i, const T &x) {
    a[++i] += x;
    for (; i <= len; i += i & -i) {
        t[i] += x;
    }
}

void set(int i, const T &x) {
    T inc = x - a[i + 1];
    add(i, inc);
}

T sum(int hi) {
    T res = 0;
    for (hi++; hi > 0; hi -= hi & -hi) {
        res += t[hi];
    }
    return res;
}

T sum(int lo, int hi) { return sum(hi) - sum(lo - 1); }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    int a[] = {10, 1, 2, 3, 4};
    fenwick_tree<int> t(5);
    for (int i = 0; i < 5; i++) {
        t.set(i, a[i]);
    }
    t.add(0, -5);
    cout << "Values: ";
    for (int i = 0; i < t.size(); i++) {
        cout << t.at(i) << " ";
    }
    cout << endl;
    assert(t.sum(1, 3) == 6);
    return 0;
}

```

Example Output

Values: 5 1 2 3 4

2.6.4 Fenwick Tree (Range Update, Range Query)

Maintain an array of numerical type, allowing for contiguous sub-arrays to be simultaneously incremented by arbitrary values (range update) and queries for the sum of contiguous sub-arrays (range query). This implementation assumes that the array is 0-based (i.e. has valid indices from 0 to `size() - 1`, inclusive).

- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `add(i, x)` increments the value at index `i` by `x`.
- `add(lo, hi, x)` adds `x` to the values at all indices from `lo` to `hi`, inclusive.
- `set(i, x)` assigns the value at index `i` to `x`.
- `sum(hi)` returns the sum of all values at indices from 0 to `hi`, inclusive.
- `sum(lo, hi)` returns the sum of all values at indices from `lo` to `hi`, inclusive.

Time Complexity:

- $O(n)$ per call to the constructor, where n is the size of the array.
- $O(1)$ per call to `size()`.
- $O(\log n)$ per call to all other operations.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(1)$ auxiliary for all operations.

```
#include <vector>

template<class T>
class fenwick_tree {
    int len;
    std::vector<T> t1, t2;

    T sum(const std::vector<T> &t, int i) {
        T res = 0;
        for (; i != 0; i -= i & -i) {
            res += t[i];
        }
        return res;
    }

    void add(std::vector<T> &t, int i, const T &x) {
        for (; i <= len + 1; i += i & -i) {
            t[i] += x;
        }
    }

public:
    fenwick_tree(int n) : len(n), t1(n + 2), t2(n + 2) {}

    int size() const { return len; }

    void add(int lo, int hi, const T &x) {
        lo++;
        hi++;
        add(t1, lo, x);
        add(t1, hi + 1, -x);
        add(t2, lo, x * (lo - 1));
        add(t2, hi + 1, -x * hi);
    }

    void add(int i, const T &x) { return add(i, i, x); }
    void set(int i, const T &x) { add(i, x - at(i)); }

    T sum(int hi) {
        hi++;
        return hi * sum(t1, hi) - sum(t2, hi);
    }

    T sum(int lo, int hi) { return sum(hi) - sum(lo - 1); }
    T at(int i) { return sum(i, i); }
};
```

Example Usage

```
#include <cassert>
#include <iostream>
using namespace std;

int main() {
    int a[] = {10, 1, 2, 3, 4};
```

```

fenwick_tree<int> t(5);
for (int i = 0; i < t.size(); i++) {
    t.set(i, a[i]);
}
t.add(0, 2, 5);
t.set(3, -5);
cout << "Values: ";
for (int i = 0; i < t.size(); i++) {
    cout << t.at(i) << " ";
}
cout << endl;
assert(t.sum(0, 4) == 27);
return 0;
}

```

Example Output

Values: 15 6 7 -5 4

2.6.5 Fenwick Tree (Compressed)

Maintain an array of numerical type, allowing for contiguous sub-arrays to be simultaneously incremented by arbitrary values (range update) and queries for the sum of contiguous sub-arrays (range query). This implementation uses `std::map` for coordinate compression, allowing for large indices to be accessed with efficient space complexity. That is, all array indices from 0 to `MAXN`, inclusive, are accessible.

- `at(i)` returns the value at index `i`.
- `add(i, x)` adds `x` to the value at index `i`.
- `add(lo, hi, x)` adds `x` to the values at all indices from `lo` to `hi`, inclusive.
- `set(i, x)` assigns the value at index `i` to `x`.
- `sum(hi)` returns the sum of all values at indices from 0 to `hi`, inclusive.
- `sum(lo, hi)` returns the sum of all values at indices from `lo` to `hi`, inclusive.

Time Complexity: $O(\log^2 \text{MAXN})$ per call to all member functions. If `std::map` is replaced with `std::unordered_map`, then the amortized running time will become $O(\log \text{MAXN})$.

Space Complexity:

- $O(n \log \text{MAXN})$ for storage of the array elements, where n is the number of distinct indices that have been accessed across all of the operations so far.
- $O(1)$ auxiliary for all operations.

```

#include <map>

template<class T>
class fenwick_tree {
    static const int MAXN = 1000000001;
    std::map<int, T> tmul, tadd;

    void add_helper(int at, int mul, T add) {
        for (int i = at; i <= MAXN; i |= i + 1) {
            tmul[i] += mul;
            tadd[i] += add;
        }
    }

public:
    void add(int lo, int hi, const T &x) {
        add_helper(lo, x, -x * (lo - 1));
        add_helper(hi, -x, x * hi);
    }

    void add(int i, const T &x) { add(i, i, x); }
}

```

```

void set(int i, const T &x) { add(i, x - at(i)); }

T sum(int hi) {
    T mul = 0, add = 0;
    for (int i = hi; i >= 0; i = (i & (i + 1)) - 1) {
        if (tmul.find(i) != tmul.end()) {
            mul += tmul[i];
        }
        if (tadd.find(i) != tadd.end()) {
            add += tadd[i];
        }
    }
    return mul * hi + add;
}

T sum(int lo, int hi) { return sum(hi) - sum(lo - 1); }
T at(int i) { return sum(i, i); }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    int a[] = {10, 1, 2, 3, 4};
    fenwick_tree<int> t;
    for (int i = 0; i < 5; i++) {
        t.set(i, a[i]);
    }
    t.add(0, 2, 5);
    t.set(3, -5);
    cout << "Values: ";
    for (int i = 0; i < 5; i++) {
        cout << t.at(i) << " ";
    }
    cout << endl;
    assert(t.sum(0, 4) == 27);
    t.add(5000000001, 5000000010, 3);
    t.add(5000000011, 5000000015, 5);
    t.set(5000000000, 10);
    assert(t.sum(0, 1000000000) == 92);
    return 0;
}

```

Example Output

Values: 15 6 7 -5 4

2.6.6 Fenwick Tree (Order Statistic)

Maintains frequencies over a 1-based integer domain and finds order statistics by binary lifting on a Fenwick tree. This is useful for dynamic multisets of bounded integer values, coordinate-compressed kth-element queries, and online rank queries.

- `initialize()` resets all frequencies.
- `add(i, delta)` adds `delta` to the frequency of value/index `i`.
- `sum(i)` returns the total frequency over indices `[1, i]`.
- `kth(k)` returns the smallest index `i` such that `sum(i) >= k`. The argument `k` is 1-based and must satisfy `1 <= k <= sum(MAXN)`.

Time Complexity:

- $O(n)$ per call to `initialize()`.
- $O(\log n)$ per call to `add(i, delta)`, `sum(i)`, and `kth(k)`.

Space Complexity: $O(n)$ for the Fenwick tree.

```
const int MAXN = 1000;
int tree[MAXN + 1];

void initialize() {
    for (int i = 0; i <= MAXN; i++) {
        tree[i] = 0;
    }
}

void add(int i, int delta) {
    for (; i <= MAXN; i += i & -i) {
        tree[i] += delta;
    }
}

int sum(int i) {
    int res = 0;
    for (; i > 0; i -= i & -i) {
        res += tree[i];
    }
    return res;
}

int kth(int k) {
    int idx = 0;
    int bit = 1;
    while (bit * 2 <= MAXN) {
        bit *= 2;
    }
    for (; bit > 0; bit >>= 1) {
        int next = idx + bit;
        if (next <= MAXN && tree[next] < k) {
            idx = next;
            k -= tree[next];
        }
    }
    return idx + 1;
}
```

Example Usage

```
#include <cassert>

int main() {
    initialize();
    add(2, 1);
    add(4, 3);
    add(7, 1);
    assert(sum(4) == 4);
    assert(kth(1) == 2);
    assert(kth(2) == 4);
    assert(kth(4) == 4);
    assert(kth(5) == 7);
    return 0;
}
```


2.6.7 2D Fenwick Tree (Simple)

Maintain a 2D array of numerical type, allowing for updates of individual cells in the matrix (point update) and queries for the sum of rectangular sub-matrices (range query). This implementation assumes that array dimensions are 1-based (i.e. rows have valid indices from 1 to `MAXR`, inclusive, and columns have valid indices from 1 to `MAXC`, inclusive).

- `initialize()` resets the data structure.
- `a[r][c]` stores the value at index (r, c) .
- `add(r, c, x)` adds `x` to the value at index (r, c) .
- `set(r, c, x)` assigns `x` to the value at index (r, c) .
- `sum(r, c)` returns the sum of the rectangle with upper-left corner $(1, 1)$ and lower-right corner (r, c) .
- `sum(r1, c1, r2, c2)` returns the sum of the rectangle with upper-left corner $(r1, c1)$ and lower-right corner $(r2, c2)$.

Time Complexity:

- $O(n \cdot m)$ per call to `initialize()`, where n is the number of rows and m is the number of columns.
- $O(\log(n) \cdot \log(m))$ per call to all other operations.

Space Complexity:

- $O(n \cdot m)$ for storage of the array elements.
- $O(1)$ auxiliary for all operations.

```
const int MAXR = 100, MAXC = 100;
int a[MAXR + 1][MAXC + 1];
int bits[MAXR + 1][MAXC + 1];

void initialize() {
    for (int i = 0; i <= MAXR; i++) {
        for (int j = 0; j <= MAXC; j++) {
            a[i][j] = bits[i][j] = 0;
        }
    }
}

void add(int r, int c, int x) {
    a[r][c] += x;
    for (int i = r; i <= MAXR; i += i & -i) {
        for (int j = c; j <= MAXC; j += j & -j) {
            bits[i][j] += x;
        }
    }
}

void set(int r, int c, int x) {
    add(r, c, x - a[r][c]);
}

int sum(int r, int c) {
    int res = 0;
    for (int i = r; i > 0; i -= i & -i) {
        for (int j = c; j > 0; j -= j & -j) {
            res += bits[i][j];
        }
    }
    return res;
}

int sum(int r1, int c1, int r2, int c2) {
    return sum(r2, c2) + sum(r1 - 1, c1 - 1) - sum(r1 - 1, c2) - sum(r2, c1 - 1);
}
```

Example Usage

```
#include <cassert>
#include <iostream>
using namespace std;

int main() {
    initialize();
    set(1, 1, 5);
    set(1, 2, 6);
    set(2, 1, 7);
    add(3, 3, 9);
    add(2, 1, -4);
    cout << "Values:" << endl;
    for (int i = 1; i <= 3; i++) {
        for (int j = 1; j <= 3; j++) {
            cout << a[i][j] << " ";
        }
        cout << endl;
    }
    assert(sum(1, 1, 1, 2) == 11);
    assert(sum(1, 1, 2, 1) == 8);
    assert(sum(1, 1, 3, 3) == 23);
    return 0;
}
```

Example Output

```
Values:
5 6 0
3 0 0
0 0 9
```

2.6.8 2D Fenwick Tree (Compressed)

Maintain a 2D array of numerical type, allowing for rectangular sub-matrices to be simultaneously incremented by arbitrary values (range update) and queries for the sum of rectangular sub-matrices (range query). This implementation uses `std::map` for coordinate compression, allowing for large indices to be accessed with efficient space complexity. That is, rows have valid indices from 0 to `MAXR`, inclusive, and columns have valid indices from 0 to `MAXC`, inclusive.

- `add(r, c, x)` adds `x` to the value at index `(r, c)`.
- `add(r1, c1, r2, c2, x)` adds `x` to all indices in the rectangle with upper-left corner `(r1, c1)` and lower-right corner `(r2, c2)`.
- `set(r, c, x)` assigns `x` to the value at index `(r, c)`.
- `sum(r, c)` returns the sum of the rectangle with upper-left corner `(0, 0)` and lower-right corner `(r, c)`.
- `sum(r1, c1, r2, c2)` returns the sum of the rectangle with upper-left corner `(r1, c1)` and lower-right corner `(r2, c2)`.
- `at(r, c)` returns the value at index `(r, c)`.

Time Complexity: $O(\log^2(\text{MAXR}) \cdot \log^2(\text{MAXC}))$ per call to all member functions. If `std::map` is replaced with `std::unordered_map`, then the amortized running time will become $O(\log(\text{MAXR}) \cdot \log(\text{MAXC}))$.

Space Complexity:

- $O(n \cdot \log(\text{MAXR}) \cdot \log(\text{MAXC}))$ for storage of the array elements, where n is the number of distinct indices that have been accessed across all of the operations so far.
- $O(1)$ auxiliary for all operations.

```
#include <map>
#include <utility>
```

```

template<class T>
class fenwick_tree_2d {
    static const int MAXR = 1000000001;
    static const int MAXC = 1000000001;
    std::map<std::pair<int, int>, T> t1, t2, t3, t4;

    template<class Map>
    void add(Map &tree, int r, int c, const T &x) {
        for (int i = r + 1; i <= MAXR; i += i & -i) {
            for (int j = c + 1; j <= MAXC; j += j & -j) {
                tree[std::make_pair(i, j)] += x;
            }
        }
    }

    void add_helper(int r, int c, const T &x) {
        add(t1, 0, 0, x);
        add(t1, 0, c, -x);
        add(t2, 0, c, x * c);
        add(t1, r, 0, -x);
        add(t3, r, 0, x * r);
        add(t1, r, c, x);
        add(t2, r, c, -x * c);
        add(t3, r, c, -x * r);
        add(t4, r, c, x * r * c);
    }

public:
    void add(int r1, int c1, int r2, int c2, const T &x) {
        add_helper(r2 + 1, c2 + 1, x);
        add_helper(r1, c2 + 1, -x);
        add_helper(r2 + 1, c1, -x);
        add_helper(r1, c1, x);
    }

    void add(int r, int c, const T &x) { add(r, c, r, c, x); }
    void set(int r, int c, const T &x) { add(r, c, x - at(r, c)); }

    T sum(int r, int c) {
        r++;
        c++;
        T s1 = 0, s2 = 0, s3 = 0, s4 = 0;
        for (int i = r; i > 0; i -= i & -i) {
            for (int j = c; j > 0; j -= j & -j) {
                const std::pair<int, int> ij(i, j);
                s1 += t1[ij];
                s2 += t2[ij];
                s3 += t3[ij];
                s4 += t4[ij];
            }
        }
        return s1 * r * c + s2 * r + s3 * c + s4;
    }

    T sum(int r1, int c1, int r2, int c2) {
        return sum(r2, c2) + sum(r1 - 1, c1 - 1) - sum(r1 - 1, c2) - sum(r2, c1 - 1);
    }

    T at(int r, int c) { return sum(r, c, r, c); }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

```

```

int main() {
    fenwick_tree_2d<int> t;
    t.set(0, 0, 5);
    t.set(0, 1, 6);
    t.set(1, 0, 7);
    t.add(2, 2, 9);
    t.add(1, 0, -4);
    t.add(1, 1, 2, 2, 5);
    cout << "Values:" << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << t.at(i, j) << " ";
        }
        cout << endl;
    }
    assert(t.sum(0, 0, 0, 1) == 11);
    assert(t.sum(0, 0, 1, 0) == 8);
    assert(t.sum(1, 1, 2, 2) == 29);
    t.set(500000000, 500000000, 100);
    assert(t.sum(0, 0, 1000000000, 1000000000) == 143);
    return 0;
}

```

Example Output

```

Values:
5 6 0
3 5 5
0 5 14

```

2.7 Disjoint Sets and Tree Structures

2.7.1 Disjoint Set Forest (Simple)

Maintain a set of elements partitioned into non-overlapping subsets. Each partition is assigned a unique representative known as the parent, or root. The following implements two well-known optimizations known as union-by-rank and path compression. This version is simplified to only work on integer elements.

- `initialize()` resets the data structure.
- `make_set(u)` creates a new partition consisting of the single element `u`, which must not have been previously added to the data structure.
- `find_root(u)` returns the unique representative of the partition containing `u`.
- `is_united(u, v)` returns whether elements `u` and `v` belong to the same partition.
- `unite(u, v)` replaces the partitions containing `u` and `v` with a single new partition consisting of the union of elements in the original partitions.

A precondition to the last three operations is that `make_set()` must have been previously called on their arguments.

Time Complexity:

- $O(1)$ per call to `initialize()` and `make_set()`.
- $O(\alpha(n))$ per call to `find_root()`, `is_united()`, and `unite()`, where n is the number of elements that has been added via `make_set()` so far, and $\alpha(n)$ is the extremely slow growing inverse of the Ackermann function (effectively a very small constant for all practical values of n).

Space Complexity:

- $O(n)$ for storage of the disjoint set forest elements.
- $O(1)$ auxiliary for all operations.

```

const int MAXN = 1000;
int num_sets, root[MAXN], rank[MAXN];

void initialize() {
    num_sets = 0;
}

void make_set(int u) {
    root[u] = u;
    rank[u] = 0;
    num_sets++;
}

int find_root(int u) {
    if (root[u] != u) {
        root[u] = find_root(root[u]);
    }
    return root[u];
}

bool is_united(int u, int v) {
    return find_root(u) == find_root(v);
}

void unite(int u, int v) {
    int ru = find_root(u), rv = find_root(v);
    if (ru == rv) {
        return;
    }
    num_sets--;
    if (rank[ru] < rank[rv]) {
        root[ru] = rv;
    } else {
        root[rv] = ru;
        if (rank[ru] == rank[rv]) {
            rank[ru]++;
        }
    }
}

```

Example Usage

```

#include <cassert>

int main() {
    initialize();
    for (char c = 'a'; c <= 'g'; c++) {
        make_set(c);
    }
    unite('a', 'b');
    unite('b', 'f');
    unite('d', 'e');
    unite('d', 'g');
    assert(num_sets == 3);
    assert(is_united('a', 'b'));
    assert(!is_united('a', 'c'));
    assert(!is_united('b', 'g'));
    assert(is_united('e', 'g'));
    return 0;
}

```

2.7.2 Disjoint Set Forest (Compressed)

Maintain a set of elements partitioned into non-overlapping subsets using a collection of trees. Each partition is assigned a unique representative known as the parent, or root. The following implements two well-known optimizations known as union-by-rank and path compression. This version uses an `std::map` for storage and coordinate compression (thus, element types must meet the requirements of key types for `std::map`).

- `make_set(u)` creates a new partition consisting of the single element `u`, which must not have been previously added to the data structure.
- `is_united(u, v)` returns whether elements `u` and `v` belong to the same partition.
- `unite(u, v)` replaces the partitions containing `u` and `v` with a single new partition consisting of the union of elements in the original partitions.
- `get_all_sets()` returns all current partitions as a vector of vectors.

A precondition to the last three operations is that `make_set()` must have been previously called on their arguments.

Time Complexity:

- $O(1)$ per call to the constructor.
- $O(\log n)$ per call to `make_set()`, where n is the number of elements that have been added via `make_set()` so far.
- $O(\alpha(n) \log n)$ per call to `is_united()` and `unite()`, where n is the number of elements that have been added via `make_set()` so far, and $\alpha(n)$ is the extremely slow growing inverse of the Ackermann function (effectively a very small constant for all practical values of n).
- $O(n)$ per call to `get_all_sets()`.

Space Complexity:

- $O(n)$ for storage of the disjoint set forest elements.
- $O(n)$ auxiliary heap space for `get_all_sets()`.
- $O(1)$ auxiliary for all other operations.

```
#include <map>
#include <vector>

template<class T>
class disjoint_set_forest {
    int num_elements, num_sets;
    std::map<T, int> id;
    std::vector<int> root, rank;

    int find_root(int u) {
        if (root[u] != u) {
            root[u] = find_root(root[u]);
        }
        return root[u];
    }

public:
    disjoint_set_forest() : num_elements(0), num_sets(0) {}

    int size() const { return num_elements; }
    int sets() const { return num_sets; }

    void make_set(const T &u) {
        if (id.find(u) != id.end()) {
            return;
        }
        id[u] = num_elements;
        root.push_back(num_elements++);
        rank.push_back(0);
        num_sets++;
    }
}
```

```

bool is_united(const T &u, const T &v) { return find_root(id[u]) == find_root(id[v]); }

void unite(const T &u, const T &v) {
    int ru = find_root(id[u]), rv = find_root(id[v]);
    if (ru == rv) {
        return;
    }
    num_sets--;
    if (rank[ru] < rank[rv]) {
        root[ru] = rv;
    } else {
        root[rv] = ru;
        if (rank[ru] == rank[rv]) {
            rank[ru]++;
        }
    }
}

std::vector<std::vector<T>> get_all_sets() {
    std::map<int, std::vector<T>> tmp;
    for (typename std::map<T, int>::iterator it = id.begin(); it != id.end(); ++it) {
        tmp[find_root(it->second)].push_back(it->first);
    }
    std::vector<std::vector<T>> res;
    for (typename std::map<int, std::vector<T>>::iterator it = tmp.begin(); it != tmp.end(); ++it) {
        res.push_back(it->second);
    }
    return res;
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    disjoint_set_forest<char> dsf;
    for (char c = 'a'; c <= 'g'; c++) {
        dsf.make_set(c);
    }
    dsf.unite('a', 'b');
    dsf.unite('b', 'f');
    dsf.unite('d', 'e');
    dsf.unite('d', 'g');
    assert(dsf.size() == 7);
    assert(dsf.sets() == 3);
    vector<vector<char>> s = dsf.get_all_sets();
    for (int i = 0; i < (int)s.size(); i++) {
        cout << (i > 0 ? ", [" : "[";
        for (int j = 0; j < (int)s[i].size(); j++) {
            cout << (j > 0 ? ", " : "") << s[i][j];
        }
        cout << "]\n";
    }
    cout << endl;
    return 0;
}

```

Example Output

```
[a, b, f], [c], [d, e, g]
```

2.7.3 Disjoint Set Forest (Rollback)

Maintains disjoint sets while supporting rollback to previous snapshots. Rollback DSU is useful for offline dynamic connectivity, divide-and-conquer over time, and backtracking search where unions must be undone.

Path compression is intentionally not used, because it is hard to undo. Union by size keeps tree height logarithmic, and every successful union records enough information to restore the previous state.

- `initialize(n)` creates singleton sets over elements `0` through `n - 1`.
- `find_root(u)` returns the representative of the set containing `u`.
- `is_united(u, v)` returns whether `u` and `v` are in the same set.
- `unite(u, v)` merges two sets and returns whether a merge occurred.
- `snapshot()` returns a token representing the current history size.
- `rollback(s)` undoes all changes made after snapshot `s`.

Time Complexity:

- $O(n)$ per call to `initialize(n)`.
- $O(\log n)$ worst-case per call to `find_root(u)`, `is_united(u, v)`, and `unite(u, v)`.
- $O(1)$ per undone union during `rollback(s)`.

Space Complexity: $O(n + q)$ for q successful unions stored in the rollback history.

```
#include <vector>

class rollback_dsu {
    struct change {
        int child, parent, parent_size;

        change(int child = -1, int parent = -1, int parent_size = 0)
            : child(child), parent(parent), parent_size(parent_size) {}
    };

    std::vector<int> root, size;
    std::vector<change> history;
    int sets;

public:
    rollback_dsu() : sets(0) {}

    explicit rollback_dsu(int n) { initialize(n); }

    void initialize(int n) {
        root.resize(n);
        size.assign(n, 1);
        history.clear();
        sets = n;
        for (int i = 0; i < n; i++) {
            root[i] = i;
        }
    }

    int count_sets() const { return sets; }

    int find_root(int u) const {
        while (root[u] != u) {
            u = root[u];
        }
        return u;
    }

    bool is_united(int u, int v) const { return find_root(u) == find_root(v); }

    bool unite(int u, int v) {
        u = find_root(u);
        v = find_root(v);
```



```

    if (u == v) {
        return false;
    }
    if (size[u] > size[v]) {
        int tmp = u;
        u = v;
        v = tmp;
    }
    history.push_back(change(u, v, size[v]));
    root[u] = v;
    size[v] += size[u];
    sets--;
    return true;
}

int snapshot() const { return history.size(); }

void rollback(int snapshot) {
    while ((int)history.size() > snapshot) {
        change c = history.back();
        history.pop_back();
        root[c.child] = c.child;
        size[c.parent] = c.parent_size;
        sets++;
    }
}
};

```

Example Usage

```

#include <cassert>

int main() {
    rollback_dsu dsu(5);
    dsu.unite(0, 1);
    int s = dsu.snapshot();
    dsu.unite(1, 2);
    dsu.unite(3, 4);
    assert(dsu.is_united(0, 2));
    assert(dsu.count_sets() == 2);
    dsu.rollback(s);
    assert(dsu.is_united(0, 1));
    assert(!dsu.is_united(0, 2));
    assert(!dsu.is_united(3, 4));
    assert(dsu.count_sets() == 4);
    return 0;
}

```

2.7.4 Lowest Common Ancestor (Sparse Table)

Given a tree, determine the lowest common ancestor of any two nodes in the tree. The lowest common ancestor of two nodes u and v is the node that has the longest distance from the root while having both u and v as its descendant. A node is considered to be a descendant of itself. `build()` applies to a global, pre-populated adjacency list `adj[]` which must only consist of nodes numbered with integers between 0 (inclusive) and the total number of nodes (exclusive), as passed in the function argument.

Time Complexity:

- $O(n \log n)$ per call to `build()`, where n is the number of nodes.
- $O(\log n)$ per call to `lca()`.

Space Complexity:

- $O(n \log n)$ to store the sparse table, where n is the number of nodes.
- $O(n)$ auxiliary stack space for `build()`.
- $O(1)$ auxiliary for `lca()`.

```
#include <vector>

const int MAXN = 1000;
std::vector<int> adj[MAXN], dp[MAXN];
int len, timer, tin[MAXN], tout[MAXN];

void dfs(int u, int p) {
    tin[u] = timer++;
    dp[u][0] = p;
    for (int i = 1; i < len; i++) {
        dp[u][i] = dp[dp[u][i - 1]][i - 1];
    }
    for (int j = 0; j < (int)adj[u].size(); j++) {
        int v = adj[u][j];
        if (v != p) {
            dfs(v, u);
        }
    }
    tout[u] = timer++;
}

void build(int nodes, int root = 0) {
    len = 1;
    while ((1 << len) <= nodes) {
        len++;
    }
    for (int i = 0; i < nodes; i++) {
        dp[i].resize(len);
    }
    timer = 0;
    dfs(root, root);
}

bool is_ancestor(int parent, int child) {
    return (tin[parent] <= tin[child]) && (tout[child] <= tout[parent]);
}

int lca(int u, int v) {
    if (is_ancestor(u, v)) {
        return u;
    }
    if (is_ancestor(v, u)) {
        return v;
    }
    for (int i = len - 1; i >= 0; i--) {
        if (!is_ancestor(dp[u][i], v)) {
            u = dp[u][i];
        }
    }
    return dp[u][0];
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    adj[0].push_back(1);
    adj[1].push_back(0);
    adj[1].push_back(2);
```

```

adj[2].push_back(1);
adj[3].push_back(1);
adj[1].push_back(3);
adj[0].push_back(4);
adj[4].push_back(0);
build(5, 0);
assert(lca(3, 2) == 1);
assert(lca(2, 4) == 0);
return 0;
}

```

2.7.5 Lowest Common Ancestor (Segment Tree)

Given a tree, determine the lowest common ancestor of any two nodes in the tree. The lowest common ancestor of two nodes u and v is the node that has the longest distance from the root while having both u and v as its descendant. A node is considered to be a descendant of itself. `build()` applies to a global, pre-populated adjacency list `adj[]` which must only consist of nodes numbered with integers between 0 (inclusive) and the total number of nodes (exclusive), as passed in the function argument.

Time Complexity:

- $O(n \log n)$ per call to `build()`, where n is the number of nodes.
- $O(\log n)$ per call to `lca()`.

Space Complexity:

- $O(n)$ for storage of the segment tree, where n is the number of nodes.
- $O(n \log n)$ auxiliary stack space for `build()`.
- $O(\log n)$ auxiliary stack space for `lca()`.

```

#include <algorithm>
#include <vector>

const int MAXN = 1000;
std::vector<int> adj[MAXN];
int len, counter, depth[MAXN], dfs_order[2 * MAXN], first[MAXN], minpos[8 * MAXN];

void dfs(int u, int d) {
    depth[u] = d;
    dfs_order[counter++] = u;
    for (int j = 0; j < (int)adj[u].size(); j++) {
        int v = adj[u][j];
        if (depth[v] == -1) {
            dfs(v, d + 1);
            dfs_order[counter++] = u;
        }
    }
}

void build(int n, int lo, int hi) {
    if (lo == hi) {
        minpos[n] = dfs_order[lo];
        return;
    }
    int lchild = 2 * n + 1, rchild = 2 * n + 2, mid = lo + (hi - lo) / 2;
    build(lchild, lo, mid);
    build(rchild, mid + 1, hi);
    minpos[n] = depth[minpos[lchild]] < depth[minpos[rchild]] ? minpos[lchild] : minpos[rchild];
}

void build(int nodes, int root) {
    std::fill(depth, depth + nodes, -1);
    std::fill(first, first + nodes, -1);
}

```

```

len = 2 * nodes - 1;
counter = 0;
dfs(root, 0);
build(0, 0, len - 1);
for (int i = 0; i < len; i++) {
    if (first[dfs_order[i]] == -1) {
        first[dfs_order[i]] = i;
    }
}
}

int get_minpos(int a, int b, int n, int lo, int hi) {
    if (a == lo && b == hi) {
        return minpos[n];
    }
    int mid = lo + (hi - lo) / 2;
    if (a <= mid && mid < b) {
        int p1 = get_minpos(a, std::min(b, mid), 2 * n + 1, lo, mid);
        int p2 = get_minpos(std::max(a, mid + 1), b, 2 * n + 2, mid + 1, hi);
        return depth[p1] < depth[p2] ? p1 : p2;
    }
    if (a <= mid) {
        return get_minpos(a, std::min(b, mid), 2 * n + 1, lo, mid);
    }
    return get_minpos(std::max(a, mid + 1), b, 2 * n + 2, mid + 1, hi);
}

int lca(int u, int v) {
    return get_minpos(std::min(first[u], first[v]), std::max(first[u], first[v]), 0, 0, len - 1);
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    adj[0].push_back(1);
    adj[1].push_back(0);
    adj[1].push_back(2);
    adj[2].push_back(1);
    adj[3].push_back(1);
    adj[1].push_back(3);
    adj[0].push_back(4);
    adj[4].push_back(0);
    build(5, 0);
    assert(lca(3, 2) == 1);
    assert(lca(2, 4) == 0);
    return 0;
}

```

2.7.6 Heavy Light Decomposition

Maintain a tree with values associated with either its edges or nodes, while supporting both dynamic queries and dynamic updates of all values on a given path between two nodes in the tree. Heavy-light decomposition partitions the nodes of the tree into disjoint paths where all nodes have degree two, except the endpoints of a path which has degree one.

The query operation is defined by an associative `join_values()` function which satisfies

`join_values(x, join_values(y, z)) = join_values(join_values(x, y), z)` for all values `x`, `y`, and `z` in the tree. The default code below assumes a numerical tree type, defining queries for the "min" of the target range. Another possible query operation is "sum", in which case the `join_values()` function should be defined to return $a + b$.

The update operation is defined by the `join_value_with_delta()` and `join_deltas()` functions, which determines the change made to values. These must satisfy:

- `join_deltas(d1, join_deltas(d2, d3)) = join_deltas(join_deltas(d1, d2), d3)`.
- `join_value_with_delta(join_values(v, ..., v), d, m)` should be equal to `join_values(join_value_with_delta(v, d, 1), ...)`, with m values on each side.
- if a sequence $d_1, \dots, d_m$ of deltas is used to update a value v , then `join_value_with_delta(v, join_deltas(d1, ..., dm), 1)` should be equivalent to m sequential calls to `join_value_with_delta(v, d_i, 1)` for $i = 1, \dots, m$. The default code below defines updates that "set" a path's edges or nodes to a new value. Another possible update operation is "increment", in which case `join_value_with_delta(v, d, len)` should be defined to return $v + d \cdot \text{len}$ and `join_deltas(d1, d2)` should be defined to return $d1 + d2$.
- `heavy_light(n, adj[], v)` constructs a new heavy light decomposition on a tree with n nodes defined by the adjacency list `adj[]`, with all values initialized to v . The adjacency list must be a size n array of vectors consisting of only the integers from 0 to $n - 1$, inclusive. No duplicate edges should exist, and the graph must be connected.
- `query(u, v)` returns the result of `join_values()` applied to all values on the path from node u to node v .
- `update(u, v, d)` modifies all values on the path from node u to node v by respectively joining them with d using `join_value_with_delta()`.

Time Complexity:

- $O(n)$ per call to the constructor, where n is the number of nodes.
- $O(\log n)$ per call to `query()` and `update()`;

Space Complexity:

- $O(n)$ for storage of the decomposition.
- $O(n)$ auxiliary stack space for the constructor.
- $O(1)$ auxiliary for `query()` and `update()`.

```
#include <algorithm>
#include <stdexcept>
#include <vector>

template<class T>
class heavy_light {
    // Set this to true to store values on edges, false to store values on nodes.
    static const bool VALUES_ON_EDGES = true;

    static T join_values(const T &a, const T &b) { return std::min(a, b); }
    static T join_value_with_delta(const T &v, const T &d, int len) { return d; }

    static T join_deltas(const T &d1, const T &d2) {
        return d2; // For "set" updates, the more recent delta prevails.
    }

    int counter, paths;
    std::vector<std::vector<T>> value, delta;
    std::vector<std::vector<bool>> pending;
    std::vector<std::vector<int>> len;
    std::vector<int> size, parent, tin, tout, path, pathlen, pathpos, pathroot;
    std::vector<int> *adj;

    void dfs(int u, int p) {
        tin[u] = counter++;
        parent[u] = p;
        size[u] = 1;
        for (int j = 0; j < (int)adj[u].size(); j++) {
            int v = adj[u][j];
            if (v != p) {
                dfs(v, u);
                size[u] += size[v];
            }
        }
    }
};
```

```

    }
}
tout[u] = counter++;
}

int new_path(int u) {
    pathroot[paths] = u;
    return paths++;
}

void build_paths(int u, int path) {
    this->path[u] = path;
    pathpos[u] = pathlen[path]++;
    for (int j = 0; j < (int)adj[u].size(); j++) {
        int v = adj[u][j];
        if (v != parent[u]) {
            build_paths(v, (2 * size[v] >= size[u]) ? path : new_path(v));
        }
    }
}

inline T join_value_with_delta(int path, int i) {
    return pending[path][i] ? join_value_with_delta(value[path][i], delta[path][i], len[path][i])
        : value[path][i];
}

void push_delta(int path, int i) {
    int d = 0;
    while ((i >> d) > 0) {
        d++;
    }
    for (d -= 2; d >= 0; d--) {
        int l = (i >> d), r = (1 ^ 1), n = 1 / 2;
        if (pending[path][n]) {
            value[path][n] = join_value_with_delta(path, n);
            delta[path][l] =
                pending[path][l] ? join_deltas(delta[path][l], delta[path][n]) : delta[path][n];
            delta[path][r] =
                pending[path][r] ? join_deltas(delta[path][r], delta[path][n]) : delta[path][n];
            pending[path][l] = pending[path][r] = true;
            pending[path][n] = false;
        }
    }
}

bool query(int path, int u, int v, T *res) {
    push_delta(path, u += value[path].size() / 2);
    push_delta(path, v += value[path].size() / 2);
    bool found = false;
    for (; u <= v; u = (u + 1) / 2, v = (v - 1) / 2) {
        if ((u & 1) != 0) {
            T value = join_value_with_delta(path, u);
            *res = found ? join_values(*res, value) : value;
            found = true;
        }
        if ((v & 1) == 0) {
            T value = join_value_with_delta(path, v);
            *res = found ? join_values(*res, value) : value;
            found = true;
        }
    }
    return found;
}

void update(int path, int u, int v, const T &d) {
    push_delta(path, u += value[path].size() / 2);
    push_delta(path, v += value[path].size() / 2);
    int tu = -1, tv = -1;

```

```

for (; u <= v; u = (u + 1) / 2, v = (v - 1) / 2) {
    if ((u & 1) != 0) {
        delta[path][u] = pending[path][u] ? join_deltas(delta[path][u], d) : d;
        pending[path][u] = true;
        if (tu == -1) {
            tu = u;
        }
    }
    if ((v & 1) == 0) {
        delta[path][v] = pending[path][v] ? join_deltas(delta[path][v], d) : d;
        pending[path][v] = true;
        if (tv == -1) {
            tv = v;
        }
    }
}
for (int i = tu; i > 1; i /= 2) {
    value[path][i / 2] =
        join_values(join_value_with_delta(path, i), join_value_with_delta(path, i ^ 1));
}
for (int i = tv; i > 1; i /= 2) {
    value[path][i / 2] =
        join_values(join_value_with_delta(path, i), join_value_with_delta(path, i ^ 1));
}
}

inline bool is_ancestor(int parent, int child) {
    return (tin[parent] <= tin[child]) && (tout[child] <= tout[parent]);
}

public:
heavy_light(int n, std::vector<int> adj[], const T &v = T())
: counter(0),
  paths(0),
  size(n),
  parent(n),
  tin(n),
  tout(n),
  path(n),
  pathlen(n),
  pathpos(n),
  pathroot(n),
  adj(adj) {
dfs(0, -1);
build_paths(0, new_path(0));
value.resize(paths);
delta.resize(paths);
pending.resize(paths);
len.resize(paths);
for (int i = 0; i < paths; i++) {
    int m = pathlen[i];
    value[i].assign(2 * m, v);
    delta[i].resize(2 * m);
    pending[i].assign(2 * m, false);
    len[i].assign(2 * m, 1);
    for (int j = 2 * m - 1; j > 1; j -= 2) {
        value[i][j / 2] = join_values(value[i][j], value[i][j ^ 1]);
        len[i][j / 2] = len[i][j] + len[i][j ^ 1];
    }
}
}

T query(int u, int v) {
    if (VALUES_ON_EDGES && u == v) {
        throw std::runtime_error("No edge between u and v to be queried.");
    }
    bool found = false;
    T res = T(), value;

```

```

int root;
while (!is_ancestor(root = pathroot[path[u]], v)) {
    if (query(path[u], 0, pathpos[u], &value)) {
        res = found ? join_values(res, value) : value;
        found = true;
    }
    u = parent[root];
}
while (!is_ancestor(root = pathroot[path[v]], u)) {
    if (query(path[v], 0, pathpos[v], &value)) {
        res = found ? join_values(res, value) : value;
        found = true;
    }
    v = parent[root];
}
if (query(
    path[u], std::min(pathpos[u], pathpos[v]) + (int)VALUES_ON_EDGES,
    std::max(pathpos[u], pathpos[v]), &value
)) {
    res = found ? join_values(res, value) : value;
    found = true;
}
if (!found) {
    throw std::runtime_error("Unexpected error: No values found.");
}
return res;
}

void update(int u, int v, const T &d) {
    if (VALUES_ON_EDGES && u == v) {
        return;
    }
    int root;
    while (!is_ancestor(root = pathroot[path[u]], v)) {
        update(path[u], 0, pathpos[u], d);
        u = parent[root];
    }
    while (!is_ancestor(root = pathroot[path[v]], u)) {
        update(path[v], 0, pathpos[v], d);
        v = parent[root];
    }
    update(
        path[u], std::min(pathpos[u], pathpos[v]) + (int)VALUES_ON_EDGES,
        std::max(pathpos[u], pathpos[v]), d
    );
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    //      w=40      w=20      w=10
    // 0-----1-----2-----3
    //                      |
    //                      -----4
    //                      w=30
    vector<int> adj[5];
    adj[0].push_back(1);
    adj[1].push_back(0);
    adj[1].push_back(2);
    adj[2].push_back(1);
    adj[2].push_back(3);
    adj[3].push_back(2);

```



```

adj[2].push_back(4);
adj[4].push_back(2);
heavy_light<int> hld(5, adj, 0);
hld.update(0, 1, 40);
hld.update(1, 2, 20);
hld.update(2, 3, 10);
hld.update(2, 4, 30);
assert(hld.query(0, 3) == 10);
assert(hld.query(2, 4) == 30);
hld.update(3, 4, 5);
assert(hld.query(1, 4) == 5);
return 0;
}

```

2.7.7 Link-Cut Tree

Maintain a forest of trees with values associated with its nodes, while supporting both dynamic queries and dynamic updates of all values on any path between two nodes in a given tree. In addition, support testing of whether two nodes are connected in the forest, as well as the merging and splitting of trees by adding or removing specific edges. Link/cut forests divide each of its trees into vertex-disjoint paths, each represented by a splay tree.

The query operation is defined by an associative `join_values()` function which satisfies

`join_values(x, join_values(y, z)) = join_values(join_values(x, y), z)` for all values `x`, `y`, and `z` in the forest. The default code below assumes a numerical forest type, defining queries for the "min" of the target range. Another possible query operation is "sum", in which case the `join_values()` function should be defined to return $a + b$.

The update operation is defined by the `join_value_with_delta()` and `join_deltas()` functions, which determines the change made to values. These must satisfy:

- `join_deltas(d1, join_deltas(d2, d3)) = join_deltas(join_deltas(d1, d2), d3)`.
- `join_value_with_delta(join_values(v, ..., v), d, m)` should be equal to `join_values(join_value_with_delta(v, d, 1), ...)`, with m values on each side.
- if a sequence $d_1, \dots, d_m$ of deltas is used to update a value v , then `join_value_with_delta(v, join_deltas(d_1, ..., d_m), 1)` should be equivalent to m sequential calls to `join_value_with_delta(v, d_i, 1)` for $i = 1, \dots, m$. The default code below defines updates that "set" a path's nodes to a new value. Another possible update operation is "increment", in which case `join_value_with_delta(v, d, len)` should be defined to return $v + d \cdot \text{len}$ and `join_deltas(d1, d2)` should be defined to return $d1 + d2$.
- `link_cut_forest()` constructs an empty forest with no trees.
- `size()` returns the number of nodes in the forest.
- `trees()` returns the number of trees in the forest.
- `make_root(i, v)` creates a new tree in the forest consisting of a single node labeled with the integer `i` and value initialized to `v`.
- `is_connected(a, b)` returns whether nodes `a` and `b` are connected.
- `link(a, b)` adds an edge between the nodes `a` and `b`, both of which must exist and not be connected.
- `cut(a, b)` removes the edge between the nodes `a` and `b`, both of which must exist and be connected.
- `query(a, b)` returns the result of `join_values()` applied to all values on the path from the node `a` to node `b`.
- `update(a, b, d)` modifies all the values on the path from node `a` to node `b` by respectively joining them with `d` using `join_value_with_delta()`.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `trees()`.
- $O(\log n)$ amortized per call to all other operations, where n is the number of nodes.

Space Complexity:

- $O(n)$ for storage of the forest, where n is the number of nodes.

- $O(1)$ auxiliary for all operations.

```

#include <algorithm>
#include <cstdint>
#include <map>
#include <stdexcept>

template<class T>
class link_cut_forest {
    static T join_values(const T &a, const T &b) { return std::min(a, b); }
    static T join_value_with_delta(const T &v, const T &d, int len) { return d; }
    static T join_deltas(const T &d1, const T &d2) {
        return d2; // For "set" updates, the more recent delta prevails.
    }
}

struct node_t {
    T value, subtree_value, delta;
    int size;
    bool rev, pending;
    node_t *left, *right, *parent;

    node_t(const T &v)
        : value(v),
          subtree_value(v),
          size(1),
          rev(false),
          pending(false),
          left(NULL),
          right(NULL),
          parent(NULL) {}

    inline bool is_root() const {
        return parent == NULL || (parent->left != this && parent->right != this);
    }

    inline T get_subtree_value() const {
        return pending ? join_value_with_delta(subtree_value, delta, size) : subtree_value;
    }

    void push() {
        if (rev) {
            rev = false;
            std::swap(left, right);
            if (left != NULL) {
                left->rev = !left->rev;
            }
            if (right != NULL) {
                right->rev = !right->rev;
            }
        }
        if (pending) {
            value = join_value_with_delta(value, delta, 1);
            subtree_value = join_value_with_delta(subtree_value, delta, size);
            if (left != NULL) {
                left->delta = left->pending ? join_deltas(left->delta, delta) : delta;
                left->pending = true;
            }
            if (right != NULL) {
                right->delta = right->pending ? join_deltas(right->delta, delta) : delta;
                right->pending = true;
            }
            pending = false;
        }
    }

    void update() {
        size = 1;
        subtree_value = value;
    }
}

```

```

    if (left != NULL) {
        subtree_value = join_values(subtree_value, left->get_subtree_value());
        size += left->size;
    }
    if (right != NULL) {
        subtree_value = join_values(subtree_value, right->get_subtree_value());
        size += right->size;
    }
}

};

int num_trees;
std::map<int, node_t *> nodes;

static void connect(node_t *child, node_t *parent, bool is_left) {
    if (child != NULL) {
        child->parent = parent;
    }
    if (is_left) {
        parent->left = child;
    } else {
        parent->right = child;
    }
}

static void rotate(node_t *n) {
    node_t *parent = n->parent, *grandparent = parent->parent;
    bool parent_is_root = parent->is_root(), is_left = (n == parent->left);
    connect(is_left ? n->right : n->left, parent, is_left);
    connect(parent, n, !is_left);
    if (parent_is_root) {
        if (n != NULL) {
            n->parent = grandparent;
        }
    } else {
        connect(n, grandparent, parent == grandparent->left);
    }
    parent->update();
}

static void splay(node_t *n) {
    while (!n->is_root()) {
        node_t *parent = n->parent, *grandparent = parent->parent;
        if (!parent->is_root()) {
            grandparent->push();
        }
        parent->push();
        n->push();
        if (!parent->is_root()) {
            if ((n == parent->left) == (parent == grandparent->left)) {
                rotate(parent);
            } else {
                rotate(n);
            }
        }
        rotate(n);
    }
    n->push();
    n->update();
}

static node_t *expose(node_t *n) {
    node_t *prev = NULL;
    for (node_t *curr = n; curr != NULL; curr = curr->parent) {
        splay(curr);
        curr->left = prev;
        prev = curr;
    }
}

```

```

    splay(n);
    return prev;
}

// Helper variables.
node_t *u, *v;

void get_uv(int a, int b) {
    typename std::map<int, node_t *>::iterator it1, it2;
    it1 = nodes.find(a);
    it2 = nodes.find(b);
    if (it1 == nodes.end() || it2 == nodes.end()) {
        throw std::runtime_error("Queried node ID does not exist in forest.");
    }
    u = it1->second;
    v = it2->second;
}

public:
link_cut_forest() : num_trees(0) {}

~link_cut_forest() {
    typename std::map<int, node_t *>::iterator it;
    for (it = nodes.begin(); it != nodes.end(); ++it) {
        delete it->second;
    }
}

int size() const { return nodes.size(); }
int trees() const { return num_trees; }

void make_root(int i, const T &v = T()) {
    if (nodes.find(i) != nodes.end()) {
        throw std::runtime_error("Cannot make a root with an existing ID.");
    }
    node_t *n = new node_t(v);
    expose(n);
    n->rev = !n->rev;
    nodes[i] = n;
    num_trees++;
}

bool is_connected(int a, int b) {
    get_uv(a, b);
    if (a == b) {
        return true;
    }
    expose(u);
    expose(v);
    return u->parent != NULL;
}

void link(int a, int b) {
    if (is_connected(a, b)) {
        throw std::runtime_error("Cannot link nodes that are already connected.");
    }
    get_uv(a, b);
    expose(u);
    u->rev = !u->rev;
    u->parent = v;
    num_trees--;
}

void cut(int a, int b) {
    get_uv(a, b);
    expose(u);
    u->rev = !u->rev;
    expose(v);
}

```

```

    if (v->right != u || u->left != NULL) {
        throw std::runtime_error("Cannot cut edge that does not exist.");
    }
    v->right->parent = NULL;
    v->right = NULL;
    num_trees++;
}

T query(int a, int b) {
    if (!is_connected(a, b)) {
        throw std::runtime_error("Cannot query nodes that are not connected.");
    }
    get_uv(a, b);
    expose(u);
    u->rev = !u->rev;
    expose(v);
    return v->get_subtree_value();
}

void update(int a, int b, const T &d) {
    if (!is_connected(a, b)) {
        throw std::runtime_error("Cannot update nodes that are not connected.");
    }
    get_uv(a, b);
    expose(u);
    u->rev = !u->rev;
    expose(v);
    v->delta = v->pending ? join_deltas(v->delta, d) : d;
    v->pending = true;
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    link_cut_forest<int> lcf;
    lcf.make_root(0, 10);
    lcf.make_root(1, 40);
    lcf.make_root(2, 20);
    lcf.make_root(3, 10);
    lcf.make_root(4, 30);
    assert(lcf.size() == 5);
    assert(lcf.trees() == 5);
    lcf.link(0, 1);
    lcf.link(1, 2);
    lcf.link(2, 3);
    lcf.link(2, 4);
    assert(lcf.trees() == 1);

    // v=10      v=40      v=20      v=10
    // 0-----1-----2-----3
    //                |
    //                -----4
    //                      v=30
    assert(lcf.query(1, 4) == 20);
    lcf.update(1, 1, 100);
    lcf.update(2, 4, 100);

    // v=10      v=100      v=100      v=10
    // 0-----1-----2-----3
    //                |
    //                -----4
    //                      v=100

```

```

assert(lcf.query(4, 4) == 100);
assert(lcf.query(0, 4) == 10);
assert(lcf.query(3, 4) == 10);
lcf.cut(1, 2);

// v=10      v=100      v=100      v=0
// 0-----1      2-----3
//              |
//              -----4
//                      v=100
assert(lcf.trees() == 2);
assert(!lcf.is_connected(1, 2));
assert(!lcf.is_connected(0, 4));
assert(lcf.is_connected(2, 3));
return 0;
}

```

Chapter 3

Optimization

3.1 Search on Answer

3.1.1 Binary Search

Binary search can be generally used to find the input value corresponding to any output value of a monotonic (strictly non-increasing or strictly non-decreasing) function in $O(\log n)$ time with respect to the domain size. This is a special case of finding the exact point at which any given monotonic Boolean function changes from true to false or vice versa. Unlike searching through an array, discrete binary search is not restricted by available memory, making it useful for handling infinitely large search spaces such as real number intervals.

- `binary_search_first_true()` takes two integers `lo` and `hi` as boundaries for the search space `[lo, hi)` (i.e. including `lo`, but excluding `hi`) and returns the smallest integer `k` in `[lo, hi)` for which the predicate `pred(k)` tests true. If `pred(k)` tests false for every `k` in `[lo, hi)`, then `hi` is returned. This function must be used on a range in which there exists a constant `k` such that `pred(x)` tests false for every `x` in `[lo, k)` and true for every `x` in `[k, hi)`.
- `binary_search_last_true()` takes two integers `lo` and `hi` as boundaries for the search space `[lo, hi)` (i.e. including `lo`, but excluding `hi`) and returns the largest integer `k` in `[lo, hi)` for which the predicate `pred(k)` tests true. If `pred(k)` tests false for every `k` in `[lo, hi)`, then `hi` is returned. This function must be used on a range in which there exists a constant `k` such that `pred(x)` tests true for every `x` in `[lo, k)` and false for every `x` in `[k, hi)`.
- `fbinary_search()` is the equivalent of `binary_search_first_true()` on floating point predicates. Since any interval of real numbers is dense, the exact target cannot be found due to floating point error. Instead, a value that is very close to the border between false and true is returned. The precision of the answer depends on the number of repetitions the function performs. Since each repetition bisects the search space, the absolute error of the answer is $1/(2^n)$ times the distance between `lo` and `hi` after n repetitions. Although it is possible to control the error by looping while `hi - lo` is greater than an arbitrary epsilon, it is simpler to let the loop run for a desired number of iterations until floating point arithmetic break down. 100 iterations is usually sufficient, since the search space will be reduced to 2^{-100} (roughly 10^{-30}) times its original size. This implementation can be modified to find the "last true" point in the range by simply interchanging the assignments of `lo` and `hi` in the if-else statements.

Time Complexity:

- $O(\log n)$ calls will be made to `pred()` in `binary_search_first_true()` and `binary_search_last_true()`, where n is the distance between `lo` and `hi`.
- $O(n)$ calls will be made to `pred()` in `fbinary_search()`, where n is the number of iterations performed.

Space Complexity: $O(1)$ auxiliary for all operations.

```

template<class Int, class IntPredicate>
Int binary_search_first_true(Int lo, Int hi, IntPredicate pred) { // 000[1]11
    Int mid, _hi = hi;
    while (lo < hi) {
        mid = lo + (hi - lo) / 2;
        if (pred(mid)) {
            hi = mid;
        } else {
            lo = mid + 1;
        }
    }
    if (!pred(lo)) {
        return _hi; // All false.
    }
    return lo;
}

template<class Int, class IntPredicate>
Int binary_search_last_true(Int lo, Int hi, IntPredicate pred) { // 11[1]000
    Int mid, _hi = hi--;
    while (lo < hi) {
        mid = lo + (hi - lo + 1) / 2;
        if (pred(mid)) {
            lo = mid;
        } else {
            hi = mid - 1;
        }
    }
    if (!pred(lo)) {
        return _hi; // All false.
    }
    return lo;
}

template<class DoublePredicate>
double fbinary_search(double lo, double hi, DoublePredicate pred) { // 000[1]11
    double mid;
    for (int i = 0; i < 100; i++) {
        mid = (lo + hi) / 2.0;
        if (pred(mid)) {
            hi = mid;
        } else {
            lo = mid;
        }
    }
    return lo;
}

```

Example Usage

```

#include <cassert>
#include <cmath>

// Simple predicate examples.
bool pred1(int x) {
    return x >= 3;
}
bool pred2(int x) {
    return false;
}
bool pred3(int x) {
    return x <= 5;
}
bool pred4(int x) {
    return true;
}

```



```

bool pred5(double x) {
    return x >= 1.2345;
}

int main() {
    assert(binary_search_first_true(0, 7, pred1) == 3);
    assert(binary_search_first_true(0, 7, pred2) == 7);
    assert(binary_search_last_true(0, 7, pred3) == 5);
    assert(binary_search_last_true(0, 7, pred4) == 6);
    assert(fabs(fbinary_search(-10.0, 10.0, pred5) - 1.2345) < 1e-15);
    return 0;
}

```

3.1.2 Exponential Search

Finds the first value where a monotone Boolean predicate becomes true when an upper bound is not known in advance. Exponential search first grows the search range by powers of two until it brackets the transition point, then finishes with ordinary binary search.

This is useful for answer-search problems over unbounded or very large integer domains. The predicate must eventually become true, or the search may overflow or run forever unless an explicit limit is added.

- `exponential_search_first_true(lo, pred)` returns the smallest integer `x >= lo` such that `pred(x)` is true.

Time Complexity: $O(\log ans)$ calls to `pred()`, where `ans` is the distance from `lo` to the first true value.

Space Complexity: $O(1)$ auxiliary.

```

template<class Int, class IntPredicate>
Int exponential_search_first_true(Int lo, IntPredicate pred) { // 000[1]11
    if (pred(lo)) {
        return lo;
    }
    Int step = 1;
    while (!pred(lo + step)) {
        step *= 2;
    }
    Int l = lo + step / 2, r = lo + step;
    while (l < r) {
        Int mid = l + (r - l) / 2;
        if (pred(mid)) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return l;
}

```

Example Usage

```

#include <cassert>

bool at_least_1000(int x) {
    return x >= 1000;
}

bool square_large_enough(long long x) {
    return x * x >= 123456789LL;
}

int main() {
    assert(exponential_search_first_true(0, at_least_1000) == 1000);
}

```

```

assert(exponential_search_first_true(5, at_least_1000) == 1000);
assert(exponential_search_first_true(0LL, square_large_enough) == 11112);
return 0;
}

```

3.2 Unimodal and Continuous Search

3.2.1 Ternary Search

Given a unimodal function $f(x)$ taking a single `double` argument, find its global maximum or minimum point to a specified absolute error.

- `ternary_search_min()` takes the domain $[lo, hi]$ of a continuous function $f(x)$ and returns a number x such that f is strictly decreasing on the interval $[lo, x]$ and strictly increasing on the interval $[x, hi]$. For the function to be correct and deterministic, such an x must exist and be unique.
- `ternary_search_max()` takes the domain $[lo, hi]$ of a continuous function $f(x)$ and returns a number x such that f is strictly increasing on the interval $[lo, x]$ and strictly decreasing on the interval $[x, hi]$. For the function to be correct and deterministic, such an x must exist and be unique.

Time Complexity: $O(\log n)$ calls will be made to $f()$, where n is the distance between `lo` and `hi` divided by the specified absolute error (epsilon).

Space Complexity: $O(1)$ auxiliary for both operations.

```

template<class UnimodalFunction>
double ternary_search_min(double lo, double hi, UnimodalFunction f, const double EPS = 1e-12) {
    double lthird, hthird;
    while (hi - lo > EPS) {
        lthird = lo + (hi - lo) / 3;
        hthird = hi - (hi - lo) / 3;
        if (f(lthird) < f(hthird)) {
            hi = hthird;
        } else {
            lo = lthird;
        }
    }
    return lo;
}

template<class UnimodalFunction>
double ternary_search_max(double lo, double hi, UnimodalFunction f, const double EPS = 1e-12) {
    double lthird, hthird;
    while (hi - lo > EPS) {
        lthird = lo + (hi - lo) / 3;
        hthird = hi - (hi - lo) / 3;
        if (f(lthird) < f(hthird)) {
            lo = lthird;
        } else {
            hi = hthird;
        }
    }
    return hi;
}

```

Example Usage

```

#include <cassert>
#include <cmath>

```

```

bool equal(double a, double b) {
    return fabs(a - b) < 1e-7;
}

// Parabola opening up with vertex at (-2, -24).
double f1(double x) {
    return 3 * x * x + 12 * x - 12;
}

// Parabola opening down with vertex at (2/19, 8366/95) ~ (0.105, 88.063).
double f2(double x) {
    return -5.7 * x * x + 1.2 * x + 88;
}

// Absolute value function shifted to the right by 30 units.
double f3(double x) {
    return fabs(x - 30);
}

int main() {
    assert(equal(ternary_search_min(-1000, 1000, f1), -2));
    assert(equal(ternary_search_max(-1000, 1000, f2), 2.0 / 19));
    assert(equal(ternary_search_min(-1000, 1000, f3), 30));
    return 0;
}

```

3.2.2 Hill Climbing

Given a continuous function $f(x, y)$ to `double` and a (possibly arbitrary) initial guess (x_0, y_0) , return a potential global minimum found through hill-climbing. Optionally, two `double` pointers `critical_x` and `critical_y` may be passed to store the input points to f at which the returned minimum value is attained.

Hill-climbing is a heuristic which starts at the guess, then considers taking a single step in each of a fixed number of directions. The direction with the best (in this case, minimum) value is chosen, and further steps are taken in it until the answer no longer improves. When this happens, the step size is reduced and the same process repeats until a desired absolute error is reached. The technique's success heavily depends on the behavior of f and the initial guess. Therefore, the result is not guaranteed to be the global minimum.

Time Complexity: $O(d \log n)$ call will be made to f , where d is the number of directions considered at each position and n is the search space that is approximately proportional to the maximum possible step size divided by the minimum possible step size.

Space Complexity: $O(1)$ auxiliary.

```

#include <cmath>
#include <cstdint>

template<class ContinuousFunction>
double find_min(
    ContinuousFunction f, double x0, double y0, double *critical_x = NULL,
    double *critical_y = NULL, const double STEP_MIN = 1e-9, const double STEP_MAX = 1e6,
    const int NUM_DIRECTIONS = 6
) {
    static const double PI = acos(-1.0);
    double x = x0, y = y0, res = f(x0, y0);
    for (double step = STEP_MAX; step > STEP_MIN;) {
        double best = res, best_x = x, best_y = y;
        bool found = false;
        for (int i = 0; i < NUM_DIRECTIONS; i++) {
            double a = 2.0 * PI * i / NUM_DIRECTIONS;
            double x2 = x + step * cos(a), y2 = y + step * sin(a);
            double value = f(x2, y2);
            if (best > value) {

```

```

        best_x = x2;
        best_y = y2;
        best = value;
        found = true;
    }
}
if (!found) {
    step /= 2.0;
} else {
    x = best_x;
    y = best_y;
    res = best;
}
}
if (critical_x != NULL && critical_y != NULL) {
    *critical_x = x;
    *critical_y = y;
}
return res;
}

```

Example Usage

```

#include <cassert>
#include <cmath>

bool eq(double a, double b) {
    return fabs(a - b) < 1e-8;
}

// Paraboloid with global minimum at f(2, 3) = 0.
double f(double x, double y) {
    return (x - 2) * (x - 2) + (y - 3) * (y - 3);
}

int main() {
    double x, y;
    assert(eq(find_min(f, 0, 0, &x, &y), 0));
    assert(eq(x, 2) && eq(y, 3));
    return 0;
}

```

3.2.3 Simulated Annealing

Given a function `f(x, y)` to minimize over two real variables, return a good approximate minimum found by simulated annealing. This is a randomized heuristic for difficult search spaces where gradients, convexity, and unimodality are not available.

At each temperature, the algorithm proposes a random nearby point. Better moves are always accepted, while worse moves are accepted with probability `exp((current - candidate) / temperature)`. This lets the search sometimes escape local minima early on, then become greedier as the temperature decreases.

Simulated annealing is not a proof of optimality. Its quality depends heavily on the objective function, the starting point, the initial temperature, the cooling rate, and the number of independent restarts.

- `anneal_min(f, x0, y0, &best_x, &best_y)` returns a small value found by the randomized search starting from `(x0, y0)`.
- `TEMPERATURE_START` controls the initial move scale and willingness to accept worse states.
- `TEMPERATURE_END` controls when the search stops.
- `COOLING_RATE` controls how quickly the temperature decreases.

Time Complexity: $O(r \log n)$ calls to `f()`, where r is the number of restarts and n is roughly `TEMPERATURE_START / TEMPERATURE_END`.

Space Complexity: $O(1)$ auxiliary.

```
#include <cmath>
#include <cstdlib>

const double TEMPERATURE_START = 10.0;
const double TEMPERATURE_END = 1e-7;
const double COOLING_RATE = 0.997;
const int NUM_RESTARTS = 8;

double random_unit() {
    return (double)rand() / RAND_MAX;
}

template<class Function>
double anneal_min(Function f, double x0, double y0, double *best_x = NULL, double *best_y = NULL) {
    double ans_x = x0, ans_y = y0, ans = f(x0, y0);
    for (int restart = 0; restart < NUM_RESTARTS; restart++) {
        double x = x0, y = y0, cur = f(x, y);
        double temperature = TEMPERATURE_START;
        while (temperature > TEMPERATURE_END) {
            double nx = x + (2.0 * random_unit() - 1.0) * temperature;
            double ny = y + (2.0 * random_unit() - 1.0) * temperature;
            double next = f(nx, ny);
            double probability = exp((cur - next) / temperature);
            if (next < cur || random_unit() < probability) {
                x = nx;
                y = ny;
                cur = next;
                if (cur < ans) {
                    ans = cur;
                    ans_x = x;
                    ans_y = y;
                }
            }
            temperature *= COOLING_RATE;
        }
    }

    if (best_x != NULL) {
        *best_x = ans_x;
    }
    if (best_y != NULL) {
        *best_y = ans_y;
    }
    return ans;
}
```

Example Usage

```
#include <cassert>

bool close(double a, double b) {
    return fabs(a - b) < 1e-3;
}

// Smooth bowl with global minimum at f(2, -3) = 0.
double f(double x, double y) {
    return (x - 2) * (x - 2) + (y + 3) * (y + 3);
}

int main() {
    srand(1);
```

```

double x, y;
double value = anneal_min(f, 0, 0, &x, &y);
assert(value < 1e-4);
assert(close(x, 2));
assert(close(y, -3));
return 0;
}

```

3.3 Dynamic Programming Optimization

3.3.1 Monotone Queue DP Optimization

Maintains the minimum or maximum value in a sliding window using a monotone queue. This is useful for dynamic programming recurrences where each transition may only come from one of the last w states, such as `dp[i] = a[i] + min(dp[j])` over `i - w <= j < i`.

The queue stores candidate (`index`, `value`) pairs in monotone order. Expired indices are removed from the front, and dominated values are removed from the back before inserting a new candidate.

- `monotone_queue<Compare>()` constructs an empty queue. Use `std::less<T>` for minimum queries and `std::greater<T>` for maximum queries.
- `push(index, value)` inserts candidate `value` at position `index`, removing dominated candidates.
- `expire(first_valid)` removes candidates with index less than `first_valid`.
- `top()` returns the best active (`index`, `value`) pair.
- `empty()` returns whether the queue has no active candidates.

Time Complexity:

- $O(n)$ for any sequence of n calls to `push(index, value)` and `expire(first_valid)`.
- $O(1)$ amortized per call to `push(index, value)` and `expire(first_valid)`.
- $O(1)$ worst-case per call to `top()` and `empty()`.

Space Complexity: $O(w)$ for a sliding window of width w .

```

#include <deque>
#include <functional>
#include <utility>

template<class T, class Compare>
class monotone_queue {
    typedef std::pair<int, T> entry;

    std::deque<entry> q;
    Compare better;

public:
    bool empty() const { return q.empty(); }

    void push(int index, const T &value) {
        while (!q.empty() && !better(q.back().second, value)) {
            q.pop_back();
        }
        q.push_back(entry(index, value));
    }

    void expire(int first_valid) {
        while (!q.empty() && q.front().first < first_valid) {
            q.pop_front();
        }
    }
}

```

```
entry top() const { return q.front(); }
};
```

Example Usage

```
#include <cassert>
#include <vector>
using namespace std;

int main() {
    int raw[] = {4, 2, 7, 1, 3, 6};
    vector<int> a(raw, raw + 6);

    monotone_queue<int, less<int>> minq;
    vector<int> window_min;
    for (int i = 0; i < (int)a.size(); i++) {
        minq.push(i, a[i]);
        minq.expire(i - 2);
        if (i >= 2) {
            window_min.push_back(minq.top().second);
        }
    }
    assert(window_min[0] == 2);
    assert(window_min[1] == 1);
    assert(window_min[2] == 1);
    assert(window_min[3] == 1);

    // dp[i] = a[i] + min(dp[j]) over max(0, i - 2) <= j < i.
    vector<int> dp(a.size());
    monotone_queue<int, less<int>> best;
    dp[0] = a[0];
    best.push(0, dp[0]);
    for (int i = 1; i < (int)a.size(); i++) {
        best.expire(i - 2);
        dp[i] = a[i] + best.top().second;
        best.push(i, dp[i]);
    }
    assert(dp[5] == 13);
    return 0;
}
```

3.3.2 Divide-and-Conquer DP Optimization

Computes one layer of a dynamic programming recurrence whose optimal transition indices are monotone. This optimization applies to recurrences of the form $\text{dp_cur}[i] = \min(\text{dp_prev}[k] + \text{cost}(k, i))$, where the minimizing index for i never decreases as i increases.

The function below computes $\text{dp_cur}[l..r]$ by evaluating the midpoint first, then recursing on the left half with a restricted upper bound on the optimal k , and on the right half with a restricted lower bound. The caller is responsible for checking that the recurrence satisfies the required monotonicity condition.

- `compute_dc_layer(dp_prev, dp_cur, l, r, opt_l, opt_r, cost)` fills $\text{dp_cur}[l..r]$ using candidate transition indices in $[\text{opt_l}, \text{opt_r}]$.
- `cost(k, i)` must return the transition cost from previous state k to current state i .

Time Complexity: $O(n \log n)$ calls to `cost(k, i)` per layer when each state has $O(n)$ possible transitions.

Space Complexity: $O(\log n)$ auxiliary stack space.

```
#include <algorithm>
#include <vector>
```

```

const long long INF = (1LL << 62);

template<class CostFunction>
void compute_dc_layer(
    const std::vector<long long> &dp_prev, std::vector<long long> &dp_cur, int l, int r, int opt_l,
    int opt_r, CostFunction cost
) {
    if (l > r) {
        return;
    }
    int mid = (l + r) / 2;
    long long best = INF;
    int best_k = opt_l;
    int upper = std::min(mid, opt_r);
    for (int k = opt_l; k <= upper; k++) {
        long long candidate = dp_prev[k] + cost(k, mid);
        if (candidate < best) {
            best = candidate;
            best_k = k;
        }
    }
    dp_cur[mid] = best;
    compute_dc_layer(dp_prev, dp_cur, l, mid - 1, opt_l, best_k, cost);
    compute_dc_layer(dp_prev, dp_cur, mid + 1, r, best_k, opt_r, cost);
}

```

Example Usage

```

#include <cassert>
using namespace std;

struct square_segment_cost {
    vector<long long> prefix;

    explicit square_segment_cost(const vector<int> &a) : prefix(a.size() + 1) {
        for (int i = 0; i < (int)a.size(); i++) {
            prefix[i + 1] = prefix[i] + a[i];
        }
    }

    long long operator()(int k, int i) const {
        long long sum = prefix[i] - prefix[k];
        return sum * sum;
    }
};

int main() {
    int raw[] = {1, 2, 3, 4};
    vector<int> a(raw, raw + 4);
    int n = a.size();
    square_segment_cost cost(a);

    vector<long long> dp_prev(n + 1, INF), dp_cur(n + 1, INF);
    dp_prev[0] = 0;
    compute_dc_layer(dp_prev, dp_cur, 1, n, 0, n - 1, cost);
    assert(dp_cur[4] == 100); // One group: (1 + 2 + 3 + 4)^2.

    vector<long long> dp_two(n + 1, INF);
    compute_dc_layer(dp_cur, dp_two, 1, n, 0, n - 1, cost);
    assert(dp_two[4] == 52); // Split as [1, 2] and [3, 4].
    return 0;
}

```


3.3.3 Knuth DP Optimization

Computes interval dynamic programming recurrences whose optimal split points are monotone. Knuth optimization applies to recurrences of the form $dp[l][r] = \min(dp[l][k] + dp[k][r]) + cost(l, r)$, where the optimal split $opt[l][r]$ satisfies $opt[l][r - 1] \leq opt[l][r] \leq opt[l + 1][r]$.

Common applications include optimal binary search trees and some range merging problems. The caller is responsible for verifying the quadrangle inequality and monotonicity assumptions for the chosen $cost(l, r)$.

- `knuth_interval_dp(n, cost, &opt)` computes minimum costs for all half-open intervals $[l, r)$ over n items.
- `cost(l, r)` must return the interval cost added after choosing the best split.
- If `opt` is not `NULL`, it is filled with the chosen split points.

Time Complexity: $O(n^2)$ calls to `cost(l, r)` and $O(n^2)$ candidate split checks.

Space Complexity: $O(n^2)$ for the `dp` and `opt` tables.

```
#include <algorithm>
#include <cstdint>
#include <vector>

const long long INF = (1LL << 62);

template<class CostFunction>
std::vector<std::vector<long long>> knuth_interval_dp(
    int n, CostFunction cost, std::vector<std::vector<int>> *opt_out = NULL
) {
    std::vector<std::vector<long long>> dp(n + 1, std::vector<long long>(n + 1, 0));
    std::vector<std::vector<int>> opt(n + 1, std::vector<int>(n + 1, 0));
    for (int i = 0; i <= n; i++) {
        opt[i][i] = i;
        if (i < n) {
            opt[i][i + 1] = i + 1;
        }
    }
    for (int len = 2; len <= n; len++) {
        for (int l = 0; l + len <= n; l++) {
            int r = l + len;
            dp[l][r] = INF;
            int start = opt[l][r - 1];
            int finish = opt[l + 1][r];
            if (start < l + 1) {
                start = l + 1;
            }
            if (finish > r - 1) {
                finish = r - 1;
            }
            for (int k = start; k <= finish; k++) {
                long long candidate = dp[l][k] + dp[k][r] + cost(l, r);
                if (candidate < dp[l][r]) {
                    dp[l][r] = candidate;
                    opt[l][r] = k;
                }
            }
        }
    }
    if (opt_out != NULL) {
        *opt_out = opt;
    }
    return dp;
}
```

Example Usage

```
#include <cassert>
```

```
using namespace std;

struct merge_cost {
    vector<long long> prefix;

    explicit merge_cost(const vector<int> &a) : prefix(a.size() + 1) {
        for (int i = 0; i < (int)a.size(); i++) {
            prefix[i + 1] = prefix[i] + a[i];
        }
    }

    long long operator()(int l, int r) const { return prefix[r] - prefix[l]; }
};

int main() {
    int raw[] = {1, 2, 3, 4};
    vector<int> a(raw, raw + 4);
    vector<vector<long long>> dp = knuth_interval_dp(a.size(), merge_cost(a));
    assert(dp[0][4] == 19);
    return 0;
}
```

3.3.4 Convex Hull Trick (Semi-Dynamic)

Given a set of pairs (m, b) specifying lines of the form $y = mx + b$, process a set of x-coordinate queries each asking to find the minimum y-value when any of the given lines are evaluated at the specified x. This is useful for dynamic programming recurrences of the form $dp[i] = \min(m[j] * x[i] + b[j])$.

The following implementation is a concise, semi-dynamic version of the convex hull optimization technique. It supports an interlaced sequence of `add_line()` and `query()` calls, as long as the preconditions of descending `m` and ascending `x` are satisfied. As a result, it may be necessary to sort the lines and queries before calling the functions. In that case, the overall time complexity will be dominated by the sorting step.

- `add_line(m, b)` inserts line $y = mx + b$. The slope `m` must be less than or equal to the slope of every line added so far.
- `query(x)` returns the minimum y-value among all inserted lines at coordinate `x`. Query coordinates must be nondecreasing across calls.

Time Complexity: $O(n)$ for any interlaced sequence of `add_line()` and `query()` calls, where n is the number of lines added. This is because the overall number of steps taken by `add_line()` and `query()` are respectively bounded by the number of lines. Thus a single call to either `add_line()` or `query()` will have an amortized $O(1)$ running time.

Space Complexity:

- $O(n)$ for storage of the lines.
- $O(1)$ auxiliary for `add_line()` and `query()`.

```
#include <vector>

std::vector<long long> M, B;
int ptr = 0;

void add_line(long long m, long long b) {
    int len = M.size();
    while (len > 1 && (B[len - 2] - B[len - 1]) * (m - M[len - 1]) >=
              (B[len - 1] - b) * (M[len - 1] - M[len - 2])) {
        len--;
    }
    M.resize(len);
    B.resize(len);
    M.push_back(m);
    B.push_back(b);
}
```

```

    B.push_back(b);
}

long long query(long long x) {
    if (ptr >= (int)M.size()) {
        ptr = (int)M.size() - 1;
    }
    while (ptr + 1 < (int)M.size() && M[ptr + 1] * x + B[ptr + 1] <= M[ptr] * x + B[ptr]) {
        ptr++;
    }
    return M[ptr] * x + B[ptr];
}

```

Example Usage

```

#include <cassert>

int main() {
    add_line(3, 0);
    add_line(2, 1);
    add_line(1, 2);
    add_line(0, 6);
    assert(query(0) == 0);
    assert(query(1) == 3);
    assert(query(2) == 4);
    assert(query(3) == 5);
    return 0;
}

```

3.3.5 Convex Hull Trick (Fully-Dynamic)

Given a set of pairs (m, b) specifying lines of the form $y = mx + b$, process a set of x-coordinate queries each asking to find the minimum y-value when any of the given lines are evaluated at the specified x. To instead have the queries optimize for maximum y-value, call the constructor with `query_max = true`. This is useful for dynamic programming recurrences of the form `dp[i] = min(m[j] * x[i] + b[j])` when line slopes and query coordinates are not monotone.

The following implementation is a fully dynamic variant of the convex hull optimization technique, using a self-balancing binary search tree (`std::set`) to support the ability to call `add_line()` and `query()` in any desired order.

- `hull_optimizer(query_max)` constructs an empty hull. By default, `query(x)` minimizes; if `query_max` is true, `query(x)` maximizes.
- `add_line(m, b)` inserts line $y = mx + b$ in arbitrary order.
- `query(x)` returns the best y-value among all inserted lines at coordinate `x`. At least one line must have been inserted.

Time Complexity: $O(\log n)$ amortized per call to `add_line()` and $O(\log n)$ per call to `query()`, where n is the number of lines added.

Space Complexity:

- $O(n)$ for storage of the lines.
- $O(1)$ auxiliary for `add_line()` and `query()`.

```

#include <cassert>
#include <climits>
#include <set>

class hull_optimizer {

```

```

struct line {
    long long m, b;
    mutable long long xhi;
    bool is_query;

    line(long long m, long long b, long long xhi = 0, bool is_query = false)
        : m(m), b(b), xhi(xhi), is_query(is_query) {}

    bool operator<(const line &l) const { return l.is_query ? xhi < l.xhi : m < l.m; }
};

std::multiset<line> hull;
bool query_max;

typedef std::multiset<line>::iterator hulliter;

static long long div_floor(long long a, long long b) { return a / b - ((a ^ b) < 0 && a % b); }

bool update_border(hulliter x, hulliter y) {
    if (y == hull.end()) {
        x->xhi = LLONG_MAX;
        return false;
    }
    if (x->m == y->m) {
        x->xhi = (x->b > y->b) ? LLONG_MAX : LLONG_MIN;
    } else {
        x->xhi = div_floor(y->b - x->b, x->m - y->m);
    }
    return x->xhi >= y->xhi;
}

public:
    hull_optimizer(bool query_max = false) : query_max(query_max) {}

    void add_line(long long m, long long b) {
        if (!query_max) {
            m = -m;
            b = -b;
        }
        hulliter z = hull.insert(line(m, b));
        hulliter y = z++;
        hulliter x = y;
        while (update_border(y, z)) {
            z = hull.erase(z);
        }
        if (x != hull.begin() && update_border(--x, y)) {
            update_border(x, y = hull.erase(y));
        }
        while ((y = x) != hull.begin() && (--x)->xhi >= y->xhi) {
            update_border(x, hull.erase(y));
        }
    }

    long long query(long long x) const {
        assert(!hull.empty());
        line q(0, 0, x, true);
        hulliter it = hull.lower_bound(q);
        long long res = it->m * x + it->b;
        return query_max ? res : -res;
    }
};

```

Example Usage

```
#include <cassert>
```

```
int main() {
    hull_optimizer h;
    h.add_line(3, 0);
    h.add_line(0, 6);
    h.add_line(1, 2);
    h.add_line(2, 1);
    assert(h.query(0) == 0);
    assert(h.query(2) == 4);
    assert(h.query(1) == 3);
    assert(h.query(3) == 5);
    return 0;
}
```

3.3.6 Li Chao Tree

Maintains a dynamic set of lines and answers minimum value queries at points in a fixed integer domain. A Li Chao tree is useful for dynamic programming recurrences of the form $dp[i] = \min(m[j] * x[i] + b[j])$, especially when line slopes and query points arrive in arbitrary order.

The implementation below stores the lower envelope of lines over the inclusive domain $[x_MIN, x_MAX]$. It supports adding arbitrary lines and querying arbitrary integer x-coordinates in $O(\log C)$, where $C = x_MAX - x_MIN + 1$.

- `li_chao_tree(lo, hi)` constructs an empty tree over integer domain $[lo, hi]$.
- `add_line(m, b)` inserts line $y = mx + b$.
- `query(x)` returns the minimum y-value among all inserted lines at coordinate x .

Time Complexity: $O(\log C)$ per call to `add_line(m, b)` and `query(x)`, where C is the domain size.

Space Complexity:

- $O(n \log C)$ worst-case node storage for n inserted lines, usually much less.
- $O(\log C)$ auxiliary stack space per operation.

```
#include <algorithm>
#include <cassert>
#include <limits>

const long long INF = std::numeric_limits<long long>::max() / 4;

struct line {
    long long m, b;

    line(long long m = 0, long long b = INF) : m(m), b(b) {}

    long long eval(long long x) const { return m * x + b; }
};

class li_chao_tree {
    struct node {
        line f;
        node *left, *right;

        node(const line &f) : f(f), left(NULL), right(NULL) {}
    };

    node *root;
    long long lo, hi;

    static void clean_up(node *n) {
        if (n == NULL) {
            return;
        }
        clean_up(n->left);
        clean_up(n->right);
    }
};
```

```

    delete n;
}

static void add_line(node *&n, long long l, long long r, line f) {
    if (n == NULL) {
        n = new node(f);
        return;
    }
    long long mid = l + (r - l) / 2;
    bool left_better = f.eval(l) < n->f.eval(l);
    bool mid_better = f.eval(mid) < n->f.eval(mid);
    if (mid_better) {
        std::swap(f, n->f);
    }
    if (l == r) {
        return;
    }
    if (left_better != mid_better) {
        add_line(n->left, l, mid, f);
    } else {
        add_line(n->right, mid + 1, r, f);
    }
}

static long long query(node *n, long long l, long long r, long long x) {
    if (n == NULL) {
        return INF;
    }
    long long res = n->f.eval(x);
    if (l == r) {
        return res;
    }
    long long mid = l + (r - l) / 2;
    if (x <= mid) {
        return std::min(res, query(n->left, l, mid, x));
    }
    return std::min(res, query(n->right, mid + 1, r, x));
}

public:
    li_chao_tree(long long lo, long long hi) : root(NULL), lo(lo), hi(hi) {}

    ~li_chao_tree() { clean_up(root); }

    void add_line(long long m, long long b) { add_line(root, lo, hi, line(m, b)); }

    long long query(long long x) const {
        assert(lo <= x && x <= hi);
        return query(root, lo, hi, x);
    }
};

```

Example Usage

```

#include <cassert>

int main() {
    li_chao_tree cht(-10, 10);
    cht.add_line(3, 0);
    cht.add_line(1, 2);
    cht.add_line(-1, 10);
    assert(cht.query(0) == 0);
    assert(cht.query(2) == 4);
    assert(cht.query(10) == 0);
    return 0;
}

```

3.4 Linear and Convex Optimization

3.4.1 Linear Programming (Simplex)

Solves a linear programming problem using Dantzig's simplex algorithm. The canonical form of a linear programming problem is to maximize (or minimize) the dot product cx , subject to $ax \leq b$ and $x \geq 0$, where x is a vector of unknowns to be solved, c is a vector of coefficients, a is a matrix of linear equation coefficients, and b is a vector of boundary coefficients.

- `simplex_solve(a, b, c, &x)` solves the linear programming problem for an m by n matrix `a` of real values, a length m vector `b`, and a length n vector `c`, returning 0 if a solution was found or -1 if there are no solutions. If a solution is found, then the vector pointed to by `x` is populated with the solution vector of length n .

Time Complexity: Polynomial (average) on the number of equations and unknowns, but exponential in the worst case.

Space Complexity: $O(m \cdot n)$ auxiliary heap space.

```
#include <cmath>
#include <limits>
#include <vector>

template<class Matrix>
int simplex_solve(
    const Matrix &a, const std::vector<double> &b, const std::vector<double> &c,
    std::vector<double> *x, const bool MAXIMIZE = true, const double EPS = 1e-10
) {
    int m = a.size(), n = c.size();
    Matrix t(m + 2, std::vector<double>(n + 2));
    t[1][1] = 0;
    for (int j = 1; j <= n; j++) {
        t[1][j + 1] = MAXIMIZE ? c[j - 1] : -c[j - 1];
    }
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            t[i + 1][j + 1] = -a[i - 1][j - 1];
        }
        t[i + 1][1] = b[i - 1];
    }
    for (int j = 1; j <= n; j++) {
        t[0][j + 1] = j;
    }
    for (int i = n + 1; i <= m + n; i++) {
        t[i - n + 1][0] = i;
    }
    double p1 = 0, p2 = 0;
    bool done = true;
    do {
        double mn = std::numeric_limits<double>::max(), xmax = 0, v;
        for (int j = 2; j <= n + 1; j++) {
            if (t[1][j] > 0 && t[1][j] > xmax) {
                p2 = j;
                xmax = t[1][j];
            }
        }
        for (int i = 2; i <= m + 1; i++) {
            v = fabs(t[i][1] / t[i][p2]);
            if (t[i][p2] < 0 && mn > v) {
                mn = v;
                p1 = i;
            }
        }
        std::swap(t[p1][0], t[0][p2]);
    } while (!done);
}
```

```

for (int i = 1; i <= m + 1; i++) {
    if (i != p1) {
        for (int j = 1; j <= n + 1; j++) {
            if (j != p2) {
                t[i][j] -= t[p1][j] * t[i][p2] / t[p1][p2];
            }
        }
    }
}
t[p1][p2] = 1.0 / t[p1][p2];
for (int j = 1; j <= n + 1; j++) {
    if (j != p2) {
        t[p1][j] *= fabs(t[p1][p2]);
    }
}
for (int i = 1; i <= m + 1; i++) {
    if (i != p1) {
        t[i][p2] *= t[p1][p2];
    }
}
for (int i = 2; i <= m + 1; i++) {
    if (t[i][1] < 0) {
        return -1;
    }
}
done = true;
for (int j = 2; j <= n + 1; j++) {
    if (t[1][j] > 0) {
        done = false;
    }
}
} while (!done);
x->clear();
for (int j = 1; j <= n; j++) {
    for (int i = 2; i <= m + 1; i++) {
        if (fabs(t[i][0] - j) < EPS) {
            x->push_back(t[i][1]);
        }
    }
}
return 0;
}

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    // Solve [x, y] that maximizes 3x + 4y, subject to x, y >= 0 and:
    // -2x + 1y <= 0
    // 1x + 0.85y <= 9
    // 1x + 2y <= 14
    const int equations = 3, unknowns = 2;
    double a[equations][unknowns] = {{-2, 1}, {1, 0.85}, {1, 2}};
    double b[equations] = {0, 9, 14};
    double c[unknowns] = {3, 4};
    vector<vector<double>> va(equations, vector<double>(unknowns));
    vector<double> vb(b, b + equations), vc(c, c + unknowns), x;
    for (int i = 0; i < equations; i++) {
        for (int j = 0; j < unknowns; j++) {
            va[i][j] = a[i][j];
        }
    }
    assert(simplex_solve(va, vb, vc, &x) == 0);
}

```



```

double maxval = 0;
for (int i = 0; i < (int)x.size(); i++) {
    maxval += c[i] * x[i];
}
cout << "Solution = " << maxval << " at (" << x[0];
for (int i = 1; i < (int)x.size(); i++) {
    cout << ", " << x[i];
}
cout << ")." << endl;
return 0;
}

```

Example Output

Solution = 33.3043 at (5.30435, 4.34783).

3.4.2 Hungarian Algorithm

Solves the minimum-cost assignment problem for a rectangular cost matrix. Given n workers and m jobs with $n \leq m$, choose a distinct job for each worker so that the total assignment cost is minimized.

The implementation below is the classical Hungarian algorithm using potentials. It is 1-indexed internally, but the input matrix `cost` is 0-indexed. The returned assignment vector has length n , where `assignment[i]` is the chosen job for worker `i`, using 0-indexed job numbers.

- `hungarian(cost, &assignment)` returns the minimum total assignment cost.
- `cost[i][j]` is the cost of assigning worker `i` to job `j`.

Time Complexity: $O(n^2 \cdot m)$, where `cost` has n rows and m columns.

Space Complexity: $O(n + m)$ auxiliary heap space, not counting the input matrix and returned assignment.

```

#include <algorithm>
#include <cassert>
#include <limits>
#include <vector>

long long hungarian(const std::vector<std::vector<long long>> &cost, std::vector<int> *assignment) {
    int n = cost.size(), m = cost.empty() ? 0 : cost[0].size();
    assert(n <= m);
    const long long INF = std::numeric_limits<long long>::max() / 4;
    std::vector<long long> u(n + 1), v(m + 1);
    std::vector<int> p(m + 1), way(m + 1);

    for (int i = 1; i <= n; i++) {
        p[0] = i;
        int j0 = 0;
        std::vector<long long> minv(m + 1, INF);
        std::vector<char> used(m + 1, false);
        do {
            used[j0] = true;
            int i0 = p[j0], j1 = 0;
            long long delta = INF;
            for (int j = 1; j <= m; j++) {
                if (used[j]) {
                    continue;
                }
                long long cur = cost[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < minv[j]) {
                    minv[j] = cur;
                    way[j] = j0;
                }
            }
            if (minv[j1] < delta) {
                delta = minv[j1];
                j1 = j1;
            }
        } while (delta > 0);
        for (int j = j1; j > 0; j = way[j]) {
            p[j] = i0;
            i0 = p[j0];
        }
        j0 = j1;
    }
    for (int i = 1; i <= n; i++) {
        assignment[i - 1] = p[i];
    }
}

```

```

    }
}
for (int j = 0; j <= m; j++) {
    if (used[j]) {
        u[p[j]] += delta;
        v[j] -= delta;
    } else {
        minv[j] -= delta;
    }
}
j0 = j1;
} while (p[j0] != 0);

do {
    int j1 = way[j0];
    p[j0] = p[j1];
    j0 = j1;
} while (j0 != 0);
}

assignment->assign(n, -1);
for (int j = 1; j <= m; j++) {
    if (p[j] != 0) {
        (*assignment)[p[j] - 1] = j - 1;
    }
}
return -v[0];
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    long long raw[3][4] = {
        {9, 2, 7, 8},
        {6, 4, 3, 7},
        {5, 8, 1, 8},
    };
    vector<vector<long long>> cost(3, vector<long long>(4));
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            cost[i][j] = raw[i][j];
        }
    }
    vector<int> assignment;
    assert(hungarian(cost, &assignment) == 9);
    assert(assignment[0] == 1);
    assert(assignment[1] == 0);
    assert(assignment[2] == 2);
    return 0;
}

```

Chapter 4

Strings

4.1 String Utilities

Common string functions, many of which are substitutes for features which are not available in standard C++, or may not be available on compilers that do not support C++11 and later. These operations are naive implementations and often depend on certain `std::string` functions that have unspecified complexity.

```
#include <cctype>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>
using std::string;
```

Integer Conversion:

- `to_str(i)` returns the string representation of integer `i`, much like `std::to_string()` in C++11 and later.
- `to_int(s)` returns the integer representation of string `s`, much like `atoi()`, except handling special cases of overflow by throwing an exception.
- `itoa(value, &str, base)` implements the non-standard C function which converts `value` into a C string, storing it into pointer `str` in the given `base`. For more generalized base conversion, see the math utilities section.

```
template<class Int>
string to_str(Int i) {
    std::ostringstream oss;
    oss << i;
    return oss.str();
}

int to_int(const string &s) {
    std::istringstream iss(s);
    int res;
    if (!(iss >> res)) {
        throw std::runtime_error("to_int failed");
    }
    return res;
}

char *itoa(int value, char *str, int base = 10) {
    if (base < 2 || base > 36) {
        *str = '\0';
        return str;
    }
```

```

}
char *ptr = str, *ptr1 = str, tmp_c;
int tmp_v;
do {
    tmp_v = value;
    value /= base;
    *ptr++ =
        "zyxwvutsrqponmlkjihgfedcba9876543210123456789"
        "abcdefghijklmnopqrstuvwxyz"[35 + (tmp_v - value * base)];
} while (value);
if (tmp_v < 0) {
    *ptr++ = '-';
}
for (*ptr-- = '\0'; ptr1 < ptr; *ptr1++ = tmp_c) {
    tmp_c = *ptr;
    *ptr-- = *ptr1;
}
return str;
}

```

Case Conversion:

- `to_upper(s)` returns `s` with all alphabetical characters converted to uppercase.
- `to_lower(s)` returns `s` with all alphabetical characters converted to lowercase.
- `to_title(s)` returns the title case representation of string `s`, where the first letter of every word (consecutive alphabetical characters) is capitalized.

```

string to_upper(const string &s) {
    string res;
    for (int i = 0; i < (int)s.size(); i++) {
        res.push_back(toupper(s[i]));
    }
    return res;
}

```

```

string to_lower(const string &s) {
    string res;
    for (int i = 0; i < (int)s.size(); i++) {
        res.push_back(tolower(s[i]));
    }
    return res;
}

```

```

string to_title(const string &s) {
    string res;
    char prev = '\0';
    for (int i = 0; i < (int)s.size(); i++) {
        if (isalpha(prev)) {
            res.push_back(tolower(s[i]));
        } else {
            res.push_back(toupper(s[i]));
        }
        prev = res.back();
    }
    return res;
}

```

Stripping:

- `lstrip(s)` strips the left side of `s` in-place (that is, the input is modified) using the given delimiters and returns a reference to the stripped string.
- `rstrip(s)` strips the right side of `s` in-place using the given delimiters and returns a reference to the stripped string.
- `strip(s)` strips both sides of `s` in-place and returns a reference to the stripped string.

```

string &lstrip(string &s, const string &delim = " \n\t\v\f\r") {
    size_t pos = s.find_first_not_of(delim);
    if (pos != string::npos) {
        s.erase(0, pos);
    }
    return s;
}

string &rstrip(string &s, const string &delim = " \n\t\v\f\r") {
    size_t pos = s.find_last_not_of(delim);
    if (pos != string::npos) {
        s.erase(pos);
    }
    return s;
}

string &strip(string &s, const string &delim = " \n\t\v\f\r") {
    return lstrip(rstrip(s));
}

```

Find and Replace:

- `find_all(haystack, needle)` returns a vector of all positions where the string `needle` appears in the string `haystack`.
- `replace(s, old, replacement)` returns a copy of `s` with all occurrences of the string `old` replaced with the given `replacement`.

```

std::vector<int> find_all(const string &haystack, const string &needle) {
    std::vector<int> res;
    size_t pos = haystack.find(needle, 0);
    while (pos != string::npos) {
        res.push_back(pos);
        pos = haystack.find(needle, pos + 1);
    }
    return res;
}

string replace(const string &s, const string &old, const string &replacement) {
    if (old.empty()) {
        return s;
    }
    string res(s);
    size_t pos = 0;
    while ((pos = res.find(old, pos)) != string::npos) {
        res.replace(pos, old.length(), replacement);
        pos += replacement.length();
    }
    return res;
}

```

Joining and Splitting:

- `join(v, delim)` returns the strings in vector `v` concatenated, separated by the given delimiter.
- `split(s, char delim)` returns a vector of tokens of `s`, split on a single character delimiter. Note that this version will not skip empty tokens. For example, `split("a::b", ":")` returns `{"a", "b"}`, not `{"a", "", "b"}`.
- `split(s, string delim)` returns a vector of tokens of `s`, split on a set of many possible single character delimiters. All characters of `delim` will be removed from `s`, and the remaining token(s) of `s` will be added sequentially to a vector and returned. Unlike the first version, empty tokens are skipped. For example, `split("a::b", ":")` returns `{"a", "b"}`, not `{"a", "", "b"}`.
- `explode(s, delim)` returns a vector of tokens of `s`, split on the entire delimiter string `delim`. Unlike the `split()` functions above, `delim` is treated as a contiguous boundary string, not merely a set of possible boundary characters. This will not skip empty tokens. For example, `explode("a:::b", "::")` yields

```
{ "a", "", "b" }, not { "a", "b" }.
```

```
string join(const std::vector<string> &v, const string &delim = " ") {
    string res;
    for (int i = 0; i < (int)v.size(); i++) {
        if (i > 0) {
            res += delim;
        }
        res += v[i];
    }
    return res;
}

std::vector<string> split(const string &s, char delim) {
    std::vector<string> res;
    std::stringstream ss(s);
    string curr;
    while (std::getline(ss, curr, delim)) {
        res.push_back(curr);
    }
    return res;
}

std::vector<string> split(const string &s, const string &delim = " \n\t\v\f\r") {
    std::vector<string> res;
    string curr;
    for (int i = 0; i < (int)s.size(); i++) {
        if (delim.find(s[i]) == string::npos) {
            curr += s[i];
        } else if (!curr.empty()) {
            res.push_back(curr);
            curr = "";
        }
    }
    if (!curr.empty()) {
        res.push_back(curr);
    }
    return res;
}

std::vector<string> explode(const string &s, const string &delim) {
    std::vector<string> res;
    size_t last = 0, next = 0;
    while ((next = s.find(delim, last)) != string::npos) {
        res.push_back(s.substr(last, (int)next - last));
        last = next + delim.size();
    }
    res.push_back(s.substr(last));
    return res;
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    assert(to_str(123) + "4" == "1234");
    assert(to_int("1234") == 1234);
    char buffer[50];
    assert(string(itoa(1750, buffer, 10)) == "1750");
    assert(string(itoa(1750, buffer, 16)) == "6d6");
    assert(string(itoa(1750, buffer, 2)) == "11011010110");

    assert(to_upper("Hello world") == "HELLO WORLD");
    assert(to_lower("Hello World") == "hello world");
}
```

```

assert(to_title("hello world") == "Hello World");

string s("  abc \n");
string t = s;
assert(lstrip(s) == "abc \n");
assert(rstrip(s) == strip(t));

vector<int> pos;
pos.push_back(0);
pos.push_back(7);
assert(find_all("abracadabra", "ab") == pos);
assert(replace("abcdabba", "ab", "00") == "00cd00ba");

assert(join(split("\n\ncde\nf", '\n'), "|") == "a|b|cde|f"); // split v1
assert(join(split("a::b,cde:,f", ":", "|"), "|") == "a|b|cde|f"); // split v2
assert(join(explode("a..b.cde...f", ".."), "|") == "a|b.cde|f");
return 0;
}

```

4.2 Hashing

4.2.1 Hash Functions

Provides a small library of non-cryptographic hash functions and hash functors for common contest keys. These helpers are useful for fingerprinting values, combining keys, seeding randomized algorithms, and defining `std::unordered_map` keys for pairs and vectors. They are not suitable for passwords, signatures, or adversarial security.

The standalone functions below are deterministic. The `int_hasher` functor adds a per-run random seed before mixing, which is useful for making integer-keyed hash tables harder to hack in open-test contests.

- `mix32(x)` mixes a 32-bit unsigned integer `x`, using the MurmurHash3 finalizer.
- `mix64(x)` mixes a 64-bit unsigned integer `x`, using the SplitMix64 finalizer.
- `hash32(x)` and `hash64(x)` hash any integer type that can be converted to `unsigned int` or `uint64`, respectively.
- `hash_combine32(h, x)` and `hash_combine64(h, x)` fold another already-hashed value into an existing hash accumulator.
- `hash_float(x)` and `hash_double(x)` hash floating-point bit patterns, normalizing `-0.0` to `0.0`. Different NaN representations may still hash differently.
- `hash_string_fnv1a32(s)` and `hash_string_fnv1a64(s)` compute FNV-1a hashes of string `s`.
- `hash_range32(first, last)` and `hash_range64(first, last)` hash a sequence of integer-like values.
- `int_hasher<Int>`, `pair_hasher<A, B>`, and `vector_hasher<T>` are hash functors for `std::unordered_map` and `std::unordered_set`.
- `generic_hasher<T>` recursively handles integer, pair, and vector keys, and falls back to `std::hash<T>` for other hashable types.

Time Complexity:

- $O(1)$ per call to the integer, combiner, and floating-point hash functions.
- $O(n)$ per call to string and range hashing functions, where n is the number of elements processed.
- $O(n)$ per call to `vector_hasher<T>::operator()`, where n is the vector size.

Space Complexity: $O(1)$ auxiliary.

```

#include <chrono>
#include <cstdint>
#include <cstring>
#include <functional>
#include <string>

```

```

#include <type_traits>
#include <unordered_map>
#include <utility>
#include <vector>

typedef unsigned long long uint64;

unsigned int mix32(unsigned int x) {
    x ^= x >> 16;
    x *= 0x85ebca6bU;
    x ^= x >> 13;
    x *= 0xc2b2ae35U;
    return x ^ (x >> 16);
}

uint64 mix64(uint64 x) {
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

template<class Int>
unsigned int hash32(Int x) {
    return mix32((unsigned int)x);
}

template<class Int>
uint64 hash64(Int x) {
    return mix64((uint64)x);
}

unsigned int hash_combine32(unsigned int h, unsigned int x) {
    return mix32(h ^ (x + 0x9e3779b9U + (h << 6) + (h >> 2)));
}

uint64 hash_combine64(uint64 h, uint64 x) {
    return mix64(h ^ (x + 0x9e3779b97f4a7c15ULL + (h << 6) + (h >> 2)));
}

unsigned int hash_float(float x) {
    if (x == 0.0f) {
        x = 0.0f; // Normalize -0.0.
    }
    unsigned int bits = 0;
    std::memcpy(&bits, &x, sizeof(bits));
    return mix32(bits);
}

uint64 hash_double(double x) {
    if (x == 0.0) {
        x = 0.0; // Normalize -0.0.
    }
    uint64 bits = 0;
    std::memcpy(&bits, &x, sizeof(bits));
    return mix64(bits);
}

unsigned int hash_string_fnv1a32(const std::string &s) {
    unsigned int h = 2166136261U;
    for (int i = 0; i < (int)s.size(); i++) {
        h ^= (unsigned char)s[i];
        h *= 16777619U;
    }
    return h;
}

uint64 hash_string_fnv1a64(const std::string &s) {

```



```

uint64 h = 14695981039346656037ULL;
for (int i = 0; i < (int)s.size(); i++) {
    h ^= (unsigned char)s[i];
    h *= 1099511628211ULL;
}
return h;
}

template<class It>
unsigned int hash_range32(It first, It last) {
    unsigned int h = 0;
    for (It it = first; it != last; ++it) {
        h = hash_combine32(h, hash32(*it));
    }
    return h;
}

template<class It>
uint64 hash_range64(It first, It last) {
    uint64 h = 0;
    for (It it = first; it != last; ++it) {
        h = hash_combine64(h, hash64(*it));
    }
    return h;
}

template<class Int>
struct int_hasher {
    std::size_t operator()(Int x) const {
        static const uint64 RANDOM = std::chrono::steady_clock::now().time_since_epoch().count();
        return (std::size_t)mix64((uint64)x + RANDOM);
    }
};

template<class T>
struct generic_hasher;

template<class T, bool IsInteger>
struct scalar_hasher {
    std::size_t operator()(const T &x) const { return std::hash<T>()(x); }
};

template<class T>
struct scalar_hasher<T, true> {
    std::size_t operator()(T x) const { return int_hasher<T>()(x); }
};

template<class A, class B>
struct pair_hasher {
    std::size_t operator()(const std::pair<A, B> &p) const {
        uint64 h = 0;
        h = hash_combine64(h, (uint64)generic_hasher<A>()(p.first));
        h = hash_combine64(h, (uint64)generic_hasher<B>()(p.second));
        return (std::size_t)h;
    }
};

template<class T>
struct vector_hasher {
    std::size_t operator()(const std::vector<T> &v) const {
        uint64 h = 0;
        for (int i = 0; i < (int)v.size(); i++) {
            h = hash_combine64(h, (uint64)generic_hasher<T>()(v[i]));
        }
        return (std::size_t)h;
    }
};

```

```

template<class T>
struct generic_hasher : scalar_hasher<T, std::is_integral<T>::value> {};

template<class A, class B>
struct generic_hasher<std::pair<A, B>> : pair_hasher<A, B> {};

template<class T>
struct generic_hasher<std::vector<T>> : vector_hasher<T> {};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    assert(mix32(123U) == mix32(123U));
    assert(mix32(123U) != mix32(124U));
    assert(hash32(-1) == hash32(-1));
    assert(hash64(123) == mix64(123ULL));
    assert(hash_float(-0.0f) == hash_float(0.0f));
    assert(hash_double(-0.0) == hash_double(0.0));
    assert(hash_string_fnv1a32("abc") == hash_string_fnv1a32("abc"));
    assert(hash_string_fnv1a64("abc") != hash_string_fnv1a64("acb"));

    vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    assert(hash_range32(v.begin(), v.end()) == hash_range32(v.begin(), v.end()));
    assert(hash_range64(v.begin(), v.end()) == hash_range64(v.begin(), v.end()));

    unordered_map<long long, int, int_hasher<long long>> count;
    count[1000000000000LL] = 7;
    assert(count[1000000000000LL] == 7);

    typedef pair<int, int> point;
    unordered_map<point, int, pair_hasher<int, int>> dist;
    dist[point(3, 4)] = 5;
    assert(dist[point(3, 4)] == 5);

    unordered_map<vector<int>, int, vector_hasher<int>> seen;
    seen[v] = 1;
    assert(seen[v] == 1);

    typedef pair<vector<int>, int> state;
    unordered_map<state, int, generic_hasher<state>> dp;
    dp[state(v, 4)] = 9;
    assert(dp[state(v, 4)] == 9);
    return 0;
}

```

4.2.2 Rolling Hash

Computes polynomial rolling hashes for sequences, supporting $O(1)$ contiguous subsequence hash queries after preprocessing. This is useful for probabilistic string matching, substring equality checks, repeated subarray detection, and binary-searching over candidate lengths.

The implementation works modulo the Mersenne prime $2^{61} - 1$ and uses `__uint128_t` for multiplication. The base `HASH_BASE` should be changed or chosen randomly for open-hacking environments. With a fixed base, hashing remains fast and practical, but it is still probabilistic and should not be used as proof of equality when exact verification is required.

By default, each sequence value is cast to `uint64` and mixed. For non-integer element types, pass a custom value hasher that maps each element to a stable nonzero value in `[1, HASH_MOD)`.

- `rolling_hash<T>(first, last)` constructs prefix hashes for any iterator range of values accepted by the value hasher.
- `rolling_hash<T>(v)` constructs prefix hashes for vector `v`.
- `get(l, r)` returns the hash of the half-open subsequence `[l, r)`.
- `hash(first, last)` returns the hash of an iterator range.
- `hash(v)` returns the hash of vector `v`.
- `concat(left, right, right_len)` returns the hash of the concatenation of a sequence with hash `left` and a sequence with hash `right` and length `right_len`.

Time Complexity:

- $O(n)$ per constructor call and whole-sequence hash, where n is the sequence length.
- $O(1)$ per call to `get(l, r)` and `concat(left, right, right_len)`.

Space Complexity:

- $O(n)$ for storage of prefix hashes and powers, where n is the maximum sequence length processed so far.
- $O(1)$ auxiliary per query.

```
#include <string>
#include <utility>
#include <vector>

typedef unsigned long long uint64;

const uint64 HASH_MOD = (1ULL << 61) - 1;
const uint64 HASH_BASE = 911382323ULL; // Change or randomize.

uint64 hash_add(uint64 a, uint64 b) {
    uint64 c = a + b;
    return c >= HASH_MOD ? c - HASH_MOD : c;
}

uint64 hash_sub(uint64 a, uint64 b) {
    return a >= b ? a - b : a + HASH_MOD - b;
}

uint64 hash_mul(uint64 a, uint64 b) {
    __uint128_t p = (__uint128_t)a * b;
    uint64 low = (uint64)p & HASH_MOD;
    uint64 high = (uint64)(p >> 61);
    uint64 res = low + high;
    if (res >= HASH_MOD) {
        res -= HASH_MOD;
    }
    return res;
}

uint64 hash_mix(uint64 x) {
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

template<class T>
struct rolling_value_hasher {
    uint64 operator()(const T &x) const { return hash_mix((uint64)x) % (HASH_MOD - 1) + 1; }
};

template<class T, class ValueHasher = rolling_value_hasher<T>>
class rolling_hash {
    static std::vector<uint64> pow_base;
```

```

std::vector<uint64> pref;
ValueHasher value_hasher;

static void ensure_powers(int n) {
    while ((int)pow_base.size() <= n) {
        pow_base.push_back(hash_mul(pow_base.back(), HASH_BASE));
    }
}

template<class It>
void build(It first, It last) {
    pref.clear();
    pref.push_back(0);
    for (It it = first; it != last; ++it) {
        pref.push_back(hash_add(hash_mul(pref.back(), HASH_BASE), value_hasher(*it)));
    }
    ensure_powers(pref.size() - 1);
}

public:
explicit rolling_hash(const ValueHasher &hasher = ValueHasher()) : value_hasher(hasher) {
    ensure_powers(0);
    pref.push_back(0);
}

template<class It>
rolling_hash(It first, It last, const ValueHasher &hasher = ValueHasher())
    : value_hasher(hasher) {
    build(first, last);
}

explicit rolling_hash(const std::vector<T> &v, const ValueHasher &hasher = ValueHasher())
    : value_hasher(hasher) {
    build(v.begin(), v.end());
}

int size() const { return (int)pref.size() - 1; }

uint64 get(int l, int r) const { return hash_sub(pref[r], hash_mul(pref[l], pow_base[r - l])); }

template<class It>
static uint64 hash(It first, It last, const ValueHasher &hasher = ValueHasher()) {
    rolling_hash<T, ValueHasher> h(first, last, hasher);
    return h.get(0, h.size());
}

static uint64 hash(const std::vector<T> &v, const ValueHasher &hasher = ValueHasher()) {
    rolling_hash<T, ValueHasher> h(v, hasher);
    return h.get(0, h.size());
}

static uint64 concat(uint64 left, uint64 right, int right_len) {
    ensure_powers(right_len);
    return hash_add(hash_mul(left, pow_base[right_len]), right);
}
};

template<class T, class ValueHasher>
std::vector<uint64> rolling_hash<T, ValueHasher>::pow_base(1, 1);

```

Example Usage

```

#include <cassert>
using namespace std;

typedef pair<int, int> point;

```

```

struct point_hasher {
    uint64 operator()(const point &p) const {
        return hash_mix((uint64)p.first * 1000003ULL + (uint64)p.second) % (HASH_MOD - 1) + 1;
    }
};

int main() {
    string s = "abracadabra";
    rolling_hash<char> hs(s.begin(), s.end());
    assert(hs.get(0, 4) == hs.get(7, 11)); // "abra" == "abra"
    assert(hs.get(0, 4) != hs.get(3, 7));  // "abra" != "acad"

    string a = "abc", b = "def";
    uint64 ab = rolling_hash<char>::concat(
        rolling_hash<char>::hash(a.begin(), a.end()), rolling_hash<char>::hash(b.begin(), b.end()),
        b.size()
    );
    string c = a + b;
    assert(ab == rolling_hash<char>::hash(c.begin(), c.end()));

    vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    v.push_back(1);
    v.push_back(2);
    rolling_hash<int> hv(v);
    assert(hv.get(0, 2) == hv.get(3, 5)); // [1, 2] == [1, 2]
    assert(hv.get(0, 3) == rolling_hash<int>::hash(v.begin(), v.begin() + 3));

    vector<point> poly;
    poly.push_back(point(1, 2));
    poly.push_back(point(3, 4));
    rolling_hash<point, point_hasher> hp(poly);
    assert((hp.get(0, 2) == rolling_hash<point, point_hasher>::hash(poly)));
    return 0;
}

```

4.2.3 Zobrist Hashing

Computes randomized XOR fingerprints for keys and sets of keys using Zobrist hashing. Each distinct key is assigned a pseudorandom 64-bit token, and a set is represented by the XOR of its members' tokens. This makes insertion and deletion the same operation: XOR the key's token once.

This technique is useful for probabilistic equality checks on sets, distinct-prefix queries, board-game states, and other unordered collections. It is not cryptographic, and collisions are possible with probability roughly $q^2/2^{64}$ over q compared fingerprints. For multisets, plain XOR only records element parity; use a summed hash or pair it with counts if multiplicity matters.

- `zobrist_hash(seed)` constructs a token generator with initial value `seed`.
- `get(x)` returns the stable token assigned to key `x`, creating it if needed.
- `toggle(h, x)` returns the hash obtained by inserting `x` into set hash `h` if absent, or removing `x` from `h` if present.
- `distinct_prefix_hashes(lo, hi)` returns prefix hashes where each distinct key in the range `[lo, hi)` contributes only on its first occurrence.

Time Complexity:

- $O(\log n)$ per call to `get(x)` and `toggle(h, x)`, where n is the number of distinct keys seen so far.
- $O(n \log n)$ per call to `distinct_prefix_hashes(lo, hi)`, where n is the distance between `lo` and `hi`.

Space Complexity:

- $O(n)$ for stored key tokens, where n is the number of distinct keys seen so far.
- $O(n)$ auxiliary for `distinct_prefix_hashes(lo, hi)`.

```
#include <map>
#include <set>
#include <vector>

typedef unsigned long long uint64;

template<class T>
class zobrist_hash {
    std::map<T, uint64> token;
    uint64 state;

    static uint64 splitmix64(uint64 x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

public:
    explicit zobrist_hash(uint64 seed = 0) : state(seed) {}

    uint64 get(const T &x) {
        typename std::map<T, uint64>::iterator it = token.find(x);
        if (it != token.end()) {
            return it->second;
        }
        state = splitmix64(state);
        token[x] = state;
        return state;
    }

    uint64 toggle(uint64 h, const T &x) { return h ^ get(x); }

    template<class It>
    std::vector<uint64> distinct_prefix_hashes(It lo, It hi) {
        std::set<T> seen;
        std::vector<uint64> res(1, 0);
        uint64 h = 0;
        for (It it = lo; it != hi; ++it) {
            if (seen.insert(*it).second) {
                h ^= get(*it);
            }
            res.push_back(h);
        }
        return res;
    }
};
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    int a[] = {1, 2, 1, 3};
    int b[] = {2, 1, 3};
    zobrist_hash<int> zh(123);
    vector<uint64> pa = zh.distinct_prefix_hashes(a, a + 4);
    vector<uint64> pb = zh.distinct_prefix_hashes(b, b + 3);

    assert(pa[2] == pb[2]); // Distinct sets {1, 2} and {2, 1}.
    assert(pa[4] == pb[3]); // Distinct sets {1, 2, 3} and {2, 1, 3}.
```

```

uint64 h = 0;
h = zh.toggle(h, 5);
assert(h != 0);
h = zh.toggle(h, 5);
assert(h == 0);
return 0;
}

```

4.3 String Searching

4.3.1 String Searching (KMP)

Given a single string (needle) and subsequent queries of texts (haystacks) to be searched, determine the first positions in which the needle occurs within the given haystacks in linear time using the Knuth-Morris-Pratt algorithm. In comparison, `std::string::find` runs in quadratic time.

- `kmp(needle)` constructs the partial match table for a string `needle` that is to be searched for subsequently in `haystack` queries.
- `find_in(haystack)` returns the first position that `needle` occurs in `haystack`, or `std::string::npos` if it cannot be found. Note that the function can be modified to return all matches by simply letting the loop run and storing the results instead of returning early.

Time Complexity:

- $O(m)$ per call to the constructor, where m is the length of `needle`.
- $O(n)$ per call to `find_in(haystack)`, where n is the length of `haystack`.

Space Complexity:

- $O(m)$ for storage of the partial match table, where m is the length of `needle`.
- $O(1)$ auxiliary space per call to `find_in(haystack)`.

```

#include <string>
#include <vector>
using std::string;

class kmp {
    string needle;
    std::vector<int> table;

public:
    kmp(const string &needle) : needle(needle), table(needle.size()) {
        for (int i = 1, j = 0; i < (int)needle.size(); i++) {
            while (j > 0 && needle[i] != needle[j]) {
                j = table[j - 1];
            }
            if (needle[i] == needle[j]) {
                j++;
            }
            table[i] = j;
        }
    }

    size_t find_in(const string &haystack) {
        int m = needle.size();
        if (m == 0) {
            return 0;
        }
        for (int i = 0, j = 0; i < (int)haystack.size(); i++) {
            while (j > 0 && needle[j] != haystack[i]) {
                j = table[j - 1];
            }

```

```

    }
    if (needle[j] == haystack[i]) {
        j++;
    }
    if (j == m) {
        return i + 1 - m;
    }
}
return string::npos;
}
};

```

Example Usage

```

#include <cassert>

int main() {
    assert(15 == kmp("ABCDABD").find_in("ABC ABCDAB ABCDABCDABDE"));
    return 0;
}

```

4.3.2 String Searching (Z Algorithm)

Given a single string (needle) and a single text (haystack) to be searched, determine the first position in which the needle occurs within the haystack in linear time using the Z algorithm. In comparison, `std::string::find` runs in quadratic time.

The `find()` function below calls the Z algorithm on the concatenation of `needle` and `haystack`, separated by a sentinel character (in this case `'\0'`), which should be chosen such that it does not occur within either of the input strings.

- `z_array(s)` constructs the Z array for a string `needle` that can be used for string searching. The Z array on an input string `s` is an array `z` where `z[i]` is the length of the longest substring starting from `s[i]` which is also a prefix of `s`.
- `find(haystack, needle)` returns the first position that `needle` occurs in `haystack`, or `std::string::npos` if it cannot be found. Note that the function can be modified to return all matches by simply letting the loop run and storing the results instead of returning early.

Time Complexity:

- $O(n)$ per call to `z_array(s)`, where n is the length of `s`.
- $O(n + m)$ per call to `find(haystack, needle)`, where n is the length of `haystack` and m is the length of `needle`.

Space Complexity:

- $O(n)$ auxiliary heap space for `z_array(s)`, where n is the length of `s`.
- $O(n + m)$ auxiliary heap space for `find(haystack, needle)` where n is the length of `haystack` and m is the length of `needle`.

```

#include <algorithm>
#include <string>
#include <vector>
using std::string;

std::vector<int> z_array(const string &s) {
    std::vector<int> z(s.size());
    for (int i = 1, l = 0, r = 0; i < (int)z.size(); i++) {
        if (i <= r) {
            z[i] = std::min(r - i + 1, z[i - l]);
        }
        while (i + z[i] < (int)z.size() && s[z[i]] == s[i + z[i]]) {

```



```

        z[i]++;
    }
    if (r < i + z[i] - 1) {
        l = i;
        r = i + z[i] - 1;
    }
}
return z;
}

size_t find(const string &haystack, const string &needle) {
    std::vector<int> z = z_array(needle + '\0' + haystack);
    for (int i = (int)needle.size() + 1; i < (int)z.size(); i++) {
        if (z[i] == (int)needle.size()) {
            return i - (int)needle.size() - 1;
        }
    }
    return string::npos;
}

```

Example Usage

```

#include <cassert>

int main() {
    assert(15 == find("ABC ABCDAB ABCDABCDABDE", "ABCDABD"));
    return 0;
}

```

4.3.3 String Searching (Aho-Corasick)

Given a set of strings (needles) and subsequent queries of texts (haystacks) to be searched, determine all positions in which needles occur within the given haystacks in linear time using the Aho-Corasick algorithm.

Note that this implementation uses an ordered map for storage of the graph, adding an additional $\log k$ factor to the time complexities of all operations, where k is the size of the alphabet (number of distinct characters used across the needles). It also uses an ordered set for storage of the precomputed output tables, adding an additional $\log m$ factor to the time complexities, where m is the number of needles. In C++11 and later, both of these containers should be replaced by their unordered versions for constant time access, thus eliminating the log factors from the time complexities.

- `aho_corasick(needles)` constructs the finite-state automaton for a set of needle strings that are to be searched for subsequently in haystack queries.
- `find_all_in(haystack, report_match)` calls the function `report_match(s, pos)` once on each occurrence of each needle that occurs in the `haystack`, where `pos` is the starting position in the `haystack` at which string `s` (a matched needle) occurs. The matches will be reported in increasing order of their ending positions within the `haystack`.

Time Complexity:

- $O(m \cdot ((\log m) + l \cdot \log k))$ per call to the constructor, where m is the number of needles, l is the maximum length for any needle, and k is the size of the alphabet used by the needles. If unordered containers are used, then the time complexity reduces to $O(m \cdot l)$, or linear on the input size.
- $O(n \cdot (\log k) + z)$ per call to `find_all_in(haystack, report_match)`, where n is the length of `haystack`, k is the size of the alphabet used by the needles, and z is the number of matches. If unordered containers are used, then the time complexity reduces to $O(n + z)$, or linear on the input size.

Space Complexity:

- $O(m \cdot l)$ for storage of the automaton, where m is the number of needles and l is the maximum length for

any needle.

- $O(1)$ auxiliary space per call to `find_all_in(haystack, report_match)`.

```
#include <map>
#include <queue>
#include <set>
#include <string>
#include <vector>
using std::string;

class aho_corasick {
    std::vector<string> needles;
    std::vector<int> fail;
    std::vector<std::map<char, int>> graph;
    std::vector<std::set<int>> out;

    int next_state(int curr, char c) {
        int next = curr;
        while (graph[next].find(c) == graph[next].end()) {
            next = fail[next];
        }
        return graph[next][c];
    }

public:
    aho_corasick(const std::vector<string> &needles) : needles(needles) {
        int total_len = 0;
        for (int i = 0; i < (int)needles.size(); i++) {
            total_len += needles[i].size();
        }
        fail.resize(total_len, -1);
        graph.resize(total_len);
        out.resize(total_len);
        int states = 1;
        std::map<char, int>::iterator it;
        for (int i = 0; i < (int)needles.size(); i++) {
            int curr = 0;
            for (int j = 0; j < (int)needles[i].size(); j++) {
                char c = needles[i][j];
                if ((it = graph[curr].find(c)) != graph[curr].end()) {
                    curr = it->second;
                } else {
                    curr = graph[curr][c] = states++;
                }
            }
            out[curr].insert(i);
        }
        std::queue<int> q;
        for (it = graph[0].begin(); it != graph[0].end(); ++it) {
            if (it->second != 0) {
                fail[it->second] = 0;
                q.push(it->second);
            }
        }
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (it = graph[u].begin(); it != graph[u].end(); ++it) {
                int v = it->second, f = fail[u];
                while (graph[f].find(it->first) == graph[f].end()) {
                    f = fail[f];
                }
                f = graph[f].find(it->first)->second;
                fail[v] = f;
                out[v].insert(out[f].begin(), out[f].end());
                q.push(v);
            }
        }
    }
};
```

```

    }
}

template<class ReportFunction>
void find_all_in(const string &haystack, ReportFunction report_match) {
    int state = 0;
    std::set<int>::iterator it;
    for (int i = 0; i < (int)haystack.size(); i++) {
        state = next_state(state, haystack[i]);
        for (it = out[state].begin(); it != out[state].end(); ++it) {
            report_match(needles[*it], i - needles[*it].size() + 1);
        }
    }
}
};

```

Example Usage

```

#include <iostream>
using namespace std;

void report_match(const string &needle, int pos) {
    cout << "Matched \"" << needle << "\" at position " << pos << "." << endl;
}

int main() {
    vector<string> needles;
    needles.push_back("a");
    needles.push_back("ab");
    needles.push_back("bab");
    needles.push_back("bc");
    needles.push_back("bca");
    needles.push_back("c");
    needles.push_back("caa");
    needles.push_back("abccab");

    aho_corasick(needles).find_all_in("abccab", report_match);
    return 0;
}

```

Example Output

```

Matched "a" at position 0.
Matched "ab" at position 0.
Matched "bc" at position 1.
Matched "c" at position 2.
Matched "c" at position 3.
Matched "a" at position 4.
Matched "ab" at position 4.
Matched "abccab" at position 0.

```

4.3.4 Palindrome Searching (Manacher)

Computes all odd- and even-length palindromic radii of a string in linear time using Manacher's algorithm. These radii can be used to test whether any substring is a palindrome in $O(1)$, find the longest palindromic substring, or count all palindromic substrings.

For odd palindromes, `odd[i]` is the radius centered at character `s[i]`, including the center character. Thus the palindrome spans `[i - odd[i] + 1, i + odd[i]]`. For even palindromes, `even[i]` is the radius centered between `s[i - 1]` and `s[i]`. Thus the palindrome spans `[i - even[i], i + even[i]]`.

- `manacher(s)` constructs the palindromic radii arrays for string `s`.
- `is_palindrome(l, r)` returns whether substring `[l, r)` is a palindrome.

- `longest_palindrome()` returns one longest palindromic substring of `s`.
- `count_palindromes()` returns the total number of palindromic substrings of `s`, counted with multiplicity by position.

Time Complexity:

- $O(n)$ per constructor call, where n is the length of `s`.
- $O(1)$ per call to `is_palindrome(l, r)`.
- $O(n)$ per call to `longest_palindrome()` and `count_palindromes()`.

Space Complexity:

- $O(n)$ for storage of the radii arrays.
- $O(1)$ auxiliary per query.

```
#include <algorithm>
#include <string>
#include <vector>
using std::string;

class manacher {
    string s;
    std::vector<int> odd, even;

public:
    explicit manacher(const string &s) : s(s), odd(s.size()), even(s.size()) {
        int n = s.size();
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = (i > r) ? 1 : std::min(odd[l + r - i], r - i + 1);
            while (0 <= i - k && i + k < n && s[i - k] == s[i + k]) {
                k++;
            }
            odd[i] = k;
            if (i + k - 1 > r) {
                l = i - k + 1;
                r = i + k - 1;
            }
        }
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = (i > r) ? 0 : std::min(even[l + r - i + 1], r - i + 1);
            while (0 <= i - k - 1 && i + k < n && s[i - k - 1] == s[i + k]) {
                k++;
            }
            even[i] = k;
            if (i + k - 1 > r) {
                l = i - k;
                r = i + k - 1;
            }
        }
    }

    bool is_palindrome(int l, int r) const {
        int len = r - l;
        if (len <= 0) {
            return true;
        }
        int mid = (l + r) / 2;
        return (len % 2) ? odd[mid] >= len / 2 + 1 : even[mid] >= len / 2;
    }

    string longest_palindrome() const {
        int best_l = 0, best_len = 0;
        for (int i = 0; i < (int)s.size(); i++) {
            int len = 2 * odd[i] - 1;
            if (len > best_len) {
                best_len = len;
                best_l = i - odd[i] + 1;
            }
        }
    }
};
```

```

    }
    len = 2 * even[i];
    if (len > best_len) {
        best_len = len;
        best_l = i - even[i];
    }
}
return s.substr(best_l, best_len);
}

long long count_palindromes() const {
    long long res = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        res += odd[i] + even[i];
    }
    return res;
}
};

```

Example Usage

```

#include <cassert>

int main() {
    manacher m("abacaba");
    assert(m.is_palindrome(0, 7));
    assert(m.is_palindrome(1, 6));
    assert(!m.is_palindrome(0, 6));
    assert(m.longest_palindrome() == "abacaba");
    assert(m.count_palindromes() == 12);

    manacher e("cabbad");
    assert(e.is_palindrome(1, 5));
    assert(e.longest_palindrome() == "abba");
    return 0;
}

```

4.4 Expression Parsing

4.4.1 Recursive Descent Parsing (Simple)

Evaluate an expression in accordance to the order of operations (parentheses, unary plus and minus signs, multiplication/division, addition/subtraction). The following is a minimalistic recursive descent implementation using iterators.

- `eval(s)` returns an evaluation of the arithmetic expression `s`.

Time Complexity: $O(n)$ per call to `eval(s)`, where n is the length of `s`.

Space Complexity: $O(n)$ auxiliary stack space for `eval(s)`, where n is the length of `s`.

```

#include <string>

template<class It>
int eval(It &it, int prec) {
    if (prec == 0) {
        int sign = 1, ret = 0;
        for (; *it == '-'; it++) {
            sign *= -1;
        }
    }
}

```

```

    if (*it == '(') {
        ret = eval(++it, 2);
        it++;
    } else
        while (*it >= '0' && *it <= '9') {
            ret = 10 * ret + (*(it++) - '0');
        }
    return sign * ret;
}

int num = eval(it, prec - 1);
while (!(prec == 2 && *it != '+' && *it != '-' || (prec == 1 && *it != '*' && *it != '/'))) {
    switch (*(it++)) {
        case '+':
            num += eval(it, prec - 1);
            break;
        case '-':
            num -= eval(it, prec - 1);
            break;
        case '*':
            num *= eval(it, prec - 1);
            break;
        case '/':
            num /= eval(it, prec - 1);
            break;
    }
}
return num;
}

int eval(const std::string &s) {
    std::string expr = s + '\0';
    std::string::iterator it = expr.begin();
    return eval(it, 2);
}

```

Example Usage

```

#include <cassert>

int main() {
    assert(eval("1++1") == 2);
    assert(eval("1+2*3+4+3*(2+2)-100") == -63);
    return 0;
}

```

4.4.2 Recursive Descent Parsing (Generic)

Evaluate an expression using a generalized parser class for custom-defined operand types, prefix unary operators, binary operators, and precedences. Typical parentheses behavior is supported, but multiplication by juxtaposition is not. Evaluation is performed using the recursive descent algorithm.

An arbitrary operand type is supported, with its string representation defined by a user-specified `is_operand()` and `eval_operand()` functions. For maximum reliability, the string representation of operands should not use characters shared by any operator. For instance, the best practice instead of accepting `-1` as a valid operand (since the `-` sign may conflict with the identical binary operator), is to specify non-negative number as operands alongside the unary operator `-`.

Operators may be non-empty strings of any length, but should not contain any parentheses or shared characters with the string representations of operands. Ideally, operators should not be prefixes or suffixes of one another, else the tokenization process may be ambiguous. For example, if `++` and `+` are both operators, then `++` may be split into either `["+", "+"]` or `["++"]` depending on the lexicographical ordering of conflicting operators.

- `parser(unary_op, binary_op)` initializes a parser with operators specified by maps `unary_op` (of operator to function pointer) and `binary_op` (of operator to pair of function pointer and operator precedence). Operator precedences should be numbered upwards starting at 0 (lowest precedence, evaluated last).
- `split(s)` returns a vector of tokens for the expression `s`, split on the given operators during construction. Each parenthesis, operator, and operand satisfying `is_operand()` will be split into a separate token. The algorithm is naive, matching operators lazily in the case of overlapping operators as mentioned above. Under these circumstances, the parse may not always succeed.
- `eval(lo, hi)` returns the evaluation of a range `[lo, hi)` of already split-up expression tokens, where `lo` and `hi` must be random-access iterators.
- `eval(s)` returns the evaluation of expression `s`, after first calling `split(s)` to obtain the tokens.

Time Complexity:

- $O(m)$ per call to the constructor, where m is the total number of operators.
- $O(nmk)$ per call to `split(s)`, where n is the length of `s`, m is the total number of operators defined for the parser instance, and k is the maximum length for any operator representation.
- $O(n \log m)$ per call to `eval(lo, hi)`, where n is the distance between `lo` and `hi` and m is the total number of operators defined for the parser instance. In C++11 and later, `std::unordered_map` may be used in place of `std::map` for storing `unary_ops` and `binary_ops`, which will eliminate the $\log m$ factor for a time complexity of $O(n)$ per call.
- $O(nmk + n \log m)$ per call to `eval(s)`, where n is the distance between `lo` and `hi`, and m and k are as defined previous.

Space Complexity:

- $O(mk)$ for storage of the m operators, of maximum length k .
- $O(n)$ auxiliary stack space for `split(s)`, `eval(lo, hi)`, and `eval(s)`, where n is the length of the argument.

```
#include <algorithm>
#include <cctype>
#include <functional>
#include <map>
#include <set>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>
using std::string;

// Define the custom operand type and representation below.
typedef double Operand;
typedef Operand (*UnaryOp)(Operand a);
typedef Operand (*BinaryOp)(Operand a, Operand b);

bool is_operand(const string &s) {
    int npoints = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        if (s[i] == '.') {
            if (++npoints > 1) {
                return false;
            }
        } else if (!isdigit(s[i])) {
            return false;
        }
    }
    return !s.empty();
}

Operand eval_operand(const string &s) {
    Operand res;
    std::stringstream ss(s);
    ss >> res;
    return res;
}
```

```

}

class parser {
public:
    typedef std::map<string, UnaryOp> unary_op_map;
    typedef std::map<string, std::pair<BinaryOp, int>> binary_op_map;
    unary_op_map unary_ops;
    binary_op_map binary_ops;
    std::set<string> ops;
    int max_precedence;

    template<class StrIt>
    Operand eval_unary(StrIt &lo, StrIt hi) {
        if (is_operand(*lo)) {
            return eval_operand(*(lo++));
        }
        unary_op_map::iterator it = unary_ops.find(*lo);
        if (it != unary_ops.end()) {
            return (it->second)(eval_unary(++lo, hi));
        }
        if (*lo != "(") {
            throw std::runtime_error("Expected \"(\" during eval.");
        }
        Operand res = eval_binary(++lo, hi, 0);
        if (*lo != ")") {
            throw std::runtime_error("Expected \")\" during eval.");
        }
        ++lo;
        return res;
    }

    template<class StrIt>
    Operand eval_binary(StrIt &lo, StrIt hi, Operand precedence) {
        if (precedence > max_precedence) {
            return eval_unary(lo, hi);
        }
        Operand v = eval_binary(lo, hi, precedence + 1);
        while (lo != hi) {
            binary_op_map::iterator it;
            it = binary_ops.find(*lo);
            if (it == binary_ops.end() || it->second.second != precedence) {
                return v;
            }
            v = (it->second.first)(v, eval_binary(++lo, hi, precedence + 1));
        }
        return v;
    }

    static string strip(string s) {
        s.erase(
            s.begin(), std::find_if(s.begin(), s.end(), std::not1(std::ptr_fun<int, int>(std::isspace)))
        );
        s.erase(
            std::find_if(s.rbegin(), s.rend(), std::not1(std::ptr_fun<int, int>(std::isspace))).base(),
            s.end()
        );
        return s;
    }

public:
    parser(const unary_op_map &unary_ops, const binary_op_map &binary_ops)
        : unary_ops(unary_ops), binary_ops(binary_ops) {
        for (unary_op_map::const_iterator it = unary_ops.begin(); it != unary_ops.end(); ++it) {
            ops.insert(it->first);
        }
        max_precedence = 0;
        for (binary_op_map::const_iterator it = binary_ops.begin(); it != binary_ops.end(); ++it) {
            ops.insert(it->first);
            max_precedence = std::max(max_precedence, it->second.second);
        }
    }
};

```



```

    }
}

std::vector<string> split(const string &s) {
    std::vector<string> res;
    for (int i = 0; i < (int)s.size(); i++) {
        if (s[i] == ' ') {
            continue;
        }
        int next_paren = s.size();
        for (int j = i; j < (int)s.size(); j++) {
            if (s[j] == '(' || s[j] == ')') {
                next_paren = j;
                break;
            }
        }
        while (i < next_paren) {
            int found = next_paren;
            string found_op;
            for (int j = i; j < next_paren && found == next_paren; j++) {
                for (std::set<string>::iterator it = ops.begin(); it != ops.end(); ++it) {
                    if (s.substr(j, it->size()) == *it) {
                        found = j;
                        found_op = *it;
                        break;
                    }
                }
            }
            string term = strip(s.substr(i, found - i));
            if (!term.empty()) {
                res.push_back(term);
                if (!is_operand(term)) {
                    throw std::runtime_error("Failed to split term: \"" + term + "\".");
                }
            }
            if (found < next_paren) {
                res.push_back(found_op);
                i = found + found_op.size();
            } else {
                i = next_paren;
            }
        }
        if (next_paren < s.size()) {
            res.push_back(string(1, s[next_paren]));
        }
    }
    return res;
}

template<class StrIt>
Operand eval(StrIt lo, StrIt hi) {
    Operand res = eval_binary(lo, hi, 0);
    if (lo != hi) {
        throw std::runtime_error("Eval failed at token " + *lo + ".");
    }
    return res;
}

Operand eval(const string &s) {
    std::vector<string> tokens = split(s);
    return eval(tokens.begin(), tokens.end());
}
};

```

Example Usage

```

#include <cassert>
#include <cmath>
using namespace std;

#define EQ(a, b) (fabs((a) - (b)) < 1e-7)

double pos(double a) {
    return +a;
}
double neg(double a) {
    return -a;
}
double add(double a, double b) {
    return a + b;
}
double sub(double a, double b) {
    return a - b;
}
double mul(double a, double b) {
    return a * b;
}
double div(double a, double b) {
    return a / b;
}

int main() {
    map<string, UnaryOp> unary_ops;
    unary_ops["+"] = pos;
    unary_ops["-"] = neg;

    map<string, pair<BinaryOp, int>> binary_ops;
    binary_ops["+"] = make_pair((BinaryOp)add, 0);
    binary_ops["-"] = make_pair((BinaryOp)sub, 0);
    binary_ops["*"] = make_pair((BinaryOp)mul, 1);
    binary_ops["/"] = make_pair((BinaryOp)div, 1);
    binary_ops["^"] = make_pair((BinaryOp)pow, 2);

    parser p(unary_ops, binary_ops);
    assert(EQ(p.eval("-+-(--(+1))"), -1));
    assert(EQ(p.eval("5*(3+3)-2-2"), 26));
    assert(EQ(p.eval("1+2*3*4+3*(+2)-100"), -69));
    assert(EQ(p.eval("3*3*3*3*3-2*2*2*2*2*2"), 473));
    assert(EQ(p.eval("3.14 + 3 * (7.7/9.8^32.9 )"), 3.14));
    assert(EQ(p.eval("5*(3+2)/-1*-2+(-2-2+3)-3-(-2)+15/2/2/2+(-2)"), 45.875));
    assert(EQ(p.eval("123456789./3/3/3*2*2*2+456/6-23/3"), 36579857.666666666));
    assert(EQ(p.eval("10/3+10/4+10/5+10/6+10/7+10/8+10/9+10/10+15*23456"), 351854.28968253968));
    assert(
        EQ(p.eval(
            "-(5-(5-(5-(5-(5-2)))))+(3-(3-(3-(3-(3+3)))))*"
            "(7-(7-(7-(7-(7-7+4*5))))))"
        ),
        117)
    );
    return 0;
}

```

4.4.3 Shunting Yard Parsing

Evaluate an expression using a generalized parser class for custom-defined operand types, prefix unary operators, binary operators, and precedences. Typical parentheses behavior is supported, but multiplication by juxtaposition is not. Evaluation is performed using the shunting yard algorithm.

An arbitrary operand type is supported, with its string representation defined by a user-specified `is_operand()` and `eval_operand()` functions. For maximum reliability, the string representation of operands should not use

characters shared by any operator. For instance, the best practice instead of accepting `-1` as a valid operand (since the `-` sign may conflict with the identical binary operator), is to specify non-negative number as operands alongside the unary operator `-`.

Operators may be non-empty strings of any length, but should not contain any parentheses or shared characters with the string representations of operands. Ideally, operators should not be prefixes or suffixes of one another, else the tokenization process may be ambiguous. For example, if `++` and `+` are both operators, then `++` may be split into either `["+", "+"]` or `["++"]` depending on the lexicographical ordering of conflicting operators.

- `parser(unary_op, binary_op)` initializes a parser with operators specified by maps `unary_op` (of operator to function pointer) and `binary_op` (of operator to pair of function pointer and operator precedence). Operator precedences should be numbered upwards starting at 0 (lowest precedence, evaluated last).
- `split(s)` returns a vector of tokens for the expression `s`, split on the given operators during construction. Each parenthesis, operator, and operand satisfying `is_operand()` will be split into a separate token. The algorithm is naive, matching operators lazily in the case of overlapping operators as mentioned above. Under these circumstances, the parse may not always succeed.
- `eval(lo, hi)` returns the evaluation of a range `[lo, hi)` of already split-up expression tokens, where `lo` and `hi` must be random-access iterators.
- `eval(s)` returns the evaluation of expression `s`, after first calling `split(s)` to obtain the tokens.

Time Complexity:

- $O(m)$ per call to the constructor, where m is the total number of operators.
- $O(nmk)$ per call to `split(s)`, where n is the length of `s`, m is the total number of operators defined for the parser instance, and k is the maximum length for any operator representation.
- $O(n \log m)$ per call to `eval(lo, hi)`, where n is the distance between `lo` and `hi` and m is the total number of operators defined for the parser instance. In C++11 and later, `std::unordered_map` may be used in place of `std::map` for storing `unary_ops` and `binary_ops`, which will eliminate the $\log m$ factor for a time complexity of $O(n)$ per call.
- $O(nmk + n \log m)$ per call to `eval(s)`, where n is the distance between `lo` and `hi`, and m and k are as defined previous.

Space Complexity:

- $O(mk)$ for storage of the m operators, of maximum length k .
- $O(n)$ auxiliary stack space for `split(s)`, `eval(lo, hi)`, and `eval(s)`, where n is the length of the argument.

```
#include <algorithm>
#include <cctype>
#include <functional>
#include <map>
#include <set>
#include <sstream>
#include <stack>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>
using std::string;

// Define the custom operand type and representation below.
typedef double Operand;
typedef Operand (*UnaryOp)(Operand a);
typedef Operand (*BinaryOp)(Operand a, Operand b);

bool is_operand(const string &s) {
    int npoints = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        if (s[i] == '.') {
            if (++npoints > 1) {
                return false;
            }
        }
        else if (!isdigit(s[i])) {

```

```

        return false;
    }
}
return !s.empty();
}

Operand eval_operand(const string &s) {
    Operand res;
    std::stringstream ss(s);
    ss >> res;
    return res;
}

class parser {
    typedef std::map<string, UnaryOp> unary_op_map;
    typedef std::map<string, std::pair<BinaryOp, int>> binary_op_map;
    unary_op_map unary_ops;
    binary_op_map binary_ops;
    std::set<string> ops;

    static string strip(string s) {
        s.erase(
            s.begin(), std::find_if(s.begin(), s.end(), std::not1(std::ptr_fun<int, int>(std::isspace)))
        );
        s.erase(
            std::find_if(s.rbegin(), s.rend(), std::not1(std::ptr_fun<int, int>(std::isspace))).base(),
            s.end()
        );
        return s;
    }

public:
    parser(const unary_op_map &unary_ops, const binary_op_map &binary_ops)
        : unary_ops(unary_ops), binary_ops(binary_ops) {
        for (unary_op_map::const_iterator it = unary_ops.begin(); it != unary_ops.end(); ++it) {
            ops.insert(it->first);
        }
        for (binary_op_map::const_iterator it = binary_ops.begin(); it != binary_ops.end(); ++it) {
            ops.insert(it->first);
        }
    }

    std::vector<string> split(const string &s) {
        std::vector<string> res;
        for (int i = 0; i < (int)s.size(); i++) {
            if (s[i] == ' ') {
                continue;
            }
            int next_paren = s.size();
            for (int j = i; j < (int)s.size(); j++) {
                if (s[j] == '(' || s[j] == ')') {
                    next_paren = j;
                    break;
                }
            }
            while (i < next_paren) {
                int found = next_paren;
                string found_op;
                for (int j = i; j < next_paren && found == next_paren; j++) {
                    for (std::set<string>::iterator it = ops.begin(); it != ops.end(); ++it) {
                        if (s.substr(j, it->size()) == *it) {
                            found = j;
                            found_op = *it;
                            break;
                        }
                    }
                }
            }
            string term = strip(s.substr(i, found - i));

```

```

    if (!term.empty()) {
        res.push_back(term);
        if (!is_operand(term)) {
            throw std::runtime_error("Failed to split term: \"" + term + "\".");
        }
    }
    if (found < next_paren) {
        res.push_back(found_op);
        i = found + found_op.size();
    } else {
        i = next_paren;
    }
}
if (next_paren < s.size()) {
    res.push_back(string(1, s[next_paren]));
}
}
return res;
}

template<class StrIt>
Operand eval(StrIt lo, StrIt hi) {
    std::stack<Operand> vals;
    std::stack<std::pair<string, bool>> ops;
    ops.push(std::make_pair("(", false));
    StrIt prev = hi;
    do {
        string curr = (lo == hi) ? ")" : *lo;
        if (is_operand(curr)) {
            vals.push(eval_operand(curr));
        } else if (curr == "(") {
            ops.push(std::make_pair(curr, false));
        } else if (
            unary_ops.find(curr) != unary_ops.end() &&
            (prev == hi || *prev == "(" || binary_ops.find(*prev) != binary_ops.end())
        ) {
            ops.push(std::make_pair(curr, true));
        } else {
            for (;;) {
                string op = ops.top().first;
                bool is_unary = ops.top().second;
                binary_op_map::iterator it1 = binary_ops.find(op);
                binary_op_map::iterator it2 = binary_ops.find(curr);
                if (!is_unary && (it1 == binary_ops.end() ? -1 : it1->second.second) <
                    (it2 == binary_ops.end() ? -1 : it2->second.second)) {
                    break;
                }
                ops.pop();
                if (op == "(") {
                    break;
                }
                Operand b = vals.top();
                vals.pop();
                if (is_unary) {
                    unary_op_map::iterator it = unary_ops.find(op);
                    if (it == unary_ops.end()) {
                        throw std::runtime_error("Failed to eval unary op: " + op);
                    }
                    vals.push((it->second)(b));
                } else {
                    Operand a = vals.top();
                    vals.pop();
                    if (it1 == binary_ops.end()) {
                        throw std::runtime_error("Failed to eval binary op: " + op);
                    }
                    vals.push((it1->second.first)(a, b));
                }
            }
        }
    }
}

```

```

        if (curr != ")") {
            ops.push(std::make_pair(*lo, false));
        }
    }
    prev = lo;
} while (lo++ != hi);
return vals.top();
}

Operand eval(const string &s) {
    std::vector<string> tokens = split(s);
    return eval(tokens.begin(), tokens.end());
}
};

```

Example Usage

```

#include <cassert>
#include <cmath>
using namespace std;

#define EQ(a, b) (fabs((a) - (b)) < 1e-7)

double pos(double a) { return +a; }
double neg(double a) { return -a; }
double add(double a, double b) { return a + b; }
double sub(double a, double b) { return a - b; }
double mul(double a, double b) { return a * b; }
double div(double a, double b) { return a / b; }

int main() {
    map<string, UnaryOp> unary_ops;
    unary_ops["+"] = pos;
    unary_ops["-"] = neg;

    map<string, pair<BinaryOp, int>> binary_ops;
    binary_ops["+"] = make_pair((BinaryOp)add, 0);
    binary_ops["-"] = make_pair((BinaryOp)sub, 0);
    binary_ops["*"] = make_pair((BinaryOp)mul, 1);
    binary_ops["/"] = make_pair((BinaryOp)div, 1);
    binary_ops["^"] = make_pair((BinaryOp)pow, 2);

    parser p(unary_ops, binary_ops);
    assert(EQ(p.eval("-+-(--(+1))"), -1));
    assert(EQ(p.eval("5*(3+3)-2-2"), 26));
    assert(EQ(p.eval("1+2*3*4+3*(+2)-100"), -69));
    assert(EQ(p.eval("3*3*3*3*3-2*2*2*2*2*2"), 473));
    assert(EQ(p.eval("3.14 + 3 * (7.7/9.8^32.9 )"), 3.14));
    assert(EQ(p.eval("5*(3+2)/-1*-2+(-2-2-2+3)-3-(-2)+15/2/2/2+(-2)"), 45.875));
    assert(EQ(p.eval("123456789./3/3/3*2*2*2+456/6-23/3"), 36579857.666666666));
    assert(EQ(p.eval("10/3+10/4+10/5+10/6+10/7+10/8+10/9+10/10+15*23456"), 351854.28968253968));
    assert(
        EQ(p.eval(
            "-(5-(5-(5-(5-(5-2)))))+(3-(3-(3-(3-(3+3)))))*"
            "(7-(7-(7-(7-(7-7+4*5)))))"
        ),
        117)
    );
    return 0;
}

```

4.5 Sequence Dynamic Programming

4.5.1 Longest Common Substring

Given two strings, determine their longest common substring (i.e. consecutive subsequence) using dynamic programming.

Time Complexity: $O(n \cdot m)$ per call to `longest_common_substring(s1, s2)`, where n and m are the lengths of `s1` and `s2`, respectively.

Space Complexity: $O(\min(n, m))$ auxiliary heap space, where n and m are the lengths of `s1` and `s2`, respectively.

```
#include <string>
#include <vector>
using std::string;

string longest_common_substring(const string &s1, const string &s2) {
    int n = s1.size(), m = s2.size();
    if (n == 0 || m == 0) {
        return "";
    }
    if (n < m) {
        return longest_common_substring(s2, s1);
    }
    std::vector<int> curr(m), prev(m);
    int pos = 0, len = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (s1[i] == s2[j]) {
                curr[j] = (i > 0 && j > 0) ? prev[j - 1] + 1 : 1;
                if (len < curr[j]) {
                    len = curr[j];
                    pos = i - curr[j] + 1;
                }
            } else {
                curr[j] = 0;
            }
        }
        curr.swap(prev);
    }
    return s1.substr(pos, len);
}
```

Example Usage

```
#include <cassert>

int main() {
    assert(longest_common_substring("bbbabca", "aababcd") == "babc");
    return 0;
}
```

4.5.2 Longest Common Subsequence

Given two strings, determine their longest common subsequence. A subsequence is a string that can be derived from the original string by deleting some elements without changing the order of the remaining elements (e.g. "ACE" is a subsequence of "ABCDE", but "BAE" is not).

- `longest_common_subsequence(s1, s2)` returns the longest common subsequence of strings `s1` and `s2` using a classic dynamic programming approach. This implementation computes $dp[i][j]$ (the length of the longest common subsequence for the length i prefix of `s1` and the length j prefix of `s2`) before following the path backwards to construct the answer.
- `hirschberg_lcs(s1, s2)` returns the longest common subsequence of strings `s1` and `s2` using the more memory efficient Hirschberg's algorithm.

Time Complexity: $O(n \cdot m)$ per call to `longest_common_subsequence(s1, s2)` as well as `hirschberg_lcs(s1, s2)`, where n and m are the lengths of `s1` and `s2`, respectively.

Space Complexity:

- $O(n \cdot m)$ auxiliary heap space for `longest_common_subsequence(s1, s2)`, where n and m are the lengths of `s1` and `s2`, respectively.
- $O(\log \max(n, m))$ auxiliary stack space and $O(\min(n, m))$ auxiliary heap space for `hirschberg_lcs(s1, s2)`, where n and m are the lengths of `s1` and `s2`, respectively.

```
#include <algorithm>
#include <iterator>
#include <string>
#include <vector>
using std::string;

string longest_common_subsequence(const string &s1, const string &s2) {
    int n = s1.size(), m = s2.size();
    std::vector<std::vector<int>> dp(n + 1, std::vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = std::max(dp[i][j - 1], dp[i - 1][j]);
            }
        }
    }
    string res;
    for (int i = n, j = m; i > 0 && j > 0;) {
        if (s1[i - 1] == s2[j - 1]) {
            res += s1[i - 1];
            i--;
            j--;
        } else if (dp[i - 1][j] >= dp[i][j - 1]) {
            i--;
        } else {
            j--;
        }
    }
    std::reverse(res.begin(), res.end());
    return res;
}

template<class It>
std::vector<int> lcs_len(It lo1, It hi1, It lo2, It hi2) {
    std::vector<int> res(std::distance(lo2, hi2) + 1, prev(res));
    for (It it1 = lo1; it1 != hi1; ++it1) {
        res.swap(prev);
        int i = 0;
        for (It it2 = lo2; it2 != hi2; ++it2) {
            res[i + 1] = (*it1 == *it2) ? prev[i] + 1 : std::max(res[i], prev[i + 1]);
            i++;
        }
    }
    return res;
}

template<class It>
```



```

void hirschberg_rec(It lo1, It hi1, It lo2, It hi2, string *res) {
    if (lo1 == hi1) {
        return;
    }
    if (lo1 + 1 == hi1) {
        if (std::find(lo2, hi2, *lo1) != hi2) {
            *res += *lo1;
        }
        return;
    }
    It mid1 = lo1 + (hi1 - lo1) / 2;
    std::reverse_iterator<It> rlo1(hi1), rmid1(mid1), rlo2(hi2), rhi2(lo2);
    std::vector<int> fwd = lcs_len(lo1, mid1, lo2, hi2);
    std::vector<int> rev = lcs_len(rlo1, rmid1, rlo2, rhi2);
    It mid2 = lo2;
    int maxlen = -1;
    for (int i = 0, j = (int)rev.size() - 1; i < (int)fwd.size(); i++, j--) {
        if (fwd[i] + rev[j] > maxlen) {
            maxlen = fwd[i] + rev[j];
            mid2 = lo2 + i;
        }
    }
    hirschberg_rec(lo1, mid1, lo2, mid2, res);
    hirschberg_rec(mid1, hi1, mid2, hi2, res);
}

string hirschberg_lcs(const string &s1, const string &s2) {
    if (s1.size() < s2.size()) {
        return hirschberg_lcs(s2, s1);
    }
    string res;
    hirschberg_rec(s1.begin(), s1.end(), s2.begin(), s2.end(), &res);
    return res;
}

```

Example Usage

```

#include <cassert>

int main() {
    assert(longest_common_subsequence("xmjyauz", "mzjawxu") == "mjau");
    assert(hirschberg_lcs("xmjyauz", "mzjawxu") == "mjau");
    return 0;
}

```

4.5.3 Sequence Alignment

Given two strings, determine their minimum-cost alignment. An alignment of two strings is a transformation of both strings by inserting gap characters `_` in some way to make the final lengths equal. The total cost of an alignment given a `gap_cost` (insertion or deletion cost) and a `sub_cost` (substitution, i.e. mismatch cost) is `gap_cost` times the number of inserted gaps, plus `sub_cost` times the number of indices at which the two aligned strings differ.

- `align_sequences(s1, s2, gap_cost, sub_cost)` returns a pair of aligned strings for strings `s1` and `s2`, using a classic dynamic programming approach. This implementation first computes $dp[i][j]$ (the cost of aligning the length i prefix of `s1` with the length j prefix of `s2`) before following the path backwards to construct the answer. For `gap_cost = sub_cost = 1`, $dp[n][m]$ will be the Levenshtein edit distance, where n and m are the lengths of `s1` and `s2`, respectively.
- `hirschberg_align_sequences(s1, s2, gap_cost, sub_cost)` returns the sequence alignment of strings `s1` and `s2` using the more memory efficient Hirschberg's algorithm.

Time Complexity: $O(n \cdot m)$ per call to `align_sequences(s1, s2)` as well as `hirschberg_align_sequences(s1, s2)`, where n and m are the lengths of `s1` and `s2`, respectively.

Space Complexity:

- $O(n \cdot m)$ auxiliary heap space for `align_sequences(s1, s2)`, where n and m are the lengths of `s1` and `s2`, respectively.
- $O(\log \max(n, m))$ auxiliary stack space and $O(\min(n, m))$ auxiliary heap space for `hirschberg_align_sequences(s1, s2)`, where n and m are the lengths of `s1` and `s2`, respectively.

```
#include <algorithm>
#include <string>
#include <utility>
#include <vector>
using std::string;

std::pair<string, string> align_sequences(
    const string &s1, const string &s2, int gap_cost = 1, int sub_cost = 1
) {
    int n = s1.size(), m = s2.size();
    std::vector<std::vector<int>>> dp(n + 1, std::vector<int>(m + 1, 0));
    for (int i = 0; i <= n; i++) {
        dp[i][0] = i * gap_cost;
    }
    for (int j = 0; j <= m; j++) {
        dp[0][j] = j * gap_cost;
    }
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            dp[i][j] = (s1[i - 1] == s2[j - 1]) ? dp[i - 1][j - 1]
                : std::min(
                    dp[i - 1][j - 1] + sub_cost,
                    std::min(dp[i - 1][j], dp[i][j - 1]) + gap_cost
                );
        }
    }
    string res1, res2;
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (s1[i - 1] == s2[j - 1] || dp[i][j] == dp[i - 1][j - 1] + sub_cost) {
            res1 += s1[--i];
            res2 += s2[--j];
        } else if (dp[i][j] == dp[i - 1][j] + gap_cost) {
            res1 += s1[--i];
            res2 += '_';
        } else if (dp[i][j] == dp[i][j - 1] + gap_cost) {
            res1 += '_';
            res2 += s2[--j];
        }
    }
    while (i > 0 || j > 0) {
        res1 += (i > 0) ? s1[--i] : '_';
        res2 += (j > 0) ? s2[--j] : '_';
    }
    std::reverse(res1.begin(), res1.end());
    std::reverse(res2.begin(), res2.end());
    return std::make_pair(res1, res2);
}

template<class It>
std::vector<int> row_cost(It lo1, It hi1, It lo2, It hi2, int gap_cost, int sub_cost) {
    std::vector<int> res(std::distance(lo2, hi2) + 1, prev(res));
    for (It it1 = lo1; it1 != hi1; ++it1) {
        res.swap(prev);
        int i = 0;
        for (It it2 = lo2; it2 != hi2; ++it2) {
            res[i + 1] = (*it1 == *it2) ? prev[i] : std::min(prev[i] + sub_cost, res[i] + gap_cost);
        }
    }
}
```

```

        i++;
    }
}
return res;
}

template<class It>
void hirschberg_rec(
    It lo1, It hi1, It lo2, It hi2, string *res1, string *res2, int gap_cost, int sub_cost
) {
    if (lo1 == hi1) {
        for (It it2 = lo2; it2 != hi2; ++it2) {
            *res1 += '_';
            *res2 += *it2;
        }
        return;
    }
    if (lo1 + 1 == hi1) {
        It pos = std::find(lo2, hi2, *lo1);
        bool insert = (pos == hi2) && (gap_cost * (hi2 - lo2 + 1) < sub_cost);
        if (lo2 == hi2 || insert) {
            *res1 += *lo1;
            *res2 += '_';
        }
        for (It it2 = lo2; it2 != hi2; ++it2) {
            *res1 += (pos == it2 || (!insert && it2 == lo2)) ? *lo1 : '_';
            *res2 += *it2;
        }
        return;
    }
    It mid1 = lo1 + (hi1 - lo1) / 2;
    std::reverse_iterator<It> rlo1(hi1), rmid1(mid1), rlo2(hi2), rhi2(lo2);
    std::vector<int> fwd = row_cost(lo1, mid1, lo2, hi2, gap_cost, sub_cost);
    std::vector<int> rev = row_cost(rlo1, rmid1, rlo2, rhi2, gap_cost, sub_cost);
    It mid2 = lo2;
    int mincost = -1;
    for (int i = 0, j = (int)rev.size() - 1; i < (int)fwd.size(); i++, j--) {
        if (mincost < 0 || fwd[i] + rev[j] < mincost) {
            mincost = fwd[i] + rev[j];
            mid2 = lo2 + i;
        }
    }
    hirschberg_rec(lo1, mid1, lo2, mid2, res1, res2, gap_cost, sub_cost);
    hirschberg_rec(mid1, hi1, mid2, hi2, res1, res2, gap_cost, sub_cost);
}

std::pair<string, string> hirschberg_align_sequences(
    const string &s1, const string &s2, int gap_cost = 1, int sub_cost = 1
) {
    if (s1.size() < s2.size()) {
        return hirschberg_align_sequences(s2, s1, gap_cost, sub_cost);
    }
    string res1, res2;
    hirschberg_rec(s1.begin(), s1.end(), s2.begin(), s2.end(), &res1, &res2, gap_cost, sub_cost);
    return std::make_pair(res1, res2);
}

```

Example Usage

```

#include <cassert>

int main() {
    assert(aligned_sequences("AGGGCT", "AGGCA", 2, 3) == make_pair(string("AGGGCT"), string("A_GGCA")));
    assert(
        hirschberg_align_sequences("AGGGCT", "AGGCA", 2, 3) ==
        make_pair(string("AGGGCT"), string("A_GGCA"))
    );
}

```

```

    );
    return 0;
}

```

4.6 Suffix Array and LCP

4.6.1 Suffix Array and LCP (Manber-Myers)

Given a string `s`, a suffix array is the array of the smallest starting positions for the sorted suffixes of `s`. That is, the i -th position of the suffix array stores the starting position of the i -th lexicographically smallest suffix of `s`. For example, `s = "cab"` has the suffixes "cab", "ab", and "b". When sorted, the indices of the suffixes are "ab", "b", and "cab", so the suffix array (assuming zero-based indices) is `[1, 2, 0]`.

For a string `s` of length n , the longest common prefix (LCP) array of length $n - 1$ stores the lengths of the longest common prefixes between all pairs of lexicographically adjacent suffixes in `s`. For example, "baa" has the sorted suffixes "a", "aa", and "baa", with an LCP array of `[1, 0]`.

- `suffix_array(s)` constructs a suffix array from the given string `s` using the original Manber-Myers gap partitioning algorithm with a comparison-based sort.
- `get_sa()` returns the constructed suffix array.
- `get_lcp()` returns the corresponding LCP array for the suffix array.
- `find(needle)` returns one position that `needle` occurs in `s` (not necessarily the first), or `std::string::npos` if it cannot be found. For a `needle` of length m , this implementation uses an $O(m \log n)$ binary search, but can be optimized to $O(m + \log n)$ by first computing the LCP-LR array using the LCP array.

Time Complexity:

- $O(n \log^2 n)$ per call to the constructor, where n is the length of `s`.
- $O(1)$ per call to `get_sa()`.
- $O(n)$ per call to `get_lcp()`, where n is the length of `s`.
- $O(m \log n)$ per call to `find(needle)`, where m is the length of `needle` and n is the length of `s`.

Space Complexity:

- $O(n)$ auxiliary for storage of the suffix and LCP arrays, where n is the length of `s`.
- $O(n)$ auxiliary heap space for the constructor.
- $O(1)$ auxiliary space for all other operations.

```

#include <algorithm>
#include <string>
#include <utility>
#include <vector>
using std::string;

class suffix_array {
    struct comp {
        const std::vector<std::pair<int, int>> &rank;
        comp(const std::vector<std::pair<int, int>> &rank) : rank(rank) {}
        bool operator()(int i, int j) { return rank[i] < rank[j]; }
    };

    string s;
    std::vector<int> sa, rank;

public:
    suffix_array(const string &s) : s(s), sa(s.size()), rank(s.size()) {
        int n = s.size();
        for (int i = 0; i < n; i++) {
            sa[i] = i;

```

```

    rank[i] = (int)s[i];
}
std::vector<std::pair<int, int>> rank2(n);
for (int gap = 1; gap < n; gap *= 2) {
    for (int i = 0; i < n; i++) {
        rank2[i] = std::make_pair(rank[i], i + gap < n ? rank[i + gap] + 1 : 0);
    }
    std::sort(sa.begin(), sa.end(), comp(rank2));
    for (int i = 0; i < n; i++) {
        rank[sa[i]] = (i > 0 && rank2[sa[i - 1]] == rank2[sa[i]]) ? rank[sa[i - 1]] : i;
    }
}
}

std::vector<int> get_sa() { return sa; }

std::vector<int> get_lcp() {
    int n = s.size();
    std::vector<int> lcp(n - 1);
    for (int i = 0, k = 0; i < n; i++) {
        if (rank[i] < n - 1) {
            int j = sa[rank[i] + 1];
            while (std::max(i, j) + k < n && s[i + k] == s[j + k]) {
                k++;
            }
            lcp[rank[i]] = k;
            if (k > 0) {
                k--;
            }
        }
    }
    return lcp;
}

size_t find(const string &needle) {
    int lo = 0, hi = (int)s.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int cmp = s.compare(sa[mid], needle.size(), needle);
        if (cmp < 0) {
            lo = mid + 1;
        } else if (cmp > 0) {
            hi = mid - 1;
        } else {
            return sa[mid];
        }
    }
    return string::npos;
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    suffix_array sa("banana");
    vector<int> sarr = sa.get_sa(), lcp = sa.get_lcp();
    int sarr_expected[] = {5, 3, 1, 0, 4, 2};
    int lcp_expected[] = {1, 3, 0, 0, 2};
    assert(equal(sarr.begin(), sarr.end(), sarr_expected));
    assert(equal(lcp.begin(), lcp.end(), lcp_expected));
    assert(sa.find("ana") == 1);
    assert(sa.find("x") == string::npos);
    return 0;
}

```

```
}

```

4.6.2 Suffix Array and LCP (Counting Sort)

Given a string `s`, a suffix array is the array of the smallest starting positions for the sorted suffixes of `s`. That is, the i -th position of the suffix array stores the starting position of the i -th lexicographically smallest suffix of `s`. For example, `s = "cab"` has the suffixes "cab", "ab", and "b". When sorted, the indices of the suffixes are "ab", "b", and "cab", so the suffix array (assuming zero-based indices) is `[1, 2, 0]`.

For a string `s` of length n , the longest common prefix (LCP) array of length $n - 1$ stores the lengths of the longest common prefixes between all pairs of lexicographically adjacent suffixes in `s`. For example, "baa" has the sorted suffixes "a", "aa", and "baa", with an LCP array of `[1, 0]`.

- `suffix_array(s)` constructs a suffix array from the given string `s` using the original Manber-Myers gap partitioning algorithm with a counting sort instead of a comparison-based sort to reduce the running time to $O(n \log n)$.
- `get_sa()` returns the constructed suffix array.
- `get_lcp()` returns the corresponding LCP array for the suffix array.
- `find(needle)` returns one position that `needle` occurs in `s` (not necessarily the first), or `std::string::npos` if it cannot be found. For a `needle` of length m , this implementation uses an $O(m \log n)$ binary search, but can be optimized to $O(m + \log n)$ by first computing the LCP-LR array using the LCP array.

Time Complexity:

- $O(n \log n)$ per call to the constructor, where n is the length of `s`.
- $O(1)$ per call to `get_sa()`.
- $O(n)$ per call to `get_lcp()`, where n is the length of `s`.
- $O(m \log n)$ per call to `find(needle)`, where m is the length of `needle` and n is the length of `s`.

Space Complexity:

- $O(n)$ auxiliary for storage of the suffix and LCP arrays, where n is the length of `s`.
- $O(n)$ auxiliary heap space for the constructor.
- $O(1)$ auxiliary space for all other operations.

```
#include <algorithm>
#include <string>
#include <utility>
#include <vector>
using std::string;

class suffix_array {
    struct comp {
        const string &s;
        comp(const string &s) : s(s) {}
        bool operator()(int i, int j) { return s[i] < s[j]; }
    };

    string s;
    std::vector<int> sa, rank;

public:
    suffix_array(const string &s) : s(s), sa(s.size()), rank(s.size()) {
        int n = s.size();
        for (int i = 0; i < n; i++) {
            sa[i] = n - 1 - i;
            rank[i] = (int)s[i];
        }
        std::stable_sort(sa.begin(), sa.end(), comp(s));
        for (int gap = 1; gap < n; gap *= 2) {
            std::vector<int> prev_rank(rank), prev_sa(sa), cnt(n);
```

```

    for (int i = 0; i < n; i++) {
        cnt[i] = i;
    }
    for (int i = 0; i < n; i++) {
        rank[sa[i]] = (i > 0 && prev_rank[sa[i - 1]] == prev_rank[sa[i]] && sa[i - 1] + gap < n &&
            prev_rank[sa[i - 1] + gap / 2] == prev_rank[sa[i] + gap / 2])
            ? rank[sa[i - 1]]
            : i;
    }
    for (int i = 0; i < n; i++) {
        int s1 = prev_sa[i] - gap;
        if (s1 >= 0) {
            sa[cnt[rank[s1]]++] = s1;
        }
    }
}

std::vector<int> get_sa() { return sa; }

std::vector<int> get_lcp() {
    int n = s.size();
    std::vector<int> lcp(n - 1);
    for (int i = 0, k = 0; i < n; i++) {
        if (rank[i] < n - 1) {
            int j = sa[rank[i] + 1];
            while (std::max(i, j) + k < n && s[i + k] == s[j + k]) {
                k++;
            }
            lcp[rank[i]] = k;
            if (k > 0) {
                k--;
            }
        }
    }
    return lcp;
}

size_t find(const string &needle) {
    int lo = 0, hi = (int)s.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int cmp = s.compare(sa[mid], needle.size(), needle);
        if (cmp < 0) {
            lo = mid + 1;
        } else if (cmp > 0) {
            hi = mid - 1;
        } else {
            return sa[mid];
        }
    }
    return string::npos;
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    suffix_array sa("banana");
    vector<int> sarr = sa.get_sa(), lcp = sa.get_lcp();
    int sarr_expected[] = {5, 3, 1, 0, 4, 2};
    int lcp_expected[] = {1, 3, 0, 0, 2};
    assert(equal(sarr.begin(), sarr.end(), sarr_expected));
}

```

```

assert(equal(lcp.begin(), lcp.end(), lcp_expected));
assert(sa.find("ana") == 1);
assert(sa.find("x") == string::npos);
return 0;
}

```

4.6.3 Suffix Array and LCP (Linear DC3)

Given a string `s`, a suffix array is the array of the smallest starting positions for the sorted suffixes of `s`. That is, the i -th position of the suffix array stores the starting position of the i -th lexicographically smallest suffix of `s`. For example, `s = "cab"` has the suffixes "cab", "ab", and "b". When sorted, the indices of the suffixes are "ab", "b", and "cab", so the suffix array (assuming zero-based indices) is `[1, 2, 0]`.

For a string `s` of length n , the longest common prefix (LCP) array of length $n - 1$ stores the lengths of the longest common prefixes between all pairs of lexicographically adjacent suffixes in `s`. For example, "baa" has the sorted suffixes "a", "aa", and "baa", with an LCP array of `[1, 0]`.

- `suffix_array(s)` constructs a suffix array from the given string `s` using the linear time DC3/skew algorithm by Karkkainen & Sanders (2003) with radix sort.
- `get_sa()` returns the constructed suffix array.
- `get_lcp()` returns the corresponding LCP array for the suffix array.
- `find(needle)` returns one position that `needle` occurs in `s` (not necessarily the first), or `std::string::npos` if it cannot be found. For a `needle` of length m , this implementation uses an $O(m \log n)$ binary search, but can be optimized to $O(m + \log n)$ by first computing the LCP-LR array using the LCP array.

Time Complexity:

- $O(n)$ per call to the constructor, where n is the length of `s`.
- $O(1)$ per call to `get_sa()`.
- $O(n)$ per call to `get_lcp()`, where n is the length of `s`.
- $O(m \log n)$ per call to `find(needle)`, where m is the length of `needle` and n is the length of `s`.

Space Complexity:

- $O(n)$ auxiliary for storage of the suffix and LCP arrays, where n is the length of `s`.
- $O(n)$ auxiliary heap space for the constructor.
- $O(1)$ auxiliary space for all other operations.

```

#include <algorithm>
#include <string>
#include <utility>
#include <vector>
using std::string;

class suffix_array {
    static bool leq(int a1, int a2, int b1, int b2) { return (a1 < b1) || (a1 == b1 && a2 <= b2); }

    static bool leq(int a1, int a2, int a3, int b1, int b2, int b3) {
        return (a1 < b1) || (a1 == b1 && leq(a2, a3, b2, b3));
    }
}

template<class It>
static void radix_pass(It a, It b, It r, int n, int K) {
    std::vector<int> cnt(K + 1);
    for (int i = 0; i < n; i++) {
        cnt[r[a[i]]]++;
    }
    for (int i = 1; i <= K; i++) {
        cnt[i] += cnt[i - 1];
    }
    for (int i = n - 1; i >= 0; i--) {

```



```

    b[--cnt[r[a[i]]]] = a[i];
}
}

template<class It>
static void suffix_array_dc3(It s, It sa, int n, int K) {
    int n0 = (n + 2) / 3, n1 = (n + 1) / 3, n2 = n / 3, n02 = n0 + n2;
    std::vector<int> s12(n02 + 3), sa12(n02 + 3), s0(n0), sa0(n0);
    s12[n02] = s12[n02 + 1] = s12[n02 + 2] = 0;
    sa12[n02] = sa12[n02 + 1] = sa12[n02 + 2] = 0;
    for (int i = 0, j = 0; i < n + n0 - n1; i++) {
        if (i % 3 != 0) {
            s12[j++] = i;
        }
    }
    radix_pass(s12.begin(), sa12.begin(), s + 2, n02, K);
    radix_pass(sa12.begin(), s12.begin(), s + 1, n02, K);
    radix_pass(s12.begin(), sa12.begin(), s, n02, K);
    int name = 0, c0 = -1, c1 = -1, c2 = -1;
    for (int i = 0; i < n02; i++) {
        if (s[sa12[i]] != c0 || s[sa12[i] + 1] != c1 || s[sa12[i] + 2] != c2) {
            name++;
            c0 = s[sa12[i]];
            c1 = s[sa12[i] + 1];
            c2 = s[sa12[i] + 2];
        }
        (sa12[i] % 3 == 1 ? s12[sa12[i] / 3] : s12[sa12[i] / 3 + n0]) = name;
    }
    if (name < n02) {
        suffix_array_dc3(s12.begin(), sa12.begin(), n02, name);
        for (int i = 0; i < n02; i++) {
            s12[sa12[i]] = i + 1;
        }
    } else {
        for (int i = 0; i < n02; i++) {
            sa12[s12[i] - 1] = i;
        }
    }
    for (int i = 0, j = 0; i < n02; i++) {
        if (sa12[i] < n0) {
            s0[j++] = 3 * sa12[i];
        }
    }
    radix_pass(s0.begin(), sa0.begin(), s, n0, K);
    for (int p = 0, t = n0 - n1, k = 0; k < n; k++) {
        int i = (sa12[t] < n0) ? 3 * sa12[t] + 1 : 3 * (sa12[t] - n0) + 2, j = sa0[p];
        if (sa12[t] < n0
            ? leq(s[i], s12[sa12[t] + n0], s[j], s12[j / 3])
            : leq(s[i], s[i + 1], s12[sa12[t] - n0 + 1], s[j], s[j + 1], s12[j / 3 + n0])) {
            sa[k] = i;
            if (++t == n02) {
                for (k++; p < n0; p++, k++) {
                    sa[k] = sa0[p];
                }
            }
        } else {
            sa[k] = j;
            if (++p == n0) {
                for (k++; t < n02; t++, k++) {
                    sa[k] = (sa12[t] < n0) ? 3 * sa12[t] + 1 : 3 * (sa12[t] - n0) + 2;
                }
            }
        }
    }
}

string s;
std::vector<int> sa;

```

```

public:
    suffix_array(const string &s) : s(s), sa(s.size() + 1) {
        int n = s.size();
        std::vector<int> scopy(s.begin(), s.end());
        scopy.resize(n + 4);
        suffix_array_dc3(scopy.begin(), sa.begin(), n + 1, 255);
        sa.erase(sa.begin());
    }

    std::vector<int> get_sa() { return sa; }

    std::vector<int> get_lcp() {
        int n = s.size();
        if (n == 0) {
            return std::vector<int>();
        }
        std::vector<int> rank(n), lcp(n - 1);
        for (int i = 0; i < n; i++) {
            rank[sa[i]] = i;
        }
        for (int i = 0, k = 0; i < n; i++) {
            if (rank[i] < n - 1) {
                int j = sa[rank[i] + 1];
                while (std::max(i, j) + k < n && s[i + k] == s[j + k]) {
                    k++;
                }
                lcp[rank[i]] = k;
                if (k > 0) {
                    k--;
                }
            }
        }
        return lcp;
    }

    size_t find(const string &needle) {
        int lo = 0, hi = (int)s.size() - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            int cmp = s.compare(sa[mid], needle.size(), needle);
            if (cmp < 0) {
                lo = mid + 1;
            } else if (cmp > 0) {
                hi = mid - 1;
            } else {
                return sa[mid];
            }
        }
        return string::npos;
    }
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    suffix_array sa("banana");
    vector<int> sarr = sa.get_sa(), lcp = sa.get_lcp();
    int sarr_expected[] = {5, 3, 1, 0, 4, 2};
    int lcp_expected[] = {1, 3, 0, 0, 2};
    assert(equal(sarr.begin(), sarr.end(), sarr_expected));
    assert(equal(lcp.begin(), lcp.end(), lcp_expected));
    assert(sa.find("ana") == 1);
}

```

```

    assert(sa.find("x") == string::npos);
    return 0;
}

```

4.7 String Data Structures

4.7.1 Trie

Maintain a map of strings to values using an ordered tree data structure. Each node corresponds to a character, and each inserted string corresponds to a path from the root to a node that is flagged as a terminal node.

- `trie()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(s, v)` adds an entry with string key `s` and value `v` to the map, returning `true` if a new entry was added or `false` if the string already exists (in which case the map is unchanged and the old value associated with the string key is preserved).
- `erase(s)` removes the entry with string key `s` from the map, returning `true` if the removal was successful or `false` if the string to be removed was not found.
- `find(s)` returns a pointer to a const value associated with string key `s`, or `NULL` if the key was not found.
- `walk(f)` calls the function `f(s, v)` on each entry of the map, in lexicographically ascending order of the string keys.

Time Complexity:

- $O(n)$ per call to `insert(s, v)`, `erase(s)`, and `find(s)`, where n is the length of `s`. Note that there is a hidden factor of $\log(\text{alphabet_size})$ which can be considered constant, since `char` can only take on `2CHAR_BIT` values. The implementation may be optimized by storing the children of nodes in an `std::unordered_map` in C++11 and later, or an array if a smaller alphabet size is guaranteed.
- $O(l)$ per call to `walk()`, where l is the total length of string keys that are currently in the map.
- $O(1)$ per call to all other operations.

Space Complexity:

- $O(l)$ for storage of the trie, where l is the total length of string keys that are currently in the map.
- $O(n)$ auxiliary stack space for construction, destruction, `walk()`, where n is the maximum length of any string that has been inserted so far.
- $O(n)$ auxiliary stack space for `erase(s)`, where n is the length of `s`.
- $O(1)$ auxiliary for all other operations.

```

#include <cstddef>
#include <map>
#include <string>
#include <utility>
using std::string;

template<class V>
class trie {
    struct node_t {
        V value;
        bool is_terminal;
        std::map<char, node_t*> children;

        node_t() : is_terminal(false) {}
    } *root;

    typedef typename std::map<char, node_t*>::iterator cit;

```

```

static bool erase(node_t *n, const string &s, int i) {
    if (i == (int)s.size()) {
        if (!n->is_terminal) {
            return false;
        }
        n->is_terminal = false;
        return true;
    }
    cit it = n->children.find(s[i]);
    if (it == n->children.end() || !erase(it->second, s, i + 1)) {
        return false;
    }
    if (it->second->children.empty()) {
        delete it->second;
        n->children.erase(it);
    }
    return true;
}

template<class KVFunction>
static void walk(node_t *n, string &s, KVFunction f) {
    if (n->is_terminal) {
        f(s, n->value);
    }
    for (cit it = n->children.begin(); it != n->children.end(); ++it) {
        s += it->first;
        walk(it->second, s, f);
        s.pop_back();
    }
}

static void clean_up(node_t *n) {
    for (cit it = n->children.begin(); it != n->children.end(); ++it) {
        clean_up(it->second);
    }
    delete n;
}

int num_terminals;

public:
    trie() : root(new node_t()), num_terminals(0) {}

    ~trie() { clean_up(root); }
    int size() const { return num_terminals; }
    bool empty() const { return num_terminals == 0; }

    bool insert(const string &s, const V &v) {
        node_t *n = root;
        for (int i = 0; i < (int)s.size(); i++) {
            cit it = n->children.find(s[i]);
            if (it == n->children.end()) {
                n->children[s[i]] = new node_t();
            }
            n = n->children[s[i]];
        }
        if (n->is_terminal) {
            return false;
        }
        num_terminals++;
        n->is_terminal = true;
        n->value = v;
        return true;
    }

    bool erase(const string &s) {
        if (erase(root, s, 0)) {
            num_terminals--;
        }
    }

```

```

        return true;
    }
    return false;
}

const V *find(const string &s) const {
    node_t *n = root;
    for (int i = 0; i < (int)s.size(); i++) {
        cit it = n->children.find(s[i]);
        if (it == n->children.end()) {
            return NULL;
        }
        n = it->second;
    }
    return n->is_terminal ? &(n->value) : NULL;
}

template<class KVFunction>
void walk(KVFunction f) const {
    string s = "";
    walk(root, s, f);
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

void print_entry(const string &k, int v) {
    cout << "(" << k << ", " << v << ")" << endl;
}

int main() {
    string s[9] = {"", "a", "to", "tea", "ted", "ten", "i", "in", "inn"};
    trie<int> t;
    assert(t.empty());
    for (int i = 0; i < 9; i++) {
        assert(t.insert(s[i], i));
    }
    t.walk(print_entry);
    assert(!t.empty());
    assert(t.size() == 9);
    assert(!t.insert(s[0], 2));
    assert(t.size() == 9);
    assert(*t.find("") == 0);
    assert(*t.find("ten") == 5);
    assert(t.erase("tea"));
    assert(t.size() == 8);
    assert(t.find("tea") == NULL);
    assert(t.erase(""));
    assert(t.find("") == NULL);
    return 0;
}

```

Example Output

```

("", 0)
("a", 1)
("i", 6)
("in", 7)
("inn", 8)
("tea", 3)
("ted", 4)

```

```
("ten", 5)
("to", 2)
```

4.7.2 Radix Tree

Maintain a map of strings to values using an ordered tree data structure. Each node corresponds to a substring of an inserted string, and each inserted string corresponds to a path from the root to a node that is flagged as a terminal node. Contrary to a regular trie, a radix tree is more space efficient as it combines chains of nodes with only a single child.

- `radix_tree()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(s, v)` adds an entry with string key `s` and value `v` to the map, returning `true` if a new entry was added or `false` if the string already exists (in which case the map is unchanged and the old value associated with the string key is preserved).
- `erase(s)` removes the entry with string key `s` from the map, returning `true` if the removal was successful or `false` if the string to be removed was not found.
- `find(s)` returns a pointer to a const value associated with string key `s`, or `NULL` if the key was not found.
- `walk(f)` calls the function `f(s, v)` on each entry of the map, in lexicographically ascending order of the string keys.

Time Complexity:

- $O(n)$ per call to `insert(s, v)`, `erase(s)`, and `find(s)`, where n is the length of `s`. Note that there is a hidden factor of $\log n$ due to map lookups, which can be considered constant amortized. The implementation may be optimized by storing the children of nodes in an `std::unordered_map` in C++11 and later, or a `std::vector<pair<string, node_t*>>`, since the only container operations required are iteration over the `(key, child)` pairs and inserting a new pair. Sticking with an (ordered) `std::map`, we can optimize all operations by using `map.lower_bound()`, a binary tree search for a child with a shared prefix, instead of iteration.
- $O(l)$ per call to `walk()`, where l is the total length of string keys that are currently in the map.
- $O(1)$ per call to all other operations.

Space Complexity:

- $O(l)$ for storage of the radix tree, where l is the total length of string keys that are currently in the map.
- $O(n)$ auxiliary stack space for construction, destruction, `walk()`, where n is the maximum length of any string that has been inserted so far.
- $O(n)$ auxiliary stack space for `erase(s)`, where n is the length of `s`.
- $O(1)$ auxiliary for all other operations.

```
#include <cstddef>
#include <map>
#include <string>
#include <utility>
using std::string;

template<class V>
class radix_tree {
    struct node_t {
        V value;
        bool is_terminal;
        std::map<string, node_t *> children;

        node_t(const V &value = V(), bool is_terminal = false)
            : value(value), is_terminal(is_terminal) {}
    } *root;

    typedef typename std::map<string, node_t *>::iterator cit;

    static int lcp_len(const string &s1, const string &s2, int s2start) {
```

```

int i = 0;
for (int j = s2start; i < (int)s1.size() && j < (int)s2.size(); i++, j++) {
    if (s1[i] != s2[j]) {
        break;
    }
}
return i;
}

```

```

static bool insert(node_t *n, const string &s, int i, const V &v) {
    if (i == (int)s.size()) {
        if (n->is_terminal) {
            return false;
        }
        n->is_terminal = true;
        return true;
    }
    for (cit it = n->children.begin(); it != n->children.end(); ++it) {
        int len = lcp_len(it->first, s, i);
        if (len == 0) {
            continue;
        }
        if (len == (int)it->first.size()) {
            return insert(it->second, s, i + len, v);
        }
        string left = it->first.substr(0, len);
        string right = it->first.substr(len);
        node_t *tmp = new node_t();
        tmp->children[right] = it->second;
        n->children.erase(it);
        n->children[left] = tmp;
        if (len == (int)s.size() - i) {
            tmp->value = v;
            tmp->is_terminal = true;
            return true;
        }
        return insert(tmp, s, i + len, v);
    }
    n->children[s.substr(i)] = new node_t(v, true);
    return true;
}

```

```

static bool erase(node_t *n, const string &s, int i) {
    if (i == (int)s.size()) {
        if (!n->is_terminal) {
            return false;
        }
        n->is_terminal = false;
        return true;
    }
    for (cit it = n->children.begin(); it != n->children.end(); ++it) {
        int len = lcp_len(it->first, s, i);
        if (len == 0) {
            continue;
        }
        node_t *child = it->second;
        if (!erase(child, s, i + len)) {
            return false;
        }
        if (child->children.empty()) {
            delete child;
            n->children.erase(it);
        } else if (child->children.size() == 1) {
            node_t *grandchild = child->children.begin()->second;
            if (!child->is_terminal) {
                string merged_key(it->first + child->children.begin()->first);
                child->value = grandchild->value;
                child->is_terminal = grandchild->is_terminal;
            }
        }
    }
}

```

```

        child->children = grandchild->children;
        delete grandchild;
        n->children.erase(it);
        n->children[merged_key] = child;
    }
}
return true;
}
return false;
}

template<class KVFunction>
static void walk(node_t *n, string &s, KVFunction f) {
    if (n->is_terminal) {
        f(s, n->value);
    }
    for (cit it = n->children.begin(); it != n->children.end(); ++it) {
        s += it->first;
        walk(it->second, s, f);
        s.pop_back();
    }
}

static void clean_up(node_t *n) {
    for (cit it = n->children.begin(); it != n->children.end(); ++it) {
        clean_up(it->second);
    }
    delete n;
}

int num_terminals;

public:
    radix_tree() : root(new node_t()), num_terminals(0) {}

    ~radix_tree() { clean_up(root); }
    int size() const { return num_terminals; }
    bool empty() const { return num_terminals == 0; }

    bool insert(const string &s, const V &v) {
        if (insert(root, s, 0, v)) {
            num_terminals++;
            return true;
        }
        return false;
    }

    bool erase(const string &s) {
        if (erase(root, s, 0)) {
            num_terminals--;
            return true;
        }
        return false;
    }

    const V *find(const string &s) const {
        node_t *n = root;
        int i = 0;
        while (i < (int)s.size()) {
            bool found = false;
            ;
            for (cit it = n->children.begin(); it != n->children.end(); ++it) {
                if (it->first[0] == s[i]) {
                    i += lcp_len(it->first, s, i);
                    n = it->second;
                    found = true;
                    break;
                }
            }

```



```

    }
    if (!found) {
        return NULL;
    }
}
return n->is_terminal ? &(n->value) : NULL;
}

template<class KVFunction>
void walk(KVFunction f) const {
    string s = "";
    walk(root, s, f);
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

void print_entry(const string &k, int v) {
    cout << "(" << k << ", " << v << ")" << endl;
}

int main() {
    string s[9] = {"", "a", "to", "tea", "ted", "ten", "i", "in", "inn"};
    radix_tree<int> t;
    assert(t.empty());
    for (int i = 0; i < 9; i++) {
        assert(t.insert(s[i], i));
    }
    t.walk(print_entry);
    assert(!t.empty());
    assert(t.size() == 9);
    assert(!t.insert(s[0], 2));
    assert(t.size() == 9);
    assert(t.find("") && *t.find("") == 0);
    assert(*t.find("ten") == 5);
    assert(t.erase("tea"));
    assert(t.size() == 8);
    assert(t.find("tea") == NULL);
    assert(t.erase(""));
    assert(t.find("") == NULL);
    return 0;
}

```

Example Output

```

("", 0)
("a", 1)
("i", 6)
("in", 7)
("inn", 8)
("tea", 3)
("ted", 4)
("ten", 5)
("to", 2)

```

4.7.3 Suffix Automaton

Maintains a compact deterministic automaton representing all substrings of a string. A suffix automaton is useful for substring membership queries, counting distinct substrings, finding longest common substrings, and many

other string dynamic programming tasks.

Each state represents an equivalence class of substrings with the same set of ending positions. The value `st[v].len` is the maximum length represented by state `v`, while `st[v].link` points to the state representing the longest proper suffix of that class.

- `suffix_automaton()` constructs an empty automaton.
- `suffix_automaton(s)` constructs the automaton for string `s`.
- `extend(c)` appends character `c` to the current string.
- `contains(t)` returns whether string `t` occurs as a substring.
- `count_distinct_substrings()` returns the number of distinct nonempty substrings.
- `longest_common_substring(t)` returns one longest substring common to the built string and string `t`.

Time Complexity:

- $O(n \log A)$ to construct the automaton for a string of length n , where A is the alphabet size, due to ordered map transitions.
- $O(m \log A)$ per call to `contains(t)` or `longest_common_substring(t)`, where m is the length of `t`.
- $O(n)$ per call to `count_distinct_substrings()`.

Space Complexity:

- $O(n)$ transition storage and $O(n)$ states.
- $O(1)$ auxiliary per character processed.

```
#include <map>
#include <string>
#include <vector>
using std::string;

class suffix_automaton {
    struct state {
        int len, link, first_pos;
        std::map<char, int> next;

        state() : len(0), link(-1), first_pos(-1) {}
    };

    std::vector<state> st;
    int last;

public:
    suffix_automaton() : st(1), last(0) {}

    explicit suffix_automaton(const string &s) : st(1), last(0) {
        for (int i = 0; i < (int)s.size(); i++) {
            extend(s[i]);
        }
    }

    void extend(char c) {
        int cur = st.size();
        st.push_back(state());
        st[cur].len = st[last].len + 1;
        st[cur].first_pos = st[cur].len - 1;

        int p = last;
        while (p != -1 && !st[p].next.count(c)) {
            st[p].next[c] = cur;
            p = st[p].link;
        }
        if (p == -1) {
            st[cur].link = 0;
        } else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len) {
```

```

    st[cur].link = q;
} else {
    int clone = st.size();
    st.push_back(st[q]);
    st[clone].len = st[p].len + 1;
    while (p != -1 && st[p].next[c] == q) {
        st[p].next[c] = clone;
        p = st[p].link;
    }
    st[q].link = st[cur].link = clone;
}
}
last = cur;
}

bool contains(const string &t) const {
    int v = 0;
    for (int i = 0; i < (int)t.size(); i++) {
        std::map<char, int>::const_iterator it = st[v].next.find(t[i]);
        if (it == st[v].next.end()) {
            return false;
        }
        v = it->second;
    }
    return true;
}

long long count_distinct_substrings() const {
    long long res = 0;
    for (int v = 1; v < (int)st.size(); v++) {
        res += st[v].len - st[st[v].link].len;
    }
    return res;
}

string longest_common_substring(const string &t) const {
    int v = 0, len = 0, best = 0, best_pos = -1;
    for (int i = 0; i < (int)t.size(); i++) {
        while (v && !st[v].next.count(t[i])) {
            v = st[v].link;
            len = st[v].len;
        }
        std::map<char, int>::const_iterator it = st[v].next.find(t[i]);
        if (it != st[v].next.end()) {
            v = it->second;
            len++;
        } else {
            v = 0;
            len = 0;
        }
        if (len > best) {
            best = len;
            best_pos = i;
        }
    }
    return best == 0 ? "" : t.substr(best_pos - best + 1, best);
}
};

```

Example Usage

```

#include <cassert>

int main() {
    suffix_automaton sa("ababa");
    assert(sa.contains("aba"));
}

```

```

assert(sa.contains("bab"));
assert(!sa.contains("abba"));
assert(sa.count_distinct_substrings() == 9);
assert(sa.longest_common_substring("zzbabab") == "baba");

suffix_automaton online;
online.extend('a');
online.extend('b');
online.extend('c');
assert(online.contains("bc"));
return 0;
}

```

4.7.4 Eertree

Maintains all distinct palindromic substrings of a string online using an eertree, also known as a palindromic tree. Each non-root node represents one distinct palindrome, and suffix links connect each palindrome to its longest proper palindromic suffix.

The structure has two roots: one of length -1 , which simplifies boundary cases, and one of length 0 , representing the empty palindrome. After each appended character, `last` points to the node for the longest palindromic suffix of the current string.

- `eertree()` constructs an empty palindromic tree.
- `eertree(s)` constructs the tree for string `s`.
- `add(c)` appends character `c` and returns `true` if this creates a new distinct palindrome.
- `count_distinct_palindromes()` returns the number of distinct nonempty palindromic substrings.
- `longest_suffix_length()` returns the length of the current longest palindromic suffix.
- `count_occurrences()` propagates occurrence counts through suffix links and returns `occ[v]` for each node `v`.

Time Complexity:

- $O(n \log A)$ to build the tree for a string of length n , where A is the alphabet size, due to ordered map transitions.
- $O(\log A)$ amortized per call to `add(c)`.
- $O(n)$ per call to `count_occurrences()`.

Space Complexity:

- $O(n)$ transition storage and $O(n)$ nodes.
- $O(1)$ auxiliary per appended character.

```

#include <map>
#include <string>
#include <vector>
using std::string;

class eertree {
    struct node {
        int len, link, occ;
        std::map<char, int> next;

        node(int len = 0) : len(len), link(0), occ(0) {}
    };

    std::vector<node> tree;
    string s;
    int last;

    int get_suffix(int v, int pos, char c) const {
        while (pos - 1 - tree[v].len < 0 || s[pos - 1 - tree[v].len] != c) {
            v = tree[v].link;

```

```

    }
    return v;
}

public:
eertree() : tree(), s(), last(1) {
    tree.push_back(node(-1));
    tree.push_back(node(0));
    tree[0].link = 0;
    tree[1].link = 0;
}

explicit eertree(const string &text) : tree(), s(), last(1) {
    tree.push_back(node(-1));
    tree.push_back(node(0));
    tree[0].link = 0;
    tree[1].link = 0;
    for (int i = 0; i < (int)text.size(); i++) {
        add(text[i]);
    }
}

bool add(char c) {
    s += c;
    int pos = s.size() - 1;
    int cur = get_suffix(last, pos, c);
    std::map<char, int>::iterator it = tree[cur].next.find(c);
    if (it != tree[cur].next.end()) {
        last = it->second;
        tree[last].occ++;
        return false;
    }

    last = tree.size();
    tree.push_back(node(tree[cur].len + 2));
    tree[last].occ = 1;
    tree[cur].next[c] = last;

    if (tree[last].len == 1) {
        tree[last].link = 1;
        return true;
    }
    int link_parent = get_suffix(tree[cur].link, pos, c);
    tree[last].link = tree[link_parent].next[c];
    return true;
}

int count_distinct_palindromes() const { return (int)tree.size() - 2; }
int longest_suffix_length() const { return tree[last].len; }

std::vector<int> count_occurrences() const {
    std::vector<int> occ(tree.size());
    for (int i = 0; i < (int)tree.size(); i++) {
        occ[i] = tree[i].occ;
    }
    for (int i = (int)tree.size() - 1; i >= 2; i--) {
        occ[tree[i].link] += occ[i];
    }
    return occ;
}
};

```

Example Usage

```
#include <cassert>
```

```

int main() {
    eertree t("abacaba");
    assert(t.count_distinct_palindromes() == 7);
    assert(t.longest_suffix_length() == 7);

    eertree online;
    assert(online.add('a'));
    assert(online.add('a'));
    assert(online.add('b'));
    assert(online.add('a'));
    assert(online.count_distinct_palindromes() == 4);
    assert(online.longest_suffix_length() == 3);

    eertree duplicate;
    assert(duplicate.add('a'));
    assert(duplicate.add('b'));
    assert(duplicate.add('c'));
    assert(!duplicate.add('a'));

    std::vector<int> occ = t.count_occurrences();
    assert((int)occ.size() == t.count_distinct_palindromes() + 2);
    return 0;
}

```

4.8 Encoding and Compression

4.8.1 Entropy and Frequency Tables

Computes byte frequency tables and empirical information-theoretic quantities for strings. These primitives are useful when analyzing compression, building coding schemes such as Huffman coding, or comparing symbol distributions.

For a string with symbol probabilities p_i , empirical entropy is $H = -\sum_i p_i \log_2 p_i$ bits per symbol. This is a lower bound on the average number of bits per symbol achievable by any uniquely decodable code for the same independent symbol model.

- `byte_frequencies(s)` returns a length-256 table of byte counts in string `s`.
- `entropy(freq)` returns empirical entropy in bits per symbol for a frequency table.
- `entropy(s)` returns empirical entropy in bits per symbol for string `s`.
- `expected_code_length(freq, length)` returns the average encoded bits per symbol for code lengths `length[c]`.

Time Complexity:

- $O(n + A)$ per call to `entropy(s)`, where n is the string length and $A = 256$.
- $O(A)$ per call to `entropy(freq)` and `expected_code_length(freq, length)`.

Space Complexity:

- $O(A)$ auxiliary heap space for `byte_frequencies(s)`.
- $O(1)$ auxiliary for the other operations.

```

#include <cmath>
#include <string>
#include <vector>
using std::string;

std::vector<int> byte_frequencies(const string &s) {
    std::vector<int> freq(256, 0);
    for (int i = 0; i < (int)s.size(); i++) {
        freq[(unsigned char)s[i]]++;
    }
}

```

```

    return freq;
}

double entropy(const std::vector<int> &freq) {
    int total = 0;
    for (int i = 0; i < (int)freq.size(); i++) {
        total += freq[i];
    }
    if (total == 0) {
        return 0;
    }
    double res = 0;
    for (int i = 0; i < (int)freq.size(); i++) {
        if (freq[i] == 0) {
            continue;
        }
        double p = (double)freq[i] / total;
        res -= p * (log(p) / log(2.0));
    }
    return res;
}

double entropy(const string &s) {
    return entropy(byte_frequencies(s));
}

double expected_code_length(const std::vector<int> &freq, const std::vector<int> &length) {
    int total = 0;
    for (int i = 0; i < (int)freq.size(); i++) {
        total += freq[i];
    }
    if (total == 0) {
        return 0;
    }
    double res = 0;
    for (int i = 0; i < (int)freq.size(); i++) {
        res += (double)freq[i] * length[i] / total;
    }
    return res;
}

```

Example Usage

```

#include <cassert>
#include <cmath>
using namespace std;

bool close(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    vector<int> freq = byte_frequencies("aabb");
    assert(freq['a'] == 2);
    assert(freq['b'] == 2);
    assert(close(entropy(freq), 1.0));
    assert(close(entropy("aaaa"), 0.0));

    vector<int> length(256, 0);
    length['a'] = 1;
    length['b'] = 1;
    assert(close(expected_code_length(freq, length), 1.0));
    return 0;
}

```

4.8.2 Run-Length Encoding

Encodes consecutive equal characters as (`character`, `count`) runs. Run-length encoding is a simple lossless compression technique that works well when strings contain long repeated blocks and poorly when repetitions are rare.

The representation below stores runs in a vector of pairs rather than formatting them as a string. This avoids ambiguity when the original text contains digits or separator characters.

- `run_length_encode(s)` returns the sequence of runs in string `s`.
- `run_length_decode(runs)` reconstructs the original string from a sequence of runs.

Time Complexity: $O(n)$ per call, where n is the input or output length.

Space Complexity:

- $O(r)$ for encoded output, where r is the number of runs.
- $O(n)$ for decoded output.

```
#include <string>
#include <utility>
#include <vector>
using std::string;

std::vector<std::pair<char, int>> run_length_encode(const string &s) {
    std::vector<std::pair<char, int>> res;
    for (int i = 0; i < (int)s.size(); i++) {
        if (res.empty() || res.back().first != s[i]) {
            res.push_back(std::make_pair(s[i], 1));
        } else {
            res.back().second++;
        }
    }
    return res;
}

string run_length_decode(const std::vector<std::pair<char, int>> &runs) {
    string res;
    for (int i = 0; i < (int)runs.size(); i++) {
        res.append(runs[i].second, runs[i].first);
    }
    return res;
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    string s = "aaabcccd";
    vector<pair<char, int>> runs = run_length_encode(s);
    assert(runs.size() == 4);
    assert(runs[0] == make_pair('a', 3));
    assert(runs[2] == make_pair('c', 4));
    assert(run_length_decode(runs) == s);
    return 0;
}
```


4.8.3 Base64 Encoding

Encodes binary data as printable ASCII using Base64, and decodes Base64 text back to bytes. Base64 is an encoding, not encryption or compression: it preserves all input data exactly, but usually increases size by about one third.

This implementation uses the standard alphabet with `+`, `/`, and `=` padding. Whitespace is not skipped by the decoder; clean the input first if line breaks or spaces may appear.

- `base64_encode(bytes)` returns the Base64 representation of string `bytes`.
- `base64_decode(text)` returns the decoded byte string. It assumes `text` is valid padded Base64.

Time Complexity: $O(n)$ per call, where n is the input length.

Space Complexity: $O(n)$ for the output string.

```
#include <string>
#include <vector>
using std::string;

const char BASE64_ALPHABET[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";

string base64_encode(const string &bytes) {
    string res;
    int val = 0, bits = -6;
    for (int i = 0; i < (int)bytes.size(); i++) {
        val = (val << 8) + (unsigned char)bytes[i];
        bits += 8;
        while (bits >= 0) {
            res.push_back(BASE64_ALPHABET[(val >> bits) & 63]);
            bits -= 6;
        }
    }
    if (bits > -6) {
        res.push_back(BASE64_ALPHABET[((val << 8) >> (bits + 8)) & 63]);
    }
    while (res.size() % 4) {
        res.push_back('=');
    }
    return res;
}

string base64_decode(const string &text) {
    std::vector<int> table(256, -1);
    for (int i = 0; i < 64; i++) {
        table[(unsigned char)BASE64_ALPHABET[i]] = i;
    }
    string res;
    int val = 0, bits = -8;
    for (int i = 0; i < (int)text.size(); i++) {
        if (text[i] == '=') {
            break;
        }
        int x = table[(unsigned char)text[i]];
        if (x < 0) {
            break;
        }
        val = (val << 6) + x;
        bits += 6;
        if (bits >= 0) {
            res.push_back((char)((val >> bits) & 255));
            bits -= 8;
        }
    }
    return res;
}
```

Example Usage

```
#include <cassert>

int main() {
    assert(base64_encode("hello") == "aGVsbG8=");
    assert(base64_decode("aGVsbG8=") == "hello");
    assert(base64_decode(base64_encode("abc123!?")) == "abc123!?");
    return 0;
}
```

4.8.4 Huffman Coding

Builds a Huffman code for a string and uses it to encode and decode that string. Huffman coding is a lossless prefix-code compression algorithm: more frequent characters receive shorter bit strings, and no code is a prefix of another code.

The implementation below stores encoded bits as a string of `'0'` and `'1'` characters for clarity. For real compression, pack those bits into bytes. The tree is also needed to decode the bit string, so compressed data normally stores enough metadata to reconstruct the same tree.

- `huffman_tree(text)` constructs a Huffman tree from the character frequencies in `text`.
- `codes()` returns a table mapping each byte value to its bit string.
- `encode(text)` returns the encoded bit string.
- `decode(bits)` returns the decoded text.

Time Complexity:

- $O(n + A \log A)$ to construct the tree, where n is the input length and A is the number of distinct characters.
- $O(n)$ to encode or decode, plus the number of produced or consumed bits.

Space Complexity:

- $O(A)$ for the tree and code table.
- $O(n)$ for the encoded or decoded output.

```
#include <queue>
#include <string>
#include <vector>
using std::string;

class huffman_tree {
    struct node {
        int freq;
        unsigned char ch;
        int left, right;

        node(int freq = 0, unsigned char ch = 0, int left = -1, int right = -1)
            : freq(freq), ch(ch), left(left), right(right) {}
    };

    struct by_frequency {
        const std::vector<node> *nodes;
        explicit by_frequency(const std::vector<node> *nodes = NULL) : nodes(nodes) {}
        bool operator()(int a, int b) const { return (*nodes)[a].freq > (*nodes)[b].freq; }
    };

    std::vector<node> nodes;
    int root;
    std::vector<string> code;

    void build_codes(int u, const string &path) {
        if (nodes[u].left == -1 && nodes[u].right == -1) {
```

```

        code[nodes[u].ch] = path.empty() ? "0" : path;
        return;
    }
    build_codes(nodes[u].left, path + '0');
    build_codes(nodes[u].right, path + '1');
}

public:
explicit huffman_tree(const string &text) : root(-1), code(256) {
    std::vector<int> freq(256, 0);
    for (int i = 0; i < (int)text.size(); i++) {
        freq[(unsigned char)text[i]]++;
    }
    by_frequency cmp(&nodes);
    std::priority_queue<int, std::vector<int>, by_frequency> pq(cmp);
    for (int c = 0; c < 256; c++) {
        if (freq[c] > 0) {
            nodes.push_back(node(freq[c], (unsigned char)c));
            pq.push((int)nodes.size() - 1);
        }
    }
    if (pq.empty()) {
        return;
    }
    while (pq.size() > 1) {
        int a = pq.top();
        pq.pop();
        int b = pq.top();
        pq.pop();
        nodes.push_back(node(nodes[a].freq + nodes[b].freq, 0, a, b));
        pq.push((int)nodes.size() - 1);
    }
    root = pq.top();
    build_codes(root, "");
}

std::vector<string> codes() const { return code; }

string encode(const string &text) const {
    string bits;
    for (int i = 0; i < (int)text.size(); i++) {
        bits += code[(unsigned char)text[i]];
    }
    return bits;
}

string decode(const string &bits) const {
    string text;
    if (root == -1) {
        return text;
    }
    int u = root;
    for (int i = 0; i < (int)bits.size(); i++) {
        u = bits[i] == '0' ? nodes[u].left : nodes[u].right;
        if (nodes[u].left == -1 && nodes[u].right == -1) {
            text.push_back((char)nodes[u].ch);
            u = root;
        }
    }
    return text;
}
};

```

Example Usage

```
#include <cassert>
```

```
int main() {  
    string text = "banana bandana";  
    huffman_tree h(text);  
    string bits = h.encode(text);  
    assert(h.decode(bits) == text);  
    assert(bits.size() < text.size() * 8);  
    return 0;  
}
```

Chapter 5

Graphs

5.1 DFS and Tree Algorithms

5.1.1 Graph Class and Depth-First Search

A graph consists of a set of objects (a.k.a. vertices, or nodes) and a set of connections (a.k.a. edges) between pairs of said objects. A graph may be stored as an adjacency list, which is a space efficient representation that is also time-efficient for traversals.

The following class implements a simple graph using adjacency lists, along with depth-first search and a few other applications. The constructor takes a Boolean argument which specifies whether the instance is a directed or undirected graph. The nodes of the graph are identified by integers indices numbered consecutively starting from 0. The total number of nodes will automatically increase based on the maximum node index passed to `add_edge()` so far.

Time Complexity:

- $O(1)$ amortized per call to `add_edge()`, or $O(\max(n, m))$ for n calls where the maximum node index passed as an argument is m .
- $O(\max(n, m))$ per call for `dfs()`, `has_cycle()`, `is_tree()`, or `is_dag()`, where n is the number of nodes and m is the number of edges.
- $O(1)$ per call to all other public member functions.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary stack space for `dfs()`, `has_cycle()`, `is_tree()`, and `is_dag()`.
- $O(1)$ auxiliary for all other public member functions.

```
#include <algorithm>
#include <vector>

class graph {
    std::vector<std::vector<int>>> adj;
    bool directed;

    template<class ReportFunction>
    void dfs(int n, std::vector<bool> &visit, ReportFunction f) const {
        f(n);
        visit[n] = true;
        std::vector<int>::const_iterator it;
        for (it = adj[n].begin(); it != adj[n].end(); ++it) {
            if (!visit[*it]) {
                dfs(*it, visit, f);
            }
        }
    }
};
```

```

    }
  }
}

bool has_cycle(int n, int prev, std::vector<bool> &visit, std::vector<bool> &onstack) const {
  visit[n] = true;
  onstack[n] = true;
  std::vector<int>::const_iterator it;
  for (it = adj[n].begin(); it != adj[n].end(); ++it) {
    if (directed && onstack[*it]) {
      return true;
    }
    if (!directed && visit[*it] && *it != prev) {
      return true;
    }
    if (!visit[*it] && has_cycle(*it, n, visit, onstack)) {
      return true;
    }
  }
  onstack[n] = false;
  return false;
}

public:
graph(bool directed = true) : directed(directed) {}

int nodes() const { return (int)adj.size(); }
std::vector<int> &operator[](int n) { return adj[n]; }

void add_edge(int u, int v) {
  int n = adj.size();
  if (u >= n || v >= n) {
    adj.resize(std::max(u, v) + 1);
  }
  adj[u].push_back(v);
  if (!directed) {
    adj[v].push_back(u);
  }
}

bool is_directed() const { return directed; }

bool has_cycle() const {
  int n = adj.size();
  std::vector<bool> visit(n, false), onstack(n, false);
  for (int i = 0; i < n; i++) {
    if (!visit[i] && has_cycle(i, -1, visit, onstack)) {
      return true;
    }
  }
  return false;
}

bool is_tree() const { return !directed && !has_cycle(); }
bool is_dag() const { return directed && !has_cycle(); }

template<class ReportFunction>
void dfs(int start, ReportFunction f) const {
  std::vector<bool> visit(adj.size(), false);
  dfs(start, visit, f);
}
};

```

Example Usage

```
#include <cassert>
```

```

#include <iostream>
using namespace std;

void print(int n) {
    cout << n << " ";
}

int main() {
    {
        graph g;
        g.add_edge(0, 1);
        g.add_edge(0, 6);
        g.add_edge(0, 7);
        g.add_edge(1, 2);
        g.add_edge(1, 5);
        g.add_edge(2, 3);
        g.add_edge(2, 4);
        g.add_edge(7, 8);
        g.add_edge(7, 11);
        g.add_edge(8, 9);
        g.add_edge(8, 10);
        cout << "DFS order: ";
        g.dfs(0, print);
        cout << endl;
        assert(g[0].size() == 3);
        assert(g.is_dag());
        assert(!g.has_cycle());
    }
    {
        graph tree(false);
        tree.add_edge(0, 1);
        tree.add_edge(0, 2);
        tree.add_edge(1, 3);
        tree.add_edge(1, 4);
        assert(tree.is_tree());
        assert(!tree.is_dag());
        tree.add_edge(2, 3);
        assert(!tree.is_tree());
    }
    return 0;
}

```

Example Output

```
DFS order: 0 1 2 3 4 5 6 7 8 9 10 11
```

5.1.2 Connected Components

Given an undirected graph, label every node with its connected component and collect the nodes in each component. `connected_components()` applies to a global, pre-populated adjacency list `adj[]` which satisfies the precondition that for every node v in `adj[u]`, node u also exists in `adj[v]`.

This is the standard graph reachability primitive behind many larger routines: run one depth-first search from every unseen node, and all nodes visited by that search belong to the same component. For directed graphs, use a strongly connected components algorithm instead.

- `component_id[u]` stores the index of the component containing node u .
- `components[k]` stores the nodes in component k .

Time Complexity: $O(\max(n, m))$ per call to `connected_components()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.

- $O(n)$ auxiliary stack space for `dfs()`.

```
#include <algorithm>
#include <vector>

const int MAXN = 100;
std::vector<int> adj[MAXN];
std::vector<std::vector<int>> components;
int component_id[MAXN];

void dfs(int u, int id) {
    component_id[u] = id;
    components[id].push_back(u);
    for (int i = 0; i < (int)adj[u].size(); i++) {
        int v = adj[u][i];
        if (component_id[v] == -1) {
            dfs(v, id);
        }
    }
}

void connected_components(int nodes) {
    components.clear();
    std::fill(component_id, component_id + nodes, -1);
    for (int u = 0; u < nodes; u++) {
        if (component_id[u] == -1) {
            components.push_back(std::vector<int>());
            dfs(u, (int)components.size() - 1);
        }
    }
}
```

Example Usage

```
#include <cassert>
using namespace std;

void add_edge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int main() {
    add_edge(0, 1);
    add_edge(1, 2);
    add_edge(3, 4);
    connected_components(6);
    assert(components.size() == 3);
    assert(component_id[0] == component_id[2]);
    assert(component_id[0] != component_id[3]);
    assert(component_id[5] == 2);
    return 0;
}
```

5.1.3 Topological Sorting

Given a directed acyclic graph, find one of possibly many orderings of the nodes such that for every edge from node u to v , u comes before v in the ordering. Depth-first search is used to traverse all nodes in post-order.

`toposort(nodes)` takes a directed graph stored as a global adjacency list with nodes indexed from 0 to `nodes - 1` and assigns a valid topological ordering to the global result vector. An error is thrown if the graph contains a cycle.

Time Complexity: $O(\max(n, m))$ per call to `toposort()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary stack space for `toposort()`.

```
#include <algorithm>
#include <stdexcept>
#include <vector>

const int MAXN = 100;
std::vector<int> adj[MAXN], res;
std::vector<bool> visit(MAXN), done(MAXN);

void dfs(int u) {
    if (visit[u]) {
        throw std::runtime_error("Not a directed acyclic graph.");
    }
    if (done[u]) {
        return;
    }
    visit[u] = true;
    for (int j = 0; j < (int)adj[u].size(); j++) {
        dfs(adj[u][j]);
    }
    visit[u] = false;
    done[u] = true;
    res.push_back(u);
}

void toposort(int nodes) {
    fill(visit.begin(), visit.end(), false);
    fill(done.begin(), done.end(), false);
    res.clear();
    for (int i = 0; i < nodes; i++) {
        if (!done[i]) {
            dfs(i);
        }
    }
    std::reverse(res.begin(), res.end());
}
```

Example Usage

```
#include <iostream>
using namespace std;

int main() {
    adj[0].push_back(3);
    adj[0].push_back(4);
    adj[1].push_back(3);
    adj[2].push_back(4);
    adj[2].push_back(7);
    adj[3].push_back(5);
    adj[3].push_back(6);
    adj[3].push_back(7);
    adj[4].push_back(6);
    toposort(8);
    cout << "The topological order:";
    for (int i = 0; i < (int)res.size(); i++) {
        cout << " " << res[i];
    }
    cout << endl;
    return 0;
}
```

```

}

```

Example Output

The topological order: 2 1 0 4 3 7 6 5

5.1.4 Tree Centers and Diameter

An unweighted tree possesses a center, centroid, and diameter. The following functions apply to a global, pre-populated adjacency list `adj[]` which satisfies the precondition that for every node v in `adj[u]`, node u also exists in `adj[v]`. Nodes in `adj[]` must be numbered with integers between 0 (inclusive) and the total number of nodes (exclusive), as passed in the function arguments.

- `find_centers()` returns a vector of either one or two tree Jordan centers. The Jordan center of a tree is the set of all nodes with minimum eccentricity, that is, the set of all nodes where the maximum distance to all other nodes in the tree is minimal.
- `find_centroid()` returns the node where all of its subtrees have a size less than or equal to $n/2$, where n is the number of nodes in the tree.
- `diameter()` returns the maximum distance between any two nodes in the tree, using a well-known double depth-first search technique.

Time Complexity: $O(\max(n, m))$ per call to `find_centers()`, `find_centroid()`, and `diameter()`, where n is the number of nodes and m is the number of edges.

Space Complexity: $O(n)$ auxiliary stack space for `find_centers()`, `find_centroid()`, and `diameter()`, where n is the number of nodes.

```

#include <utility>
#include <vector>

const int MAXN = 100;
std::vector<int> adj[MAXN];

std::vector<int> find_centers(int nodes) {
    std::vector<int> leaves, degree(nodes);
    for (int i = 0; i < nodes; i++) {
        degree[i] = adj[i].size();
        if (degree[i] <= 1) {
            leaves.push_back(i);
        }
    }
    int removed = leaves.size();
    while (removed < nodes) {
        std::vector<int> nleaves;
        for (int i = 0; i < (int)leaves.size(); i++) {
            int u = leaves[i];
            for (int j = 0; j < (int)adj[u].size(); j++) {
                int v = adj[u][j];
                if (--degree[v] == 1) {
                    nleaves.push_back(v);
                }
            }
        }
        leaves = nleaves;
        removed += leaves.size();
    }
    return leaves;
}

int find_centroid(int nodes, int u = 0, int p = -1) {
    int count = 1;
    bool good_center = true;
    for (int j = 0; j < (int)adj[u].size(); j++) {

```

```

    int v = adj[u][j];
    if (v == p) {
        continue;
    }
    int res = find_centroid(nodes, v, u);
    if (res >= 0) {
        return res;
    }
    int size = -res;
    good_center &= (size <= nodes / 2);
    count += size;
}
good_center &= (nodes - count <= nodes / 2);
return good_center ? u : -count;
}

std::pair<int, int> dfs(int u, int p, int depth) {
    std::pair<int, int> res = std::make_pair(depth, u);
    for (int j = 0; j < (int)adj[u].size(); j++) {
        if (adj[u][j] != p) {
            res = max(res, dfs(adj[u][j], u, depth + 1));
        }
    }
    return res;
}

int diameter() {
    int furthest_node = dfs(0, -1, 0).second;
    return dfs(furthest_node, -1, 0).first;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    int nodes = 6;
    adj[0].push_back(1);
    adj[1].push_back(0);
    adj[1].push_back(2);
    adj[2].push_back(1);
    adj[1].push_back(4);
    adj[4].push_back(1);
    adj[3].push_back(4);
    adj[4].push_back(3);
    adj[4].push_back(5);
    adj[5].push_back(4);
    vector<int> centers = find_centers(nodes);
    assert(centers.size() == 2 && centers[0] == 1 && centers[1] == 4);
    assert(find_centroid(nodes) == 4);
    assert(diameter() == 3);
    return 0;
}

```

5.1.5 Centroid Decomposition

Given a tree, build its centroid decomposition. A centroid is a node whose removal splits the current subtree into connected components of size at most half of that subtree. Recursively choosing centroids creates a decomposition tree of height $O(\log n)$, which is useful for queries based on distances to marked nodes, nearest special node, and other "path through a centroid" problems.

`build_centroid_tree()` applies to a global, pre-populated adjacency list `adj[]` which satisfies the precondition that for every node v in `adj[u]`, node u also exists in `adj[v]`.

- `centroid_parent[u]` stores the parent of node u in the centroid tree.
- The root centroid has parent `-1`.

Time Complexity: $O(n \log n)$ per call to `build_centroid_tree()`, where n is the number of nodes.

Space Complexity:

- $O(n)$ for `subtree_size[]`, `removed[]`, and `centroid_parent[]`.
- $O(n)$ auxiliary stack space for the recursive searches.

```
#include <algorithm>
#include <vector>

const int MAXN = 100;
std::vector<int> adj[MAXN];
int subtree_size[MAXN], centroid_parent[MAXN];
bool removed[MAXN];

int get_size(int u, int p) {
    subtree_size[u] = 1;
    for (int i = 0; i < (int)adj[u].size(); i++) {
        int v = adj[u][i];
        if (v != p && !removed[v]) {
            subtree_size[u] += get_size(v, u);
        }
    }
    return subtree_size[u];
}

int get_centroid(int u, int p, int total) {
    for (int i = 0; i < (int)adj[u].size(); i++) {
        int v = adj[u][i];
        if (v != p && !removed[v] && subtree_size[v] > total / 2) {
            return get_centroid(v, u, total);
        }
    }
    return u;
}

void decompose(int entry, int parent) {
    int total = get_size(entry, -1);
    int centroid = get_centroid(entry, -1, total);
    centroid_parent[centroid] = parent;
    removed[centroid] = true;
    for (int i = 0; i < (int)adj[centroid].size(); i++) {
        int v = adj[centroid][i];
        if (!removed[v]) {
            decompose(v, centroid);
        }
    }
}

void build_centroid_tree(int nodes) {
    std::fill(removed, removed + nodes, false);
    std::fill(centroid_parent, centroid_parent + nodes, -1);
    decompose(0, -1);
}
```

Example Usage

```
#include <cassert>
using namespace std;
```

```

void add_edge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int main() {
    add_edge(0, 1);
    add_edge(1, 2);
    add_edge(1, 3);
    add_edge(3, 4);
    add_edge(3, 5);
    add_edge(5, 6);
    build_centroid_tree(7);
    assert(centroid_parent[3] == -1);
    assert(centroid_parent[1] == 3);
    assert(centroid_parent[5] == 3);
    return 0;
}

```

5.1.6 Prufer Code

Encode and decode labeled trees using Prufer codes. A Prufer code is a sequence of length $n - 2$ that uniquely represents a labeled tree on nodes $0, 1, \dots, n - 1$. It is useful for counting labeled trees, generating test cases, and converting between trees and compact sequences.

- `encode_prufer()` takes a tree adjacency list and returns its Prufer code.
- `decode_prufer()` takes a Prufer code and returns the corresponding tree edges.

Both functions choose the smallest available leaf at each step. This makes the implementation deterministic and matches the usual textbook convention.

Time Complexity: $O(n \log n)$ per call to `encode_prufer()` or `decode_prufer()`, where n is the number of nodes in the tree.

Space Complexity: $O(n)$ auxiliary space per call to `encode_prufer()` or `decode_prufer()`.

```

#include <set>
#include <utility>
#include <vector>

std::vector<int> encode_prufer(const std::vector<std::vector<int>> &tree) {
    int nodes = tree.size();
    std::vector<int> degree(nodes), parent(nodes, -1), code;
    std::set<int> leaves;
    if (nodes <= 2) {
        return code;
    }
    int root = 0;
    for (int u = 0; u < nodes; u++) {
        degree[u] = tree[u].size();
        if (degree[u] == 1) {
            leaves.insert(u);
        } else {
            root = u;
        }
    }
    std::vector<int> stack(1, root);
    parent[root] = -2;
    while (!stack.empty()) {
        int u = stack.back();
        stack.pop_back();
        for (int i = 0; i < (int)tree[u].size(); i++) {
            int v = tree[u][i];
            if (parent[v] == -1) {

```

```

        parent[v] = u;
        stack.push_back(v);
    }
}
}
for (int i = 0; i < nodes - 2; i++) {
    int leaf = *leaves.begin();
    leaves.erase(leaves.begin());
    int p = parent[leaf];
    code.push_back(p);
    if (--degree[p] == 1) {
        leaves.insert(p);
    }
}
return code;
}

std::vector<std::pair<int, int>> decode_prufer(const std::vector<int> &code) {
    int nodes = code.size() + 2;
    std::vector<int> degree(nodes, 1);
    std::set<int> leaves;
    std::vector<std::pair<int, int>> edges;
    for (int i = 0; i < (int)code.size(); i++) {
        degree[code[i]]++;
    }
    for (int u = 0; u < nodes; u++) {
        if (degree[u] == 1) {
            leaves.insert(u);
        }
    }
    for (int i = 0; i < (int)code.size(); i++) {
        int leaf = *leaves.begin();
        leaves.erase(leaves.begin());
        int u = code[i];
        edges.push_back(std::make_pair(leaf, u));
        if (--degree[u] == 1) {
            leaves.insert(u);
        }
    }
    edges.push_back(std::make_pair(*leaves.begin(), *leaves.rbegin()));
    return edges;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<vector<int>> tree(5);
    tree[0].push_back(3);
    tree[1].push_back(3);
    tree[2].push_back(3);
    tree[3].push_back(0);
    tree[3].push_back(1);
    tree[3].push_back(2);
    tree[3].push_back(4);
    tree[4].push_back(3);
    vector<int> code = encode_prufer(tree);
    assert(code.size() == 3 && code[0] == 3 && code[1] == 3 && code[2] == 3);
    vector<pair<int, int>> edges = decode_prufer(code);
    assert(edges.size() == 4);
    return 0;
}

```

5.1.7 Eulerian Cycles

A Eulerian trail is a path in a graph which contains every edge exactly once. An Eulerian cycle or circuit is an Eulerian trail which begins and ends on the same node. A directed graph has an Eulerian cycle if and only if every node has an in-degree equal to its out-degree, and all of its nodes with nonzero degree belong to a single strongly connected component. An undirected graph has an Eulerian cycle if and only if every node has even degree, and all of its nodes with nonzero degree belong to a single connected component.

Given a graph as an adjacency list along with the starting node of the cycle, both functions below return a vector containing all nodes reachable from the starting node in an order which forms an Eulerian cycle. The first node of the cycle will be repeated as the last element of the vector. All nodes of input adjacency lists to both functions must be between 0 and `MAXN - 1`, inclusive. In addition, `euler_cycle_undirected()` requires that for every node v which is found in `adj[u]`, node u must also be found in `adj[v]`.

Time Complexity: $O(\max(n, m))$ per call to either function, where n and m are the numbers of nodes and edges respectively.

Space Complexity:

- $O(n)$ auxiliary heap space for `euler_cycle_directed()`, where n is the number of nodes.
- $O(n^2)$ auxiliary heap space for `euler_cycle_undirected()`, where n is the number of nodes. This can be reduced to $O(m)$ auxiliary heap space on the number of edges if the `used[][]` bit matrix is replaced with an `std::unordered_set<std::pair<int, int>>`.

```
#include <algorithm>
#include <bitset>
#include <vector>

const int MAXN = 100;

std::vector<int> euler_cycle_directed(std::vector<int> adj[], int u) {
    std::vector<int> stack, curr_edge(MAXN), res;
    stack.push_back(u);
    while (!stack.empty()) {
        u = stack.back();
        stack.pop_back();
        while (curr_edge[u] < (int)adj[u].size()) {
            stack.push_back(u);
            u = adj[u][curr_edge[u]++];
        }
        res.push_back(u);
    }
    std::reverse(res.begin(), res.end());
    return res;
}

std::vector<int> euler_cycle_undirected(std::vector<int> adj[], int u) {
    std::bitset<MAXN> used[MAXN];
    std::vector<int> stack, curr_edge(MAXN), res;
    stack.push_back(u);
    while (!stack.empty()) {
        u = stack.back();
        stack.pop_back();
        while (curr_edge[u] < (int)adj[u].size()) {
            int v = adj[u][curr_edge[u]++];
            int mn = std::min(u, v), mx = std::max(u, v);
            if (!used[mn][mx]) {
                used[mn][mx] = true;
                stack.push_back(u);
                u = v;
            }
        }
        res.push_back(u);
    }
    std::reverse(res.begin(), res.end());
}
```

```
    return res;
}
```

Example Usage

```
#include <iostream>
using namespace std;

int main() {
    {
        vector<int> g[5], cycle;
        g[0].push_back(1);
        g[1].push_back(2);
        g[2].push_back(0);
        g[1].push_back(3);
        g[3].push_back(4);
        g[4].push_back(1);
        cycle = euler_cycle_directed(g, 0);
        cout << "Eulerian cycle from 0 (directed):";
        for (int i = 0; i < (int)cycle.size(); i++) {
            cout << " " << cycle[i];
        }
        cout << endl;
    }
    {
        vector<int> g[5], cycle;
        g[0].push_back(1);
        g[1].push_back(0);
        g[1].push_back(2);
        g[2].push_back(1);
        g[2].push_back(0);
        g[0].push_back(2);
        g[1].push_back(3);
        g[3].push_back(1);
        g[3].push_back(4);
        g[4].push_back(3);
        g[4].push_back(1);
        g[1].push_back(4);
        cycle = euler_cycle_undirected(g, 2);
        cout << "Eulerian cycle from 2 (undirected):";
        for (int i = 0; i < (int)cycle.size(); i++) {
            cout << " " << cycle[i];
        }
        cout << endl;
    }
    return 0;
}
```

Example Output

```
Eulerian cycle from 0 (directed): 0 1 3 4 1 2 0
Eulerian cycle from 2 (undirected): 2 1 3 4 1 0 2
```

5.2 BFS and Shortest Path

5.2.1 Shortest Path (BFS)

Given a starting node in an unweighted, directed graph, visit every connected node and determine the minimum distance to each such node. Optionally, output the shortest path to a specific destination node using the shortest-path tree from the predecessor array `pred[]`. `bfs()` applies to a global, pre-populated adjacency list `adj[]` which consists of only nodes numbered with integers between 0 (inclusive) and the total number of nodes

(exclusive), as passed in the function argument.

Time Complexity: $O(\max(n, m))$ per call to `bfs()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary heap space for `bfs()`.

```
#include <queue>
#include <utility>
#include <vector>

const int MAXN = 100, INF = 0x3f3f3f3f;
std::vector<int> adj[MAXN];
int dist[MAXN], pred[MAXN];

void bfs(int nodes, int start) {
    std::vector<bool> visit(nodes, false);
    for (int i = 0; i < nodes; i++) {
        dist[i] = INF;
        pred[i] = -1;
    }
    std::queue<std::pair<int, int>> q;
    q.push(std::make_pair(start, 0));
    while (!q.empty()) {
        int u = q.front().first;
        int d = q.front().second;
        q.pop();
        visit[u] = true;
        for (int j = 0; j < (int)adj[u].size(); j++) {
            int v = adj[u][j];
            if (visit[v]) {
                continue;
            }
            dist[v] = d + 1;
            pred[v] = u;
            q.push(std::make_pair(v, d + 1));
        }
    }
}
```

Example Usage

```
#include <iostream>
using namespace std;

void print_path(int dest) {
    vector<int> path;
    for (int j = dest; pred[j] != -1; j = pred[j]) {
        path.push_back(pred[j]);
    }
    cout << "Take the path: ";
    while (!path.empty()) {
        cout << path.back() << "->";
        path.pop_back();
    }
    cout << dest << "." << endl;
}

int main() {
    int start = 0, dest = 3;
    adj[0].push_back(1);
    adj[0].push_back(3);
    adj[1].push_back(2);
```

```

adj[1].push_back(3);
adj[2].push_back(3);
adj[0].push_back(3);
bfs(4, start);
cout << "The shortest distance from " << start << " to " << dest << " is " << dist[dest] << "."
    << endl;
print_path(dest);
return 0;
}

```

Example Output

The shortest distance from 0 to 3 is 2.
Take the path: 0->1->3.

5.2.2 Shortest Path (0-1 BFS)

Given a starting node in a weighted graph whose edge weights are only 0 or 1, compute the shortest distance to every reachable node. `zero_one_bfs()` applies to a global, pre-populated adjacency list `adj[]` where each edge is stored as `(neighbor, weight)`.

This is a specialized version of Dijkstra's algorithm. Because every relaxation changes the distance by either 0 or 1, a deque maintains nodes in nondecreasing distance order: push weight-0 relaxations to the front and weight-1 relaxations to the back.

Time Complexity: $O(\max(n, m))$ per call to `zero_one_bfs()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary deque space for `zero_one_bfs()`.

```

#include <algorithm>
#include <deque>
#include <utility>
#include <vector>

const int MAXN = 100, INF = 0x3f3f3f3f;
std::vector<std::pair<int, int>> adj[MAXN];
int dist[MAXN], pred[MAXN];

void zero_one_bfs(int nodes, int start) {
    std::fill(dist, dist + nodes, INF);
    std::fill(pred, pred + nodes, -1);
    std::deque<int> dq;
    dist[start] = 0;
    dq.push_front(start);
    while (!dq.empty()) {
        int u = dq.front();
        dq.pop_front();
        for (int i = 0; i < (int)adj[u].size(); i++) {
            int v = adj[u][i].first;
            int w = adj[u][i].second;
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pred[v] = u;
                if (w == 0) {
                    dq.push_front(v);
                } else {
                    dq.push_back(v);
                }
            }
        }
    }
}

```

```

    }
}

```

Example Usage

```

#include <cassert>
using namespace std;

void add_edge(int u, int v, int w) {
    adj[u].push_back(make_pair(v, w));
}

int main() {
    add_edge(0, 1, 1);
    add_edge(0, 2, 0);
    add_edge(2, 1, 0);
    add_edge(1, 3, 1);
    add_edge(2, 3, 1);
    zero_one_bfs(4, 0);
    assert(dist[1] == 0);
    assert(dist[3] == 1);
    assert(pred[1] == 2);
    return 0;
}

```

5.2.3 Shortest Path (Dijkstra)

Given a starting node in a weighted, directed graph with nonnegative weights only, visit every connected node and determine the minimum distance to each such node. Optionally, output the shortest path to a specific destination node using the shortest-path tree from the predecessor array `pred[]`. `dijkstra()` applies to a global, pre-populated adjacency list `adj[]` which must only consist of nodes numbered with integers between 0 (inclusive) and the total number of nodes (exclusive), as passed in the function argument.

Since `std::priority_queue` is by default a max-heap, we simulate a min-heap by negating node distances before pushing them and negating them again after popping them. Alternatively, the container can be declared with the following template arguments (`#include <functional>` to access `std::greater`):

```
priority_queue<pair<int, int>, vector<pair<int, int> >, greater<pair<int, int> > > pq;
```

Dijkstra's algorithm may be modified to support negative edge weights by allowing nodes to be re-visited (removing the visited array check in the inner for-loop). This is known as the Shortest Path Faster Algorithm (SPFA), which has a larger running time of $O(n \cdot m)$ on the number of nodes and edges respectively. While it is as slow in the worst case as the Bellman-Ford algorithm, the SPFA still tends to outperform in the average case.

Time Complexity: $O(m \log n)$ for `dijkstra()`, where m is the number of edges and n is the number of nodes.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(m)$ auxiliary heap space for `dijkstra()`.

```

#include <queue>
#include <utility>
#include <vector>

const int MAXN = 100, INF = 0x3f3f3f3f;
std::vector<std::pair<int, int>> adj[MAXN];
int dist[MAXN], pred[MAXN];

void dijkstra(int nodes, int start) {
    std::vector<bool> visit(nodes, false);
    for (int i = 0; i < nodes; i++) {

```

```

    dist[i] = INF;
    pred[i] = -1;
}
dist[start] = 0;
std::priority_queue<std::pair<int, int>> pq;
pq.push(std::make_pair(0, start));
while (!pq.empty()) {
    int u = pq.top().second;
    pq.pop();
    visit[u] = true;
    for (int j = 0; j < (int)adj[u].size(); j++) {
        int v = adj[u][j].first;
        if (visit[v]) {
            continue;
        }
        if (dist[v] > dist[u] + adj[u][j].second) {
            dist[v] = dist[u] + adj[u][j].second;
            pred[v] = u;
            pq.push(std::make_pair(-dist[v], v));
        }
    }
}
}
}

```

Example Usage

```

#include <iostream>
using namespace std;

void print_path(int dest) {
    vector<int> path;
    for (int j = dest; pred[j] != -1; j = pred[j]) {
        path.push_back(pred[j]);
    }
    cout << "Take the path: ";
    while (!path.empty()) {
        cout << path.back() << "->";
        path.pop_back();
    }
    cout << dest << "." << endl;
}

int main() {
    int start = 0, dest = 3;
    adj[0].push_back(make_pair(1, 2));
    adj[0].push_back(make_pair(3, 8));
    adj[1].push_back(make_pair(2, 2));
    adj[1].push_back(make_pair(3, 4));
    adj[2].push_back(make_pair(3, 1));
    dijkstra(4, start);
    cout << "The shortest distance from " << start << " to " << dest << " is " << dist[dest] << "."
        << endl;
    print_path(dest);
    return 0;
}

```

Example Output

The shortest distance from 0 to 3 is 5.
Take the path: 0->1->2->3.

5.2.4 Shortest Path (Bellman-Ford)

Given a starting node in a weighted, directed graph with possibly negative weights, visit every connected node and determine the minimum distance to each such node. Optionally, output the shortest path to a specific destination node using the shortest-path tree from the predecessor array `pred[]`. `bellman_ford()` applies to a global, pre-populated edge list which must only consist of nodes numbered with integers between 0 (inclusive) and the total number of nodes (exclusive), as passed in the function argument.

This function will also detect whether the graph contains negative-weighted cycles, in which case there is no shortest path and an error will be thrown.

Time Complexity: $O(n \cdot m)$ per call to `bellman_ford()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary heap space for `bellman_ford()`, where n is the number of nodes.

```
#include <stdexcept>
#include <vector>

struct edge {
    int u, v, w;
}; // Edge from u to v with weight w.

const int MAXN = 100, INF = 0x3f3f3f3f;
std::vector<edge> e;
int dist[MAXN], pred[MAXN];

void bellman_ford(int nodes, int start) {
    for (int i = 0; i < nodes; i++) {
        dist[i] = INF;
        pred[i] = -1;
    }
    dist[start] = 0;
    for (int i = 0; i < nodes; i++) {
        for (int j = 0; j < (int)e.size(); j++) {
            if (dist[e[j].v] > dist[e[j].u] + e[j].w) {
                dist[e[j].v] = dist[e[j].u] + e[j].w;
                pred[e[j].v] = e[j].u;
            }
        }
    }
    // Optional: Report negative-weighted cycles.
    for (int i = 0; i < (int)e.size(); i++) {
        if (dist[e[i].v] > dist[e[i].u] + e[i].w) {
            throw std::runtime_error("Negative-weight cycle found.");
        }
    }
}
```

Example Usage

```
#include <iostream>
using namespace std;

void print_path(int dest) {
    vector<int> path;
    for (int j = dest; pred[j] != -1; j = pred[j]) {
        path.push_back(pred[j]);
    }
    cout << "Take the path: ";
    while (!path.empty()) {
```

```

    cout << path.back() << "->";
    path.pop_back();
}
cout << dest << "." << endl;
}

int main() {
    int start = 0, dest = 2;
    e.push_back((edge){0, 1, 1});
    e.push_back((edge){1, 2, 2});
    e.push_back((edge){0, 2, 5});
    bellman_ford(3, start);
    cout << "The shortest distance from " << start << " to " << dest << " is " << dist[dest] << "."
        << endl;
    print_path(dest);
    return 0;
}

```

Example Output

The shortest distance from 0 to 2 is 3.
Take the path: 0->1->2.

5.2.5 Shortest Path (SPFA)

Given a starting node in a weighted directed graph, compute shortest paths even when some edge weights are negative. `spfa()` applies to a global, pre-populated adjacency list `adj[]`, where each edge is stored as `(neighbor, weight)`.

The Shortest Path Faster Algorithm is a queue-based optimization of Bellman-Ford. It is often fast on benign inputs, but it still has Bellman-Ford's worst-case behavior and can be forced to run in $O(n \cdot m)$. Prefer Dijkstra for nonnegative weights, and use SPFA mainly when negative edges are present and the input is not adversarial.

`spfa()` returns `false` if it detects a reachable negative cycle, and returns `true` otherwise.

Time Complexity: $O(n \cdot m)$ in the worst case for `spfa()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary queue space for `spfa()`.

```

#include <algorithm>
#include <queue>
#include <utility>
#include <vector>

const int MAXN = 100, INF = 0x3f3f3f3f;
std::vector<std::pair<int, int>> adj[MAXN];
int dist[MAXN], pred[MAXN], relax_count[MAXN];
bool in_queue[MAXN];

bool spfa(int nodes, int start) {
    std::fill(dist, dist + nodes, INF);
    std::fill(pred, pred + nodes, -1);
    std::fill(relax_count, relax_count + nodes, 0);
    std::fill(in_queue, in_queue + nodes, false);
    std::queue<int> q;
    dist[start] = 0;
    q.push(start);
    in_queue[start] = true;
    while (!q.empty()) {
        int u = q.front();

```

```

    q.pop();
    in_queue[u] = false;
    for (int i = 0; i < (int)adj[u].size(); i++) {
        int v = adj[u][i].first;
        int w = adj[u][i].second;
        if (dist[u] != INF && dist[v] > dist[u] + w) {
            dist[v] = dist[u] + w;
            pred[v] = u;
            if (!in_queue[v]) {
                q.push(v);
                in_queue[v] = true;
                if (++relax_count[v] >= nodes) {
                    return false;
                }
            }
        }
    }
}
return true;
}

```

Example Usage

```

#include <cassert>
using namespace std;

void add_edge(int u, int v, int w) {
    adj[u].push_back(make_pair(v, w));
}

int main() {
    add_edge(0, 1, 4);
    add_edge(0, 2, 5);
    add_edge(1, 2, -2);
    add_edge(2, 3, 3);
    assert(spfa(4, 0));
    assert(dist[3] == 5);
    assert(pred[2] == 1);
    return 0;
}

```

5.2.6 Shortest Path (Floyd-Warshall)

Given a weighted, directed graph with possibly negative weights, determine the minimum distance between all pairs of start and destination nodes in the graph. Optionally, output the shortest path between two nodes using the shortest-path tree precomputed into the `parent[][]` array. `floyd_warshall()` applies to a global adjacency matrix `dist[][]`, which must be initialized using `initialize()` and subsequently populated with weights. After the function call, `dist[u][v]` will have been modified to contain the shortest path from u to v , for all pairs of valid nodes u and v .

This function will also detect whether the graph contains negative-weighted cycles, in which case there is no shortest path and an error will be thrown.

Time Complexity:

- $O(n^2)$ per call to `initialize()`, where n is the number of nodes.
- $O(n^3)$ per call to `floyd_warshall()`.

Space Complexity:

- $O(n^2)$ for storage of the graph, where n is the number of nodes.
- $O(n^2)$ auxiliary heap space for `initialize()` and `floyd_warshall()`.

```

#include <stdexcept>

const int MAXN = 100, INF = 0x3f3f3f3f;
int dist[MAXN][MAXN], parent[MAXN][MAXN];

void initialize(int nodes) {
    for (int i = 0; i < nodes; i++) {
        for (int j = 0; j < nodes; j++) {
            dist[i][j] = (i == j) ? 0 : INF;
            parent[i][j] = j;
        }
    }
}

void floyd_warshall(int nodes) {
    for (int k = 0; k < nodes; k++) {
        for (int i = 0; i < nodes; i++) {
            for (int j = 0; j < nodes; j++) {
                if (dist[i][j] > dist[i][k] + dist[k][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                    parent[i][j] = parent[i][k];
                }
            }
        }
    }
    // Optional: Report negative-weighted cycles.
    for (int i = 0; i < nodes; i++) {
        if (dist[i][i] < 0) {
            throw std::runtime_error("Negative-weight cycle found.");
        }
    }
}

```

Example Usage

```

#include <iostream>
using namespace std;

void print_path(int u, int v) {
    cout << "Take the path " << u;
    while (u != v) {
        u = parent[u][v];
        cout << "->" << u;
    }
    cout << "." << endl;
}

int main() {
    initialize(3);
    int start = 0, dest = 2;
    dist[0][1] = 1;
    dist[1][2] = 2;
    dist[0][2] = 5;
    floyd_warshall(3);
    cout << "The shortest distance from " << start << " to " << dest << " is " << dist[start][dest]
        << "." << endl;
    print_path(start, dest);
    return 0;
}

```

Example Output

The shortest distance from 0 to 2 is 3.
Take the path: 0->1->2.

5.3 Connectivity

5.3.1 Strongly Connected Components (Kosaraju)

Given a directed graph, determine the strongly connected components, that is, the set of all strongly (maximally) connected subgraphs. A subgraph is strongly connected if there is a path between each pair of nodes. Condensing the strongly connected components of a graph into single nodes will result in a directed acyclic graph. `kosaraju()` applies to a global, pre-populated adjacency list `adj[]` which must only consist of nodes numbered with integers between 0 (inclusive) and the total number of nodes (exclusive), as passed in the function argument.

Time Complexity: $O(\max(n, m))$ per call to `kosaraju()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ auxiliary heap space for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary stack space for `kosaraju()`.

```
#include <algorithm>
#include <vector>

const int MAXN = 100;
std::vector<int> adj[MAXN], rev[MAXN];
std::vector<bool> visit(MAXN);
std::vector<std::vector<int>> scc;

void dfs(std::vector<int> g[], std::vector<int> &res, int u) {
    visit[u] = true;
    for (int j = 0; j < (int)g[u].size(); j++) {
        if (!visit[g[u][j]]) {
            dfs(g, res, g[u][j]);
        }
    }
    res.push_back(u);
}

void kosaraju(int nodes) {
    std::fill(visit.begin(), visit.end(), false);
    std::vector<int> order;
    for (int i = 0; i < nodes; i++) {
        rev[i].clear();
        if (!visit[i]) {
            dfs(adj, order, i);
        }
    }
    std::reverse(order.begin(), order.end());
    std::fill(visit.begin(), visit.end(), false);
    for (int i = 0; i < nodes; i++) {
        for (int j = 0; j < (int)adj[i].size(); j++) {
            rev[adj[i][j]].push_back(i);
        }
    }
    scc.clear();
    for (int i = 0; i < (int)order.size(); i++) {
        if (visit[order[i]]) {
            continue;
        }
        std::vector<int> component;
        dfs(rev, component, order[i]);
        scc.push_back(component);
    }
}
```

Example Usage

```
#include <iostream>
using namespace std;

int main() {
    adj[0].push_back(1);
    adj[1].push_back(2);
    adj[1].push_back(4);
    adj[1].push_back(5);
    adj[2].push_back(3);
    adj[2].push_back(6);
    adj[3].push_back(2);
    adj[3].push_back(7);
    adj[4].push_back(0);
    adj[4].push_back(5);
    adj[5].push_back(6);
    adj[6].push_back(5);
    adj[7].push_back(3);
    adj[7].push_back(6);
    kosaraju(8);
    cout << "Components:" << endl;
    for (int i = 0; i < (int)scc.size(); i++) {
        for (int j = 0; j < (int)scc[i].size(); j++) {
            cout << scc[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Example Output

```
1 4 0
7 3 2
5 6
```

5.3.2 Strongly Connected Components (Tarjan)

Given a directed graph, determine the strongly connected components. The strongly connected components of a graph is the set of all strongly (maximally) connected subgraphs. A subgraph is strongly connected if there is a path between each pair of nodes. Condensing the strongly connected components of a graph into single nodes will result in a directed acyclic graph. `tarjan()` applies to a global, pre-populated adjacency list `adj[]` which must only consist of nodes numbered with integers between 0 (inclusive) and the total number of nodes (exclusive), as passed in the function argument.

Time Complexity: $O(\max(n, m))$ per call to `tarjan()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary stack space for `tarjan()`.

```
#include <algorithm>
#include <vector>

const int MAXN = 100, INF = 0x3f3f3f3f;
std::vector<int> adj[MAXN], currstack;
int timer, lowlink[MAXN];
std::vector<bool> visit(MAXN);
std::vector<std::vector<int>> scc;
```

```

void dfs(int u) {
    lowlink[u] = timer++;
    visit[u] = true;
    currstack.push_back(u);
    bool is_component_root = true;
    int v;
    for (int j = 0; j < (int)adj[u].size(); j++) {
        v = adj[u][j];
        if (!visit[v]) {
            dfs(v);
        }
        if (lowlink[u] > lowlink[v]) {
            lowlink[u] = lowlink[v];
            is_component_root = false;
        }
    }
    if (!is_component_root) {
        return;
    }
    std::vector<int> component;
    do {
        v = currstack.back();
        visit[v] = true;
        currstack.pop_back();
        lowlink[v] = INF;
        component.push_back(v);
    } while (u != v);
    scc.push_back(component);
}

void tarjan(int nodes) {
    scc.clear();
    currstack.clear();
    std::fill(lowlink, lowlink + nodes, 0);
    std::fill(visit.begin(), visit.end(), false);
    timer = 0;
    for (int i = 0; i < nodes; i++) {
        if (!visit[i]) {
            dfs(i);
        }
    }
}

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    adj[0].push_back(1);
    adj[1].push_back(2);
    adj[1].push_back(4);
    adj[1].push_back(5);
    adj[2].push_back(3);
    adj[2].push_back(6);
    adj[3].push_back(2);
    adj[3].push_back(7);
    adj[4].push_back(0);
    adj[4].push_back(5);
    adj[5].push_back(6);
    adj[6].push_back(5);
    adj[7].push_back(3);
    adj[7].push_back(6);
    tarjan(8);
    cout << "Components:" << endl;
    for (int i = 0; i < (int)scc.size(); i++) {

```

```

    for (int j = 0; j < (int)scc[i].size(); j++) {
        cout << scc[i][j] << " ";
    }
    cout << endl;
}
return 0;
}

```

Example Output

```

Components:
5 6
7 3 2
4 1 0

```

5.3.3 Cut-points, Bridges and Bridge Forest

Given an undirected graph, compute the following properties of the graph using Tarjan's algorithm. `tarjan()` applies to a global, pre-populated adjacency list `adj[]` which satisfies the precondition that for every node v in `adj[u]`, node u also exists in `adj[v]`. Nodes in `adj[]` must be numbered with integers between 0 (inclusive) and the total number of nodes (exclusive), as passed in the function arguments. `get_block_forest()` applies to a global vector `block[]` which must be already precomputed by a call to `tarjan()`.

A cut-point (i.e. cut-node, or articulation point) is any node whose removal increases the number of connected components in the graph.

A bridge is an edge such that when deleted, the number of connected components in the graph is increased. An edge is a bridge if and only if it is not part of any cycle.

Condensing each maximal component without a bridge into a single node, we can decompose any connected graph into a bridge-block tree (a.k.a. bridge-tree or block-tree), with bridges connecting each block. An unconnected graph will thus decompose into a "bridge-block forest."

Time Complexity: $O(\max(n, m))$ per call to `tarjan()` and `get_block_forest()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges
- $O(n)$ auxiliary stack space for `tarjan()`.
- $O(1)$ auxiliary stack space for `get_block_forest()`.

```

#include <algorithm>
#include <vector>

const int MAXN = 100;
int timer, lowlink[MAXN], tin[MAXN], comp[MAXN];
std::vector<bool> visit(MAXN);
std::vector<int> adj[MAXN], block_forest[MAXN];
std::vector<int> currstack, cutpoints;
std::vector<std::vector<int>> block;
std::vector<std::pair<int, int>> bridges;

void dfs(int u, int p) {
    visit[u] = true;
    lowlink[u] = tin[u] = timer++;
    currstack.push_back(u);
    int v, children = 0;
    bool cutpoint = false;
    for (int j = 0; j < (int)adj[u].size(); j++) {
        v = adj[u][j];
        if (v == p) {
            continue;

```

```

    }
    if (visit[v]) {
        lowlink[u] = std::min(lowlink[u], tin[v]);
    } else {
        dfs(v, u);
        lowlink[u] = std::min(lowlink[u], lowlink[v]);
        cutpoint |= (lowlink[v] >= tin[u]);
        if (lowlink[v] > tin[u]) {
            bridges.push_back(std::make_pair(u, v));
        }
        children++;
    }
}
if (p == -1) {
    cutpoint = (children >= 2);
}
if (cutpoint) {
    cutpoints.push_back(u);
}
if (lowlink[u] == tin[u]) {
    std::vector<int> component;
    do {
        v = currstack.back();
        currstack.pop_back();
        component.push_back(v);
    } while (u != v);
    block.push_back(component);
}
}

void tarjan(int nodes) {
    block.clear();
    bridges.clear();
    cutpoints.clear();
    currstack.clear();
    std::fill(lowlink, lowlink + nodes, 0);
    std::fill(tin, tin + nodes, 0);
    std::fill(visit.begin(), visit.end(), false);
    timer = 0;
    for (int i = 0; i < nodes; i++) {
        if (!visit[i]) {
            dfs(i, -1);
        }
    }
}

void get_block_forest(int nodes) {
    std::fill(comp, comp + nodes, 0);
    for (int i = 0; i < nodes; i++) {
        block_forest[i].clear();
    }
    for (int i = 0; i < (int)block.size(); i++) {
        for (int j = 0; j < (int)block[i].size(); j++) {
            comp[block[i][j]] = i;
        }
    }
    for (int i = 0; i < nodes; i++) {
        for (int j = 0; j < (int)adj[i].size(); j++) {
            if (comp[i] != comp[adj[i][j]]) {
                block_forest[comp[i]].push_back(comp[adj[i][j]]);
            }
        }
    }
}

```

Example Usage

```

#include <iostream>
using namespace std;

void add_edge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int main() {
    add_edge(0, 1);
    add_edge(0, 5);
    add_edge(1, 2);
    add_edge(1, 5);
    add_edge(3, 7);
    add_edge(4, 5);
    tarjan(8);
    get_block_forest(8);
    cout << "Cut-points:";
    for (int i = 0; i < (int)cutpoints.size(); i++) {
        cout << " " << cutpoints[i];
    }
    cout << endl << "Bridges:" << endl;
    for (int i = 0; i < (int)bridges.size(); i++) {
        cout << bridges[i].first << " " << bridges[i].second << endl;
    }
    cout << "Blocks, or Edge-Biconnected Components:" << endl;
    for (int i = 0; i < (int)block.size(); i++) {
        for (int j = 0; j < (int)block[i].size(); j++) {
            cout << block[i][j] << " ";
        }
        cout << endl;
    }
    cout << "Adjacency List for Bridge-Block Forest:" << endl;
    for (int i = 0; i < (int)block.size(); i++) {
        cout << i << " =>";
        for (int j = 0; j < (int)block_forest[i].size(); j++) {
            cout << " " << block_forest[i][j];
        }
        cout << endl;
    }
    return 0;
}

```

Example Output

```

Cut-points: 5 1
Bridges:
1 2
5 4
3 7
Blocks, a.k.a. Edge-Biconnected Components:
2
4
5 1 0
7
3
6
Adjacency List for Bridge-Block Forest:
0 => 2
1 => 2
2 => 0 1
3 => 4
4 => 3
5 =>

```

5.3.4 Bridge Finding (Online)

Maintain the number of bridges in an undirected graph while edges are inserted one at a time. The graph starts with isolated nodes. After each call to `add_edge()`, `bridges` stores the current number of bridges.

This data structure maintains two disjoint-set forests:

- `dsu_2ecc[]` tracks the current 2-edge-connected components.
- `dsu_cc[]` tracks the ordinary connected components.

When an inserted edge joins two different connected components, it creates a new bridge. When it joins two nodes already in the same connected component, it creates a cycle and every bridge on the path between the two endpoints stops being a bridge.

Time Complexity: $O(\log n)$ amortized per call to `add_edge()` in this implementation, where n is the number of nodes.

Space Complexity: $O(n)$ for the disjoint-set arrays and auxiliary parent links.

```
#include <algorithm>
#include <vector>

struct online_bridges {
    std::vector<int> dsu_2ecc, dsu_cc, dsu_cc_size, parent, last_visit;
    int lca_iteration, bridges;

    online_bridges(int nodes = 0) { init(nodes); }

    void init(int nodes) {
        dsu_2ecc.resize(nodes);
        dsu_cc.resize(nodes);
        dsu_cc_size.assign(nodes, 1);
        parent.assign(nodes, -1);
        last_visit.assign(nodes, 0);
        lca_iteration = 0;
        bridges = 0;
        for (int i = 0; i < nodes; i++) {
            dsu_2ecc[i] = dsu_cc[i] = i;
        }
    }

    int find_2ecc(int v) {
        if (v == -1) {
            return -1;
        }
        if (dsu_2ecc[v] == v) {
            return v;
        }
        return dsu_2ecc[v] = find_2ecc(dsu_2ecc[v]);
    }

    int find_cc(int v) {
        v = find_2ecc(v);
        if (dsu_cc[v] == v) {
            return v;
        }
        return dsu_cc[v] = find_cc(dsu_cc[v]);
    }

    void make_root(int v) {
        v = find_2ecc(v);
        int root = v, child = -1;
        while (v != -1) {
            int p = find_2ecc(parent[v]);
            parent[v] = child;
            dsu_cc[v] = root;
            child = v;
        }
    }
};
```

```

    v = p;
}
dsu_cc_size[root] = dsu_cc_size[child];
}

void merge_path(int a, int b) {
    lca_iteration++;
    std::vector<int> path_a, path_b;
    int lca = -1;
    while (lca == -1) {
        if (a != -1) {
            a = find_2ecc(a);
            path_a.push_back(a);
            if (last_visit[a] == lca_iteration) {
                lca = a;
                break;
            }
            last_visit[a] = lca_iteration;
            a = parent[a];
        }
        if (b != -1) {
            b = find_2ecc(b);
            path_b.push_back(b);
            if (last_visit[b] == lca_iteration) {
                lca = b;
                break;
            }
            last_visit[b] = lca_iteration;
            b = parent[b];
        }
    }
    for (int i = 0; i < (int)path_a.size(); i++) {
        dsu_2ecc[path_a[i]] = lca;
        if (path_a[i] == lca) {
            break;
        }
    }
    bridges--;
}
for (int i = 0; i < (int)path_b.size(); i++) {
    dsu_2ecc[path_b[i]] = lca;
    if (path_b[i] == lca) {
        break;
    }
}
    bridges--;
}
}

void add_edge(int a, int b) {
    a = find_2ecc(a);
    b = find_2ecc(b);
    if (a == b) {
        return;
    }
    int ca = find_cc(a), cb = find_cc(b);
    if (ca != cb) {
        bridges++;
        if (dsu_cc_size[ca] > dsu_cc_size[cb]) {
            std::swap(a, b);
            std::swap(ca, cb);
        }
        make_root(a);
        parent[a] = b;
        dsu_cc[a] = b;
        dsu_cc_size[cb] += dsu_cc_size[a];
    } else {
        merge_path(a, b);
    }
}
}

```



```
};
```

Example Usage

```
#include <cassert>

int main() {
    online_bridges graph(4);
    graph.add_edge(0, 1);
    assert(graph.bridges == 1);
    graph.add_edge(1, 2);
    assert(graph.bridges == 2);
    graph.add_edge(2, 0);
    assert(graph.bridges == 0);
    graph.add_edge(2, 3);
    assert(graph.bridges == 1);
    return 0;
}
```

5.3.5 2-SAT

Solve a Boolean formula in 2-CNF, where each clause contains at most two literals. A clause like $(a \vee b)$ is represented by the implications $\neg a \rightarrow b$ and $\neg b \rightarrow a$. The formula is satisfiable if and only if no variable and its negation belong to the same strongly connected component of the implication graph.

Variables are numbered from 0 to $n - 1$. A literal is represented by `literal(variable, value)`, where `value == true` means the variable itself and `value == false` means its negation.

- `add_or(a, b)` adds the clause $(a \vee b)$.
- `add_true(a)` forces literal `a` to be true.
- `satisfiable()` returns whether all added clauses can be satisfied and stores one valid assignment in `answer[]`.

Time Complexity: $O(\max(n, m))$ per call to `satisfiable()`, where n is the number of variables and m is the number of clauses.

Space Complexity: $O(\max(n, m))$ for the implication graph and DFS stacks.

```
#include <algorithm>
#include <vector>

struct two_sat {
    int variables;
    std::vector<std::vector<int>> graph, reverse_graph;
    std::vector<int> order, component, answer;

    two_sat(int n = 0) { init(n); }

    void init(int n) {
        variables = n;
        graph.assign(2 * n, std::vector<int>());
        reverse_graph.assign(2 * n, std::vector<int>());
        answer.assign(n, false);
    }

    int literal(int variable, bool value) { return 2 * variable + (value ? 0 : 1); }
    int neg(int x) { return x ^ 1; }

    void add_implication(int a, int b) {
        graph[a].push_back(b);
        reverse_graph[b].push_back(a);
    }
}
```

```

void add_or(int a, int b) {
    add_implication(neg(a), b);
    add_implication(neg(b), a);
}

void add_true(int a) { add_implication(neg(a), a); }
void add_false(int a) { add_implication(a, neg(a)); }

void dfs_order(int u, std::vector<bool> &visited) {
    visited[u] = true;
    for (int i = 0; i < (int)graph[u].size(); i++) {
        int v = graph[u][i];
        if (!visited[v]) {
            dfs_order(v, visited);
        }
    }
    order.push_back(u);
}

void dfs_component(int u, int id) {
    component[u] = id;
    for (int i = 0; i < (int)reverse_graph[u].size(); i++) {
        int v = reverse_graph[u][i];
        if (component[v] == -1) {
            dfs_component(v, id);
        }
    }
}

bool satisfiable() {
    order.clear();
    component.assign(2 * variables, -1);
    std::vector<bool> visited(2 * variables, false);
    for (int i = 0; i < 2 * variables; i++) {
        if (!visited[i]) {
            dfs_order(i, visited);
        }
    }
    std::reverse(order.begin(), order.end());
    int id = 0;
    for (int i = 0; i < (int)order.size(); i++) {
        if (component[order[i]] == -1) {
            dfs_component(order[i], id++);
        }
    }
    for (int i = 0; i < variables; i++) {
        if (component[2 * i] == component[2 * i + 1]) {
            return false;
        }
        answer[i] = component[2 * i] > component[2 * i + 1];
    }
    return true;
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    two_sat solver(3);
    int x0 = solver.literal(0, true);
    int not_x0 = solver.literal(0, false);
    int x1 = solver.literal(1, true);
    int x2 = solver.literal(2, true);
}

```

```

solver.add_or(x0, x1);
solver.add_or(not_x0, x2);
solver.add_true(x0);
assert(solver.satisfiable());
assert(solver.answer[0]);
return 0;
}

```

5.4 Spanning Trees

5.4.1 Minimum Spanning Tree (Prim)

Given a connected, undirected, weighted graph with possibly negative weights, its minimum spanning tree is a subgraph which is a tree that connects all nodes with a subset of its edges such that their total weight is minimized. `prim()` applies to a global, pre-populated adjacency list `adj[]` which must only consist of nodes numbered with integers between 0 (inclusive) and the total number of nodes (exclusive), as passed in the function argument. If the input graph is not connected, then this implementation will find the minimum spanning forest.

Since `std::priority_queue` is by default a max-heap, we simulate a min-heap by negating node distances before pushing them and negating them again after popping them. To modify this implementation to find the maximum spanning tree, the two negation steps can be skipped to prioritize the max edges.

Time Complexity: $O(m \log n)$ per call to `prim()`, where m is the number of edges and n is the number of nodes.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(m)$ auxiliary heap space for `prim()`.

```

#include <queue>
#include <utility>
#include <vector>

const int MAXN = 100;
std::vector<std::pair<int, int>> adj[MAXN], mst;

int prim(int nodes) {
    mst.clear();
    std::vector<bool> visit(nodes);
    int total_dist = 0;
    for (int i = 0; i < nodes; i++) {
        if (visit[i]) {
            continue;
        }
        visit[i] = true;
        std::priority_queue<std::pair<int, std::pair<int, int>>> pq;
        for (int j = 0; j < (int)adj[i].size(); j++) {
            pq.push(std::make_pair(-adj[i][j].second, std::make_pair(i, adj[i][j].first)));
        }
        while (!pq.empty()) {
            int u = pq.top().second.first;
            int v = pq.top().second.second;
            int w = -pq.top().first;
            pq.pop();
            if (visit[u] && !visit[v]) {
                visit[v] = true;
                if (v != i) {
                    mst.push_back(std::make_pair(u, v));
                    total_dist += w;
                }
                for (int j = 0; j < (int)adj[v].size(); j++) {

```

```

        pq.push(std::make_pair(-adj[v][j].second, std::make_pair(v, adj[v][j].first)));
    }
}
}
}
return total_dist;
}

```

Example Usage

```

#include <iostream>
using namespace std;

void add_edge(int u, int v, int w) {
    adj[u].push_back(make_pair(v, w));
    adj[v].push_back(make_pair(u, w));
}

int main() {
    add_edge(0, 1, 4);
    add_edge(1, 2, 6);
    add_edge(2, 0, 3);
    add_edge(3, 4, 1);
    add_edge(4, 5, 2);
    add_edge(5, 6, 3);
    add_edge(6, 4, 4);
    cout << "Total distance: " << prim(7) << endl;
    for (int i = 0; i < (int)mst.size(); i++) {
        cout << mst[i].first << " <-> " << mst[i].second << endl;
    }
    return 0;
}

```

Example Output

```

Total distance: 13
0 <-> 2
0 <-> 1
3 <-> 4
4 <-> 5
5 <-> 6

```

5.4.2 Minimum Spanning Tree (Kruskal)

Given a connected, undirected, weighted graph with possibly negative weights, its minimum spanning tree is a subgraph which is a tree that connects all nodes with a subset of its edges such that their total weight is minimized. `kruskal()` applies to a global, pre-populated edge list `edges`. If the input graph is not connected, then this implementation will find the minimum spanning forest.

Time Complexity: $O(m \log n)$ per call to `kruskal()`, where m is the number of edges and n is the number of nodes.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges
- $O(n)$ auxiliary stack space for `kruskal()`.

```

#include <algorithm>
#include <utility>
#include <vector>

const int MAXN = 100;

```

```

std::vector<std::pair<int, std::pair<int, int>>> edges;
int root[MAXN];
std::vector<std::pair<int, int>> mst;

int find_root(int x) {
    if (root[x] != x) {
        root[x] = find_root(root[x]);
    }
    return root[x];
}

int kruskal(int nodes) {
    mst.clear();
    std::sort(edges.begin(), edges.end());
    int total_dist = 0;
    for (int i = 0; i < nodes; i++) {
        root[i] = i;
    }
    for (int i = 0; i < (int)edges.size(); i++) {
        int u = find_root(edges[i].second.first);
        int v = find_root(edges[i].second.second);
        if (u != v) {
            root[u] = root[v];
            mst.push_back(edges[i].second);
            total_dist += edges[i].first;
        }
    }
    return total_dist;
}

```

Example Usage

```

#include <iostream>
using namespace std;

void add_edge(int u, int v, int w) {
    edges.push_back(make_pair(w, make_pair(u, v)));
}

int main() {
    add_edge(0, 1, 4);
    add_edge(1, 2, 6);
    add_edge(2, 0, 3);
    add_edge(3, 4, 1);
    add_edge(4, 5, 2);
    add_edge(5, 6, 3);
    add_edge(6, 4, 4);
    cout << "Total distance: " << kruskal(7) << endl;
    for (int i = 0; i < (int)mst.size(); i++) {
        cout << mst[i].first << " <-> " << mst[i].second << endl;
    }
    return 0;
}

```

Example Output

```

Total distance: 13
3 <-> 4
4 <-> 5
2 <-> 0
5 <-> 6
0 <-> 1

```

5.5 Flow

5.5.1 Maximum Flow (Ford-Fulkerson)

Given a flow network with integer capacities, find the maximum flow from a given source node to a given sink node. The flow along each edge may not exceed its capacity, and flow is conserved at every node other than the source and sink. `ford_fulkerson()` applies to global variables `nodes`, `source`, `sink`, and `cap[][]` which is an adjacency matrix that will be modified by the function call.

The Ford-Fulkerson algorithm is only optimal on graphs with integer capacities, as there exists certain real-valued flow inputs for which the algorithm never terminates. The Edmonds-Karp algorithm is an improvement using breadth-first search, addressing this problem.

Time Complexity: $O(n^2 \cdot f)$ per call to `ford_fulkerson()`, where n is the number of nodes and f is the maximum flow.

Space Complexity:

- $O(n^2)$ for storage of the flow network, where n is the number of nodes.
- $O(n)$ auxiliary stack space for `ford_fulkerson()`.

```
#include <algorithm>
#include <vector>

const int MAXN = 100, INF = 0x3f3f3f3f;
int nodes, source, sink, cap[MAXN][MAXN];
std::vector<bool> visit(MAXN);

int dfs(int u, int f) {
    if (u == sink) {
        return f;
    }
    visit[u] = true;
    for (int v = 0; v < nodes; v++) {
        if (!visit[v] && cap[u][v] > 0) {
            int flow = dfs(v, std::min(f, cap[u][v]));
            if (flow > 0) {
                cap[u][v] -= flow;
                cap[v][u] += flow;
                return flow;
            }
        }
    }
    return 0;
}

int ford_fulkerson() {
    int max_flow = 0;
    for (;;) {
        std::fill(visit.begin(), visit.end(), false);
        int flow = dfs(source, INF);
        if (flow == 0) {
            break;
        }
        max_flow += flow;
    }
    return max_flow;
}
```

Example Usage

```
#include <cassert>
```

```

int main() {
    nodes = 6;
    source = 0;
    sink = 5;
    cap[0][1] = 3;
    cap[0][2] = 3;
    cap[1][2] = 2;
    cap[1][3] = 3;
    cap[2][4] = 2;
    cap[3][4] = 1;
    cap[3][5] = 2;
    cap[4][5] = 3;
    assert(ford_ulkerson() == 5);
    return 0;
}

```

5.5.2 Maximum Flow (Edmonds-Karp)

Given a flow network with integer capacities, find the maximum flow from a given source node to a given sink node. The flow along each edge may not exceed its capacity, and flow is conserved at every node other than the source and sink. `edmonds_karp()` applies to a global adjacency list `adj[]` that will be modified by the function call.

The Edmonds-Karp algorithm will also support real-valued flow capacities. As such, this implementation will work as intended upon changing the appropriate variables to doubles.

Time Complexity: $O(\min(n \cdot m^2, m \cdot f))$ per call to `edmonds_karp()`, where n is the number of nodes, m is the number of edges, and f is the maximum flow.

Space Complexity: $O(\max(n, m))$ for storage of the flow network, where n is the number of nodes and m is the number of edges.

```

#include <algorithm>
#include <queue>
#include <vector>

struct edge {
    int u, v, rev, cap, f;
};

const int MAXN = 100, INF = 0x3f3f3f3f;
std::vector<edge> adj[MAXN];

void add_edge(int u, int v, int cap) {
    adj[u].push_back((edge){u, v, (int)adj[v].size(), cap, 0});
    adj[v].push_back((edge){v, u, (int)adj[u].size() - 1, 0, 0});
}

int edmonds_karp(int nodes, int source, int sink) {
    int max_flow = 0;
    for (;;) {
        std::vector<edge *> pred(nodes, (edge *)0);
        std::queue<int> q;
        q.push(source);
        while (!q.empty() && !pred[sink]) {
            int u = q.front();
            q.pop();
            for (int j = 0; j < (int)adj[u].size(); j++) {
                edge &e = adj[u][j];
                if (!pred[e.v] && e.cap > e.f) {
                    pred[e.v] = &e;
                    q.push(e.v);
                }
            }
        }
    }
}

```

```

    if (!pred[sink]) {
        break;
    }
    int flow = INF;
    for (int u = sink; u != source; u = pred[u]->u) {
        flow = std::min(flow, pred[u]->cap - pred[u]->f);
    }
    for (int u = sink; u != source; u = pred[u]->u) {
        pred[u]->f += flow;
        adj[pred[u]->v][pred[u]->rev].f -= flow;
    }
    max_flow += flow;
}
return max_flow;
}

```

Example Usage

```

#include <cassert>

int main() {
    add_edge(0, 1, 3);
    add_edge(0, 2, 3);
    add_edge(1, 2, 2);
    add_edge(1, 3, 3);
    add_edge(2, 4, 2);
    add_edge(3, 4, 1);
    add_edge(3, 5, 2);
    add_edge(4, 5, 3);
    assert(edmonds_karp(6, 0, 5) == 5);
    return 0;
}

```

5.5.3 Maximum Flow (Dinic)

Given a flow network with integer capacities, find the maximum flow from a given source node to a given sink node. The flow along each edge may not exceed its capacity, and flow is conserved at every node other than the source and sink. `dinic()` applies to a global adjacency list `adj[]` that will be modified by the function call.

Dinic's algorithm will also support real-valued flow capacities. As such, this implementation will work as intended upon changing the appropriate variables to doubles.

Time Complexity: $O(n^2 \cdot m)$ per call to `dinic()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the flow network, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary stack and heap space for `dinic()`.

```

#include <algorithm>
#include <queue>
#include <vector>

struct edge {
    int v, rev, cap, f;
};

const int MAXN = 100, INF = 0x3f3f3f3f;
std::vector<edge> adj[MAXN];
int dist[MAXN], ptr[MAXN];

void add_edge(int u, int v, int cap) {

```



```

    adj[u].push_back((edge){v, (int)adj[v].size(), cap, 0});
    adj[v].push_back((edge){u, (int)adj[u].size() - 1, 0, 0});
}

bool dinic_bfs(int nodes, int source, int sink) {
    std::fill(dist, dist + nodes, -1);
    dist[source] = 0;
    std::queue<int> q;
    q.push(source);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int j = 0; j < (int)adj[u].size(); j++) {
            edge &e = adj[u][j];
            if (dist[e.v] < 0 && e.f < e.cap) {
                dist[e.v] = dist[u] + 1;
                q.push(e.v);
            }
        }
    }
    return dist[sink] >= 0;
}

int dinic_dfs(int u, int f, int sink) {
    if (u == sink) {
        return f;
    }
    for (; ptr[u] < (int)adj[u].size(); ptr[u]++) {
        edge &e = adj[u][ptr[u]];
        if (dist[e.v] == dist[u] + 1 && e.f < e.cap) {
            int flow = dinic_dfs(e.v, std::min(f, e.cap - e.f), sink);
            if (flow > 0) {
                e.f -= flow;
                adj[e.v][e.rev].f += flow;
                return flow;
            }
        }
    }
    return 0;
}

int dinic(int nodes, int source, int sink) {
    int flow, max_flow = 0;
    while (dinic_bfs(nodes, source, sink)) {
        std::fill(ptr, ptr + nodes, 0);
        while ((flow = dinic_dfs(source, INF, sink)) != 0) {
            max_flow += flow;
        }
    }
    return max_flow;
}

```

Example Usage

```

#include <cassert>

int main() {
    add_edge(0, 1, 3);
    add_edge(0, 2, 3);
    add_edge(1, 2, 2);
    add_edge(1, 3, 3);
    add_edge(2, 4, 2);
    add_edge(3, 4, 1);
    add_edge(3, 5, 2);
    add_edge(4, 5, 3);
    assert(dinic(6, 0, 5) == 5);
}

```

```

    return 0;
}

```

5.5.4 Maximum Flow (Push-Relabel)

Given a flow network with integer capacities, find the maximum flow from a given source node to a given sink node. The flow along each edge may not exceed its capacity, and flow is conserved at every node other than the source and sink. `push_relabel()` applies to a global adjacency matrix `cap[][]` and returns the maximum flow.

Although the push-relabel algorithm is considered one of the most efficient maximum flow algorithms, it cannot take advantage of the magnitude of the maximum flow being less than n^3 (in which case the Ford-Fulkerson or Edmonds-Karp algorithms may be more efficient).

Time Complexity: $O(n^3)$ per call to `push_relabel()`, where n is the number of nodes.

Space Complexity:

- $O(n^2)$ for storage of the flow network, where n is the number of nodes.
- $O(n)$ auxiliary heap space for `push_relabel()`.

```

#include <algorithm>
#include <vector>

const int MAXN = 100, INF = 0x3f3f3f3f;
int cap[MAXN][MAXN], f[MAXN][MAXN];

int push_relabel(int nodes, int source, int sink) {
    std::vector<int> e(nodes, 0), h(nodes, 0), maxh(nodes, 0);
    for (int i = 0; i < nodes; i++) {
        std::fill(f[i], f[i] + nodes, 0);
    }
    h[source] = nodes - 1;
    for (int i = 0; i < nodes; i++) {
        f[source][i] = cap[source][i];
        f[i][source] = -f[source][i];
        e[i] = cap[source][i];
    }
    int size = 0;
    for (;;) {
        if (size == 0) {
            for (int i = 0; i < nodes; i++) {
                if (i != source && i != sink && e[i] > 0) {
                    if (size != 0 && h[i] > h[maxh[0]]) {
                        size = 0;
                    }
                    maxh[size++] = i;
                }
            }
        }
        if (size == 0) {
            break;
        }
        while (size != 0) {
            int i = maxh[size - 1];
            bool pushed = false;
            for (int j = 0; j < nodes && e[i] != 0; j++) {
                if (h[i] == h[j] + 1 && cap[i][j] - f[i][j] > 0) {
                    int df = std::min(cap[i][j] - f[i][j], e[i]);
                    f[i][j] += df;
                    f[j][i] -= df;
                    e[i] -= df;
                    e[j] += df;
                    if (e[i] == 0) {
                        size--;
                    }
                }
            }
        }
    }
}

```

```

        }
        pushed = true;
    }
}
if (pushed) {
    continue;
}
h[i] = INF;
for (int j = 0; j < nodes; j++) {
    if (h[i] > h[j] + 1 && cap[i][j] - f[i][j] > 0) {
        h[i] = h[j] + 1;
    }
}
if (h[i] > h[maxh[0]]) {
    size = 0;
    break;
}
}
}
int max_flow = 0;
for (int i = 0; i < nodes; i++) {
    max_flow += f[source][i];
}
return max_flow;
}

```

Example Usage

```

#include <cassert>

int main() {
    cap[0][1] = 3;
    cap[0][2] = 3;
    cap[1][2] = 2;
    cap[1][3] = 3;
    cap[2][4] = 2;
    cap[3][4] = 1;
    cap[3][5] = 2;
    cap[4][5] = 3;
    assert(push_relabel(6, 0, 5) == 5);
    return 0;
}

```

5.5.5 Min-Cost Max-Flow

Given a flow network with integer capacities and edge costs, find a minimum-cost flow from a source node to a sink node. Calling `min_cost_flow(source, sink, target_flow)` sends up to `target_flow` units of flow and returns `(flow, cost)`. If the returned flow is smaller than `target_flow`, the network cannot carry the requested amount.

This implementation uses shortest augmenting paths with SPFA, so it supports negative edge costs as long as the residual graph has no reachable negative-cost cycle. For dense or adversarial inputs, a potential-based Dijkstra version is usually faster when reduced costs are nonnegative.

Time Complexity: $O(F \cdot n \cdot m)$ per call to `min_cost_flow()`, where F is the amount of flow sent, n is the number of nodes, and m is the number of edges.

Space Complexity: $O(\max(n, m))$ for storage of the residual network and auxiliary arrays.

```

#include <algorithm>
#include <queue>
#include <utility>
#include <vector>

```

```

struct edge {
    int v, rev, cap, cost;
    edge(int v_, int rev_, int cap_, int cost_) : v(v_), rev(rev_), cap(cap_), cost(cost_) {}
};

const int MCMF_INF = 0x3f3f3f3f;

struct min_cost_max_flow {
    std::vector<std::vector<edge>> adj;

    min_cost_max_flow(int nodes = 0) { init(nodes); }

    void init(int nodes) { adj.assign(nodes, std::vector<edge>()); }

    void add_edge(int u, int v, int cap, int cost) {
        adj[u].push_back(edge(v, (int)adj[v].size(), cap, cost));
        adj[v].push_back(edge(u, (int)adj[u].size() - 1, 0, -cost));
    }

    std::pair<int, int> min_cost_flow(int source, int sink, int target_flow) {
        int nodes = adj.size(), flow = 0, cost = 0;
        std::vector<int> dist(nodes), parent_node(nodes), parent_edge(nodes);
        std::vector<bool> in_queue(nodes);
        while (flow < target_flow) {
            std::fill(dist.begin(), dist.end(), MCMF_INF);
            std::fill(in_queue.begin(), in_queue.end(), false);
            std::queue<int> q;
            dist[source] = 0;
            q.push(source);
            in_queue[source] = true;
            while (!q.empty()) {
                int u = q.front();
                q.pop();
                in_queue[u] = false;
                for (int i = 0; i < (int)adj[u].size(); i++) {
                    edge &e = adj[u][i];
                    if (e.cap > 0 && dist[e.v] > dist[u] + e.cost) {
                        dist[e.v] = dist[u] + e.cost;
                        parent_node[e.v] = u;
                        parent_edge[e.v] = i;
                        if (!in_queue[e.v]) {
                            q.push(e.v);
                            in_queue[e.v] = true;
                        }
                    }
                }
            }
            if (dist[sink] == MCMF_INF) {
                break;
            }
            int aug = target_flow - flow;
            for (int v = sink; v != source; v = parent_node[v]) {
                aug = std::min(aug, adj[parent_node[v]][parent_edge[v]].cap);
            }
            for (int v = sink; v != source; v = parent_node[v]) {
                edge &e = adj[parent_node[v]][parent_edge[v]];
                e.cap -= aug;
                adj[v][e.rev].cap += aug;
                cost += aug * e.cost;
            }
            flow += aug;
        }
        return std::make_pair(flow, cost);
    }
};

```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    min_cost_max_flow graph(4);
    graph.add_edge(0, 1, 2, 1);
    graph.add_edge(0, 2, 1, 5);
    graph.add_edge(1, 2, 1, 1);
    graph.add_edge(1, 3, 1, 3);
    graph.add_edge(2, 3, 2, 1);
    pair<int, int> result = graph.min_cost_flow(0, 3, 3);
    assert(result.first == 3);
    assert(result.second == 13);
    return 0;
}
```

5.6 Maximum Matching

5.6.1 Maximum Bipartite Matching (Kuhn)

Given two sets of nodes $A = \{0, 1, \dots, n_1 - 1\}$ and $B = \{0, 1, \dots, n_2 - 1\}$ such that $n_1 < n_2$, as well as a set of edges E mapping nodes from set A to set B , find the largest possible subset of E containing no edges that share the same node. `kuhn()` applies to a global, pre-populated adjacency list `adj[]` which must only consist of nodes numbered with integers between 0 (inclusive) and the total number of nodes (exclusive), as passed in the function argument.

Time Complexity: $O(m \cdot (n_1 + n_2))$ per call to `kuhn()`, where m is the number of edges.

Space Complexity: $O(n_1 + n_2)$ auxiliary stack space for `kuhn()`.

```
#include <algorithm>
#include <vector>

const int MAXN = 100;
int match[MAXN];
std::vector<bool> visit(MAXN);
std::vector<int> adj[MAXN];

bool dfs(int u) {
    visit[u] = true;
    for (int j = 0; j < (int)adj[u].size(); j++) {
        int v = match[adj[u][j]];
        if (v == -1 || (!visit[v] && dfs(v))) {
            match[adj[u][j]] = u;
            return true;
        }
    }
    return false;
}

int kuhn(int n1, int n2) {
    std::fill(visit.begin(), visit.end(), false);
    std::fill(match, match + n2, -1);
    int matches = 0;
    for (int i = 0; i < n1; i++) {
        std::fill(visit.begin(), visit.begin() + n1, false);
        if (dfs(i)) {
            matches++;
        }
    }
}
```

```

    }
    return matches;
}

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    int n1 = 3, n2 = 4;
    adj[0].push_back(1);
    adj[1].push_back(0);
    adj[1].push_back(1);
    adj[1].push_back(2);
    adj[2].push_back(2);
    adj[2].push_back(3);
    cout << "Matched " << kuhn(n1, n2) << " pair(s):" << endl;
    for (int i = 0; i < n2; i++) {
        if (match[i] != -1) {
            cout << match[i] << " " << i << endl;
        }
    }
    return 0;
}

```

Example Output

```

Matched 3 pair(s):
1 0
0 1
2 2

```

5.6.2 Maximum Bipartite Matching (Hopcroft-Karp)

Given two sets of nodes $A = \{0, 1, \dots, n_1 - 1\}$ and $B = \{0, 1, \dots, n_2 - 1\}$ such that $n_1 < n_2$, as well as a set of edges E mapping nodes from set A to set B , find the largest possible subset of E containing no edges that share the same node. `hopcroft_karp()` applies to a global, pre-populated adjacency list `adj[]` which must only consist of nodes numbered with integers between 0 (inclusive) and the total number of nodes (exclusive), as passed in the function argument.

Time Complexity: $O(m \cdot \sqrt{n_1 + n_2})$ per call to `hopcroft_karp()`, where m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n_1 + n_2)$ auxiliary stack and heap space for `hopcroft_karp()`.

```

#include <algorithm>
#include <queue>
#include <vector>

const int MAXN = 100;
std::vector<int> adj[MAXN];
std::vector<bool> used(MAXN), visit(MAXN);
int match[MAXN], dist[MAXN];

void bfs(int n1, int n2) {
    std::fill(dist, dist + n1, -1);
    std::queue<int> q;
    for (int u = 0; u < n1; u++) {
        if (!used[u]) {

```

```

        q.push(u);
        dist[u] = 0;
    }
}
while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int j = 0; j < (int)adj[u].size(); j++) {
        int v = match[adj[u][j]];
        if (v >= 0 && dist[v] < 0) {
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
}
}

bool dfs(int u) {
    visit[u] = true;
    for (int j = 0; j < (int)adj[u].size(); j++) {
        int v = match[adj[u][j]];
        if (v < 0 || (!visit[v] && dist[v] == dist[u] + 1 && dfs(v))) {
            match[adj[u][j]] = u;
            used[u] = true;
            return true;
        }
    }
    return false;
}

int hopcroft_karp(int n1, int n2) {
    std::fill(match, match + n2, -1);
    std::fill(used.begin(), used.end(), false);
    int res = 0;
    for (;;) {
        bfs(n1, n2);
        std::fill(visit.begin(), visit.end(), false);
        int f = 0;
        for (int u = 0; u < n1; u++) {
            if (!used[u] && dfs(u)) {
                f++;
            }
        }
        if (f == 0) {
            return res;
        }
        res += f;
    }
    return res;
}

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    int n1 = 3, n2 = 4;
    adj[0].push_back(1);
    adj[1].push_back(0);
    adj[1].push_back(1);
    adj[1].push_back(2);
    adj[2].push_back(2);
    adj[2].push_back(3);
    cout << "Matched " << hopcroft_karp(n1, n2) << " pair(s):" << endl;
    for (int i = 0; i < n2; i++) {

```

```

    if (match[i] != -1) {
        cout << match[i] << " " << i << endl;
    }
}
return 0;
}

```

Example Output

```

Matched 3 pair(s):
1 0
0 1
2 2

```

5.6.3 Maximum Graph Matching (Edmonds)

Given a directed graph, determine a maximum subset of its edges such that no node is shared between different edges in the resulting subset. `edmonds()` applies to a global, pre-populated adjacency list `adj[]` which must only consist of nodes numbered with integers between 0 (inclusive) and the total number of nodes (exclusive), as passed in the function argument.

Time Complexity: $O(n^3)$ per call to `edmonds()`, where n is the number of nodes.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary heap space for `edmonds()`, where n is the number of nodes.

```

#include <queue>
#include <vector>

const int MAXN = 100;
std::vector<int> adj[MAXN];
int p[MAXN], base[MAXN], match[MAXN];

int lca(int nodes, int u, int v) {
    std::vector<bool> used(nodes);
    for (;;) {
        u = base[u];
        used[u] = true;
        if (match[u] == -1) {
            break;
        }
        u = p[match[u]];
    }
    for (;;) {
        v = base[v];
        if (used[v]) {
            return v;
        }
        v = p[match[v]];
    }
}

void mark_path(std::vector<bool> &blossom, int u, int b, int child) {
    for (; base[u] != b; u = p[match[u]]) {
        blossom[base[u]] = true;
        blossom[base[match[u]]] = true;
        p[u] = child;
        child = match[u];
    }
}

int find_path(int nodes, int root) {

```



```

std::vector<bool> used(nodes);
for (int i = 0; i < nodes; ++i) {
    p[i] = -1;
    base[i] = i;
}
used[root] = true;
std::queue<int> q;
q.push(root);
while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int j = 0; j < (int)adj[u].size(); j++) {
        int v = adj[u][j];
        if (base[u] == base[v] || match[u] == v) {
            continue;
        }
        if (v == root || (match[v] != -1 && p[match[v]] != -1)) {
            int curr_base = lca(nodes, u, v);
            std::vector<bool> blossom(nodes);
            mark_path(blossom, u, curr_base, v);
            mark_path(blossom, v, curr_base, u);
            for (int i = 0; i < nodes; i++) {
                if (blossom[base[i]]) {
                    base[i] = curr_base;
                    if (!used[i]) {
                        used[i] = true;
                        q.push(i);
                    }
                }
            }
        }
        else if (p[v] == -1) {
            p[v] = u;
            if (match[v] == -1) {
                return v;
            }
            v = match[v];
            used[v] = true;
            q.push(v);
        }
    }
}
return -1;
}

int edmonds(int nodes) {
    for (int i = 0; i < nodes; i++) {
        match[i] = -1;
    }
    for (int i = 0; i < nodes; i++) {
        if (match[i] == -1) {
            int u, pu, ppu;
            for (u = find_path(nodes, i); u != -1; u = ppu) {
                pu = p[u];
                ppu = match[pu];
                match[u] = pu;
                match[pu] = u;
            }
        }
    }
    int matches = 0;
    for (int i = 0; i < nodes; i++) {
        if (match[i] != -1) {
            matches++;
        }
    }
    return matches / 2;
}

```

Example Usage

```
#include <iostream>
using namespace std;

int main() {
    int nodes = 4;
    adj[0].push_back(1);
    adj[1].push_back(0);
    adj[1].push_back(2);
    adj[2].push_back(1);
    adj[2].push_back(3);
    adj[3].push_back(2);
    adj[3].push_back(0);
    adj[0].push_back(3);
    cout << "Matched " << edmonds(nodes) << " pair(s):" << endl;
    for (int i = 0; i < nodes; i++) {
        if (match[i] != -1 && i < match[i]) {
            cout << i << " " << match[i] << endl;
        }
    }
    return 0;
}
```

Example Output

```
Matched 2 pair(s):
0 1
2 3
```

5.7 Hard Problems

5.7.1 Maximum Clique (Bron-Kerbosch)

Given an undirected graph, `max_clique()` returns the size of the maximum clique, that is, the largest subset of nodes such that all pairs of nodes in the subset are connected by an edge. `max_clique_weighted()` additionally uses a global array `w[]` specifying a weight value for each node, returning the clique in the graph that has maximum total weight.

Both functions apply to a global, pre-populated adjacency matrix `adj[][]` which must satisfy the condition that `adj[u][v]` is `true` if and only if `adj[v][u]` is `true`, for all pairs of nodes u and v respectively between 0 (inclusive) and the total number of nodes (exclusive) as passed in the function argument. Note that `max_clique_weighted()` is an efficient implementation using bitmasks of unsigned 64-bit integers, thus requiring the number of nodes to be less than 64.

Time Complexity: $O(3^{n/3})$ per call to `max_clique()` and `max_clique_weighted()`, where n is the number of nodes.

Space Complexity:

- $O(n^2)$ for storage of the graph, where n is the number of nodes.
- $O(n)$ auxiliary stack space for `max_clique()` and `max_clique_weighted()`.

```
#include <algorithm>
#include <bitset>
#include <vector>

const int MAXN = 35;
typedef std::bitset<MAXN> bits;
typedef unsigned long long uint64;

bool adj[MAXN][MAXN];
```

```

int w[MAXN];

int rec(int nodes, bits &curr, bits &pool, bits &excl) {
    if (pool.none() && excl.none()) {
        return curr.count();
    }
    int ans = 0, u = 0;
    for (int v = 0; v < nodes; v++) {
        if (pool[v] || excl[v]) {
            u = v;
        }
    }
    for (int v = 0; v < nodes; v++) {
        if (!pool[v] || adj[u][v]) {
            continue;
        }
        bits ncurr, npool, nexcl;
        for (int i = 0; i < nodes; i++) {
            ncurr[i] = curr[i];
        }
        ncurr[v] = true;
        for (int j = 0; j < nodes; j++) {
            npool[j] = pool[j] && adj[v][j];
            nexcl[j] = excl[j] && adj[v][j];
        }
        ans = std::max(ans, rec(nodes, ncurr, npool, nexcl));
        pool[v] = false;
        excl[v] = true;
    }
    return ans;
}

int max_clique(int nodes) {
    bits curr, excl, pool;
    for (int i = 0; i < nodes; i++) {
        pool[i] = true;
    }
    return rec(nodes, curr, pool, excl);
}

int rec(const std::vector<uint64> &g, uint64 curr, uint64 pool, uint64 excl) {
    if (pool == 0 && excl == 0) {
        int res = 0;
        while (curr != 0) {
            int u = __builtin_ctzll(curr);
            res += w[u];
            curr &= curr - 1;
        }
        return res;
    }
    if (pool == 0) {
        return -1;
    }
    int res = -1, pivot = __builtin_ctzll(pool | excl);
    uint64 z = pool & ~g[pivot];
    while (z != 0) {
        int u = __builtin_ctzll(z);
        res = std::max(res, rec(g, curr | (1ULL << u), pool & g[u], excl & g[u]));
        pool ^= 1ULL << u;
        excl |= 1ULL << u;
        z &= z - 1;
    }
    return res;
}

int max_clique_weighted(int nodes) {
    std::vector<uint64> g(nodes, 0);
    for (int i = 0; i < nodes; i++) {

```

```

    for (int j = 0; j < nodes; j++) {
        if (adj[i][j]) {
            g[i] |= 1ULL << j;
        }
    }
}
return rec(g, 0, (1ULL << nodes) - 1, 0);
}

```

Example Usage

```

#include <cassert>

void add_edge(int u, int v) {
    adj[u][v] = true;
    adj[v][u] = true;
}

int main() {
    add_edge(0, 1);
    add_edge(0, 2);
    add_edge(0, 3);
    add_edge(1, 2);
    add_edge(1, 3);
    add_edge(2, 3);
    add_edge(3, 4);
    add_edge(4, 2);
    w[0] = 10;
    w[1] = 20;
    w[2] = 30;
    w[3] = 40;
    w[4] = 50;
    assert(max_clique(5) == 4);
    assert(max_clique_weighted(5) == 120);
    return 0;
}

```

5.7.2 Graph Coloring

Given an undirected graph, assign a color to every node such that no pair of adjacent nodes have the same color, and that the total number of colors used is minimized. `color_graph()` applies to a global, pre-populated adjacency matrix `adj[][]` which must satisfy the condition that `adj[u][v]` is `true` if and only if `adj[v][u]` is `true`, for all pairs of nodes u and v respectively between 0 (inclusive) and the total number of nodes (exclusive) as passed in the function argument.

Time Complexity: Exponential on the number of nodes per call to `color_graph()`.

Space Complexity:

- $O(n^2)$ for storage of the graph, where n is the number of nodes.
- $O(n)$ auxiliary stack and heap space for `color_graph()`.

```

#include <algorithm>
#include <vector>

const int MAXN = 30;
int adj[MAXN][MAXN], min_colors, color[MAXN];
int curr[MAXN], id[MAXN + 1], degree[MAXN + 1];

void rec(int lo, int hi, int n, int used_colors) {
    if (used_colors >= min_colors) {
        return;
    }
}

```

```

    }
    if (n == hi) {
        for (int i = lo; i < hi; i++) {
            color[id[i]] = curr[i];
        }
        min_colors = used_colors;
        return;
    }
    std::vector<bool> used(used_colors + 1);
    for (int i = 0; i < n; i++) {
        if (adj[id[n]][id[i]]) {
            used[curr[i]] = true;
        }
    }
    for (int i = 0; i <= used_colors; i++) {
        if (!used[i]) {
            int tmp = curr[n];
            curr[n] = i;
            rec(lo, hi, n + 1, std::max(used_colors, i + 1));
            curr[n] = tmp;
        }
    }
}

int color_graph(int nodes) {
    for (int i = 0; i <= nodes; i++) {
        id[i] = i;
        degree[i] = 0;
    }
    int res = 1, lo = 0;
    for (int hi = 1; hi <= nodes; hi++) {
        int best = hi;
        for (int i = hi; i < nodes; i++) {
            if (adj[id[hi - 1]][id[i]]) {
                degree[id[i]]++;
            }
            if (degree[id[best]] < degree[id[i]]) {
                best = i;
            }
        }
        std::swap(id[hi], id[best]);
        if (degree[id[hi]] == 0) {
            min_colors = nodes + 1;
            std::fill(curr, curr + nodes, 0);
            rec(lo, hi, lo, 0);
            lo = hi;
            res = std::max(res, min_colors);
        }
    }
    return res;
}

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

void add_edge(int u, int v) {
    adj[u][v] = true;
    adj[v][u] = true;
}

int main() {
    add_edge(0, 1);
    add_edge(0, 4);
}

```

```

add_edge(1, 3);
add_edge(1, 4);
add_edge(2, 3);
add_edge(2, 4);
add_edge(3, 4);
int colors = color_graph(5);
cout << "Colored using " << colors << " color(s):" << endl;
for (int i = 0; i < colors; i++) {
    cout << "Color " << i + 1 << ":";
    for (int j = 0; j < 5; j++) {
        if (color[j] == i) {
            cout << " " << j;
        }
    }
    cout << endl;
}
return 0;
}

```

Example Output

```

Colored using 3 color(s):
Color 1: 0 3
Color 2: 1 2
Color 3: 4

```

5.7.3 Shortest Hamiltonian Cycle (TSP)

Given a weighted graph, determine a cycle of minimum total distance which visits each node exactly once and returns to the starting node. This is known as the traveling salesman problem (TSP). Since this implementation uses bitmasks with 32-bit integers, the maximum number of nodes must be less than 32.

`shortest_hamiltonian_cycle()` applies to a global adjacency matrix `adj[][]`, which must be populated with `add_edge()` before the function call.

Time Complexity: $O(2^n \cdot n^2)$ per call to `shortest_hamiltonian_cycle()`, where n is the number of nodes.

Space Complexity:

- $O(n^2)$ for storage of the graph, where n is the number of nodes.
- $O(2^n \cdot n)$ auxiliary heap space for `shortest_hamiltonian_cycle()`.

```

#include <algorithm>

const int MAXN = 20, INF = 0x3f3f3f3f;
int adj[MAXN][MAXN], dp[1 << MAXN][MAXN], order[MAXN];

void add_edge(int u, int v, int w) {
    adj[u][v] = w;
    adj[v][u] = w; // Remove this line if the graph is directed.
}

int shortest_hamiltonian_cycle(int nodes) {
    int max_mask = (1 << nodes) - 1;
    for (int i = 0; i <= max_mask; i++) {
        std::fill(dp[i], dp[i] + nodes, INF);
    }
    dp[1][0] = 0;
    for (int mask = 1; mask <= max_mask; mask += 2) {
        for (int i = 1; i < nodes; i++) {
            if ((mask & 1 << i) != 0) {
                for (int j = 0; j < nodes; j++) {
                    if ((mask & 1 << j) != 0) {
                        dp[mask][i] = std::min(dp[mask][i], dp[mask ^ (1 << i)][j] + adj[j][i]);
                    }
                }
            }
        }
    }
}

```

```

    }
  }
}
int res = INF + INF;
for (int i = 1; i < nodes; i++) {
  res = std::min(res, dp[max_mask][i] + adj[i][0]);
}
int mask = max_mask, old = 0;
for (int i = nodes - 1; i >= 1; i--) {
  int bj = -1;
  for (int j = 1; j < nodes; j++) {
    if ((mask & 1 << j) != 0 &&
        (bj == -1 || dp[mask][bj] + adj[bj][old] > dp[mask][j] + adj[j][old])) {
      bj = j;
    }
  }
  order[i] = bj;
  mask ^= 1 << bj;
  old = bj;
}
return res;
}

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
  int nodes = 5;
  add_edge(0, 1, 1);
  add_edge(0, 2, 10);
  add_edge(0, 3, 1);
  add_edge(0, 4, 10);
  add_edge(1, 2, 10);
  add_edge(1, 3, 10);
  add_edge(1, 4, 1);
  add_edge(2, 3, 1);
  add_edge(2, 4, 1);
  add_edge(3, 4, 10);
  cout << "The shortest hamiltonian cycle has length " << shortest_hamiltonian_cycle(nodes) << "."
        << endl;
  cout << "Take the path: ";
  for (int i = 0; i < nodes; i++) {
    cout << order[i] << "->";
  }
  cout << order[0] << "." << endl;
  return 0;
}

```

Example Output

```

The shortest hamiltonian cycle has length 5.
Take the path: 0->3->2->4->1->0.

```

5.7.4 Shortest Hamiltonian Path

Given a weighted, directed graph, determine a path of minimum total distance which visits each node exactly once. Unlike the traveling salesman problem, we do not have to return to the starting vertex. Since this implementation uses bitmasks with 32-bit integers, the maximum number of nodes must be less than 32.

`shortest_hamiltonian_path()` applies to a global adjacency matrix `adj[][]` which must be populated before the function call.

Time Complexity: $O(2^n \cdot n^2)$ per call to `shortest_hamiltonian_path()`, where n is the number of nodes.

Space Complexity:

- $O(n^2)$ for storage of the graph, where n is the number of nodes.
- $O(2^n \cdot n^2)$ auxiliary heap space for `shortest_hamiltonian_path()`.

```
#include <algorithm>

const int MAXN = 20, INF = 0x3f3f3f3f;
int adj[MAXN][MAXN], dp[1 << MAXN][MAXN], order[MAXN];

int shortest_hamiltonian_path(int nodes) {
    int max_mask = (1 << nodes) - 1;
    for (int i = 0; i <= max_mask; i++) {
        std::fill(dp[i], dp[i] + nodes, INF);
    }
    for (int i = 0; i < nodes; i++) {
        dp[1 << i][i] = 0;
    }
    for (int mask = 1; mask <= max_mask; mask += 2) {
        for (int i = 0; i < nodes; i++) {
            if ((mask & 1 << i) != 0) {
                for (int j = 0; j < nodes; j++) {
                    if ((mask & 1 << j) != 0)
                        dp[mask][i] = std::min(dp[mask][i], dp[mask ^ (1 << i)][j] + adj[j][i]);
                }
            }
        }
    }
    int res = INF + INF;
    for (int i = 1; i < nodes; i++) {
        res = std::min(res, dp[max_mask][i]);
    }
    int mask = max_mask, old = -1;
    for (int i = nodes - 1; i >= 0; i--) {
        int bj = -1;
        for (int j = 0; j < nodes; j++) {
            if ((mask & 1 << j) != 0 && (bj == -1 || dp[mask][bj] + (old == -1 ? 0 : adj[bj][old]) >
                dp[mask][j] + (old == -1 ? 0 : adj[j][old]))) {
                bj = j;
            }
        }
        order[i] = bj;
        mask ^= (1 << bj);
        old = bj;
    }
    return res;
}
```

Example Usage

```
#include <iostream>
using namespace std;

int main() {
    int nodes = 3;
    adj[0][1] = 1;
    adj[0][2] = 1;
    adj[1][0] = 7;
    adj[1][2] = 2;
    adj[2][0] = 3;
    adj[2][1] = 5;
    cout << "The shortest hamiltonian path has length " << shortest_hamiltonian_path(nodes) << " "
        << endl
        << "Take the path: " << order[0];
}
```



```
for (int i = 1; i < nodes; i++) {  
    cout << "->" << order[i];  
}  
cout << "." << endl;  
return 0;  
}
```

Example Output

The shortest hamiltonian path has length 3.
Take the path: 0->1->2.

Chapter 6

Mathematics

6.1 Math Utilities

Common mathematic constants and functions, many of which are substitutes for features which are not available in standard C++, or may not be available on compilers that do not support C++11 and later.

Time Complexity: $O(1)$ for all operations.

Space Complexity: $O(1)$ auxiliary for all operations.

```
#include <algorithm>
#include <cfloat>
#include <climits>
#include <cmath>
#include <cstdlib>
#include <limits>
#include <string>
#include <vector>

#ifndef M_PI
const double M_PI = acos(-1.0);
#endif
#ifndef M_E
const double M_E = exp(1.0);
#endif
const double M_PHI = (1.0 + sqrt(5.0)) / 2.0;
const double M_INF = std::numeric_limits<double>::infinity();
const double M_NAN = std::numeric_limits<double>::quiet_NaN();

#ifndef isnan
#define isnan(x) ((x) != (x))
#endif
```

Epsilon Comparisons:

- `EQ()`, `NE()`, `LT()`, `GT()`, `LE()`, and `GE()` relationally compare two values x and y accounting for absolute error. For any x , the range of values considered equal barring absolute error is $[x - \text{EPS}, x + \text{EPS}]$. Values outside of this range are considered not equal (strictly less or strictly greater).
- `rEQ()` returns whether x and y are equal barring relative error. For any x , the range of values considered equal is $[x(1 - \text{EPS}), x(1 + \text{EPS})]$.

```
const double EPS = 1e-9;
```

```

#define EQ(x, y) (fabs((x) - (y)) <= EPS)
#define NE(x, y) (fabs((x) - (y)) > EPS)
#define LT(x, y) ((x) < (y) - EPS)
#define GT(x, y) ((x) > (y) + EPS)
#define LE(x, y) ((x) <= (y) + EPS)
#define GE(x, y) ((x) >= (y) - EPS)
#define rEQ(x, y) (fabs((x) - (y)) <= EPS * fabs(x))

```

Sign Functions:

- `sgn(x)` returns -1 (if $x < 0$), 0 (if $x = 0$), or 1 (if $x > 0$). Unlike `signbit()` or `copysign()`, this does not handle the sign of `NaN`.
- `signbit_(x)` is analogous to `std::signbit()` in C++11 and later, returning whether the sign bit of the floating point number is set to true. If so, then `x` is considered "negative." Note that this works as expected on `+0.0`, `-0.0`, `Inf`, `-Inf`, `NaN`, as well as `-NaN`. Warning: This assumes that the sign bit is the leading (most significant) bit in the internal representation of the IEEE floating point value.
- `copysign(x, y)` is analogous to `std::copysign()` in C++11 and later, returning a number with the magnitude of `x` but the sign of `y`.

```

template<class T>
int sgn(const T &x) {
    return (T(0) < x) - (x < T(0));
}

template<class Double>
bool signbit_(Double x) {
    return (((unsigned char *)&x)[sizeof(x) - 1] >> (CHAR_BIT - 1)) & 1;
}

template<class Double>
Double copysign_(Double x, Double y) {
    return signbit_(y) ? -fabs(x) : fabs(x);
}

```

Rounding Functions:

- `floor0(x)` returns `x` rounded down, symmetrically towards zero. This function is analogous to `trunc()` in C++11 and later.
- `ceil0(x)` returns `x` rounded up, symmetrically away from zero. This function is analogous to `round()` in C++11 and later.
- `round_half_up(x)` returns `x` rounded half up, towards positive infinity.
- `round_half_down(x)` returns `x` rounded half down, towards negative infinity.
- `round_half_to0(x)` returns `x` rounded half down, symmetrically towards zero.
- `round_half_from0(x)` returns `x` rounded half up, symmetrically away from zero.
- `round_half_even(x)` returns `x` rounded half to even, using banker's rounding.
- `round_half_alternate(x)` returns `x` rounded, where ties are broken by alternating rounds towards positive and negative infinity.
- `round_half_alternate0(x)` returns `x` rounded, where ties are broken by alternating symmetric rounds towards and away from zero.
- `round_half_random(x)` returns `x` rounded, where ties are broken randomly.
- `round_n_places(x, n, f)` returns `x` rounded to `n` digits after the decimal, using the specified rounding function `f(x)`.

```

template<class Double>
Double floor0(const Double &x) {
    Double res = floor(fabs(x));
    return (x < 0.0) ? -res : res;
}

```

```

template<class Double>
Double ceil0(const Double &x) {
    Double res = ceil(fabs(x));
    return (x < 0.0) ? -res : res;
}

template<class Double>
Double round_half_up(const Double &x) {
    return floor(x + 0.5);
}

template<class Double>
Double round_half_down(const Double &x) {
    return ceil(x - 0.5);
}

template<class Double>
Double round_half_to0(const Double &x) {
    Double res = round_half_down(fabs(x));
    return (x < 0.0) ? -res : res;
}

template<class Double>
Double round_half_from0(const Double &x) {
    Double res = round_half_up(fabs(x));
    return (x < 0.0) ? -res : res;
}

template<class Double>
Double round_half_even(const Double &x, const Double &eps = 1e-9) {
    if (x < 0.0) {
        return -round_half_even(-x, eps);
    }
    Double ipart;
    modf(x, &ipart);
    if (x - (ipart + 0.5) < eps) {
        return (fmod(ipart, 2.0) < eps) ? ipart : ceil0(ipart + 0.5);
    }
    return round_half_from0(x);
}

template<class Double>
Double round_half_alterate(const Double &x) {
    static bool up = true;
    return (up = !up) ? round_half_up(x) : round_half_down(x);
}

template<class Double>
Double round_half_alterate0(const Double &x) {
    static bool up = true;
    return (up = !up) ? round_half_from0(x) : round_half_to0(x);
}

template<class Double>
Double round_half_random(const Double &x) {
    return (rand() % 2 == 0) ? round_half_from0(x) : round_half_to0(x);
}

template<class Double, class RoundingFunction>
Double round_n_places(const Double &x, unsigned int n, RoundingFunction f) {
    return f(x * pow(10, n)) / pow(10, n);
}

```

Error Function:

- `erf(x)` returns the error encountered in integrating the normal distribution. Its value is $2/\sqrt{\pi} \int_0^x e^{-t^2} dt$. This function is analogous to `erf(x)` in C++11 and later.

- `erfc_(x)` returns the error function complement, that is, $1 - \text{erf_}(x)$. This function is analogous to `erfc(x)` in C++11 and later.

```
#define ERF_EPS 1e-14

double erfc_(double x);

double erf_(double x) {
    if (signbit_(x)) {
        return -erf_(-x);
    }
    if (fabs(x) > 2.2) {
        return 1.0 - erfc_(x);
    }
    double sum = x, term = x, xx = x * x;
    int j = 1;
    do {
        term *= xx / j;
        sum -= term / (2 * (j++) + 1);
        term *= xx / j;
        sum += term / (2 * (j++) + 1);
    } while (fabs(term) > sum * ERF_EPS);
    return 2 / sqrt(M_PI) * sum;
}

double erfc_(double x) {
    if (fabs(x) < 2.2) {
        return 1.0 - erf_(x);
    }
    if (signbit_(x)) {
        return 2.0 - erfc_(-x);
    }
    double a = 1, b = x, c = x, d = x * x + 0.5, q1, q2 = 0, n = 1.0, t;
    do {
        t = a * n + b * x;
        a = b;
        b = t;
        t = c * n + d * x;
        c = d;
        d = t;
        n += 0.5;
        q1 = q2;
        q2 = b / d;
    } while (fabs(q1 - q2) > q2 * ERF_EPS);
    return 1 / sqrt(M_PI) * exp(-x * x) * q2;
}

#undef ERF_EPS
```

Gamma Functions:

- `tgamma_(x)` returns the gamma function of `x`. Unlike the `tgamma()` function in C++11 and later, this version only supports positive `x`, returning `NaN` if `x` is less than or equal to 0.
- `lgamma_(x)` returns the natural logarithm of the absolute value of the gamma function of `x`. Unlike the `lgamma()` function in C++11 and later, this version only supports positive `x`, returning `NaN` if `x` is less than or equal to 0.

```
double lgamma_(double x);

double tgamma_(double x) {
    if (x <= 0) {
        return M_NAN;
    }
    if (x < 1e-3) {
        return 1.0 / (x * (1.0 + 0.57721566490153286060651209 * x));
    }
}
```

```

}
if (x < 12) {
    double y = x;
    int n = 0;
    bool arg_was_less_than_one = (y < 1);
    if (arg_was_less_than_one) {
        y += 1;
    } else {
        n = (int)floor(y) - 1;
        y -= n;
    }
    static const double p[] = {-1.71618513886549492533811e+0, 2.47656508055759199108314e+1,
                                -3.79804256470945635097577e+2, 6.29331155312818442661052e+2,
                                8.66966202790413211295064e+2, -3.14512729688483675254357e+4,
                                -3.61444134186911729807069e+4, 6.64561438202405440627855e+4};
    static const double q[] = {-3.08402300119738975254353e+1, 3.15350626979604161529144e+2,
                                -1.01515636749021914166146e+3, -3.10777167157231109440444e+3,
                                2.25381184209801510330112e+4, 4.75584627752788110767815e+3,
                                -1.34659959864969306392456e+5, -1.15132259675553483497211e+5};

    double num = 0, den = 1, z = y - 1;
    for (int i = 0; i < 8; i++) {
        num = (num + p[i]) * z;
        den = den * z + q[i];
    }
    double result = num / den + 1;
    if (arg_was_less_than_one) {
        result /= (y - 1);
    } else {
        for (int i = 0; i < n; i++) {
            result *= y++;
        }
    }
    return result;
}
return (x > 171.624) ? 2 * DBL_MAX : exp(lgamma(x));
}

double lgamma_(double x) {
    if (x <= 0) {
        return M_NAN;
    }
    if (x < 12) {
        return log(fabs(tgamma_(x)));
    }
    static const double c[8] = {1.0 / 12, -1.0 / 360, 1.0 / 1260, -1.0 / 1680,
                                1.0 / 1188, -691.0 / 360360, 1.0 / 156, -3617.0 / 122400};
    double z = 1.0 / (x * x), sum = c[7];
    for (int i = 6; i >= 0; i--) {
        sum = sum * z + c[i];
    }
    return (x - 0.5) * log(x) - x + 0.91893853320467274178032973640562 + sum / x;
}

```

Base Conversion:

- Given an integer in base *a* as a vector *d* of digits (where *d*[0] is the least significant digit), `convert_base(d, a, b)` returns a vector of the integer's digits when converted base *b* (again with index 0 storing the least significant digit). The actual value of the entire integer to be converted must be able to fit within an unsigned 64-bit integer for intermediate storage.
- `to_base(x, b)` returns the digits of the unsigned integer *x* in base *b*, where index 0 of the result stores the least significant digit.
- `to_roman(x)` returns the Roman numeral representation of the unsigned integer *x* as an `std::string`.

```

std::vector<int> convert_base(const std::vector<int> &d, int a, int b) {
    unsigned long long x = 0, power = 1;

```

```

    for (int i = 0; i < (int)d.size(); i++) {
        x += d[i] * power;
        power *= a;
    }
    int n = ceil(log(x + 1) / log(b));
    std::vector<int> res;
    for (int i = 0; i < n; i++) {
        res.push_back(x % b);
        x /= b;
    }
    return res;
}

std::vector<int> to_base(unsigned int x, int b = 10) {
    std::vector<int> res;
    while (x != 0) {
        res.push_back(x % b);
        x /= b;
    }
    return res;
}

std::string to_roman(unsigned int x) {
    static const std::string h[] = {"", "C", "CC", "CCC", "CD", "D", "DC", "DCC", "DCCC", "CM"};
    static const std::string t[] = {"", "X", "XX", "XXX", "XL", "L", "LX", "LXX", "LXXX", "XC"};
    static const std::string o[] = {"", "I", "II", "III", "IV", "V", "VI", "VII", "VIII", "IX"};
    std::string prefix(x / 1000, 'M');
    x %= 1000;
    return prefix + h[x / 100] + t[x / 10 % 10] + o[x % 10];
}

```

Example Usage

```

#include <cassert>
#include <iostream>

int main() {
    assert(EQ(M_PI, 3.14159265359));
    assert(EQ(M_E, 2.718281828459));
    assert(EQ(M_PHI, 1.61803398875));

    double x = -12345.6789;
    assert((-M_INF < x) && (x < M_INF));
    assert((M_INF + x == M_INF) && (M_INF - x == M_INF));
    assert((M_INF + M_INF == M_INF) && (-M_INF - M_INF == -M_INF));
    assert((M_NAN != x) && (M_NAN != M_INF) && (M_NAN != M_NAN));
    assert(!(M_NAN < x) && !(M_NAN > x) && !(M_NAN <= x) && !(M_NAN >= x));
    assert(isnan(0.0 * M_INF) && isnan(0.0 * -M_INF) && isnan(M_INF / -M_INF));
    assert(isnan(M_NAN) && isnan(-M_NAN) && isnan(M_INF - M_INF));

    assert(sgn(x) == -1 && sgn(0.0) == 0 && sgn(5678) == 1);
    assert(signbit(x) && !signbit(0.0) && signbit(-0.0));
    assert(!signbit(M_INF) && signbit(-M_INF));
    assert(!signbit(M_NAN) && signbit(-M_NAN));
    assert(copysign(1.0, +2.0) == +1.0 && copysign(M_INF, -2.0) == -M_INF);
    assert(copysign(1.0, -2.0) == -1.0 && std::signbit(copysign(M_NAN, -2.0)));

    assert(EQ(floor0(1.5), 1.0) && EQ(ceil0(1.5), 2.0));
    assert(EQ(floor0(-1.5), -1.0) && EQ(ceil0(-1.5), -2.0));
    assert(EQ(round_half_up(+1.5), +2) && EQ(round_half_down(+1.5), +1));
    assert(EQ(round_half_up(-1.5), -1) && EQ(round_half_down(-1.5), -2));
    assert(EQ(round_half_to0(+1.5), +1) && EQ(round_half_from0(+1.5), +2));
    assert(EQ(round_half_to0(-1.5), -1) && EQ(round_half_from0(-1.5), -2));
    assert(EQ(round_half_even(+1.5), +2) && EQ(round_half_even(-1.5), -2));
    assert(NE(round_half_alternate(+1.5), round_half_alternate(+1.5)));
    assert(NE(round_half_alternate0(-1.5), round_half_alternate0(-1.5)));
}

```

```

assert(EQ(round_n_places(-1.23456, 3, round_half_to0<double>), -1.235));

assert(EQ(erf_(1.0), 0.8427007929) && EQ(erf_(-1.0), -0.8427007929));
assert(EQ(tgamma_(0.5), 1.7724538509) && EQ(tgamma_(1.0), 1.0));
assert(EQ(lgamma_(0.5), 0.5723649429) && EQ(lgamma_(1.0), 0.0));

int digits[] = {6, 5, 4, 3, 2, 1};
std::vector<int> base20 = to_base(123456, 20);
assert(convert_base(base20, 20, 10) == std::vector<int>(digits, digits + 6));
assert(to_roman(1234) == "MCCXXXIV");
assert(to_roman(5678) == "MMMMDCLXXVIII");
return 0;
}

```

6.2 Combinatorics

6.2.1 Combinatorial Calculations

The following functions implement common operations in combinatorics. All inputs must be non-negative. All return values and table entries are computed as 64-bit integers modulo an input argument m or p .

- `factorial(n, m)` returns $n! \bmod m$.
- `factorialp(n, p)` returns $n! \bmod p$, where p is prime.
- `binomial_table(n, m)` returns rows 0 to n of Pascal's triangle as a table t such that $t[i][j] = \binom{i}{j} \bmod m$.
- `permute(n, k, m)` returns $(n \text{ permute } k) \bmod m$.
- `choose(n, k, p)` returns $\binom{n}{k} \bmod p$, where p is prime.
- `multichoose(n, k, p)` returns $(n \text{ multichoose } k) \bmod p$, where p is prime.
- `catalan(n, p)` returns the n th Catalan number mod p , where p is prime.
- `partitions(n, m)` returns the number of partitions of n , mod m .
- `partitions(n, k, m)` returns the number of partitions of n into k parts, mod m .
- `stirling1(n, k, m)` returns the (n, k) unsigned Stirling number of the 1st kind mod m .
- `stirling2(n, k, m)` returns the (n, k) Stirling number of the 2nd kind mod m .
- `eulerian1(n, k, m)` returns the (n, k) Eulerian number of the 1st kind mod m , where $n > k$.
- `eulerian2(n, k, m)` returns the (n, k) Eulerian number of the 2nd kind mod m , where $n > k$.

Time Complexity:

- $O(n)$ for `factorial(n, m)`.
- $O(p \log n)$ for `factorialp(n, p)`.
- $O(n^2)$ for `binomial_table(n, m)`.
- $O(k)$ for `permute(n, k, p)`.
- $O(\min(k, n - k))$ for `choose(n, k, p)`.
- $O(k)$ for `multichoose(n, k, p)`.
- $O(n)$ for `catalan(n, p)`.
- $O(n^2)$ for `partitions(n, m)`.
- $O(n \cdot k)$ for `partitions(n, k, m)`, `stirling1(n, k, m)`, `stirling2(n, k, m)`, `eulerian1(n, k, m)`, and `eulerian2(n, k, m)`.

Space Complexity:

- $O(n^2)$ auxiliary heap space for `binomial_table(n, m)`.
- $O(n \cdot k)$ auxiliary heap space for `partitions(n, k, m)`, `stirling1(n, k, m)`, `stirling2(n, k, m)`, `eulerian1(n, k, m)`, and `eulerian2(n, k, m)`.
- $O(1)$ auxiliary for all other operations.

```
#include <vector>
```



```

typedef long long int64;
typedef std::vector<std::vector<int64>> table;

int64 factorial(int n, int m = 1000000007) {
    int64 res = 1;
    for (int i = 2; i <= n; i++) {
        res = (res * i) % m;
    }
    return res % m;
}

int64 factorialp(int64 n, int64 p = 1000000007) {
    int64 res = 1;
    while (n > 1) {
        if (n / p % 2 == 1) {
            res = res * (p - 1) % p;
        }
        int max = n % p;
        for (int i = 2; i <= max; i++) {
            res = (res * i) % p;
        }
        n /= p;
    }
    return res % p;
}

table binomial_table(int n, int64 m = 1000000007) {
    table t(n + 1);
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= i; j++) {
            if (i < 2 || j == 0 || i == j) {
                t[i].push_back(1);
            } else {
                t[i].push_back((t[i - 1][j - 1] + t[i - 1][j]) % m);
            }
        }
    }
    return t;
}

int64 permute(int n, int k, int64 m = 1000000007) {
    if (n < k) {
        return 0;
    }
    int64 res = 1;
    for (int i = 0; i < k; i++) {
        res = res * (n - i) % m;
    }
    return res % m;
}

int64 mulmod(int64 x, int64 n, int64 m) {
    int64 a = 0, b = x % m;
    for (; n > 0; n >= 1) {
        if (n & 1) {
            a = (a + b) % m;
        }
        b = (b << 1) % m;
    }
    return a % m;
}

int64 powmod(int64 x, int64 n, int64 m) {
    int64 a = 1, b = x;
    for (; n > 0; n >= 1) {
        if (n & 1) {
            a = mulmod(a, b, m);
        }
    }
}

```

```

    b = mulmod(b, b, m);
}
return a % m;
}

int64 choose(int n, int k, int64 p = 1000000007) {
    if (n < k) {
        return 0;
    }
    if (k > n - k) {
        k = n - k;
    }
    int64 num = 1, den = 1;
    for (int i = 0; i < k; i++) {
        num = num * (n - i) % p;
    }
    for (int i = 1; i <= k; i++) {
        den = den * i % p;
    }
    return num * powmod(den, p - 2, p) % p;
}

int64 multichoose(int n, int k, int64 p = 1000000007) {
    return choose(n + k - 1, k, p);
}

int64 catalan(int n, int64 p = 1000000007) {
    return choose(2 * n, n, p) * powmod(n + 1, p - 2, p) % p;
}

int64 partitions(int n, int64 m = 1000000007) {
    std::vector<int64> t(n + 1, 0);
    t[0] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = i; j <= n; j++) {
            t[j] = (t[j] + t[j - i]) % m;
        }
    }
    return t[n] % m;
}

int64 partitions(int n, int k, int64 m = 1000000007) {
    table t(n + 1, std::vector<int64>(k + 1, 0));
    t[0][1] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = 1, h = k < i ? k : i; j <= h; j++) {
            t[i][j] = (t[i - 1][j - 1] + t[i - j][j]) % m;
        }
    }
    return t[n][k] % m;
}

int64 stirling1(int n, int k, int64 m = 1000000007) {
    table t(n + 1, std::vector<int64>(k + 1, 0));
    t[0][0] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            t[i][j] = (i - 1) * t[i - 1][j] % m;
            t[i][j] = (t[i][j] + t[i - 1][j - 1]) % m;
        }
    }
    return t[n][k] % m;
}

int64 stirling2(int n, int k, int64 m = 1000000007) {
    table t(n + 1, std::vector<int64>(k + 1, 0));
    t[0][0] = 1;
    for (int i = 1; i <= n; i++) {

```

```

    for (int j = 1; j <= k; j++) {
        t[i][j] = j * t[i - 1][j] % m;
        t[i][j] = (t[i][j] + t[i - 1][j - 1]) % m;
    }
}
return t[n][k] % m;
}

int64 eulerian1(int n, int k, int64 m = 1000000007) {
    if (k > n - 1 - k) {
        k = n - 1 - k;
    }
    table t(n + 1, std::vector<int64>(k + 1, 1));
    for (int j = 1; j <= k; j++) {
        t[0][j] = 0;
    }
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            t[i][j] = (i - j) * t[i - 1][j - 1] % m;
            t[i][j] = (t[i][j] + ((j + 1) * t[i - 1][j] % m)) % m;
        }
    }
    return t[n][k] % m;
}

int64 eulerian2(int n, int k, int64 m = 1000000007) {
    table t(n + 1, std::vector<int64>(k + 1, 1));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            if (i == j) {
                t[i][j] = 0;
            } else {
                t[i][j] = (j + 1) * t[i - 1][j] % m;
                t[i][j] = ((2 * i - 1 - j) * t[i - 1][j - 1] % m + t[i][j]) % m;
            }
        }
    }
    return t[n][k] % m;
}

```

Example Usage

```

#include <cassert>

int main() {
    table t = binomial_table(10);
    for (int i = 0; i < (int)t.size(); i++) {
        for (int j = 0; j < (int)t[i].size(); j++) {
            assert(t[i][j] == choose(i, j));
        }
    }
    assert(factorial(10) == 3628800);
    assert(factorialp(123456) == 639390503);
    assert(permute(10, 4) == 5040);
    assert(choose(20, 7) == 77520);
    assert(multichoose(20, 7) == 657800);
    assert(catalan(10) == 16796);
    assert(partitions(4) == 5);
    assert(partitions(100, 5) == 38225);
    assert(stirling1(4, 2) == 11);
    assert(stirling2(4, 3) == 6);
    assert(eulerian1(9, 5) == 88234);
    assert(eulerian2(8, 3) == 195800);
    return 0;
}

```

6.2.2 Enumerating Arrangements

For the purposes of this section, we define a "size k arrangement of n " to be a permutation of a size k subset of the integers from 0 to $n - 1$, for $0 \leq k \leq n$. There are n permute k possible arrangements, but n^k possible arrangements if repeated values are allowed.

- `next_arrangement(n, k, a)` tries to rearrange `a[]` to the next lexicographically greater arrangement, returning true if such an arrangement exists or false if the array is already in descending order (in which case `a[]` is unchanged). The input `a[]` must consist of exactly k distinct integers in the range $[0, n)$.
- `arrangement_by_rank(n, k, r)` returns the size k arrangement of n which is lexicographically ranked r out of all size k arrangements of n , where r is a zero-based rank in the range $[0, n \text{ permute } k)$.
- `rank_by_arrangement(n, k, a)` returns an integer representing the zero-based rank of arrangement `a[]`, which must consist of exactly k distinct integers in the range $[0, n)$.
- `next_arrangement_with_repeats(n, k, a)` tries to rearrange `a[]` to the next lexicographically greater arrangement with repeats, returning true if such an arrangement exists or false if the array is already in descending order (in which case `a[]` is unchanged). The input `a[]` must consist of exactly k (not necessarily distinct) integers in the range $[0, n)$. If `a[]` were interpreted as a k digit integer in base n , this function could be thought of as incrementing the integer.

Time Complexity:

- $O(n \cdot k)$ for `next_arrangement()`, `arrangement_by_rank()`, and `rank_by_arrangement()`.
- $O(k)$ for `next_arrangement_with_repeats()`.

Space Complexity:

- $O(n)$ auxiliary heap space for `next_arrangement()`, `arrangement_by_rank()`, and `rank_by_arrangement()`.
- $O(1)$ auxiliary for `next_arrangement_with_repeats()`.

```
#include <algorithm>
#include <vector>

bool next_arrangement(int n, int k, int a[]) {
    std::vector<bool> used(n);
    for (int i = 0; i < k; i++) {
        used[a[i]] = true;
    }
    for (int i = k - 1; i >= 0; i--) {
        used[a[i]] = false;
        for (int j = a[i] + 1; j < n; j++) {
            if (!used[j]) {
                a[i++] = j;
                used[j] = true;
                for (int x = 0; x < k; x++) {
                    if (!used[x]) {
                        a[i++] = x;
                    }
                }
                return true;
            }
        }
    }
    return false;
}

long long n_permute_k(int n, int k) {
    long long res = 1;
    for (int i = 0; i < k; i++) {
        res *= n - i;
    }
    return res;
}

std::vector<int> arrangement_by_rank(int n, int k, long long r) {
```

```

std::vector<int> values(n), res(k);
for (int i = 0; i < n; i++) {
    values[i] = i;
}
for (int i = 0; i < k; i++) {
    long long count = n_permute_k(n - 1 - i, k - 1 - i);
    int pos = r / count;
    res[i] = values[pos];
    std::copy(values.begin() + pos + 1, values.end(), values.begin() + pos);
    r %= count;
}
return res;
}

long long rank_by_arrangement(int n, int k, int a[]) {
    long long res = 0;
    std::vector<bool> used(n);
    for (int i = 0; i < k; i++) {
        int count = 0;
        for (int j = 0; j < a[i]; j++) {
            if (!used[j]) {
                count++;
            }
        }
        res += count * n_permute_k(n - i - 1, k - i - 1);
        used[a[i]] = true;
    }
    return res;
}

bool next_arrangement_with_repeats(int n, int k, int a[]) {
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - 1) {
            a[i]++;
            std::fill(a + i + 1, a + k, 0);
            return true;
        }
    }
    return false;
}

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

template<class It>
void print_range(It lo, It hi) {
    cout << "{";
    for (; lo != hi; ++lo) {
        cout << *lo << (lo == hi - 1 ? " " : ",");
    }
    cout << "} ";
}

int main() {
    {
        int n = 4, k = 3, a[] = {0, 1, 2};
        cout << n << " permute " << k << " arrangements:" << endl;
        int count = 0;
        do {
            print_range(a, a + k);
            vector<int> b = arrangement_by_rank(n, k, count);
            assert(equal(a, a + k, b.begin()));
            assert(rank_by_arrangement(n, k, a) == count);

```

```

    count++;
} while (next_arrangement(n, k, a));
cout << endl;
}
{
    int n = 4, k = 2, a[] = {0, 0};
    cout << endl << n << "~" << k << " arrangements with repeats:" << endl;
    do {
        print_range(a, a + k);
    } while (next_arrangement_with_repeats(n, k, a));
    cout << endl;
}
return 0;
}

```

Example Output

```

4 permute 3 arrangements:
{0,1,2} {0,1,3} {0,2,1} {0,2,3} {0,3,1} {0,3,2} {1,0,2} {1,0,3} {1,2,0} {1,2,3}
{1,3,0} {1,3,2} {2,0,1} {2,0,3} {2,1,0} {2,1,3} {2,3,0} {2,3,1} {3,0,1} {3,0,2}
{3,1,0} {3,1,2} {3,2,0} {3,2,1}

4^2 arrangements with repeats:
{0,0} {0,1} {0,2} {0,3} {1,0} {1,1} {1,2} {1,3} {2,0} {2,1} {2,2} {2,3} {3,0}
{3,1} {3,2} {3,3}

```

6.2.3 Enumerating Permutations

A permutation is an ordered list consisting of n (not necessarily distinct) elements.

- `next_permutation(lo, hi)` is analogous to `std::next_permutation(lo, hi)`, taking two BidirectionalIterators `lo` and `hi` as a range `[lo, hi)` for which the function tries to rearrange to the next lexicographically greater permutation. The function returns true if such a permutation exists, or false if the range is already in descending order (in which case the values are unchanged). This implementation requires an ordering on the set of possible elements defined by the `<` operator on the iterator's value type.
- `next_permutation(n, a)` is analogous to `next_permutation()`, except that it takes an array `a[]` of size n instead of a range.
- `next_permutation(x)` returns the next lexicographically greater permutation of the binary digits of the integer `x`, that is, the lowest integer greater than `x` with the same number of 1-bits. This can be used to generate combinations of a set of n items by treating each 1 bit as whether to "take" the item at the corresponding position.
- `permutation_by_rank(n, r)` returns the permutation of the integers in the range $[0, n)$ which is lexicographically ranked r , where r is a zero-based rank in the range $[0, n!)$.
- `rank_by_permutation(n, a)` returns an integer representing the zero-based rank of permutation `a[]`, which must be a permutation of the integers $[0, n)$.
- `permutation_cycles(n, a)` returns the decomposition of the permutation `a[]` into cycles. A permutation cycle is a subset of a permutation whose elements are consecutively swapped, relative to a sorted set. For example, `{3, 1, 0, 2}` decomposes to `{0, 3, 2}` and `{1}`, meaning that starting from the sorted order `{0, 1, 2, 3}`, the 0th value is replaced by the 3rd, the 3rd by the 2nd, and the 2nd by the 0th ($0 \rightarrow 3 \rightarrow 2 \rightarrow 0$).

Time Complexity:

- $O(n^2)$ per call to `next_permutation(lo, hi)`, where n is the distance between `lo` and `hi`.
- $O(n^2)$ per call to `next_permutation(n, a)`, `permutation_by_rank(n, r)`, and `rank_by_permutation(n, a)`.
- $O(1)$ per call to `next_permutation(x)`.
- $O(n)$ per call to `permutation_cycles()`.

Space Complexity:

- $O(1)$ auxiliary for `next_permutation()` and `next_permutation()`.
- $O(n)$ auxiliary heap space for `permutation_by_rank()`, `rank_by_permutation()`, and `permutation_cycles()`.

```

#include <algorithm>
#include <vector>

template<class It>
bool next_permutation_(It lo, It hi) {
    if (lo == hi) {
        return false;
    }
    It i = lo;
    if (++i == hi) {
        return false;
    }
    i = hi;
    --i;
    for (;;) {
        It j = i;
        if (*--i < *j) {
            It k = hi;
            while (!(*i < *--k)) {
            }
            std::iter_swap(i, k);
            std::reverse(j, hi);
            return true;
        }
        if (i == lo) {
            std::reverse(lo, hi);
            return false;
        }
    }
}

template<class T>
bool next_permutation(int n, T a[]) {
    for (int i = n - 2; i >= 0; i--) {
        if (a[i] < a[i + 1]) {
            for (int j = n - 1; j--;) {
                if (a[i] < a[j]) {
                    std::swap(a[i++], a[j]);
                    for (j = n - 1; i < j; i++, j--) {
                        std::swap(a[i], a[j]);
                    }
                    return true;
                }
            }
        }
    }
    return false;
}

long long next_permutation(long long x) {
    long long s = x & -x, r = x + s;
    return r | (((x ^ r) >> 2) / s);
}

std::vector<int> permutation_by_rank(int n, long long x) {
    std::vector<long long> factorial(n);
    std::vector<int> values(n), res(n);
    factorial[0] = 1;
    for (int i = 1; i < n; i++) {
        factorial[i] = i * factorial[i - 1];
    }
    for (int i = 0; i < n; i++) {
        values[i] = i;
    }
    for (int i = 0; i < n; i++) {
        int pos = x / factorial[n - 1 - i];
        res[i] = values[pos];
        std::copy(values.begin() + pos + 1, values.end(), values.begin() + pos);
    }
}

```

```

    x %= factorial[n - 1 - i];
}
return res;
}

long long rank_by_permutation(int n, int a[]) {
    std::vector<long long> factorial(n);
    factorial[0] = 1;
    for (int i = 1; i < n; i++) {
        factorial[i] = i * factorial[i - 1];
    }
    long long res = 0;
    for (int i = 0; i < n; i++) {
        int v = a[i];
        for (int j = 0; j < i; j++) {
            if (a[j] < a[i]) {
                v--;
            }
        }
        res += v * factorial[n - 1 - i];
    }
    return res;
}

typedef std::vector<std::vector<int>> cycles;

cycles permutation_cycles(int n, int a[]) {
    std::vector<bool> visit(n);
    cycles res;
    for (int i = 0; i < n; i++) {
        if (!visit[i]) {
            int j = i;
            std::vector<int> curr;
            do {
                curr.push_back(j);
                visit[j] = true;
                j = a[j];
            } while (j != i);
            res.push_back(curr);
        }
    }
    return res;
}

```

Example Usage

```

#include <bitset>
#include <cassert>
#include <iostream>
using namespace std;

template<class It>
void print_range(It lo, It hi) {
    cout << "{";
    for (; lo != hi; ++lo) {
        cout << *lo << (lo == hi - 1 ? " " : ",");
    }
    cout << "} ";
}

int main() {
    {
        const int n = 4;
        int a[] = {0, 1, 2, 3}, b[n], c[n];
        for (int i = 0; i < n; i++) {
            b[i] = c[i] = a[i];

```



```

}
cout << "Permutations of [0, " << n << "):" << endl;
int count = 0;
do {
    print_range(a, a + n);
    assert(equal(b, b + n, a));
    assert(equal(c, c + n, a));
    vector<int> d = permutation_by_rank(n, count);
    assert(equal(d.begin(), d.end(), a));
    assert(rank_by_permutation(n, a) == count);
    count++;
    std::next_permutation(b, b + n);
    next_permutation(c, c + n);
} while (next_permutation(n, a));
cout << endl;
}
{ // Permutations of binary digits.
    const int n = 5;
    cout << "\nPermutations of 2 zeros and 3 ones:" << endl;
    int lo = bitset<5>(string("00111")).to_ulong();
    int hi = bitset<6>(string("100011")).to_ulong();
    do {
        cout << bitset<n>(lo).to_string() << " ";
    } while ((lo = next_permutation(lo)) != hi);
    cout << endl;
}
{ // Decomposition into cycles.
    const int n = 4;
    int a[] = {3, 1, 0, 2};
    cout << "\nDecomposition of {3,1,0,2} into cycles:" << endl;
    cycles c = permutation_cycles(n, a);
    for (int i = 0; i < (int)c.size(); i++) {
        print_range(c[i].begin(), c[i].end());
    }
    cout << endl;
}
return 0;
}
}

```

Example Output

```

Permutations of [0, 4):
{0,1,2,3} {0,1,3,2} {0,2,1,3} {0,2,3,1} {0,3,1,2} {0,3,2,1} {1,0,2,3} {1,0,3,2}
{1,2,0,3} {1,2,3,0} {1,3,0,2} {1,3,2,0} {2,0,1,3} {2,0,3,1} {2,1,0,3} {2,1,3,0}
{2,3,0,1} {2,3,1,0} {3,0,1,2} {3,0,2,1} {3,1,0,2} {3,1,2,0} {3,2,0,1} {3,2,1,0}

Permutations of 2 zeros and 3 ones:
00111 01011 01101 01110 10011 10101 10110 11001 11010 11100

Decomposition of {3,1,0,2} into cycles:
{0,3,2} {1}

```

6.2.4 Enumerating Combinations

A combination is a subset of size k chosen from a total of n (not necessarily distinct) elements, where order does not matter.

- `next_combination(lo, mid, hi)` takes random-access iterators `lo`, `mid`, and `hi` as a range `[lo, hi)` of n elements for which the function will rearrange such that the k elements in `[lo, mid)` become the next lexicographically greater combination. The function returns true if such a combination exists, or false if `[lo, mid)` already consists of the lexicographically greatest combination of the elements in `[lo, hi)` (in which case the values are unchanged). This implementation requires an ordering on the set of possible elements defined by `operator <` on the iterator's value type.

- `next_combination(n, k, a)` rearranges `a[]` to become the next lexicographically greater combination of k distinct integers in the range $[0, n)$. The array `a[]` must consist of k distinct integers in the range $[0, n)$.
- `next_combination_mask(x)` interprets the bits of an integer `x` as a mask with 1-bits specifying the chosen items for a combination and returns the mask of the next lexicographically greater combination (that is, the lowest integer greater than `x` with the same number of 1 bits). Note that this does not generate combinations in the same order as `next_combination()`, nor does it work if the corresponding n items are not distinct (in that case, duplicate combinations will be generated).
- `combination_by_rank(n, k, r)` returns the combination of k distinct integers in the range $[0, n)$ that is lexicographically ranked r , where r is a zero-based rank in the range $[0, \binom{n}{k})$.
- `rank_by_combination(n, k, a)` returns an integer representing the zero-based rank of combination `a[]`, which must consist of k distinct integers in $[0, n)$.
- `next_combination_with_repeats(n, k, a)` rearranges `a[]` to become the next lexicographically greater combination of k (not necessarily distinct) integers in the range $[0, n)$. The array `a[]` must consist of k integers in the range $[0, n)$. Note that there is a total of n multichoose k combinations if repetition is allowed, where n multichoose $k = \binom{n+k-1}{k}$.

Time Complexity:

- $O(n)$ per call to `next_combination(lo, hi)`, where n is the distance between `lo` and `hi`.
- $O(k)$ per call to `next_combination(n, k, a)` and `next_combination_with_repeats(n, k, a)`.
- $O(1)$ per call to `next_combination_mask(x)`.
- $O(n \cdot k)$ per call to `combination_by_rank()` and `rank_by_combination()`.

Space Complexity:

- $O(k)$ auxiliary heap space for `combination_by_rank(n, k, r)`.
- $O(1)$ auxiliary for all other operations.

```
#include <algorithm>
#include <iterator>
#include <vector>

template<class It>
bool next_combination(It lo, It mid, It hi) {
    if (lo == mid || mid == hi) {
        return false;
    }
    It l = mid - 1, h = hi - 1;
    int len1 = 1, len2 = 1;
    while (l != lo && !(*l < *h)) {
        --l;
        ++len1;
    }
    if (l == lo && !(*l < *h)) {
        std::rotate(lo, mid, hi);
        return false;
    }
    for (; mid < h; ++len2) {
        if (!(*l < *--h)) {
            ++h;
            break;
        }
    }
    if (len1 == 1 || len2 == 1) {
        std::iter_swap(l, h);
    } else if (len1 == len2) {
        std::swap_ranges(l, mid, h);
    } else {
        std::iter_swap(l++, h++);
        int total = (--len1) + (--len2), gcd = total;
        for (int i = len1; i != 0; i--) {
            std::swap(gcd %= i, i);
        }
        int skip = total / gcd - 1;
    }
}
```

```

    for (int i = 0; i < gcd; i++) {
        It curr = (i < len1) ? (1 + i) : (h + (i - len1));
        int k = i;
        typename std::iterator_traits<It>::value_type prev(*curr);
        for (int j = 0; j < skip; j++) {
            k = (k + len1) % total;
            It next = (k < len1) ? (1 + k) : (h + (k - len1));
            *curr = *next;
            curr = next;
        }
        *curr = prev;
    }
}
return true;
}

bool next_combination(int n, int k, int a[]) {
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - k + i) {
            a[i]++;
            while (++i < k) {
                a[i] = a[i - 1] + 1;
            }
            return true;
        }
    }
    return false;
}

long long next_combination_mask(long long x) {
    long long s = x & -x, r = x + s;
    return r | ((x ^ r) >> 2) / s;
}

long long n_choose_k(long long n, long long k) {
    if (k > n - k) {
        k = n - k;
    }
    long long res = 1;
    for (int i = 0; i < k; i++) {
        res = res * (n - i) / (i + 1);
    }
    return res;
}

std::vector<int> combination_by_rank(int n, int k, long long r) {
    std::vector<int> res(k);
    int count = n;
    for (int i = 0; i < k; i++) {
        int j = 1;
        for (;;) {
            long long am = n_choose_k(count - j, k - 1 - i);
            if (r < am) {
                break;
            }
            r -= am;
        }
        res[i] = (i > 0) ? (res[i - 1] + j) : (j - 1);
        count -= j;
    }
    return res;
}

long long rank_by_combination(int n, int k, int a[]) {
    long long res = 0;
    int prev = -1;
    for (int i = 0; i < k; i++) {
        for (int j = prev + 1; j < a[i]; j++) {

```

```

        res += n_choose_k(n - 1 - j, k - 1 - i);
    }
    prev = a[i];
}
return res;
}

bool next_combination_with_repeats(int n, int k, int a[]) {
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - 1) {
            for (++a[i]; ++i < k;) {
                a[i] = a[i - 1];
            }
            return true;
        }
    }
    return false;
}

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

template<class It>
void print_range(It lo, It hi) {
    cout << "{";
    for (; lo != hi; ++lo) {
        cout << *lo << (lo == hi - 1 ? "" : ",");
    }
    cout << "} ";
}

int main() {
    {
        int k = 3;
        string s = "11234";
        cout << "\" " << s << "\" choose " << k << ":" << endl;
        do {
            cout << s.substr(0, k) << " ";
        } while (next_combination(s.begin(), s.begin() + k, s.end()));
        cout << endl;
    }
    { // Unordered combinations using masks.
        int n = 5, k = 3;
        string char_set = "abcde"; // Must be distinct.
        cout << "\"\n\" " << char_set << "\" choose " << k << " with masks:" << endl;
        long long mask = 0, dest = 0;
        for (int i = 0; i < k; i++) {
            mask |= (1 << i);
        }
        for (int i = k - 1; i < n; i++) {
            dest |= (1 << i);
        }
        do {
            for (int i = 0; i < n; i++) {
                if ((mask >> i) & 1) {
                    cout << char_set[i];
                }
            }
            cout << " ";
            mask = next_combination_mask(mask);
        } while (mask != dest);
        cout << endl;
    }
}

```

```

{ // Combinations of distinct integers from 0 to n - 1.
  int n = 5, k = 3, a[] = {0, 1, 2};
  cout << "\n" << n << " choose " << k << ":" << endl;
  int count = 0;
  do {
    print_range(a, a + k);
    vector<int> b = combination_by_rank(n, k, count);
    assert(equal(a, a + k, b.begin()));
    assert(rank_by_combination(n, k, a) == count);
    count++;
  } while (next_combination(n, k, a));
  cout << endl;
}
{ // Combinations with repeats.
  int n = 3, k = 2, a[] = {0, 0};
  cout << "\n" << n << " multichoose " << k << ":" << endl;
  do {
    print_range(a, a + k);
  } while (next_combination_with_repeats(n, k, a));
  cout << endl;
}
return 0;
}

```

Example Output

```

"11234" choose 3:
112 113 114 123 124 134 234

"abcde" choose 3 with masks:
abc abd acd bcd abe ace bce ade bde

5 choose 3:
{0,1,2} {0,1,3} {0,1,4} {0,2,3} {0,2,4} {0,3,4} {1,2,3} {1,2,4} {1,3,4} {2,3,4}

3 multichoose 2:
{0,0} {0,1} {0,2} {1,1} {1,2} {2,2}

```

6.2.5 Enumerating Partitions

A partition of a natural number n is a way to write n as a sum of positive integers where the order of the addends does not matter.

- `next_partition(p)` takes a reference to a vector `p[]` of positive integers as a partition of n for which the function will re-assign to become the next lexicographically greater partition. The function returns true if such a partition exists, or false if `p[]` already consists of the lexicographically greatest partition (i.e. the single integer n).
- `partition_by_rank(n, r)` returns the partition of n that is lexicographically ranked r if addends in each partition were sorted in non-increasing order, where r is a zero-based rank in the range $[0, \text{partitions}(n))$.
- `rank_by_partition(p)` returns an integer representing the zero-based rank of the partition specified by vector `p[]`, which must consist of positive integers sorted in non-increasing order.
- `generate_increasing_partitions(n, f)` calls the function `f(lo, hi)` on strictly increasing partitions of n in lexicographically increasing order of partition, where `lo` and `hi` are random-access iterators to a range `[lo, hi)` of integers. Note that non-strictly increasing partitions like $\{1, 1, 1, 1\}$ are skipped.

Time Complexity:

- $O(n)$ per call to `next_partition()`.
- $O(n^2)$ per call to `partition_by_rank(n, r)` and `rank_by_partition(p)`.
- $O(p(n))$ per call to `generate_increasing_partitions(n, f)`, where $p(n)$ is the number of partitions of n .

Space Complexity:

- $O(1)$ auxiliary for `next_partition()`.
- $O(n^2)$ auxiliary heap space for `partition_function()`, `partition_by_rank()`, and `rank_by_partition()`.
- $O(n)$ auxiliary stack space for `generate_increasing_partitions()`.

```
#include <vector>

bool next_partition(std::vector<int> &p) {
    int n = p.size();
    if (n <= 1) {
        return false;
    }
    int s = p[n - 1] - 1, i = n - 2;
    p.pop_back();
    for (; i > 0 && p[i] == p[i - 1]; i--) {
        s += p[i];
        p.pop_back();
    }
    for (p[i]++; s > 0; s--) {
        p.push_back(1);
    }
    return true;
}

long long partition_function(int a, int b) {
    static std::vector<std::vector<long long>> p(1, std::vector<long long>(1, 1));
    if (a >= (int)p.size()) {
        int old = p.size();
        p.resize(a + 1);
        p[0].resize(a + 1);
        for (int i = 1; i <= a; i++) {
            p[i].resize(a + 1);
            for (int j = old; j <= i; j++) {
                p[i][j] = p[i - 1][j - 1] + p[i - j][j];
            }
        }
    }
    return p[a][b];
}

std::vector<int> partition_by_rank(int n, long long r) {
    std::vector<int> res;
    for (int i = n, j; i > 0; i -= j) {
        for (j = 1;; j++) {
            long long count = partition_function(i, j);
            if (r < count) {
                break;
            }
            r -= count;
        }
        res.push_back(j);
    }
    return res;
}

long long rank_by_partition(const std::vector<int> &p) {
    long long res = 0;
    int sum = 0;
    for (int i = 0; i < (int)p.size(); i++) {
        sum += p[i];
    }
    for (int i = 0; i < (int)p.size(); i++) {
        for (int j = 0; j < p[i]; j++) {
            res += partition_function(sum, j);
        }
        sum -= p[i];
    }
    return res;
}
```

```

}

typedef void (*ReportFunction)(std::vector<int>::iterator, std::vector<int>::iterator);

void generate_increasing_partitions(
    int left, int prev, int i, std::vector<int> &p, ReportFunction f
) {
    if (left == 0) {
        f(p.begin(), p.begin() + i);
        return;
    }
    for (p[i] = prev + 1; p[i] <= left; p[i]++) {
        generate_increasing_partitions(left - p[i], p[i], i + 1, p, f);
    }
}

void generate_increasing_partitions(int n, ReportFunction f) {
    std::vector<int> p(n, 0);
    generate_increasing_partitions(n, 0, 0, p, f);
}

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

template<class It>
void print_range(It lo, It hi) {
    cout << "{";
    for (; lo != hi; ++lo) {
        cout << *lo << (lo == hi - 1 ? "" : ",");
    }
    cout << "} ";
}

int main() {
    {
        int n = 4;
        vector<int> a(n, 1);
        cout << "Partitions of " << n << ":" << endl;
        int count = 0;
        do {
            print_range(a.begin(), a.end());
            vector<int> b = partition_by_rank(n, count);
            assert(equal(a.begin(), a.end(), b.begin()));
            assert(rank_by_partition(a) == count);
            count++;
        } while (next_partition(a));
        cout << endl;
    }
    {
        int n = 8;
        cout << "\nIncreasing partitions of " << n << ":" << endl;
        generate_increasing_partitions(n, print_range);
        cout << endl;
    }
    return 0;
}

```

Example Output

```

Partitions of 4:
{1,1,1,1} {2,1,1} {2,2} {3,1} {4}

```

```
Increasing partitions of 8:
{1,2,5} {1,3,4} {1,7} {2,6} {3,5} {8}
```

6.2.6 Enumerating Generic Combinatorial Sequences

Enumerate combinatorial sequences by inheriting an abstract class. Child classes of `abstract_enumerator` must implement the `count()` function which should return the number of combinatorial sequences starting with the given prefix.

- `to_rank(a)` returns an integer representing the zero-based rank of the combinatorial sequence `a`.
- `from_rank(r)` returns a combinatorial sequence of integers that is lexicographically ranked `r`, where `r` is a zero-based rank in the range `[0, total_count())`.
- `enumerate(f)` calls the function `f(lo, hi)` on every specified combinatorial sequence in lexicographically increasing order, where `lo` and `hi` are two random-access iterators to a range `[lo, hi)` of integers.

Time Complexity: $O(n^2)$ calls will be made to `count()` per call to all operations, where n is the length of the combinatorial sequence.

Space Complexity: $O(n)$ auxiliary heap space per call to all operations.

```
#include <vector>

class abstract_enumerator {
protected:
    int range, length;

    abstract_enumerator(int r, int l) : range(r), length(l) {}
    virtual long long count(const std::vector<int> &prefix) { return 0; }
    std::vector<int> next(std::vector<int> &a) { return from_rank(to_rank(a) + 1); }
    long long total_count() { return count(std::vector<int>(0)); }

public:
    long long to_rank(const std::vector<int> &a) {
        long long res = 0;
        for (int i = 0; i < (int)a.size(); i++) {
            std::vector<int> prefix(a.begin(), a.end());
            prefix.resize(i + 1);
            for (prefix[i] = 0; prefix[i] < a[i]; prefix[i]++) {
                res += count(prefix);
            }
        }
        return res;
    }

    std::vector<int> from_rank(long long r) {
        std::vector<int> a(length);
        for (int i = 0; i < (int)a.size(); i++) {
            std::vector<int> prefix(a.begin(), a.end());
            prefix.resize(i + 1);
            for (prefix[i] = 0; prefix[i] < range; ++prefix[i]) {
                long long curr = count(prefix);
                if (r < curr) {
                    break;
                }
                r -= curr;
            }
            a[i] = prefix[i];
        }
        return a;
    }

    void enumerate(void (*f)(std::vector<int>::iterator, std::vector<int>::iterator)) {
        long long total = total_count();
        for (long long i = 0; i < total; i++) {
            std::vector<int> curr = from_rank(i);
```



```

        f(curr.begin(), curr.end());
    }
}
};

class arrangement_enumerator : public abstract_enumerator {
public:
    arrangement_enumerator(int n, int k) : abstract_enumerator(n, k) {}

    long long count(const std::vector<int> &prefix) {
        int n = prefix.size();
        for (int i = 0; i < n - 1; i++) {
            if (prefix[i] == prefix[n - 1]) {
                return 0;
            }
        }
        long long res = 1;
        for (int i = 0; i < length - n; i++) {
            res *= range - n - i;
        }
        return res;
    }
};

class permutation_enumerator : public arrangement_enumerator {
public:
    permutation_enumerator(int n) : arrangement_enumerator(n, n) {}
};

class combination_enumerator : public abstract_enumerator {
    std::vector<std::vector<long long>> table;

public:
    combination_enumerator(int n, int k)
        : abstract_enumerator(n, k), table(n + 1, std::vector<long long>(n + 1)) {
        for (int i = 0; i <= n; i++) {
            for (int j = 0; j <= i; j++) {
                table[i][j] = (j == 0) ? 1 : table[i - 1][j - 1] + table[i - 1][j];
            }
        }
    }

    long long count(const std::vector<int> &prefix) {
        int n = prefix.size();
        if (n >= 2 && prefix[n - 1] <= prefix[n - 2]) {
            return 0;
        }
        if (n == 0) {
            return table[range][length - n];
        }
        return table[range - prefix[n - 1] - 1][length - n];
    }
};

class partition_enumerator : public abstract_enumerator {
    std::vector<std::vector<long long>> table;

public:
    partition_enumerator(int n)
        : abstract_enumerator(n + 1, n), table(n + 1, std::vector<long long>(n + 1)) {
        std::vector<std::vector<long long>> tmp(table);
        tmp[0][0] = 1;
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                tmp[i][j] = tmp[i - 1][j - 1] + tmp[i - j][j];
            }
        }
        for (int i = 1; i <= n; i++) {

```

```

        for (int j = 1; j <= n; j++) {
            table[i][j] = tmp[i][j] + table[i][j - 1];
        }
    }

long long count(const std::vector<int> &prefix) {
    int n = (int)prefix.size(), sum = 0;
    for (int i = 0; i < n; i++) {
        sum += prefix[i];
    }
    if (sum == range - 1) {
        return 1;
    }
    if (sum > range - 1 || (n > 0 && prefix[n - 1] == 0) ||
        (n >= 2 && prefix[n - 1] > prefix[n - 2])) {
        return 0;
    }
    if (n == 0) {
        return table[range - sum - 1][range - 1];
    }
    return table[range - sum - 1][prefix[n - 1]];
}
};

```

Example Usage

```

#include <iostream>
using namespace std;

template<class It>
void print_range(It lo, It hi) {
    cout << "{";
    for (; lo != hi; ++lo) {
        cout << *lo << (lo == hi - 1 ? "" : ",");
    }
    cout << "} ";
}

int main() {
    {
        cout << "3 permute 2 arrangement_enumerator:" << endl;
        arrangement_enumerator arr(3, 2);
        arr.enumerate(print_range);
        cout << endl;
    }
    {
        cout << "\nPermutatations of [0, 3):" << endl;
        permutation_enumerator perm(3);
        perm.enumerate(print_range);
        cout << endl;
    }
    {
        cout << "\n4 choose 3 combinations:" << endl;
        combination_enumerator comb(4, 3);
        comb.enumerate(print_range);
        cout << endl;
    }
    {
        cout << "\nPartition of 4:" << endl;
        partition_enumerator part(4);
        part.enumerate(print_range);
        cout << endl;
    }
    return 0;
}

```

Example Output

```

3 permute 2 arrangements:
{0,1} {0,2} {1,0} {1,2} {2,0} {2,1}

Permutatations of [0, 3):
{0,1,2} {0,2,1} {1,0,2} {1,2,0} {2,0,1} {2,1,0}

4 choose 3 combinations:
{0,1,2} {0,1,3} {0,2,3} {1,2,3}

Partition of 4:
{1,1,1,1} {2,1,1,0} {2,2,0,0} {3,1,0,0} {4,0,0,0}

```

6.3 Number Theory

6.3.1 GCD, LCM, Mod Inverse, Chinese Remainder

Common number theory operations relating to modular arithmetic.

- `gcd(a, b)` and `gcd2(a, b)` both return the greatest common division of `a` and `b` using the Euclidean algorithm.
- `lcm(a, b)` returns the lowest common multiple of `a` and `b`.
- `extended_euclid(a, b)` and `extended_euclid2(a, b)` both return a pair (x, y) of integers such that $\text{gcd}(a, b) = ax + by$.
- `mod(a, b)` returns the value of `a mod b` under the true Euclidean definition of modulo, that is, the smallest nonnegative integer m satisfying $a + bn = m$ for some integer n . Note that this is identical to the remainder operator `%` in C++ for nonnegative operands `a` and `b`, but the result will differ when an operand is negative.
- `mod_inverse(a, m)` and `mod_inverse2(a, m)` both return an integer x such that $ax \equiv 1 \pmod{m}$, where the arguments must satisfy $m > 0$ and $\text{gcd}(a, m) = 1$.
- `generate_inverse(p)` returns a vector v of integers where for each index i in the vector, $i \cdot v[i] \equiv 1 \pmod{p}$, where the argument p is prime.
- `simple_restore(n, a, p)` and `garner_restore(n, a, p)` both return the solution x for the system of simultaneous congruences $x \equiv a[i] \pmod{p[i]}$ for all indices i in $[0, n)$, where `p[]` consists of pairwise coprime integers. The solution x is guaranteed to be unique by the Chinese remainder theorem.

Time Complexity:

- $O(\log(a + b))$ per call to `gcd(a, b)`, `gcd2(a, b)`, `lcm(a, b)`, `extended_euclid(a, b)`, `extended_euclid2(a, b)`, `mod_inverse(a, b)`, and `mod_inverse2(a, b)`.
- $O(1)$ for `mod(a, b)`.
- $O(p)$ for `generate_inverse(p)`.
- Exponential for `simple_restore(n, a, p)`.
- $O(n^2)$ for `garner_restore(n, a, p)`.

Space Complexity:

- $O(\log(a + b))$ auxiliary stack space for `gcd2(a, b)`, `extended_euclid2(a, b)`, and `mod_inverse2(a, b)`.
- $O(p)$ auxiliary heap space for `generate_inverse(p)`.
- $O(n)$ auxiliary heap space for `garner_restore(n, a, p)`.
- $O(1)$ auxiliary space for all other operations.

```

#include <cstdlib>
#include <utility>
#include <vector>

template<class Int>
Int gcd(Int a, Int b) {

```

```

    while (b != 0) {
        Int t = b;
        b = a % b;
        a = t;
    }
    return abs(a);
}

template<class Int>
Int gcd2(Int a, Int b) {
    return (b == 0) ? abs(a) : gcd(b, a % b);
}

template<class Int>
Int lcm(Int a, Int b) {
    return abs(a / gcd(a, b) * b);
}

template<class Int>
std::pair<Int, Int> extended_euclid(Int a, Int b) {
    Int x = 1, y = 0, x1 = 0, y1 = 1;
    while (b != 0) {
        Int q = a / b, prev_x1 = x1, prev_y1 = y1, prev_b = b;
        x1 = x - q * x1;
        y1 = y - q * y1;
        b = a - q * b;
        x = prev_x1;
        y = prev_y1;
        a = prev_b;
    }
    return (a > 0) ? std::make_pair(x, y) : std::make_pair(-x, -y);
}

template<class Int>
std::pair<Int, Int> extended_euclid2(Int a, Int b) {
    if (b == 0) {
        return (a > 0) ? std::make_pair(1, 0) : std::make_pair(-1, 0);
    }
    std::pair<Int, Int> r = extended_euclid2(b, a % b);
    return std::make_pair(r.second, r.first - a / b * r.second);
}

template<class Int>
Int mod(Int a, Int m) {
    Int r = a % m;
    return (r >= 0) ? r : (r + m);
}

template<class Int>
Int mod_inverse(Int a, Int m) {
    a = mod(a, m);
    return (a == 0) ? 0 : mod((1 - m * mod_inverse(m % a, a)) / a, m);
}

template<class Int>
Int mod_inverse2(Int a, Int m) {
    return mod(extended_euclid(a, m).first, m);
}

std::vector<int> generate_inverse(int p) {
    std::vector<int> res(p);
    res[1] = 1;
    for (int i = 2; i < p; i++) {
        res[i] = (p - (p / i) * res[p % i] % p) % p;
    }
    return res;
}

```

```

long long simple_restore(int n, int a[], int p[]) {
    long long res = 0, m = 1;
    for (int i = 0; i < n; i++) {
        while (res % p[i] != a[i]) {
            res += m;
        }
        m *= p[i];
    }
    return res;
}

long long garner_restore(int n, int a[], int p[]) {
    if (n == 0) {
        return 0;
    }
    std::vector<int> x(a, a + n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            x[i] = mod_inverse((long long)p[j], (long long)p[i]) * (x[i] - x[j]);
        }
        x[i] = (x[i] % p[i] + p[i]) % p[i];
    }
    long long res = x[0], m = 1;
    for (int i = 1; i < n; i++) {
        m *= p[i - 1];
        res += x[i] * m;
    }
    return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    {
        for (int steps = 0; steps < 10000; steps++) {
            int a = rand() % 200 - 10, b = rand() % 200 - 10, g = gcd(a, b);
            assert(g == gcd2(a, b));
            if (g == 1 && b > 1) {
                int inv = mod_inverse(a, b);
                assert(inv == mod_inverse2(a, b) && mod(a * inv, b) == 1);
            }
            pair<int, int> res = extended_euclid(a, b);
            assert(res == extended_euclid2(a, b));
            assert(g == a * res.first + b * res.second);
        }
    }
    {
        int p = 17;
        std::vector<int> res = generate_inverse(p);
        for (int i = 0; i < p; i++) {
            if (i > 0) {
                assert(mod(i * res[i], p) == 1);
            }
        }
    }
    {
        int n = 3, a[] = {2, 3, 1}, m[] = {3, 4, 5};
        int x = simple_restore(n, a, m);
        assert(x == garner_restore(n, a, m));
        for (int i = 0; i < n; i++) {
            assert(mod(x, m[i]) == a[i]);
        }
        assert(x == 11);
    }
}

```

```

    }
    return 0;
}

```

6.3.2 Modular Arithmetic

Wraps arithmetic modulo a compile-time constant in a small value type. This is a common contest helper for dynamic programming, combinatorics, polynomial operations, and any calculation where all answers are taken modulo a number such as `1000000007`.

The implementation is intentionally close to common contest "Mint" templates: normalization happens at construction, arithmetic operators are overloaded, mixed integer operations are supported through implicit construction, and combinations can be computed from lazy factorial tables.

Division uses the extended Euclidean algorithm, so the divisor only needs to be coprime to the modulus. For the factorial-table combination helper, the usual contest assumption is that `MOD` is prime and the requested factorials are invertible modulo `MOD`.

- `modular<T>(x)` constructs the residue class of integer `x` modulo `T::value`.
- `value()` and `operator()()` return the stored representative in `[0, MOD)`.
- `pow(e)` returns this value raised to nonnegative exponent `e`.
- `inv()` returns the multiplicative inverse, asserting it exists.
- Operators `+`, `-`, `*`, `/`, comparison, increment, decrement, and stream I/O are overloaded.
- `mod_combinatorics<Mint>::choose(n, k)` returns $\binom{n}{k}$ using lazy factorial and inverse-factorial tables.

Time Complexity:

- $O(1)$ per addition, subtraction, multiplication, comparison, and stream output.
- $O(\log e)$ per call to `pow(e)`.
- $O(\log \text{MOD})$ per call to `inv()` and division.
- $O(n)$ total table growth to answer combinations up to size `n`.

Space Complexity:

- $O(1)$ auxiliary for modular arithmetic.
- $O(n)$ for factorial and inverse-factorial tables grown through `choose(n, k)`.

```

#include <cassert>
#include <iostream>
#include <vector>

template<class T>
T inverse(T a, T m) {
    T original_m = m;
    T u = 0, v = 1;
    while (a != 0) {
        T q = m / a;
        m -= q * a;
        T tmp = a;
        a = m;
        m = tmp;
        u -= q * v;
        tmp = v;
        v = u;
        u = tmp;
    }
    assert(m == 1);
    u %= original_m;
    return u < 0 ? u + original_m : u;
}

template<class T>

```

```

class modular {
public:
    typedef typename T::value_type value_type;

private:
    value_type v;

public:
    modular(long long x = 0) { v = normalize(x); }

    static value_type mod() { return T::value; }

    static value_type normalize(long long x) {
        if (-(long long)mod() <= x && x < (long long)mod()) {
            value_type y = (value_type)x;
            return y < 0 ? y + mod() : y;
        }
        x %= mod();
        if (x < 0) {
            x += mod();
        }
        return (value_type)x;
    }

    value_type value() const { return v; }
    value_type operator()() const { return v; }

    modular pow(long long e) const {
        assert(e >= 0);
        modular base = *this, res = 1;
        while (e > 0) {
            if (e & 1) {
                res *= base;
            }
            base *= base;
            e >>= 1;
        }
        return res;
    }

    modular inv() const {
        assert(v != 0);
        return modular(inverse((long long)v, (long long)mod()));
    }

    modular &operator+=(const modular &other) {
        v += other.v;
        if (v >= mod()) {
            v -= mod();
        }
        return *this;
    }

    modular &operator-=(const modular &other) {
        v -= other.v;
        if (v < 0) {
            v += mod();
        }
        return *this;
    }

    modular &operator*=(const modular &other) {
        v = normalize((long long)v * other.v);
        return *this;
    }

    modular operator-() const { return modular(-v); }
    modular &operator++() { return *this += 1; }

```

```

modular &operator--() { return *this -= 1; }
modular &operator/=(const modular &other) { return *this *= other.inv(); }

modular operator++(int) {
    modular res = *this;
    ++*this;
    return res;
}

modular operator--(int) {
    modular res = *this;
    --*this;
    return res;
}

friend modular operator+(modular a, const modular &b) { return a += b; }

friend modular operator-(modular a, const modular &b) { return a -= b; }

friend modular operator*(modular a, const modular &b) { return a *= b; }

friend modular operator/(modular a, const modular &b) { return a /= b; }

friend bool operator==(const modular &a, const modular &b) { return a.v == b.v; }

friend bool operator!=(const modular &a, const modular &b) { return !(a == b); }

friend bool operator<(const modular &a, const modular &b) { return a.v < b.v; }

friend std::ostream &operator<<(std::ostream &os, const modular &x) { return os << x.v; }

friend std::istream &operator>>(std::istream &is, modular &x) {
    long long value;
    is >> value;
    x = modular(value);
    return is;
}
};

template<class Mint>
class mod_combinatorics {
    std::vector<Mint> fact, inv_fact;

    void ensure(int n) {
        int old = fact.size();
        if (old > n) {
            return;
        }
        fact.resize(n + 1);
        inv_fact.resize(n + 1);
        for (int i = old; i <= n; i++) {
            fact[i] = fact[i - 1] * i;
        }
        inv_fact[n] = fact[n].inv();
        for (int i = n; i > old; i--) {
            inv_fact[i - 1] = inv_fact[i] * i;
        }
    }

public:
    mod_combinatorics() : fact(1, Mint(1)), inv_fact(1, Mint(1)) {}

    Mint factorial(int n) {
        ensure(n);
        return fact[n];
    }

    Mint choose(int n, int k) {

```



```

    if (k < 0 || k > n) {
        return 0;
    }
    ensure(n);
    return fact[n] * inv_fact[k] * inv_fact[n - k];
}

Mint permute(int n, int k) {
    if (k < 0 || k > n) {
        return 0;
    }
    ensure(n);
    return fact[n] * inv_fact[n - k];
}

Mint multichoose(int n, int k) { return choose(n + k - 1, k); }
};

struct mod_1000000007 {
    typedef int value_type;
    static const int value = 1000000007;
};

typedef modular<mod_1000000007> Mint;

```

Example Usage

```

#include <cassert>
#include <sstream>
using namespace std;

int main() {
    Mint a = 1000000008LL;
    Mint b = -2;
    assert(a.value() == 1);
    assert(b() == Mint::mod() - 2);
    assert(a + b == -1);
    assert(2 + Mint(3) == 5);
    assert(Mint(2) * 3 == 6);
    assert(Mint(2).pow(10) == 1024);
    assert(Mint(5) / 5 == 1);

    Mint x = 0;
    x++;
    ++x;
    assert(x == 2);
    stringstream ss("1000000008");
    ss >> x;
    assert(x == 1);

    mod_combinatorics<Mint> comb;
    assert(comb.factorial(5) == 120);
    assert(comb.choose(5, 2) == 10);
    assert(comb.permute(5, 2) == 20);
    assert(comb.multichoose(3, 2) == 6);
    return 0;
}

```

6.3.3 Prime Generation

Generate prime numbers using the Sieve of Eratosthenes.

- `sieve(n)` returns a vector of all the primes less than or equal to `n`.

- `sieve(lo, hi)` returns a vector of all the primes in the range `[lo, hi]`.

Time Complexity:

- $O(n \log(\log(n)))$ per call to `sieve(n)`.
- $O(\sqrt{hi} \cdot \log(\log(hi - lo)))$ per call to `sieve(lo, hi)`.

Space Complexity:

- $O(n)$ auxiliary heap space per call to `sieve(n)`.
- $O(hi - lo + \sqrt{hi})$ auxiliary heap space per call to `sieve(lo, hi)`.

```
#include <cmath>
#include <vector>

std::vector<int> sieve(int n) {
    std::vector<bool> prime(n + 1, true);
    int sqrtn = ceil(sqrt(n));
    for (int i = 2; i <= sqrtn; i++) {
        if (prime[i]) {
            for (int j = i * i; j <= n; j += i) {
                prime[j] = false;
            }
        }
    }
    std::vector<int> res;
    for (int i = 2; i <= n; i++) {
        if (prime[i]) {
            res.push_back(i);
        }
    }
    return res;
}

std::vector<int> sieve(int lo, int hi) {
    int sqrt_hi = ceil(sqrt(hi)), fourth_root_hi = ceil(sqrt(sqrt_hi));
    std::vector<bool> prime1(sqrt_hi + 1, true), prime2(hi - lo + 1, true);
    for (int i = 2; i <= fourth_root_hi; i++) {
        if (prime1[i]) {
            for (int j = i * i; j <= sqrt_hi; j += i) {
                prime1[j] = false;
            }
        }
    }
    for (int i = 2, n = hi - lo; i <= sqrt_hi; i++) {
        if (prime1[i]) {
            for (int j = (lo / i) * i - lo; j <= n; j += i) {
                if (j >= 0 && j + lo != i) {
                    prime2[j] = false;
                }
            }
        }
    }
    std::vector<int> res;
    for (int i = (lo > 1) ? lo : 2; i <= hi; i++) {
        if (prime2[i - lo]) {
            res.push_back(i);
        }
    }
    return res;
}
```

Example Usage

```
#include <ctime>
#include <iostream>
```

```
using namespace std;

int main() {
    int pmax = 10000000;
    vector<int> p;
    time_t start;
    double delta;

    start = clock();
    p = sieve(pmax);
    delta = (double)(clock() - start) / CLOCKS_PER_SEC;
    cout << "sieve(n=" << pmax << "): " << delta << "s" << endl;

    int l = 1000000000, h = 1005000000;
    start = clock();
    p = sieve(l, h);
    delta = (double)(clock() - start) / CLOCKS_PER_SEC;
    cout << "sieve([" << l << ", " << h << "]): " << delta << "s" << endl;
    return 0;
}
```

Example Output

```
sieve(n=10000000): 0.059s
atkins(n=10000000): 0.08s
sieve([1000000000, 1005000000]): 0.034s
```

6.3.4 Primality Testing

Determine whether an integer n is prime. This can be done deterministically by testing all numbers under $\sqrt{n}$ using trial division, probabilistically using the Miller-Rabin test, or deterministically using the Miller-Rabin test if the maximum input is known ($2^{63} - 1$ for the purposes here).

- `is_prime(n)` returns whether the integer `n` is prime using an optimized trial division technique based on the fact that all primes greater than 6 must take the form $6n + 1$ or $6n - 1$.
- `is_probable_prime(n, k)` returns true if the integer `n` is prime, or false with an error probability of $(1/4)^k$ if `n` is composite. In other words, the result is guaranteed to be correct if `n` is prime, but could be wrong with probability $(1/4)^k$ if `n` is composite. This implementation uses exponentiation by squaring to support all signed 64-bit integers (up to and including $2^{63} - 1$).
- `is_prime_fast(n)` returns whether the signed 64-bit integer `n` is prime using a fully deterministic version of the Miller-Rabin test.

Time Complexity:

- $O(\sqrt{n})$ per call to `is_prime(n)`.
- $O(k \log^3(n))$ per call to `is_probable_prime(n, k)`.
- $O(\log^3(n))$ per call to `is_prime_fast(n)`.

Space Complexity: $O(1)$ auxiliary space for all operations.

```
#include <cstdlib>

template<class Int>
bool is_prime(Int n) {
    if (n == 2 || n == 3) {
        return true;
    }
    if (n < 2 || n % 2 == 0 || n % 3 == 0) {
        return false;
    }
    for (Int i = 5, w = 4; i * i <= n; i += w) {
        if (n % i == 0) {
            return false;
        }
    }
    return true;
}
```

```

    }
    w = 6 - w;
}
return true;
}

typedef unsigned long long uint64;

uint64 mulmod(uint64 x, uint64 n, uint64 m) {
    uint64 a = 0, b = x % m;
    for (; n > 0; n >>= 1) {
        if (n & 1) {
            a = (a + b) % m;
        }
        b = (b << 1) % m;
    }
    return a % m;
}

uint64 powmod(uint64 x, uint64 n, uint64 m) {
    uint64 a = 1, b = x;
    for (; n > 0; n >>= 1) {
        if (n & 1) {
            a = mulmod(a, b, m);
        }
        b = mulmod(b, b, m);
    }
    return a % m;
}

uint64 rand64u() {
    return ((uint64)(rand() & 0xf) << 60) | ((uint64)(rand() & 0x7fff) << 45) |
        ((uint64)(rand() & 0x7fff) << 30) | ((uint64)(rand() & 0x7fff) << 15) |
        ((uint64)(rand() & 0x7fff));
}

bool is_probable_prime(long long n, int k = 20) {
    if (n == 2 || n == 3) {
        return true;
    }
    if (n < 2 || n % 2 == 0 || n % 3 == 0) {
        return false;
    }
    uint64 s = n - 1, p = n - 1;
    while (!(s & 1)) {
        s >>= 1;
    }
    for (int i = 0; i < k; i++) {
        uint64 x, r = powmod(rand64u() % p + 1, s, n);
        for (x = s; x != p && r != 1 && r != p; x <<= 1) {
            r = mulmod(r, r, n);
        }
        if (r != p && !(x & 1)) {
            return false;
        }
    }
    return true;
}

bool is_prime_fast(long long n) {
    static const int np = 9, p[] = {2, 3, 5, 7, 11, 13, 17, 19, 23};
    for (int i = 0; i < np; i++) {
        if (n % p[i] == 0) {
            return n == p[i];
        }
    }
    if (n < p[np - 1]) {
        return false;
    }

```

```

}
uint64 t;
int s = 0;
for (t = n - 1; !(t & 1); t >>= 1) {
    s++;
}
for (int i = 0; i < np; i++) {
    uint64 r = powmod(p[i], t, n);
    if (r == 1) {
        continue;
    }
    bool ok = false;
    for (int j = 0; j < s && !ok; j++) {
        ok |= (r == (uint64)n - 1);
        r = mulmod(r, r, n);
    }
    if (!ok) {
        return false;
    }
}
return true;
}

```

Example Usage

```

#include <cassert>

int main() {
    int len = 20;
    long long tests[] = {
        -1,
        0,
        1,
        2,
        3,
        4,
        5,
        1000000LL,
        772023803LL,
        792904103LL,
        813815117LL,
        834753187LL,
        855718739LL,
        876717799LL,
        897746119LL,
        2147483647LL,
        5705234089LL,
        5914686649LL,
        6114145249LL,
        6339503641LL,
        6548531929LL
    };
    for (int i = 0; i < len; i++) {
        bool p = is_prime(tests[i]);
        assert(p == is_prime_fast(tests[i]));
        assert(p == is_probable_prime(tests[i]));
    }
    return 0;
}

```

6.3.5 Integer Factorization

Compute the prime factorization of an integer. In the following implementations, the prime factorization of n is represented as a sorted vector of prime integers which together multiply to n . Note that factors are duplicated in the vector in accordance to their multiplicity in the prime factorization of n . For 0, 1, and prime numbers, the prime factorization is considered to be a vector consisting of a single element - the input itself.

- `prime_factorize(n)` returns the prime factorization of `n` using trial division.
- `get_divisors(n)` returns a sorted vector of all (not merely prime) divisors of `n` using trial division.
- `fermat(n)` returns a factor of `n` (possibly 1 or itself) that is not necessarily prime. This algorithm is efficient for integers with two factors near $\sqrt{n}$, but is roughly as slow as trial division otherwise.
- `pollards_rho_brent(n)` returns a factor of `n` that is not necessarily prime using Pollard's rho algorithm with Brent's optimization. If `n` is prime, then `n` itself is returned. While this algorithm is non-deterministic and may fail to detect factors on certain runs of the same input, it can be placed in a loop to deterministically factor large integers, as done in `prime_factorize_big()`.
- `prime_factorize_big(n, trial_division_cutoff)` returns the prime factorization of a 64-bit integer `n` using a combination of trial division, the Miller-Rabin primality test, and Pollard's rho algorithm. `trial_division_cutoff` specifies the largest factor to test with trial division before falling back to the rho algorithm. This supports 64-bit integers up to and including $2^{63} - 1$.

Time Complexity:

- $O(\sqrt{n})$ per call to `prime_factorize(n)`, `get_divisors(n)`, and `fermat(n)`.
- Unknown, but approximately $O(n^{1/4})$ per call to `pollards_rho_brent(n)` and `prime_factorize_big(n)`.

Space Complexity: $O(f)$ auxiliary heap space for all operations, where f is the number of factors returned.

```
#include <algorithm>
#include <cmath>
#include <cstdlib>
#include <vector>

template<class Int>
std::vector<Int> prime_factorize(Int n) {
    if (n <= 3) {
        return std::vector<Int>(1, n);
    }
    std::vector<Int> res;
    for (Int i = 2;; i++) {
        int p = 0, q = n / i, r = n - q * i;
        if (i > q || (i == q && r > 0)) {
            break;
        }
        while (r == 0) {
            p++;
            n = q;
            q = n / i;
            r = n - q * i;
        }
        for (int j = 0; j < p; j++) {
            res.push_back(i);
        }
    }
    if (n > 1) {
        res.push_back(n);
    }
    return res;
}

template<class Int>
std::vector<Int> get_divisors(Int n) {
    if (n <= 1) {
        return (n < 1) ? std::vector<Int>() : std::vector<Int>(1, 1);
    }
}
```

```

std::vector<Int> res;
for (Int i = 1; i * i <= n; i++) {
    if (n % i == 0) {
        res.push_back(i);
        if (i * i != n) {
            res.push_back(n / i);
        }
    }
}
std::sort(res.begin(), res.end());
return res;
}

long long fermat(long long n) {
    if (n % 2 == 0) {
        return 2;
    }
    long long x = sqrt(n), y = 0, r = x * x - y * y - n;
    while (r != 0) {
        if (r < 0) {
            r += x + x + 1;
            x++;
        } else {
            r -= y + y + 1;
            y++;
        }
    }
    return (x == y) ? (x + y) : (x - y);
}

typedef unsigned long long uint64;

uint64 mulmod(uint64 x, uint64 n, uint64 m) {
    uint64 a = 0, b = x % m;
    for (; n > 0; n >= 1) {
        if (n & 1) {
            a = (a + b) % m;
        }
        b = (b << 1) % m;
    }
    return a % m;
}

uint64 powmod(uint64 x, uint64 n, uint64 m) {
    uint64 a = 1, b = x;
    for (; n > 0; n >= 1) {
        if (n & 1) {
            a = mulmod(a, b, m);
        }
        b = mulmod(b, b, m);
    }
    return a % m;
}

uint64 rand64u() {
    return ((uint64)(rand() & 0xf) << 60) | ((uint64)(rand() & 0x7fff) << 45) |
        ((uint64)(rand() & 0x7fff) << 30) | ((uint64)(rand() & 0x7fff) << 15) |
        ((uint64)(rand() & 0x7fff));
}

uint64 gcd(uint64 a, uint64 b) {
    while (b != 0) {
        uint64 t = b;
        b = a % b;
        a = t;
    }
    return a;
}

```

```

long long pollards_rho_brent(long long n) {
    if (n % 2 == 0) {
        return 2;
    }
    uint64 y = rand64u() % (n - 1) + 1;
    uint64 c = rand64u() % (n - 1) + 1;
    uint64 m = rand64u() % (n - 1) + 1;
    uint64 g = 1, r = 1, q = 1, ys = 0, x = 0;
    for (r = 1; g == 1; r <= 1) {
        x = y;
        for (int i = 0; i < r; i++) {
            y = (mulmod(y, y, n) + c) % n;
        }
        for (long long k = 0; k < r && g == 1; k += m) {
            ys = y;
            long long lim = std::min(m, r - k);
            for (int j = 0; j < lim; j++) {
                y = (mulmod(y, y, n) + c) % n;
                q = mulmod(q, (x > y) ? (x - y) : (y - x), n);
            }
            g = gcd(q, n);
        }
    }
    if (g == n) {
        do {
            ys = (mulmod(ys, ys, n) + c) % n;
            g = gcd((x > ys) ? (x - ys) : (ys - x), n);
        } while (g <= 1);
    }
    return g;
}

bool is_prime(long long n) {
    static const int np = 9, p[] = {2, 3, 5, 7, 11, 13, 17, 19, 23};
    for (int i = 0; i < np; i++) {
        if (n % p[i] == 0) {
            return n == p[i];
        }
    }
    if (n < p[np - 1]) {
        return false;
    }
    uint64 t;
    int s = 0;
    for (t = n - 1; !(t & 1); t >>= 1) {
        s++;
    }
    for (int i = 0; i < np; i++) {
        uint64 r = powmod(p[i], t, n);
        if (r == 1) {
            continue;
        }
        bool ok = false;
        for (int j = 0; j < s && !ok; j++) {
            ok |= (r == (uint64)n - 1);
            r = mulmod(r, r, n);
        }
        if (!ok) {
            return false;
        }
    }
    return true;
}

std::vector<long long> prime_factorize_big(
    long long n, long long trial_division_cutoff = 1000000LL
) {

```



```

if (n <= 3) {
    return std::vector<long long>(1, n);
}
std::vector<long long> res;
for (; n % 2 == 0; n /= 2) {
    res.push_back(2);
}
for (; n % 3 == 0; n /= 3) {
    res.push_back(3);
}
for (int i = 5, w = 4; i <= trial_division_cutoff && i * i <= n; i += w) {
    for (; n % i == 0; n /= i) {
        res.push_back(i);
    }
    w = 6 - w;
}
for (long long p; n > trial_division_cutoff && !is_prime(n); n /= p) {
    do {
        p = pollards_rho_brent(n);
    } while (p == n);
    res.push_back(p);
}
if (n != 1) {
    res.push_back(n);
}
sort(res.begin(), res.end());
return res;
}

```

Example Usage

```

#include <cassert>
#include <set>
using namespace std;

void validate(long long n, const vector<long long> &factors) {
    if (n == 1 || is_prime(n)) {
        assert(factors == vector<long long>(1, n));
        return;
    }
    long long prod = 1;
    for (int i = 0; i < factors.size(); i++) {
        assert(is_prime(factors[i]));
        prod *= factors[i];
    }
    assert(prod == n);
}

int main() {
    { // Small tests.
        for (int i = 1; i <= 10000; i++) {
            vector<long long> v1 = prime_factorize((long long)i);
            vector<long long> v2 = prime_factorize_big(i);
            validate(i, v1);
            assert(v1 == v2);
            vector<int> d = get_divisors(i);
            set<int> s(d.begin(), d.end());
            assert(d.size() == s.size());
            for (int j = 1; j <= i; j++) {
                if (i % j == 0) {
                    assert(s.count(j));
                }
            }
        }
    }
    { // Fermat works best for numbers with two factors close to each other.

```

```

long long n = 1000003LL * 100000037;
assert(fermat(n) == 1000003);
}
{ // Large tests.
const int ntests = 7;
const long long tests[] = {
    3LL * 3 * 5 * 7 * 9949 * 9967 * 1000003,
    2LL * 1000003 * 1000000007,
    999961LL * 1000033,
    357267896789127671LL,
    2LL * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 3 * 3 * 3 * 3 * 5 * 5 * 5 * 7 * 7 * 11 * 13 * 17 * 19 * 23 * 29 *
    31 * 37,
    2LL * 2 * 2 * 2 * 2 * 2 * 2 * 3 * 3 * 3 * 3 * 5 * 5 * 7 * 7 * 35336848213,
    2LL * 2 * 2 * 2 * 2 * 2 * 2 * 3 * 3 * 3 * 3 * 5 * 5 * 7 * 7 * 186917 * 186947,
};
for (int i = 0; i < ntests; i++) {
    validate(tests[i], prime_factorize_big(tests[i]));
}
}
return 0;
}

```

6.3.6 Euler's Totient Function

Euler's totient function `phi(n)` returns the number of positive integers less than or equal to n that are relatively prime to n . That is, `phi(n)` is the number of integers k in the range $[1, n]$ for which $\gcd(n, k) = 1$. The computation of `phi(1..n)` can be performed simultaneously, as done so by `phi_table(n)` which returns a vector `v` such that `v[i]` stores `phi(i)` for i in the range $[0, n]$.

Time Complexity: $O(n \log(\log(n)))$ per call to `phi(n)` and `phi_table(n)`.

Space Complexity:

- $O(1)$ auxiliary space for `phi(n)`.
- $O(n)$ auxiliary heap space for `phi_table(n)`.

```

#include <vector>

int phi(int n) {
    int res = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0) {
                n /= i;
            }
            res -= res / i;
        }
    }
    if (n > 1) {
        res -= res / n;
    }
    return res;
}

std::vector<int> phi_table(int n) {
    std::vector<int> res(n + 1);
    for (int i = 0; i <= n; i++) {
        res[i] = i;
    }
    for (int i = 1; i <= n; i++) {
        for (int j = 2 * i; j <= n; j += i) {
            res[j] -= res[i];
        }
    }
}

```

```
    return res;
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    assert(phi(1) == 1);
    assert(phi(9) == 6);
    assert(phi(1234567) == 1224720);
    const int n = 1000;
    vector<int> v = phi_table(n);
    for (int i = 0; i <= n; i++) {
        assert(v[i] == phi(i));
    }
    return 0;
}
```

6.3.7 Binary Exponentiation

Given three unsigned 64-bit integers x , n , and m , `powmod()` returns x raised to the power of n (modulo m). `mulmod()` returns x multiplied by n (modulo m). Despite the fact that both functions use unsigned 64-bit integers for arguments and intermediate calculations, arguments x and n must not exceed $2^{63} - 1$ (the maximum value of a signed 64-bit integer) for the result to be correctly computed without overflow.

Binary exponentiation, also known as exponentiation by squaring, decomposes the exponentiation into a logarithmic number of multiplications while avoiding overflow. To further prevent overflow in the intermediate squaring computations, multiplication is performed using a similar principle of repeated addition.

Time Complexity: $O(\log n)$ per call to `mulmod()` and `powmod()`, where n is the second argument.

Space Complexity: $O(1)$ auxiliary.

```
typedef unsigned long long uint64;

uint64 mulmod(uint64 x, uint64 n, uint64 m) {
    uint64 a = 0, b = x % m;
    for (; n > 0; n >>= 1) {
        if (n & 1) {
            a = (a + b) % m;
        }
        b = (b << 1) % m;
    }
    return a % m;
}

uint64 powmod(uint64 x, uint64 n, uint64 m) {
    uint64 a = 1, b = x;
    for (; n > 0; n >>= 1) {
        if (n & 1) {
            a = mulmod(a, b, m);
        }
        b = mulmod(b, b, m);
    }
    return a % m;
}
```

Example Usage

```
#include <cassert>

int main() {
    assert(powmod(2, 10, 1000000007) == 1024);
    assert(powmod(2, 62, 1000000) == 387904);
    assert(powmod(10001, 10001, 100000) == 10001);
    return 0;
}
```

6.4 Arbitrary Precision Arithmetic

6.4.1 Big Integers (Simple)

Perform simple arithmetic operations on arbitrary precision big integers whose digits are internally represented as an `std::string` in little-endian order.

- `bigint(n)` constructs a big integer from a long `long` (default: 0).
- `bigint(s)` constructs a big integer from a string `s`, which must strictly consist of a sequence of numeric digits, optionally preceded by a minus sign.
- `str()` returns the string representation of the big integer.
- `comp(a, b)` returns -1 , 0 , or 1 depending on whether the big integers `a` and `b` compare less, equal, or greater, respectively.
- `add(a, b)` returns the sum of big integers `a` and `b`.
- `sub(a, b)` returns the difference of big integers `a` and `b`.
- `mul(a, b)` returns the product of big integers `a` and `b`.
- `div(a, b)` returns the quotient of big integers `a` and `b`.

Time Complexity:

- $O(n)$ per call to the constructor, `str()`, `comp()`, `add()`, and `sub()`, where n is total number of digits in the arguments and result for each operation.
- $O(n \cdot m)$ per call to `mul(a, b)` and `div(a, b)` where n is the number of digits in `a` and m is the number of digits in `b`.

Space Complexity:

- $O(n)$ for storage of the big integer, where n is the number of the digits.
- $O(n)$ auxiliary heap space for `str()`, `add()`, `sub()`, `mul()`, and `div()`, where n is the total number of digits in the arguments and result for each operation.

```
#include <algorithm>
#include <cctype>
#include <stdexcept>
#include <string>

class bigint {
    std::string digits;
    int sign;

    void normalize() {
        size_t pos = digits.find_last_not_of('0');
        if (pos != std::string::npos) {
            digits.erase(pos + 1);
        }
        if (digits.empty()) {
            digits = "0";
        }
        if (digits.size() == 1 && digits[0] == '0') {
            sign = 1;
        }
    }
};
```

```

    return;
}
}

static int comp(const std::string &a, const std::string &b, int asign, int bsign) {
    if (asign != bsign) {
        return asign < bsign ? -1 : 1;
    }
    if (a.size() != b.size()) {
        return a.size() < b.size() ? -asign : asign;
    }
    for (int i = (int)a.size() - 1; i >= 0; i--) {
        if (a[i] != b[i]) {
            return a[i] < b[i] ? -asign : asign;
        }
    }
    return 0;
}

static bigint add(const std::string &a, const std::string &b, int asign, int bsign) {
    if (asign != bsign) {
        return (asign == 1) ? sub(a, b, asign, 1) : sub(b, a, bsign, 1);
    }
    bigint res;
    res.sign = asign;
    res.digits.resize(std::max(a.size(), b.size()) + 1, '0');
    for (int i = 0, carry = 0; i < (int)res.digits.size(); i++) {
        int d = carry;
        if (i < (int)a.size()) {
            d += a[i] - '0';
        }
        if (i < (int)b.size()) {
            d += b[i] - '0';
        }
        res.digits[i] = '0' + (d % 10);
        carry = d / 10;
    }
    res.normalize();
    return res;
}

static bigint sub(const std::string &a, const std::string &b, int asign, int bsign) {
    if (asign == -1 || bsign == -1) {
        return add(a, b, asign, -bsign);
    }
    bigint res;
    if (comp(a, b, asign, bsign) < 0) {
        res = sub(b, a, bsign, asign);
        res.sign = -1;
        return res;
    }
    res.digits.assign(a.size(), '0');
    for (int i = 0, borrow = 0; i < (int)res.digits.size(); i++) {
        int d = (i < (int)b.size() ? a[i] - b[i] : a[i] - '0') - borrow;
        if (a[i] > '0') {
            borrow = 0;
        }
        if (d < 0) {
            d += 10;
            borrow = 1;
        }
        res.digits[i] = '0' + (d % 10);
    }
    res.normalize();
    return res;
}

public:

```

```

bigint(long long n = 0) {
    sign = (n < 0) ? -1 : 1;
    if (n == 0) {
        digits = "0";
        return;
    }
    for (n = (n > 0) ? n : -n; n > 0; n /= 10) {
        digits += '0' + (n % 10);
    }
    normalize();
}

bigint(const std::string &s) {
    if (s.empty() || (s[0] == '-' && s.size() == 1)) {
        throw std::runtime_error("Invalid string format to construct bigint.");
    }
    digits.assign(s.rbegin(), s.rend());
    if (s[0] == '-') {
        sign = -1;
        digits.erase(digits.size() - 1);
    } else {
        sign = 1;
    }
    if (digits.find_first_not_of("0123456789") != std::string::npos) {
        throw std::runtime_error("Invalid string format to construct bigint.");
    }
    normalize();
}

std::string to_string() const {
    return (sign < 0 ? "-" : "") + std::string(digits.rbegin(), digits.rend());
}

friend int comp(const bigint &a, const bigint &b) {
    return comp(a.digits, b.digits, a.sign, b.sign);
}

friend bigint add(const bigint &a, const bigint &b) {
    return add(a.digits, b.digits, a.sign, b.sign);
}

friend bigint sub(const bigint &a, const bigint &b) {
    return sub(a.digits, b.digits, a.sign, b.sign);
}

friend bigint mul(const bigint &a, const bigint &b) {
    bigint res, row(a);
    for (int i = 0; i < (int)b.digits.size(); i++) {
        for (int j = 0; j < (b.digits[i] - '0'); j++) {
            res = add(res.digits, row.digits, res.sign, row.sign);
        }
        if (row.digits.size() > 1 || row.digits[0] != '0') {
            row.digits.insert(0, "0");
        }
    }
    res.sign = a.sign * b.sign;
    res.normalize();
    return res;
}

friend bigint div(const bigint &a, const bigint &b) {
    bigint res, row;
    res.digits.assign(a.digits.size(), '0');
    for (int i = (int)a.digits.size() - 1; i >= 0; i--) {
        row.digits.insert(row.digits.begin(), a.digits[i]);
        while (comp(row.digits, b.digits, row.sign, 1) > 0) {
            res.digits[i]++;
            row = sub(row.digits, b.digits, row.sign, 1);
        }
    }
}

```

```

    }
}
res.sign = a.sign * b.sign;
res.normalize();
return res;
}
};

```

Example Usage

```

#include <cassert>

int main() {
    bigint a("-9899819294989142124"), b("12398124981294214");
    assert(add(a, b).to_string() == "-9887421170007847910");
    assert(sub(a, b).to_string() == "-9912217419970436338");
    assert(mul(a, b).to_string() == "-122739196911503356525379735104870536");
    assert(div(a, b).to_string() == "-798");
    assert(comp(a, b) == -1 && comp(a, a) == 0 && comp(b, a) == 1);
    return 0;
}

```

6.4.2 Big Integers

Perform operations on arbitrary precision big integers internally represented as a vector of base-10⁹ digits in little-endian order. Typical arithmetic operations involving mixed numeric primitives and strings are supported using templates and operator overloading, as long as at least one operand is a `bigint` at any given level of evaluation.

- `bigint(n)` constructs a big integer from a long long (default: 0).
- `bigint(s)` constructs a big integer from a C string or an `std::string s`.
- `operator =` is defined to copy from another big integer or to assign from an 64-bit integer primitive.
- `size()` returns the number of digits in the base-10 representation.
- Operators `>>` and `<<` are defined to support stream-based input and output.
- `v.to_string()`, `v.to_llong()`, `v.to_double()`, and `v.to_ldouble()` return the big integer `v` converted to an `std::string`, `long long`, `double`, and `long double` respectively. For the latter three data types, overflow behavior is based on that of inputting from `std::istream`.
- `v.abs()` returns the absolute value of big integer `v`.
- `a.comp(b)` returns `-1`, `0`, or `1` depending on whether the big integers `a` and `b` compare less, equal, or greater, respectively.
- Operators `<`, `>`, `<=`, `>=`, `==`, `!=`, `+`, `-`, `*`, `/`, `%`, `++`, `-`, `+=`, `-=`, `*=`, `/=`, and `%=` are defined analogous to those on integer primitives. Addition, subtraction, and comparisons are performed using the standard linear algorithms. Multiplication is performed using a combination of the grade school algorithm (for smaller inputs) and either the Karatsuba algorithm (if the `USE_FFT_MULT` flag is set to false) or the Schonhage-Strassen algorithm (if `USE_FFT_MULT` is set to true). Division and modulo are computed simultaneously using the grade school method.
- `a.div(b)` returns a pair consisting of the quotient and remainder.
- `v.pow(n)` returns `v` raised to the power of `n`.
- `v.sqrt()` returns the integral part of the square root of big integer `v`.
- `v.nth_root(n)` returns the integral part of the n th root of big integer `v`.
- `rand(n)` returns a random, positive big integer with n digits.

Time Complexity:

- $O(n)$ per call to the constructors, `size()`, `to_string()`, `to_llong()`, `to_double()`, `to_ldouble()`, `abs()`, `comp()`, `rand()`, and all comparison and arithmetic operators except multiplication, division, and modulo, where n is total number of digits in the arguments and result for each operation.

- $O(n \cdot \log(n) \cdot \log(\log(n)))$ or $O(n^{1.585})$ per call to multiplication operations, depending on whether `USE_FFT_MULT` is set to true or false.
- $O(n \cdot m)$ per call to division and modulo operations, where n and m are the number of digits in the dividend and divisor, respectively.
- $O(M(m) \log n)$ per call to `pow(n)`, where m is the length of the big integer.

Space Complexity:

- $O(n)$ for storage of the big integer.
- $O(n)$ auxiliary heap space for negation, addition, subtraction, multiplication, division, `abs()`, `sqrt()`, `pow()`, and `nth_root()`.
- $O(1)$ auxiliary space for all other operations.

```
#include <algorithm>
#include <cmath>
#include <complex>
#include <cstdlib>
#include <cstring>
#include <iomanip>
#include <istream>
#include <ostream>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

class bigint {
    static const int BASE = 1000000000, BASE_DIGITS = 9;
    static const bool USE_FFT_MULT = true;

    typedef std::vector<int> vint;
    typedef std::vector<long long> vll;
    typedef std::vector<std::complex<double>> vcd;

    vint digits;
    int sign;

    void normalize() {
        while (!digits.empty() && digits.back() == 0) {
            digits.pop_back();
        }
        if (digits.empty()) {
            sign = 1;
        }
    }

    void read(int n, const char *s) {
        sign = 1;
        digits.clear();
        int pos = 0;
        while (pos < n && (s[pos] == '-' || s[pos] == '+')) {
            if (s[pos] == '-') {
                sign = -sign;
            }
            pos++;
        }
        for (int i = n - 1; i >= pos; i -= BASE_DIGITS) {
            int x = 0;
            for (int j = std::max(pos, i - BASE_DIGITS + 1); j <= i; j++) {
                x = x * 10 + s[j] - '0';
            }
            digits.push_back(x);
        }
        normalize();
    }
}
```



```

static int comp(const vint &a, const vint &b, int asign, int bsign) {
    if (asign != bsign) {
        return asign < bsign ? -1 : 1;
    }
    if (a.size() != b.size()) {
        return a.size() < b.size() ? -asign : asign;
    }
    for (int i = (int)a.size() - 1; i >= 0; i--) {
        if (a[i] != b[i]) {
            return a[i] < b[i] ? -asign : asign;
        }
    }
    return 0;
}

static bigint add(const vint &a, const vint &b, int asign, int bsign) {
    if (asign != bsign) {
        return (asign == 1) ? sub(a, b, asign, 1) : sub(b, a, bsign, 1);
    }
    bigint res;
    res.digits = a;
    res.sign = asign;
    int carry = 0, size = (int)std::max(a.size(), b.size());
    for (int i = 0; i < size || carry; i++) {
        if (i == (int)res.digits.size()) {
            res.digits.push_back(0);
        }
        res.digits[i] += carry + (i < (int)b.size() ? b[i] : 0);
        carry = (res.digits[i] >= BASE) ? 1 : 0;
        if (carry) {
            res.digits[i] -= BASE;
        }
    }
    return res;
}

static bigint sub(const vint &a, const vint &b, int asign, int bsign) {
    if (asign == -1 || bsign == -1) {
        return add(a, b, asign, -bsign);
    }
    bigint res;
    if (comp(a, b, asign, bsign) < 0) {
        res = sub(b, a, bsign, asign);
        res.sign = -1;
        return res;
    }
    res.digits = a;
    res.sign = asign;
    for (int i = 0, borrow = 0; i < (int)a.size() || borrow; i++) {
        res.digits[i] -= borrow + (i < (int)b.size() ? b[i] : 0);
        borrow = res.digits[i] < 0;
        if (borrow) {
            res.digits[i] += BASE;
        }
    }
    res.normalize();
    return res;
}

static vint convert_base(const vint &digits, int l1, int l2) {
    vll p(std::max(l1, l2) + 1);
    p[0] = 1;
    for (int i = 1; i < (int)p.size(); i++) {
        p[i] = p[i - 1] * 10;
    }
    vint res;
    long long curr = 0;

```

```

for (int i = 0, curr_digits = 0; i < (int)digits.size(); i++) {
    curr += digits[i] * p[curr_digits];
    curr_digits += 11;
    while (curr_digits >= 12) {
        res.push_back((int)(curr % p[12]));
        curr /= p[12];
        curr_digits -= 12;
    }
}
res.push_back((int)curr);
while (!res.empty() && res.back() == 0) {
    res.pop_back();
}
return res;
}

```

```

template<class It>
static vll karatsuba(It alo, It ahi, It blo, It bhi) {
    int n = std::distance(alo, ahi), k = n / 2;
    vll res(n * 2);
    if (n <= 32) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                res[i + j] += alo[i] * blo[j];
            }
        }
        return res;
    }
    vll a1b1 = karatsuba(alo, alo + k, blo, blo + k);
    vll a2b2 = karatsuba(alo + k, ahi, blo + k, bhi);
    vll a2(alo + k, ahi), b2(blo + k, bhi);
    for (int i = 0; i < k; i++) {
        a2[i] += alo[i];
        b2[i] += blo[i];
    }
    vll r = karatsuba(a2.begin(), a2.end(), b2.begin(), b2.end());
    for (int i = 0; i < (int)a1b1.size(); i++) {
        r[i] -= a1b1[i];
        res[i] += a1b1[i];
    }
    for (int i = 0; i < (int)a2b2.size(); i++) {
        r[i] -= a2b2[i];
        res[i + n] += a2b2[i];
    }
    for (int i = 0; i < (int)r.size(); i++) {
        res[i + k] += r[i];
    }
    return res;
}

```

```

template<class It>
static vcd fft(It lo, It hi, bool invert = false) {
    int n = std::distance(lo, hi), k = 0, high1 = -1;
    while ((1 << k) < n) {
        k++;
    }
    std::vector<int> rev(n, 0);
    for (int i = 1; i < n; i++) {
        if (!(i & (i - 1))) {
            high1++;
        }
        rev[i] = rev[i ^ (1 << high1)];
        rev[i] |= (1 << (k - high1 - 1));
    }
    vcd roots(n), res(n);
    for (int i = 0; i < n; i++) {
        double alpha = 2 * 3.14159265358979323846 * i / n;
        roots[i] = std::complex<double>(cos(alpha), sin(alpha));
    }
}

```

```

    res[i] = *(lo + rev[i]);
}
for (int len = 1; len < n; len <= 1) {
    vcd tmp(n);
    int rstep = roots.size() / (len < 1);
    for (int pdest = 0; pdest < n; pdest += len) {
        int p = pdest;
        for (int i = 0; i < len; i++) {
            std::complex<double> c = roots[i * rstep] * res[p + len];
            tmp[pdest] = res[p] + c;
            tmp[pdest + len] = res[p] - c;
            pdest++;
            p++;
        }
    }
    res.swap(tmp);
}
if (invert) {
    for (int i = 0; i < (int)res.size(); i++) {
        res[i] /= n;
    }
    std::reverse(res.begin() + 1, res.end());
}
return res;
}

public:
    bigint() : sign(1) {}
    bigint(int v) { *this = (long long)v; }
    bigint(long long v) { *this = v; }
    bigint(const char *s) { read(strlen(s), s); }
    bigint(const std::string &s) { read(s.size(), s.c_str()); }

    void operator=(const bigint &v) {
        sign = v.sign;
        digits = v.digits;
    }

    void operator=(long long v) {
        sign = 1;
        if (v < 0) {
            sign = -1;
            v = -v;
        }
        digits.clear();
        for (; v > 0; v /= BASE) {
            digits.push_back(v % BASE);
        }
    }

    int size() const {
        if (digits.empty()) {
            return 1;
        }
        std::ostringstream oss;
        oss << digits.back();
        return oss.str().length() + BASE_DIGITS * (digits.size() - 1);
    }

    friend std::istream &operator>>(std::istream &in, bigint &v) {
        std::string s;
        in >> s;
        v.read(s.size(), s.c_str());
        return in;
    }

    friend std::ostream &operator<<(std::ostream &out, const bigint &v) {
        if (v.sign == -1) {

```

```

    out << '-';
}
out << (v.digits.empty() ? 0 : v.digits.back());
for (int i = (int)v.digits.size() - 2; i >= 0; i--) {
    out << std::setw(BASE_DIGITS) << std::setfill('0') << v.digits[i];
}
return out;
}

std::string to_string() const {
    std::ostringstream oss;
    if (sign == -1) {
        oss << '-';
    }
    oss << (digits.empty() ? 0 : digits.back());
    for (int i = (int)digits.size() - 2; i >= 0; i--) {
        oss << std::setw(BASE_DIGITS) << std::setfill('0') << digits[i];
    }
    return oss.str();
}

long long to_llong() const {
    long long res = 0;
    for (int i = (int)digits.size() - 1; i >= 0; i--) {
        res = res * BASE + digits[i];
    }
    return res * sign;
}

double to_double() const {
    std::stringstream ss(to_string());
    double res;
    ss >> res;
    return res;
}

long double to_ldouble() const {
    std::stringstream ss(to_string());
    long double res;
    ss >> res;
    return res;
}

int comp(const bigint &v) const { return comp(digits, v.digits, sign, v.sign); }
bool operator<(const bigint &v) const { return comp(v) < 0; }
bool operator>(const bigint &v) const { return comp(v) > 0; }
bool operator<=(const bigint &v) const { return comp(v) <= 0; }
bool operator>=(const bigint &v) const { return comp(v) >= 0; }
bool operator==(const bigint &v) const { return comp(v) == 0; }
bool operator!=(const bigint &v) const { return comp(v) != 0; }

template<class T> friend bool operator<(const T &a, const bigint &b) { return bigint(a) < b; }
template<class T> friend bool operator>(const T &a, const bigint &b) { return bigint(a) > b; }
template<class T> friend bool operator<=(const T &a, const bigint &b) { return bigint(a) <= b; }
template<class T> friend bool operator>=(const T &a, const bigint &b) { return bigint(a) >= b; }
template<class T> friend bool operator==(const T &a, const bigint &b) { return bigint(a) == b; }
template<class T> friend bool operator!=(const T &a, const bigint &b) { return bigint(a) != b; }

bigint abs() const { bigint res(*this); res.sign = 1; return res; }
bigint operator-() const { bigint res(*this); res.sign = -sign; return res; }
bigint operator+(const bigint &v) const { return add(digits, v.digits, sign, v.sign); }
bigint operator-(const bigint &v) const { return sub(digits, v.digits, sign, v.sign); }

void operator*=(int v) {
    if (v < 0) {
        sign = -sign;
        v = -v;
    }
}

```

```

    for (int i = 0, carry = 0; i < (int)digits.size() || carry; i++) {
        if (i == (int)digits.size()) {
            digits.push_back(0);
        }
        long long curr = digits[i] * (long long)v + carry;
        carry = (int)(curr / BASE);
        digits[i] = (int)(curr % BASE);
    }
    normalize();
}

bigint operator*(int v) const {
    bigint res(*this);
    res *= v;
    return res;
}

bigint operator*(const bigint &v) const {
    static const int TEMP_BASE = 10000, TEMP_BASE_DIGITS = 4;
    vint a = convert_base(digits, BASE_DIGITS, TEMP_BASE_DIGITS);
    vint b = convert_base(v.digits, BASE_DIGITS, TEMP_BASE_DIGITS);
    int n = 1;
    while (n < 2 * (int)std::max(a.size(), b.size())) {
        n <<= 1;
    }
    a.resize(n, 0);
    b.resize(n, 0);
    vll c;
    if (USE_FFT_MULT) {
        vcd at = fft(a.begin(), a.end()), bt = fft(b.begin(), b.end());
        for (int i = 0; i < n; i++) {
            at[i] *= bt[i];
        }
        at = fft(at.begin(), at.end(), true);
        c.resize(n);
        for (int i = 0; i < n; i++) {
            c[i] = at[i].real() + 0.5;
        }
    } else {
        c = karatsuba(a.begin(), a.end(), b.begin(), b.end());
    }
    bigint res;
    res.sign = sign * v.sign;
    for (int i = 0, carry = 0; i < (int)c.size(); i++) {
        long long d = c[i] + carry;
        res.digits.push_back(d % TEMP_BASE);
        carry = d / TEMP_BASE;
    }
    res.digits = convert_base(res.digits, TEMP_BASE_DIGITS, BASE_DIGITS);
    res.normalize();
    return res;
}

bigint &operator/=(int v) {
    if (v == 0) {
        throw std::runtime_error("Division by zero in bigint.");
    }
    if (v < 0) {
        sign = -sign;
        v = -v;
    }
    for (int i = (int)digits.size() - 1, rem = 0; i >= 0; i--) {
        long long curr = digits[i] + rem * (long long)BASE;
        digits[i] = (int)(curr / v);
        rem = (int)(curr % v);
    }
    normalize();
    return *this;
}

```

```

}

bigint operator/(int v) const {
    bigint res(*this);
    res /= v;
    return res;
}

int operator%(int v) const {
    if (v == 0) {
        throw std::runtime_error("Division by zero in bigint.");
    }
    if (v < 0) {
        v = -v;
    }
    int m = 0;
    for (int i = (int)digits.size() - 1; i >= 0; i--) {
        m = (digits[i] + m * (long long)BASE) % v;
    }
    return m * sign;
}

std::pair<bigint, bigint> div(const bigint &v) const {
    if (v == 0) {
        throw std::runtime_error("Division by zero in bigint.");
    }
    if (comp(digits, v.digits, 1, 1) < 0) {
        return std::make_pair(0, *this);
    }
    int norm = BASE / (v.digits.back() + 1);
    bigint an = abs() * norm, bn = v.abs() * norm, q, r;
    q.digits.resize(an.digits.size());
    for (int i = (int)an.digits.size() - 1; i >= 0; i--) {
        r *= BASE;
        r += an.digits[i];
        int s1 = (r.digits.size() <= bn.digits.size()) ? 0 : r.digits[bn.digits.size()];
        int s2 = (r.digits.size() <= bn.digits.size() - 1) ? 0 : r.digits[bn.digits.size() - 1];
        int d = ((long long)s1 * BASE + s2) / bn.digits.back();
        for (r -= bn * d; r < 0; r += bn) {
            d--;
        }
        q.digits[i] = d;
    }
    q.sign = sign * v.sign;
    r.sign = sign;
    q.normalize();
    r.normalize();
    return std::make_pair(q, r / norm);
}

bigint operator/(const bigint &v) const { return div(v).first; }
bigint operator%(const bigint &v) const { return div(v).second; }
bigint operator++(int) { bigint t(*this); operator++; return t; }
bigint operator--(int) { bigint t(*this); operator--; return t; }
bigint &operator++() { *this = *this + bigint(1); return *this; }
bigint &operator--() { *this = *this - bigint(1); return *this; }
bigint &operator+=(const bigint &v) { *this = *this + v; return *this; }
bigint &operator-=(const bigint &v) { *this = *this - v; return *this; }
bigint &operator*=(const bigint &v) { *this = *this * v; return *this; }
bigint &operator/=(const bigint &v) { *this = *this / v; return *this; }
bigint &operator%=(const bigint &v) { *this = *this % v; return *this; }

template<class T>
friend bigint operator+(const T &a, const bigint &b) { return bigint(a) + b; }

template<class T>
friend bigint operator-(const T &a, const bigint &b) { return bigint(a) - b; }

```

```

bigint pow(int n) const {
    if (n == 0) {
        return bigint(1);
    }
    if (*this == 0 || n < 0) {
        return bigint(0);
    }
    bigint x(*this), res(1);
    for (; n != 0; n >= 1) {
        if (n & 1) {
            res *= x;
        }
        x *= x;
    }
    return res;
}

bigint sqrt() const {
    if (sign == -1) {
        throw std::runtime_error("Cannot take square root of a negative number.");
    }
    bigint v(*this);
    while (v.digits.empty() || v.digits.size() % 2 == 1) {
        v.digits.push_back(0);
    }
    int n = v.digits.size();
    int ldig = (int)::sqrt((double)v.digits[n - 1] * BASE + v.digits[n - 2]);
    int norm = BASE / (ldig + 1);
    v *= norm;
    v *= norm;
    while (v.digits.empty() || v.digits.size() % 2 == 1) {
        v.digits.push_back(0);
    }
    bigint r((long long)v.digits[n - 1] * BASE + v.digits[n - 2]);
    int q = ldig = (int)::sqrt((double)v.digits[n - 1] * BASE + v.digits[n - 2]);
    bigint res;
    for (int j = n / 2 - 1; j >= 0; j--) {
        for (; q-- > 0) {
            bigint r1 = (r - (res * 2 * BASE + q) * q) * BASE * BASE +
                (j > 0 ? (long long)v.digits[2 * j - 1] * BASE + v.digits[2 * j - 2] : 0);
            if (r1 >= 0) {
                r = r1;
                break;
            }
        }
        res = res * BASE + q;
        if (j > 0) {
            int sz1 = res.digits.size(), sz2 = r.digits.size();
            int d1 = (sz1 + 2 < sz2) ? r.digits[sz1 + 2] : 0;
            int d2 = (sz1 + 1 < sz2) ? r.digits[sz1 + 1] : 0;
            int d3 = (sz1 < sz2) ? r.digits[sz1] : 0;
            q = ((long long)d1 * BASE * BASE + (long long)d2 * BASE + d3) / (ldig * 2);
        }
    }
    res.normalize();
    return res / norm;
}

bigint nth_root(int n) const {
    if (sign == -1 && n % 2 == 0) {
        throw std::runtime_error("Cannot take even root of a negative number.");
    }
    if (*this == 0 || n < 0) {
        return bigint(0);
    }
    if (n >= size()) {
        int p = 1;
        while (comp(bigint(p).pow(n)) > 0) {

```

```

    p++;
}
return comp(bigint(p).pow(n)) < 0 ? p - 1 : p;
}
bigint lo(bigint(10).pow((int)ceil((double)size() / n) - 1)), hi(lo * 10), mid;
while (lo < hi) {
    mid = (lo + hi) / 2;
    int cmp = comp(digits, mid.pow(n).digits, 1, 1);
    if (lo < mid && cmp > 0) {
        lo = mid;
    } else if (mid < hi && cmp < 0) {
        hi = mid;
    } else {
        return (sign == -1) ? -mid : mid;
    }
}
return (sign == -1) ? -(mid + 1) : (mid + 1);
}

static bigint rand(int n) {
    if (n == 0) {
        return bigint(0);
    }
    std::string s(1, '1' + (::rand() % 9));
    for (int i = 1; i < n; i++) {
        s += '0' + (::rand() % 10);
    }
    return bigint(s);
}

friend int comp(const bigint &a, const bigint &b) { return a.comp(b); }
friend bigint abs(const bigint &v) { return v.abs(); }
friend bigint pow(const bigint &v, int n) { return v.pow(n); }
friend bigint sqrt(const bigint &v) { return v.sqrt(); }
friend bigint nth_root(const bigint &v, int n) { return v.nth_root(n); }
};

```

Example Usage

```

#include <cassert>

int main() {
    bigint a("-9899819294989142124"), b("12398124981294214");
    assert(a + b == "-9887421170007847910");
    assert(a - b == "-9912217419970436338");
    assert(a * b == "-122739196911503356525379735104870536");
    assert(a / b == "-798");
    assert(bigint(20).pow(12345).size() == 16062);
    assert(bigint("9812985918924981892491829").nth_root(4) == 1769906);
    for (int i = -100; i <= 100; i++) {
        if (i >= 0) {
            assert(bigint(i).sqrt() == (int)sqrt(i));
        }
        for (int j = -100; j <= 100; j++) {
            assert(bigint(i) + bigint(j) == i + j);
            assert(bigint(i) - bigint(j) == i - j);
            assert(bigint(i) * bigint(j) == i * j);
            if (j != 0) {
                assert(bigint(i) / bigint(j) == i / j);
            }
            if (0 < i && i <= 10 && 0 < j && j <= 10) {
                assert(bigint(i).nth_root(j) == (long long)(pow(i, 1.0 / j) + 1E-5));
                long long p = 1;
                for (int k = 0; k < j; k++) {
                    p *= i;
                }
            }
        }
    }
}

```



```

        assert(bigint(i).pow(j) == p);
    }
}
}
for (int i = 0; i < 20; i++) {
    int n = rand() % 100 + 1;
    bigint a(bigint::rand(n)), s(a.sqrt()), xx(s * s), yy(s + 1);
    yy *= yy;
    assert(xx <= a && a < yy);
    bigint b(bigint::rand(rand() % n + 1) + 1), q(a / b);
    xx = q * b;
    yy = b * (q + 1);
    assert(a >= xx && a < yy);
}
bigint x(-6);
assert(x.to_string() == "-6");
assert(x.to_llong() == -6LL);
assert(x.to_double() == -6.0);
assert(x.to_ldouble() == -6.0);
return 0;
}

```

6.4.3 Rational Numbers

Perform operations on rational numbers internally represented as two integers, a numerator and a denominator. The template integer type must support streamed input/output, comparisons, and arithmetic operations. Overflow is not checked for in internal operations.

- `rational(n)` constructs a rational with numerator `n` and denominator 1.
- `rational(n, d)` constructs a rational with numerator `n` and denominator `d`.
- `operator >>` inputs a rational using the next integer from the stream as the numerator and 1 as the denominator.
- `operator <<` outputs a rational as a string consisting of possibly a minus sign followed by the numerator, followed by a slash, followed by the denominator.
- `v.to_string()`, `v.to_llong()`, `v.to_double()`, and `v.to_ldouble()` return the big integer `v` converted to an `std::string`, `long long`, `double`, and `long double` respectively.
- Operators `<`, `>`, `<=`, `>=`, `==`, `!=`, `+`, `-`, `*`, `/`, `%`, `++`, `-`, `+=`, `-=`, `*=`, `/=`, and `%=` are defined analogous to those on numerical primitives.

Time Complexity:

- $O(\log(n + d))$ per call to constructor `rational(n, d)`.
- $O(1)$ per call to all other operations, assuming that corresponding operations on the template integer type are $O(1)$ as well.

Space Complexity:

- $O(1)$ for storage of the rational.
- $O(1)$ auxiliary space for all operations.

```

#include <istream>
#include <ostream>
#include <sstream>
#include <string>

template<class Int = long long>
class rational {
    Int num, den;

public:
    rational() : num(0), den(1) {}
    rational(const Int &n) : num(n), den(1) {}

```

```

template<class T1, class T2>
rational(const T1 &n, const T2 &d) : num(n), den(d) {
    if (den == 0) {
        throw std::runtime_error("Division by zero in rational.");
    }
    if (den < 0) {
        num = -num;
        den = -den;
    }
    Int a(num < 0 ? -num : num), b(den), tmp;
    while (a != 0 && b != 0) {
        tmp = a % b;
        a = b;
        b = tmp;
    }
    Int gcd = (b == 0) ? a : b;
    num /= gcd;
    den /= gcd;
}

friend std::istream &operator>>(std::istream &in, rational &r) {
    std::string s;
    in >> r.num;
    r.den = 1;
    return in;
}

friend std::ostream &operator<<(std::ostream &out, const rational &r) {
    out << r.num << "/" << r.den;
    return out;
}

std::string to_string() const {
    std::stringstream ss;
    ss << num << " " << den;
    std::string n, d;
    ss >> n >> d;
    return n + "/" + d;
}

long long to_llong() const {
    std::stringstream ss;
    ss << num << " " << den;
    long long n, d;
    ss >> n >> d;
    return n / d;
}

double to_double() const {
    std::stringstream ss;
    ss << num << " " << den;
    double n, d;
    ss >> n >> d;
    return n / d;
}

long double to_ldouble() const {
    long double n, d;
    std::stringstream ss;
    ss << num << " " << den;
    ss >> n >> d;
    return n / d;
}

bool operator<(const rational &r) const { return num * r.den < r.num * den; }
bool operator>(const rational &r) const { return r.num * den < num * r.den; }
bool operator<=(const rational &r) const { return !(r < *this); }

```

```

bool operator>=(const rational &r) const { return !(*this < r); }
bool operator==(const rational &r) const { return num == r.num && den == r.den; }
bool operator!=(const rational &r) const { return num != r.num || den != r.den; }

template<class T>
friend bool operator<(const T &a, const rational &b) { return rational(a) < b; }

template<class T>
friend bool operator>(const T &a, const rational &b) { return rational(a) > b; }

template<class T>
friend bool operator<=(const T &a, const rational &b) { return rational(a) <= b; }

template<class T>
friend bool operator>=(const T &a, const rational &b) { return rational(a) >= b; }

template<class T>
friend bool operator==(const T &a, const rational &b) { return rational(a) == b; }

template<class T>
friend bool operator!=(const T &a, const rational &b) { return rational(a) != b; }

rational abs() const { return rational(num < 0 ? -num : num, den); }
friend rational abs(const rational &r) { return r.abs(); }
Int floor() const { return num < 0 ? -((-num + den - 1) / den) : num / den; }
Int ceil() const { return num < 0 ? -(-num / den) : (num + den - 1) / den; }

rational operator+(const rational &r) const {
    return rational(num * r.den + r.num * den, den * r.den);
}

rational operator-(const rational &r) const {
    return rational(num * r.den - r.num * den, r.den * den);
}

rational operator*(const rational &r) const { return rational(num * r.num, r.den * den); }
rational operator/(const rational &r) const { return rational(num * r.den, den * r.num); }

rational operator%(const rational &r) const {
    return *this - r * rational(num * r.den / (r.num * den), 1);
}

template<class T>
friend rational operator+(const T &a, const rational &b) { return rational(a) + b; }

template<class T>
friend rational operator-(const T &a, const rational &b) { return rational(a) - b; }

template<class T>
friend rational operator*(const T &a, const rational &b) { return rational(a) * b; }

template<class T>
friend rational operator/(const T &a, const rational &b) { return rational(a) / b; }

template<class T>
friend rational operator%(const T &a, const rational &b) { return rational(a) % b; }

rational operator-() const { return rational(-num, den); }

rational operator++(int) {
    rational t(*this);
    operator++;
    return t;
}

rational operator--(int) {
    rational t(*this);
    operator--;
    return t;
}

```

```

}
rational &operator++() {
    *this = *this + 1;
    return *this;
}
rational &operator--() {
    *this = *this - 1;
    return *this;
}
rational &operator+=(const rational &r) {
    *this = *this + r;
    return *this;
}
rational &operator-=(const rational &r) {
    *this = *this - r;
    return *this;
}
rational &operator*=(const rational &r) {
    *this = *this * r;
    return *this;
}
rational &operator/=(const rational &r) {
    *this = *this / r;
    return *this;
}
rational &operator%=(const rational &r) {
    *this = *this % r;
    return *this;
}
};

```

Example Usage

```

#include <cassert>
#include <cmath>

int main() {
#define EQ(a, b) (fabs((a) - (b)) <= 1E-9)
    typedef rational<long long> rational;

    assert(rational(-21, 1) % 2 == -1);
    rational r(rational(-53, 10) % rational(-17, 10));
    assert(EQ(r.to_ldouble(), fmod(-5.3, -1.7)));
    assert(r.to_string() == "-1/5");
    return 0;
}

```

6.5 Linear Algebra

6.5.1 Matrix Utilities

Basic matrix operations defined on a two-dimensional vector of numeric values.

- `make_matrix(r, c, v)` constructs and returns a matrix with `r` rows and `c` columns where the value at every index is initialized to `v`.
- `make_matrix(a)` returns a matrix constructed from the two dimensional array `a`.
- `identity_matrix(n)` returns the n by n identity matrix, that is, a matrix where `a[i][j]` equals 1 (if $i = j$), or 0 otherwise, for every i and j in $[0, n)$.
- `rows(a)` returns the number of rows r in an r by c matrix `a`.
- `columns(a)` returns the number of columns c in an r by c matrix `a`.

- `a[i][j]` may be used to access or modify the entry at row i , column j of an r by c matrix `a`, for every i in $[0, r)$ and j in $[0, c)$.
- Operators `<`, `>`, `<=`, `>=`, `==`, and `!=` define lexicographical comparison based on that of `std::vector`.
- Operators `+`, `-`, `*`, `/`, `+=`, `-=`, `*=`, and `/=` define scalar addition, subtraction, multiplication, and division involving a matrix and a numeric scalar value.
- Operators `*` and `*=` define vector and matrix multiplication.
- Operators `^` and `^=` define matrix exponentiation of a square matrix a by an integer power p .
- `power_sum(a, p)` returns the power sum of a square matrix a up to an integer power p , that is, $a + a^2 + \dots + a^p$.
- `transpose(a)` returns the transpose of an r by c matrix `a`, that is, a new c by r matrix b such that $a[i][j] = b[j][i]$ for every i in $[0, r)$ and j in $[0, c)$.
- `transpose_in_place(a)` assigns the square matrix `a` to its transpose, returning a reference to the modified argument itself.
- `rotate(a, d)` returns the matrix `a` rotated d degrees clockwise. A negative d specifies a counter-clockwise rotation, and d must be a multiple of 90.
- `rotate_in_place(a, d)` assigns the square matrix `a` to its rotation by d degrees clockwise, returning a reference to the modified argument itself. A negative d specifies a counter-clockwise rotation, and d must be a multiple of 90.

Time Complexity:

- $O(n \cdot m)$ for construction, output, comparison, and scalar arithmetic of n by m matrices.
- $O(1)$ for `rows(a)` and `columns(a)`.
- $O(n \cdot m)$ for matrix-matrix addition and subtraction of n by m matrices.
- $O(n \cdot m \cdot \log(p))$ for exponentiation of an n by m matrix to power p .
- $O(n \cdot m \cdot \log^2(p))$ for power sum of an n by m matrix to power p .
- $O(n \cdot m \cdot k)$ for multiplication of an n by m matrix by an m by k matrix.
- $O(n \cdot m)$ for `transpose()`, `transpose_in_place()`, `rotate()`, and `rotate_in_place()` of n by m matrices.

Space Complexity:

- $O(1)$ auxiliary space for `rows()`, `columns()`, `a[i][j]` access, comparison operators, and in-place operations.
- $O(n \cdot m \cdot \log(p))$ auxiliary stack and heap space for exponentiation of an n by m matrix to power p , as well as the power sum of an n by m matrix up to power p .
- $O(n \cdot m)$ auxiliary heap space for all non-in-place operations returning an n by m matrix, `transpose()`, and `rotate()`.

```
#include <algorithm>
#include <cstdint>
#include <iomanip>
#include <ostream>
#include <stdexcept>
#include <vector>

typedef std::vector<std::vector<int>> matrix;

matrix make_matrix(int r, int c) {
    return matrix(r, matrix::value_type(c));
}

template<class T>
matrix make_matrix(int r, int c, const T &v) {
    return matrix(r, matrix::value_type(c, v));
}

template<class T, size_t r, size_t c>
matrix make_matrix(T (&a)[r][c]) {
    matrix res(r, matrix::value_type(c));
    for (size_t i = 0; i < r; i++) {
        for (size_t j = 0; j < c; j++) {
            res[i][j] = a[i][j];
        }
    }
}
```

```

    }
}
return res;
}

matrix identity_matrix(int n) {
    matrix res(n, matrix::value_type(n, 0));
    for (int i = 0; i < n; i++) {
        res[i][i] = 1;
    }
    return res;
}

int rows(const matrix &a) {
    return a.size();
}

int columns(const matrix &a) {
    return a.empty() ? 0 : a[0].size();
}

std::ostream &operator<<(std::ostream &out, const matrix &a) {
    static const int W = 10, P = 5;
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(a); j++) {
            out << std::setw(W) << std::fixed << std::setprecision(P) << a[i][j];
        }
        out << std::endl;
    }
    return out;
}

template<class T>
matrix &operator+=(matrix &a, const T &v) {
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(a); j++) {
            a[i][j] += v;
        }
    }
    return a;
}

template<class T>
matrix &operator-=(matrix &a, const T &v) {
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(a); j++) {
            a[i][j] -= v;
        }
    }
    return a;
}

template<class T>
matrix &operator*=(matrix &a, const T &v) {
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(a); j++) {
            a[i][j] *= v;
        }
    }
    return a;
}

template<class T>
matrix &operator/=(matrix &a, const T &v) {
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(a); j++) {
            a[i][j] /= v;
        }
    }
}

```

```

    }
    return a;
}

matrix &operator+=(matrix &a, const matrix &b) {
    if (rows(a) != rows(b) || columns(a) != columns(b)) {
        throw std::runtime_error("Invalid dimensions for matrix addition.");
    }
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(a); j++) {
            a[i][j] += b[i][j];
        }
    }
    return a;
}

matrix &operator-=(matrix &a, const matrix &b) {
    if (rows(a) != rows(b) || columns(a) != columns(b)) {
        throw std::runtime_error("Invalid dimensions for matrix addition.");
    }
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(a); j++) {
            a[i][j] -= b[i][j];
        }
    }
    return a;
}

matrix operator+(const matrix &a, const matrix &b) {
    matrix c(a);
    return c += b;
}

matrix operator-(const matrix &a, const matrix &b) {
    matrix c(a);
    return c -= b;
}

template<class T>
matrix &operator*=(matrix &a, const std::vector<T> &v) {
    if (columns(a) != (int)v.size() || v.empty()) {
        throw std::runtime_error("Invalid dimensions for matrix multiplication.");
    }
    for (int i = 0; i < rows(a); i++) {
        a[i][0] *= v[0];
        for (int j = 1; j < columns(a); j++) {
            a[i][0] += a[i][j] * v[j];
        }
    }
    for (int i = 0; i < rows(a); i++) {
        a[i].resize(1);
    }
    return a;
}

matrix operator*(const matrix &a, const matrix &b) {
    if (columns(a) != rows(b)) {
        throw std::runtime_error("Invalid dimensions for matrix multiplication.");
    }
    matrix res = make_matrix(rows(a), columns(b), 0);
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(b); j++) {
            for (int k = 0; k < rows(b); k++) {
                res[i][j] += a[i][k] * b[k][j];
            }
        }
    }
    return res;
}

```

```

}

matrix &operator*=(matrix &a, const matrix &b) { return a = a * b; }

template<class T>
matrix operator+(const matrix &a, const T &v) { matrix m(a); return m += v; }

template<class T>
matrix operator-(const matrix &a, const T &v) { matrix m(a); return m -= v; }

template<class T>
matrix operator*(const matrix &a, const T &v) { matrix m(a); return m *= v; }

template<class T>
matrix operator/(const matrix &a, const T &v) { matrix m(a); return m /= v; }

template<class T>
matrix operator+(const T &v, const matrix &a) { return a + v; }

template<class T>
matrix operator-(const T &v, const matrix &a) { return a - v; }

template<class T>
matrix operator*(const T &v, const matrix &a) { return a * v; }

template<class T>
matrix operator/(const T &v, const matrix &a) { return a / v; }

matrix operator^(const matrix &a, unsigned int p) {
    if (rows(a) != columns(a)) {
        throw std::runtime_error("Matrix must be square for exponentiation.");
    }
    if (p == 0) {
        return identity_matrix(rows(a));
    }
    return (p % 2 == 0) ? (a * a) ^ (p / 2) : a * (a ^ (p - 1));
}

matrix operator^=(matrix &a, unsigned int p) {
    return a = a ^ p;
}

matrix power_sum(const matrix &a, unsigned int p) {
    if (rows(a) != columns(a)) {
        throw std::runtime_error("Matrix must be square for power_sum.");
    }
    if (p == 0) {
        return make_matrix(rows(a), rows(a));
    }
    return (p % 2 == 0) ? power_sum(a, p / 2) * (identity_matrix(rows(a)) + (a ^ (p / 2)))
        : (a + a * power_sum(a, p - 1));
}

matrix transpose(const matrix &a) {
    matrix res = make_matrix(columns(a), rows(a));
    for (int i = 0; i < rows(res); i++) {
        for (int j = 0; j < columns(res); j++) {
            res[i][j] = a[j][i];
        }
    }
    return res;
}

matrix &transpose_in_place(matrix &a) {
    if (rows(a) != columns(a)) {
        throw std::runtime_error("Matrix must be square for transpose_in_place.");
    }
    for (int i = 0; i < rows(a); i++) {

```



```

        for (int j = i + 1; j < columns(a); j++) {
            std::swap(a[i][j], a[j][i]);
        }
    }
    return a;
}

matrix rotate(const matrix &a, int degrees = 90) {
    if (degrees % 90 != 0) {
        throw std::runtime_error("Rotation must be by a multiple of 90 degrees.");
    }
    if (degrees < 0) {
        degrees = 360 - ((-degrees) % 360);
    }
    matrix res;
    switch (degrees % 360) {
        case 90: {
            res = make_matrix(columns(a), rows(a));
            for (int i = 0; i < columns(a); i++) {
                for (int j = 0; j < rows(a); j++) {
                    res[i][j] = a[rows(a) - j - 1][i];
                }
            }
            break;
        }
        case 180: {
            res = make_matrix(rows(a), columns(a));
            for (int i = 0; i < rows(a); i++) {
                for (int j = 0; j < columns(a); j++) {
                    res[i][j] = a[rows(a) - i - 1][columns(a) - j - 1];
                }
            }
            break;
        }
        case 270: {
            res = make_matrix(columns(a), rows(a));
            for (int i = 0; i < columns(a); i++) {
                for (int j = 0; j < rows(a); j++) {
                    res[i][j] = a[j][columns(a) - i - 1];
                }
            }
            break;
        }
        default: {
            res = a;
        }
    }
    return res;
}

matrix &rotate_in_place(matrix &a, int degrees = 90) {
    if (degrees % 90 != 0) {
        throw std::runtime_error("Rotation must be by a multiple of 90 degrees.");
    }
    if (degrees % 180 != 0 && rows(a) != columns(a)) {
        throw std::runtime_error("Matrix must be square for rotate_in_place.");
    }
    if (degrees < 0) {
        degrees = 360 - ((-degrees) % 360);
    }
    int n = rows(a);
    switch (degrees % 360) {
        case 90: {
            transpose_in_place(a);
            for (int i = 0; i < n; i++) {
                std::reverse(a[i].begin(), a[i].end());
            }
            break;
        }
    }
}

```

```

}
case 180: {
    for (int i = 0; i < columns(a); i++) {
        for (int j = 0, k = n - 1; j < k; j++, k--) {
            std::swap(a[i][j], a[i][k]);
        }
    }
    for (int j = 0; j < n; j++) {
        for (int i = 0, k = columns(a) - 1; i < k; i++, k--) {
            std::swap(a[i][j], a[k][j]);
        }
    }
    break;
}
case 270: {
    transpose_in_place(a);
    for (int j = 0; j < n; j++) {
        for (int i = 0, k = columns(a) - 1; i < k; i++, k--) {
            std::swap(a[i][j], a[k][j]);
        }
    }
    break;
}
}
return a;
}

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    int a[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int a90[3][2] = {{4, 1}, {5, 2}, {6, 3}};
    int a180[2][3] = {{6, 5, 4}, {3, 2, 1}};
    int a270[3][2] = {{3, 6}, {2, 5}, {1, 4}};
    cout << make_matrix(a) << endl;
    assert(rotate(make_matrix(a), -270) == make_matrix(a90));
    assert(rotate(make_matrix(a), -180) == make_matrix(a180));
    assert(rotate(make_matrix(a), -90) == make_matrix(a270));
    assert(rotate(make_matrix(a), 0) == make_matrix(a));
    assert(rotate(make_matrix(a), 90) == make_matrix(a90));
    assert(rotate(make_matrix(a), 180) == make_matrix(a180));
    assert(rotate(make_matrix(a), 270) == make_matrix(a270));
    assert(rotate(make_matrix(a), 360) == make_matrix(a));

    int b[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    for (int d = -360; d <= 360; d += 90) {
        matrix m = make_matrix(b);
        assert(rotate_in_place(m, d) == rotate(make_matrix(b), d));
    }

    matrix m = make_matrix(5, 5, 10) + 10;
    int v[] = {1, 2, 3, 4, 5}, mv[5][1] = {{300}, {300}, {300}, {300}, {300}};
    assert(m * vector<int>(v, v + 5) == make_matrix(mv));

    m[0][0] += 5;
    assert(m[0][0] == 25 && m[1][1] == 20);
    assert(power_sum(m, 3) == m + m * m + (m ^ 3));
    return 0;
}

```

6.5.2 Row Reduction

Converts a matrix to reduced row echelon form using Gaussian elimination to solve a system of linear equations as well as compute the determinant. In practice, this method is prone to rounding error on certain matrices. For a more accurate algorithm for solving systems of linear equations, LU decomposition with row partial pivoting should be used.

- `row_reduce(a)` assigns the matrix `a` to its reduced row echelon form, returning a reference to the modified argument itself.
- `solve_system(a, b, &x)` solves the system of linear equations $ax = b$ given an r by c matrix `a` of real values, and a length r vector `b`, returning 0 if there is one solution, -1 if there are zero solutions, or -2 if there are infinite solutions. If there is exactly one solution, then the vector pointed to by `x` is populated with the solution vector of length c .

Time Complexity: $O(r^2 \cdot c)$ per call to `row_reduce(a)` and `solve_system(a)`, where r and c are the number of rows and columns of `a` respectively.

Space Complexity:

- $O(1)$ auxiliary for `row_reduce(a)`.
- $O(r \cdot c)$ auxiliary heap space for `solve_system(a)`.

```
#include <cmath>
#include <cstdint>
#include <stdexcept>
#include <vector>

const double EPS = 1e-9;

template<class Matrix>
Matrix &row_reduce(Matrix &a) {
    if (a.empty()) {
        return a;
    }
    int r = a.size(), c = a[0].size(), lead = 0;
    for (int row = 0; row < r && lead < c; row++) {
        int i = row;
        while (fabs(a[i][lead]) < EPS) {
            if (++i == r) {
                i = row;
                if (++lead == c) {
                    return a;
                }
            }
        }
        std::swap(a[i], a[row]);
        typename Matrix::value_type::value_type lv = a[row][lead];
        for (int j = 0; j < c; j++) {
            a[row][j] /= lv;
        }
        for (int i = 0; i < r; i++) {
            if (i != row) {
                lv = a[i][lead];
                for (int j = 0; j < c; j++) {
                    a[i][j] -= lv * a[row][j];
                }
            }
        }
        for (int j = 0; j < lead; j++) {
            a[row][j] = 0;
        }
        a[row][lead++] = 1;
    }
    return a;
}
```

```

template<class Matrix, class T>
int solve_system(const Matrix &a, const std::vector<T> &b, std::vector<T> *x) {
    if (x == NULL || a.empty() || a.size() != b.size()) {
        return -1;
    }
    int r = a.size(), c = a[0].size();
    if (r < c) {
        return -2;
    }
    Matrix m(a);
    for (int i = 0; i < r; i++) {
        m[i].push_back(b[i]);
    }
    row_reduce(m);
    for (int i = 0; i < r; i++) {
        int lead = -1;
        for (int j = 0; j < c && lead < 0; j++) {
            if (fabs(m[i][j]) > EPS) {
                lead = j;
            }
        }
        if (lead < 0 && fabs(m[i][c]) > EPS) {
            return -1;
        }
        if (lead > i) {
            return -2;
        }
    }
    x->resize(c);
    for (int i = 0; i < c; i++) {
        (*x)[i] = m[i][c];
    }
    return 0;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    const int equations = 3, unknowns = 3;
    const int a[equations][unknowns] = {{-1, 2, 5}, {1, 0, -6}, {-4, 2, 2}};
    const int b[equations] = {3, 1, -2};
    vector<vector<double>> m(equations);
    for (int i = 0; i < equations; i++) {
        m[i].assign(a[i], a[i] + unknowns);
    }
    vector<double> x;
    assert(solve_system(m, vector<double>(b, b + equations), &x) == 0);
    for (int i = 0; i < equations; i++) {
        double sum = 0;
        for (int j = 0; j < unknowns; j++) {
            sum += a[i][j] * x[j];
        }
        assert(fabs(sum - b[i]) < EPS);
    }
    return 0;
}

```

6.5.3 Determinant and Inverse

Computes the determinant and inverse of a square matrix using Gaussian elimination. The inverse of a matrix a is another matrix b such that ab equals the identity matrix. The inverse of a exists if and only if the determinant of a is nonzero. In this case, a is called invertible or non-singular. In practice, simple Gaussian elimination is prone to rounding error on certain matrices. For a more accurate algorithm for solving systems of linear equations, use LU decomposition with row partial pivoting.

- `det_naive(a)` returns the determinant of an n by n matrix `a`, using the classic divide-and-conquer algorithm by Laplace expansions.
- `det(a)` returns the determinant of an n by n matrix `a` using Gaussian elimination.
- `invert(a)` assigns the n by n matrix `a` to its inverse (if it exists), returning a reference to the modified argument itself. If `a` is not invertible, then its assigned values after the function call will be undefined (+/-Inf or +/-NaN).

Time Complexity:

- $O(n!)$ per call to `det_naive()`, where n is the dimension of the matrix.
- $O(n^3)$ per call to `det()` and `invert()` where n is the dimension of the matrix.

Space Complexity:

- $O(n)$ auxiliary stack space and $O(n! \cdot n)$ auxiliary heap space for `det_naive()`, where n is the dimension of the matrix.
- $O(n^2)$ auxiliary heap space for `det()` and `invert()`.

```
#include <cmath>
#include <map>
#include <vector>

template<class SquareMatrix>
double det_naive(const SquareMatrix &a) {
    int n = a.size();
    if (n == 1) {
        return a[0][0];
    }
    if (n == 2) {
        return a[0][0] * a[1][1] - a[0][1] * a[1][0];
    }
    double res = 0;
    SquareMatrix temp(n - 1, typename SquareMatrix::value_type(n - 1));
    for (int p = 0; p < n; p++) {
        int h = 0, k = 0;
        for (int i = 1; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (j == p) {
                    continue;
                }
                temp[h][k++] = a[i][j];
            }
            if (k == n - 1) {
                h++;
                k = 0;
            }
        }
        res += (p % 2 == 0 ? 1 : -1) * a[0][p] * det_naive(temp);
    }
    return res;
}

template<class SquareMatrix>
double det(const SquareMatrix &a, double EPS = 1e-10) {
    SquareMatrix b(a);
    int n = a.size();
    double res = 1.0;
```

```

std::vector<bool> used(n, false);
for (int i = 0; i < n; i++) {
    int p;
    for (p = 0; p < n; p++) {
        if (!used[p] && fabs(b[p][i]) > EPS) {
            break;
        }
    }
    if (p >= n) {
        return 0;
    }
    res *= b[p][i];
    used[p] = true;
    double z = 1.0 / b[p][i];
    for (int j = 0; j < n; j++) {
        b[p][j] *= z;
    }
    for (int j = 0; j < n; j++) {
        if (j != p) {
            z = b[j][i];
            for (int k = 0; k < n; k++) {
                b[j][k] -= z * b[p][k];
            }
        }
    }
}
}
return res;
}

template<class SquareMatrix>
SquareMatrix &invert(SquareMatrix &a) {
    int n = a.size();
    for (int i = 0; i < n; i++) {
        a[i].resize(2 * n);
        for (int j = n; j < n * 2; j++) {
            a[i][j] = (i == j - n ? 1 : 0);
        }
    }
    for (int i = 0; i < n; i++) {
        double z = a[i][i];
        for (int j = i; j < n * 2; j++) {
            a[i][j] /= z;
        }
        for (int j = 0; j < n; j++) {
            if (i != j) {
                double z = a[j][i];
                for (int k = 0; k < n * 2; k++) {
                    a[j][k] -= z * a[i][k];
                }
            }
        }
    }
    for (int i = 0; i < n; i++) {
        a[i].erase(a[i].begin(), a[i].begin() + n);
    }
    return a;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    const int n = 3, a[n][n] = {{6, 1, 1}, {4, -2, 5}, {2, 8, 7}};
    vector<vector<double>> m(n), inv, res(n, vector<double>(n, 0));

```

```

for (int i = 0; i < n; i++) {
    m[i] = vector<double>(a[i], a[i] + n);
}
double d = det(m);
assert(fabs(d - det_naive(m)) < 1e-10);
invert(inv = m);
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        for (int k = 0; k < n; k++) {
            res[i][j] += a[i][k] * inv[k][j];
        }
    }
}
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        assert(fabs(res[i][j] - (i == j ? 1 : 0)) < 1e-10);
    }
}
return 0;
}

```

6.5.4 LU Decomposition

The LU decomposition of a matrix a with row-partial pivoting is a factorization of a (after some rows are possibly permuted by a permutation matrix p) as a product of a lower triangular matrix l and an upper triangular matrix u . This factorization can be used to tackle many common problems in linear algebra such as solving systems of linear equations and computing determinants. An improvement on basic row reduction, LU decomposition by row-partial pivoting keeps the relative magnitude of matrix values small, thus reducing the relative error due to rounding in computed solutions.

- `lu_decompose(a, &p1col)` assigns the r by c matrix `a` to merged LU decomposition matrix `lu`, returning either 0 or 1 denoting the "sign" of the permutation parity (0 if the number of overall row swaps performed is even, or 1 if it is odd), or -1 denoting a degenerate matrix (i.e. singular for square matrices). The merged matrix `lu` has `lu[i][j] = l[i][j]` for $i > j$ and `lu[i][j] = u[i][j]` for $i \leq j$. Note that the algorithm always yields an atomic lower triangular matrix for which the diagonal entries `l[i][i]` are always equal to 1, so this is not explicitly stored in the resulting merged matrix. For general i and j , the values of the lower and upper triangular matrices should be accessed via the `getl(lu, i, j)` and `getu(lu, i, j)` functions. Optionally, a `vector<int>` pointer `p1col` may be passed to return the permutation vector `p1col` where `p1col[i]` stores the only column that is equal to 1 in row i of the permutation matrix p (all other columns in row i of p are implicitly 0). The resulting permutation matrix p corresponding to `p1col` will satisfy $pa = lu$.
- `solve_system(a, b, &x)` solves the system of linear equations $ax = b$ given an r by c matrix `a` of real values, and a length r vector `b`, returning 0 if there is one solution or -1 if there are zero or infinite solutions. If there is exactly one solution, then the vector pointed to by `x` is populated with the solution vector of length c .
- `det(a)` returns the determinant of an n by n matrix `a` using LU decomposition.
- `invert(a)` assigns the n by n matrix `a` to its inverse (if it exists), returning 0 if the inversion was successful or -1 if `a` has no inverse.

Time Complexity:

- $O(r^2 \cdot c)$ per call to `lu_decompose(a)` and `solve_system(a, b)`, where r and c are the number of rows and columns respectively, in accordance to the functions' descriptions above.
- $O(n^3)$ per call to `det(a)` and `inverse(a)`, where n is the dimension of `a`.

Space Complexity:

- $O(1)$ auxiliary for `lu_decompose()`.
- $O(n^2)$ for `det(a)` and `inverse(a)`.
- $O(r \cdot c)$ auxiliary heap space for `solve_system(a, b)`.

```

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <limits>
#include <vector>

template<class Matrix>
int lu_decompose(Matrix &a, std::vector<int> *p1col = NULL, const double EPS = 1e-10) {
    int r = a.size(), c = a[0].size(), parity = 0;
    if (p1col != NULL) {
        p1col->resize(r);
        for (int i = 0; i < r; i++) {
            (*p1col)[i] = i;
        }
    }
    for (int i = 0; i < r && i < c; i++) {
        int pi = i;
        for (int k = i + 1; k < r; k++) {
            if (fabs(a[k][i]) > fabs(a[pi][i])) {
                pi = k;
            }
        }
        if (fabs(a[pi][i]) < EPS) {
            return -1;
        }
        if (pi != i) {
            if (p1col != NULL) {
                std::iter_swap(p1col->begin() + i, p1col->begin() + pi);
            }
            std::iter_swap(a.begin() + i, a.begin() + pi);
            parity = 1 - parity;
        }
        for (int j = i + 1; j < r; j++) {
            a[j][i] /= a[i][i];
            for (int k = i + 1; k < c; k++) {
                a[j][k] -= a[j][i] * a[i][k];
            }
        }
    }
    return parity;
}

template<class Matrix>
double getl(const Matrix &lu, int i, int j) {
    return i > j ? lu[i][j] : (i < j ? 0 : 1);
}

template<class Matrix>
double getu(const Matrix &lu, int i, int j) {
    return i <= j ? lu[i][j] : 0;
}

template<class Matrix, class T>
int solve_system(
    const Matrix &a, const std::vector<T> &b, std::vector<T> *x, const double EPS = 1e-10
) {
    int r = a.size(), c = a[0].size();
    if (x == NULL || a.empty() || a.size() != b.size() || r < c) {
        return -1;
    }
    x->resize(c);
    std::vector<int> p1col;
    Matrix lu;
    int status = lu_decompose(lu = a, &p1col, EPS);
    if (status < 0) {
        return status;
    }
    for (int i = 0; i < c; i++) {

```



```

    (*x)[i] = b[picol[i]];
    for (int k = 0; k < i; k++) {
        (*x)[i] -= getl(lu, i, k) * (*x)[k];
    }
}
for (int i = c - 1; i >= 0; i--) {
    for (int k = i + 1; k < c; k++) {
        (*x)[i] -= getu(lu, i, k) * (*x)[k];
    }
    (*x)[i] /= getu(lu, i, i);
}
for (int i = 0; i < r; i++) {
    double val = 0;
    for (int j = 0; j < c; j++) {
        val += a[i][j] * (*x)[j];
    }
    if (fabs(val - b[i]) / b[i] > EPS) {
        return -1;
    }
}
return 0;
}

template<class SquareMatrix>
double det(const SquareMatrix &a) {
    int n = a.size();
    SquareMatrix lu;
    int status = lu_decompose(lu = a);
    if (status < 0) {
        return 0;
    }
    double res = 1;
    for (int i = 0; i < n; i++) {
        res *= lu[i][i];
    }
    return status == 0 ? res : -res;
}

template<class SquareMatrix>
int invert(SquareMatrix &a) {
    int n = a.size();
    std::vector<int> pcol;
    int status = lu_decompose(a, &pcol);
    if (status < 0) {
        return status;
    }
    SquareMatrix ia(n, typename SquareMatrix::value_type(n, 0));
    for (int j = 0; j < n; j++) {
        for (int i = 0; i < n; i++) {
            if (pcol[i] == j) {
                ia[i][j] = 1.0;
            } else {
                ia[i][j] = 0.0;
            }
            for (int k = 0; k < i; k++) {
                ia[i][j] -= getl(a, i, k) * ia[k][j];
            }
        }
        for (int i = n - 1; i >= 0; i--) {
            for (int k = i + 1; k < n; k++) {
                ia[i][j] -= getu(a, i, k) * ia[k][j];
            }
            ia[i][j] /= getu(a, i, i);
        }
    }
    a.swap(ia);
    return 0;
}

```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    { // Solve a system.
        const int equations = 3, unknowns = 3;
        const int a[equations][unknowns] = {{-1, 2, 5}, {1, 0, -6}, {-4, 2, 2}};
        const int b[equations] = {3, 1, -2};
        vector<vector<double>> m(equations);
        for (int i = 0; i < equations; i++) {
            m[i].assign(a[i], a[i] + unknowns);
        }
        vector<double> x;
        assert(solve_system(m, vector<double>(b, b + equations), &x) == 0);
        for (int i = 0; i < equations; i++) {
            double sum = 0;
            for (int j = 0; j < unknowns; j++) {
                sum += a[i][j] * x[j];
            }
            assert(fabs(sum - b[i]) < 1e-10);
        }
    }
    { // Find the determinant.
        const int n = 3, a[n][n] = {{1, 3, 5}, {2, 4, 7}, {1, 1, 0}};
        vector<vector<double>> m(n);
        for (int i = 0; i < n; i++) {
            m[i] = vector<double>(a[i], a[i] + n);
        }
        assert(fabs(det(m) - 4) < 1e-10);
    }
    { // Find the inverse.
        const int n = 3, a[n][n] = {{6, 1, 1}, {4, -2, 5}, {2, 8, 7}};
        vector<vector<double>> m(n), res(n, vector<double>(n, 0));
        for (int i = 0; i < n; i++) {
            m[i] = vector<double>(a[i], a[i] + n);
        }
        assert(invert(m) == 0);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                for (int k = 0; k < n; k++) {
                    res[i][j] += a[i][k] * m[k][j];
                }
            }
        }
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                assert(fabs(res[i][j] - (i == j ? 1 : 0)) < 1e-10);
            }
        }
    }
    return 0;
}
```

6.6 Root Finding and Calculus

6.6.1 Root Finding (Bracketing)

Finds an x in an interval $[a, b]$ for a continuous function f such that $f(x) = 0$. By the intermediate value theorem, a root must exist in $[a, b]$ if the signs of $f(a)$ and $f(b)$ differ. The answer is found with an absolute error of roughly $1/2^n$, where n is the number of iterations. Although it is possible to control the error by looping while

$b - a$ is greater than an arbitrary epsilon, it is simpler to let the loop run for a desired number of iterations until floating point arithmetic break down. 100 iterations is usually sufficient, since the search space will be reduced to 2^{-100} (roughly 10^{-30}) times its original size.

- `bisection_root(f, a, b)` returns a root in an interval $[a, b]$ for a continuous function f where $\text{sgn}(f(a)) \neq \text{sgn}(f(b))$, using the bisection method.
- `falsi_illinois_root(f, a, b)` returns a root in an interval $[a, b]$ for a continuous function f where $\text{sgn}(f(a)) \neq \text{sgn}(f(b))$, using the Illinois algorithm variant of the false position (a.k.a. regula falsi) method.

Time Complexity: $O(n)$ calls will be made to `f()` in `bisection_root()` and `falsi_illinois_root()`, where n is the number of iterations performed.

Space Complexity: $O(1)$ auxiliary space for both operations.

```
#include <stdexcept>

template<class ContinuousFunction>
double bisection_root(ContinuousFunction f, double a, double b, const int ITERATIONS = 100) {
    if (a > b || f(a) * f(b) > 0) {
        throw std::runtime_error("Must give [a, b] where sgn(f(a)) != sgn(f(b)).");
    }
    double m;
    for (int i = 0; i < ITERATIONS; i++) {
        m = a + (b - a) / 2;
        if (f(a) * f(m) >= 0) {
            a = m;
        } else {
            b = m;
        }
    }
    return m;
}

template<class ContinuousFunction>
double falsi_illinois_root(ContinuousFunction f, double a, double b, const int ITERATIONS = 100) {
    if (a > b || f(a) * f(b) > 0) {
        throw std::runtime_error("Must give [a, b] where sgn(f(a)) != sgn(f(b)).");
    }
    double m, fm, fa = f(a), fb = f(b);
    int side = 0;
    for (int i = 0; i < ITERATIONS; i++) {
        m = (fa * b - fb * a) / (fa - fb);
        fm = f(m);
        if (fb * fm > 0) {
            b = m;
            fb = fm;
            if (side < 0) {
                fa /= 2;
            }
            side = -1;
        } else if (fa * fm > 0) {
            a = m;
            fa = fm;
            if (side > 1) {
                fb /= 2;
            }
            side = 1;
        } else {
            break;
        }
    }
    return m;
}
```

Example Usage

```

#include <cassert>
#include <cmath>

double f(double x) {
    return x * x - 4 * sin(x);
}

int main() {
    assert(fabs(f(bisection_root(f, 1, 3))) < 1e-10);
    assert(fabs(f(falsi_illinois_root(f, 1, 3))) < 1e-10);
    return 0;
}

```

6.6.2 Root Finding (Iteration)

Finds an x for a continuous function f such that $f(x) = 0$ using iterative approximation by an initial guess that is close to the answer. Newton's method requires an explicit definition of the function's derivative while the secant method starts with two initial guesses and approximates the derivative using the secant slope from the previous iteration. For n iterations and a good initial guess, the methods below compute approximately 2^n digits of precision, with the secant method converging approximately 1.6 times slower than Newton's.

- `newton_root(f, fprime, x0)` returns a root x for a function `f` with derivative `fprime` using an initial guess `x0` which should be relatively close to x .
- `secant_root(f, x0, x1)` returns a root x for a function `f` using two initial guesses `x0` and `x1` which should be relatively close to x .

Time Complexity: $O(n)$ calls will be made to `f()` in `newton_root()` and `secant_root()`, where n is the number of iterations performed.

Space Complexity: $O(1)$ auxiliary space for both operations.

```

#include <cmath>
#include <stdexcept>

template<class ContinuousFunction>
double newton_root(
    ContinuousFunction f, ContinuousFunction fprime, double x0, const double EPS = 1e-15,
    const int ITERATIONS = 100
) {
    double x = x0, error = EPS + 1;
    for (int i = 0; error > EPS && i < ITERATIONS; i++) {
        double xnew = x - f(x) / fprime(x);
        error = fabs(xnew - x);
        x = xnew;
    }
    if (error > EPS) {
        throw std::runtime_error("Newton's method failed to converge.");
    }
    return x;
}

template<class ContinuousFunction>
double secant_root(
    ContinuousFunction f, double x0, double x1, const double EPS = 1e-15, const int ITERATIONS = 100
) {
    double xold = x0, fxold = f(x0), x = x1, error = EPS + 1;
    for (int i = 0; error > EPS && i < ITERATIONS; i++) {
        double fx = f(x);
        double xnew = x - fx * ((x - xold) / (fx - fxold));
        xold = x;
        fxold = fx;
        error = fabs(xnew - x);
        x = xnew;
    }
}

```

```

}
if (error > EPS) {
    throw std::runtime_error("Secant method failed to converge.");
}
return x;
}

```

Example Usage

```

#include <cassert>

double f(double x) {
    return x * x - 4 * sin(x);
}

double fprime(double x) {
    return 2 * x - 4 * cos(x);
}

int main() {
    assert(fabs(f(newton_root(f, fprime, 3))) < 1e-10);
    assert(fabs(f(secant_root(f, 3, 2))) < 1e-10);
    return 0;
}

```

6.6.3 Polynomial Root Finding (Differentiation)

Finds every root x for a polynomial p such that $p(x) = 0$ by differentiation. Each adjacent pair of local extrema is searched using the bisection method, where local extrema are recursively found by finding the root of the derivative.

- `horner_eval(p, x)` evaluates the polynomial `p` of degree d (represented as a vector of size $d + 1$ where `p[i]` stores the coefficient for the x^i term) at `x`, using Horner's method.
- `find_one_root(p, a, b, EPS)` returns a root in the interval $[a, b]$ for a polynomial `p` where $\text{sgn}(f(a)) \neq \text{sgn}(f(b))$, using the bisection method. If this precondition is not satisfied, then `NaN` is returned. The root is found to a tolerance of `EPS` in absolute or relative error (whichever is reached first).
- `find_all_roots(p, a, b, EPS)` returns a vector of all roots in the interval $[a, b]$ for a polynomial `p` using the bisection method. The roots are found to a tolerance of `EPS` in absolute or relative error (whichever is reached first).

Time Complexity:

- $O(n)$ per call to `horner_eval()`, where n is the degree of the polynomial.
- $O(n \log p)$ per call to `find_one_root()`, where n is the degree of the polynomial and $p = -\log_{10}(\text{EPS})$ is the number of digits of absolute or relative precision that is desired.
- $O(n^3 \log p)$ per call to `find_all_roots()`, where n is the degree of the polynomial and $p = -\log_{10}(\text{EPS})$ is the number of digits of absolute or relative precision that is desired.

Space Complexity:

- $O(1)$ auxiliary space for `horner_eval()` and `find_one_root()`.
- $O(n^2)$ auxiliary heap and $O(n)$ auxiliary stack space for `find_all_roots()`, where n is the degree of the polynomial.

```

#include <cmath>
#include <limits>
#include <utility>
#include <vector>

double horner_eval(const std::vector<double> &p, double x) {

```

```

double res = p.back();
for (int i = (int)p.size() - 2; i >= 0; i--) {
    res = res * x + p[i];
}
return res;
}

double find_one_root(const std::vector<double> &p, double a, double b, const double EPS = 1e-15) {
    double pa = horner_eval(p, a), pb = horner_eval(p, b);
    bool paneg = pa < 0, pbneg = pb < 0;
    if (paneg == pbneg) {
        return std::numeric_limits<double>::quiet_NaN();
    }
    while (b - a > EPS && a * (1 + EPS) < b && a < b * (1 + EPS)) {
        double m = a + (b - a) / 2;
        if ((horner_eval(p, m) < 0) == paneg) {
            a = m;
        } else {
            b = m;
        }
    }
    return a;
}

std::vector<double> find_all_roots(
    const std::vector<double> &p, double a = -1e20, double b = 1e20, const double EPS = 1e-15
) {
    std::vector<double> pprime;
    for (int i = 1; i < (int)p.size(); i++) {
        pprime.push_back(p[i] * i);
    }
    if (pprime.empty()) {
        return std::vector<double>();
    }
    std::vector<double> res, r = find_all_roots(pprime, a, b, EPS);
    r.push_back(b);
    for (int i = 0; i < (int)r.size(); i++) {
        double root = find_one_root(p, i == 0 ? a : r[i - 1], r[i], EPS);
        if (!std::isnan(root) && (res.empty() || root != res.back())) {
            res.push_back(root);
        }
    }
    return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    { // -1 + 2x - 6x^2 + 2x^3
        int poly[] = {-1, 2, -6, 2};
        vector<double> p(poly, poly + 4), roots = find_all_roots(p);
        assert(roots.size() == 1 && fabs(horner_eval(p, roots[0])) < 1e-10);
    }
    { // -20 + 4x + 3x^2
        int poly[] = {-20, 4, 3};
        vector<double> p(poly, poly + 3), roots = find_all_roots(p);
        assert(roots.size() == 2);
        assert(fabs(horner_eval(p, roots[0])) < 1e-10);
        assert(fabs(horner_eval(p, roots[1])) < 1e-10);
    }
    return 0;
}

```

6.6.4 Polynomial Root Finding (Laguerre)

Finds every complex root x for a polynomial p with complex coefficients such that $p(x) = 0$ using Laguerre's method.

- `horner_eval(p, x)` evaluates the complex polynomial p of degree d (represented as a vector of size $d + 1$ where `p[i]` stores the complex coefficient for the x^i term) at x , using Horner's method, returning a pair where the first value is a vector of sub-evaluations and the second value is the final result $p(x)$.
- `find_one_root(p, x0)` returns a complex root x for a polynomial p (represented as a vector of size $d + 1$ where `p[i]` stores the complex coefficient for the x^i term) using an initial guess x_0 which should be relatively close to x . The root is found to a tolerance of `EPS` in absolute or relative error (whichever is reached first).
- `find_all_roots(p)` returns a vector of all complex roots for a complex polynomial p . The roots are found to a tolerance of `EPS` in absolute or relative error (whichever is reached first).

Time Complexity:

- $O(n)$ per call to `horner_eval()`, where n is the degree of the polynomial.
- $O(n \log p)$ per call to `find_one_root()`, where n is the degree of the polynomial and $p = -\log_{10}(\text{EPS})$ is the number of digits of absolute or relative precision that is desired.
- $O(n^2 \log p)$ per call to `find_all_roots()`, where n is the degree of the polynomial and $p = -\log_{10}(\text{EPS})$ is the number of digits of absolute or relative precision that is desired.

Space Complexity:

- $O(n)$ auxiliary heap space and $O(1)$ auxiliary stack space for `horner_eval()` and `find_one_root()`, where n is the degree of the polynomial.
- $O(n)$ auxiliary heap and $O(1)$ auxiliary stack space per for `find_one_root()` and `find_all_roots()`, where n is the degree of the polynomial.

```
#include <complex>
#include <cstdlib>
#include <vector>

typedef std::complex<double> cdouble;
typedef std::vector<cdouble> cpoly;

std::pair<cdouble, cpoly> horner_eval(const cpoly &p, const cdouble &x) {
    int n = p.size();
    cpoly b(std::max(1, n - 1));
    for (int i = n - 1; i > 0; i--) {
        b[i - 1] = p[i] + (i < n - 1 ? b[i] * x : 0);
    }
    return std::make_pair(p[0] + b[0] * x, b);
}

cpoly derivative(const cpoly &p) {
    int n = p.size();
    cpoly res(std::max(1, n - 1));
    for (int i = 1; i < n; i++) {
        res[i - 1] = p[i] * cdouble(i);
    }
    return res;
}

int comp(const cdouble &a, const cdouble &b, const double EPS = 1e-15) {
    double diff = std::abs(a) - std::abs(b);
    return (diff < -EPS) ? -1 : (diff > EPS ? 1 : 0);
}

cdouble find_one_root(
    const cpoly &p, const cdouble &x0, const double EPS = 1e-15, const int ITERATIONS = 10000
) {
    cdouble x = x0;
    int n = p.size() - 1;

```

```

cpoly p1 = derivative(p), p2 = derivative(p1);
for (int i = 0; i < ITERATIONS; i++) {
    cdouble y0 = horner_eval(p, x).first;
    if (comp(y0, 0, EPS) == 0) {
        break;
    }
    cdouble g = horner_eval(p1, x).first / y0;
    cdouble h = g * g - horner_eval(p2, x).first / y0;
    cdouble r = std::sqrt(cdouble(n - 1) * (h * cdouble(n) - g * g));
    cdouble d1 = g + r, d2 = g - r;
    cdouble a = cdouble(n) / (comp(d1, d2, EPS) > 0 ? d1 : d2);
    x -= a;
    if (comp(a, 0, EPS) == 0) {
        break;
    }
}
return x;
}

std::vector<cdouble> find_all_roots(
    const cpoly &p, const double EPS = 1e-15, const int ITERATIONS = 10000
) {
    std::vector<cdouble> res;
    cpoly q = p;
    while (q.size() > 2) {
        cdouble z = cdouble(rand(), rand()) / (double)RAND_MAX;
        z = find_one_root(p, find_one_root(q, z, EPS, ITERATIONS), EPS, ITERATIONS);
        q = horner_eval(q, z).second;
        res.push_back(z);
    }
    res.push_back(-q[0] / q[1]);
    return res;
}

```

Example Usage

```

#include <cstdio>
#include <iostream>
using namespace std;

void print_roots(const vector<cdouble> &x) {
    for (int i = 0; i < (int)x.size(); i++) {
        printf("(%.5lf, %.5lf)\n", x[i].real(), x[i].imag());
    }
}

int main() {
    { // 140 - 13x - 8x^2 + x^3 = (x + 4)(x - 5)(x - 7)
        printf("Roots of 140 - 13x - 8x^2 + x^3:\n");
        cpoly p;
        p.push_back(140);
        p.push_back(-13);
        p.push_back(-8);
        p.push_back(1);
        print_roots(find_all_roots(p));
    }
    { // (-24+36i) + (-26+12i)x + (-30+40i)x^2 + (-26+12i)x^3 + (-6+4i)x^4
      // = ((2 + 3i)x + 6)(x + i)(2x + (6 + 4i))(xi + 1):
        printf("Roots of ((2 + 3i)x + 6)(x + i)(2x + (6 + 4i))(xi + 1):\n");
        cpoly p;
        p.push_back(cdouble(-24, 36));
        p.push_back(cdouble(-26, 12));
        p.push_back(cdouble(-30, 40));
        p.push_back(cdouble(-26, 12));
        p.push_back(cdouble(-6, 4));
        print_roots(find_all_roots(p));
    }
}

```



```

}
return 0;
}

```

Example Output

```

Roots of 140 - 13x - 8x^2 + x^3:
(5.00000, 0.00000)
(-4.00000, -0.00000)
(7.00000, -0.00000)
Roots of ((2 + 3i)x + 6)(x + i)(2x + (6 + 4i))(xi + 1):
(0.00000, 1.00000)
(0.00000, -1.00000)
(-0.92308, 1.38462)
(-3.00000, -2.00000)

```

6.6.5 Polynomial Root Finding (Ehrlich-Aberth)

Finds every complex root x for a polynomial p such that $p(x) = 0$ using the Ehrlich-Aberth method. This is a simultaneous iteration: every root estimate is updated using both the Newton correction and a repulsion term from the other estimates.

This routine is intended for well-scaled polynomials when all complex roots are wanted. If only real roots are needed, a real-root isolator such as derivative recursion with bisection is usually more reliable. Multiple or tightly clustered roots may converge slowly, and very large or very small root scales may require rescaling the input or using multiprecision arithmetic.

- `eval_with_derivative(p, x)` returns `p(x)` and `p'(x)` for a polynomial `p` represented as a vector where `p[i]` stores the coefficient for the x^i term.
- `find_all_roots(p, EPS, ITERATIONS)` returns a vector of all complex roots for a complex polynomial `p`. A `vector<LD>` overload is provided for polynomials with real coefficients. The roots are found to a tolerance of `EPS` in absolute or relative error (whichever is reached first), and zero roots are removed exactly before the simultaneous iteration starts.

Time Complexity:

- $O(n)$ per call to `eval_with_derivative(p, x)`, where n is the degree of the polynomial.
- $O(n^2p)$ per call to `find_all_roots(p, EPS, ITERATIONS)`, where n is the degree of the polynomial and p is the number of iterations required to reach the desired precision.

Space Complexity:

- $O(1)$ auxiliary space for `eval_with_derivative(p, x)`.
- $O(n)$ auxiliary heap space for `find_all_roots(p, EPS, ITERATIONS)`, where n is the degree of the polynomial.

```

#include <algorithm>
#include <cmath>
#include <complex>
#include <vector>

typedef long double LD;
typedef std::complex<LD> cdouble;
typedef std::vector<cdouble> cpoly;

const LD PI = acos(-1.0L);
const LD ZERO_EPS = 1e-30L; // Treat coefficients and denominators this small as zero.
const LD ROOT_EPS = 1e-18L; // Stop once root updates are this small relative to the root.
const LD CLEAN_EPS = 1e-12L; // Round tiny real or imaginary output parts down to zero.
const LD CHECK_EPS = 1e-12L; // Residual tolerance used by the example assertions.

bool is_zero(const cdouble &z, const LD EPS = ZERO_EPS) {
    return std::abs(z) <= EPS;
}

```

```

bool is_finite(const cdouble &z) {
    return std::isfinite((double)z.real()) && std::isfinite((double)z.imag());
}

std::pair<cdouble, cdouble> eval_with_derivative(const cpoly &p, const cdouble &x) {
    cdouble value = p.back(), derivative = 0;
    for (int i = (int)p.size() - 2; i >= 0; i--) {
        derivative = derivative * x + value;
        value = value * x + p[i];
    }
    return std::make_pair(value, derivative);
}

LD root_bound(const cpoly &p) {
    LD res = 0;
    int n = p.size() - 1;
    for (int i = 0; i + 1 < (int)p.size(); i++) {
        LD ratio = std::abs(p[i] / p.back());
        if (ratio > 0) {
            res = std::max(res, powl(ratio, 1.0L / (n - i)));
        }
    }
    return 2 * std::max((LD)1, res);
}

bool root_less(const cdouble &a, const cdouble &b) {
    if (a.real() != b.real()) {
        return a.real() < b.real();
    }
    return a.imag() < b.imag();
}

cdouble clean_root(const cdouble &z) {
    LD real = fabs1(z.real()) < CLEAN_EPS ? 0 : z.real();
    LD imag = fabs1(z.imag()) < CLEAN_EPS ? 0 : z.imag();
    return cdouble(real, imag);
}

cpoly find_all_roots(cpoly p, const LD EPS = ROOT_EPS, const int ITERATIONS = 2000) {
    while (!p.empty() && is_zero(p.back())) {
        p.pop_back();
    }
    cpoly roots;
    while (p.size() > 1 && is_zero(p[0])) {
        roots.push_back(0);
        p.erase(p.begin());
    }
    if (p.size() <= 1) {
        return roots;
    }
    LD scale = 0;
    for (int i = 0; i < (int)p.size(); i++) {
        scale = std::max(scale, std::abs(p[i]));
    }
    for (int i = 0; i < (int)p.size(); i++) {
        p[i] /= scale;
    }
    int n = p.size() - 1;
    if (n == 1) {
        roots.push_back(-p[0] / p[1]);
        return roots;
    }
    cpoly z(n);
    LD radius = root_bound(p), offset = PI / (2 * n);
    LD max_radius = 2 * radius;
    for (int i = 0; i < n; i++) {
        LD angle = offset + 2 * PI * i / n;

```

```

    z[i] = radius * cdouble(cosl(angle), sinl(angle));
}
for (int it = 0; it < ITERATIONS; it++) {
    bool done = true;
    cpoly next = z;
    for (int i = 0; i < n; i++) {
        std::pair<cdouble, cdouble> ev = eval_with_derivative(p, z[i]);
        if (std::abs(ev.first) <= EPS) {
            continue;
        }
        cdouble repulsion = 0;
        for (int j = 0; j < n; j++) {
            if (i != j) {
                cdouble diff = z[i] - z[j];
                if (std::abs(diff) <= EPS * EPS) {
                    LD angle = 2 * PI * (i + 1) / (n + 1);
                    diff = EPS * (1 + std::abs(z[i])) * cdouble(cosl(angle), sinl(angle));
                }
                repulsion += cdouble(1) / diff;
            }
        }
        cdouble denom = ev.second - ev.first * repulsion;
        if (is_zero(denom)) {
            continue;
        }
        cdouble step = ev.first / denom;
        if (!is_finite(step) && !is_zero(ev.second)) {
            step = ev.first / ev.second;
        }
        if (!is_finite(step)) {
            continue;
        }
        LD limit = 2 * max_radius;
        if (std::abs(step) > limit) {
            step *= limit / std::abs(step);
        }
        next[i] = z[i] - step;
        if (!is_finite(next[i])) {
            next[i] = z[i];
            continue;
        }
        if (std::abs(next[i]) > max_radius) {
            next[i] *= max_radius / std::abs(next[i]);
        }
        if (std::abs(step) > EPS * (1 + std::abs(next[i]))) {
            done = false;
        }
    }
    z = next;
    if (done) {
        break;
    }
}
for (int i = 0; i < n; i++) {
    roots.push_back(clean_root(z[i]));
}
std::sort(roots.begin(), roots.end(), root_less);
return roots;
}

std::vector<cdouble> find_all_roots(
    const std::vector<LD> &p, const LD EPS = ROOT_EPS, const int ITERATIONS = 2000
) {
    cpoly q(p.begin(), p.end());
    return find_all_roots(q, EPS, ITERATIONS);
}

```

Example Usage

```
#include <cassert>
#include <cstdio>
using namespace std;

cdouble eval(const cpoly &p, const cdouble &x) {
    return eval_with_derivative(p, x).first;
}

void print_roots(const vector<cdouble> &x) {
    for (int i = 0; i < (int)x.size(); i++) {
        printf("(%.5Lf, %.5Lf)\n", x[i].real(), x[i].imag());
    }
}

int main() {
    { // -1 + 2x - 6x^2 + 2x^3
        printf("Roots of -1 + 2x - 6x^2 + 2x^3:\n");
        LD poly[] = {-1, 2, -6, 2};
        vector<LD> p(poly, poly + 4);
        vector<cdouble> roots = find_all_roots(p);
        assert(roots.size() == 3);
        for (int i = 0; i < (int)roots.size(); i++) {
            assert(abs(eval(cpoly(p.begin(), p.end()), roots[i])) < CHECK_EPS);
        }
        print_roots(roots);
    }
    { // (-24+36i) + (-26+12i)x + (-30+40i)x^2 + (-26+12i)x^3 + (-6+4i)x^4
      // = ((2 + 3i)x + 6)(x + i)(2x + (6 + 4i))(xi + 1):
        printf("Roots of ((2 + 3i)x + 6)(x + i)(2x + (6 + 4i))(xi + 1):\n");
        cpoly p;
        p.push_back(cdouble(-24, 36));
        p.push_back(cdouble(-26, 12));
        p.push_back(cdouble(-30, 40));
        p.push_back(cdouble(-26, 12));
        p.push_back(cdouble(-6, 4));
        vector<cdouble> roots = find_all_roots(p);
        assert(roots.size() == 4);
        for (int i = 0; i < (int)roots.size(); i++) {
            assert(abs(eval(p, roots[i])) < CHECK_EPS);
        }
        print_roots(roots);
    }
    return 0;
}
```

Example Output

```
Roots of -1 + 2x - 6x^2 + 2x^3:
(0.15098, -0.40314)
(0.15098, 0.40314)
(2.69805, 0.00000)
Roots of ((2 + 3i)x + 6)(x + i)(2x + (6 + 4i))(xi + 1):
(-3.00000, -2.00000)
(-0.92308, 1.38462)
(0.00000, -1.00000)
(0.00000, 1.00000)
```

6.6.6 Integration (Simpson)

Computes the definite integral from a to b for a continuous function f using Simpson's approximation:
 $\text{integral} \sim [f(a) + 4f((a+b)/2) + f(b)](b-a)/6$.

- `simpsons(f, a, b)` returns the definite integral for a function f from a to b , to a tolerance of `EPS` in absolute error.

Time Complexity: $O(p)$ per call to `integrate()`, where $p = -\log_{10}(\text{EPS})$ is the number of digits of absolute precision that is desired.

Space Complexity: $O(p)$ auxiliary stack and $O(1)$ auxiliary heap space, where $p = -\log_{10}(\text{EPS})$ is the number of digits of absolute precision that is desired.

```
#include <cmath>

template<class ContinuousFunction>
double simpsons(ContinuousFunction f, double a, double b) {
    return (f(a) + 4 * f((a + b) / 2) + f(b)) * (b - a) / 6;
}

template<class ContinuousFunction>
double integrate(ContinuousFunction f, double a, double b, const double EPS = 1e-15) {
    double m = (a + b) / 2;
    double am = simpsons(f, a, m);
    double mb = simpsons(f, m, b);
    double ab = simpsons(f, a, b);
    if (fabs(am + mb - ab) < EPS) {
        return ab;
    }
    return integrate(f, a, m) + integrate(f, m, b);
}
```

Example Usage

```
#include <cassert>
#include <cstdio>
using namespace std;

double f(double x) {
    return sin(x);
}

int main() {
    double PI = acos(-1.0);
    assert(fabs(integrate(f, 0.0, PI / 2) - 1) < 1e-10);
    return 0;
}
```

6.7 Game Theory

6.7.1 Nim

Solves normal-play Nim and related impartial games where each position is the disjoint sum of independent piles. In normal play, the player who makes the last move wins. A Nim position is losing exactly when the xor of all pile sizes is 0.

This is the base case of the Sprague-Grundy theorem: every finite impartial game position is equivalent to a Nim pile of size equal to its Grundy number, and sums of games combine by xor.

- `nim_sum(piles)` returns the xor of all pile sizes.
- `first_player_wins(piles)` returns whether the player to move has a winning strategy.
- `winning_move(piles)` returns a move `(pile, new_size)` that moves to xor 0, or `(-1, -1)` if the position is losing.

Time Complexity: $O(n)$ per call, where n is the number of piles.

Space Complexity: $O(1)$ auxiliary.

```
#include <utility>
#include <vector>

int nim_sum(const std::vector<int> &piles) {
    int x = 0;
    for (int i = 0; i < (int)piles.size(); i++) {
        x ^= piles[i];
    }
    return x;
}

bool first_player_wins(const std::vector<int> &piles) {
    return nim_sum(piles) != 0;
}

std::pair<int, int> winning_move(const std::vector<int> &piles) {
    int x = nim_sum(piles);
    if (x == 0) {
        return std::make_pair(-1, -1);
    }
    for (int i = 0; i < (int)piles.size(); i++) {
        int target = piles[i] ^ x;
        if (target < piles[i]) {
            return std::make_pair(i, target);
        }
    }
    return std::make_pair(-1, -1);
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    int raw[] = {3, 4, 5};
    vector<int> piles(raw, raw + 3);
    assert(nim_sum(piles) == 2);
    assert(first_player_wins(piles));
    pair<int, int> move = winning_move(piles);
    piles[move.first] = move.second;
    assert(nim_sum(piles) == 0);

    int losing_raw[] = {1, 2, 3};
    vector<int> losing(losing_raw, losing_raw + 3);
    assert(!first_player_wins(losing));
    assert(winning_move(losing).first == -1);
    return 0;
}
```

6.7.2 Sprague-Grundy

Computes Grundy numbers for finite impartial games. The Grundy number $g[p]$ of a position p is the minimum excluded nonnegative integer (MEX) of the Grundy numbers reachable in one move. A position is losing exactly when its Grundy number is 0.

For a sum of independent impartial games, the combined Grundy number is the xor of the component Grundy numbers. This is the Sprague-Grundy theorem.

- `mex(values)` returns the minimum excluded nonnegative integer.
- `subtraction_game_grundy(max_stones, moves)` computes Grundy numbers for a game where a move subtracts one value in `moves` from the pile.
- `sum_grundy(grundies)` returns the xor of component Grundy numbers.

Time Complexity: $O(nm)$ for `subtraction_game_grundy(max_stones, moves)`, where n is `max_stones` and m is the number of allowed moves.

Space Complexity: $O(n + m)$ auxiliary heap space.

```
#include <vector>

int mex(const std::vector<int> &values) {
    std::vector<bool> seen(values.size() + 1, false);
    for (int i = 0; i < (int)values.size(); i++) {
        if (0 <= values[i] && values[i] < (int)seen.size()) {
            seen[values[i]] = true;
        }
    }
    for (int i = 0; i < (int)seen.size(); i++) {
        if (!seen[i]) {
            return i;
        }
    }
    return seen.size();
}

std::vector<int> subtraction_game_grundy(int max_stones, const std::vector<int> &moves) {
    std::vector<int> grundy(max_stones + 1, 0);
    for (int stones = 1; stones <= max_stones; stones++) {
        std::vector<int> reachable;
        for (int i = 0; i < (int)moves.size(); i++) {
            if (moves[i] <= stones) {
                reachable.push_back(grundy[stones - moves[i]]);
            }
        }
        grundy[stones] = mex(reachable);
    }
    return grundy;
}

int sum_grundy(const std::vector<int> &grundies) {
    int x = 0;
    for (int i = 0; i < (int)grundies.size(); i++) {
        x ^= grundies[i];
    }
    return x;
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    int moves_raw[] = {1, 3, 4};
    vector<int> moves(moves_raw, moves_raw + 3);
    vector<int> g = subtraction_game_grundy(10, moves);
    assert(g[0] == 0);
    assert(g[1] == 1);
    assert(g[2] == 0);
    assert(g[4] == 2);
    assert(g[7] == 0);

    int heaps_raw[] = {g[5], g[7], g[10]};
```

```

vector<int> heaps(heaps_raw, heaps_raw + 3);
assert(sum_grundy(heaps) == (g[5] ^ g[7] ^ g[10]));
return 0;
}

```

6.7.3 Grundy Numbers on DAG

Computes Grundy numbers for impartial games whose positions form a directed acyclic graph. Each vertex is a game position, and each outgoing edge is a legal move. Terminal vertices have Grundy number 0; all other vertices take the MEX of their successors' Grundy numbers.

The implementation uses memoized DFS. It assumes the graph is acyclic; if cycles are present, the usual finite impartial-game Grundy recurrence is not directly valid without additional analysis.

- `grundy_on_dag(g)` returns the Grundy number of every vertex in graph `g`.
- `g[u]` contains all positions reachable from position `u` in one move.

Time Complexity: $O(n + m + s)$, where n is the number of vertices, m is the number of edges, and s is the total cost of MEX sets over outgoing edges.

Space Complexity: $O(n + d)$ auxiliary heap and stack space, where d is DFS depth.

```

#include <vector>

int grundy_dfs(int u, const std::vector<std::vector<int>> &g, std::vector<int> *memo) {
    if ((*memo)[u] != -1) {
        return (*memo)[u];
    }
    std::vector<bool> seen(g[u].size() + 1, false);
    for (int i = 0; i < (int)g[u].size(); i++) {
        int v = g[u][i];
        int x = grundy_dfs(v, g, memo);
        if (x < (int)seen.size()) {
            seen[x] = true;
        }
    }
    int res = 0;
    while (res < (int)seen.size() && seen[res]) {
        res++;
    }
    (*memo)[u] = res;
    return res;
}

std::vector<int> grundy_on_dag(const std::vector<std::vector<int>> &g) {
    std::vector<int> memo(g.size(), -1);
    for (int u = 0; u < (int)g.size(); u++) {
        grundy_dfs(u, g, &memo);
    }
    return memo;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<vector<int>> g(5);
    g[0].push_back(1);
    g[0].push_back(2);
    g[1].push_back(3);
}

```



```

g[2].push_back(3);
g[2].push_back(4);
vector<int> Grundy = Grundy_on_dag(g);
assert(Grundy[3] == 0);
assert(Grundy[4] == 0);
assert(Grundy[1] == 1);
assert(Grundy[2] == 1);
assert(Grundy[0] == 0);
return 0;
}

```

6.7.4 Minimax and Alpha-Beta Pruning

Searches a finite two-player, zero-sum, perfect-information game tree using minimax and alpha-beta pruning. Minimax assumes both players play optimally: the maximizing player chooses the child with largest value, and the minimizing player chooses the child with smallest value.

Alpha-beta pruning computes the same value as minimax, but avoids exploring branches that cannot affect the final answer. It is most effective when good moves are searched first.

The example game is "take 1 or 2 stones": starting with `stones` stones, players alternate taking 1 or 2 stones, and the player who takes the last stone wins. The evaluation returns 1 for a win for the player to move at the root and -1 for a loss.

- `minimax(stones, maximizing)` returns the exact game-tree value.
- `alpha_beta(stones, maximizing, alpha, beta)` returns the same value, with pruning.
- `best_take(stones)` returns an optimal first move, either 1 or 2.

Time Complexity: $O(b^d)$ in the worst case, where b is branching factor and d is search depth. Alpha-beta can reduce this substantially with good move ordering.

Space Complexity: $O(d)$ auxiliary recursion stack space.

```

#include <algorithm>

int minimax(int stones, bool maximizing) {
    if (stones == 0) {
        return maximizing ? -1 : 1;
    }
    if (maximizing) {
        int best = -2;
        for (int take = 1; take <= 2 && take <= stones; take++) {
            best = std::max(best, minimax(stones - take, false));
        }
        return best;
    } else {
        int best = 2;
        for (int take = 1; take <= 2 && take <= stones; take++) {
            best = std::min(best, minimax(stones - take, true));
        }
        return best;
    }
}

int alpha_beta(int stones, bool maximizing, int alpha = -2, int beta = 2) {
    if (stones == 0) {
        return maximizing ? -1 : 1;
    }
    if (maximizing) {
        int value = -2;
        for (int take = 1; take <= 2 && take <= stones; take++) {
            value = std::max(value, alpha_beta(stones - take, false, alpha, beta));
            alpha = std::max(alpha, value);
        }
    }
}

```

```

        if (alpha >= beta) {
            break;
        }
    }
    return value;
} else {
    int value = 2;
    for (int take = 1; take <= 2 && take <= stones; take++) {
        value = std::min(value, alpha_beta(stones - take, true, alpha, beta));
        beta = std::min(beta, value);
        if (alpha >= beta) {
            break;
        }
    }
    return value;
}
}

int best_take(int stones) {
    int best_move = 1, best_value = -2;
    for (int take = 1; take <= 2 && take <= stones; take++) {
        int value = alpha_beta(stones - take, false);
        if (value > best_value) {
            best_value = value;
            best_move = take;
        }
    }
    return best_move;
}

```

Example Usage

```

#include <cassert>

int main() {
    assert(minimax(1, true) == 1);
    assert(minimax(3, true) == -1);
    for (int stones = 1; stones <= 10; stones++) {
        assert(minimax(stones, true) == alpha_beta(stones, true));
    }
    assert(best_take(4) == 1); // Move to losing state with 3 stones.
    assert(best_take(5) == 2); // Move to losing state with 3 stones.
    return 0;
}

```

Chapter 7

Geometry

7.1 Geometric Classes

7.1.1 Point

A two-dimensional real-valued point class supporting epsilon comparisons. Operations include element-wise arithmetic, `norm()`, `arg()`, `dot()`, `cross()`, `proj()`, `normalize()`, `rotate90()`, `rotateCW()`, `rotateCCW()`, and `reflect()`. See also `std::complex`.

Time Complexity: $O(1)$ per call to the constructor and all other operations.

Space Complexity:

- $O(1)$ for storage of the point.
- $O(1)$ auxiliary for all operations.

```
#include <cmath>
#include <ostream>
#include <utility>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LT(a, b) ((a) < (b) - EPS)

struct point {
    double x, y;

    point() : x(0), y(0) {}
    point(double x, double y) : x(x), y(y) {}
    point(const point &p) : x(p.x), y(p.y) {}
    point(const std::pair<double, double> &p) : x(p.first), y(p.second) {}

    bool operator<(const point &p) const { return EQ(x, p.x) ? LT(y, p.y) : LT(x, p.x); }
    bool operator>(const point &p) const { return EQ(x, p.x) ? LT(p.y, y) : LT(p.x, x); }
    bool operator==(const point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
    bool operator!=(const point &p) const { return !(*this == p); }
    bool operator<=(const point &p) const { return !(*this > p); }
    bool operator>=(const point &p) const { return !(*this < p); }
    point operator+(const point &p) const { return point(x + p.x, y + p.y); }
    point operator-(const point &p) const { return point(x - p.x, y - p.y); }
    point operator+(double v) const { return point(x + v, y + v); }
    point operator-(double v) const { return point(x - v, y - v); }
    point operator*(double v) const { return point(x * v, y * v); }
    point operator/(double v) const { return point(x / v, y / v); }
    point &operator+=(const point &p) { x += p.x; y += p.y; return *this; }
```

```

point &operator--=(const point &p) { x -= p.x; y -= p.y; return *this; }
point &operator+=(double v) { x += v; y += v; return *this; }
point &operator-=(double v) { x -= v; y -= v; return *this; }
point &operator*=(double v) { x *= v; y *= v; return *this; }
point &operator/=(double v) { x /= v; y /= v; return *this; }
friend point operator+(double v, const point &p) { return p + v; }
friend point operator*(double v, const point &p) { return p * v; }

double sqnorm() const { return x * x + y * y; }
double norm() const { return sqrt(x * x + y * y); }
double arg() const { return atan2(y, x); }
double dot(const point &p) const { return x * p.x + y * p.y; }
double cross(const point &p) const { return x * p.y - y * p.x; }
double proj(const point &p) const { return dot(p) / p.norm(); }

// Returns a proportional unit vector (p, q) = c(x, y) where p^2 + q^2 = 1.
point normalize() const { return (EQ(x, 0) && EQ(y, 0)) ? point(0, 0) : (point(x, y) / norm()); }

// Returns (x, y) rotated 90 degrees clockwise about the origin.
point rotate90() const { return point(-y, x); }

// Returns (x, y) rotated t radians clockwise about the origin.
point rotateCW(double t) const { return point(x * cos(t) + y * sin(t), y * cos(t) - x * sin(t)); }

// Returns (x, y) rotated t radians counter-clockwise about the origin.
point rotateCCW(double t) const {
    return point(x * cos(t) - y * sin(t), x * sin(t) + y * cos(t));
}

// Returns (x, y) rotated t radians clockwise about point p.
point rotateCW(const point &p, double t) const { return (*this - p).rotateCW(t) + p; }

// Returns (x, y) rotated t radians counter-clockwise about the point p.
point rotateCCW(const point &p, double t) const { return (*this - p).rotateCCW(t) + p; }

// Returns (x, y) reflected across point p.
point reflect(const point &p) const { return point(2 * p.x - x, 2 * p.y - y); }

// Returns (x, y) reflected across the line containing points p and q.
point reflect(const point &p, const point &q) const {
    if (p == q) {
        return reflect(p);
    }
    point r(*this - p), s = q - p;
    r = point(r.x * s.x + r.y * s.y, r.x * s.y - r.y * s.x) / s.sqnorm();
    r = point(r.x * s.x - r.y * s.y, r.x * s.y + r.y * s.x) + p;
    return r;
}

friend double sqnorm(const point &p) { return p.sqnorm(); }
friend double norm(const point &p) { return p.norm(); }
friend double arg(const point &p) { return p.arg(); }
friend double dot(const point &p, const point &q) { return p.dot(q); }
friend double cross(const point &p, const point &q) { return p.cross(q); }
friend double proj(const point &p, const point &q) { return p.proj(q); }
friend point normalize(const point &p) { return p.normalize(); }
friend point rotate90(const point &p) { return p.rotate90(); }
friend point rotateCW(const point &p, double t) { return p.rotateCW(t); }
friend point rotateCCW(const point &p, double t) { return p.rotateCCW(t); }
friend point rotateCW(const point &p, const point &q, double t) { return p.rotateCW(q, t); }
friend point rotateCCW(const point &p, const point &q, double t) { return p.rotateCCW(q, t); }
friend point reflect(const point &p, const point &q) { return p.reflect(q); }
friend point reflect(const point &p, const point &a, const point &b) { return p.reflect(a, b); }

friend std::ostream &operator<<(std::ostream &out, const point &p) {
    return out << "(" << (fabs(p.x) < EPS ? 0 : p.x) << ", " << (fabs(p.y) < EPS ? 0 : p.y) << ")";
}
};

```

Example Usage

```
#include <cassert>
#define pt point

const double PI = acos(-1.0);

int main() {
    pt p(-10, 3), q;
    assert(pt(-18, 29) == p + pt(-3, 9) * 6 / 2 - pt(-1, 1));
    assert(EQ(109, p.sqnorm()));
    assert(EQ(10.44030650891, p.norm()));
    assert(EQ(2.850135859112, p.arg()));
    assert(EQ(0, p.dot(pt(3, 10))));
    assert(EQ(0, p.cross(pt(10, -3))));
    assert(EQ(10, p.proj(pt(-10, 0))));
    assert(EQ(1, p.normalize().norm()));
    assert(pt(-3, -10) == p.rotate90());
    assert(pt(3, 12) == p.rotateCW(pt(1, 1), PI / 2));
    assert(pt(1, -10) == p.rotateCCW(pt(2, 2), PI / 2));
    assert(pt(10, -3) == p.reflect(pt(0, 0)));
    assert(pt(-10, -3) == p.reflect(pt(-2, 0), pt(5, 0)));
    return 0;
}
```

7.1.2 Line

A straight line in two dimensions supporting epsilon comparisons. The line is represented by the form $ax + by + c = 0$, where the coefficients are normalized so that b is always 1 except for when the line is vertical, in which case $b = 0$. Operations include `horizontal()`, `vertical()`, `slope()`, evaluating y at some x with `y()` (and vice versa with `x()`), checking if a point falls on the line with `contains()`, checking if another line is parallel or perpendicular with `is_parallel()` or `is_perpendicular()`, and finding the `parallel()` or `perpendicular()` line through a point.

Time Complexity: $O(1)$ per call to the constructor and all other operations.

Space Complexity:

- $O(1)$ for storage of the line.
- $O(1)$ auxiliary for all operations.

```
#include <cmath>
#include <limits>
#include <ostream>

const double EPS = 1e-9;
const double M_NAN = std::numeric_limits<double>::quiet_NaN();

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LT(a, b) ((a) < (b) - EPS)

struct line {
    double a, b, c;

    line() : a(0), b(0), c(0) {} // Invalid or uninitialized line.

    line(double a, double b, double c) {
        if (!EQ(b, 0)) {
            this->a = a / b;
            this->c = c / b;
            this->b = 1;
        } else {
            this->c = c / a;

```

```

        this->a = 1;
        this->b = 0;
    }
}

template<class Point>
line(double slope, const Point &p) {
    a = -slope;
    b = 1;
    c = slope * p.x - p.y;
}

template<class Point>
line(const Point &p, const Point &q) : a(0), b(0), c(0) {
    if (EQ(p.x, q.x)) {
        if (NE(p.y, q.y)) { // Vertical line.
            a = 1;
            b = 0;
            c = -p.x;
        } // Else, invalid line.
    } else {
        a = -(p.y - q.y) / (p.x - q.x);
        b = 1;
        c = -(a * p.x) - (b * p.y);
    }
}

bool operator==(const line &l) const { return EQ(a, l.a) && EQ(b, l.b) && EQ(c, l.c); }
bool operator!=(const line &l) const { return !(*this == l); }

// Returns whether the line is initialized and normalized.
bool valid() const {
    if (EQ(a, 0)) {
        return !EQ(b, 0);
    }
    return EQ(b, 1) || (EQ(b, 0) && EQ(a, 1));
}

bool horizontal() const { return valid() && EQ(a, 0); }
bool vertical() const { return valid() && EQ(b, 0); }
double slope() const { return (!valid() || EQ(b, 0)) ? M_NAN : -a; }

// Solve for x at a given y. If the line is horizontal, then either -INF, INF,
// or NAN is returned based on whether y is below, above, or on the line.
double x(double y) const {
    if (!valid() || EQ(a, 0)) {
        return M_NAN; // Invalid or horizontal line.
    }
    return (-c - b * y) / a;
}

// Solve for y at a given x. If the line is vertical, then either -INF, INF,
// or NAN is returned based on whether x is left of, right of, or on the line.
double y(double x) const {
    if (!valid() || EQ(b, 0)) {
        return M_NAN; // Invalid or vertical line.
    }
    return (-c - a * x) / b;
}

template<class Point>
bool contains(const Point &p) const {
    return EQ(a * p.x + b * p.y + c, 0);
}

bool is_parallel(const line &l) const { return EQ(a, l.a) && EQ(b, l.b); }
bool is_perpendicular(const line &l) const { return EQ(-a * l.a, b * l.b); }

```

```

// Return the parallel line passing through point p.
template<class Point>
line parallel(const Point &p) const {
    return line(a, b, -a * p.x - b * p.y);
}

// Return the perpendicular line passing through point p.
template<class Point>
line perpendicular(const Point &p) const {
    return line(-b, a, b * p.x - a * p.y);
}

friend std::ostream &operator<<(std::ostream &out, const line &l) {
    return out << (fabs(l.a) < EPS ? 0 : l.a) << "x" << std::showpos << (fabs(l.b) < EPS ? 0 : l.b)
        << "y" << (fabs(l.c) < EPS ? 0 : l.c) << "=0" << std::noshowpos;
}
};

```

Example Usage

```

#include <cassert>

struct point {
    double x, y;
    point(double x, double y) : x(x), y(y) {}
};

int main() {
    line l(2, -5, -8);
    line para = line(2, -5, -8).parallel(point(-6, -2));
    line perp = line(2, -5, -8).perpendicular(point(-6, -2));
    assert(l.is_parallel(para) && l.is_perpendicular(perp));
    assert(l.slope() == 0.4);
    assert(para == line(-0.4, 1, -0.4)); // -0.4x + y - 0.4 = 0.
    assert(perp == line(2.5, 1, 17));    // 2.5x + y + 17 = 0.
    return 0;
}

```

7.1.3 Circle

A circle in two dimensions supporting epsilon comparisons. The circle centered at (h, k) is represented by the relation $(x - h)^2 + (y - k)^2 = r^2$, where the radius r is normalized to a non-negative number. Operations include constructing a circle from a line segment, constructing a circumcircle, accessing `center()`, checking if a point falls inside the circle with `contains()` or on its edge with `on_edge()`, and constructing an incircle with `incircle()`.

Time Complexity: $O(1)$ per call to the constructors and all other operations.

Space Complexity:

- $O(1)$ for storage of the circle.
- $O(1)$ auxiliary for all operations.

```

#include <cmath>
#include <ostream>
#include <stdexcept>
#include <utility>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LE(a, b) ((a) <= (b) + EPS)
#define LT(a, b) ((a) < (b) - EPS)

```

```

typedef std::pair<double, double> point;
#define x first
#define y second

double sqnorm(const point &a) {
    return a.x * a.x + a.y * a.y;
}
double norm(const point &a) {
    return sqrt(sqnorm(a));
}

struct circle {
    double h, k, r;

    circle() : h(0), k(0), r(0) {}
    circle(double r) : h(0), k(0), r(fabs(r)) {}
    circle(const point &o, double r) : h(o.x), k(o.y), r(fabs(r)) {}
    circle(double h, double k, double r) : h(h), k(k), r(fabs(r)) {}

    // Circle with the line segment ab as a diameter.
    circle(const point &a, const point &b) {
        h = (a.x + b.x) / 2.0;
        k = (a.y + b.y) / 2.0;
        r = norm(point(a.x - h, a.y - k));
    }

    // Circumcircle of three points.
    circle(const point &a, const point &b, const point &c) {
        double an = sqnorm(point(b.x - c.x, b.y - c.y));
        double bn = sqnorm(point(a.x - c.x, a.y - c.y));
        double cn = sqnorm(point(a.x - b.x, a.y - b.y));
        double wa = an * (bn + cn - an);
        double wb = bn * (an + cn - bn);
        double wc = cn * (an + bn - cn);
        double w = wa + wb + wc;
        if (EQ(w, 0)) {
            throw std::runtime_error("No circumcircle from collinear points.");
        }
        h = (wa * a.x + wb * b.x + wc * c.x) / w;
        k = (wa * a.y + wb * b.y + wc * c.y) / w;
        r = norm(point(a.x - h, a.y - k));
    }

    // Circle of radius r that contains points a and b. In the general case, there
    // will be two possible circles and only one is chosen arbitrarily. However if
    // the diameter is equal to dist(a, b) = 2*r, then there is only one possible
    // center. If points a and b are identical, then there are infinite circles.
    // If the points are too far away relative to the radius, then there is no
    // possible circle. In the latter two cases, an exception is thrown.
    circle(const point &a, const point &b, double r) : r(fabs(r)) {
        if (LE(r, 0) && a == b) { // Circle with zero area.
            h = a.x;
            k = a.y;
            return;
        }
        double d = norm(point(b.x - a.x, b.y - a.y));
        if (EQ(d, 0)) {
            throw std::runtime_error("Identical points, infinite circles.");
        }
        if (LT(r * 2.0, d)) {
            throw std::runtime_error("Points too far away to make circle.");
        }
        double v = sqrt(r * r - d * d / 4.0) / d;
        point m((a.x + b.x) / 2.0, (a.y + b.y) / 2.0);
        h = m.x + v * (a.y - b.y);
        k = m.y + v * (b.x - a.x);
        // The other answer is (h, k) = (m.x - v*(a.y - b.y), m.y - v*(b.x - a.x)).
    }
}

```



```

}

bool operator==(const circle &c) const { return EQ(h, c.h) && EQ(k, c.k) && EQ(r, c.r); }
bool operator!=(const circle &c) const { return !(*this == c); }
point center() const { return point(h, k); }
bool contains(const point &p) const { return LE(sqnorm(point(p.x - h, p.y - k)), r * r); }
bool on_edge(const point &p) const { return EQ(sqnorm(point(p.x - h, p.y - k)), r * r); }

friend std::ostream &operator<<(std::ostream &out, const circle &c) {
    return out << std::showpos << "(" << (fabs(c.h) < EPS ? 0 : c.h) << ")^2+"
        << "(" << (fabs(c.k) < EPS ? 0 : c.k) << ")^2" << std::noshowpos << "="
        << (fabs(c.r) < EPS ? 0 : c.r * c.r);
}
};

// Returns the circle inscribed inside the triangle abc.
circle incircle(const point &a, const point &b, const point &c) {
    double al = norm(point(b.x - c.x, b.y - c.y));
    double bl = norm(point(a.x - c.x, a.y - c.y));
    double cl = norm(point(a.x - b.x, a.y - b.y));
    double l = al + bl + cl;
    point p(a.x - c.x, a.y - c.y), q(b.x - c.x, b.y - c.y);
    return EQ(l, 0) ? circle(a.x, a.y, 0)
        : circle(
            (al * a.x + bl * b.x + cl * c.x) / l, (al * a.y + bl * b.y + cl * c.y) / l,
            fabs(p.x * q.y - p.y * q.x) / l
        );
}

```

Example Usage

```

#include <cassert>

int main() {
    circle c(-2, 5, sqrt(10));
    assert(c == circle(point(-2, 5), sqrt(10)));
    assert(c == circle(point(1, 6), point(-5, 4)));
    assert(c == circle(point(-3, 2), point(-3, 8), point(-1, 8)));
    assert(c == incircle(point(-12, 5), point(3, 0), point(0, 9)));
    assert(c.contains(point(-2, 8)) && !c.contains(point(-2, 9)));
    assert(c.on_edge(point(-1, 2)) && !c.on_edge(point(-1.01, 2)));
    return 0;
}

```

7.1.4 Triangle

Common triangle calculations in two dimensions.

- `triangle_area(a, b, c)` returns the area of the triangle abc .
- `triangle_area_sides(s1, s2, s3)` returns the area of a triangle with side lengths s_1 , s_2 , and s_3 . The given lengths must be non-negative and form a valid triangle.
- `triangle_area_medians(m1, m2, m3)` returns the area of a triangle with medians of lengths m_1 , m_2 , and m_3 . The median of a triangle is a line segment joining a vertex to the midpoint of the opposing edge.
- `triangle_area_altitudes(h1, h2, h3)` returns the area of a triangle with altitudes h_1 , h_2 , and h_3 . An altitude of a triangle is the shortest line between a vertex and the infinite line that is extended from its opposite edge.
- `same_side(p1, p2, a, b)` returns whether points p_1 and p_2 lie on the same side of the line containing points a and b . If one or both points lie exactly on the line, then the result will depend on the setting of `EDGE_IS_SAME_SIDE`.
- `point_in_triangle(p, a, b, c)` returns whether point p lies within the triangle abc . If the point lies on or close to an edge (by roughly `EPS`), then the result will depend on the setting of `EDGE_IS_SAME_SIDE` in the

function above.

Time Complexity: $O(1)$ for all operations.

Space Complexity: $O(1)$ auxiliary for all operations.

```
#include <algorithm>
#include <cmath>
#include <utility>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LT(a, b) ((a) < (b) - EPS)
#define GT(a, b) ((a) > (b) + EPS)
#define LE(a, b) ((a) <= (b) + EPS)
#define GE(a, b) ((a) >= (b) - EPS)

typedef std::pair<double, double> point;
#define x first
#define y second

double cross(const point &a, const point &b) {
    return a.x * b.y - a.y * b.x;
}

double triangle_area(const point &a, const point &b, const point &c) {
    point ac(a.x - c.x, a.y - c.y), bc(b.x - c.x, b.y - c.y);
    return fabs(cross(ac, bc)) / 2.0;
}

double triangle_area_sides(double s1, double s2, double s3) {
    double s = (s1 + s2 + s3) / 2.0;
    return sqrt(s * (s - s1) * (s - s2) * (s - s3));
}

double triangle_area_medians(double m1, double m2, double m3) {
    return 4.0 * triangle_area_sides(m1, m2, m3) / 3.0;
}

double triangle_area_altitudes(double h1, double h2, double h3) {
    if (EQ(h1, 0) || EQ(h2, 0) || EQ(h3, 0)) {
        return 0;
    }
    double x = h1 * h1, y = h2 * h2, z = h3 * h3;
    double v = 2.0 / (x * y) + 2.0 / (x * z) + 2.0 / (y * z);
    return 1.0 / sqrt(v - 1.0 / (x * x) - 1.0 / (y * y) - 1.0 / (z * z));
}

bool same_side(const point &p1, const point &p2, const point &a, const point &b) {
    static const bool EDGE_IS_SAME_SIDE = true;
    point ab(b.x - a.x, b.y - a.y);
    point p1a(p1.x - a.x, p1.y - a.y), p2a(p2.x - a.x, p2.y - a.y);
    double c1 = cross(ab, p1a), c2 = cross(ab, p2a);
    return EDGE_IS_SAME_SIDE ? GE(c1 * c2, 0) : GT(c1 * c2, 0);
}

bool point_in_triangle(const point &p, const point &a, const point &b, const point &c) {
    return same_side(p, a, b, c) && same_side(p, b, a, c) && same_side(p, c, a, b);
}
```

Example Usage

```
#include <cassert>

int main() {
```

```

assert(EQ(6, triangle_area(point(0, -1), point(4, -1), point(0, -4))));
assert(EQ(6, triangle_area_sides(3, 4, 5)));
assert(EQ(6, triangle_area_medians(3.605551275, 2.5, 4.272001873)));
assert(EQ(6, triangle_area_altitudes(3, 4, 2.4)));

assert(point_in_triangle(point(0, 0), point(-1, 0), point(0, -2), point(4, 0)));
assert(!point_in_triangle(point(0, 1), point(-1, 0), point(0, -2), point(4, 0)));
assert(point_in_triangle(point(-2.44, 0.82), point(-1, 0), point(-3, 1), point(4, 0)));
assert(!point_in_triangle(point(-2.44, 0.7), point(-1, 0), point(-3, 1), point(4, 0)));
return 0;
}

```

7.1.5 Rectangle

Common rectangle calculations in two dimensions.

- `rectangle_area(a, b)` returns the area of a rectangle with opposing vertices a and b .
- `point_in_rectangle(p, v, w, h)` returns whether point p lies within the rectangle defined by a vertex v at (x, y) , a width of w , and a height of h . Note that negative widths and heights are supported. If the point lies on or close to an edge (by roughly `EPS`), then the result will depend on the setting of `EDGE_IS_INSIDE`.
- `point_in_rectangle(p, a, b)` returns whether point p lies within the rectangle with opposing vertices a and b . If the point lies on or close to an edge (by roughly `EPS`), then the result will depend on the setting of `EDGE_IS_INSIDE`.
- `rectangle_intersection(a1, b1, a2, b2, &p, &q)` determines the intersection region of the rectangle with opposing vertices a_1 and b_1 and the rectangle with opposing vertices a_2 and b_2 . Returns -1 if the rectangles are completely disjoint, 0 if the rectangles partially intersect, 1 if the first rectangle is completely inside the second, and 2 if the second rectangle is completely inside the first. If there is an intersection, the opposing vertices of the intersection rectangle will be stored into pointers `p` and `q` if they are not `NULL`. If the intersection is a single point or line segment, then the result will depend on the setting of `EDGE_IS_INSIDE` within `point_in_rectangle()`.

Time Complexity: $O(1)$ for all operations.

Space Complexity: $O(1)$ auxiliary for all operations.

```

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <utility>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LT(a, b) ((a) < (b) - EPS)
#define GT(a, b) ((a) > (b) + EPS)
#define LE(a, b) ((a) <= (b) + EPS)
#define GE(a, b) ((a) >= (b) - EPS)

typedef std::pair<double, double> point;
#define x first
#define y second

double rectangle_area(const point &a, const point &b) {
    return fabs((a.x - b.x) * (a.y - b.y));
}

bool point_in_rectangle(const point &p, const point &v, double w, double h) {
    static const bool EDGE_IS_INSIDE = true;
    if (w < 0) {
        return point_in_rectangle(p, point(v.x + w, v.y), -w, h);
    }
    if (h < 0) {

```

```

    return point_in_rectangle(p, point(v.x, v.y + h), w, -h);
}
return EDGE_IS_INSIDE ? (GE(p.x, v.x) && LE(p.x, v.x + w) && GE(p.y, v.y) && LE(p.y, v.y + h))
    : (GT(p.x, v.x) && LT(p.x, v.x + w) && GT(p.y, v.y) && LT(p.y, v.y + h));
}

bool point_in_rectangle(const point &p, const point &a, const point &b) {
    double xl = std::min(a.x, b.x), yl = std::min(a.y, b.y);
    double xh = std::max(a.x, b.x), yh = std::max(a.y, b.y);
    return point_in_rectangle(p, point(xl, yl), xh - xl, yh - yl);
}

int rectangle_intersection(
    const point &a1, const point &b1, const point &a2, const point &b2, point *p = NULL,
    point *q = NULL
) {
    bool a1in2 = point_in_rectangle(a1, a2, b2);
    bool b1in2 = point_in_rectangle(b1, a2, b2);
    if (a1in2 && b1in2) {
        if (p != NULL && q != NULL) {
            *p = std::min(a1, b1);
            *q = std::max(a1, b1);
        }
        return 1; // Rectangle 1 completely inside 2.
    }
    if (!a1in2 && !b1in2) {
        if (point_in_rectangle(a2, a1, b1)) {
            if (p != NULL && q != NULL) {
                *p = std::min(a2, b2);
                *q = std::max(a2, b2);
            }
            return 2; // Rectangle 2 completely inside 1.
        }
        return -1; // Completely disjoint.
    }
    if (p != NULL && q != NULL) {
        if (a1in2) {
            *p = a1;
            *q = (a1 < b1) ? std::max(a2, b2) : std::min(a2, b2);
        } else {
            *p = b1;
            *q = (b1 < a1) ? std::max(a2, b2) : std::min(a2, b2);
        }
        if (*p > *q) {
            std::swap(p, q);
        }
    }
    return 0;
}

```

Example Usage

```

#include <cassert>

bool EQP(const point &a, const point &b) {
    return EQ(a.x, b.x) && EQ(a.y, b.y);
}

int main() {
    assert(EQ(20, rectangle_area(point(1, 1), point(5, 6))));

    assert(point_in_rectangle(point(0, -1), point(0, -3), 3, 2));
    assert(point_in_rectangle(point(2, -2), point(3, -3), -3, 2));
    assert(!point_in_rectangle(point(0, 0), point(3, -1), -3, -2));
    assert(point_in_rectangle(point(2, -2), point(3, -3), point(0, -1)));
    assert(!point_in_rectangle(point(-1, -2), point(3, -3), point(0, -1)));
}

```

```

point p, q;
assert(-1 == rectangle_intersection(point(0, 0), point(1, 1), point(2, 2), point(3, 3)));
assert(0 == rectangle_intersection(point(1, 1), point(7, 7), point(5, 5), point(0, 0), &p, &q));
assert(EQP(p, point(1, 1)) && EQP(q, point(5, 5)));
assert(1 == rectangle_intersection(point(1, 1), point(0, 0), point(0, 0), point(1, 10), &p, &q));
assert(EQP(p, point(0, 0)) && EQP(q, point(1, 1)));
assert(2 == rectangle_intersection(point(0, 5), point(5, 7), point(1, 6), point(2, 5), &p, &q));
assert(EQP(p, point(1, 6)) && EQP(q, point(2, 5)));

return 0;
}

```

7.2 Elementary Geometric Calculations

7.2.1 Angles

Angle calculations in two dimensions. The constants `DEG` and `RAD` may be used as multipliers to convert between degrees and radians. For example, if t is a value in radians, then $t \cdot \text{DEG}$ is the equivalent angle in degrees.

- `reduce_deg(t)` takes an angle t degrees and returns an equivalent angle in the range $[0, 360)$ degrees. E.g. -630 becomes 90 .
- `reduce_rad(t)` takes an angle t radians and returns an equivalent angle in the range $[0, 2\pi)$ radians. E.g. 720.5 becomes 0.5 .
- `polar_point(r, t)` returns a two-dimensional Cartesian point given radius r and angle t radians in polar coordinates (see `std::polar()`).
- `polar_angle(p)` returns the angle in radians of the line segment from $(0, 0)$ to point p , relative counterclockwise to the positive x -axis.
- `angle(a, o, b)` returns the smallest angle in radians formed by the points a, o, b with vertex at point o .
- `angle_between(a, b)` returns the angle in radians of segment from point a to point b , relative counterclockwise to the positive x -axis.
- `angle_between(a1, b1, a2, b2)` returns the smaller angle in radians between two lines $a_1x + b_1y + c_1 = 0$ and $a_2x + b_2y + c_2 = 0$, limited to $[0, \pi/2]$.
- `cross(a, b, o)` returns the magnitude (Euclidean norm) of the three-dimensional cross product between points a and b where the z -component is implicitly zero and the origin is implicitly shifted to point o . This operation is also equal to double the signed area of the triangle from these three points.
- `turn(a, o, b)` returns -1 if the path $a \rightarrow o \rightarrow b$ forms a left turn on the plane, 0 if the path forms a straight line segment, or 1 if it forms a right turn.

Time Complexity: $O(1)$ for all operations.

Space Complexity: $O(1)$ auxiliary for all operations.

```

#include <cmath>
#include <utility>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LT(a, b) ((a) < (b) - EPS)
#define GT(a, b) ((a) > (b) + EPS)

typedef std::pair<double, double> point;
#define x first
#define y second

double sqnorm(const point &a) {
    return a.x * a.x + a.y * a.y;
}

```

```

}
double norm(const point &a) {
    return sqrt(sqnorm(a));
}

const double PI = acos(-1.0), DEG = PI / 180, RAD = 180 / PI;

double reduce_deg(double t) {
    if (t < -360) {
        return reduce_deg(fmod(t, 360));
    }
    if (t < 0) {
        return t + 360;
    }
    return (t >= 360) ? fmod(t, 360) : t;
}

double reduce_rad(double t) {
    if (t < -2 * PI) {
        return reduce_rad(fmod(t, 2 * PI));
    }
    if (t < 0) {
        return t + 2 * PI;
    }
    return (t >= 2 * PI) ? fmod(t, 2 * PI) : t;
}

point polar_point(double r, double t) {
    return point(r * cos(t), r * sin(t));
}

double polar_angle(const point &p) {
    double t = atan2(p.y, p.x);
    return (t < 0) ? (t + 2 * PI) : t;
}

double angle(const point &a, const point &o, const point &b) {
    point u(o.x - a.x, o.y - a.y), v(o.x - b.x, o.y - b.y);
    return acos((u.x * v.x + u.y * v.y) / (norm(u) * norm(v)));
}

double angle_between(const point &a, const point &b) {
    double t = atan2(a.x * b.y - a.y * b.x, a.x * b.x + a.y * b.y);
    return (t < 0) ? (t + 2 * PI) : t;
}

double angle_between(const double &a1, const double &b1, const double &a2, const double &b2) {
    double t = atan2(a1 * b2 - a2 * b1, a1 * a2 + b1 * b2);
    if (t < 0) {
        t += PI;
    }
    return GT(t, PI / 2) ? (PI - t) : t;
}

double cross(const point &a, const point &b, const point &o = point(0, 0)) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

int turn(const point &a, const point &o, const point &b) {
    double c = cross(a, b, o);
    return LT(c, 0) ? -1 : (GT(c, 0) ? 1 : 0);
}

```

Example Usage

```
#include <cassert>
```

```

bool EQP(const point &a, const point &b) {
    return EQ(a.x, b.x) && EQ(a.y, b.y);
}

int main() {
    assert(EQ(123, reduce_deg(-8 * 360 + 123)));
    assert(EQ(1.2345, reduce_rad(2 * PI * 8 + 1.2345)));
    assert(EQP(polar_point(4, PI), point(-4, 0)));
    assert(EQP(polar_point(4, -PI / 2), point(0, -4)));
    assert(EQ(45, polar_angle(point(5, 5)) * RAD));
    assert(EQ(135 * DEG, polar_angle(point(-4, 4))));
    assert(EQ(90 * DEG, angle(point(5, 0), point(0, 5), point(-5, 0))));
    assert(EQ(225 * DEG, angle_between(point(0, 5), point(5, -5))));
    assert(-1 == cross(point(0, 1), point(1, 0), point(0, 0)));
    assert(1 == turn(point(0, 1), point(0, 0), point(-5, -5)));
    return 0;
}

```

7.2.2 Distances

Distance calculations in two dimensions for points, lines, and line segments.

- `dist(a, b)` and `sqdist(a, b)` respectively return the distance and squared distance between points a and b .
- `line_dist(p, a, b, c)` returns the distance from point p to the line $ax + by + c = 0$. If the line is invalid (i.e. $a = b = 0$), then `-INF`, `INF`, or `NaN` is returned based on the sign of c .
- `line_dist(p, a, b)` returns the distance from point p to the infinite line containing points a and b . If the line is invalid (i.e. $a = b$), then the distance from p to the single point is returned.
- `line_dist(a1, b1, c1, a2, b2, c2)` returns the distance between two lines. If the lines are non-parallel then the distance is considered to be 0. Otherwise, the distance is considered to be the perpendicular distance from any point on one line to the other line.
- `seg_dist(p, a, b)` returns the distance from point p to the line segment ab .
- `seg_dist(a, b, c, d)` returns the minimum distance from any point on the line segment ab to any point on the line segment cd . This is 0 if the segments touch or intersect.

Time Complexity: $O(1)$ for all operations.

Space Complexity: $O(1)$ auxiliary for all operations.

```

#include <algorithm>
#include <cmath>
#include <utility>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LE(a, b) ((a) <= (b) + EPS)
#define GE(a, b) ((a) >= (b) - EPS)

typedef std::pair<double, double> point;
#define x first
#define y second

double sqnorm(const point &a) {
    return a.x * a.x + a.y * a.y;
}

double norm(const point &a) {
    return sqrt(sqnorm(a));
}

double dot(const point &a, const point &b) {

```

```

    return a.x * b.x + a.y * b.y;
}

double cross(const point &a, const point &b) {
    return a.x * b.y - a.y * b.x;
}

double dist(const point &a, const point &b) {
    return norm(point(b.x - a.x, b.y - a.y));
}

double sqdist(const point &a, const point &b) {
    return sqnorm(point(b.x - a.x, b.y - a.y));
}

double line_dist(const point &p, double a, double b, double c) {
    return fabs(a * p.x + b * p.y + c) / sqrt(a * a + b * b);
}

double line_dist(const point &p, const point &a, const point &b) {
    if (EQ(a.x, b.x) && EQ(a.y, b.y)) {
        return dist(p, a);
    }
    double u = ((p.x - a.x) * (b.x - a.x) + (p.y - a.y) * (b.y - a.y)) / sqdist(a, b);
    return norm(point(a.x + u * (b.x - a.x) - p.x, a.y + u * (b.y - a.y) - p.y));
}

double line_dist(double a1, double b1, double c1, double a2, double b2, double c2) {
    if (EQ(a1 * b2, a2 * b1)) {
        double factor = EQ(b1, 0) ? (a1 / a2) : (b1 / b2);
        return EQ(c1, c2 * factor) ? 0 : fabs(c2 * factor - c1) / sqrt(a1 * a1 + b1 * b1);
    }
    return 0;
}

double seg_dist(const point &p, const point &a, const point &b) {
    if (EQ(a.x, b.x) && EQ(a.y, b.y)) {
        return dist(p, a);
    }
    point ab(b.x - a.x, b.y - a.y), ap(p.x - a.x, p.y - a.y);
    double n = sqnorm(ab), d = dot(ab, ap);
    if (LE(d, 0) || EQ(n, 0)) {
        return norm(ap);
    }
    return GE(d, n) ? norm(point(ap.x - ab.x, ap.y - ab.y))
        : norm(point(ap.x - ab.x * (d / n), ap.y - ab.y * (d / n)));
}

double seg_dist(const point &a, const point &b, const point &c, const point &d) {
    if (EQ(a.x, b.x) && EQ(a.y, b.y)) {
        return seg_dist(a, c, d);
    }
    if (EQ(c.x, d.x) && EQ(c.y, d.y)) {
        return seg_dist(c, a, b);
    }
    point ab(b.x - a.x, b.y - a.y);
    point ac(c.x - a.x, c.y - a.y);
    point cd(d.x - c.x, d.y - c.y);
    double c1 = cross(ab, cd), c2 = cross(ac, ab);
    if (EQ(c1, 0) && EQ(c2, 0)) {
        double t0 = dot(ac, ab) / sqnorm(ab), t1 = t0 + dot(cd, ab) / sqnorm(ab);
        if (LE(std::min(t0, t1), 1) && LE(0, std::max(t0, t1))) {
            return 0;
        }
    }
    else if (!EQ(c1, 0)) {
        double t = cross(ac, cd) / c1, u = c2 / c1;
        if (LE(0, t) && LE(t, 1) && LE(0, u) && LE(u, 1)) {
            return 0;
        }
    }
}

```



```

    }
}
return std::min(
    std::min(seg_dist(a, c, d), seg_dist(b, c, d)), std::min(seg_dist(c, a, b), seg_dist(d, a, b))
);
}

point closest_point(const point &a, const point &b, const point &p) {
    if (EQ(a.x, b.x) && EQ(a.y, b.y)) {
        return a;
    }
    point ap(p.x - a.x, p.y - a.y), ab(b.x - a.x, b.y - a.y);
    double t = dot(ap, ab) / sqnorm(ab);
    return (t <= 0) ? a : ((t >= 1) ? b : point(a.x + t * ab.x, a.y + t * ab.y));
}

```

Example Usage

```

#include <cassert>

int main() {
    assert(EQ(5, dist(point(-1, -1), point(2, 3))));
    assert(EQ(25, sqdist(point(-1, -1), point(2, 3))));
    assert(EQ(1.2, line_dist(point(2, 1), -4, 3, -1)));
    assert(EQ(0.8, line_dist(point(3, 3), point(-1, -1), point(2, 3))));
    assert(EQ(1.2, line_dist(point(2, 1), point(-1, -1), point(2, 3))));
    assert(EQ(0, line_dist(-4, 3, -1, 8, 6, 2)));
    assert(EQ(0.8, line_dist(-4, 3, -1, -8, 6, -10)));
    assert(EQ(1.0, seg_dist(point(3, 3), point(-1, -1), point(2, 3))));
    assert(EQ(1.2, seg_dist(point(2, 1), point(-1, -1), point(2, 3))));
    assert(EQ(0, seg_dist(point(0, 2), point(3, 3), point(-1, -1), point(2, 3))));
    assert(EQ(0.6, seg_dist(point(-1, 0), point(-2, 2), point(-1, -1), point(2, 3))));
    return 0;
}

```

7.2.3 Line Intersection

Intersection and closest point calculations in two dimensions for straight lines and line segments.

- `line_intersection(a1, b1, c1, a2, b2, c2, &p)` determines whether the lines $a_1x + b_1y + c_1 = 0$ and $a_2x + b_2y + c_2 = 0$ intersect, returning -1 if there is no intersection because the lines are parallel, 0 if there is exactly one intersection (in which case the intersection point is stored into pointer `p` if it's not `NULL`), or 1 if there are infinite intersections because the lines are identical.
- `line_intersection(p1, p2, p3, p4, &p)` determines whether the infinite lines (not segments) through points p_1 , p_2 and through points p_3 and p_4 intersect, returning -1 if there is no intersection because the lines are parallel, 0 if there is exactly one intersection (in which case the intersection point is stored into pointer `p` if it's not `NULL`), or 1 if there are infinite intersections because the lines are identical.
- `seg_intersection(a, b, c, d, &p, &q)` determines whether the line segment ab intersects the line segment cd , returning -1 if the segments do not intersect, 0 if there is exactly one intersection point (in which case it is stored into pointer `p` if it's not `NULL`), or 1 if the intersection is another line segment (in which case the two endpoints are stored into pointers `p` and `q` if they are not `NULL`). If the segments are barely touching (close within `EPS`), then the result will depend on the setting of `TOUCH_IS_INTERSECT`.
- `closest_point(a, b, c, p)` returns the point on line $ax + by + c = 0$ that is closest to point p . Note that the result always lies on the line through p which is perpendicular to the line $ax + by + c = 0$.
- `closest_point(a, b, p)` returns the point on segment ab closest to point p .

Time Complexity: $O(1)$ for all operations.

Space Complexity: $O(1)$ auxiliary for all operations.

```

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <utility>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LT(a, b) ((a) < (b) - EPS)
#define LE(a, b) ((a) <= (b) + EPS)

typedef std::pair<double, double> point;
#define x first
#define y second

double sqnorm(const point &a) { return a.x * a.x + a.y * a.y; }
double norm(const point &a) { return sqrt(sqnorm(a)); }
double dot(const point &a, const point &b) { return a.x * b.x + a.y * b.y; }
double cross(const point &a, const point &b) { return a.x * b.y - a.y * b.x; }

bool point_on_segment(const point &p, const point &a, const point &b) {
    return EQ(cross(point(p.x - a.x, p.y - a.y), point(b.x - a.x, b.y - a.y)), 0) &&
        LE(std::min(a.x, b.x), p.x) && LE(p.x, std::max(a.x, b.x)) &&
        LE(std::min(a.y, b.y), p.y) && LE(p.y, std::max(a.y, b.y));
}

int line_intersection(
    double a1, double b1, double c1, double a2, double b2, double c2, point *p = NULL
) {
    if (EQ(a1, a2) && EQ(b1, b2)) {
        return EQ(c1, c2) ? 1 : -1;
    }
    if (p != NULL) {
        p->x = (b1 * c1 - b1 * c2) / (a2 * b1 - a1 * b2);
        if (!EQ(b1, 0)) {
            p->y = -(a1 * p->x + c1) / b1;
        } else {
            p->y = -(a2 * p->x + c2) / b2;
        }
    }
    return 0;
}

int line_intersection(
    const point &p1, const point &p2, const point &p3, const point &p4, point *p = NULL
) {
    double a1 = p2.y - p1.y, b1 = p1.x - p2.x;
    double c1 = -(p1.x * p2.y - p2.x * p1.y);
    double a2 = p4.y - p3.y, b2 = p3.x - p4.x;
    double c2 = -(p3.x * p4.y - p4.x * p3.y);
    double x = -(c1 * b2 - c2 * b1), y = -(a1 * c2 - a2 * c1);
    double det = a1 * b2 - a2 * b1;
    if (EQ(det, 0)) {
        return (EQ(x, 0) && EQ(y, 0)) ? 1 : -1;
    }
    if (p != NULL) {
        *p = point(x / det, y / det);
    }
    return 0;
}

int seg_intersection(
    const point &a, const point &b, const point &c, const point &d, point *p = NULL, point *q = NULL
) {
    static const bool TOUCH_IS_INTERSECT = true;
    point ab(b.x - a.x, b.y - a.y);
    point ac(c.x - a.x, c.y - a.y);
    point cd(d.x - c.x, d.y - c.y);

```

```

if (EQ(sqnorm(ab), 0)) {
    if (TOUCH_IS_INTERSECT && point_on_segment(a, c, d)) {
        if (p != NULL) *p = a;
        return 0;
    }
    return -1;
}
if (EQ(sqnorm(cd), 0)) {
    if (TOUCH_IS_INTERSECT && point_on_segment(c, a, b)) {
        if (p != NULL) *p = c;
        return 0;
    }
    return -1;
}
double c1 = cross(ab, cd), c2 = cross(ac, ab);
if (EQ(c1, 0) && EQ(c2, 0)) { // Collinear.
    double t0 = dot(ac, ab) / sqnorm(ab);
    double t1 = t0 + dot(cd, ab) / sqnorm(ab);
    double mint = std::min(t0, t1), maxt = std::max(t0, t1);
    bool overlap = TOUCH_IS_INTERSECT ? (LE(mint, 1) && LE(0, maxt)) : (LT(mint, 1) && LT(0, maxt));
    if (overlap) {
        point res1 = std::max(std::min(a, b), std::min(c, d));
        point res2 = std::min(std::max(a, b), std::max(c, d));
        if (res1 == res2) {
            if (p != NULL) {
                *p = res1;
            }
            return 0; // Collinear and meeting at an endpoint.
        }
        if (p != NULL && q != NULL) {
            *p = res1;
            *q = res2;
        }
        return 1; // Collinear and overlapping.
    } else {
        return -1; // Collinear and disjoint.
    }
}
if (EQ(c1, 0)) {
    return -1; // Parallel and disjoint.
}
double t = cross(ac, cd) / c1, u = c2 / c1;
bool t_between_01 = TOUCH_IS_INTERSECT ? (LE(0, t) && LE(t, 1)) : (LT(0, t) && LT(t, 1));
bool u_between_01 = TOUCH_IS_INTERSECT ? (LE(0, u) && LE(u, 1)) : (LT(0, u) && LT(u, 1));
if (t_between_01 && u_between_01) {
    if (p != NULL) {
        *p = point(a.x + t * ab.x, a.y + t * ab.y);
    }
    return 0; // Non-parallel with one intersection.
}
return -1; // Non-parallel with no intersections.
}

point closest_point(double a, double b, double c, const point &p) {
    if (EQ(a, 0)) {
        return point(p.x, -c / b); // Horizontal line.
    }
    if (EQ(b, 0)) {
        return point(-c / a, p.y); // Vertical line.
    }
    point res;
    line_intersection(a, b, c, -b, a, b * p.x - a * p.y, &res);
    return res;
}

point closest_point(const point &a, const point &b, const point &p) {
    if (a == b) return a;
    point ap(p.x - a.x, p.y - a.y), ab(b.x - a.x, b.y - a.y);

```

```

double t = dot(ap, ab) / sqnorm(ab);
if (t <= 0) return a;
if (t >= 1) return b;
return point(a.x + t * ab.x, a.y + t * ab.y);
}

```

Example Usage

```

#include <cassert>
#define point point

bool EQP(const point &a, const point &b) {
    return EQ(a.x, b.x) && EQ(a.y, b.y);
}

int main() {
    point p, q;

    assert(line_intersection(-1, 1, 0, 1, 1, -3, &p) == 0);
    assert(EQP(p, point(1.5, 1.5)));
    assert(line_intersection(point(0, 0), point(1, 1), point(0, 4), point(4, 0), &p) == 0);
    assert(EQP(p, point(2, 2)));

    {
#define test(a, b, c, d, e, f, g, h) \
    seg_intersection(point(a, b), point(c, d), point(e, f), point(g, h), &p, &q)

        // Intersection is a point.
        assert(0 == test(-4, 0, 4, 0, 0, -4, 0, 4) && EQP(p, point(0, 0)));
        assert(0 == test(0, 0, 10, 10, 2, 2, 16, 4) && EQP(p, point(2, 2)));
        assert(0 == test(-2, 2, -2, -2, -2, 0, 0, 0) && EQP(p, point(-2, 0)));
        assert(0 == test(0, 4, 4, 4, 4, 0, 4, 8) && EQP(p, point(4, 4)));

        // Intersection is a segment.
        assert(1 == test(10, 10, 0, 0, 2, 2, 6, 6));
        assert(EQP(p, point(2, 2)) && EQP(q, point(6, 6)));
        assert(1 == test(6, 8, 14, -2, 14, -2, 6, 8));
        assert(EQP(p, point(6, 8)) && EQP(q, point(14, -2)));

        // No intersection.
        assert(-1 == test(6, 8, 8, 10, 12, 12, 4, 4));
        assert(-1 == test(-4, 2, -8, 8, 0, 0, -4, 6));
        assert(-1 == test(4, 4, 4, 6, 0, 2, 0, 0));
        assert(-1 == test(4, 4, 6, 4, 0, 2, 0, 0));
        assert(-1 == test(-2, -2, 4, 4, 10, 10, 6, 6));
        assert(-1 == test(0, 0, 2, 2, 4, 0, 1, 4));
        assert(-1 == test(2, 2, 2, 8, 4, 4, 6, 4));
        assert(-1 == test(4, 2, 4, 4, 0, 8, 10, 0));
    }

    assert(EQP(point(2.5, 2.5), closest_point(-1, -1, 5, point(0, 0))));
    assert(EQP(point(3, 0), closest_point(1, 0, -3, point(0, 0))));
    assert(EQP(point(0, 3), closest_point(0, 1, -3, point(0, 0))));

    assert(EQP(point(3, 0), closest_point(point(3, 0), point(3, 3), point(0, 0))));
    assert(EQP(point(2, -1), closest_point(point(2, -1), point(4, -1), point(0, 0))));
    assert(EQP(point(4, -1), closest_point(point(2, -1), point(4, -1), point(5, 0))));
    return 0;
}

```

7.2.4 Circle Intersection

Circle tangent and intersection calculations in two dimensions.

- `tangent(c, p, &l1, &l2)` determines the line(s) tangent to circle c that pass through point p , returning -1 if there is no tangent line because p is strictly inside c , 0 if there is exactly one tangent line because p is on the boundary of c (in which case the line will be stored into pointer `l1` if it's not `NULL`), or 1 if there are two tangent lines because p is strictly outside of c (in which case the lines will be stored into pointers `l1` and `l2` if they are not `NULL`).
- `intersection(c, l, &p, &q)` determines the intersection between the circle c and line l , returning -1 if there is no intersection, 0 if the line has one intersection point because the line is tangent (in which case it will be stored into pointer `p` if it's not `NULL`), or 1 if there are two intersection points because the line crosses through the circle (in which case they will be stored into pointers `p` and `q` if they are not `NULL`).
- `intersection(c1, c2, &p, &q)` determines the intersection points between two circles c_1 and c_2 , returning -2 if circle c_2 completely encloses circle c_1 , -1 if circle c_1 completely encloses circle c_2 , 0 if the circles are completely disjoint, 1 if the circles are tangent with one intersection (stored in `p`), 2 if the circles intersect at two points (stored in `p` and `q`), 3 if the circles are equal and intersect at infinite points.
- `intersection_area(c1, c2)` returns the intersection area of circles c_1 and c_2 .

Time Complexity: $O(1)$ for all operations.

Space Complexity: $O(1)$ auxiliary for all operations.

```
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <utility>

const double EPS = 1e-9, PI = acos(-1.0);

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define NE(a, b) (fabs((a) - (b)) > EPS)
#define LT(a, b) ((a) < (b) - EPS)
#define GT(a, b) ((a) > (b) + EPS)
#define LE(a, b) ((a) <= (b) + EPS)
#define GE(a, b) ((a) >= (b) - EPS)

typedef std::pair<double, double> point;
#define x first
#define y second

double sqnorm(const point &a) {
    return a.x * a.x + a.y * a.y;
}

double norm(const point &a) {
    return sqrt(sqnorm(a));
}

struct circle {
    double h, k, r;

    circle(double h, double k, double r) {
        this->h = h;
        this->k = k;
        this->r = r;
    }
};

struct line {
    double a, b, c;

    line() : a(0), b(0), c(0) {}

    line(double a, double b, double c) {
        if (!EQ(b, 0)) {
            this->a = a / b;
            this->c = c / b;
            this->b = 1;
        }
    }
};
```

```

    } else {
        this->c = c / a;
        this->a = 1;
        this->b = 0;
    }
}

line(const point &p, const point &q) : a(0), b(0), c(0) {
    if (EQ(p.x, q.x)) {
        if (NE(p.y, q.y)) { // Vertical line.
            a = 1;
            b = 0;
            c = -p.x;
        } // Else, invalid line.
    } else {
        a = -(p.y - q.y) / (p.x - q.x);
        b = 1;
        c = -(a * p.x) - (b * p.y);
    }
}
};

int tangent(const circle &c, const point &p, line *l1 = NULL, line *l2 = NULL) {
    point vop(p.x - c.h, p.y - c.k);
    if (EQ(sqnorm(vop), c.r * c.r)) { // Point on an edge.
        if (l1 != 0) { // Get perpendicular line through p.
            *l1 = line(point(c.h, c.k), p);
            *l1 = line(-l1->b, l1->a, l1->b * p.x - l1->a * p.y);
        }
        return 0;
    }
    if (LE(sqnorm(vop), c.r * c.r)) {
        return -1; // Point inside circle, no intersection.
    }
    point q(vop.x / c.r, vop.y / c.r);
    double n = sqnorm(q), d = q.y * sqrt(sqnorm(q) - 1.0);
    point t1((q.x - d) / n, c.k), t2((q.x + d) / n, c.k);
    if (NE(q.y, 0)) { // Common case.
        t1.y += c.r * (1.0 - t1.x * q.x) / q.y;
        t2.y += c.r * (1.0 - t2.x * q.x) / q.y;
    } else { // Point at center horizontal, y = 0.
        d = c.r * sqrt(1.0 - t1.x * t1.x);
        t1.y += d;
        t2.y -= d;
    }
    t1.x = t1.x * c.r + c.h;
    t2.x = t2.x * c.r + c.h;
    //note: here, t1 and t2 are the two points of tangencies
    if (l1 != NULL && l2 != NULL) {
        *l1 = line(p, t1);
        *l2 = line(p, t2);
    }
    return 1;
}

int intersection(const circle &c, const line &l, point *p = NULL, point *q = NULL) {
    double v = c.h * l.a + c.k * l.b + l.c;
    double aabb = l.a * l.a + l.b * l.b;
    double disc = v * v / aabb - c.r * c.r;
    if (disc > EPS) {
        return -1;
    }
    double x0 = -l.a * l.c / aabb, y0 = -l.b * v / aabb;
    if (disc > -EPS) {
        if (p != NULL) {
            *p = point(x0 + c.h, y0 + c.k);
        }
        return 0;
    }
}

```

```

    }
    double k = sqrt(std::max(0.0, disc / -aabb));
    if (p != NULL && q != NULL) {
        *p = point(x0 + k * l.b + c.h, y0 - k * l.a + c.k);
        *q = point(x0 - k * l.b + c.h, y0 + k * l.a + c.k);
    }
    return 1;
}

int intersection(const circle &c1, const circle &c2, point *p = NULL, point *q = NULL) {
    if (EQ(c1.h, c2.h) && EQ(c1.k, c2.k)) {
        return EQ(c1.r, c2.r) ? 3 : (c1.r > c2.r ? -1 : -2);
    }
    point d12(point(c2.h - c1.h, c2.k - c1.k));
    double d = norm(d12);
    if (GT(d, c1.r + c2.r)) {
        return 0;
    }
    if (LT(d, fabs(c1.r - c2.r))) {
        return c1.r > c2.r ? -1 : -2;
    }
    double a = (c1.r * c1.r - c2.r * c2.r + d * d) / (2 * d);
    double x0 = c1.h + (d12.x * a / d), y0 = c1.k + (d12.y * a / d);
    double s = sqrt(c1.r * c1.r - a * a), rx = -d12.y * s / d, ry = d12.x * s / d;
    if (EQ(rx, 0) && EQ(ry, 0)) {
        if (p != NULL) {
            *p = point(x0, y0);
        }
        return 1;
    }
    if (p != NULL && q != NULL) {
        *p = point(x0 - rx, y0 - ry);
        *q = point(x0 + rx, y0 + ry);
    }
    return 2;
}

double intersection_area(const circle &c1, const circle &c2) {
    double r = std::min(c1.r, c2.r), R = std::max(c1.r, c2.r);
    double d = norm(point(c2.h - c1.h, c2.k - c1.k));
    if (LE(d, R - r)) {
        return PI * r * r;
    }
    if (GE(d, R + r)) {
        return 0;
    }
    return r * r * acos((d * d + r * r - R * R) / 2 / d / r) +
        R * R * acos((d * d + R * R - r * r) / 2 / d / R) -
        0.5 * sqrt((-d + r + R) * (d + r - R) * (d - r + R) * (d + r + R));
}

```

Example Usage

```

#include <cassert>

bool EQP(const point &a, const point &b) {
    return EQ(a.x, b.x) && EQ(a.y, b.y);
}

bool EQL(const line &l1, const line &l2) {
    return EQ(l1.a, l2.a) && EQ(l1.b, l2.b) && EQ(l1.c, l2.c);
}

int main() {
    line l1, l2;
    assert(-1 == tangent(circle(0, 0, 4), point(1, 1), &l1, &l2));
}

```

```

assert(0 == tangent(circle(0, 0, sqrt(2)), point(1, 1), &l1, &l2));
assert(EQL(l1, line(-1, -1, 2)));
assert(1 == tangent(circle(0, 0, 2), point(2, 2), &l1, &l2));
assert(EQL(l1, line(0, -2, 4)));
assert(EQL(l2, line(2, 0, -4)));

point p, q;
assert(-1 == intersection(circle(1, 1, 3), line(5, 3, -30), &p, &q));
assert(0 == intersection(circle(1, 1, 3), line(0, 1, -4), &p, &q));
assert(EQP(p, point(1, 4)));
assert(1 == intersection(circle(1, 1, 3), line(0, 1, -1), &p, &q));
assert(EQP(p, point(4, 1)));
assert(EQP(q, point(-2, 1)));

assert(-2 == intersection(circle(1, 1, 1), circle(0, 0, 3), &p, &q));
assert(-1 == intersection(circle(0, 0, 3), circle(1, 1, 1), &p, &q));
assert(0 == intersection(circle(5, 0, 4), circle(-5, 0, 4), &p, &q));
assert(1 == intersection(circle(-5, 0, 5), circle(5, 0, 5), &p, &q));
assert(EQP(p, point(0, 0)));
assert(2 == intersection(circle(-0.5, 0, 1), circle(0.5, 0, 1), &p, &q));
assert(EQP(p, point(0, -sqrt(3) / 2)));
assert(EQP(q, point(0, sqrt(3) / 2)));

// Each circle passes through the other's center.
double r = 3, a = intersection_area(circle(-r / 2, 0, r), circle(r / 2, 0, r));
assert(EQ(a, r * r * (2 * PI / 3 - sqrt(3) / 2)));
return 0;
}

```

7.3 Intermediate Geometric Calculations

7.3.1 Polygon Sorting and Area

Given a list of distinct points in two-dimensions, order them into a valid polygon and determine the area.

- `mean_center(lo, hi)` returns the arithmetic mean of a range `[lo, hi)` of points, where `lo` and `hi` must be random-access iterators. This point is mathematically guaranteed to lie within the non-self-intersecting closed polygon constructed by sorting all other points clockwise about it. Note that this is different from the geometric centroid (a.k.a. barycenter) of a polygon.
- `cw_comp(a, b, c)` returns whether point *a* compares clockwise "before" point *b* when using *c* as a central reference point.
- `cw_comp_class(c)` constructs a wrapper class of `cw_comp()` that may be passed to `std::sort()` a range of points clockwise to produce a valid polygon.
- `ccw_comp_class(c)` constructs a wrapper class of `cw_comp()` that may be passed to `std::sort()` a range of points counter-clockwise to produce a valid polygon.
- `polygon_area(lo, hi)` returns the area of the polygon specified by the range `[lo, hi)` of points, where `lo` and `hi` must be BidirectionalIterators. The points are interpreted as a polygon based on the order given in the range. The input polygon does not have to be sorted using the methods above, but must be given in some ordering that yields a valid non-self-intersecting closed polygon. Optionally, the last point may be equal to the first point in the input without affecting the result. The area is computed using the shoelace formula.

Time Complexity:

- $O(n)$ per call to `mean_center(lo, hi)` and `polygon_area(lo, hi)`, where *n* is the distance between `lo` and `hi`.
- $O(1)$ per call to `cw_comp()` and the related class comparators.

Space Complexity: $O(1)$ auxiliary for all operations.

```
#include <algorithm>
```



```

#include <cmath>
#include <stdexcept>
#include <utility>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LT(a, b) ((a) < (b) - EPS)
#define GE(a, b) ((a) >= (b) - EPS)

typedef std::pair<double, double> point;
#define x first
#define y second

double sqnorm(const point &a) {
    return a.x * a.x + a.y * a.y;
}

double cross(const point &a, const point &b) {
    return a.x * b.y - a.y * b.x;
}

template<class It>
point mean_center(It lo, It hi) {
    if (lo == hi) {
        throw std::runtime_error("Cannot get center of an empty range.");
    }
    double x_sum = 0, y_sum = 0, num_points = hi - lo;
    for (; lo != hi; ++lo) {
        x_sum += lo->x;
        y_sum += lo->y;
    }
    return point(x_sum / num_points, y_sum / num_points);
}

bool cw_comp(const point &a, const point &b, const point &c) {
    if (GE(a.x - c.x, 0) && LT(b.x - c.x, 0)) {
        return true;
    }
    if (LT(a.x - c.x, 0) && GE(b.x - c.x, 0)) {
        return false;
    }
    if (EQ(a.x - c.x, 0) && EQ(b.x - c.x, 0)) {
        if (GE(a.y - c.y, 0) || GE(b.y - c.y, 0)) {
            return a.y > b.y;
        }
        return b.y > a.y;
    }
    point ac(a.x - c.x, a.y - c.y), bc(b.x - c.x, b.y - c.y);
    double det = cross(ac, bc);
    if (EQ(det, 0)) {
        return sqnorm(ac) > sqnorm(bc);
    }
    return det < 0;
}

struct cw_comp_class {
    point c;
    cw_comp_class(const point &c) : c(c) {}
    bool operator()(const point &a, const point &b) const { return cw_comp(a, b, c); }
};

struct ccw_comp_class {
    point c;
    ccw_comp_class(const point &c) : c(c) {}
    bool operator()(const point &a, const point &b) const { return cw_comp(b, a, c); }
};

```

```

template<class It>
double polygon_area(It lo, It hi) {
    if (lo == hi) {
        return 0;
    }
    double area = 0;
    if (*lo != *--hi) {
        area += (lo->x - hi->x) * (lo->y + hi->y);
    }
    for (It i = hi, j = --hi; i != lo; --i, --j) {
        area += (i->x - j->x) * (i->y + j->y);
    }
    return fabs(area / 2.0);
}

```

Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

int main() {
    // Irregular pentagon with only the vertex (1, 2) not on its convex hull.
    // The ordering here is already sorted in ccw order around their mean center,
    // though we will shuffle them to verify our sorting comparator.
    point points[] = {point(1, 3), point(1, 2), point(2, 1), point(0, 0), point(-1, 3)};
    vector<point> v(points, points + 5);
    std::random_shuffle(v.begin(), v.end());
    point c = mean_center(v.begin(), v.end());
    assert(EQ(c.x, 0.6) && EQ(c.y, 1.8));
    sort(v.begin(), v.end(), cw_comp_class(c));
    for (int i = 0; i < (int)v.size(); i++) {
        assert(v[i] == points[i]);
    }
    assert(EQ(polygon_area(v.begin(), v.end()), 5));
    return 0;
}

```

7.3.2 Point-in-Polygon (Ray Casting)

Given a point p and a polygon in two dimensions, determine whether p lies inside the polygon using a ray casting algorithm.

- `point_in_polygon(p, lo, hi)` returns whether p lies within the polygon defined by the range `[lo, hi)` of points specifying the vertices in either clockwise or counter-clockwise order, where `lo` and `hi` must be random-access iterators. If p lies barely on an edge (within `EPS`), then the result will depend on the setting of `EDGE_IS_INSIDE`.

Time Complexity: $O(n)$ per call to `point_in_polygon(p, lo, hi)`, where n is the distance between `lo` and `hi`.

Space Complexity: $O(1)$ auxiliary.

```

#include <cmath>
#include <utility>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LE(a, b) ((a) <= (b) + EPS)
#define GT(a, b) ((a) > (b) + EPS)

typedef std::pair<double, double> point;

```

```

#define x first
#define y second

double cross(const point &a, const point &b, const point &o = point(0, 0)) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

template<class It>
bool point_in_polygon(const point &p, It lo, It hi) {
    static const bool EDGE_IS_INSIDE = true;
    bool ans = 0;
    for (It i = lo, j = hi - 1; i != hi; j = i++) {
        if (EQ(i->y, p.y) && (EQ(i->x, p.x) || (EQ(j->y, p.y) && (LE(i->x, p.x) || LE(j->x, p.x))))) {
            return EDGE_IS_INSIDE;
        }
        if (GT(i->y, p.y) != GT(j->y, p.y)) {
            double det = cross(*i, *j, p);
            if (EQ(det, 0)) {
                return EDGE_IS_INSIDE;
            }
            if (GT(det, 0) != GT(j->y, i->y)) {
                ans = !ans;
            }
        }
    }
    return ans;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // Irregular trapezoid.
    point p[] = {point(-1, 3), point(1, 3), point(2, 1), point(0, 0)};
    assert(point_in_polygon(point(1, 2), p, p + 4));
    assert(point_in_polygon(point(0, 3), p, p + 4));
    assert(!point_in_polygon(point(0, 3.01), p, p + 4));
    assert(!point_in_polygon(point(2, 2), p, p + 4));
    return 0;
}

```

7.3.3 Convex Hull and Diametral Pair

Given a list of points in two dimensions, determine the convex hull using the monotone chain algorithm, and the diameter of the points using the method of rotating calipers. The convex hull is the smallest convex polygon (a polygon such that every line crossing through it will only do so once) that contains all of its points.

- `convex_hull(lo, hi)` returns the convex hull as a vector of polygon vertices in clockwise order, given a range `[lo, hi)` of points where `lo` and `hi` must be random-access iterators. The input range will be sorted lexicographically (by x , then by y) after the function call. Note that to produce the hull points in counter-clockwise order, replace every `GE()` comparison with `LE()`. To have the first point on the hull repeated as the last in the resulting vector, the final `res.resize(k - 1)` may be changed to `res.resize(k)`.
- `diametral_pair(lo, hi)` returns a maximum diametral pair given a range `[lo, hi)` of points where `lo` and `hi` must be random-access iterators. The input range will be sorted lexicographically (by x , then by y) after the function call.

Time Complexity: $O(n \log n)$ per call to `convex_hull(lo, hi)` and `diametral_pair(lo, hi)`, where n is the distance between `lo` and `hi`.

Space Complexity: $O(n)$ auxiliary for storage of the convex hull in both operations.

```

#include <algorithm>
#include <cmath>
#include <utility>
#include <vector>

const double EPS = 1e-9;

#define GE(a, b) ((a) >= (b) - EPS)

typedef std::pair<double, double> point;
#define x first
#define y second

double sqnorm(const point &a) {
    return a.x * a.x + a.y * a.y;
}

double cross(const point &a, const point &b, const point &o = point(0, 0)) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

template<class It>
std::vector<point> convex_hull(It lo, It hi) {
    int k = 0;
    if (hi - lo <= 1) {
        return std::vector<point>(lo, hi);
    }
    std::vector<point> res(2 * (int)(hi - lo));
    std::sort(lo, hi);
    for (It it = lo; it != hi; ++it) {
        while (k >= 2 && GE(cross(res[k - 1], *it, res[k - 2]), 0)) {
            k--;
        }
        res[k++] = *it;
    }
    int t = k + 1;
    for (It it = hi - 2; it != lo - 1; --it) {
        while (k >= t && GE(cross(res[k - 1], *it, res[k - 2]), 0)) {
            k--;
        }
        res[k++] = *it;
    }
    res.resize(k - 1);
    return res;
}

template<class It>
std::pair<point, point> diametral_pair(It lo, It hi) {
    std::vector<point> h = convex_hull(lo, hi);
    int m = h.size();
    if (m == 1) {
        return std::make_pair(h[0], h[0]);
    }
    if (m == 2) {
        return std::make_pair(h[0], h[1]);
    }
    int k = 1;
    while (fabs(cross(h[0], h[(k + 1) % m], h[m - 1])) > fabs(cross(h[0], h[k], h[m - 1]))) {
        k++;
    }
    double maxdist = 0, d;
    std::pair<point, point> res;
    for (int i = 0, j = k; i <= k && j < m; i++) {
        d = sqnorm(point(h[i].x - h[j].x, h[i].y - h[j].y));
        if (d > maxdist) {
            maxdist = d;
            res = std::make_pair(h[i], h[j]);
        }
    }
    return res;
}

```

```

    res = std::make_pair(h[i], h[j]);
}
while (j < m && fabs(cross(h[(i + 1) % m], h[(j + 1) % m], h[i])) >
        fabs(cross(h[(i + 1) % m], h[j], h[i]))) {
    d = sqnorm(point(h[i].x - h[(j + 1) % m].x, h[i].y - h[(j + 1) % m].y));
    if (d > maxdist) {
        maxdist = d;
        res = std::make_pair(h[i], h[(j + 1) % m]);
    }
    j++;
}
}
return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    { // Irregular pentagon with only the vertex (1, 2) not on the hull.
        vector<point> v;
        v.push_back(point(1, 3));
        v.push_back(point(1, 2));
        v.push_back(point(2, 1));
        v.push_back(point(0, 0));
        v.push_back(point(-1, 3));
        std::random_shuffle(v.begin(), v.end());
        vector<point> h;
        h.push_back(point(-1, 3));
        h.push_back(point(1, 3));
        h.push_back(point(2, 1));
        h.push_back(point(0, 0));
        assert(convex_hull(v.begin(), v.end()) == h);
    }
    {
        vector<point> v;
        v.push_back(point(0, 0));
        v.push_back(point(3, 0));
        v.push_back(point(0, 3));
        v.push_back(point(1, 1));
        v.push_back(point(4, 4));
        pair<point, point> res = diametral_pair(v.begin(), v.end());
        assert(res.first == point(0, 0));
        assert(res.second == point(4, 4));
    }
    return 0;
}

```

7.3.4 Minimum Enclosing Circle

Given a list of points in two dimensions, find the circle with smallest area which contains all the given points using a randomized algorithm.

- `minimum_enclosing_circle(lo, hi)` returns the minimum enclosing circle given a range `[lo, hi)` of points, where `lo` and `hi` must be random-access iterators. The input range will be shuffled after the function call, though this is only to avoid the worst-case running time and is not necessary for correctness.

Time Complexity: $O(n)$ on average per call to `minimum_enclosing_circle(lo, hi)`, where n is the distance between `lo` and `hi`.

Space Complexity: $O(1)$ auxiliary.

```

#include <algorithm>
#include <cmath>
#include <stdexcept>
#include <utility>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LE(a, b) ((a) <= (b) + EPS)

typedef std::pair<double, double> point;
#define x first
#define y second

double sqnorm(const point &a) {
    return a.x * a.x + a.y * a.y;
}

double norm(const point &a) {
    return sqrt(sqnorm(a));
}

struct circle {
    double h, k, r;

    circle() : h(0), k(0), r(0) {}
    circle(double h, double k, double r) : h(h), k(k), r(fabs(r)) {}

    // Circle with the line segment ab as a diameter.
    circle(const point &a, const point &b) {
        h = (a.x + b.x) / 2.0;
        k = (a.y + b.y) / 2.0;
        r = norm(point(a.x - h, a.y - k));
    }

    // Circumcircle of three points.
    circle(const point &a, const point &b, const point &c) {
        double an = sqnorm(point(b.x - c.x, b.y - c.y));
        double bn = sqnorm(point(a.x - c.x, a.y - c.y));
        double cn = sqnorm(point(a.x - b.x, a.y - b.y));
        double wa = an * (bn + cn - an);
        double wb = bn * (an + cn - bn);
        double wc = cn * (an + bn - cn);
        double w = wa + wb + wc;
        if (EQ(w, 0)) {
            throw std::runtime_error("No circumcircle from collinear points.");
        }
        h = (wa * a.x + wb * b.x + wc * c.x) / w;
        k = (wa * a.y + wb * b.y + wc * c.y) / w;
        r = norm(point(a.x - h, a.y - k));
    }

    bool contains(const point &p) const { return LE(sqnorm(point(p.x - h, p.y - k)), r * r); }
};

template<class It>
circle minimum_enclosing_circle(It lo, It hi) {
    if (lo == hi) {
        return circle(0, 0, 0);
    }
    if (lo + 1 == hi) {
        return circle(lo->x, lo->y, 0);
    }
    std::random_shuffle(lo, hi);
    circle res(*lo, *(lo + 1));
    for (It i = lo + 2; i != hi; ++i) {
        if (res.contains(*i)) {
            continue;
        }
    }
}

```

```

    }
    res = circle(*lo, *i);
    for (It j = lo + 1; j != i; ++j) {
        if (res.contains(*j)) {
            continue;
        }
        res = circle(*i, *j);
        for (It k = lo; k != j; ++k) {
            if (!res.contains(*k)) {
                res = circle(*i, *j, *k);
            }
        }
    }
}
}
return res;
}

```

Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

int main() {
    vector<point> v;
    v.push_back(point(0, 0));
    v.push_back(point(0, 1));
    v.push_back(point(1, 0));
    v.push_back(point(1, 1));
    circle res = minimum_enclosing_circle(v.begin(), v.end());
    assert(EQ(res.h, 0.5) && EQ(res.k, 0.5) && EQ(res.r, 1 / sqrt(2)));
    return 0;
}

```

7.3.5 Closest Pair

Given a list of points in two dimensions, find the closest pair among them using a divide and conquer algorithm.

- `closest_pair(lo, hi, &res)` returns the minimum Euclidean distance between any pair of points in the range `[lo, hi)`, where `lo` and `hi` must be random-access iterators. The input range will be sorted lexicographically (by x , then by y) after the function call. If there is an answer, the closest pair will be stored into pointer `res`.

Time Complexity: $O(n \log^2 n)$ per call to `closest_pair(lo, hi, &res)`, where n is the distance between `lo` and `hi`.

Space Complexity: $O(n \log^2 n)$ auxiliary stack space for `closest_pair(lo, hi, &res)`, where n is the distance between `lo` and `hi`.

```

#include <algorithm>
#include <cmath>
#include <limits>
#include <utility>
#include <vector>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LT(a, b) ((a) < (b) - EPS)

typedef std::pair<double, double> point;
#define x first

```

```

#define y second

double sqnorm(const point &a) { return a.x * a.x + a.y * a.y; }
double norm(const point &a) { return sqrt(sqnorm(a)); }
bool cmp_x(const point &a, const point &b) { return (a.x == b.x) ? (a.y < b.y) : (a.x < b.x); }
bool cmp_y(const point &a, const point &b) { return (a.y == b.y) ? (a.x < b.x) : (a.y < b.y); }

template<class It>
double closest_pair(
    It lo, It hi, std::pair<point, point> *res = NULL,
    double mindist = std::numeric_limits<double>::max(), bool sort_x = true
) {
    if (lo == hi) {
        return std::numeric_limits<double>::max();
    }
    if (sort_x) {
        std::sort(lo, hi, cmp_x);
    }
    It mid = lo + (hi - lo) / 2;
    double midx = mid->x;
    double d1 = closest_pair(lo, mid, res, mindist, false);
    mindist = std::min(mindist, d1);
    double d2 = closest_pair(mid + 1, hi, res, mindist, false);
    mindist = std::min(mindist, d2);
    std::sort(lo, hi, cmp_y);
    std::vector<It> t;
    for (It it = lo; it != hi; ++it) {
        if (fabs(it->x - midx) < mindist) {
            t.push_back(it);
        }
    }
    for (int i = 0; i < (int)t.size(); i++) {
        for (int j = i + 1; j < (int)t.size(); j++) {
            point a(*t[i]), b(*t[j]);
            if (b.y - a.y >= mindist) {
                break;
            }
            double dist = norm(point(a.x - b.x, a.y - b.y));
            if (mindist > dist) {
                mindist = dist;
                if (res) {
                    *res = std::make_pair(a, b);
                }
            }
        }
    }
    return mindist;
}

```

Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

int main() {
    vector<point> v;
    v.push_back(point(2, 3));
    v.push_back(point(12, 30));
    v.push_back(point(40, 50));
    v.push_back(point(5, 1));
    v.push_back(point(12, 10));
    v.push_back(point(3, 4));
    pair<point, point> res;
    assert(EQ(closest_pair(v.begin(), v.end(), &res), sqrt(2)));
    assert(res.first == point(2, 3));
}

```



```

    assert(res.second == point(3, 4));
    return 0;
}

```

7.3.6 Segment Intersection Finding

Given a list of line segments in two dimensions, determine whether any pair of segments intersect using a sweep line algorithm.

- `find_intersection(lo, hi, &res1, &res2)` returns whether any pair of segments intersect given a range `[lo, hi)` of segments, where `lo` and `hi` are random-access iterators. If an intersection is found, then one such pair of segments will be stored into pointers `res1` and `res2`. If some segments are barely touching (close within `EPS`), then the result will depend on the setting of `TOUCH_IS_INTERSECT`.

Time Complexity: $O(n \log n)$ per call to `find_intersection(lo, hi, &res1, &res2)`, where n is the distance between `lo` and `hi`.

Space Complexity: $O(n)$ auxiliary heap space for `find_intersection(lo, hi, &res1, &res2)`, where n is the distance between `lo` and `hi`.

```

#include <algorithm>
#include <cassert>
#include <cmath>
#include <set>
#include <utility>
#include <vector>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LT(a, b) ((a) < (b) - EPS)
#define LE(a, b) ((a) <= (b) + EPS)

typedef std::pair<double, double> point;
#define x first
#define y second

double sqnorm(const point &a) { return a.x * a.x + a.y * a.y; }
double norm(const point &a) { return sqrt(sqnorm(a)); }
double dot(const point &a, const point &b) { return a.x * b.x + a.y * b.y; }

double cross(const point &a, const point &b, const point &o = point(0, 0)) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

bool point_on_segment(const point &p, const point &a, const point &b) {
    return EQ(cross(p, b, a), 0) && LE(std::min(a.x, b.x), p.x) && LE(p.x, std::max(a.x, b.x)) &&
        LE(std::min(a.y, b.y), p.y) && LE(p.y, std::max(a.y, b.y));
}

int seg_intersection(
    const point &a, const point &b, const point &c, const point &d, point *p = NULL, point *q = NULL
) {
    static const bool TOUCH_IS_INTERSECT = true;
    point ab(b.x - a.x, b.y - a.y);
    point ac(c.x - a.x, c.y - a.y);
    point cd(d.x - c.x, d.y - c.y);
    if (EQ(sqnorm(ab), 0)) {
        if (TOUCH_IS_INTERSECT && point_on_segment(a, c, d)) {
            if (p != NULL) *p = a;
            return 0;
        }
    }
    return -1;
}

```

```

}
if (EQ(sqnorm(cd), 0)) {
    if (TOUCH_IS_INTERSECT && point_on_segment(c, a, b)) {
        if (p != NULL) *p = c;
        return 0;
    }
    return -1;
}
double c1 = cross(ab, cd), c2 = cross(ac, ab);
if (EQ(c1, 0) && EQ(c2, 0)) { // Collinear.
    double t0 = dot(ac, ab) / sqnorm(ab);
    double t1 = t0 + dot(cd, ab) / sqnorm(ab);
    double mint = std::min(t0, t1), maxt = std::max(t0, t1);
    bool overlap = TOUCH_IS_INTERSECT ? (LE(mint, 1) && LE(0, maxt)) : (LT(mint, 1) && LT(0, maxt));
    if (overlap) {
        point res1 = std::max(std::min(a, b), std::min(c, d));
        point res2 = std::min(std::max(a, b), std::max(c, d));
        if (res1 == res2) {
            if (p != NULL) {
                *p = res1;
            }
            return 0; // Collinear and meeting at an endpoint.
        }
        if (p != NULL && q != NULL) {
            *p = res1;
            *q = res2;
        }
        return 1; // Collinear and overlapping.
    } else {
        return -1; // Collinear and disjoint.
    }
}
if (EQ(c1, 0)) {
    return -1; // Parallel and disjoint.
}
double t = cross(ac, cd) / c1, u = c2 / c1;
bool t_between_01 = TOUCH_IS_INTERSECT ? (LE(0, t) && LE(t, 1)) : (LT(0, t) && LT(t, 1));
bool u_between_01 = TOUCH_IS_INTERSECT ? (LE(0, u) && LE(u, 1)) : (LT(0, u) && LT(u, 1));
if (t_between_01 && u_between_01) {
    if (p != NULL) {
        *p = point(a.x + t * ab.x, a.y + t * ab.y);
    }
    return 0; // Non-parallel with one intersection.
}
return -1; // Non-parallel with no intersections.
}

struct segment {
    point p, q;

    segment() {}
    segment(const point &p, const point &q) : p(min(p, q)), q(max(p, q)) {}
};

double get_y(const segment &s, double x) {
    if (EQ(s.p.x, s.q.x)) {
        return s.p.y;
    }
    return s.p.y + (s.q.y - s.p.y) * (x - s.p.x) / (s.q.x - s.p.x);
}

template<class It>
struct segment_order {
    It lo;

    segment_order(It lo) : lo(lo) {}

    bool operator()(It a, It b) const {

```

```

    if (a == b) {
        return false;
    }
    double x = std::max(a->p.x, b->p.x);
    double ay = get_y(*a, x), by = get_y(*b, x);
    return (ay == by) ? (a - lo < b - lo) : (ay < by);
}
};

template<class SegIt>
struct event {
    point p;
    int type;
    SegIt seg;

    event() {}
    event(const point &p, int type, SegIt seg) : p(p), type(type), seg(seg) {}

    bool operator<(const event &rhs) const {
        if (p.x != rhs.p.x) {
            return p.x < rhs.p.x;
        }
        if (type != rhs.type) {
            return rhs.type < type;
        }
        return p.y < rhs.p.y;
    }
};

bool intersect(const segment &s1, const segment &s2) {
    return seg_intersection(s1.p, s1.q, s2.p, s2.q) >= 0;
}

template<class It>
bool find_intersection(It lo, It hi, segment *res1, segment *res2) {
    std::vector<event<It>> e;
    for (It it = lo; it != hi; ++it) {
        if (it->p > it->q) {
            std::swap(it->p, it->q);
        }
        e.push_back(event<It>(it->p, 1, it));
        e.push_back(event<It>(it->q, -1, it));
    }
    std::sort(e.begin(), e.end());
    typedef std::set<It, segment_order<It>> active_set;
    active_set s((segment_order<It>(lo)));
    std::vector<typename active_set::iterator> position(hi - lo);
    for (int i = 0; i < (int)e.size(); i++) {
        It seg = e[i].seg;
        if (e[i].type == 1) {
            typename active_set::iterator it = s.insert(seg).first;
            position[seg - lo] = it;
            typename active_set::iterator next = it;
            if (++next != s.end() && intersect(**next, *seg)) {
                *res1 = **next;
                *res2 = *seg;
                return true;
            }
            if (it != s.begin() && intersect(**--it, *seg)) {
                *res1 = **it;
                *res2 = *seg;
                return true;
            }
        }
        else {
            typename active_set::iterator it = position[seg - lo];
            typename active_set::iterator next = it;
            if (++next != s.end() && it != s.begin()) {
                typename active_set::iterator prev = it;

```

```

        if (intersect(**next, **--prev)) {
            *res1 = **next;
            *res2 = **prev;
            return true;
        }
    }
    s.erase(it);
}
return false;
}

```

Example Usage

```

#include <vector>
using namespace std;

int main() {
    vector<segment> v;
    v.push_back(segment(point(0, 0), point(2, 2)));
    v.push_back(segment(point(3, 0), point(0, -1)));
    v.push_back(segment(point(0, 2), point(2, -2)));
    v.push_back(segment(point(0, 3), point(9, 0)));
    segment res1, res2;
    assert(find_intersection(v.begin(), v.end(), &res1, &res2));
    assert(res1.p == point(0, 0) && res1.q == point(2, 2));
    assert(res2.p == point(0, 2) && res2.q == point(2, -2));
    return 0;
}

```

7.4 Advanced Geometric Computations

7.4.1 Convex Polygon Cut

Given a convex polygon (a polygon such that every line crossing through it will only do so once) in two dimensions, and two points specifying an infinite line, cut off the right part of the polygon, and return the resulting left part.

- `convex_cut(lo, hi, p, q)` returns the points of the left side of a polygon, in clockwise order, after it has been cut by the line containing points p and q . The original convex polygon is given by the range `[lo, hi)` of points in clockwise order, where `lo` and `hi` must be random-access iterators.

Time Complexity: $O(n)$ per call to `convex_cut(lo, hi, p, q)`, where n is the distance between `lo` and `hi`.

Space Complexity: $O(n)$ auxiliary for storage of the resulting convex cut.

```

#include <cmath>
#include <cstddef>
#include <stdexcept>
#include <utility>
#include <vector>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LT(a, b) ((a) < (b) - EPS)
#define GT(a, b) ((a) > (b) + EPS)

typedef std::pair<double, double> point;
#define x first
#define y second

```

```

double cross(const point &a, const point &b, const point &o = point(0, 0)) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

int turn(const point &a, const point &o, const point &b) {
    double c = cross(a, b, o);
    return LT(c, 0) ? -1 : (GT(c, 0) ? 1 : 0);
}

int line_intersection(
    const point &p1, const point &p2, const point &p3, const point &p4, point *p = NULL
) {
    double a1 = p2.y - p1.y, b1 = p1.x - p2.x;
    double c1 = -(p1.x * p2.y - p2.x * p1.y);
    double a2 = p4.y - p3.y, b2 = p3.x - p4.x;
    double c2 = -(p3.x * p4.y - p4.x * p3.y);
    double x = -(c1 * b2 - c2 * b1), y = -(a1 * c2 - a2 * c1);
    double det = a1 * b2 - a2 * b1;
    if (EQ(det, 0)) {
        return (EQ(x, 0) && EQ(y, 0)) ? 1 : -1;
    }
    if (p != NULL) {
        *p = point(x / det, y / det);
    }
    return 0;
}

template<class It>
std::vector<point> convex_cut(It lo, It hi, const point &p, const point &q) {
    if (EQ(p.x, q.x) && EQ(p.y, q.y)) {
        throw std::runtime_error("Cannot cut using line from identical points.");
    }
    std::vector<point> res;
    for (It i = lo, j = hi - 1; i != hi; j = i++) {
        int d1 = turn(q, p, *j), d2 = turn(q, p, *i);
        if (d1 >= 0) {
            res.push_back(*j);
        }
        if (d1 * d2 < 0) {
            point r;
            line_intersection(p, q, *j, *i, &r);
            res.push_back(r);
        }
    }
    return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    {
        vector<point> v;
        v.push_back(point(1, 3));
        v.push_back(point(2, 2));
        v.push_back(point(2, 1));
        v.push_back(point(0, 0));
        v.push_back(point(-1, 3));
        // Cut using the vertical line through (0, 0).
        vector<point> c;
        c.push_back(point(-1, 3));
        c.push_back(point(0, 3));
        c.push_back(point(0, 0));
    }
}

```

```

    assert(convex_cut(v.begin(), v.end(), point(0, 0), point(0, 1)) == c);
}
{ // On a non-convex input, the result may be multiple disjoint polygons!
    vector<point> v;
    v.push_back(point(0, 0));
    v.push_back(point(2, 2));
    v.push_back(point(0, 4));
    v.push_back(point(3, 4));
    v.push_back(point(3, 0));
    vector<point> c;
    c.push_back(point(1, 0));
    c.push_back(point(0, 0));
    c.push_back(point(1, 1));
    c.push_back(point(1, 3));
    c.push_back(point(0, 4));
    c.push_back(point(1, 4));
    assert(convex_cut(v.begin(), v.end(), point(1, 0), point(1, 4)) == c);
}
return 0;
}

```

7.4.2 Polygon Intersection and Union

Given two polygons, determine the areas of their intersection and union using a sweep line algorithm and the inclusion-exclusion principle.

- `intersection_area(lo1, hi1, lo2, hi2)` returns the intersection area of two polygons respectively specified by two ranges `[lo1, hi1)` and `[lo2, hi2)` of vertices in clockwise order, where `lo1`, `hi1`, `lo2`, and `hi2` must be random-access iterators.
- `union_area(lo1, hi1, lo2, hi2)` returns the union area of two polygons respectively specified by two ranges `[lo1, hi1)` and `[lo2, hi2)` of vertices in clockwise order, where `lo1`, `hi1`, `lo2`, and `hi2` must be random-access iterators.

Time Complexity: $O(n^2 \log n)$ per call to `intersection_area(lo1, hi1, lo2, hi2)` and `union_area(lo1, hi1, lo2, hi2)` where n is the sum of distances between `lo1` and `hi1` and `lo2` and `hi2` respectively.

Space Complexity: $O(n)$ auxiliary heap space for `intersection_area(lo1, hi1, lo2, hi2)` and `union_area(lo1, hi1, lo2, hi2)`, where n is the sum of distances between `lo1` and `hi1` and `lo2` and `hi2` respectively.

```

#include <algorithm>
#include <cmath>
#include <set>
#include <utility>
#include <vector>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LT(a, b) ((a) < (b) - EPS)
#define LE(a, b) ((a) <= (b) + EPS)

typedef std::pair<double, double> point;
#define x first
#define y second

double sqnorm(const point &a) {
    return a.x * a.x + a.y * a.y;
}

double dot(const point &a, const point &b) {

```

```

    return a.x * b.x + a.y * b.y;
}

double cross(const point &a, const point &b, const point &o = point(0, 0)) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

int seg_intersection(
    const point &a, const point &b, const point &c, const point &d, point *p = NULL, point *q = NULL
) {
    static const bool TOUCH_IS_INTERSECT = true;
    point ab(b.x - a.x, b.y - a.y);
    point ac(c.x - a.x, c.y - a.y);
    point cd(d.x - c.x, d.y - c.y);
    double c1 = cross(ab, cd), c2 = cross(ac, ab);
    if (EQ(c1, 0) && EQ(c2, 0)) { // Collinear.
        double t0 = dot(ac, ab) / sqnorm(ab);
        double t1 = t0 + dot(cd, ab) / sqnorm(ab);
        double mint = std::min(t0, t1), maxt = std::max(t0, t1);
        bool overlap = TOUCH_IS_INTERSECT ? (LE(mint, 1) && LE(0, maxt)) : (LT(mint, 1) && LT(0, maxt));
        if (overlap) {
            point res1 = std::max(std::min(a, b), std::min(c, d));
            point res2 = std::min(std::max(a, b), std::max(c, d));
            if (res1 == res2) {
                if (p != NULL) {
                    *p = res1;
                }
                return 0; // Collinear and meeting at an endpoint.
            }
            if (p != NULL && q != NULL) {
                *p = res1;
                *q = res2;
            }
            return 1; // Collinear and overlapping.
        } else {
            return -1; // Collinear and disjoint.
        }
    }
    if (EQ(c1, 0)) {
        return -1; // Parallel and disjoint.
    }
    double t = cross(ac, cd) / c1, u = c2 / c1;
    bool t_between_01 = TOUCH_IS_INTERSECT ? (LE(0, t) && LE(t, 1)) : (LT(0, t) && LT(t, 1));
    bool u_between_01 = TOUCH_IS_INTERSECT ? (LE(0, u) && LE(u, 1)) : (LT(0, u) && LT(u, 1));
    if (t_between_01 && u_between_01) {
        if (p != NULL) {
            *p = point(a.x + t * ab.x, a.y + t * ab.y);
        }
        return 0; // Non-parallel with one intersection.
    }
    return -1; // Non-parallel with no intersections.
}

int line_intersection(
    const point &p1, const point &p2, const point &p3, const point &p4, point *p = NULL
) {
    double a1 = p2.y - p1.y, b1 = p1.x - p2.x;
    double c1 = -(p1.x * p2.y - p2.x * p1.y);
    double a2 = p4.y - p3.y, b2 = p3.x - p4.x;
    double c2 = -(p3.x * p4.y - p4.x * p3.y);
    double x = -(c1 * b2 - c2 * b1), y = -(a1 * c2 - a2 * c1);
    double det = a1 * b2 - a2 * b1;
    if (EQ(det, 0)) {
        return (EQ(x, 0) && EQ(y, 0)) ? 1 : -1;
    }
    if (p != NULL) {
        *p = point(x / det, y / det);
    }
}

```

```

    return 0;
}

struct event {
    double y;
    int mask_delta;

    event(double y = 0, int mask_delta = 0) {
        this->y = y;
        this->mask_delta = mask_delta;
    }

    bool operator<(const event &e) const {
        if (y != e.y) {
            return y < e.y;
        }
        return mask_delta < e.mask_delta;
    }
};

template<class It>
double intersection_area(It lo1, It hi1, It lo2, It hi2) {
    It plo[2] = {lo1, lo2}, phi[] = {hi1, hi2};
    std::set<double> xs;
    for (It i1 = lo1; i1 != hi1; ++i1) {
        xs.insert(i1->x);
    }
    for (It i2 = lo2; i2 != hi2; ++i2) {
        xs.insert(i2->x);
    }
    for (It i1 = lo1, j1 = hi1 - 1; i1 != hi1; j1 = i1++) {
        for (It i2 = lo2, j2 = hi2 - 1; i2 != hi2; j2 = i2++) {
            point p;
            if (seg_intersection(*i1, *j1, *i2, *j2, &p) == 0) {
                xs.insert(p.x);
            }
        }
    }
    std::vector<double> xsa(xs.begin(), xs.end());
    double res = 0;
    for (int k = 0; k < (int)xsa.size() - 1; k++) {
        double x = (xsa[k] + xsa[k + 1]) / 2;
        point sweep0(x, 0), sweep1(x, 1);
        std::vector<event> events;
        for (int poly = 0; poly < 2; poly++) {
            It lo = plo[poly], hi = phi[poly];
            double area = 0;
            for (It i = lo, j = hi - 1; i != hi; j = i++) {
                area += (j->x - i->x) * (j->y + i->y);
            }
            for (It j = lo, i = hi - 1; j != hi; i = j++) {
                point p;
                if (line_intersection(*j, *i, sweep0, sweep1, &p) == 0) {
                    double y = p.y, x0 = i->x, x1 = j->x;
                    int sgn_area = (area < 0 ? -1 : (area > 0 ? 1 : 0));
                    if (x0 < x && x1 > x) {
                        events.push_back(event(y, sgn_area * (1 << poly)));
                    } else if (x0 > x && x1 < x) {
                        events.push_back(event(y, -sgn_area * (1 << poly)));
                    }
                }
            }
        }
    }
    std::sort(events.begin(), events.end());
    double a = 0;
    int mask = 0;
    for (int j = 0; j < (int)events.size(); j++) {
        if (mask == 3) {

```



```

        a += events[j].y - events[j - 1].y;
    }
    mask += events[j].mask_delta;
}
res += a * (xsa[k + 1] - xsa[k]);
}
return res;
}

template<class It>
double polygon_area(It lo, It hi) {
    if (lo == hi) {
        return 0;
    }
    double area = 0;
    if (*lo != *--hi) {
        area += (lo->x - hi->x) * (lo->y + hi->y);
    }
    for (It i = hi, j = --hi; i != lo; --i, --j) {
        area += (i->x - j->x) * (i->y + j->y);
    }
    return fabs(area / 2.0);
}

template<class It>
double union_area(It lo1, It hi1, It lo2, It hi2) {
    return polygon_area(lo1, hi1) + polygon_area(lo2, hi2) - intersection_area(lo1, hi1, lo2, hi2);
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<point> p, s;
    // Irregular pentagon a triangle of area 1.5 overlapping quadrant 2.
    p.push_back(point(1, 3));
    p.push_back(point(1, 2));
    p.push_back(point(2, 1));
    p.push_back(point(0, 0));
    p.push_back(point(-1, 3));
    // Square of area 12.5 in quadrant 2.
    s.push_back(point(0, 0));
    s.push_back(point(0, 3));
    s.push_back(point(-3, 3));
    s.push_back(point(-3, 0));
    assert(EQ(1.5, intersection_area(p.begin(), p.end(), s.begin(), s.end())));
    assert(EQ(12.5, union_area(p.begin(), p.end(), s.begin(), s.end())));
    return 0;
}

```

7.4.3 Delaunay Triangulation (Simple)

Given a set P of two dimensional points, the Delaunay triangulation of P is a set of non-overlapping triangles that covers the entire convex hull of P such that no point in P lies within the circumcircle of any of the resulting triangles. For any point p in the convex hull of P (but not necessarily in P), the nearest point is guaranteed to be a vertex of the enclosing triangle from the triangulation.

The triangulation may not exist (e.g. for a set of collinear points), or may not be unique if it does exist. The following program assumes its existence and produces one such valid result using a simple algorithm which encases each triangle in a circle and rejects the triangle if another point in the tessellation is within the generalized circle.

- `delaunay_triangulation(lo, hi)` returns a Delaunay triangulation for the input range `[lo, hi)` of points, where `lo` and `hi` must be random-access iterators, or an empty vector if a triangulation does not exist.

Time Complexity: $O(n^4)$ per call to `delaunay_triangulation(lo, hi)`, where n is the distance between `lo` and `hi`.

Space Complexity: $O(n)$ auxiliary heap space for storage of the Delaunay triangulation.

```
#include <algorithm>
#include <cmath>
#include <utility>
#include <vector>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LT(a, b) ((a) < (b) - EPS)
#define LE(a, b) ((a) <= (b) + EPS)

typedef std::pair<double, double> point;
#define x first
#define y second

double sqnorm(const point &a) {
    return a.x * a.x + a.y * a.y;
}

double dot(const point &a, const point &b) {
    return a.x * b.x + a.y * b.y;
}

double cross(const point &a, const point &b, const point &o = point(0, 0)) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

int seg_intersection(
    const point &a, const point &b, const point &c, const point &d, point *p = NULL, point *q = NULL
) {
    static const bool TOUCH_IS_INTERSECT = false; // false is important!
    point ab(b.x - a.x, b.y - a.y);
    point ac(c.x - a.x, c.y - a.y);
    point cd(d.x - c.x, d.y - c.y);
    double c1 = cross(ab, cd), c2 = cross(ac, ab);
    if (EQ(c1, 0) && EQ(c2, 0)) { // Collinear.
        double t0 = dot(ac, ab) / sqnorm(ab);
        double t1 = t0 + dot(cd, ab) / sqnorm(ab);
        double mint = std::min(t0, t1), maxt = std::max(t0, t1);
        bool overlap = TOUCH_IS_INTERSECT ? (LE(mint, 1) && LE(0, maxt)) : (LT(mint, 1) && LT(0, maxt));
        if (overlap) {
            point res1 = std::max(std::min(a, b), std::min(c, d));
            point res2 = std::min(std::max(a, b), std::max(c, d));
            if (res1 == res2) {
                if (p != NULL) {
                    *p = res1;
                }
                return 0; // Collinear and meeting at an endpoint.
            }
            if (p != NULL && q != NULL) {
                *p = res1;
                *q = res2;
            }
            return 1; // Collinear and overlapping.
        } else {
            return -1; // Collinear and disjoint.
        }
    }
    if (EQ(c1, 0)) {
        return -1; // Parallel and disjoint.
    }
}
```

```

double t = cross(ac, cd) / c1, u = c2 / c1;
bool t_between_01 = TOUCH_IS_INTERSECT ? (LE(0, t) && LE(t, 1)) : (LT(0, t) && LT(t, 1));
bool u_between_01 = TOUCH_IS_INTERSECT ? (LE(0, u) && LE(u, 1)) : (LT(0, u) && LT(u, 1));
if (t_between_01 && u_between_01) {
    if (p != NULL) {
        *p = point(a.x + t * ab.x, a.y + t * ab.y);
    }
    return 0; // Non-parallel with one intersection.
}
return -1; // Non-parallel with no intersections.
}

struct triangle {
    point a, b, c;

    triangle(const point &a, const point &b, const point &c) : a(a), b(b), c(c) {}

    bool operator==(const triangle &t) const {
        return EQ(a.x, t.a.x) && EQ(a.y, t.a.y) && EQ(b.x, t.b.x) && EQ(b.y, t.b.y) && EQ(c.x, t.c.x) &&
            EQ(c.y, t.c.y);
    }
};

template<class It>
std::vector<triangle> delaunay_triangulation(It lo, It hi) {
    int n = hi - lo;
    std::vector<double> x, y, z;
    for (It it = lo; it != hi; ++it) {
        x.push_back(it->x);
        y.push_back(it->y);
        z.push_back(sqnorm(*it));
    }
    std::vector<triangle> res;
    for (int i = 0; i < n - 2; i++) {
        for (int j = i + 1; j < n; j++) {
            for (int k = i + 1; k < n; k++) {
                if (j == k) {
                    continue;
                }
                double nx = (y[j] - y[i]) * (z[k] - z[i]) - (y[k] - y[i]) * (z[j] - z[i]);
                double ny = (x[k] - x[i]) * (z[j] - z[i]) - (x[j] - x[i]) * (z[k] - z[i]);
                double nz = (x[j] - x[i]) * (y[k] - y[i]) - (x[k] - x[i]) * (y[j] - y[i]);
                if (LE(0, nz)) {
                    continue;
                }
                point s1[] = {lo[i], lo[j], lo[k], lo[i]};
                for (int m = 0; m < n; m++) {
                    if (nx * (x[m] - x[i]) + ny * (y[m] - y[i]) + nz * (z[m] - z[i]) > 0) {
                        goto skip;
                    }
                }
                // Handle four points on a circle.
                for (int t = 0; t < (int)res.size(); t++) {
                    point s2[] = {res[t].a, res[t].b, res[t].c, res[t].a};
                    for (int u = 0; u < 3; u++) {
                        for (int v = 0; v < 3; v++) {
                            if (seg_intersection(s1[u], s1[u + 1], s2[v], s2[v + 1]) == 0) {
                                goto skip;
                            }
                        }
                    }
                }
                res.push_back(triangle(lo[i], lo[j], lo[k]));
            }
        }
        skip:;
    }
    return res;
}

```

```
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    vector<point> v;
    v.push_back(point(1, 3));
    v.push_back(point(1, 2));
    v.push_back(point(2, 1));
    v.push_back(point(0, 0));
    v.push_back(point(-1, 3));
    vector<triangle> t;
    t.push_back(triangle(point(1, 3), point(1, 2), point(-1, 3)));
    t.push_back(triangle(point(1, 3), point(2, 1), point(1, 2)));
    t.push_back(triangle(point(1, 2), point(2, 1), point(0, 0)));
    t.push_back(triangle(point(1, 2), point(0, 0), point(-1, 3)));
    assert(delaunay_triangulation(v.begin(), v.end()) == t);
    return 0;
}
```

7.4.4 Delaunay Triangulation (Fast)

Given a set P of two dimensional points, the Delaunay triangulation of P is a set of non-overlapping triangles that covers the entire convex hull of P such that no point in P lies within the circumcircle of any of the resulting triangles. For any point p in the convex hull of P (but not necessarily in P), the nearest point is guaranteed to be a vertex of the enclosing triangle from the triangulation.

The triangulation may not exist (e.g. for a set of collinear points), or may not be unique if it does exist. The following program produces one such valid result using the Guibas-Stolfi divide-and-conquer algorithm with a quad-edge data structure. A full description is available in the original paper:

<https://people.eecs.berkeley.edu/~jrs/meshpapers/GuibasStolfi.pdf>

- `delaunay_triangulation(lo, hi)` returns a Delaunay triangulation for the input range `[lo, hi)` of points, or an empty vector if a triangulation does not exist. Duplicate points are ignored.

Time Complexity: $O(n \log n)$ per call to `delaunay_triangulation(lo, hi)`, where n is the distance between `lo` and `hi`.

Space Complexity: $O(n)$ auxiliary heap space for storage of the Delaunay triangulation.

```
#include <algorithm>
#include <cmath>
#include <utility>
#include <vector>

const double EPS = 1e-9;

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define LT(a, b) ((a) < (b) - EPS)
#define GT(a, b) ((a) > (b) + EPS)

typedef std::pair<double, double> point;
#define x first
#define y second

double cross(const point &a, const point &b, const point &o = point(0, 0)) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}
```

```

long double incircle(const point &a, const point &b, const point &c, const point &d) {
    long double adx = a.x - d.x, ady = a.y - d.y;
    long double bdx = b.x - d.x, bdy = b.y - d.y;
    long double cdx = c.x - d.x, cdy = c.y - d.y;
    long double abdet = adx * bdy - bdx * ady;
    long double bcdet = bdx * cdy - cdx * bdy;
    long double cadet = cdx * ady - adx * cdy;
    long double alift = adx * adx + ady * ady;
    long double blift = bdx * bdx + bdy * bdy;
    long double clift = cdx * cdx + cdy * cdy;
    return alift * bcdet + blift * cadet + clift * abdet;
}

struct edge {
    point origin;
    edge *rot, *onext;
    bool primal, deleted, used;

    edge *rev() const { return rot->rot; }
    edge *lnext() const { return rot->rev()->onext->rot; }
    edge *oprev() const { return rot->onext->rot; }
    point dest() const { return rev()->origin; }
};

struct quad_edge_pool {
    std::vector<edge *> edges;

    ~quad_edge_pool() {
        for (int i = 0; i < (int)edges.size(); i++) {
            delete edges[i];
        }
    }

    edge *make_edge(const point &from, const point &to) {
        edge *e1 = new edge, *e2 = new edge, *e3 = new edge, *e4 = new edge;
        edges.push_back(e1);
        edges.push_back(e2);
        edges.push_back(e3);
        edges.push_back(e4);
        e1->origin = from;
        e2->origin = to;
        e1->rot = e3;
        e2->rot = e4;
        e3->rot = e2;
        e4->rot = e1;
        e1->onext = e1;
        e2->onext = e2;
        e3->onext = e4;
        e4->onext = e3;
        e1->primal = e2->primal = true;
        e3->primal = e4->primal = false;
        e1->deleted = e2->deleted = e3->deleted = e4->deleted = false;
        e1->used = e2->used = e3->used = e4->used = false;
        return e1;
    }

    void delete_edge(edge *e) {
        splice(e, e->oprev());
        splice(e->rev(), e->rev()->oprev());
        e->deleted = e->rot->deleted = e->rev()->deleted = e->rev()->rot->deleted = true;
    }

    edge *connect(edge *a, edge *b) {
        edge *e = make_edge(a->dest(), b->origin);
        splice(e, a->lnext());
        splice(e->rev(), b);
        return e;
    }
}

```

```

}

static void splice(edge *a, edge *b) {
    std::swap(a->onext->rot->onext, b->onext->rot->onext);
    std::swap(a->onext, b->onext);
}

};

bool left_of(const point &p, edge *e) {
    return GT(cross(e->origin, e->dest(), p), 0);
}

bool right_of(const point &p, edge *e) {
    return LT(cross(e->origin, e->dest(), p), 0);
}

bool in_circle(const point &a, const point &b, const point &c, const point &d) {
    return incircle(a, b, c, d) > EPS;
}

std::pair<edge *, edge *> build_triangulation(
    std::vector<point> &p, int lo, int hi, quad_edge_pool &pool
) {
    if (hi - lo == 1) {
        edge *a = pool.make_edge(p[lo], p[hi]);
        return std::make_pair(a, a->rev());
    }
    if (hi - lo == 2) {
        edge *a = pool.make_edge(p[lo], p[lo + 1]);
        edge *b = pool.make_edge(p[lo + 1], p[hi]);
        pool.splice(a->rev(), b);
        int side = GT(cross(p[lo], p[lo + 1], p[hi]), 0)
            ? 1
            : (LT(cross(p[lo], p[lo + 1], p[hi]), 0) ? -1 : 0);
        if (side == 0) {
            return std::make_pair(a, b->rev());
        }
        edge *c = pool.connect(b, a);
        return side == 1 ? std::make_pair(a, b->rev()) : std::make_pair(c->rev(), c);
    }
    int mid = (lo + hi) / 2;
    edge *ldo, *ldi, *rdi, *rdo;
    std::pair<edge *, edge *> left = build_triangulation(p, lo, mid, pool);
    std::pair<edge *, edge *> right = build_triangulation(p, mid + 1, hi, pool);
    ldo = left.first;
    ldi = left.second;
    rdi = right.first;
    rdo = right.second;
    for (;;) {
        if (left_of(rdi->origin, ldi)) {
            ldi = ldi->lnext();
        } else if (right_of(ldi->origin, rdi)) {
            rdi = rdi->rev()->onext;
        } else {
            break;
        }
    }
    edge *base = pool.connect(rdi->rev(), ldi);
    if (ldi->origin == ldo->origin) {
        ldo = base->rev();
    }
    if (rdi->origin == rdo->origin) {
        rdo = base;
    }
    for (;;) {
        edge *lcand = base->rev()->onext;
        if (right_of(lcand->dest(), base)) {
            while (in_circle(base->dest(), base->origin, lcand->dest(), lcand->onext->dest())) {

```

```

        edge *next = lcand->onext;
        pool.delete_edge(lcand);
        lcand = next;
    }
}
edge *rcand = base->oprev();
if (right_of(rcand->dest(), base)) {
    while (in_circle(base->dest(), base->origin, rcand->dest(), rcand->oprev()->dest())) {
        edge *prev = rcand->oprev();
        pool.delete_edge(rcand);
        rcand = prev;
    }
}
bool lvalid = right_of(lcand->dest(), base);
bool rvalid = right_of(rcand->dest(), base);
if (!lvalid && !rvalid) {
    break;
}
if (!lvalid ||
    (rvalid && in_circle(lcand->dest(), lcand->origin, rcand->origin, rcand->dest()))) {
    base = pool.connect(rcand, base->rev());
} else {
    base = pool.connect(base->rev(), lcand->rev());
}
}
return std::make_pair(ldo, rdo);
}

struct triangle {
    point a, b, c;

    triangle(const point &a, const point &b, const point &c) : a(a), b(b), c(c) {
        if (b < a && b < c) {
            this->a = b;
            this->b = c;
            this->c = a;
        } else if (c < a && c < b) {
            this->a = c;
            this->b = a;
            this->c = b;
        }
    }
};

bool operator==(const triangle &t) const { return a == t.a && b == t.b && c == t.c; }

bool operator<(const triangle &t) const {
    return a != t.a ? a < t.a : (b != t.b ? b < t.b : c < t.c);
}

};

template<class It>
std::vector<triangle> delaunay_triangulation(It lo, It hi) {
    std::vector<point> p(lo, hi);
    std::sort(p.begin(), p.end());
    p.erase(std::unique(p.begin(), p.end()), p.end());
    if (p.size() < 3) {
        return std::vector<triangle>();
    }
    quad_edge_pool pool;
    build_triangulation(p, 0, p.size() - 1, pool);
    std::vector<triangle> res;
    for (int i = 0; i < (int)pool.edges.size(); i++) {
        edge *start = pool.edges[i];
        if (!start->primal || start->deleted || start->used) {
            continue;
        }
        std::vector<point> face;
        edge *curr = start;

```

```

do {
    curr->used = true;
    face.push_back(curr->origin);
    curr = curr->lnext();
} while (curr != start);
if (face.size() == 3 && GT(cross(face[0], face[1], face[2]), 0)) {
    res.push_back(triangle(face[0], face[1], face[2]));
}
}
std::sort(res.begin(), res.end());
return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<point> v;
    v.push_back(point(1, 3));
    v.push_back(point(1, 2));
    v.push_back(point(2, 1));
    v.push_back(point(0, 0));
    v.push_back(point(-1, 3));
    vector<triangle> t;
    t.push_back(triangle(point(-1, 3), point(0, 0), point(1, 2)));
    t.push_back(triangle(point(-1, 3), point(1, 2), point(1, 3)));
    t.push_back(triangle(point(0, 0), point(2, 1), point(1, 2)));
    t.push_back(triangle(point(1, 2), point(2, 1), point(1, 3)));
    assert(delaunay_triangulation(v.begin(), v.end()) == t);
    return 0;
}

```

7.5 Combined Geometry Template

7.5.1 Geometry Library in One File

This combines the next few subsections in this chapter in a cross-dependent manner. All of these algorithms apply to a two-dimensional Cartesian plane.

Time Complexity: $O(1)$ for all operations.

Space Complexity:

- $O(1)$ for storage of all data types.
- $O(1)$ auxiliary for all operations.

```

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <limits>
#include <ostream>
#include <stdexcept>
#include <utility>
#include <vector>

const double M_NAN = std::numeric_limits<double>::quiet_NaN();
const double EPS = 1e-9;

```



```

#define EQ(a, b) (fabs((a) - (b)) <= EPS)
#define NE(a, b) (fabs((a) - (b)) > EPS)
#define LT(a, b) ((a) < (b) - EPS)
#define GT(a, b) ((a) > (b) + EPS)
#define LE(a, b) ((a) <= (b) + EPS)
#define GE(a, b) ((a) >= (b) - EPS)

// A two-dimensional point class that behaves like std::complex while supporting
// epsilon comparisons.
struct point {
    double x, y;

    point() : x(0), y(0) {}
    point(double x, double y) : x(x), y(y) {}
    point(const point &p) : x(p.x), y(p.y) {}
    point(const std::pair<double, double> &p) : x(p.first), y(p.second) {}

    bool operator<(const point &p) const { return EQ(x, p.x) ? LT(y, p.y) : LT(x, p.x); }
    bool operator>(const point &p) const { return EQ(x, p.x) ? LT(p.y, y) : LT(p.x, x); }
    bool operator==(const point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
    bool operator!=(const point &p) const { return !(*this == p); }
    bool operator<=(const point &p) const { return !(*this > p); }
    bool operator>=(const point &p) const { return !(*this < p); }
    point operator+(const point &p) const { return point(x + p.x, y + p.y); }
    point operator-(const point &p) const { return point(x - p.x, y - p.y); }
    point operator+(double v) const { return point(x + v, y + v); }
    point operator-(double v) const { return point(x - v, y - v); }
    point operator*(double v) const { return point(x * v, y * v); }
    point operator/(double v) const { return point(x / v, y / v); }
    point &operator+=(const point &p) { x += p.x; y += p.y; return *this; }
    point &operator-=(const point &p) { x -= p.x; y -= p.y; return *this; }
    point &operator+=(double v) { x += v; y += v; return *this; }
    point &operator-=(double v) { x -= v; y -= v; return *this; }
    point &operator*=(double v) { x *= v; y *= v; return *this; }
    point &operator/=(double v) { x /= v; y /= v; return *this; }
    friend point operator+(double v, const point &p) { return p + v; }
    friend point operator*(double v, const point &p) { return p * v; }

    double sqnorm() const { return x * x + y * y; }
    double norm() const { return sqrt(x * x + y * y); }
    double arg() const { return atan2(y, x); }
    double dot(const point &p) const { return x * p.x + y * p.y; }
    double cross(const point &p) const { return x * p.y - y * p.x; }
    double proj(const point &p) const { return dot(p) / p.norm(); }

    // Returns a proportional unit vector (p, q) = c(x, y) where p^2 + q^2 = 1.
    point normalize() const { return (EQ(x, 0) && EQ(y, 0)) ? point(0, 0) : (point(x, y) / norm()); }

    // Returns (x, y) rotated 90 degrees clockwise about the origin.
    point rotate90() const { return point(-y, x); }

    // Returns (x, y) rotated t radians clockwise about the origin.
    point rotateCW(double t) const { return point(x * cos(t) + y * sin(t), y * cos(t) - x * sin(t)); }

    // Returns (x, y) rotated t radians counter-clockwise about the origin.
    point rotateCCW(double t) const {
        return point(x * cos(t) - y * sin(t), x * sin(t) + y * cos(t));
    }

    // Returns (x, y) rotated t radians clockwise about point p.
    point rotateCW(const point &p, double t) const { return (*this - p).rotateCW(t) + p; }

    // Returns (x, y) rotated t radians counter-clockwise about the point p.
    point rotateCCW(const point &p, double t) const { return (*this - p).rotateCCW(t) + p; }

    // Returns (x, y) reflected across point p.
    point reflect(const point &p) const { return point(2 * p.x - x, 2 * p.y - y); }

```

```

// Returns (x, y) reflected across the line containing points p and q.
point reflect(const point &p, const point &q) const {
    if (p == q) {
        return reflect(p);
    }
    point r(*this - p), s = q - p;
    r = point(r.x * s.x + r.y * s.y, r.x * s.y - r.y * s.x) / s.sqnorm();
    r = point(r.x * s.x - r.y * s.y, r.x * s.y + r.y * s.x) + p;
    return r;
}

friend double sqnorm(const point &p) { return p.sqnorm(); }
friend double norm(const point &p) { return p.norm(); }
friend double arg(const point &p) { return p.arg(); }
friend double dot(const point &p, const point &q) { return p.dot(q); }
friend double cross(const point &p, const point &q) { return p.cross(q); }
friend double proj(const point &p, const point &q) { return p.proj(q); }
friend point normalize(const point &p) { return p.normalize(); }
friend point rotate90(const point &p) { return p.rotate90(); }
friend point rotateCW(const point &p, double t) { return p.rotateCW(t); }
friend point rotateCCW(const point &p, double t) { return p.rotateCCW(t); }
friend point rotateCW(const point &p, const point &q, double t) { return p.rotateCW(q, t); }
friend point rotateCCW(const point &p, const point &q, double t) { return p.rotateCCW(q, t); }
friend point reflect(const point &p, const point &q) { return p.reflect(q); }
friend point reflect(const point &p, const point &a, const point &b) { return p.reflect(a, b); }

friend std::ostream &operator<<(std::ostream &out, const point &p) {
    return out << "(" << (fabs(p.x) < EPS ? 0 : p.x) << "," << (fabs(p.y) < EPS ? 0 : p.y) << ")";
}
};

// A two-dimensional line class stored of the form ax + by + c = 0, normalized
// such that b is always either 1 (for normal line) or 0 (for vertical lines).
struct line {
    double a, b, c;

    line() : a(0), b(0), c(0) {} // Invalid or uninitialized line.

    line(double a, double b, double c) {
        if (!EQ(b, 0)) {
            this->a = a / b;
            this->c = c / b;
            this->b = 1;
        } else {
            this->c = c / a;
            this->a = 1;
            this->b = 0;
        }
    }

    line(double slope, const point &p) {
        a = -slope;
        b = 1;
        c = slope * p.x - p.y;
    }

    line(const point &p, const point &q) : a(0), b(0), c(0) {
        if (EQ(p.x, q.x)) {
            if (NE(p.y, q.y)) { // Vertical line.
                a = 1;
                b = 0;
                c = -p.x;
            } // Else, invalid line.
        } else {
            a = -(p.y - q.y) / (p.x - q.x);
            b = 1;
            c = -(a * p.x) - (b * p.y);
        }
    }
}

```

```

}

bool operator==(const line &l) const { return EQ(a, l.a) && EQ(b, l.b) && EQ(c, l.c); }
bool operator!=(const line &l) const { return !(*this == l); }

// Returns whether the line is initialized and normalized.
bool valid() const {
    if (EQ(a, 0)) {
        return !EQ(b, 0);
    }
    return EQ(b, 1) || (EQ(b, 0) && EQ(a, 1));
}

bool horizontal() const { return valid() && EQ(a, 0); }
bool vertical() const { return valid() && EQ(b, 0); }
double slope() const { return (!valid() || EQ(b, 0)) ? M_NAN : -a; }

// Solve for x at a given y. If the line is horizontal, then either -INF, INF,
// or NAN is returned based on whether y is below, above, or on the line.
double x(double y) const {
    if (!valid() || EQ(a, 0)) {
        return M_NAN; // Invalid or horizontal line.
    }
    return (-c - b * y) / a;
}

// Solve for y at a given x. If the line is vertical, then either -INF, INF,
// or NAN is returned based on whether x is left of, right of, or on the line.
double y(double x) const {
    if (!valid() || EQ(b, 0)) {
        return M_NAN; // Invalid or vertical line.
    }
    return (-c - a * x) / b;
}

bool contains(const point &p) const { return EQ(a * p.x + b * p.y + c, 0); }
bool is_parallel(const line &l) const { return EQ(a, l.a) && EQ(b, l.b); }
bool is_perpendicular(const line &l) const { return EQ(-a * l.a, b * l.b); }

// Return the parallel line passing through point p.
line parallel(const point &p) const { return line(a, b, -a * p.x - b * p.y); }

// Return the perpendicular line passing through point p.
line perpendicular(const point &p) const { return line(-b, a, b * p.x - a * p.y); }

friend std::ostream &operator<<(std::ostream &out, const line &l) {
    return out << (fabs(l.a) < EPS ? 0 : l.a) << "x" << std::showpos << (fabs(l.b) < EPS ? 0 : l.b)
        << "y" << (fabs(l.c) < EPS ? 0 : l.c) << "=0" << std::noshowpos;
}
};

const double PI = acos(-1.0), DEG = PI / 180, RAD = 180 / PI;

// Returns t degrees reduced to the range [0, 360). E.g. -630 becomes 90.
double reduce_deg(double t) {
    if (t < -360) {
        return reduce_deg(fmod(t, 360));
    }
    if (t < 0) {
        return t + 360;
    }
    return (t >= 360) ? fmod(t, 360) : t;
}

// Returns t radians reduced to the range [0, 2*pi). E.g. 720.5 becomes 0.5.
double reduce_rad(double t) {
    if (t < -2 * PI) {
        return reduce_rad(fmod(t, 2 * PI));
    }

```

```

}
if (t < 0) {
    return t + 2 * PI;
}
return (t >= 2 * PI) ? fmod(t, 2 * PI) : t;
}

// Returns a two-dimensional Cartesian point given radius r and angle t radians
// in polar coordinates, analogous to std::polar().
point polar_point(double r, double t) {
    return point(r * cos(t), r * sin(t));
}

// Returns the angle in radians of the segment from (0, 0) to point p, relative
// counterclockwise to the positive x-axis.
double polar_angle(const point &p) {
    double t = arg(p);
    return (t < 0) ? (t + 2 * PI) : t;
}

// Returns the smallest angle in radians formed by the points a, o, b with
// vertex at point o.
double angle(const point &a, const point &o, const point &b) {
    point u(o - a), v(o - b);
    return acos(u.dot(v) / (norm(u) * norm(v)));
}

// Returns the angle in radians of segment from point a to point b, relative
// counterclockwise to the positive x-axis.
double angle_between(const point &a, const point &b) {
    double t = atan2(a.cross(b), a.dot(b)); // Equivalently, b.arg() - a.arg().
    return (t < 0) ? (t + 2 * PI) : t;
}

// Returns the smaller angle in radians between two lines, limited to [0, PI/2].
double angle_between(const line &l1, const line &l2) {
    double t = atan2(l1.a * l2.b - l2.a * l1.b, l1.a * l2.a + l1.b * l2.b);
    if (t < 0) {
        t += PI;
    }
    return GT(t, PI / 2) ? (PI - t) : t;
}

// Returns the magnitude (Euclidean norm) of the three-dimensional cross product
// between points a and b where the z-component is implicitly zero and the
// origin is implicitly shifted to point o. This operation is also equal to
// double the signed area of the triangle from these three points.
double cross(const point &a, const point &b, const point &o = point(0, 0)) {
    return (a - o).cross(b - o);
}

// Returns -1 if the path a->o->b forms a left turn on the plane, 0 if the path
// forms a straight line segment (collinear), or 1 if it forms a right turn.
int turn(const point &a, const point &o, const point &b) {
    double c = cross(a, b, o);
    return LT(c, 0) ? -1 : (GT(c, 0) ? 1 : 0);
}

// Returns the distance and squared distance between points a and b.
double dist(const point &a, const point &b) {
    return norm(b - a);
}
double sqdist(const point &a, const point &b) {
    return sqnorm(b - a);
}

// Returns the distance from point p to line l. If l is invalid (l.a = l.b = 0),
// then -INF, INF, or NaN is returned based on the sign of l.c.

```

```

double line_dist(const point &p, const line &l) {
    return fabs(l.a * p.x + l.b * p.y + l.c) / sqrt(l.a * l.a + l.b * l.b);
}

// Returns the distance from point p to the line (not segment) containing points
// a and b. If a = b, then the distance from p to the single point is returned.
double line_dist(const point &p, const point &a, const point &b) {
    return (a == b) ? dist(p, a) : norm(a + (p - a).dot(b - a) * (b - a) / sqdist(a, b) - p);
}

// Returns the distance between two lines. If the lines are non-parallel then
// the distance is considered to be 0. Otherwise, the distance is considered to
// be the perpendicular distance from any point on one line to the other line.
double line_dist(const line &l1, const line &l2) {
    if (EQ(l1.a * l2.b, l2.a * l1.b)) {
        double factor = EQ(l1.b, 0) ? (l1.a / l2.a) : (l1.b / l2.b);
        return EQ(l1.c, l2.c * factor) ? 0
            : fabs(l2.c * factor - l1.c) / sqrt(l1.a * l1.a + l1.b * l1.b);
    }
    return 0;
}

// Returns the distance from point p to the line segment ab.
double seg_dist(const point &p, const point &a, const point &b) {
    if (a == b) {
        return dist(p, a);
    }
    point ab(b - a), ap(p - a);
    double n = sqnorm(ab), d = ab.dot(ap);
    if (LE(d, 0) || EQ(n, 0)) {
        return norm(ap);
    }
    return GE(d, n) ? norm(ap - ab) : norm(ap - ab * (d / n));
}

// Determines whether lines l1 and l2 intersect. Returns -1 if there is no
// intersection because the lines are parallel, 0 if there is exactly one
// intersection (in which case the intersection point is stored into pointer p
// if it's not NULL), or 1 if there are infinite intersections because the lines
// are identical.
int line_intersection(const line &l1, const line &l2, point *p = NULL) {
    if (l1.is_parallel(l2)) {
        return (l1 == l2) ? 1 : -1;
    }
    if (p != NULL) {
        p->x = (l1.b * l2.c - l2.b * l1.c) / (l1.a * l2.b - l2.a * l1.b);
        if (!EQ(l1.b, 0)) {
            p->y = -(l1.a * p->x + l1.c) / l1.b;
        } else {
            p->y = -(l2.a * p->x + l2.c) / l2.b;
        }
    }
    return 0;
}

// Determines whether the infinite lines (not segments) through points p1, p2
// and through points p3, p4 intersect. Returns -1 if there is no intersection
// because the lines are parallel, 0 if there is exactly one intersection (in
// which case the intersection point is stored into pointer p if it's not NULL),
// or 1 if there are infinite intersections because the lines are identical.
int line_intersection(
    const point &p1, const point &p2, const point &p3, const point &p4, point *p = NULL
) {
    double a1 = p2.y - p1.y, b1 = p1.x - p2.x;
    double c1 = -(p1.x * p2.y - p2.x * p1.y);
    double a2 = p4.y - p3.y, b2 = p3.x - p4.x;
    double c2 = -(p3.x * p4.y - p4.x * p3.y);
    double x = -(c1 * b2 - c2 * b1), y = -(a1 * c2 - a2 * c1);

```

```

double det = a1 * b2 - a2 * b1;
if (EQ(det, 0)) {
    return (EQ(x, 0) && EQ(y, 0)) ? 1 : -1;
}
if (p != NULL) {
    *p = point(x / det, y / det);
}
return 0;
}

// Determines whether the line segment ab intersects the line segment cd.
// Returns -1 if the segments do not intersect, 0 if there is exactly one
// intersection point (in which case it is stored into pointer p if it's not
// NULL), or 1 if the intersection is another line segment (in which case the
// two endpoints are stored into pointers p and q if they are not NULL). If the
// segments are barely touching (close within EPS), then the result will depend
// on the setting of TOUCH_IS_INTERSECT.
int seg_intersection(
    const point &a, const point &b, const point &c, const point &d, point *p = NULL, point *q = NULL
) {
    static const bool TOUCH_IS_INTERSECT = true;
    point ab(b - a), ac(c - a), cd(d - c);
    if (EQ(sqnorm(ab), 0)) {
        if (TOUCH_IS_INTERSECT && seg_dist(a, c, d) < EPS) {
            if (p != NULL) *p = a;
            return 0;
        }
        return -1;
    }
    if (EQ(sqnorm(cd), 0)) {
        if (TOUCH_IS_INTERSECT && seg_dist(c, a, b) < EPS) {
            if (p != NULL) *p = c;
            return 0;
        }
        return -1;
    }
    double c1 = ab.cross(cd), c2 = ac.cross(ab);
    if (EQ(c1, 0) && EQ(c2, 0)) { // Collinear.
        double t0 = ac.dot(ab) / sqnorm(ab);
        double t1 = t0 + cd.dot(ab) / sqnorm(ab);
        double mint = std::min(t0, t1), maxt = std::max(t0, t1);
        bool overlap = TOUCH_IS_INTERSECT ? (LE(mint, 1) && LE(0, maxt)) : (LT(mint, 1) && LT(0, maxt));
        if (overlap) {
            point res1 = std::max(std::min(a, b), std::min(c, d));
            point res2 = std::min(std::max(a, b), std::max(c, d));
            if (res1 == res2) {
                if (p != NULL) {
                    *p = res1;
                }
                return 0; // Collinear and meeting at an endpoint.
            }
            if (p != NULL && q != NULL) {
                *p = res1;
                *q = res2;
            }
            return 1; // Collinear and overlapping.
        }
        return -1; // Collinear and disjoint.
    }
    if (EQ(c1, 0)) {
        return -1; // Parallel and disjoint.
    }
    double t = ac.cross(cd) / c1, u = c2 / c1;
    bool t_between_01 = TOUCH_IS_INTERSECT ? (LE(0, t) && LE(t, 1)) : (LT(0, t) && LT(t, 1));
    bool u_between_01 = TOUCH_IS_INTERSECT ? (LE(0, u) && LE(u, 1)) : (LT(0, u) && LT(u, 1));
    if (t_between_01 && u_between_01) {
        if (p != NULL) {

```

```

    *p = a + t * ab;
}
return 0; // Non-parallel with one intersection.
}
return -1; // Non-parallel with no intersections.
}

// Returns the minimum distance from any point on the line segment ab to any
// point on the line segment cd. This is 0 if the segments touch or intersect.
double seg_dist(const point &a, const point &b, const point &c, const point &d) {
    return (seg_intersection(a, b, c, d) >= 0) ? 0
        : std::min(
            std::min(seg_dist(a, c, d), seg_dist(b, c, d)),
            std::min(seg_dist(c, a, b), seg_dist(d, a, b))
        );
}

// Returns the point on line l that is closest to point p. Note that the result
// always lies on the line through p that is perpendicular to l.
point closest_point(const line &l, const point &p) {
    if (EQ(l.a, 0)) {
        return point(p.x, -l.c); // Horizontal line.
    }
    if (EQ(l.b, 0)) {
        return point(-l.c, p.y); // Vertical line.
    }
    point res;
    line_intersection(l, l.perpendicular(p), &res);
    return res;
}

// Returns the point on line segment ab that is closest to point p.
point closest_point(const point &a, const point &b, const point &p) {
    if (a == b) {
        return a;
    }
    point ap(p - a), ab(b - a);
    double t = ap.dot(ab) / sqnorm(ab);
    return (t <= 0) ? a : ((t >= 1) ? b : point(a.x + t * ab.x, a.y + t * ab.y));
}

// A two-dimensional circle class represented by  $(x - h)^2 + (y - k)^2 = r^2$ ,
// where (h, k) is the center and r is the radius.
struct circle {
    double h, k, r;

    circle() : h(0), k(0), r(0) {}
    circle(double r) : h(0), k(0), r(fabs(r)) {}
    circle(const point &o, double r) : h(o.x), k(o.y), r(fabs(r)) {}
    circle(double h, double k, double r) : h(h), k(k), r(fabs(r)) {}

    // Circle with the line segment ab as a diameter.
    circle(const point &a, const point &b) {
        h = (a.x + b.x) / 2.0;
        k = (a.y + b.y) / 2.0;
        r = norm(a - point(h, k));
    }

    // Circumcircle of three points.
    circle(const point &a, const point &b, const point &c) {
        double an = sqnorm(b - c), bn = sqnorm(a - c), cn = sqnorm(a - b);
        double wa = an * (bn + cn - an);
        double wb = bn * (an + cn - bn);
        double wc = cn * (an + bn - cn);
        double w = wa + wb + wc;
        if (EQ(w, 0)) {
            throw std::runtime_error("No circle from collinear points.");
        }
    }
}

```

```

    h = (wa * a.x + wb * b.x + wc * c.x) / w;
    k = (wa * a.y + wb * b.y + wc * c.y) / w;
    r = norm(a - point(h, k));
}

// Circle of radius r that contains points a and b. In the general case, there
// will be two possible circles and only one is chosen arbitrarily. However if
// the diameter is equal to dist(a, b) = 2*r, then there is only one possible
// center. If points a and b are identical, then there are infinite circles.
// If the points are too far away relative to the radius, then there is no
// possible circle. In the latter two cases, an exception is thrown.
circle(const point &a, const point &b, double r) : r(fabs(r)) {
    if (LE(r, 0) && a == b) { // Circle with zero area.
        h = a.x;
        k = a.y;
        return;
    }
    double d = norm(b - a);
    if (EQ(d, 0)) {
        throw std::runtime_error("Identical points, infinite circles.");
    }
    if (LT(r * 2.0, d)) {
        throw std::runtime_error("Points too far away to make circle.");
    }
    double v = sqrt(r * r - d * d / 4.0) / d;
    point m = (a + b) / 2.0;
    h = m.x + v * (a.y - b.y);
    k = m.y + v * (b.x - a.x);
    // The other answer is (h, k) = (m.x - v*(a.y - b.y), m.y - v*(b.x - a.x)).
}

bool operator==(const circle &c) const { return EQ(h, c.h) && EQ(k, c.k) && EQ(r, c.r); }
bool operator!=(const circle &c) const { return !(*this == c); }

point center() const { return point(h, k); }
bool contains(const point &p) const { return LE(sqnorm(p - center()), r * r); }
bool on_edge(const point &p) const { return EQ(sqnorm(p - center()), r * r); }

friend std::ostream &operator<<(std::ostream &out, const circle &c) {
    return out << std::showpos << "(x" << -(fabs(c.h) < EPS ? 0 : c.h) << ")^2+"
        << "(y" << -(fabs(c.k) < EPS ? 0 : c.k) << ")^2" << std::noshowpos << "=="
        << (fabs(c.r) < EPS ? 0 : c.r * c.r);
}

};

// Returns the circle inscribed inside the triangle abc.
circle incircle(const point &a, const point &b, const point &c) {
    double al = norm(b - c), bl = norm(a - c), cl = norm(a - b), p = al + bl + cl;
    return EQ(p, 0) ? circle(a.x, a.y, 0)
        : circle(
            (al * a.x + bl * b.x + cl * c.x) / p, (al * a.y + bl * b.y + cl * c.y) / p,
            fabs(cross(a, b, c)) / p
        );
}

// Determines the line(s) tangent to circle c that passes through point p.
// Returns -1 if there is no tangent line because p is strictly inside c, 0 if
// there is exactly one tangent line because p is on the boundary of c (in which
// case the line will be stored into pointer l1 if it's not NULL), or 1 if there
// are two tangent lines because p is strictly outside of c (in which case the
// lines will be stored into pointers l1 and l2 if they are not NULL).
int tangent(const circle &c, const point &p, line *l1 = NULL, line *l2 = NULL) {
    if (c.on_edge(p)) {
        if (l1 != NULL) {
            *l1 = line(point(c.h, c.k), p).perpendicular(p);
        }
        return 0;
    }
}

```



```

    if (c.contains(p)) {
        return -1;
    }
    point q = (p - c.center()) / c.r;
    double n = sqnorm(q), d = q.y * sqrt(sqnorm(q) - 1.0);
    point t1((q.x - d) / n, c.k), t2((q.x + d) / n, c.k);
    if (NE(q.y, 0)) {
        t1.y += c.r * (1.0 - t1.x * q.x) / q.y;
        t2.y += c.r * (1.0 - t2.x * q.x) / q.y;
    } else {
        d = c.r * sqrt(1.0 - t1.x * t1.x);
        t1.y += d;
        t2.y -= d;
    }
    t1.x = t1.x * c.r + c.h;
    t2.x = t2.x * c.r + c.h;
    if (l1 != NULL && l2 != NULL) {
        *l1 = line(p, t1);
        *l2 = line(p, t2);
    }
    return 1;
}

// Determines the intersection between the circle c and line l. Returns -1 if
// there is no intersection, 0 if the line is one intersection point because the
// line is tangent (in which case it will be stored into pointer p if it's not
// NULL), or 1 if there are two intersection points because the line crosses
// through the circle (in which case they will be stored into pointers p and q
// if they are not NULL).
int intersection(const circle &c, const line &l, point *p = NULL, point *q = NULL) {
    if (!l.valid()) {
        throw std::runtime_error("Invalid line for intersection.");
    }
    double v = c.h * l.a + c.k * l.b + l.c;
    double aabb = l.a * l.a + l.b * l.b;
    double disc = v * v / aabb - c.r * c.r;
    if (disc > EPS) {
        return -1;
    }
    double x0 = -l.a * l.c / aabb, y0 = -l.b * v / aabb;
    if (disc > -EPS) {
        if (p != NULL) {
            *p = point(x0 + c.h, y0 + c.k);
        }
        return 0;
    }
    double k = sqrt(std::max(0.0, disc / -aabb));
    if (p != NULL && q != NULL) {
        *p = point(x0 + k * l.b + c.h, y0 - k * l.a + c.k);
        *q = point(x0 - k * l.b + c.h, y0 + k * l.a + c.k);
    }
    return 1;
}

// Determines the intersection points between two circles c1 and c2.
// Returns: -2 if circle c2 completely encloses circle c1,
//          -1 if circle c1 completely encloses circle c2,
//           0 if the circles are completely disjoint,
//           1 if the circles are tangent with one intersection (stored in p),
//           2 if the circles intersect at two points (stored in p and q),
//           3 if the circles are equal and intersect at infinite points.
int intersection(const circle &c1, const circle &c2, point *p = NULL, point *q = NULL) {
    if (EQ(c1.h, c2.h) && EQ(c1.k, c2.k)) {
        return EQ(c1.r, c2.r) ? 3 : (c1.r > c2.r ? -1 : -2);
    }
    point d12(c2.center() - c1.center());
    double d = norm(d12);
    if (GT(d, c1.r + c2.r)) {

```

```

    return 0;
}
if (LT(d, fabs(c1.r - c2.r))) {
    return c1.r > c2.r ? -1 : -2;
}
double a = (c1.r * c1.r - c2.r * c2.r + d * d) / (2 * d);
double x0 = c1.h + (d12.x * a / d), y0 = c1.k + (d12.y * a / d);
double s = sqrt(c1.r * c1.r - a * a), rx = -d12.y * s / d, ry = d12.x * s / d;
if (EQ(rx, 0) && EQ(ry, 0)) {
    if (p != NULL) {
        *p = point(x0, y0);
    }
    return 1;
}
if (p != NULL && q != NULL) {
    *p = point(x0 - rx, y0 - ry);
    *q = point(x0 + rx, y0 + ry);
}
return 2;
}

// Returns the intersection area of circles c1 and c2.
double intersection_area(const circle &c1, const circle &c2) {
    double r = std::min(c1.r, c2.r), R = std::max(c1.r, c2.r);
    double d = norm(c2.center() - c1.center());
    if (LE(d, R - r)) {
        return PI * r * r;
    }
    if (GE(d, R + r)) {
        return 0;
    }
    return r * r * acos((d * d + r * r - R * R) / 2 / d / r) +
        R * R * acos((d * d + R * R - r * r) / 2 / d / R) -
        0.5 * sqrt((-d + r + R) * (d + r - R) * (d - r + R) * (d + r + R));
}

// Returns the area of the triangle abc.
double triangle_area(const point &a, const point &b, const point &c) {
    return fabs(cross(a, b, c)) / 2.0;
}

// Returns the area of a triangle with side lengths s1, s2, and s3. The given
// lengths must be non-negative and form a valid triangle.
double triangle_area_sides(double s1, double s2, double s3) {
    double s = (s1 + s2 + s3) / 2.0;
    return sqrt(s * (s - s1) * (s - s2) * (s - s3));
}

// Returns the area of a triangle with medians of lengths m1, m2, and m3. The
// median of a triangle is a line segment joining a vertex to the midpoint of
// the opposing edge.
double triangle_area_medians(double m1, double m2, double m3) {
    return 4.0 * triangle_area_sides(m1, m2, m3) / 3.0;
}

// Returns the area of a triangle with altitudes h1, h2, and h3. An altitude of
// a triangle is the shortest line between a vertex and the infinite line that
// is extended from its opposite edge.
double triangle_area_altitudes(double h1, double h2, double h3) {
    if (EQ(h1, 0) || EQ(h2, 0) || EQ(h3, 0)) {
        return 0;
    }
    double x = h1 * h1, y = h2 * h2, z = h3 * h3;
    double v = 2.0 / (x * y) + 2.0 / (x * z) + 2.0 / (y * z);
    return 1.0 / sqrt(v - 1.0 / (x * x) - 1.0 / (y * y) - 1.0 / (z * z));
}

// Returns whether points p1 and p2 lie on the same side of the line containing

```

```

// points a and b. If one or both points lie exactly on the line, then the
// result will depend on the setting of EDGE_IS_SAME_SIDE.
bool same_side(const point &p1, const point &p2, const point &a, const point &b) {
    static const bool EDGE_IS_SAME_SIDE = true;
    point ab(b - a);
    double c1 = ab.cross(p1 - a), c2 = ab.cross(p2 - a);
    return EDGE_IS_SAME_SIDE ? GE(c1 * c2, 0) : GT(c1 * c2, 0);
}

// Returns whether point p lies within the triangle abc. If the point lies on or
// close to an edge (by roughly EPS), then the result will depend on the setting
// of EDGE_IS_SAME_SIDE in the function above.
bool point_in_triangle(const point &p, const point &a, const point &b, const point &c) {
    return same_side(p, a, b, c) && same_side(p, b, a, c) && same_side(p, c, a, b);
}

// Returns the area of a rectangle with opposing vertices a and b.
double rectangle_area(const point &a, const point &b) {
    return fabs((a.x - b.x) * (a.y - b.y));
}

// Returns whether point p lies within the rectangle defined by a vertex at v
// (x, y), a width of w, and a height of h. Note that negative widths and
// heights are supported. If the point lies on or close to an edge (by roughly
// EPS), then the result will depend on the setting of EDGE_IS_INSIDE.
bool point_in_rectangle(const point &p, const point &v, double w, double h) {
    static const bool EDGE_IS_INSIDE = true;
    if (w < 0) {
        return point_in_rectangle(p, point(v.x + w, v.y), -w, h);
    }
    if (h < 0) {
        return point_in_rectangle(p, point(v.x, v.y + h), w, -h);
    }
    return EDGE_IS_INSIDE ? (GE(p.x, v.x) && LE(p.x, v.x + w) && GE(p.y, v.y) && LE(p.y, v.y + h))
        : (GT(p.x, v.x) && LT(p.x, v.x + w) && GT(p.y, v.y) && LT(p.y, v.y + h));
}

// Returns whether point p lies within the rectangle with opposing vertices a
// and b. If the point lies on or close to an edge (by roughly EPS), then the
// result will depend on the setting of EDGE_IS_INSIDE in the function above.
bool point_in_rectangle(const point &p, const point &a, const point &b) {
    double x1 = std::min(a.x, b.x), y1 = std::min(a.y, b.y);
    double xh = std::max(a.x, b.x), yh = std::max(a.y, b.y);
    return point_in_rectangle(p, point(x1, y1), xh - x1, yh - y1);
}

// Determines the intersection region of the rectangle with opposing vertices a1
// and b1 and the rectangle with opposing vertices a2 and b2. Returns -1 if the
// rectangles are completely disjoint, 0 if the rectangles partially intersect,
// 1 if the first rectangle is completely inside the second, and 2 if the second
// rectangle is completely inside the first. If there is an intersection, the
// opposing vertices of the intersection rectangle will be stored into pointers
// p and q if they are not NULL. If the intersection is a single point or line
// segment, then the result will depend on the setting of EDGE_IS_INSIDE in the
// point_in_rectangle function above.
int rectangle_intersection(
    const point &a1, const point &b1, const point &a2, const point &b2, point *p = NULL,
    point *q = NULL
) {
    bool a1in2 = point_in_rectangle(a1, a2, b2);
    bool b1in2 = point_in_rectangle(b1, a2, b2);
    if (a1in2 && b1in2) {
        if (p != NULL && q != NULL) {
            *p = std::min(a1, b1);
            *q = std::max(a1, b1);
        }
        return 1; // Rectangle 1 completely inside 2.
    }
}

```

```

if (!a1in2 && !b1in2) {
    if (point_in_rectangle(a2, a1, b1)) {
        if (p != NULL && q != NULL) {
            *p = std::min(a2, b2);
            *q = std::max(a2, b2);
        }
        return 2; // Rectangle 2 completely inside 1.
    }
    return -1; // Completely disjoint.
}
if (p != NULL && q != NULL) {
    if (a1in2) {
        *p = a1;
        *q = (a1 < b1) ? std::max(a2, b2) : std::min(a2, b2);
    } else {
        *p = b1;
        *q = (b1 < a1) ? std::max(a2, b2) : std::min(a2, b2);
    }
    if (*p > *q) {
        std::swap(p, q);
    }
}
return 0;
}

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

#define pt point
#define EQP(p, q) (EQ((p).x, (q).x) && EQ((p).y, (q).y))

int main() {
    pt p, q;

    p = point(-10, 3);
    assert(pt(-18, 29) == p + pt(-3, 9) * 6 / 2 - pt(-1, 1));
    assert(EQ(109, p.sqnorm()));
    assert(EQ(10.44030650891, p.norm()));
    assert(EQ(2.850135859112, p.arg()));
    assert(EQ(0, p.dot(pt(3, 10))));
    assert(EQ(0, p.cross(pt(10, -3))));
    assert(EQ(10, p.proj(pt(-10, 0))));
    assert(EQ(1, p.normalize().norm()));
    assert(pt(-3, -10) == p.rotate90());
    assert(pt(3, 12) == p.rotateCW(pt(1, 1), PI / 2));
    assert(pt(1, -10) == p.rotateCCW(pt(2, 2), PI / 2));
    assert(pt(10, -3) == p.reflect(pt(0, 0)));
    assert(pt(-10, -3) == p.reflect(pt(-2, 0), pt(5, 0)));

    line l(2, -5, -8);
    line para = line(2, -5, -8).parallel(pt(-6, -2));
    line perp = line(2, -5, -8).perpendicular(pt(-6, -2));
    assert(l.is_parallel(para) && l.is_perpendicular(perp));
    assert(l.slope() == 0.4);
    assert(para == line(-0.4, 1, -0.4)); // -0.4x + y - 0.4 = 0.
    assert(perp == line(2.5, 1, 17));   // 2.5x + y + 17 = 0.

    assert(EQ(angle_between(l, perp), 90 * DEG));

    assert(EQ(123, reduce_deg(-8 * 360 + 123)));
    assert(EQ(1.2345, reduce_rad(2 * PI * 8 + 1.2345)));
    assert(polar_point(4, PI) == pt(-4, 0));
    assert(polar_point(4, -PI / 2) == pt(0, -4));
}

```

```

assert(EQ(45, polar_angle(pt(5, 5)) * RAD));
assert(EQ(135, polar_angle(pt(-4, 4)) * RAD));
assert(EQ(90, angle(pt(5, 0), pt(0, 5), pt(-5, 0)) * RAD));
assert(EQ(225, angle_between(pt(0, 5), pt(5, -5)) * RAD));
assert(-1 == cross(pt(0, 1), pt(1, 0), pt(0, 0)));
assert(1 == turn(pt(0, 1), pt(0, 0), pt(-5, -5)));

assert(EQ(5, dist(pt(-1, -1), pt(2, 3))));
assert(EQ(25, sqdist(pt(-1, -1), pt(2, 3))));
assert(EQ(1.2, line_dist(pt(2, 1), line(-4, 3, -1))));
assert(EQ(0.8, line_dist(pt(3, 3), pt(-1, -1), pt(2, 3))));
assert(EQ(1.2, line_dist(pt(2, 1), pt(-1, -1), pt(2, 3))));
assert(EQ(0.0, line_dist(line(-4, 3, -1), line(8, 6, 2))));
assert(EQ(0.8, line_dist(line(-4, 3, -1), line(-8, 6, -10))));
assert(EQ(1.0, seg_dist(pt(3, 3), pt(-1, -1), pt(2, 3))));
assert(EQ(1.2, seg_dist(pt(2, 1), pt(-1, -1), pt(2, 3))));
assert(EQ(0.0, seg_dist(pt(0, 2), pt(3, 3), pt(-1, -1), pt(2, 3))));
assert(EQ(0.6, seg_dist(pt(-1, 0), pt(-2, 2), pt(-1, -1), pt(2, 3))));

assert(line_intersection(line(-1, 1, 0), line(1, 1, -3), &p) == 0);
assert(p == pt(1.5, 1.5));
assert(line_intersection(pt(0, 0), pt(1, 1), pt(0, 4), pt(4, 0), &p) == 0);
assert(p == pt(2, 2));

{
#define test(a, b, c, d, e, f, g, h) \
    seg_intersection(pt(a, b), pt(c, d), pt(e, f), pt(g, h), &p, &q)

    // Intersection is a point.
    assert(0 == test(-4, 0, 4, 0, 0, -4, 0, 4) && p == pt(0, 0));
    assert(0 == test(0, 0, 10, 10, 2, 2, 16, 4) && p == pt(2, 2));
    assert(0 == test(-2, 2, -2, -2, -2, 0, 0, 0) && p == pt(-2, 0));
    assert(0 == test(0, 4, 4, 4, 4, 0, 4, 8) && p == pt(4, 4));

    // Intersection is a segment.
    assert(1 == test(10, 10, 0, 0, 2, 2, 6, 6));
    assert(p == pt(2, 2) && q == pt(6, 6));
    assert(1 == test(6, 8, 14, -2, 14, -2, 6, 8));
    assert(p == pt(6, 8) && q == pt(14, -2));

    // No intersection.
    assert(-1 == test(6, 8, 8, 10, 12, 12, 4, 4));
    assert(-1 == test(-4, 2, -8, 8, 0, 0, -4, 6));
    assert(-1 == test(4, 4, 4, 6, 0, 2, 0, 0));
    assert(-1 == test(4, 4, 6, 4, 0, 2, 0, 0));
    assert(-1 == test(-2, -2, 4, 4, 10, 10, 6, 6));
    assert(-1 == test(0, 0, 2, 2, 4, 0, 1, 4));
    assert(-1 == test(2, 2, 2, 8, 4, 4, 6, 4));
    assert(-1 == test(4, 2, 4, 4, 0, 8, 10, 0));
}

assert(pt(2.5, 2.5) == closest_point(line(-1, -1, 5), pt(0, 0)));
assert(pt(3, 0) == closest_point(line(1, 0, -3), pt(0, 0)));
assert(pt(0, 3) == closest_point(line(0, 1, -3), pt(0, 0)));

assert(pt(3, 0) == closest_point(pt(3, 0), pt(3, 3), pt(0, 0)));
assert(pt(2, -1) == closest_point(pt(2, -1), pt(4, -1), pt(0, 0)));
assert(pt(4, -1) == closest_point(pt(2, -1), pt(4, -1), pt(5, 0)));

circle c(-2, 5, sqrt(10));
assert(c == circle(point(-2, 5), sqrt(10)));
assert(c == circle(point(1, 6), point(-5, 4)));
assert(c == circle(point(-3, 2), point(-3, 8), point(-1, 8)));
assert(c == incircle(point(-12, 5), point(3, 0), point(0, 9)));
assert(c.contains(point(-2, 8)) && !c.contains(point(-2, 9)));
assert(c.on_edge(point(-1, 2)) && !c.on_edge(point(-1.01, 2)));

line l1, l2;

```

```

assert(-1 == tangent(circle(0, 0, 4), pt(1, 1), &l1, &l2));
assert(0 == tangent(circle(0, 0, sqrt(2)), pt(1, 1), &l1, &l2));
assert(l1 == line(-1, -1, 2));
assert(1 == tangent(circle(0, 0, 2), pt(2, 2), &l1, &l2));
assert(l1 == line(0, -2, 4));
assert(l2 == line(2, 0, -4));

assert(-1 == intersection(circle(1, 1, 3), line(5, 3, -30), &p, &q));
assert(0 == intersection(circle(1, 1, 3), line(0, 1, -4), &p, &q));
assert(p == pt(1, 4));
assert(1 == intersection(circle(1, 1, 3), line(0, 1, -1), &p, &q));
assert(p == pt(4, 1));
assert(q == pt(-2, 1));

assert(-2 == intersection(circle(1, 1, 1), circle(0, 0, 3), &p, &q));
assert(-1 == intersection(circle(0, 0, 3), circle(1, 1, 1), &p, &q));
assert(0 == intersection(circle(5, 0, 4), circle(-5, 0, 4), &p, &q));
assert(1 == intersection(circle(-5, 0, 5), circle(5, 0, 5), &p, &q));
assert(p == pt(0, 0));
assert(2 == intersection(circle(-0.5, 0, 1), circle(0.5, 0, 1), &p, &q));
assert(p == pt(0, -sqrt(3) / 2));
assert(q == pt(0, sqrt(3) / 2));

// Each circle passes through the other's center.
double r = 3, a = intersection_area(circle(-r / 2, 0, r), circle(r / 2, 0, r));
assert(EQ(a, r * r * (2 * PI / 3 - sqrt(3) / 2)));

assert(EQ(6, triangle_area(pt(0, -1), pt(4, -1), pt(0, -4))));
assert(EQ(6, triangle_area_sides(3, 4, 5)));
assert(EQ(6, triangle_area_medians(3.605551275, 2.5, 4.272001873)));
assert(EQ(6, triangle_area_altitudes(3, 4, 2.4)));

assert(point_in_triangle(pt(0, 0), pt(-1, 0), pt(0, -2), pt(4, 0)));
assert(!point_in_triangle(pt(0, 1), pt(-1, 0), pt(0, -2), pt(4, 0)));
assert(point_in_triangle(pt(-2.44, 0.82), pt(-1, 0), pt(-3, 1), pt(4, 0)));
assert(!point_in_triangle(pt(-2.44, 0.7), pt(-1, 0), pt(-3, 1), pt(4, 0)));

assert(EQ(20, rectangle_area(pt(1, 1), pt(5, 6))));

assert(point_in_rectangle(pt(0, -1), pt(0, -3), 3, 2));
assert(point_in_rectangle(pt(2, -2), pt(3, -3), -3, 2));
assert(!point_in_rectangle(pt(0, 0), pt(3, -1), -3, -2));
assert(point_in_rectangle(pt(2, -2), pt(3, -3), pt(0, -1)));
assert(!point_in_rectangle(pt(-1, -2), pt(3, -3), pt(0, -1)));

assert(-1 == rectangle_intersection(pt(0, 0), pt(1, 1), pt(2, 2), pt(3, 3)));
assert(0 == rectangle_intersection(pt(1, 1), pt(7, 7), pt(5, 5), pt(0, 0), &p, &q));
assert(EQP(p, pt(1, 1)) && EQP(q, pt(5, 5)));
assert(1 == rectangle_intersection(pt(1, 1), pt(0, 0), pt(0, 0), pt(1, 10), &p, &q));
assert(EQP(p, pt(0, 0)) && EQP(q, pt(1, 1)));
assert(2 == rectangle_intersection(pt(0, 5), pt(5, 7), pt(1, 6), pt(2, 5), &p, &q));
assert(EQP(p, pt(1, 6)) && EQP(q, pt(2, 5)));

return 0;
}

```